

A SELF-STUDY BOOK FOR DESIGN ROUNDS

System Design for Interviews

LLD and HLD

One procedure. Seven steps. Twelve chapters that climb three tiers.

- TIER 1** Low-level design — classes, SOLID, patterns, one small design that runs. TCS, Accenture, Infosys, Wipro, Capgemini, Cognizant
- TIER 2** Mid-scale HLD — one service done properly: API, schema, cache, a few thousand requests per second. Grab, Sea/Shopee, GIC, DBS, Agoda, SCB
- TIER 3** Large-scale distributed — sharding, quorums, CAP trade-offs, multi-region, failure modes. Google, Amazon, Microsoft, Uber, Adobe

12 chapters · estimation · OOP & SOLID · design patterns · classic LLD · databases · caching · load balancing & API · queues · sharding · 10 HLD case studies

Code is JavaScript first (ES2022), Python where it reads more clearly.

A NOTE ON WHERE THIS MATERIAL COMES FROM

Every design, problem, code listing, diagram and answer in this book is **original**. Each one was written for this book. Nothing here is copied from any published book, course, blog or question bank.

No company's real interview question is reproduced, quoted or paraphrased. Real interview questions are private property and are not public documents. No engineer's account of a real interview was used.

When a design is labelled, for example, "Grab · pattern" or "Amazon · pattern", that label means one thing only: the problem was written to match the **publicly documented shape** of that company's design round — the kind of system it tends to ask about, the depth it tends to push to, and the time it gives you. The label is never a claim that this question, or anything like it, was ever asked at that company.

Company names are used only to describe that shape. This book is not affiliated with, endorsed by, or connected to any company named in it. All trademarks belong to their owners.

Architecture descriptions of well-known systems are **teaching sketches**, not accounts of how any real company builds anything. Treat them as a model to reason with, not as inside knowledge.

Every number was worked twice and every code listing was run before it was printed. If you still find a mistake, trust your own working and re-check it — that habit is worth more than any answer key.

Contents

How to use this book	2
Chapter 1 · How to Run a Design Interview	4
Chapter 2 · Back-of-the-Envelope Estimation	37
Chapter 3 · OOP & SOLID for Interviews	67
Chapter 4 · Design Patterns That Get Asked	106
Chapter 5 · Classic LLD Problems	147
Chapter 6 · Databases & Data Modelling	169
Chapter 7 · Caching & CDNs	210
Chapter 8 · Load Balancing, Proxies & API Design	238
Chapter 9 · Queues, Streams & Async Processing	277
Chapter 10 · Replication, Sharding & Consistency	310
Chapter 11 · HLD Case Studies I	347
Chapter 12 · HLD Case Studies II	398
Appendix · Last-Week Quick Reference	444

How to use this book

Read these two pages once. Then start Chapter 1.

1 • What the three tiers mean in this book

In the other books of this series a tier is a difficulty level. Here a tier is a **different kind of round**. A company usually runs one of them, not all three.

Tier	Who runs this round	What you must produce	Length
0 Warm-up	nobody — it is you	One mechanism alone: a single class, one cache policy, one index, one queue guarantee. No story, no scale. You are building the reflex.	5–10 min
1 LLD	TCS, Accenture, Infosys, Wipro, Capgemini, Cognizant	Classes, fields, methods, SOLID, one or two patterns, and code that would run. No servers, no QPS. Marks come from clean boundaries and the awkward case.	45 min
2 Mid-scale HLD	Grab, Sea/Shopee, GIC, DBS, Agoda, SCB, LINE MAN	One service done properly: 3–5 endpoints, a schema with the key chosen, a cache with an invalidation rule, a few thousand RPS. Depth beats breadth.	45–60 min
3 Large-scale	Google, Amazon, Microsoft, Uber, Adobe, Goldman Sachs	Sharding key and its hotspot, replication and the consistency promise, a CAP decision you defend, multi-region, and what breaks first. Follow-ups are half the round.	45–60 min

Work Tier 1 first even for a Tier-3 company. Every Tier-3 design is built on the Tier-1 and Tier-2 ones before it. A distributed design is a low-level design with a network in the middle — and the network is the part that fails.

2 • Inside every chapter, and the six rules

Idea in one page	The mechanism, its one formula or invariant, and the sentence you say out loud.	1 Numbers	Thirty seconds of arithmetic ticks the “did they estimate?” box. Zero seconds ticks nothing. Do one division out loud.
Warm-up	Bare mechanics. Short. Where the vocabulary becomes automatic.	2 Decide	Every trade-off names both sides and picks one . Say which, why, and what you gave up.
Tier 1 → 2 → 3	The same idea as classes, then as one service, then as many machines.	3 Draw early	Boxes and arrows by minute 20, with the read path and write path marked separately.
Seven steps	Clarify, estimate, API, data model, diagram, deep dive, trade-offs. Never skipped, sometimes shrunk.	4 Code runs	Every listing here was run with node first. In the room, write code you could run, not pseudo-code that hides the hard part.
Interview drill	What the interviewer pushes on next, and the answer that survives the push.	5 Depth	Two parts solved properly beat eight parts named. Short on time? Deepen, do not widen.
Practice + key	Timed sets per tier. The key gives the full design, not a hint.	6 Failure	Name what breaks, how you notice, and what the user sees meanwhile. This separates senior from junior.
Revision card	Numbers, checklist, traps in the order they bite. Re-read this the night before.		

3 • The study plan — eight weeks

One hour a day on weekdays, two on Saturday. Each checkpoint is closed-book, on paper, against a timer.

Week	Chapters	Daily target	Checkpoint (closed book, timed)
1	Ch 1, 2	1 section + the arithmetic drills	Seven steps from memory in 2 min. Storage for a 20 M-user photo app in 5 min.
2	Ch 3	1 SOLID principle + rewrite one bad class	Split a 60-line god class. Name the principle each split serves.

3	Ch 4	1 pattern, typed from scratch	Strategy, observer and factory from memory in JavaScript. 30 min, must run.
4	Ch 5	1 classic LLD problem, full 45 min	Vending machine end to end on paper in 45 min: classes, states, code.
5	Ch 6, 7	1 section + draw the schema	Schema + index plan for a booking system, then a cache and its invalidation rule.
6	Ch 8, 9	1 section + one API written out	A rate-limited, idempotent payment-submit endpoint, with the retry story.
7	Ch 10, 11	1 case study, full 45 min	A URL shortener and a news feed, each with a sharding key and a defended CAP choice.
8	Ch 12 + back matter	1 case study + revise the cards	Two 45-min mock rounds, out loud, recorded. Then watch the recording.

4 • The crash plan — five days

Use this only when the interview is this week. It buys a pass, not mastery.

Day	Do exactly this	Hours
1	Chapter 1 in full. Chapter 2's revision card — memorise the latency numbers and the five estimation lines . Nothing else.	3
2	Chapter 3 skim, then Chapter 4's six patterns, typing each one once. Then one problem from Chapter 5 end to end.	3
3	Chapters 6 and 7: "The idea in one page" and the revision card of each. Then the SQL-vs-NoSQL and cache-pattern tables until you can reproduce them.	3
4	Chapters 8, 9, 10: revision cards only, plus the trade-off table in the appendix. Learn the sharding-key hotspot argument cold.	3
5	Two case studies from Chapter 11 spoken out loud against a timer, then the appendix quick reference twice.	4

TRAP

The crash plan skips the practice sets — that is what makes it a crash plan. With three weeks, run the eight-week plan at double speed instead. The practice sets are where a design moves from "I read it" to "I can produce it".

How to Run a Design Interview

You are not asked to be right. You are asked to be organised.

The idea in one page

A design interview is 45 minutes long. You are given one line of problem (“design a service that shortens links”), a whiteboard, and silence. Most candidates fail not because they lack knowledge, but because they have no **procedure**. They talk for 30 minutes, draw six boxes, and never produce a number, an endpoint, or a table.

This chapter gives you the procedure. It is seven steps. You will use the same seven steps for a 40-line class design and for a 40-machine cluster. Only the size of the numbers changes.

THE SEVEN STEPS — MEMORISE THIS LIST

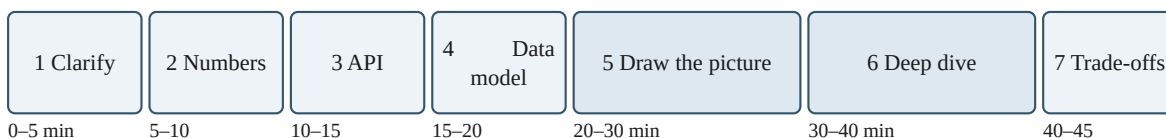
1. **Clarify.** Ask questions until the problem has a boundary. Write the boundary down.
2. **Estimate.** Turn users into requests per second, bytes per year, bits per second.
3. **API.** Write 3 to 5 concrete endpoints with their payloads.
4. **Data model.** Write the tables or classes, with fields and the chosen key.
5. **Draw.** One picture, boxes and arrows, request path marked.
6. **Deep dive.** Pick the one or two genuinely hard parts and solve them properly.
7. **Trade-offs.** Name both sides, decide, then say what breaks at 10x.

Never skip a step. If you are short on time, **shrink** a step; do not delete it. An interviewer scores “did the candidate estimate?” as yes or no. Thirty seconds of arithmetic gets the yes. Zero seconds gets the no.

The time budget

45 minutes is the common length. Here is where the minutes go. Print this on the inside of your eyelids.

time runs this way



If you are still in step 1 at minute 12, you have already lost 20 marks.

The 45-minute budget. The two wide blocks are where the marks are.

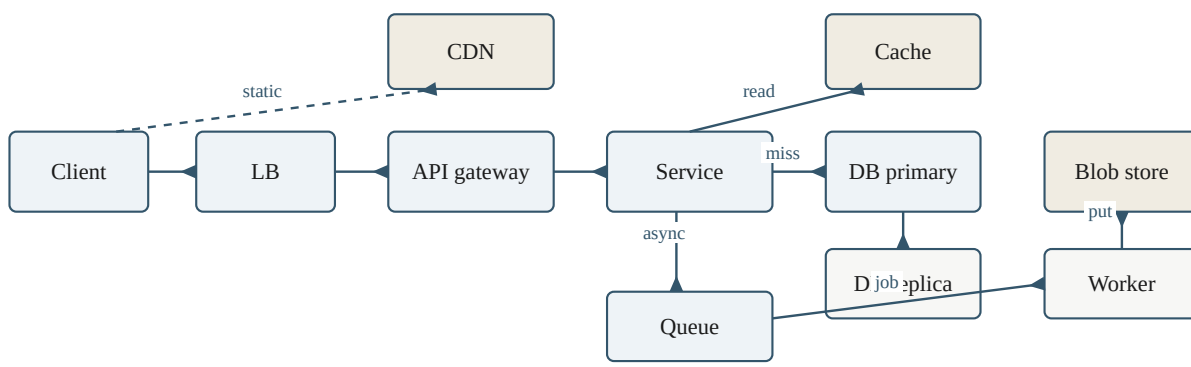
Step	Minutes	The single artefact you must leave on the board
1 Clarify	5	a written list: 3 in-scope features, 3 out-of-scope
2 Estimate	5	peak QPS, storage per year, peak bandwidth
3 API	5	3–5 endpoints with request and response fields
4 Data model	5	2–4 tables with the partition key circled
5 Draw	10	one diagram with the read path and write path marked
6 Deep dive	10	the hard part, solved, with the failure case handled
7 Trade-offs	5	two options, one decision, one 10x remark

TRAP

The single most common failure: the candidate spends 20 minutes on step 1, drawing out a perfect requirement list, and then has 10 minutes for everything technical. Clarifying is not free marks. It is a **gate** you pass through, not a room you live in. Five minutes. Then move.

The skeleton you draw for almost every HLD

Before you know anything about the problem, you already know 80% of the picture. Almost every internet service has this shape. Learn to draw it in 60 seconds, then **delete** the boxes the problem does not need.



The default skeleton. Draw it, then delete what you do not need.

The dashed arrow is the static path: images, CSS, video. It never touches your service. Saying that sentence out loud is worth a mark on its own.

What the interviewer is actually filling in

At the end of the hour the interviewer types a short form. It usually has boxes close to these five. Every minute you spend should be buying a tick in one of them.

Box	Tick if...	Cross if...
Scope control	you named what you would NOT build	you tried to build everything
Numbers	you produced a peak QPS and a storage figure	you said “it will be a lot of traffic”
Concrete design	endpoints, fields, keys are on the board	only unlabelled boxes are on the board
Depth	you solved one hard thing end to end	you stayed shallow everywhere
Judgement	you compared two options and chose one	you said “it depends” and stopped

SHORTCUT

The sentence that buys the Judgement tick, every time: “Option A gives me X but costs me Y. Option B is the reverse. Because this product is read-heavy and the user tolerates a few seconds of staleness, I choose A, and I would revisit it if writes grew past Z.” Name both. Decide. Give the condition that would flip the decision.

Step 1 — Clarify

The prompt you are given is deliberately under-specified. Your job is to cut it down to something buildable in 45 minutes. There are six questions that fit almost every problem. Ask four of them. Never ask more than six.

THE SIX CLARIFYING QUESTIONS

1. **Who uses it and how many?** (“Is this 10 thousand users or 100 million?”)

2. **What are the top three actions?** (“Post, read feed, follow. Anything else?”)
3. **Read-heavy or write-heavy, and what ratio?** (“Roughly 100 reads per write?”)
4. **How fresh must the data be?** (“Can a reader see a 10-second-old value?”)
5. **What must never break?** (“Is losing one message acceptable? Is double charging?”)
6. **One region or many?** (“India only, or India plus Singapore?”)

And one sentence you always add yourself, without being asked: “I will treat login, payments, and the admin console as out of scope.”

Why each answer changes the design

This is the part candidates miss. Do not ask a question and then ignore the answer. Say out loud what the answer just changed.

Question	If the answer is A	If the answer is B
How many users?	10 K: one server, one Postgres, done	100 M: sharding, cache tier, CDN
Read/write ratio?	100:1 read-heavy: cache + read replicas	1:1: cache barely helps; size the writes
Freshness?	10 s stale is fine: cache aggressively, async	must be exact: read the primary, pay latency
What must never break?	analytics: lose 0.1%, nobody dies	money: idempotency keys, ledger, no deletes
One region?	one: single primary, simple	many: conflict handling, CAP decision needed
Attachments?	text only: DB is enough	files: blob store + CDN + signed URLs

Example 1 [warm-up]

Prompt: “Design a system where college clubs can post events and students can see them.” Write four clarifying questions and say what each one changes.

SOLUTION

1. **“How many students and clubs?”** — 5,000 students means one server. 5 million means a feed fan-out problem. It decides whether we even need a cache.
2. **“Does a student see every club’s events, or only clubs they joined?”** — “every” means one shared list that can be cached once for everybody. “only joined” means a per-student query, which is 1000x more cache entries.
3. **“Do events have posters or video?”** — text-only keeps everything in the database. Media means a blob store, a CDN, and an upload endpoint.
4. **“If an event edit takes 30 seconds to appear, is that a problem?”** — if yes, we read the primary database. If no, we can serve a cached list and refresh it in the background, which is far cheaper.

Then the scope sentence: “I will build post-event, list-events, and join-club. I am leaving out login, notifications, and the moderation queue.”

TRAP

Do not ask a question whose answer you will not use. Asking “what is the tech stack?” or “is it on AWS or GCP?” wastes a minute and earns nothing. The interviewer is listening for whether the question **branches** your design.

Step 2 — Estimate

Chapter 2 is entirely about this. Here is only the part you need to run the interview: the four numbers you must produce, and the order to produce them in.

FOUR NUMBERS, ALWAYS IN THIS ORDER

1. Peak QPS.

$$\text{requests/day} = \text{DAU} \times \text{actions per user per day}$$

$$\text{average QPS} = \frac{\text{requests/day}}{86,400}$$

$$\text{peak QPS} \approx 3 \times \text{average QPS}$$

2. Storage per year.

$$\text{bytes/day} = \text{new rows/day} \times \text{bytes per row}$$

$$\text{bytes/year} = \text{bytes/day} \times 365$$

3. Peak bandwidth.

$$\text{bytes/s} = \text{peak QPS} \times \text{average response size}$$

(multiply by 8 for bits per second)

4. Server count.

$$\text{servers} = \frac{\text{peak QPS}}{\text{QPS one server handles}} \times 1.5 \text{ (headroom)}$$

Round everything. 86,400 seconds in a day. Call it 100,000. You are then about 14% low, which is fine for a 30-second estimate — but say out loud that you rounded.

Example 2 [warm-up]

A service has 5 million daily users. Each opens the app 8 times a day and each open makes 3 API calls. Find requests per day, average QPS and peak QPS.

SOLUTION

Requests per day.

$$5,000,000 \times 8 \times 3 = 120,000,000 \text{ requests/day}$$

Average QPS. Divide by the seconds in a day.

$$120,000,000 \times \frac{1,000}{86,400} = 1,388.9 \approx 1,400 \text{ QPS}$$

Check with the rounded divisor: $120,000,000 \times \frac{1,000}{100,000} = 1,200$. Close enough to know the answer is “about one to one and a half thousand”.

Peak QPS. Traffic is not flat. Use 3x.

$$1,400 \times 3 = 4,200 \text{ QPS at peak}$$

Answer: 1,400 QPS average, 4,200 QPS peak

Where does 3x come from? Real consumer traffic in one country has a busy evening. Roughly 12–15% of a day’s traffic lands in the busiest hour, versus 4.2% if it were perfectly flat. $15/4.2 \approx 3.6$. So 3x is a safe, honest, defensible multiplier. Say that sentence and the interviewer stops questioning the number.

Step 3 — The API surface

Boxes and arrows are cheap. Endpoints are not. Writing three endpoints proves you have actually decided what the system does.

HOW TO WRITE AN ENDPOINT IN 20 SECONDS

Give four things and nothing more:

1. **method and path** — POST /v1/orders
2. **request fields** — { userId, items[], couponCode }
3. **response fields** — { orderId, total, status }
4. **the one non-obvious header or rule** — “the client sends Idempotency-Key; a repeat with the same key returns the first response instead of creating a second order.”

Rules that earn marks:

- Version the path: /v1/. Costs two characters, shows experience.
- Every list endpoint is paginated: ?limit=20&cursor=... Never ?page=7.
- Every write endpoint that touches money or sends a message is idempotent.
- Return an id, not the whole object, from a create.

Example 3 [warm-up]

Write the API surface for the club-events service from Example 1.

SOLUTION

Four endpoints is enough

TEXT

```
POST /v1/clubs/{clubId}/events
  body { title, startsAt, venue, description }
  resp 201 { eventId }
  note header Idempotency-Key: if the phone retries, no duplicate event

GET /v1/feed?limit=20&cursor=<opaque>
  resp 200 { events: [ {eventId, clubId, title, startsAt, venue} ],
              nextCursor }
  note cursor, not page number: new events keep arriving at the top,
        so page 2 would show repeats

POST /v1/clubs/{clubId}/members
  resp 204
  note joining is idempotent by nature; joining twice is still joined

GET /v1/events/{eventId}
  resp 200 { eventId, clubId, title, startsAt, venue, description, rsvpCount }
```

Notice what is **not** here: no GET /v1/everything, no admin endpoints, no login. Saying “login is out of scope” in step 1 lets you skip it here without looking lazy.

TRAP

?page=7 on a feed that is constantly gaining new items at the top is a real bug, not a style choice. If three new events arrive while the student reads page 1, then page 2 starts three items later than it should, and the student sees three events twice and misses none — or with deletions, misses some entirely. Cursor pagination (“give me what comes after this exact item”) has no such hole.

Step 4 — The data model

Two to four tables. Fields. And one thing circled: **the key**.

THE THREE KEY DECISIONS

Primary key — what identifies a row. **Partition key** — which shard the row lives on. This is the decision that decides whether the system scales. Choose the field you always have when you read. **Index** — the extra lookup path. Every index costs write speed and disk. Add one only when you can name the query it serves.

Write it in this shape — it is fast on a whiteboard and unambiguous:

```
events(event_id PK, club_id, title, starts_at, venue, created_at)
  index on (club_id, starts_at)  -- "this club's upcoming events"
```

Example 4 [warm-up]

Data model for the club-events service. Say which key you partition on and why.

SOLUTION

Three tables

TEXT

```
clubs(club_id PK, name, college_id, created_at)

events(event_id PK, club_id, title, starts_at, venue, body, created_at)
  index (club_id, starts_at)      -- club page
  index (college_id, starts_at)  -- college-wide feed

memberships(student_id, club_id, joined_at)
  PK (student_id, club_id)       -- composite; also stops double-join
  index (club_id)               -- "who is in this club"
```

Partition key: college_id. Reason: every read is scoped to one college. A student at college 7 never reads college 12's events. So all of college 7's rows can live on one shard, and every feed query touches exactly one shard. That is the property you want — a **single-shard read**.

Why not partition on event_id? It spreads writes beautifully, but then building one feed needs a query to every shard and a merge. With 50 shards that is 50 network calls for one screen. Rejected.

Why is memberships keyed (student_id, club_id) and not a fresh id? Because the composite key makes "join twice" impossible at the database level, for free. A uniqueness rule enforced by the database beats a uniqueness rule enforced by application code that runs on 20 servers at once.

Step 5 — Draw the picture

RULES FOR A WHITEBOARD DIAGRAM

1. Left to right: client, edge, service, storage. Never scatter boxes randomly.
2. Label **every** arrow with what flows on it, not just with an arrowhead.
3. Mark the **read path** and the **write path** differently (solid and dashed).
4. Put the numbers on the boxes: "3 app servers", "DB 500 GB", "cache 16 GB".
5. Draw dependencies you will not build (auth, payments) as a grey box labelled "existing".
6. Leave space on the right. Step 6 will add boxes there.

SHORTCUT

Narrate as you draw: "a write comes in here, hits the load balancer, the service validates it, writes to the primary, and publishes an event to the queue; the worker picks it up and updates the search index." One sentence per arrow. The interviewer is following your voice, not your handwriting.

Step 6 — Deep dive

You have 10 minutes and you cannot design everything properly. So choose. The interviewer wants to see one thing done to the bottom.

HOW TO PICK THE DEEP DIVE

Pick the part where a **naïve answer is visibly wrong**. Usually one of these five:

1. **The hot key.** One item gets 1000x the traffic of the others (a flash sale item, a celebrity account). What happens to that shard?
2. **The fan-out.** One write causes a million writes (a post to a million followers).
3. **The uniqueness.** Two requests race to grab the same seat, id, or username.
4. **The retry.** The network times out. Did the payment happen or not?
5. **The big list.** A query that must scan or sort something huge to answer.

If the interviewer has already hinted (“what if one user is very popular?”), that hint **is** the deep dive. Take it.

Step 7 — Trade-offs, failures, and 10x

THE CLOSING THREE MINUTES, SCRIPTED

Trade-off. “I chose X over Y. X gives me A, Y would have given me B. For this product A matters more because (reason tied to the user). If (condition) changed, I would switch to Y.”

Failure modes. Walk one component at a time and say what happens when it dies:

- Cache dies -> all traffic hits the database. Is the database sized for that? If not, add a request coalescing layer or accept a degraded mode.
- Queue backs up -> messages are late, not lost. What is the alert threshold?
- One shard dies -> that slice of users is down. Replica promotes in N seconds.
- The whole region dies -> is there a plan, or do we accept an outage? Say which.

10x. “If traffic grew 10x, the first thing to break is (component), because (measured limit). I would fix it by (change).” Pick the real bottleneck, not a generic one.

TRAP

“It depends” is the most expensive phrase in the interview. It is not wrong — it is **empty**. The recovery is to add three words: “It depends on X. Here X is (value), so I choose (option).” Now it is judgement instead of avoidance.

Warm-up

WARM-UP

build the reflex

Example 5 [warm-up]

Sort these into **functional** requirements (what it does) and **non-functional** requirements (how well it does it): (a) a user can upload a photo, (b) the feed loads in under 400 ms, (c) a user can follow another user, (d) no post is ever lost, (e) it works when one data centre is down, (f) a post can be deleted.

SOLUTION

Functional: (a) upload a photo, (c) follow a user, (f) delete a post. These are features. A product manager writes them.

Non-functional: (b) 400 ms, (d) durability, (e) availability. These are properties. They are what actually drives the architecture.

The useful habit: list functional requirements to fix the **scope**, list non-functional requirements to fix the **shape**. A 400 ms budget forces a cache. A “never lost” rule forces replication and acknowledgements. Neither is visible in the feature list.

Answer: Functional: a, c, f. Non-functional: b, d, e.

Example 6 [warm-up]

You are 22 minutes into a 45-minute interview and you have finished steps 1 to 4. You have not drawn anything. How do you spend the remaining 23 minutes?

SOLUTION

You are 2 minutes behind. Do not try to win them back by rushing — cut instead.

- Minutes 22–31 (9 min): **draw**. This is non-negotiable; it is the artefact everything else hangs on.
- Minutes 31–40 (9 min): **one** deep dive, not two. Say out loud: “there are two hard parts here, the hot key and the fan-out. I will do the hot key properly; ask me about fan-out if you want it.”
- Minutes 40–45 (5 min): trade-offs, failure modes, 10x. Never cut this. It is where the Judgement tick lives and it is the last thing the interviewer hears.

The thing you cut is the **second** deep dive, because depth on one topic scores higher than shallowness on two.

Example 7 [warm-up]

A candidate is asked to design a photo-sharing app. In minute 3 they say: “First let me design the login system with OAuth and refresh tokens.” What has gone wrong, and what should they say instead?

SOLUTION

What went wrong: they picked a sub-problem that (i) the interviewer did not ask for, (ii) every app has, so it shows nothing specific, and (iii) will eat ten minutes. This is **scope drift**, and it is the number one silent killer.

What to say instead: “Authentication is a solved problem and every service has it, so I will assume there is an existing identity service that gives me a verified userId on each request. I will spend my time on the parts specific to photo sharing: upload, storage, and feed generation. Tell me if you would rather I design auth.”

That sentence does three things: it shows you know what auth involves, it hands the choice back to the interviewer, and it buys back ten minutes.

Example 8 [warm-up]

Which of these is a **hot key** problem?

- (a) a shard holding 10 TB (b) one product getting 40% of all reads (c) a slow SQL query (d) a full disk

SOLUTION

(b). A hot key is when the **traffic** to one key is far above the average, so the one shard that owns that key is overloaded while the other shards idle. Adding shards does not help — the key still lives on exactly one of them.

(a) is a capacity problem: rebalance or add shards, and it goes away. (c) is a query problem: add an index or rewrite it. (d) is an operations problem: delete, archive, or grow the volume.

The fix for a hot key is different from all three: replicate the hot key to many nodes and read from any of them, or put it behind a cache, or split the one key into k sub-keys and sum them.

Answer: (b)

Example 9 [warm-up]

Turn this vague requirement into a number you can design against: “The feed should feel fast.”

SOLUTION

“Fast” is not designable. Convert it into a **percentile budget**:

“p99 latency of GET /v1/feed under 400 ms, measured at the load balancer, for a cold client on 4G.”

Four things make it usable:

1. **Which operation.** Feed read, not upload.
2. **Which percentile.** p99, not average. Average hides the 1% of users who wait 4 seconds; those are the users who uninstall.
3. **The number.** 400 ms.
4. **Where measured.** At the load balancer, so it excludes the user's phone network but includes all of your own work.

Now it drives design: 400 ms total, minus 80 ms of round trip, leaves 320 ms of server budget. A single database query that takes 250 ms no longer fits, so you know you need a cache before you have drawn anything.

PRACTICE · WARM-UP target: 12 min

1. Write four clarifying questions for “design a system to book badminton courts at a sports complex”. For each, say what the answer changes.
2. A service has 800,000 daily users, each doing 12 actions per day. Find requests per day, average QPS, and peak QPS at 3x.
3. Write three endpoints for a service that lets a user shorten a URL and follow it.
4. Give three non-functional requirements for a school report-card portal, each with a number attached.
5. A candidate finishes all seven steps in 30 minutes and then stops talking. What should they do with the remaining 15 minutes? Give two options.

ANSWER KEY

1. (i) How many courts and how far ahead can you book? — decides whether the whole availability grid fits in memory. (ii) Can two people book the same slot? — decides whether you need a database uniqueness constraint or a full locking scheme. (iii) Is payment taken at booking? — decides whether you need idempotency and a refund path. (iv) Do bookings repeat weekly? — a recurring booking is a completely different data model (rule rows, not slot rows).
2. $800\{, \}000 \times 12 = 9\{, \}600\{, \}000$ per day. $9\{, \}600\{, \}000/86\{, \}400 = 111.1$ QPS average. Peak = $111.1 \times 3 = 333$ QPS. This is small: two app servers and one database will do.
3. `POST /v1/links {longUrl, customAlias?} -> 201 {shortCode}; GET /{shortCode} -> 301 Location: <longUrl>; GET /v1/links/{shortCode}/stats -> 200 {clicks, createdAt}`. The redirect is a bare path, not `/v1/`, because it must be short — worth saying.
4. p99 page load under 1.5 s on a 3G connection; results published within 5 minutes of the teacher pressing publish; zero marks lost — every write is durable before the teacher sees “saved”.
5. Option A: offer a second deep dive — “I have time; shall I take the fan-out problem properly?” Option B: go back and strengthen the weakest step — usually the numbers. Do **not** sit in silence, and do not invent new features.

Tier 1 — the seven steps on a low-level design

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

In an LLD round there are no servers. But the seven steps still apply — they just mean smaller things.

Step	In HLD it means	In LLD it means
1 Clarify	which features, which scale	which use cases, which rules vary
2 Estimate	QPS, storage, bandwidth	how many objects, which lookups must be $O(1)$
3 API	HTTP endpoints	the public methods of the main class
4 Data model	tables and keys	classes, fields, and the maps that index them
5 Draw	architecture boxes	a class picture: owns, uses, implements

6 Deep dive	the hot key or fan-out	the rule that changes most often
7 Trade-offs	CAP, cost	inheritance vs composition; where to put the logic

Example 10 [Infosys · pattern]

Design the lending desk of a college library. A member borrows a physical book and returns it. Late returns are fined. Run all seven steps and produce working code.

SOLUTION

Step 1 — Clarify (what I asked, and what the answer changed).

Question	Answer, and what it decided
Two members want “Operating Systems”. Is that one thing or two?	Two. A library owns several physical copies of one title. So I need two classes: Title (the work) and Copy (the barcode on the shelf). This single answer is the whole design.
Do all members get the same loan period?	No — students 14 days, staff 30 days, and it may change next year. So the rule must be swappable, not an if inside the desk.
How is the fine computed?	Rupees 2 per day, capped at Rupees 100. The cap and the rate change per library, so this is also a swappable rule.
Reservations, renewals, e-books?	Out of scope — I said so and the interviewer agreed.

Step 2 — Estimate (the LLD version). A college library: about 40,000 copies, 6,000 members, roughly 300 issues a day. Everything fits in memory easily. What matters is which lookups must be instant: “is barcode BC-1 on loan?” happens on every return, so that must be a hash map lookup, not a scan of all loans. That one sentence is the whole estimation step here, and it directly creates the `openLoanByBarcode` map below.

Step 3 — API (public methods).

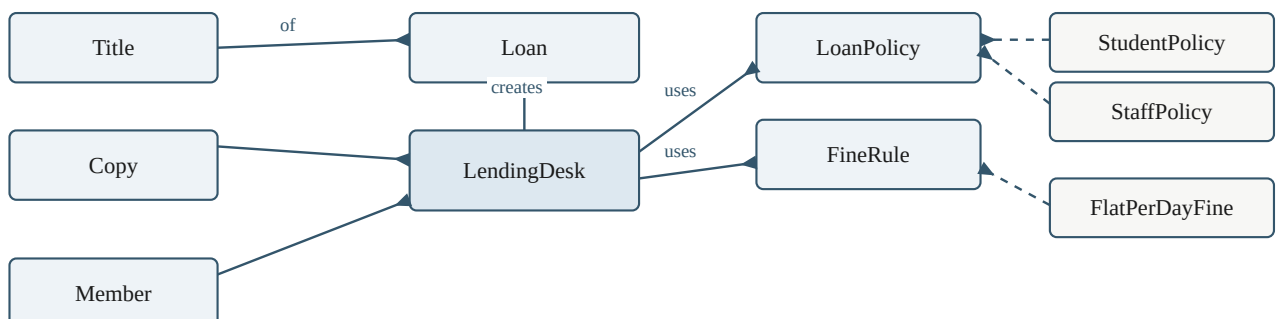
The only four methods the outside world needs

TEXT

```
addCopy(Copy c)
issue(barcode, member, today)    -> Loan      (throws if not allowed)
giveBack(barcode, today)        -> fine in paise
openLoanCount(member)           -> int        (internal helper)
```

Fines are in **paise** (integer), never in rupees as a double. Money in floating point is a classic rejection.

Step 4 — Data model (classes and the maps that index them).



One **Title**, many **Copy**. **LendingDesk** holds the maps and owns nothing else.

The class picture. Solid = owns/uses, dashed = implements.

Step 5 — The code. This runs as written with node.

Entities: the **Title/Copy** split is the key idea

JS

```

const CopyStatus = Object.freeze({
  ON_SHELF: "ON_SHELF", ON_LOAN: "ON_LOAN", LOST: "LOST", REPAIR: "REPAIR",
});
const Tier = Object.freeze({ STUDENT: "STUDENT", STAFF: "STAFF" });

class Title {
  // the WORK: "Operating Systems"
  constructor(isbn, name, author) {
    this.isbn = isbn; this.name = name; this.author = author;
  }
}

class Copy {
  // the PHYSICAL BOOK: one barcode each
  constructor(barcode, title) {
    this.barcode = barcode; this.title = title;
    this.status = CopyStatus.ON_SHELF;
  }
}

class Member {
  constructor(memberId, name, tier) {
    this.memberId = memberId; this.name = name; this.tier = tier;
  }
}

class Loan {
  constructor(loanId, copy, member, issuedOnDay, dueOnDay) {
    this.loanId = loanId; this.copy = copy; this.member = member;
    this.issuedOnDay = issuedOnDay; this.dueOnDay = dueOnDay;
    this.returnedOnDay = null; // null while the loan is open
  }
  get isOpen() { return this.returnedOnDay === null; }
}

```

Dates here are plain day numbers (day 0, day 14) so the example stays about the design and not about time zones. In real code use a date library, and store the due date as a UTC instant, because a library that opens branches in two countries will otherwise disagree with itself about when “tomorrow” is.

Step 6 — Deep dive: the rule that changes most often. Loan length and fine rate change every year and differ per library. Putting them in `if` statements inside `LendingDesk` means editing the desk whenever a rule changes. Putting each rule in its own small object means **adding** a class instead of editing a tested one. That is the Strategy pattern, and this is the honest reason to use it.

Two rules, four tiny classes

JS

```

class StudentPolicy { get loanDays() { return 14; } get maxOpenLoans() { return 3; } }
class StaffPolicy { get loanDays() { return 30; } get maxOpenLoans() { return 10; } }

class FlatPerDayFine {
  constructor(perDayPaise, capPaise) {
    this.perDayPaise = perDayPaise; this.capPaise = capPaise;
  }
  fineInPaise(daysLate) {
    if (daysLate <= 0) return 0; // on time or early
    return Math.min(this.capPaise, this.perDayPaise * daysLate); // cap applied
  }
}

```



```

last
}
}

class LendingError extends Error {}

```

A language point you will be asked about. JavaScript has no interface keyword. LendingDesk simply calls `policy.loanDays`, and any object with that property works — this is duck typing, and it gives you the same swappability with less ceremony.

But a service-company interviewer will often ask this question in Java vocabulary. **What they expect to hear:** “LoanPolicy is an interface with `int loanDays()` and `int maxOpenLoans()`; StudentPolicy and StaffPolicy implement it; LendingDesk depends on the interface, not the concrete class.” Say that sentence in Java terms when asked in Java terms, then add: “in JavaScript the same design needs no interface declaration, because the call site only requires the method to exist.”

The desk: every map exists to make one lookup $O(1)$

JS

```

class LendingDesk {
  constructor(fineRule) {
    this.fineRule = fineRule;
    this.copies = new Map();           // barcode -> Copy
    this.openLoanByBarcode = new Map(); // barcode -> Loan (0(1) return)
    this.loansByMember = new Map();    // memberId -> Loan[]
    this.policies = new Map([
      [Tier.STUDENT, new StudentPolicy()],
      [Tier.STAFF, new StaffPolicy()],
    ]);
    this.seq = 0;
  }

  addCopy(copy) { this.copies.set(copy.barcode, copy); }

  openLoanCount(member) {
    const loans = this.loansByMember.get(member.memberId) ?? [];
    return loans.filter(l => l.isOpen).length;
  }

  issue(barcode, member, todayDay) {
    const copy = this.copies.get(barcode);
    if (!copy) throw new LendingError(`no such copy: ${barcode}`);
    if (copy.status !== CopyStatus.ON_SHELF) throw new LendingError("copy not on shelf");
    const policy = this.policies.get(member.tier);
    if (this.openLoanCount(member) >= policy.maxOpenLoans)
      throw new LendingError("loan limit hit");

    const loan = new Loan(`L${++this.seq}`, copy, member,
      todayDay, todayDay + policy.loanDays);
    copy.status = CopyStatus.ON_LOAN;
    this.openLoanByBarcode.set(barcode, loan);
    if (!this.loansByMember.has(member.memberId))
      this.loansByMember.set(member.memberId, []);
    this.loansByMember.get(member.memberId).push(loan);
    return loan;
  }
}

```

```

giveBack(barcode, todayDay) {
  const loan = this.openLoanByBarcode.get(barcode);
  if (!loan) throw new LendingError("copy was not on loan");
  this.openLoanByBarcode.delete(barcode);
  loan.returnedOnDay = todayDay;
  loan.copy.status = CopyStatus.ON_SHELF;
  return this.fineRule.fineInPaise(todayDay - loan.dueOnDay);
}
}

```

The test run

JS

```

const desk = new LendingDesk(new FlatPerDayFine(200, 10000)); // Rs 2/day, cap Rs 100
const t = new Title("978-1", "Operating Systems", "Rao");
const c1 = new Copy("BC-1", t), c2 = new Copy("BC-2", t);
desk.addCopy(c1); desk.addCopy(c2);

const riya = new Member("M1", "Riya", Tier.STUDENT);
const l = desk.issue("BC-1", riya, 0);
console.log(`issued ${l.loanId}, due on day ${l.dueOnDay}`);
console.log(`copy status = ${c1.status}`);

try { desk.issue("BC-1", riya, 0); }
catch (e) { console.log(`second issue blocked: ${e.message}`); }

console.log(`fine on time      = ${desk.giveBack("BC-1", 14)}`);
desk.issue("BC-2", riya, 0);
console.log(`fine 5 days late   = ${desk.giveBack("BC-2", 19)}`);
desk.issue("BC-2", riya, 0);
console.log(`fine 200 days late = ${desk.giveBack("BC-2", 214)}`);

```

Run it with node and this is the actual output:

Measured output

TEXT

```

issued L1, due on day 14
copy status = ON_LOAN
second issue blocked: copy not on shelf
fine on time      = 0
fine 5 days late   = 1000
fine 200 days late = 10000

```

Read the last three lines. On time: 0 paise. Five days late: $200 \times 5 = 1000$ paise, which is Rupees 10. Two hundred days late: $200 \times 200 = 40,000$ paise, but the cap is 10,000 paise, so Rupees 100. The cap works.

TRAP

Why Map and not a plain object for copies? A plain object turns every key into a string, so a numeric barcode 1234 and the string "1234" become the same entry, and keys like constructor or toString collide with things already on the object. Map keeps the key's type, keeps insertion order, and has a real .size. In an LLD round, say "Map because the keys are data, not property names." It is a one-line answer that shows you know the difference.

Step 7 — Trade-offs, decided.

Choice	The two sides	Decision
Fine logic: inside <code>Loan</code> or in a separate <code>FineRule</code> ?	Inside <code>Loan</code> is fewer classes and easier to read. Separate means one more interface but the rule can change without touching the entity.	Separate. The rule changes yearly; the entity does not. Keep data and changing policy apart.
Tier behaviour: subclass <code>Member</code> or give <code>Member</code> a <code>tier</code> field?	A <code>StudentMember</code> subclass reads naturally. But a student who becomes staff would need a new object and every reference updated.	Field plus policy map. Composition survives the change of tier; inheritance does not.
<code>status</code> on <code>Copy</code> , or derive it from the loans map?	A field can drift out of sync with the map — two sources of truth. Deriving is always correct but costs a lookup.	Keep the field, but only <code>LendingDesk</code> may write it. One writer means it cannot drift. If this were multi-threaded I would derive instead.
Money as rupees in a float, or paise in an integer?	Floats are convenient. In JavaScript every number is a float, and <code>0.1 + 0.2</code> prints <code>0.30000000000000004</code> .	Integer paise. Never floating point for money. No exceptions.

What breaks at 10x? Nothing — 400,000 copies still fit in memory. What breaks is **multi-desk**: two librarians on two machines both calling `issue("BC-1", ...)`. Both read `ON_SHELF`, both succeed, one book leaves twice. The fix moves the design out of memory: make `openLoanByBarcode` a database table with `barcode` as a unique key on open loans, so the second insert fails. That is the bridge from LLD to HLD, and it is a very good sentence to end on.

TRAP

The most common wrong answer to this question is a single `Book` class holding both the title and the barcode. It seems fine until the interviewer says “the library has five copies of that book”. Then every field is wrong at once. When a question mentions a real-world object, always ask: **is this one thing, or a type of thing and its instances?**

Example 11 [TCS NQT · pattern]

Run only **step 1** for: “Design a parking lot.” Give five clarifying questions with the design consequence of each.

SOLUTION

1. **“How many floors and how many spot sizes?”** — one size means a single counter is enough. Three sizes (bike, car, van) means a counter per size per floor, and the “find a spot” method becomes a search, not a decrement.
2. **“Is the fee by hour, by slab, or flat?”** — decides whether `FeeStrategy` needs the entry time only, or entry plus exit plus a slab table. It is the rule most likely to vary, so it becomes an interface.
3. **“Can a van occupy two car spots?”** — if yes, allocation is no longer “find one free spot”; it is “find two adjacent free spots”, which is a different algorithm and needs spots to know their neighbours.
4. **“What happens if the ticket is lost?”** — forces a lost-ticket flat fee and a lookup by number plate, which means an index on plate that you would otherwise not build.
5. **“Do we need to show live free-spot counts on a display board?”** — if yes, a counter per floor per size must be kept in sync with allocation, and now you have a concurrency question at the entry gates.

Then: “I will leave out payments, monthly passes, and reservations.”

Example 12 [Capgemini · pattern]

Here is a class. Name three things wrong with it **for an interview**, and fix each.

Submitted by a candidate

JS

```

class Order {
  constructor(id, items, total, status) {
    this.id = id; this.items = items; this.total = total; this.status = status;
  }
  saveToDatabase() { /* SQL here */ }
  sendEmail() { /* SMTP here */ }
  calculateTotal(customerType) {
    if (customerType === "GOLD") return this.total * 0.8;
    else if (customerType === "SILVER") return this.total * 0.9;
    else return this.total;
  }
}

```

SOLUTION

Problem 1 — it does three unrelated jobs. Order holds data, talks to a database, and sends email. Three reasons to change one class. If the mail provider changes, you edit Order. If the schema changes, you edit Order. **Fix:** keep Order as data plus order rules. Move persistence to OrderRepository, move mail to NotificationService.

Problem 2 — the discount is a chain of ifs on a string. Adding a PLATINUM tier means editing this method, re-testing it, and re-deploying it. A typo in the string silently gives **no** discount instead of failing loudly. **Fix:** one small class per rule, chosen from a map.

Problem 3 — money in a floating-point number, in a field anyone can overwrite. JavaScript numbers are all doubles, so rupee amounts accumulate tiny errors; and `order.total = -5` is legal from anywhere in the program. **Fix:** integer paise in a `#private` field, computed once in the constructor.

Both fixes together

JS

```

class PercentOffPolicy {
  constructor(percentOff) { this.percentOff = percentOff; }
  applyToPaise(p) { return p - Math.floor((p * this.percentOff) / 100); }
}

const DISCOUNTS = new Map([ // adding PLATINUM adds a row, not an if
  ["GOLD", new PercentOffPolicy(20)],
  ["SILVER", new PercentOffPolicy(10)],
  ["NONE", new PercentOffPolicy(0)],
]);

class Order {
  #totalPaise; // truly private, not just a naming habit
  constructor(id, itemIds, subtotalPaise, discount) {
    this.id = id; this.itemIds = itemIds;
    this.#totalPaise = discount.applyToPaise(subtotalPaise);
  }
  get totalPaise() { return this.#totalPaise; }
}

console.log("0.1 + 0.2 =", 0.1 + 0.2);
console.log("equals 0.3? =", 0.1 + 0.2 === 0.3);
const o = new Order("o1", ["i1"], 99995, DISCOUNTS.get("GOLD"));
console.log("GOLD on 99995 p =", o.totalPaise, "paise");
console.log("visible fields =", Object.keys(o));

```

Measured output

TEXT

```
0.1 + 0.2      = 0.30000000000000004
equals 0.3?    = false
GOLD on 9995 p = 7996 paise
visible fields = [ 'id', 'itemIds' ]
```

Look at line 2. `0.1 + 0.2 === 0.3` is **false**. That is not a JavaScript quirk to laugh at — it is why Rupees 999.95 becomes Rupees 999.9499999999999 after enough arithmetic, and why a reconciliation report ends up one paisa out. Integers only. Line 4 shows `#totalPaise` really is invisible from outside.

Say this line: “The test for whether a rule belongs in its own class is: does it change on a different schedule from the rest of the class? Discounts change every festival season. The order’s fields do not.”

Example 13 [Cognizant · pattern]

Run steps 3 and 4 (API and data model) for a hostel room allocation system: students apply, a warden allocates, a student can swap with another student if both agree.

SOLUTION

Step 3 — API.

Five endpoints

TEXT

```
POST /v1/applications
  body { studentId, preferredBlock, roommatePrefStudentId? }
  resp 201 { applicationId }

POST /v1/allocations
  body { applicationId, roomId, bedNo }
  resp 201 { allocationId }
  note warden only; fails if that bed is already allocated

GET /v1/students/{studentId}/allocation
  resp 200 { roomId, bedNo, block, allocatedAt }

POST /v1/swaps
  body { fromAllocationId, toAllocationId }
  resp 201 { swapId, status: "PENDING_OTHER" }

POST /v1/swaps/{swapId}/accept
  resp 200 { status: "DONE" }
  note both sides must accept; this is the second accept
```

The swap is **two** calls, not one, because it needs two people’s consent. Modelling a two-party agreement as a single endpoint is a common mistake — there is no moment at which both students are making the same request.

Step 4 — Data model.

Four tables; note the unique constraint

TEXT

```
rooms(room_id PK, block, floor, capacity)

beds(bed_id PK, room_id, bed_no)
  UNIQUE (room_id, bed_no)

allocations(allocation_id PK, student_id, bed_id, allocated_at, released_at)
  UNIQUE (bed_id) WHERE released_at IS NULL -- one live occupant per bed
  UNIQUE (student_id) WHERE released_at IS NULL -- one live bed per student
```

```
swaps(swap_id PK, from_alloc_id, to_alloc_id, state, created_at)
state in (PENDING_OTHER, DONE, CANCELLED, EXPIRED)
```

The two partial unique constraints are the whole design. They mean that even if two wardens click “allocate” at the same instant, the database refuses the second one. Without them you need a lock, a queue, or hope.

Why released_at instead of deleting the row? Because next year someone will ask “who lived in 3B last year?”. Soft-release keeps history at the cost of every query carrying `WHERE released_at IS NULL`. That is a trade I would take here; for a high-write table I would not, and would move old rows to an archive table instead.

PRACTICE · TIER 1 target: 35 min

1. Run all seven steps for a **vending machine**. You must produce: the clarifying questions, the public methods, the class list, a state list, and one trade-off decided.
2. For a **movie ticket booking** LLD, name the one rule most likely to change and put it behind an interface. Write the interface and two implementations.
3. A candidate models a chess game with a single `Board` class containing a `move(int, int, int, int)` method holding all the rules for all six piece types. Give two concrete problems and the fix.
4. Write the data model for a **split-the-bill** app: a group, members, expenses, and who owes whom. State your primary keys.
5. A class has a method `process(String type)` with a 7-branch `if/else` on `type`. Give the two-sentence argument for replacing it, and the one situation where you would leave it alone.

ANSWER KEY

1. Clarify: how many item slots; does it give change and in which coins; can an item be reserved while paying; what happens if payment succeeds and dispensing fails. Methods: `insertCoin`, `selectItem`, `dispense`, `refund`, `restock`. Classes: `Machine`, `Slot`, `Item`, `CoinBox`, `MachineState`. States: `IDLE`, `ITEM_SELECTED`, `HAS_MONEY`, `DISPENSING`, `OUT_OF_SERVICE`. Trade-off: one big switch on state versus a `State` object per state — choose the `State` object, because each state then owns exactly the transitions that are legal from it, and an illegal transition becomes impossible rather than merely untested.
2. Pricing. One method, `priceInPaise(show, seat)`, implemented by `WeekdayFlatPrice` and `WeekendPremiumPrice`, chosen from a `Map` keyed by day type. Seat class (gold / silver) is a second axis: either compose two rule objects or pass the seat into the one rule, as shown.
3. (i) Adding a new piece means editing `Board`, which is also responsible for occupancy and check detection — so a bishop bug can break the king. (ii) `move` cannot be unit-tested per piece. Fix: one small class per piece, each with `canMove(board, from, to)`; `Board` only checks occupancy, turn order, and whether the king is left in check. (In Java vocabulary the interviewer will call the shared shape an interface `Piece`; in JavaScript it is just the method every piece class defines.)
4. `groups(group_id PK, name)`; `members(group_id, user_id, PK(group_id, user_id))`; `expenses(expense_id PK, group_id, payer_id, amount_paise, description, created_at)`; `expense_shares(expense_id, user_id, share_paise, PK(expense_id, user_id))`. Balances are computed by summing shares, not stored — a stored balance is a second source of truth that drifts.
5. Replace it when the branches change at different times or are added by different people; each branch becomes a class and adding an eighth type stops touching tested code. Leave it alone when the set of types is fixed by an external standard and will never grow (HTTP methods, days of the week) — there, seven classes is more code for zero flexibility you will ever use.

Tier 2 — the seven steps on one real service

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 14 [Shopee · pattern]

Design a coupon redemption service for a South-East Asian shopping app. During checkout the app asks “is this coupon valid for this cart?” and, if the user confirms, “redeem it”. Run all seven steps.

SOLUTION

Step 1 — Clarify.

Question	Answer, and what it decided
How many buyers per day, and how many use a coupon?	8 million daily buyers; about 15% apply a coupon. That is 1.2 M redemptions a day — small. The validation traffic is the real load.
How many times is a coupon checked before it is redeemed?	About 10 — the app re-validates on every cart change. So validation is 12 M/day. This is the number that sizes the service.
Can one coupon be used twice by the same user?	No. Per-user limit, usually 1. This is the hard part of the whole design.
Is there a global cap, like “first 10,000 users”?	Yes. That is a hot counter and a flash-sale problem.
Is a slightly stale “still available” answer acceptable?	For validation , yes — it is only a preview. For redemption , no: it must be exact. This split is the key architectural insight.

Out of scope: creating coupons in the admin console, payments, and the cart service itself.

Step 2 — Estimate. Every division shown.

THE ARITHMETIC

Redemptions per day.

$$8\{, \}000\{, \}000 \times 0.15 = 1\{, \}200\{, \}000 \text{ redemptions/day}$$

Average redemption QPS.

$$1\{, \}200\{, \}\frac{000}{86}\{, \}400 = 13.9 \approx 14 \text{ QPS}$$

Peak redemption QPS. Checkout is concentrated: about 20% of a day’s orders land in the busiest hour.

$$1\{, \}200\{, \}000 \times 0.20 = 240\{, \}000 \text{ in that hour}$$

$$240\{, \}\frac{000}{3}\{, \}600 = 66.7 \approx 67 \text{ QPS}$$

Validation traffic — 10 checks per redemption.

$$1\{, \}200\{, \}000 \times 10 = 12\{, \}000\{, \}000 \text{ validations/day}$$

$$12\{, \}000\{, \}\frac{000}{86}\{, \}400 = 138.9 \approx 139 \text{ QPS average}$$

$$\frac{12\{, \}000\{, \}000 \times 0.20}{3}\{, \}600 = 2\{, \}400\{, \}\frac{000}{3}\{, \}600 = 666.7 \approx 667 \text{ QPS peak}$$

Storage. A redemption row: coupon id 16 B, user id 8 B, order id 16 B, timestamp 8 B, amount 8 B, status 1 B, plus index and row overhead — call it 120 B.

$$1\{, \}200\{, \}000 \times 120 \text{ B} = 144\{, \}000\{, \}000 \text{ B} = 144 \text{ MB/day}$$

$$144 \text{ MB} \times 365 = 52\{, \}560 \text{ MB} = 52.6 \text{ GB/year}$$

Coupon definitions. 200,000 live coupons $\times$ 300 B = 60 MB. That is the whole catalogue — it fits in memory on every server, which is a very useful fact.

Reading the numbers. 667 QPS at peak is **small**. This is not a distributed systems problem; it is a correctness problem. 52.6 GB a year fits on one database for many years. Say this out loud — recognising that a system is small is itself a scored signal, because it stops you from over-engineering.

Step 3 — API.

Three endpoints; note the reservation

TEXT

```
POST /v1/coupons/validate
  body { userId, couponCode, cartTotalPaise, itemIds[] }
  resp 200 { valid: true, discountPaise: 5000, reason: null }
        200 { valid: false, discountPaise: 0, reason: "ALREADY_USED" }
  note  read-only, cacheable, may be a few seconds stale, no side effect

POST /v1/coupons/reserve
  body { userId, couponCode, orderId }
  header Idempotency-Key: <orderId>
  resp 201 { reservationId, expiresAt }
        409 { reason: "EXHAUSTED" | "ALREADY_USED" }
  note  exact, takes one unit from the global cap, held for 10 minutes

POST /v1/coupons/commit
  body { reservationId, paymentId }
  resp 200 { redemptionId }
  note  called after payment succeeds; converts reservation to redemption
```

Why three calls and not one? Because payment sits between “the user chose the coupon” and “the order exists”. If you decrement the global cap at validate time, every browsing user burns a coupon. If you decrement only after payment, two users can both pay for the last unit. A short reservation solves both: it is taken at a moment the user has committed, and it expires by itself if payment fails.

Step 4 — Data model.

Four tables; the unique key is the whole correctness story

TEXT

```
coupons(coupon_code PK, kind, value_paise, min_cart_paise,
        starts_at, ends_at, global_cap, per_user_cap, status)

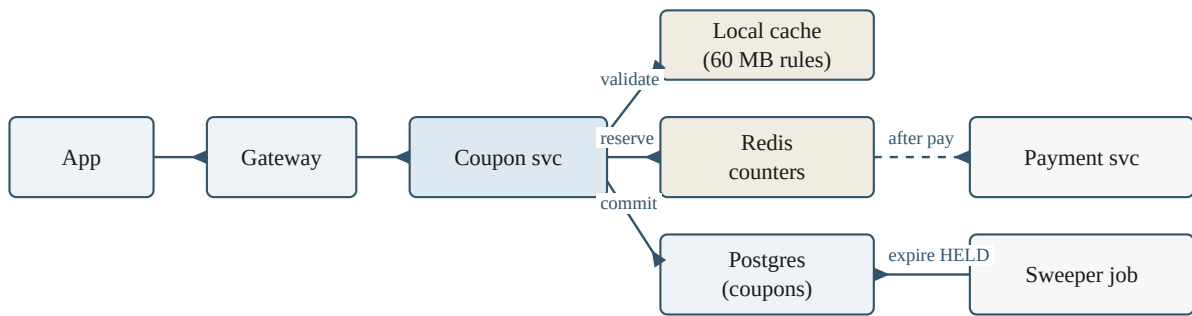
coupon_counters(coupon_code, shard_no, used_count,
                PK (coupon_code, shard_no))      -- see deep dive

reservations(reservation_id PK, coupon_code, user_id, order_id,
              expires_at, state)
  UNIQUE (coupon_code, user_id) WHERE state IN ('HELD', 'COMMITTED')
  index (expires_at) WHERE state = 'HELD'      -- the sweeper reads this

redemptions(redemption_id PK, coupon_code, user_id, order_id,
             amount_paise, redeemed_at)
  UNIQUE (coupon_code, user_id)                -- per-user cap = 1, enforced
  UNIQUE (order_id, coupon_code)                -- retry safety
```

Partition key: coupon_code. Every single query names a coupon, so every query hits one shard. The price of this choice is that one popular coupon puts all its load on one shard — which is exactly the deep dive.

Step 5 — Draw it.



Validate never touches Postgres. Only reserve and commit do — and they are 67 QPS, not 667.

Validate is cached and cheap. Reserve and commit take the exact path.

Step 6 — Deep dive 1: the global cap under a flash sale.

The cap is “first 10,000 users only”. Naive version:

```
SELECT used_count FROM coupon_counters WHERE coupon_code = 'FLASH50';
if (used_count < 10000) UPDATE ... SET used_count = used_count + 1;
```

Two requests read 9,999 at the same moment. Both pass the check. Both increment. Now 10,001 people have the coupon. This is a **lost update** and it will happen.

Fix 1 — make the check and the increment one atomic statement.

```
UPDATE coupon_counters SET used_count = used_count + 1
  WHERE coupon_code = 'FLASH50' AND used_count < 10000;
-- rows affected = 1 -> you got it.  rows affected = 0 -> sold out.
```

The database evaluates the condition and the write under one row lock. No race. This alone fixes correctness.

Fix 2 — the row is now a hot key. Every request for FLASH50 queues behind one row lock. If each update holds the lock for 1 ms, that row can serve at most 1,000 updates per second, and a flash sale can deliver far more than that. So **split the counter**: give the coupon 10 shards, each with a cap of 1,000.

Sharded counter — pick a shard from the user id

TEXT

```
shard = hash(userId) % 10
UPDATE coupon_counters SET used_count = used_count + 1
  WHERE coupon_code = 'FLASH50' AND shard_no = :shard AND used_count < 1000;
```

Ten independent rows, ten independent locks, roughly ten times the throughput.

The cost, stated honestly: the shards drain unevenly. If shard 3 fills first, a user hashing to shard 3 is told “sold out” while units still remain on shard 7. So you will hand out fewer than 10,000 in practice. **How many fewer?** Do not guess — measure.

How many units does sharding strand? Simulate it.

JS

```
const hash32 = (s) => { // deterministic 32-bit string hash
  (FNV-1a)
  let h = 0x811c9dc5;
  for (let i = 0; i < s.length; i++) {
    h ^= s.charCodeAt(i);
    h = Math.imul(h, 0x01000193) >>> 0;
  }
  return h >>> 0;
};

const CAP = 10000, USERS = 10000;
```

```

const run = (shards, retries) => {
  const perShard = CAP / shards;
  const used = new Array(shards).fill(0);
  let granted = 0;
  for (let i = 0; i < USERS; i++) {
    const start = hash32(`u${i}`) % shards;
    for (let k = 0; k <= retries; k++) { // try neighbours on a miss
      const s = (start + k) % shards;
      if (used[s] < perShard) { used[s]++; granted++; break; }
    }
  }
  return CAP - granted; // units nobody could claim
};

for (const shards of [1, 10, 100, 1000]) {
  const row = [0, 1, 2].map(r => `retries=${r}: ${run(shards, r)}`).join(" ");
  console.log(`shards=${String(shards).padStart(4)} stranded units -> ${row}`);
}

```

Measured output

TEXT

shards= 1	stranded units ->	retries=0: 0	retries=1: 0	retries=2: 0
shards= 10	stranded units ->	retries=0: 44	retries=1: 29	retries=2: 19
shards= 100	stranded units ->	retries=0: 219	retries=1: 114	retries=2: 78
shards=1000	stranded units ->	retries=0: 1155	retries=1: 729	retries=2: 592

Read the table carefully, because it contradicts the fear.

- 10 shards strand **44 units out of 10,000** — 0.44%. That is nothing.
- 1,000 shards strand 1,155 — 11.6%. That is a real loss.
- Two retries roughly halve the loss at every shard count.

The rule this gives you: **shard just enough**. The stranding cost grows with the number of shards, so pick the smallest count that clears your throughput need.

Decision: 10 shards with 2 retries — roughly 10x the write throughput for a 0.19% shortfall. If the cap were small (say 100 units), I would not shard at all: one row handles 1,000 updates a second, 100 units are gone in a moment anyway, and exactness matters more than throughput when the prize is tiny.

Step 6 — Deep dive 2: the reservation that never gets committed.

A user reserves, then closes the app. The unit is held forever. Two ways to release it:

Approach 1 — A background sweeper job (chosen)

Every 30 seconds, run `UPDATE reservations SET state='EXPIRED' WHERE state='HELD' AND expires_at < now()` and decrement the matching counter shard. The index `(expires_at)` WHERE `state='HELD'` makes this a cheap range scan — the sweeper only reads rows that are actually due.

Approach 2 — Lazy expiry at read time (rejected as the only mechanism)

When someone asks for the coupon, notice any expired holds and release them then. Costs nothing when idle. But if nobody asks, units stay locked forever — and the whole point of a flash sale is that people **are** asking, so the lazy path fires exactly when you least want extra work in the request.

Decision: the sweeper, plus lazy expiry as a safety net. The sweeper is the guarantee; lazy expiry covers the 30-second window between sweeps. Running both is safe because the state transition `HELD -> EXPIRED` is idempotent: whoever gets there second changes zero rows.

Step 7 — Trade-offs, failure modes, 10x.

Choice	Both sides	Decision
Counters in Redis or in Postgres?	Redis: 100 K ops/s, atomic DECR, but a crash can lose the last second of writes. Postgres: durable and transactional with the redemption row, but 10 K writes/s per node.	Postgres. Peak here is 67 reserve/s — three orders of magnitude below the limit. Durability is worth more than speed we do not need. Redis is used only as a read cache.
Validate exact or cached?	Exact means every keystroke in the cart hits the database — 667 QPS of exact reads for an answer that is only a preview. Cached means a user may briefly see “valid” for a coupon that just sold out.	Cached, 5-second TTL. The reserve call is the source of truth and it returns a clean 409. Showing “sorry, just sold out” at checkout is acceptable; melting the database to avoid it is not.
Per-user cap in code or as a unique index?	Code is flexible (caps of 2, 3, n). A unique index handles only cap = 1, but it cannot be beaten by any race.	Unique index for cap = 1 (which is 95% of coupons), plus a counted check for caps above 1. Use the strongest tool that fits the common case.

Failure modes.

- **Redis dies.** Validation falls through to Postgres. 667 QPS of simple indexed reads is well within one Postgres node. Degraded, not down. Acceptable.
- **Coupon service dies.** Checkout must not die with it. Decision: the cart treats a timeout as “no discount available”, shows the cart without the coupon, and lets the user retry. Never block a purchase because a discount service is slow.
- **Payment succeeds but commit fails.** The reservation stays HELD, then expires — the user paid full price and lost the discount. Fix: commit is retried from a durable outbox written in the same transaction as the payment record, and it is idempotent on reservationId, so retrying is always safe.
- **Sweeper dies.** Units leak slowly. Alert on `count(state='HELD' AND expires_at < now() - 5 min) > 100`.

At 10x (80 M buyers/day). Validation goes to 6,670 QPS peak and redemption to 670 QPS. The first thing to break is **not** the database — it is the single hot coupon row during a national sale. I would move the counter for campaign coupons only into a sharded in-memory counter with periodic write-back, and keep everyday coupons exactly as they are. Two paths, chosen by coupon type, because the shapes of the traffic are genuinely different.

Example 15 [Grab · pattern]

Run **step 2 only** for a driver-location service: 200,000 drivers are online, each sends its position every 4 seconds. Payload is 40 bytes. Locations are kept for 24 hours for dispute resolution.

SOLUTION

Write QPS. Every driver sends one ping per 4 seconds, so in any one second, a quarter of the drivers send.

$$200\{, \} \frac{000}{4} = 50\{, \} 000 \text{ writes per second}$$

Bandwidth in.

$$50\{, \} 000 \times 40 \text{ B} = 2\{, \} 000\{, \} 000 \text{ B/s} = 2 \text{ MB/s} = 16 \text{ Mbps}$$

Tiny. The bytes are not the problem; the **operation count** is.

Pings per day. Seconds in a day divided by 4, times drivers:

$$86\{, \} \frac{400}{4} = 21\{, \} 600 \text{ pings per driver per day}$$

$$200\{, \} 000 \times 21\{, \} 600 = 4\{, \} 320\{, \} 000\{, \} 000 \text{ pings/day}$$

Storage for 24 hours.

$$4\{, \}320\{, \}000\{, \}000 \times 40 \text{ B} = 172\{, \}800\{, \}000\{, \}000 \text{ B} = 172.8 \text{ GB/day}$$

The conclusion you must state. 50,000 writes per second is far past a single relational database (roughly 5–20 K writes/s). And 4.3 billion rows a day is not something you keep in Postgres. So:

1. **Current position** goes in memory (one key per driver, overwritten each ping). That is 200,000 keys, about 40 MB — trivially small, and the write rate is what in-memory stores are built for.
2. **The 24-hour history** is a different product with a different shape. Append it to a log (Kafka) and land it in a time-series or columnar store, compressed. It is never read in the hot path.

Splitting “the latest value” from “the history” is the move. They have the same data and completely different access patterns, so they get different storage.

Example 16 [Agoda · pattern]

Run **step 7 only**: you have designed a hotel search that reads from a search index refreshed from the booking database every 60 seconds. Give the trade-off, two failure modes, and the 10x answer.

SOLUTION

The trade-off, named and decided. Reading from the search index gives fast, rich queries (filter by price, stars, map box, free text) that a transactional database cannot serve at that speed. Reading from the booking database directly gives perfect freshness but every search becomes a heavy multi-table scan, and search traffic is 100x booking traffic — it would crush the database that the actual bookings depend on.

Decision: search from the index. The cost is that a room can appear available for up to 60 seconds after it is taken. That is acceptable **because the booking step re-checks availability against the real database**, so the worst outcome is a clear “just taken, here are similar rooms” message rather than a double booking. I would reduce the 60 seconds to 5 for the top 1% of properties by traffic, where a stale hit is most likely.

Failure modes.

- **The refresh pipeline stalls.** Results silently get older and older. This is dangerous precisely because nothing errors. Fix: publish the index’s newest timestamp as a metric and alert when $\text{now} - \text{newest} > 5 \text{ minutes}$. An index that is merely stale looks healthy; you must measure staleness directly.
- **The index cluster dies.** Search is the front door of the product, so “no results” is close to “site down”. Fix: a fallback that serves a cached list of popular hotels per city with filters disabled, plus a banner. Degraded, browsable, still bookable.

At 10x. The first thing to break is the **refresh**, not the queries. Reads scale by adding index replicas, which is easy. But the refresh is one pipeline rebuilding the same documents, and 10x the bookings means 10x the change events into one writer. I would switch from periodic full refresh to change-data-capture with partial document updates, so the work is proportional to what changed rather than to catalogue size.

PRACTICE · TIER 2 target: 40 min

1. Run steps 1 and 2 for a **food-delivery order-tracking service**: 5 million orders a day, each order sends 6 status updates, and every customer polls the status every 5 seconds while waiting (average wait 25 minutes). Find the polling QPS and then argue for or against polling.
2. Run step 3 and 4 for a **wallet top-up service**. There must be no possibility of a double credit when the client retries.
3. Run step 6 for a **“5 free articles a month” paywall**: the counter must be per user per month and must survive the user clearing cookies.
4. For a URL shortener at 50 million redirects a day, compute the peak QPS and decide whether the redirect should be served from a cache, a database, or both. Show the arithmetic.
5. Give a trade-off, decided, for: “store the user’s uploaded image in the database as a BLOB, or in object storage with the URL in the database”.

ANSWER KEY

1. Status writes: $5\{, \}000\{, \}000 \times 6 = 30\{, \}000\{, \}000/\text{day} = 30\{, \}000\{, \}000/86\{, \}400 = 347$ QPS average; dinner peak, 25% of orders in 2 hours, gives $(0.25 \times 30\{, \}000\{, \}000)/7\{, \}200 = 1\{, \}042$ QPS. Polling: each waiting customer polls $25 \times 60/5 = 300$ times, so $5\{, \}000\{, \}000 \times 300 = 1.5$ billion polls/day = $17\{, \}361$ QPS average, and roughly 52,000 at peak — **50x the write traffic, for data that changed 6 times**. Argue against polling: use a push (websocket or mobile push) for status changes, and keep a slow poll (every 30 s) only as a reconnect safety net. If you must poll, serve it from a cache with a 3-second TTL and return **304 Not Modified** with an ETag so the body is not re-sent.
2. `POST /v1/wallets/{walletId}/topups` with `Idempotency-Key: <clientId>`, body `{ amountPaise, paymentRef }`, response `201 { topupId, newBalancePaise }`. Tables: `wallets(wallet_id PK, balance_paise, version)`; `ledger(entry_id PK, wallet_id, delta_paise, reason, idempotency_key, created_at)` with `UNIQUE (wallet_id, idempotency_key)`. The unique index is the guarantee: the second attempt fails the insert, and the handler then returns the **first** attempt's response. Balance is updated in the same transaction as the ledger insert, never separately.
3. Key the counter (`userId, yyyyymm`) and store it server-side, so clearing cookies changes nothing — that is the point of keying on the user. For logged-out readers you have no user id; decide explicitly: count by a signed device cookie plus IP, accept that it is beatable, and cap the loss by making the free allowance small. Reset is implicit (a new month is a new key), so there is no monthly reset job, and no midnight thundering herd. Write path: `INSERT ... ON CONFLICT (user_id, yyyyymm) DO UPDATE SET n = counter.n + 1 RETURNING n` — one atomic statement, no read-then-write race.
4. $50\{, \}000\{, \}000/86\{, \}400 = 578.7$ QPS average; $\times 3 = 1\{, \}736$ QPS peak. Both: the cache serves it because a redirect is the same tiny answer for everyone and must be fast (target under 20 ms); the database is the source of truth and fills the cache on a miss. A pure cache loses links on restart; a pure database wastes a query on data that never changes after creation. Cache hit rate will be very high because link popularity is extremely skewed.
5. BLOB in the database: one system, transactional with its row, easy backup — but images are 2 MB each, so the database grows 1000x faster than its rows, backups get slow, and every read of the image occupies a database connection. Object storage: cheap, effectively unlimited, and served by the CDN without touching your servers — but you now have two systems that can disagree (a row pointing at a missing object). **Decision: object storage**, with an orphan-sweeper job and writes ordered upload-then-insert so a failure leaves an unreferenced object (harmless, cleaned up) rather than a broken link (visible to users).

Tier 3 — the seven steps at scale

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 17 [Google · pattern]

Design a feature-flag delivery service. Engineers turn features on and off for a percentage of users without deploying code. Every backend service in the company must see a flag change quickly. Run all seven steps.

SOLUTION

Step 1 — Clarify.

Question	Answer, and what it decided
How many application instances read flags?	About 40,000 worldwide across 2,000 services. This is the fleet size that sets the fan-out.
How often is a flag evaluated ?	On nearly every request. 60 M users, 25 page loads each, 8 flags per load. That is 12 billion evaluations a day — which immediately means evaluation cannot be a network call.
How fast must a change reach every instance?	Under 30 seconds is fine; under 5 seconds is nice. Not milliseconds.
What is the worst failure?	The flag service being down must never take down a product service. That outranks freshness and is the design's main constraint.
Do flags depend on user attributes?	Yes — country, app version, and a percentage rollout by user id.

How many flags?

About 5,000 live. At 400 B each the whole rule set is 2 MB.

Step 2 — Estimate.

THE TWO NUMBERS THAT DECIDE THE ARCHITECTURE

Evaluations per day.

$$60\{, \}000\{, \}000 \text{ users} \times 25 \text{ loads} = 1\{, \}500\{, \}000\{, \}000 \text{ page loads/day}$$
$$1\{, \}500\{, \}000\{, \}000 \times 8 \text{ flags} = 12\{, \}000\{, \}000\{, \}000 \text{ evaluations/day}$$
$$12\{, \}000\{, \}000\{, \}000 \times \frac{000}{86}\{, \}400 = 138\{, \}889 \text{ evaluations per second}$$

If each evaluation were an RPC at 1 ms, that is 139,000 QPS of pure overhead and 1 ms added to every flag check. **Therefore evaluation must happen in-process.** The service ships **rules**, not **answers**.

Poll traffic. 40,000 instances, each polling every 30 s:

$$40\{, \}000 \times \frac{000}{30} = 1\{, \}333 \text{ polls per second}$$

That is small — and, importantly, **flat**. There is no peak hour for a poll loop.

Bandwidth if each poll returns the full 2 MB rule set.

$$40\{, \}000 \times 2 \text{ MB} = 80\{, \}000 \text{ MB} = 80 \text{ GB per 30-second cycle}$$
$$80 \frac{\text{GB}}{30} \text{ s} = 2.67 \text{ GB/s} = 21.3 \text{ Gbps, continuously}$$

Bandwidth with conditional requests. The client sends the version it holds; if nothing changed the server replies 304 Not Modified, about 200 B.

$$40\{, \}000 \times 200 \text{ B} = 8\{, \}000\{, \}000 \text{ B} = 8 \text{ MB per cycle}$$
$$8 \frac{\text{MB}}{30} \text{ s} = 267 \text{ KB/s}$$
$$80 \frac{\text{GB}}{8} \text{ MB} = 10\{, \}000 \times \text{less traffic}$$

That 10,000x is the whole design. One HTTP header turns a 21 Gbps problem into a 267 KB/s problem. Say the number.

Step 3 — API.

Two read paths and one write path

TEXT

```
GET /v1/ruleset
  header If-None-Match: "v-8817"
  resp   200 { version: "v-8819", flags: [ ... ] }   (full set, gzipped)
         304 (empty body)                           (nothing changed)
  note   this is the poll. 40,000 callers. Cacheable at the edge.

GET /v1/stream          (Server-Sent Events, optional fast path)
  resp   event: ruleset-changed data: { version: "v-8819" }
  note   a nudge, not the data; the client then calls GET /v1/ruleset

PUT /v1/flags/{flagKey}
  body   { enabled, rollout: { percent: 25, salt: "checkout-v2" },
         rules: [ {attr:"country", op:"in", values:["SG","TH"]} ] }
```

```
resp 200 { version: "v-8819" }
note human-triggered, a few hundred calls a day. Audited.
```

Step 4 — Data model.

Small tables; the version number does the heavy lifting

TEXT

```
flags(flag_key PK, description, owner_team, created_at, archived_at)

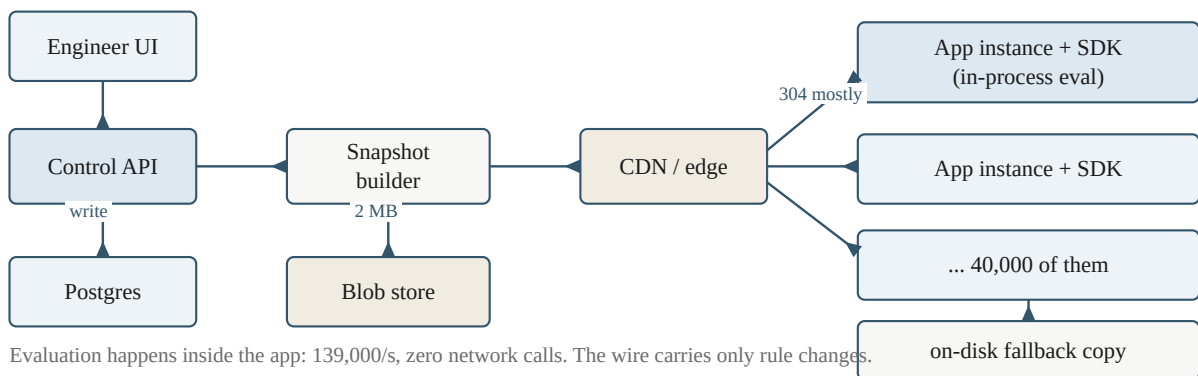
flag_versions(flag_key, version_no, rules_json, enabled, rollout_percent,
              rollout_salt, author, created_at,
              PK (flag_key, version_no))    -- append-only: full history, free
rollback

ruleset_snapshots(version PK, blob_url, byte_size, built_at)
              -- the whole 2 MB set, pre-built

audit(audit_id PK, flag_key, author, before_json, after_json, at)
```

flag_versions is append-only. An edit writes a new row; it never overwrites. Rollback is then “serve version 12 again”, not “hope someone remembers what it was”. For a system whose whole purpose is changing production behaviour, an immutable history is not optional.

Step 5 — Draw it.



Rules flow out; nothing flows in on the hot path. The SDK is the availability story.

Step 6 — Deep dive 1: the SDK must never be able to break a product service.

This is the requirement that outranks everything. Four rules, and the reason for each:

1. **Evaluate locally, always.** No network call on the hot path, so the flag service being slow cannot make checkout slow. This also gives 0 ms of added latency across 139,000 evaluations per second.
2. **Persist the last good snapshot to local disk.** On process start the SDK loads the disk copy **first** and only then tries the network. So a cold start during a flag service outage still gets real rules — possibly hours old, which is vastly better than no rules.
3. **Every call site supplies a default.** `flags.isOn("new-checkout", default=false)`. If there is no snapshot at all — brand-new machine, empty disk, service down — the code still runs and takes the safe branch. The default is in the caller’s code, so it ships with the code that needs it.
4. **Fail open to the old value, never to an exception.** A malformed snapshot is rejected wholesale and the previous one is kept. Partial application of a bad ruleset is worse than being stale.

What this costs: every instance carries 2 MB of memory and there is a window, up to 30 seconds, where instances disagree. So a flag flip is not atomic across the fleet. **Therefore the rule:** flags may never be used for anything requiring fleet-wide agreement at an instant — no “switch database now” flags. They gate **behaviour**, not **coordination**. Naming that boundary is the senior part of this answer.

Step 6 — Deep dive 2: percentage rollout that is stable and independent.

“Turn this on for 25% of users” has three hidden requirements:

1. **Sticky.** The same user must get the same answer on every request, forever, on every machine — otherwise the UI flickers between old and new.
2. **Independent.** A user in the 25% for flag A must not be more likely to be in the 25% for flag B, or your experiments correlate and your measurements are worthless.
3. **Monotonic.** Going 25% -> 50% must **add** users, never swap them out.

Three lines satisfy all three — then prove it

JS

```
const hash32 = (s) => { // FNV-1a: deterministic, no library
  let h = 0x811c9dc5;
  for (let i = 0; i < s.length; i++) {
    h ^= s.charCodeAt(i);
    h = Math.imul(h, 0x01000193) >>> 0; // imul keeps the result 32-bit
  }
  return h >>> 0;
};

const bucketOf = (salt, userId) => hash32(`${salt}:${userId}`) % 10000; // 0..9999
const isOn = (salt, userId, percent) => bucketOf(salt, userId) < percent * 100;

const users = Array.from({ length: 100000 }, (_, i) => `user-${i}`);

// 1. sticky, and the proportion is right
const on25 = users.filter(u => isOn("checkout-v2", u, 25));
console.log("target 25% ->", (100 * on25.length / users.length).toFixed(2) + "%");
const u0 = on25[0];
console.log(`${u0} bucket =`, bucketOf("checkout-v2", u0),
  "-> on?", isOn("checkout-v2", u0, 25), isOn("checkout-v2", u0, 25));

// 2. monotonic: 25% -> 50% must only ADD users
const on50 = new Set(users.filter(u => isOn("checkout-v2", u, 50)));
console.log("target 50% ->", (100 * on50.size / users.length).toFixed(2) + "%");
console.log("users dropped when going 25% -> 50% =",
  on25.filter(u => !on50.has(u)).length);

// 3. independent: a different flag picks a different 25%
const onOther = new Set(users.filter(u => isOn("new-search", u, 25)));
console.log("overlap of two 25% flags =",
  (100 * on25.filter(u => onOther.has(u)).length /
on25.length).toFixed(2) +
  "% (expected ~25%)");

// 4. the trap: userId % 100 is NOT independent
const idNum = (u) => Number(u.split("-")[1]);
const modA = users.filter(u => idNum(u) % 100 < 25);
const modB = new Set(users.filter(u => idNum(u) % 100 < 25));
console.log("overlap with userId % 100 =",
  (100 * modA.filter(u => modB.has(u)).length / modA.length).toFixed(2) +
  "%");
```

Measured output

TEXT


```

target 25%    -> 24.99%
user-0 bucket = 288 -> on? true true
target 50%    -> 50.08%
users dropped when going 25% -> 50% = 0
overlap of two 25% flags = 24.18% (expected ~25%)
overlap with userId % 100 = 100.00%

```

Now read the output line by line — each line proves one requirement.

- **24.99% for a 25% target.** The hash spreads users evenly; you get the share you asked for without keeping any list of who is in.
- **true true for the same user.** Sticky. No randomness, no clock, no machine identity, so all 40,000 instances compute the same answer for that user forever.
- **0 users dropped going 25% -> 50%.** Monotonic. Everyone below bucket 2500 is still below 5000. Raising a rollout can never take the feature away from someone.
- **24.18% overlap between two different flags.** Independent. That is exactly what chance predicts for two unrelated 25% groups, which is what you want: being in one experiment tells you nothing about being in another.
- **100.00% overlap with userId % 100.** This is the trap. Every flag using `id % 100` picks the **same** users. Your “25% experiment” is then always run on the same quarter of your customers — and because ids are usually handed out in order, that quarter is your oldest, least typical users. Every measurement you take is biased, and nothing in the code looks wrong.

The other trap: calling `random()` per request. It is not sticky, so the same user sees the new checkout on one click and the old one on the next. Users report this as “the site is broken”, and they are right.

Step 7 — Trade-offs, failures, 10x.

Choice	Both sides	Decision
Push (streaming) or pull (polling)?	Push gives 1 s propagation but needs 40,000 live connections, reconnect storms, and per-connection state. Pull is stateless, cacheable at the CDN, trivially scalable — but 30 s slower.	Pull as the guaranteed path; push as an optional nudge. Correctness never depends on the stream. If it drops, propagation degrades from 1 s to 30 s and nothing else changes.
Ship rules, or ship per-user answers?	Answers are simpler for the client and let the server change logic centrally — but require a network call per evaluation: 139,000 QPS and latency on every request. Rules need SDKs in every language.	Ship rules. The evaluation count makes the other option arithmetically impossible, not merely expensive.
Strong consistency across regions, or eventual?	Strong means a global consensus write before any instance sees a change: hundreds of ms per write and a global outage if the quorum is unreachable. Eventual means regions can briefly disagree.	Eventual, and say why via CAP. A partition forces a choice between consistency and availability. Here availability wins outright, because an unreachable flag service must not stop 2,000 product services from serving. The cost — a few seconds of disagreement — is invisible to users for behaviour flags.
One global ruleset or one per service?	One global blob is 2 MB everywhere, simple to build and cache, but ships each service 4,900 flags it never reads. Per-service slices are smaller but mean 40,000 different objects, so the CDN hit rate collapses.	One global blob. 2 MB of memory per instance is nothing, and a single cacheable object is what makes the CDN path work. Revisit if the blob passes 50 MB.

Failure modes.

- **Control API down.** Nothing changes; every instance keeps serving its snapshot. Engineers cannot flip flags — an incident, not an outage. This is the correct failure shape.
- **A bad ruleset is published** (bad JSON, or a flag turned on for 100% by mistake). This is the dangerous one: it propagates everywhere in 30 seconds **by design**. Mitigation: schema-validate at write, require a second approver above 10% rollout, and keep a one-click “revert to previous version” that is itself just publishing an older snapshot. Recovery time must be under 60 seconds.
- **CDN down in one region.** SDKs fall back to the origin. Poll traffic at origin rises from near zero to 1,333 QPS. Size the origin for full fleet load; at 1,333 QPS of mostly-304 responses that is a handful of machines. Cheap insurance.
- **Clock skew on an instance.** Time-based rules (“on after 09:00”) misfire. Evaluate time rules against a timestamp stamped into the snapshot by the builder, not the instance’s own clock.

At 10x (400,000 instances). Poll traffic becomes $400 \times \{, \}000/30 = 13 \{, \}333$ QPS — still nothing, and the CDN absorbs it. The thing that actually breaks first is the **snapshot builder**: one process rebuilding a growing blob on every flag edit, with edits arriving faster. I would make the blob a base snapshot plus signed deltas, so the SDK fetches only the changes since its version and the builder’s work becomes proportional to the edit, not to the catalogue. Second concern is the blast radius of a bad publish: at 400,000 instances I would add staged rollout of the **ruleset itself** — 1% of instances, then 10%, then all — so a bad publish is caught by health metrics before it reaches the fleet.

Example 18 [Amazon · pattern]

Run **step 6 only**. A write-heavy service stores every row keyed by `user_id`, sharded by `hash(user_id) % 64`. One customer is an enterprise account generating 60% of all writes. Describe the problem precisely and give three fixes with a decision.

SOLUTION

The problem, precisely. Hashing distributes **keys** evenly, not **traffic**. There is one key here with enormous traffic, and a hash puts one key on exactly one shard. So one of the 64 shards takes 60% of all writes while the other 63 share the remaining 40% — about 0.63% each. Adding shards does not help: 128 shards still puts that key on one of them. The shard saturates, its replication lag grows, and reads from its replicas go stale. This is a **hot partition**, and it is the classic failure of hash sharding.

Approach 1 — Give the big tenant its own shard (operationally simplest)

Route by a lookup table: enterprise customers get dedicated shards, everyone else hashes into the shared pool. Isolates the noisy tenant completely and lets you size its hardware separately. **Cost:** a routing table you must keep consistent everywhere, and it only works when you can **name** the big tenants in advance. A tenant that grows suddenly still melts a shared shard before you notice.

Approach 2 — Add a sub-key: `hash(user_id + bucket)` (best throughput)

Spread one tenant across k sub-partitions: the write key becomes `user_id : (writeSeq % 16)`, so that tenant’s writes hit 16 shards instead of 1. **Cost:** any read that wants “all rows for this user” must now query all 16 buckets and merge. You have traded write skew for read fan-out. Acceptable only if reads are already scoped (by time range, say) or rare.

Approach 3 — Buffer writes through a log and batch them (best when writes are appends)

Send writes to a partitioned log keyed by user, then have a consumer batch 1,000 rows into one database write. The database sees 1/1000th of the operations. **Cost:** writes become visible after a delay (tens of ms to seconds), and you must handle consumer lag and replay. Unacceptable for read-your-own-write flows.

Decision. Combine 1 and 3: **dedicated shards for named large tenants** (it is the change with the least blast radius and no read-path cost), **plus batching through a log** for the append-only event tables, which is where the volume actually is. I would not take option 2 here, because the product’s main query is “show me this user’s recent rows”, and turning every such read into 16 queries would make the common case worse in order to fix an uncommon one.

What I would also add, regardless: per-tenant rate limiting at the gateway, and a per-shard write-rate alert. The real failure was not the hash — it was that nobody was watching a single shard’s share of traffic until it was 60%.

Example 19 [Microsoft · pattern]

Run **step 7 only**: your service runs in Singapore and Frankfurt with a single primary database in Singapore. The network between regions is cut for 12 minutes. Walk the CAP decision explicitly for two different features: (a) a “like” button, (b) a bank transfer.

SOLUTION

The setup. During a partition, Frankfurt cannot reach the Singapore primary. You must choose: refuse writes in Frankfurt (consistency, unavailable) or accept writes locally and reconcile later (available, inconsistent). CAP says you may not keep both. There is no clever third option — but the choice can be made **per feature**, and that is the answer the interviewer wants.

(a) The like button — choose AP. Accept likes in Frankfurt into a local store, and merge when the link returns.

- **Why it is safe:** a like count is a commutative counter. Adding 5 here and 3 there gives 8 either way; the merge is addition and cannot conflict. Nobody is harmed by a count that is 12 minutes behind.
- **The one real edge:** a user may like, see the count not move (because they are reading a different replica), and like again. Fix with a per-user unique key (`post_id`, `user_id`) so a like is idempotent — then even duplicate submissions merge to one.
- **Decision: available. Refusing likes for 12 minutes is a worse product with no compensating benefit.**

(b) The bank transfer — choose CP. Refuse the write in Frankfurt. Return “temporarily unavailable, please retry”.

- **Why:** two partitions both approving a withdrawal from the same balance creates money that does not exist. Reconciliation cannot fix it, because the money has already left. The damage is unbounded and irreversible; a 12-minute outage is bounded and reversible.
- **Do not simply error, though.** Accept the request into a durable queue, return 202 Accepted with a reference number, tell the user “we will confirm within a few minutes”, and process it when the primary is reachable. The user keeps a receipt, the money moves exactly once, and the **perceived** availability is much better than a red error. This is the senior move: you did not break CAP, you moved the wait to a place where waiting is acceptable.
- **Decision: consistent. Never let money diverge.**

The sentence that ties it together. “CAP is not a property of the system; it is a choice you make per operation. In one product I will be AP for likes and CP for transfers, in the same hour, in the same partition.”

And the number that matters more than CAP. A partition lasting 12 minutes is rare; replica lag of 200 ms happens constantly. Most real “consistency bugs” are not CAP at all — they are a user writing to the primary and then reading a replica that has not caught up. Fix that with read-your-writes: pin a user to the primary for a few seconds after their write, or route by a token carrying the write’s position in the log.

PRACTICE · TIER 3 target: 45 min

1. Run all seven steps for **a distributed rate limiter** used by an API gateway across 40 machines: “100 requests per minute per API key”. State exactly what happens when the shared counter store is unreachable.
2. You must choose between a single global database with 200 ms cross-region latency and per-region databases with eventual convergence, for **a user profile service**. Decide, and give the one feature that would flip your decision.
3. Run step 2 and step 6 for **a service that fans out a post to followers**: 20 million posts a day, average 300 followers. Then handle the account with 50 million followers.
4. Design the **failure-mode section only** for a payment gateway: list six components and, for each, what happens when it fails and what the user sees.
5. A senior engineer says “just add a cache”. Give three specific questions you would ask before agreeing, and say what a wrong answer to each would cost.

ANSWER KEY

1. Clarify: per key or per key per endpoint; is a small overshoot acceptable; hard or soft limit. Estimate: if the gateway handles 50,000 QPS, a shared counter store sees 50,000 increments/s — within one Redis node (100 K ops/s) but with no headroom, so shard by key. API: `internal allow(key) -> {allowed, remaining, resetAt}; return 429 with Retry-After` and `X-RateLimit-Remaining`. Data: one counter per (key, minuteBucket) with a TTL of two minutes — expiry is automatic, no cleanup job. Deep dive: a fixed one-minute window lets a caller send 100 at 10:00:59 and 100 at 10:01:00 — 200 in one second. Use a sliding window counter (weight the previous bucket by the fraction of the window still covered) or a token bucket. Failure: if the store is unreachable, **fail open** — allow the request and log it. A rate limiter that takes down the API it protects has inverted its own purpose; the cost of briefly allowing extra traffic is far lower than the cost of a total outage. For a limiter guarding something expensive (SMS sending, money), fail **closed** instead, and say so.

2. Per-region databases. A profile is read constantly and written rarely, and almost every read is by the owner or by people near them, so a local read at 5 ms beats a global read at 200 ms on every page load. Conflicts are rare (one person editing their own profile) and last-writer-wins per field is acceptable. **The feature that flips it:** a globally unique username or handle. Uniqueness cannot be decided locally — two regions can both accept “riya” — so that one field needs a single global authority (or a consensus store) even if everything else stays regional. Note the shape of the answer: split the data by what it needs, rather than forcing one answer on the whole service.

3. Fan-out writes: $20\{, \}000\{, \}000 \times 300 = 6\{, \}000\{, \}000\{, \}000$ feed rows/day = $6\{, \}000\{, \}000\{, \}000/86\{, \}400 = 69\{, \}444$ writes/s sustained. That is large but survivable with a partitioned log and batched inserts. The celebrity breaks it: one post to 50 million followers is 50 million writes; spread over 60 seconds that is $50\{, \}000\{, \}000/60 = 833\{, \}333$ writes/s from a **single** post — twelve times the entire normal load. Fix: hybrid. Fan out on write for normal accounts; for accounts above a threshold (say 100,000 followers) do not fan out at all — store the post once and merge it into the feed at read time. A reader follows few celebrities, so the read-time merge is a handful of extra lookups, and it is bounded. State the threshold and say it should be tunable, because the right value is an operational fact, not a design constant.

4. Card network timeout: user sees “processing”, order held in **PENDING**, reconciled by a job that queries the network by reference; never charge twice, never assume failure. Our database primary down: writes refused, **202 Accepted** into a durable queue, user gets a reference number. Fraud service down: decide in advance — for low-value transactions approve without it (fail open, cap exposure by amount); for high-value, decline (fail closed). Queue backed up: settlement is late, not lost; alert on queue age, not queue length. Webhook endpoint of the merchant down: retry with exponential backoff for 24 hours, then dead-letter and alert the merchant. Entire region down: read-only mode in the secondary showing existing transactions, new payments queued; state the recovery point and recovery time objectives explicitly.

5. (i) “What is the hit rate going to be, and how do you know?” — a cache in front of a uniform-access dataset does nothing; you added a hop and a bug source for zero benefit. **(ii) “What happens when the cache is empty or restarts?”** — if the database cannot survive 100% of traffic even briefly, the cache is not an optimisation, it is a single point of failure with no fallback; you need request coalescing and staged warm-up. **(iii) “How does an entry become wrong, and who removes it?”** — unanswered, you will ship stale data and never know; the cost is a bug class that is invisible in testing and appears only under concurrency.

Interview drill — what the interviewer pushes on

These are the pushes that come in almost every design round, and the answer that scores. Read the answer out loud once. The wording matters as much as the content.

The push	The answer that scores
“How would you scale this?”	“Which part? Reads scale by adding replicas and a cache. Writes scale by sharding on X. Right now the binding constraint is (the one you computed), so that is where I would spend first.”
“What if the cache goes down?”	“All traffic goes to the database — that is (number) QPS. One node handles roughly (number), so I need N nodes, or a degraded mode. I would add request coalescing so 10,000 misses for the same key become one database read.”
“Is this consistent?”	“Within one shard, yes — it is a single-row transaction. Across shards, no, and I chose that: the alternative is a distributed transaction, which costs latency on every write to remove a rare anomaly.”
“Why did you choose SQL here?”	“Because the access pattern is relational and I need a multi-row transaction for X. The data is (size), which one node handles for years. I would move

	to a key-value store if the access pattern became a single-key lookup at (scale)."
"What if I told you traffic is 100x?"	"Then (specific component) breaks first, because (its measured limit). I would change (specific thing). The next thing to break after that is (second component)."
"How do you handle a duplicate request?"	"An idempotency key from the client, stored with a unique index. A repeat returns the stored response instead of doing the work again. I keep keys for 24 hours."
"Where is the single point of failure?"	"(Name it honestly — there is always one.) Today it is X. I would remove it by Y, and until then the blast radius is Z."
"Your estimate looks wrong."	"Let me redo it out loud. (Redo the division.) You are right, it is (new number) — which changes (what it changes)." Never defend a wrong number.
"Why not just use a bigger machine?"	"For this size, that is genuinely the right first answer and it is cheaper than my time. It stops working at (number), because (the limit), and then I shard."
"How do you test this?"	"Unit tests on the rule logic; one integration test per failure mode I listed; and a load test at 3x peak to check the headroom I claimed is real."
"How do you deploy a schema change?"	"Expand, migrate, contract. Add the new column and write both, backfill, switch reads, then drop the old column in a later release. Never in one step."
"We are out of time. Summarise."	"Three sentences: what it does, the one number that shapes it, and the one trade-off I chose. Nothing else."

SHORTCUT

The recovery move. If you get stuck, say: "Let me go back to the numbers." Re-deriving QPS or storage out loud takes 30 seconds, restarts your thinking, and looks like method rather than panic. It is the single most useful habit in the whole round.

TRAP

Three behaviours that lose the room, in order of how often they happen:

1. **Silence.** Ten seconds of thinking is fine; thirty is not. Say what you are weighing.
2. **Arguing with the interviewer.** If they push, they are either testing you or they know something. Both cases mean: consider it out loud.
3. **Name-dropping.** "I would use Kafka and Cassandra and Kubernetes." Naming tools without naming the property you need from them reads as a memorised list. Say the property first: "I need an ordered, replayable log per key — Kafka gives me that."

ONE-PAGE REVISION CARD

The seven steps. Clarify -> Estimate -> API -> Data model -> Draw -> Deep dive -> Trade-offs. Never skip; shrink instead.

The budget (45 min). 5 / 5 / 5 / 5 / 10 / 10 / 5.

The six clarifying questions. How many users. Top three actions. Read:write ratio. Freshness tolerance. What must never break. One region or many. Plus: “auth, payments and admin are out of scope.”

The four numbers, in order.

requests/day	DAU $\times$ actions per user
average QPS	requests/day / 86{, }400 (round to 100,000)
peak QPS	average $\times$ 3
storage/year	rows/day $\times$ bytes/row $\times$ 365

The API rules. Version the path. Cursor pagination, never page numbers. Idempotency key on every write that spends money or sends a message. Return an id from a create.

The data model rules. Name the partition key and justify it by the read pattern. Every index must name the query it serves. Enforce uniqueness with a database constraint, not with application code.

Pick the deep dive from this list. Hot key. Fan-out. Uniqueness race. Retry after timeout. Big list scan.

The trade-off sentence. “A gives me X, B gives me Y. This product needs X because (user reason), so I choose A. If (condition) changed, I would switch.”

Failure walk. Cache dies. Queue backs up. One shard dies. One region dies. A bad config is published. For each: what breaks, what the user sees, what alerts.

Things that lose marks. “It depends” with no choice. Designing auth unasked. No numbers. Unlabelled boxes. Arguing. Tool names with no reason attached.

Things that gain marks. A stated out-of-scope list. A division done out loud. A correct “this is small, one server is enough”. A named single point of failure. A decision with the condition that would reverse it.

The recovery line. “Let me go back to the numbers.”

Back-of-the-Envelope Estimation

Thirty seconds of arithmetic decides the next thirty minutes of design.

The idea in one page

In step 2 of the seven steps you turn a sentence into numbers. That is all estimation is. You are not trying to be exact. You are trying to land in the right **order of magnitude** so that the rest of the design is honest.

Here is why it matters. Two candidates design the same photo app.

- Candidate A says “we will need a lot of storage, so let us use a distributed file system”. Twenty minutes later they are drawing replication protocols.
- Candidate B spends 40 seconds and says “3.2 million uploads a day at 600 KB stored is 1.9 TB a day, so 700 TB a year. That is too big for one disk but small enough that object storage plus a CDN is the whole answer. No custom file system.”

Candidate B has **earned the right** to skip a hard problem. That is what the number bought. Candidate A has to argue.

THE ONE RULE THAT MAKES ESTIMATION EASY

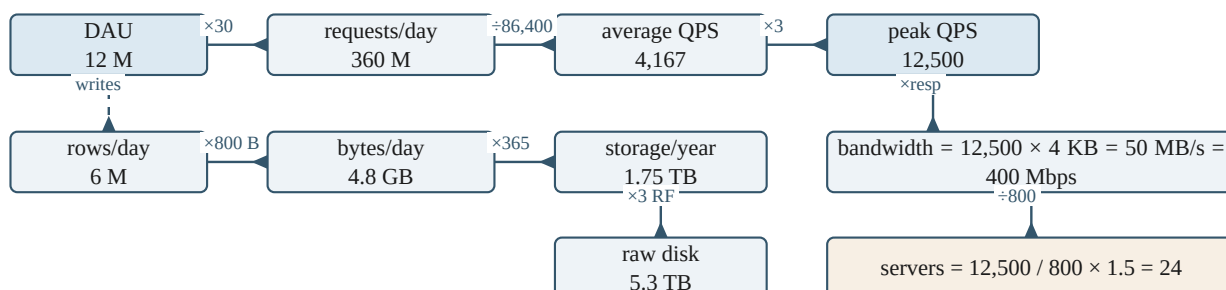
Keep one significant figure, and say out loud that you are rounding.

4,166.67 becomes “about four thousand”. 1,752 GB becomes “under 2 TB”. You are choosing between “one server” and “one hundred servers”. Being 20% off never changes that choice. Being 100x off always does.

An estimate is **good** if it is within about 2x of the truth. It is **acceptable** at 3x. It is **wrong** at 10x, because at 10x you pick the wrong architecture.

The five quantities, always in this order

#	Quantity	Why it is next
1	Peak QPS	decides how many servers, and whether you need a cache at all
2	Storage per year	decides one database or many shards
3	Peak bandwidth	decides CDN yes or no, and how big the network bill is
4	Memory for the hot set	decides the cache size, which decides the cache cost
5	Machine count and cost	the sentence that ends the estimate



The estimation pipeline. One arrow, one multiplication. Never skip an arrow.

TRAP

The most common estimation mistake is not arithmetic. It is **forgetting the multiplier that lives behind the API**. A user posts once. The system writes 300 rows, because 300 followers each get an inbox entry. The write API looks tiny; the write load is 300x bigger. Always ask: “one user action equals how many stored rows?”

The number cards

You cannot derive these in the room. Memorise them. There are five cards and each one fits on a postcard.

Card 1 — time

SECONDS, AND THE DIVIDE-BY-86,400 TRICK

Period	Seconds	Use it for
1 day	$86,400 \approx 10^5$	turning a daily count into QPS
1 month (30 d)	$2,592,000 \approx 2.6 \times 10^6$	monthly bills
1 year (365 d)	$31,536,000 \approx 3.15 \times 10^7$	yearly storage

The single most useful fact in this book:

$$1 \text{ million per day} = \frac{1,000,000}{86,400} = 11.574 \approx \mathbf{11.6} \text{ per second}$$

So: take the daily count **in millions** and multiply by 11.6.

SHORTCUT

Worked, three times, so the reflex sticks.

- 5 million/day $\rightarrow 5 \times 11.6 = 58$ per second. Check: $5,000,000 / 86,400 = 57.87$. ✓
- 360 million/day $\rightarrow 360 \times 11.6 = 4,176$ per second. Check: $360,000,000 / 86,400 = 4,166.7$. ✓ (0.2% high)
- 30 billion/day = 30,000 million $\rightarrow 30,000 \times 11.6 = 348,000$ per second. Check: $30 \times 10^9 / 86,400 = 347,222$. ✓

The trick is never more than 0.3% off, and you can do it in your head.

Card 2 — bytes

POWERS OF TWO, AND THE BYTE LADDER

Power	Exact	Call it	Mental hook
2^{10}	1,024	1 thousand	1 KB
2^{20}	1,048,576	1 million	1 MB
2^{30}	1,073,741,824	1 billion	1 GB
2^{32}	4,294,967,296	4.3 billion	the limit of a 32-bit id
2^{40}	1.10×10^{12}	1 trillion	1 TB
2^{50}	1.13×10^{15}	1 quadrillion	1 PB
2^{63}	9.22×10^{18}	9.2 quintillion	the limit of a signed 64-bit id

The ladder that does most of the work:

$$\begin{aligned} 1 \text{ KB} \times 1 \text{ million} &= 1 \text{ GB} & 1 \text{ KB} \times 1 \text{ billion} &= 1 \text{ TB} \\ 1 \text{ MB} \times 1 \text{ million} &= 1 \text{ TB} & 1 \text{ MB} \times 1 \text{ billion} &= 1 \text{ PB} \end{aligned}$$

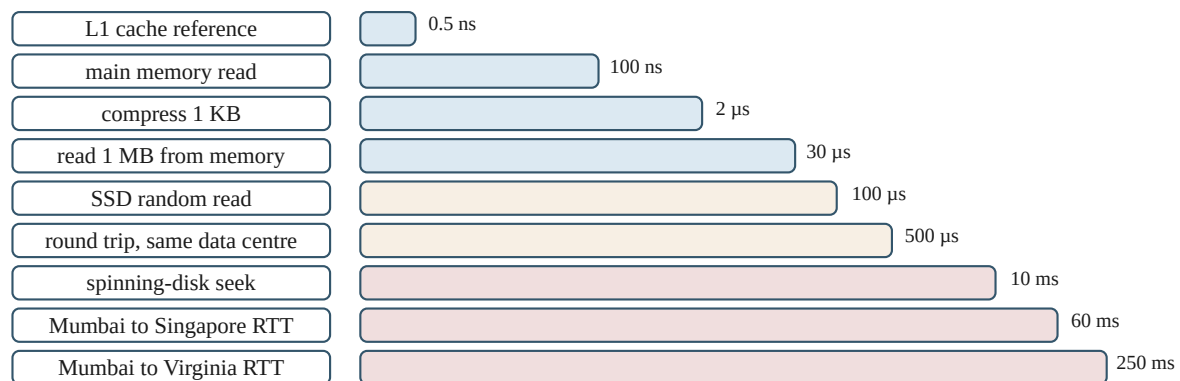
In interviews use **decimal** units: $1 \text{ GB} = 10^9$ bytes. The 7% gap against 2^{30} is inside your rounding error, and the arithmetic is ten times faster. Say “I am using decimal GB” once and move on.

TRAP

1 GB and 1 Gb are different by a factor of 8. Storage and disk are quoted in **bytes** (capital B). Network links are quoted in **bits** (small b). A “1 Gbps” network card moves $1,000,000,000 / 8 = 125 \text{ MB}$ per second, not 1,000 MB per second. Multiply by 8 when going from MB/s to Mbps; divide by 8 going the other way.

Card 3 — latency

These are the numbers that tell you whether a design can possibly meet its budget. They are approximate and they change slowly with hardware, but the **ratios** between them have been stable for twenty years, and the ratios are what you reason with.



The latency ladder. Each bar is one step of ten on a log scale — the bars are not to linear scale, no picture could be.

THE SAME LADDER, SCALED TO HUMAN TIME (1 NANOSECOND = 1 SECOND)

This is the version you should actually remember, because it makes the gaps feel real.

Operation	Real time	If 1 ns were 1 second
read from L1 cache	0.5 ns	half a second
read from main memory	100 ns	under 2 minutes
read 1 MB from memory	30 μs	8 hours
one random read from an SSD	100 μs	just over a day
one network round trip inside a data centre	500 μs	6 days
one seek on a spinning disk	10 ms	4 months
Mumbai ↔ Singapore round trip	60 ms	2 years
Mumbai ↔ Virginia round trip	250 ms	8 years

SHORTCUT

Three sentences you can derive from that table, and they win arguments.

1. **Memory is about 5,000x faster than a same-data-centre network hop** ($500 \mu\text{s} / 100 \text{ ns} = 5,000$). So one in-process cache lookup is free compared to one Redis call. Do not put a 40-byte value in Redis if you can hold it locally.
2. **A cross-continent round trip costs 250 ms, and you cannot fix it with a faster server.** Light in glass does about 200,000 km per second. Mumbai to Virginia and back is roughly 25,000 km of cable, so $25,000 / 200,000 = 0.125 \text{ s}$ of pure physics, before any equipment. The only fix is a copy of the data near the user.
3. **Chatty beats nothing, batched beats chatty.** 100 sequential same-DC calls cost $100 \times 0.5 \text{ ms} = 50 \text{ ms}$. One batched call costs 0.5 ms. Always say this when you draw a loop that calls a service.

Card 4 — what one machine does

THROUGHPUT OF ONE ORDINARY BOX (SAFE INTERVIEW FIGURES)

Component	Per second	What limits it
Node.js API server, light work	1,000–2,000 req	one CPU core per process
Node.js API server, per core, 2 ms of CPU	500 req	1/0.002
Nginx / load balancer	50,000+ req	almost free, it just copies bytes
MySQL / Postgres, simple indexed reads	10,000–20,000	buffer pool hit rate
MySQL / Postgres, writes with fsync	3,000–5,000	disk flush, replication
Redis, simple GET/SET	80,000–100,000	single-threaded command loop
Kafka, one partition	10 MB/s+	sequential disk write, very fast
One machine, open WebSocket connections	$\approx 100,000$ held	memory per socket
1 Gbps network card	125 MB/s	the link
10 Gbps network card	1,250 MB/s	the link

Use the **low** end of each range. If your design works at 3,000 database writes per second, it also works at 5,000. The reverse is not true.

Nobody will check these to the decimal. What is checked is whether you **have** a number at all, and whether you used it. “One Postgres box does about 5,000 writes a second; I need 12,000, so one box is not enough and I shard into 4” is a complete, defensible piece of reasoning. “The database might struggle” is not.

Card 5 — how big is one of those?

SIZES TO QUOTE WITHOUT THINKING

Thing	Size	Note
32-bit integer	4 B	id space only 4.29×10^9
64-bit integer / timestamp	8 B	use this for ids and times
UUID, raw bytes	16 B	as text with dashes it is 36 B
a short text post	300 B	140 characters plus metadata
one chat message row	400–600 B	body + sender + room + time + index
one feed/inbox row (ids only)	50–80 B	postId, authorId, ts, score

a JSON API response	2–10 KB	this is what you bill bandwidth on
a compressed phone photo	300 KB – 1 MB	before thumbnails
a photo thumbnail set	50–150 KB	3 sizes
1 minute of 1080p video at 2.5 Mbps	18.75 MB	$2.5 \times 60/8$
1 hour of 1080p video at 2.5 Mbps	1.125 GB	18.75×60
one log line	200–500 B	logs are usually bigger than your data

Card 6 — the peak factor

Traffic is never flat. You size for the peak, not the average.

WHICH MULTIPLIER TO USE, AND THE SENTENCE THAT DEFENDS IT

Product shape	Peak / average	Why
global, many time zones	2x	one region's night is another's morning
one country, consumer app	3x	everybody is awake in the same evening
one country, work app	4–5x	nothing happens at night or on Sunday
event-driven (sale, match, results day)	10–50x	everybody arrives in one minute

The defence, said in one breath: “A perfectly flat day puts $100\%/24 = 4.2\%$ of traffic in each hour. A real Indian consumer app puts 12–15% of its day into the 8–10 p.m. hour. $15/4.2 = 3.6$, so 3x is a conservative peak factor.” Now the number is yours, not a guess.

The five-step estimate, as a template

Every estimate in this book follows these five lines. Write them on the board as five lines and fill them in. It takes 90 seconds.

THE TEMPLATE — COPY IT EXACTLY

1. traffic $\text{DAU} \times \text{actions/user}$ $=$ ____ /day
 $\div 86,400$ $=$ ____ QPS average
 $\times \text{peak factor}$ $=$ ____ QPS peak
2. storage $\text{new rows/day} \times \text{bytes/row}$ $=$ ____ /day
 $\times 365 \times \text{years} \times \text{replication}$ $=$ ____ total
3. bandwidth $\text{peak QPS} \times \text{response bytes}$ $=$ ____ B/s ($\times 8 = \text{bits/s}$)
4. memory $\text{hot items} \times \text{bytes/item}$ $=$ ____ cache size
5. machines $\text{peak QPS} \div \text{per-server QPS} \times 1.5$ $=$ ____ servers

Then one closing sentence: “so this is a **small** / **medium** / **large** system, and the binding constraint is ____.”

That last sentence is worth more than all the arithmetic. It tells the interviewer you know what the numbers were **for**.

The same template as code — run it and change the inputs

JS

```
const SEC_PER_DAY = 86_400;

function estimate({ dau, actionsPerUser, peakFactor = 3,
  bytesPerRow, rowsPerDay, respBytes, qpsPerServer }) {
  const reqPerDay = dau * actionsPerUser;
```

```

const avgQps    = reqPerDay / SEC_PER_DAY;
const peakQps   = avgQps * peakFactor;
const bytesDay  = rowsPerDay * bytesPerRow;
const bytesYear = bytesDay * 365;
const peakBps   = peakQps * respBytes;           // bytes per second
const servers   = Math.ceil((peakQps / qpsPerServer) * 1.5);
return { reqPerDay, avgQps, peakQps, bytesDay, bytesYear, peakBps, servers };
}

const fmt = n =>
  n >= 1e12 ? (n / 1e12).toFixed(2) + " T" :
  n >= 1e9  ? (n / 1e9).toFixed(2)  + " G" :
  n >= 1e6  ? (n / 1e6).toFixed(2)  + " M" :
  n >= 1e3  ? (n / 1e3).toFixed(2)  + " k" : n.toFixed(2) + " ";

const r = estimate({
  dau: 12_000_000, actionsPerUser: 30, peakFactor: 3,
  rowsPerDay: 6_000_000, bytesPerRow: 800,
  respBytes: 4_000, qpsPerServer: 800,
});

console.log("requests/day :", fmt(r.reqPerDay));
console.log("average QPS  :", fmt(r.avgQps));
console.log("peak QPS     :", fmt(r.peakQps));
console.log("storage/day   :", fmt(r.bytesDay) + "B");
console.log("storage/year  :", fmt(r.bytesYear) + "B");
console.log("peak egress  :", fmt(r.peakBps) + "B/s =", fmt(r.peakBps * 8) + "bit/s");
console.log("app servers  :", r.servers);

```

node estimate.js

TEXT

```

requests/day : 360.00 M
average QPS  : 4.17 k
peak QPS     : 12.50 k
storage/day   : 4.80 GB
storage/year  : 1.75 TB
peak egress   : 50.00 MB/s = 400.00 Mbit/s
app servers   : 24

```

You will not run code in the interview. Write this once at home, change the inputs ten times, and the five lines become muscle memory. That is the point of the snippet.

Warm-up

WARM-UP

build the reflex

Example 1 [warm-up]

How many seconds are in a day, a 30-day month, and a 365-day year? Give the rounded version you would actually use.

| SOLUTION

Day. $24 \times 60 \times 60$. Do it in two steps: $24 \times 60 = 1,440$ minutes, then $1,440 \times 60 = 86,400$ seconds. Round to $10^5 = 100,000$.

Month (30 days). $86,400 \times 30 = 2,592,000$. Round to 2.6×10^6 .

Year (365 days). $86,400 \times 365$. Break it up: $86,400 \times 300 = 25,920,000$ and $86,400 \times 65 = 5,616,000$. Add: 31,536,000. Round to 3.15×10^7 , or “about 31.5 million”.

Which rounding is safe? Using 100,000 instead of 86,400 makes your QPS number $86,400 / 100,000 = 0.864$, so 13.6% **low**. That is fine for a first pass, but say it: “I rounded the day to 100,000 seconds, so my real QPS is about 15% higher.”

Answer: $86,400 \cdot 2,592,000 \cdot 31,536,000$

Example 2 [warm-up]

A service has 12 million daily active users. Each does 30 actions a day. Find requests per day, average QPS, and peak QPS for a single-country consumer app.

SOLUTION

Requests per day.

$$12,000,000 \times 30 = 360,000,000 \text{ requests/day}$$

Average QPS. Use the 11.6 trick first, then check by division.

$$360 \text{ million/day} \times 11.6 = 4,176 \text{ QPS}$$

$$\text{Check: } \frac{360,000,000}{86,400} = 4,166.7 \text{ QPS}$$

The trick was 0.2% high. Call it **4,200 QPS**.

Peak QPS. One country, consumer, so 3x.

$$4,166.7 \times 3 = 12,500 \text{ QPS at peak}$$

Answer: 360 M/day · ≈ 4,170 QPS average · 12,500 QPS peak

Example 3 [warm-up]

That service writes 6 million new rows a day. A row is 800 bytes. How much storage for five years, at replication factor 3?

SOLUTION

Per day.

$$6,000,000 \times 800 = 4,800,000,000 \text{ bytes} = 4.8 \text{ GB/day}$$

Per year.

$$4.8 \times 365 = 1,752 \text{ GB} \approx 1.75 \text{ TB/year}$$

Five years.

$$1.75 \times 5 = 8.76 \text{ TB}$$

Times replication factor 3 (three copies so a disk failure loses nothing):

$$8.76 \times 3 = 26.3 \text{ TB of raw disk}$$

The sentence that matters: 26 TB of raw disk is four or five ordinary servers. This is a **small** system. No sharding needed for storage reasons for years. Say that.

Answer: 8.76 TB of data, ≈ 26 TB of raw disk at RF 3 — small

TRAP

Students forget the replication factor and then forget indexes. Two habits fix it: (i) always multiply the final storage by 3, and (ii) add 30% for indexes if the table has two or more secondary indexes. $8.76 \times 1.3 \times 3 = 34.2$ TB. Still small, but now the number is honest.

Example 4 [warm-up]

Peak is 12,500 QPS and the average response is 4 KB. What is the outbound bandwidth, in MB/s and in Mbps?

SOLUTION

Use 4,000 bytes for 4 KB — decimal units, and say so.

$$12,500 \times 4,000 = 50,000,000 \text{ bytes/second} = 50 \text{ MB/s}$$

To bits, multiply by 8:

$$50 \times 8 = 400 \text{ Mbps}$$

Read it. A single 1 Gbps network card does 125 MB/s. You are asking for 50 MB/s, so one card covers it with room to spare. Bandwidth is **not** your problem here. Request count is.

Answer: 50 MB/s = 400 Mbps — comfortable on one 1 Gbps link

Example 5 [warm-up]

You are choosing the type of the id column. The service creates 50 million new rows a day. Is a 32-bit integer enough? Is a 64-bit integer enough?

SOLUTION

32-bit signed vs unsigned. An unsigned 32-bit integer holds $2^{32} = 4,294,967,296 \approx 4.29 \times 10^9$ values.

$$\frac{4,294,967,296}{50,000,000} = 85.9 \text{ days}$$

Under three months. A 32-bit id is **not** enough. You would run out before the next financial year, and the outage would come at 3 a.m. with no warning.

64-bit signed. $2^{63} = 9.22 \times 10^{18}$ values.

$$9.22 \times \frac{10^{18}}{50,000,000} = 1.84 \times 10^{11} \text{ days}$$

$$1.84 \times \frac{10^{11}}{365} = 5.05 \times 10^8 \text{ years}$$

Five hundred million years. Use 64-bit.

Cost of the choice: 4 extra bytes per row. At 50 million rows a day that is $50,000,000 \times 4 = 200$ MB a day, 73 GB a year. Nothing. Take the 64-bit id.

Answer: 32-bit lasts 86 days — no. 64-bit lasts $\approx$ 500 million years — yes.

Example 6 [warm-up]

Your code reads a value from main memory. Your colleague's code reads the same value from a Redis server in the same data centre. How many times faster is yours?

SOLUTION

From the latency card: main memory read is about 100 ns. A round trip inside one data centre is about $500 \mu\text{s} = 500,000$ ns.

$$\frac{500,000}{100} = 5,000$$

Yours is **five thousand times** faster.

What to do with that fact. It does not mean “never use Redis”. Redis holds things that must be **shared** between 24 servers. It means: do not put a value in Redis that every server could just hold itself — a feature flag, a small config table, a currency rate. Hold those in process and refresh them every 30 seconds.

Answer: About 5,000x faster

Example 7 [warm-up]

You want to cache 20 million hot items. Each cached entry is about 300 bytes including the key. How much memory, and does it fit on one Redis node?

SOLUTION

$$20,000,000 \times 300 = 6,000,000,000 \text{ bytes} = 6 \text{ GB}$$

A Redis node with 16 GB of memory holds this easily, and you should **not** fill a cache node past about 70% because Redis needs headroom for its own structures and for fragmentation.

$$16 \times 0.7 = 11.2 \text{ GB usable} > 6 \text{ GB needed} \quad \checkmark$$

One node. Add a replica for failover, which is 2 nodes total, not 2x the data.

Answer: 6 GB — one 16 GB node, plus one replica

Example 8 [warm-up]

Peak is 12,500 QPS. One application server handles 800 QPS. How many servers?

SOLUTION

Raw count.

$$\frac{12,500}{800} = 15.625 \rightarrow 16 \text{ servers}$$

But 16 is wrong to ship. At exactly 16 servers you are at 100% capacity at peak, so: one deploy, one crash, or one slightly bad day and you are over.

Multiply by 1.5 for headroom:

$$15.625 \times 1.5 = 23.4 \rightarrow \mathbf{24} \text{ servers}$$

Sanity check against zones. 24 splits evenly into 3 availability zones as 8+8+8. If one whole zone dies you are left with 16, which is exactly the raw requirement. That is a clean answer, and saying it earns a mark.

Answer: 24 servers — 8 per zone in 3 zones

Example 9 [warm-up]

A candidate says “we get 1 billion requests a day, that’s about 1 million per second.” Find the mistake and give the correct figure.

SOLUTION

They divided by 1,000 somewhere instead of by 86,400 — probably they thought “a billion is a thousand millions, and a day is about a thousand seconds”. A day is 86,400 seconds, not 1,000.

$$\frac{1,000,000,000}{86,400} = 11,574 \text{ per second}$$

Or with the trick: 1,000 million $\times$ 11.6 = 11,600. $\checkmark$

Why this matters so much. 1 million QPS and 11,600 QPS are different **systems**. 11,600 QPS is roughly 15 ordinary servers and one sharded database. 1 million QPS is a custom edge fleet, a global cache tier, and a team of fifty. An 86x error changes every later decision. This is exactly the failure mode the “order of magnitude” rule exists to prevent.

Answer: $\approx$ 11,600 QPS, not 1 million. The candidate was 86x high.

PRACTICE · WARM-UP target: 10 min

1. 3 million DAU, 18 actions each per day. Requests/day, average QPS, peak at 3x?
2. Rows are 1.4 KB, 2.5 million rows a day. Storage for one year, and raw disk at RF 3?
3. Peak 9,000 QPS, average response 2.5 KB. Bandwidth in MB/s and Mbps?
4. Peak 9,000 QPS, one server does 1,200 QPS. How many servers with 1.5x headroom?
5. 50 million hot keys, 120 bytes each. Cache size in GB?
6. Which is slower, and by how much: one SSD random read, or one round trip to a server in the same data centre?

ANSWER KEY

1. $3,000,000 \times 18 = 54,000,000/\text{day}$. $54,000,000 / 86,400 = 625$ QPS. Peak $625 \times 3 = 1,875$ QPS. Tiny — two servers and one database.
2. $2,500,000 \times 1,400 = 3,500,000,000 \text{ B} = 3.5 \text{ GB/day}$. $3.5 \times 365 = 1,277.5 \text{ GB} \approx 1.28 \text{ TB/year}$. At RF 3: $1.28 \times 3 = 3.83 \text{ TB}$ of raw disk.
3. $9,000 \times 2,500 = 22,500,000 \text{ B/s} = 22.5 \text{ MB/s}$. In bits: $22.5 \times 8 = 180 \text{ Mbps}$.
4. $9,000 / 1,200 = 7.5$, so 8 raw. With headroom $7.5 \times 1.5 = 11.25 \rightarrow 12$ servers (4 per zone across 3 zones).
5. $50,000,000 \times 120 = 6,000,000,000 \text{ B} = 6 \text{ GB}$.
6. The network round trip: $500 \mu\text{s}$ against $100 \mu\text{s}$ for the SSD read, so the network is **5x slower**. This surprises people. Reading from a local disk can beat asking another machine that has the answer in memory.

Tier 1 — estimation inside a low-level design

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

An LLD round has no servers and no QPS. But it still has numbers, and they still decide the design. The three questions that come up are:

How much memory does this structure take?	decides whether “just keep it all in a Map” is a real answer or a joke
How slow is this lookup, in seconds of CPU per second of wall clock?	decides whether you need an index map or a scan is fine
How many things are in flight at once?	decides pool sizes, queue sizes, and whether you need a lock at all

Measuring memory instead of guessing it

You can measure exactly how much memory a JavaScript structure takes. Do this once at home and you will never guess wrong again.

mem.js — run with: `node --expose-gc mem.js <which>`

JS

```
const which = process.argv[2];
const N = 1_000_000;

function snap() {
  global.gc(); global.gc(); // settle the heap twice
  const m = process.memoryUsage();
  return m.heapUsed + m.arrayBuffers; // typed arrays live outside the heap
}
```



```

}

const before = snap();
let keep; // keep a reference or GC eats it
if (which === "map") { const m = new Map();
    for (let i = 0; i < N; i++) m.set("user:" + i, i); keep = m; }
if (which === "array") { const a = new Array(N);
    for (let i = 0; i < N; i++) a[i] = i; keep = a; }
if (which === "typed") { const a = new Int32Array(N);
    for (let i = 0; i < N; i++) a[i] = i; keep = a; }
if (which === "obj") { const a = new Array(N);
    for (let i = 0; i < N; i++)
        a[i] = { id: i, floor: i % 5, size: i % 3, free: true };
    keep = a; }
if (which === "objmap") { const m = new Map();
    for (let i = 0; i < N; i++)
        m.set(i, { id: i, floor: i % 5, size: i % 3, free:
true });
    keep = m; }

const after = snap();

console.log(which.padEnd(7),
    ((after - before) / 1e6).toFixed(1).padStart(7), "MB ",
    ((after - before) / N).toFixed(1).padStart(6), "bytes each",
    keep ? "" : "");

```

Measured on Node v24, one million entries each

TEXT

```

$ for w in map array typed obj objmap; do node --expose-gc mem.js $w; done
map          61.4 MB      61.4 bytes each
array        8.0 MB       8.0 bytes each
typed        4.0 MB       4.0 bytes each
obj         64.0 MB      64.0 bytes each
objmap      85.4 MB      85.4 bytes each

```

THE JAVASCRIPT MEMORY CARD — CARRY THESE FIVE NUMBERS

Structure	Bytes per item	Where it goes
Int32Array element	4	exactly 4 — it is a raw buffer
Array of small integers	8	one 64-bit tagged slot each
object with 4 small fields	≈ 64	header + 4 slots, rounded up
Map entry, string key	≈ 61	key string + hash slot + load factor
Map entry, int key → object value	≈ 85	≈ 21 for the slot, ≈ 64 for the object

Rule of thumb for an interview: “a JavaScript object costs about 60–90 bytes, a Map entry adds about 20–30 bytes on top of what it holds, and a typed array costs exactly the element size.” That sentence is enough.

Example 10 [TCS NQT · pattern]

You are designing the in-memory index for a parking lot. Every spot is an object with `id`, `floor`, `size`, `free`. You also keep `Map<spotId, Spot>` so lookup is $O(1)$. How much memory for 200,000 spots? For 20 million?

SOLUTION

Use the measured number for `Map<key, object>`: about 85 bytes per entry, which already includes the object.
200,000 spots.

$$200,000 \times 85 = 17,000,000 \text{ bytes} = 17 \text{ MB}$$

Nothing. Keep everything in memory, no question.

20 million spots.

$$20,000,000 \times 85 = 1,700,000,000 \text{ bytes} = 1.7 \text{ GB}$$

Now it matters. Node's default old-space heap limit is often around 2–4 GB, and 1.7 GB of live data plus garbage-collection headroom is uncomfortable. Two honest options:

Option A — keep objects	Option B — columns of typed arrays
1.7 GB, code stays readable	<code>Int32Array</code> for floor, size; <code>Uint8Array</code> for free
GC must walk 20 M objects on every major collection	GC walks 3 buffers, not 20 M objects
ship it, it is simple	$20,000,000 \times (4 + 4 + 1) = 180 \text{ MB}$

$$\text{Option B} = 180 \text{ MB} \text{ versus } 1.7 \text{ GB} = 9.4 \times \text{smaller}$$

Decision: Option A at 200,000 spots, Option B above about 5 million. Below 5 million ($5,000,000 \times 85 = 425 \text{ MB}$) the readability is worth more than the memory. Above it, the garbage collector pause becomes the user-visible problem, not the memory, and typed arrays remove the pause as well as the bytes.

Answer: 17 MB at 200 K — trivial. 1.7 GB at 20 M — switch to typed arrays (180 MB).

Example 11 [Infosys · pattern]

An LRU cache must fit in 512 MB. Keys are 24-byte strings, values are 180-byte objects, and the linked-list node overhead is about 80 bytes. How many entries fit? Write the sizing function.

SOLUTION

Per entry.

$$24 + 180 + 80 = 284 \text{ bytes}$$

Entries.

$$\frac{512,000,000}{284} = 1,802,816.9 \rightarrow 1,802,816 \text{ entries}$$

Round down — a partial entry does not exist. Call it “about 1.8 million”.

cache.js — sizing and the hit rate the database needs

JS

```
function cacheFit({ budgetBytes, keyBytes, valueBytes, overheadBytes = 80 }) {
  const perEntry = keyBytes + valueBytes + overheadBytes;
  return { perEntry, entries: Math.floor(budgetBytes / perEntry) };
}

function neededHitRate({ peakQps, dbMaxQps }) {
  if (dbMaxQps >= peakQps) return 0; // no cache needed at all
  return 1 - dbMaxQps / peakQps;
}

const c = cacheFit({ budgetBytes: 512e6, keyBytes: 24, valueBytes: 180 });
console.log("per entry :", c.perEntry, "bytes");
console.log("entries   :", c.entries.toLocaleString("en-US"));
```

```

for (const [peak, dbMax] of [[3000, 2000], [30000, 2000], [30000, 20000]]) {
  const h = neededHitRate({ peakQps: peak, dbMaxQps: dbMax });
  console.log(`peak ${peak} qps, db can do ${dbMax} -> cache must hit
  ${(h*100).toFixed(1)}%`);
}

const dbQps = (peak, hit) => Math.round(peak * (1 - hit));
console.log("peak 30,000 qps at 95% hit -> db sees", dbQps(30000, 0.95), "qps");
console.log("peak 30,000 qps at 90% hit -> db sees", dbQps(30000, 0.90), "qps (2x
more!)");

```

node cache.js

TEXT

```

per entry : 284 bytes
entries   : 1,802,816
peak 3000 qps, db can do 2000 -> cache must hit 33.3%
peak 30000 qps, db can do 2000 -> cache must hit 93.3%
peak 30000 qps, db can do 20000 -> cache must hit 33.3%
peak 30,000 qps at 95% hit -> db sees 1500 qps
peak 30,000 qps at 90% hit -> db sees 3000 qps (2x more!)

```

The line worth saying out loud: going from a 95% hit rate to a 90% hit rate does not make the database 5% busier. It makes it **twice** as busy, because the database only ever sees the misses, and misses went from 5% to 10%. Cache hit rate is a number about **misses**, and misses are what you are paying for.

Answer: 1,802,816 entries at 284 bytes each

TRAP

A cache miss rate of 5% sounds excellent until you multiply it out. At 30,000 peak QPS that is 1,500 database reads a second. If your database does 20,000 reads a second you are fine. If it does 2,000, you are at 75% of the whole database on **misses alone**, and the first cache restart takes you to 30,000 and kills it. Always compute the miss traffic, never just quote the hit rate.

Example 12 [Capgemini · pattern]

A library has 40,000 copies. The return operation finds the open loan by scanning a list of all loans. 300 returns happen per second at the busiest minute. Is the scan acceptable? Show the CPU arithmetic.

SOLUTION

Cost of one scan. A tight loop over an array of objects touches memory at roughly 100 ns per element once you include the pointer chase and the comparison.

$$40,000 \times 100 \text{ ns} = 4,000,000 \text{ ns} = 4 \text{ ms per return}$$

Cost per second of wall clock.

$$300 \text{ returns/s} \times 4 \text{ ms} = 1,200 \text{ ms of CPU per second} = 1.2 \text{ seconds}$$

You need 1.2 seconds of CPU for every 1 second that passes. One core cannot do it. The event loop falls behind, the queue grows, and latency climbs without limit. This is the definition of **saturation**.

The fix and its cost. Keep Map<barcode, Loan> for open loans.

$$\text{lookup} = O(1) \approx 100 \text{ ns, so } 300 \times 100 \text{ ns} = 30 \mu\text{s of CPU per second}$$

$$1,200,000 \mu\text{s} \rightarrow 30 \mu\text{s} = 40,000 \times \text{less CPU}$$

Memory cost of the map: at most 40,000 entries at ≈ 85 bytes = 3.4 MB. You traded 3.4 MB for 40,000x less CPU.

The general rule you should state: a linear scan is fine while

$$\text{items} \times \text{rate} \times 100 \text{ ns} < 0.1 \text{ second of CPU per second}$$

which means $\text{items} \times \text{rate} < 1,000,000$. Here $40,000 \times 300 = 12$ million, twelve times over budget. At 40,000 items and 20 returns per second ($= 800,000$) a scan would have been perfectly fine.

Answer: No — 1.2 s of CPU per second. Index it with a Map: 3.4 MB buys a 40,000x saving.

SHORTCUT

The items $\times$ rate $< 1,000,000$ rule. Multiply how many things you scan by how often you scan them. Under a million, scan away and keep the code simple. Over a million, build the index. This one inequality answers “should I add a hash map here?” in every LLD round you will ever sit.

Example 13 [Wipro · pattern]

Your service handles 2,000 requests per second and each request takes 50 ms end to end. How many requests are in flight at any instant? Now the database slows down and requests take 500 ms. What happens to your connection pool of 200?

SOLUTION

This is **Little’s Law**, and it is the only queueing formula you need:

$$L = \lambda \times W$$

where L is the number of items in the system, λ is the arrival rate, and W is the time each item spends inside.

Healthy case.

$$L = 2,000 \text{ req/s} \times 0.050 \text{ s} = 100 \text{ requests in flight}$$

A pool of 200 connections is comfortable — you use half of it.

Slow case. Nothing about arrivals changed. Only W changed.

$$L = 2,000 \times 0.500 = 1,000 \text{ requests in flight}$$

You need 1,000 connections and you have 200. So 800 requests queue **for a connection**, on top of the 500 ms they will then wait for the database. Latency does not go from 50 ms to 500 ms; it goes to 500 ms plus the queueing time, and the queue keeps growing because arrivals never slowed down.

This is why a small slowdown becomes an outage. A 10x latency increase caused a 10x concurrency increase, which blew a fixed-size pool, which turned “slow” into “failing”.

Little's Law and retry amplification — the two numbers that explain most outages

JS

```
const inFlight = (rps, seconds) => rps * seconds;
console.log("2000 rps x 50 ms =", inFlight(2000, 0.050), "in flight");
console.log("2000 rps x 500 ms =", inFlight(2000, 0.500), "in flight");

const amplify = layers => Math.pow(3, layers); // 3 attempts at each layer
console.log("3 tries at each of 3 layers =", amplify(3), "x the original load");
console.log("1,000 rps of real traffic becomes", 1000 * amplify(3), "rps in an incident");
```

node little.js

TEXT

```
2000 rps x 50 ms = 100 in flight
2000 rps x 500 ms = 1000 in flight
3 tries at each of 3 layers = 27 x the original load
1,000 rps of real traffic becomes 27000 rps in an incident
```

The fix, and the decision. Two options:

- **Grow the pool to 1,000.** Cheap to type. But now 1,000 connections hit a database that was already slow, and connection memory on the database side is real ($1,000 \times \text{a few MB}$). You have moved the queue, not removed it.
- **Keep the pool at 200 and fail fast** — a 200 ms timeout on acquiring a connection, then return 503 and shed load.

Decision: keep the pool small and shed load. A bounded pool is a bulkhead. It means a slow database makes 15% of requests fail quickly instead of making 100% of requests hang. Fast failure is recoverable; a growing queue is not.

Answer: 100 in flight healthy, 1,000 when slow. Bound the pool and shed load.

Example 14 [Cognizant · pattern]

You must store 20 million integer sensor readings in a Node process. Compare a plain Array with an Int32Array. Which do you pick and when does the choice stop mattering?

SOLUTION

From the measured card: Array of small integers is 8 bytes per element, Int32Array is exactly 4.

Array: $20,000,000 \times 8 = 160,000,000$ bytes = 160 MB

Int32Array: $20,000,000 \times 4 = 80,000,000$ bytes = 80 MB

Half the memory. But memory is not the real win.

The real win is the garbage collector. An Int32Array is one buffer outside the JavaScript heap. The collector treats it as a single object. A plain Array of 20 million slots is 20 million things the collector must consider on a major collection. That is what turns into a 200 ms pause that your p99 latency notices.

When does the choice stop mattering? Below about 100,000 elements:

$100,000 \times 8 = 800$ KB versus $100,000 \times 4 = 400$ KB

A 400 KB difference is not worth losing push, map, filter and holes.

Decision: plain Array under 100,000 elements, Int32Array (or Float64Array) above about 1 million. Between the two, pick whichever the surrounding code makes cleaner, because neither number will ever appear in a bug report.

Answer: 160 MB vs 80 MB. Typed array above ≈ 1 M elements, mostly for GC, not bytes.

PRACTICE · TIER 1 target: 18 min

1. A Map<int, object> holds 5 million entries, each object has four small fields. Memory, using the measured 85 bytes per entry?
2. A service does 600 requests per second and each takes 120 ms. How many are in flight? Is a pool of 50 enough?
3. You scan a list of 200,000 items on every call, 500 calls per second. Seconds of CPU per second? Does it pass the $\text{items} \times \text{rate} < 1,000,000$ rule?
4. 20 million integers: Array vs Int32Array in MB?
5. An in-memory seat map holds 5,000 shows $\times$ 200 seats, each seat 40 bytes. Total?
6. A cache budget is 2 GB, entries are 24 B key + 400 B value + 80 B overhead. Entries?

ANSWER KEY

1. $5,000,000 \times 85 = 425,000,000$ B = 425 MB. Fits in one Node process, but it is over a third of a 1 GB container — size the container at 2 GB.
2. $L = 600 \times 0.120 = 72$ in flight. A pool of 50 is **not** enough — 22 requests queue at all times. Size it to at least 100, or cut the 120 ms.

3. $200,000 \times 100 \text{ ns} = 20 \text{ ms}$ per call. $500 \times 20 \text{ ms} = 10,000 \text{ ms} = 10 \text{ seconds}$ of CPU per second. You would need 10 cores just for the scan. The rule: $200,000 \times 500 = 100 \text{ million}$, which is 100x over the one-million budget. Index it.
4. $20,000,000 \times 8 = 160 \text{ MB}$ against $20,000,000 \times 4 = 80 \text{ MB}$.
5. $5,000 \times 200 = 1,000,000 \text{ seats}$. $1,000,000 \times 40 = 40,000,000 \text{ B} = 40 \text{ MB}$. Trivially in memory — which is exactly why a cinema booking system can hold the whole seat map in Redis.
6. Per entry $24 + 400 + 80 = 504 \text{ B}$. $2,000,000,000 / 504 = 3,968,253$ entries, so “about 4 million”.

Tier 2 — estimating one real service

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Now the numbers get big enough to change the architecture. Each example below is a full step-2 of the seven steps: the arithmetic, and then the **one sentence** about what it decided.

Example 15 [Shopee · pattern]

A photo-sharing feature. 8 million daily users in one country. Each uploads 0.4 photos a day and views 60 photos a day. An upload is stored at 500 KB plus a 100 KB thumbnail set. A viewed photo sends about 150 KB over the wire (mostly thumbnails). Produce the full step-2.

SOLUTION

Line 1 — traffic.

Uploads:

$$8,000,000 \times 0.4 = 3,200,000 \text{ uploads/day}$$

$$\frac{3,200,000}{86,400} = 37.0 \text{ uploads/s average}$$

Photo uploads cluster hard in the evening, so use 4x:

$$37.0 \times 4 = 148 \text{ uploads/s at peak}$$

Views:

$$8,000,000 \times 60 = 480,000,000 \text{ views/day}$$

$$\frac{480,000,000}{86,400} = 5,555.6 \text{ views/s average}$$

$$5,555.6 \times 3 = 16,667 \text{ views/s at peak}$$

The read:write ratio is $60/0.4 = 150 : 1$. Say that out loud. It immediately justifies every cache and every read replica you are about to draw.

Line 2 — storage.

$$500 \text{ KB} + 100 \text{ KB} = 600 \text{ KB stored per upload}$$

$$3,200,000 \times 600,000 = 1.92 \times 10^{12} \text{ bytes} = 1.92 \text{ TB/day}$$

$$1.92 \times 365 = 700.8 \text{ TB/year}$$

Line 3 — bandwidth (this is the number that decides the architecture).

$$16,667 \text{ views/s} \times 150,000 \text{ bytes} = 2.5 \times 10^9 \text{ bytes/s} = 2,500 \text{ MB/s}$$

$$2,500 \times 8 = 20,000 \text{ Mbps} = 20 \text{ Gbps at peak}$$

Line 4 — memory. The hot set is “photos posted in the last 24 hours”, roughly 3.2 million objects of metadata at $\approx 400 \text{ B}$:

$$3,200,000 \times 400 = 1.28 \text{ GB}$$

Metadata cache: one small Redis node. The **images** are not cached in Redis; they are cached in the CDN.

Line 5 — machines. At 95% CDN offload the origin serves 5% of the bytes:

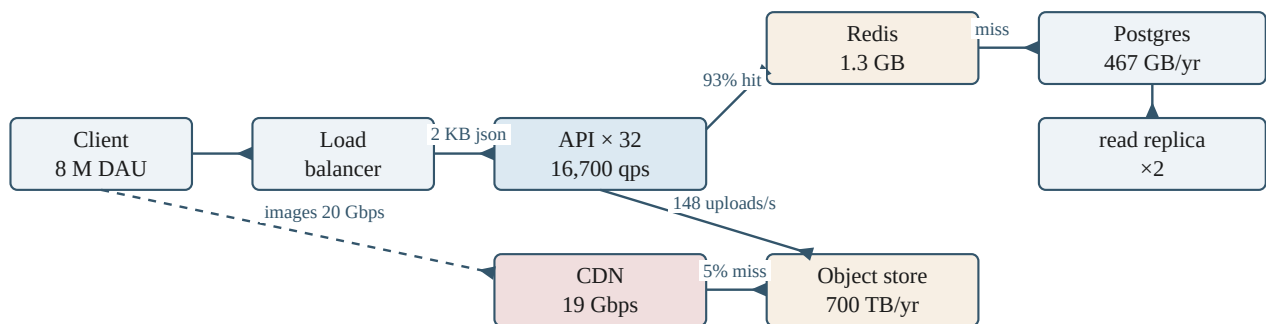
$$20 \text{ Gbps} \times 0.05 = 1 \text{ Gbps at origin}$$

and the API (metadata only, $\approx 2 \text{ KB}$ responses) at 16,667 QPS with 800 QPS per server:

$$\frac{16,667}{800} \times 1.5 = 31.3 \rightarrow 32 \text{ API servers}$$

The closing sentence. “700 TB a year means object storage, not a database — the database stores 400-byte metadata rows, roughly $3,200,000 \times 400 \times 365 = 467 \text{ GB}$ a year, which is one ordinary Postgres box. 20 Gbps of egress means a CDN is mandatory, not optional: without it I need 16 ten-gigabit links at the origin. So the binding constraint here is **bandwidth**, and the whole design is ‘metadata in Postgres, bytes in object storage, delivery by CDN’.”

Answer: 3.2 M uploads/day · 16,700 views/s peak · 1.92 TB/day · 20 Gbps peak · CDN mandatory



The same service with the numbers written on it. This is what the board should look like after step 2 and step 5.

Example 16 [Grab · pattern]

A chat service. 5 million daily users, 40 messages sent per user per day. A stored message row is 500 bytes. At peak, 20% of daily users hold an open connection. One machine holds 80,000 connections. Produce step-2 and say how many connection servers.

SOLUTION

Line 1 — traffic.

$$5,000,000 \times 40 = 200,000,000 \text{ messages/day}$$

$$\frac{200,000,000}{86,400} = 2,314.8 \text{ messages/s average}$$

$$2,314.8 \times 3 = 6,944 \text{ messages/s at peak}$$

But that is only the send side. Every message in a 1:1 chat is **delivered** once and usually read once, and group chats multiply it. Take an average delivery fan-out of 2:

$$6,944 \times 2 = 13,889 \text{ delivery pushes/s at peak}$$

Line 2 — storage.

$$200,000,000 \times 500 = 100,000,000,000 \text{ bytes} = 100 \text{ GB/day}$$

$$100 \times 365 = 36,500 \text{ GB} = 36.5 \text{ TB/year}$$

Five years at RF 3:

$$36.5 \times 5 \times 3 = 547.5 \text{ TB of raw disk}$$

Line 3 — bandwidth. A message on the wire is small, say 300 B including framing:

$$13,889 \times 300 = 4,166,700 \text{ bytes/s} \approx 4.2 \text{ MB/s} = 33 \text{ Mbps}$$

Bandwidth is a non-issue. Do not spend a minute on it — say so and move on.

Line 4 — connections. This is the interesting number for chat.

$$5,000,000 \times 0.20 = 1,000,000 \text{ concurrent connections at peak}$$

$$\frac{1,000,000}{80,000} = 12.5 \rightarrow 13 \text{ connection servers}$$

But a chat connection server holds **state** (which user is on which box), so losing one matters. Size to survive losing one availability zone out of three, which means the remaining two zones must carry everything:

$$\frac{13}{2/3} = 19.5 \rightarrow 20 \text{ servers, spread } 7 / 7 / 6$$

Memory per connection. 1,000,000 connections over 13 boxes is 77,000 each. At roughly 20 KB of kernel and userspace buffers per socket:

$$80,000 \times 20,000 = 1.6 \times 10^9 \text{ bytes} = 1.6 \text{ GB just for sockets}$$

So a connection server needs 8 GB, not 2 GB. That is a real, specific requirement, and it comes straight out of the arithmetic.

The closing sentence. “The binding constraint for chat is not QPS — 6,944 writes a second is four database shards at 5,000 writes each with room to spare. It is the **million open connections**, which is 20 stateful machines and a routing table that says which machine holds which user. That routing table is the thing I would deep-dive.”

Answer: 6,944 msg/s peak · 36.5 TB/yr · 1 M connections · 20 connection servers

TRAP

Do not compute a chat service’s connection count from **daily** users directly. “5 million DAU means 5 million connections” is wrong by 5x and it inflates the design into something absurd. Concurrency is DAU times the fraction online **at the same moment**. For a consumer messaging app, 15–25% at the evening peak is a defensible assumption. State the fraction you are using, and why.

Example 17 [Agoda · pattern]

Driver location pings. 200,000 drivers are online at peak. Each phone sends its position every 4 seconds. A ping is 60 bytes. How many writes per second? How much storage if you keep every raw ping? Decide a retention policy.

SOLUTION

Writes per second.

$$200,000 \text{ drivers} / 4 \text{ seconds} = 50,000 \text{ writes/s}$$

This is a **huge** number and it arrives with no user pressing anything. Compare it with the actual business events: maybe 3,000 ride requests a minute = 50 per second. The telemetry is 1,000x the business traffic.

Storage if you keep everything.

$$200,000 \times 86,400 / 4 = 4,320,000,000 \text{ pings/day}$$

$$4,320,000,000 \times 60 = 259,200,000,000 \text{ bytes} = 259.2 \text{ GB/day}$$

$$259.2 \times 365 = 94,608 \text{ GB} = 94.6 \text{ TB/year}$$

At RF 3: $94.6 \times 3 = 284 \text{ TB}$ a year, for data nobody reads after five minutes.

Bandwidth.

$$50,000 \times 60 = 3,000,000 \text{ bytes/s} = 3 \text{ MB/s} = 24 \text{ Mbps}$$

Tiny. The problem is **write operations**, not bytes.

The decision, with both sides named.

Keep every ping forever	Keep the latest only, plus a downsample
replay any incident perfectly, any dispute answerable	7 days raw, then 1 point per minute
284 TB/year of disk and the backups to match	$259.2 \times 7 = 1.81$ TB raw window
50,000 durable writes/s needs ≈ 10 database shards	current position in Redis: 1 node does 100,000 ops/s

Decision: current position in Redis, raw pings to a 7-day log, then downsample to one point per minute.

Downsampled volume:

$$200,000 \times 1440 \text{ minutes} = 288,000,000 \text{ points/day}$$

$$288,000,000 \times 60 = 17.3 \text{ GB/day} = 6.3 \text{ TB/year}$$

$$94.6 \text{ TB/yr} \rightarrow 6.3 \text{ TB/yr} = 15 \times \text{less}$$

The reason: 99.9% of reads want **one** value — “where is driver 8812 right now?” — and that is a single Redis key. History is a compliance and analytics need, and analytics is happy with one point per minute. Paying 15x for a resolution nobody queries is the wrong trade.

Answer: 50,000 writes/s · 259 GB/day raw · keep current in Redis, 7-day raw, then downsample to 6.3 TB/yr

Example 18 [DBS · pattern]

Catalogue reads. Peak is 30,000 reads per second. One database replica serves 2,000 reads per second. Two choices: add replicas, or add a cache. Compute both and decide.

SOLUTION

Option A — replicas only.

$$\frac{30,000}{2,000} = 15 \text{ replicas}$$

With 1.5x headroom: $15 \times 1.5 = 22.5 \rightarrow 23$ replicas.

Each replica is a full copy of the data. If the catalogue is 800 GB:

$$23 \times 800 \text{ GB} = 18.4 \text{ TB of disk}$$

And every write must be shipped to 23 machines, so replication lag gets worse as you add replicas — the exact opposite of what you wanted.

Option B — cache in front. Required hit rate so the database sees at most 2,000:

$$\text{hit rate} = 1 - \frac{2,000}{30,000} = 1 - 0.0667 = 93.3\%$$

Is 93.3% realistic? Catalogue traffic is extremely skewed — a small number of products get most of the views. Suppose 2 million products are “hot” and a row is 1.5 KB:

$$2,000,000 \times 1,500 = 3,000,000,000 \text{ bytes} = 3 \text{ GB}$$

3 GB fits in one Redis node with a replica. So: 2 machines instead of 23.

The numbers side by side.

	A: 23 replicas	B: Redis + 2 replicas
machines	23 database boxes	2 Redis + 2 database
disk	18.4 TB	3 GB memory + 1.6 TB disk
reads at database	30,000/s	2,000/s
failure if cache dies	n/a	30,000/s at 2 replicas — it dies
staleness	replication lag, ≈ 100 ms	TTL, up to 60 s

Decision: Option B, with two protections.

1. **Request coalescing.** When 5,000 requests miss the same key at once, only one goes to the database and the rest wait for its answer. Without this, one expired hot key sends 5,000 simultaneous queries. With it, it sends 1.
2. **Enough replicas to survive a cold cache in a degraded mode.** Not 23 — that defeats the point — but 4, plus a rule that on a cold start the service serves a reduced catalogue (top 10,000 products) until the cache warms. Say this out loud; “what happens when the cache is empty” is the follow-up you will get.

Why B and not A: the access pattern is heavily skewed, and a cache is the correct tool for skew. 23 replicas pay full price for every product, including the 90% of the catalogue that is read once a week.

Answer: A = 23 replicas, 18.4 TB. B = one 3 GB Redis at 93.3% hit. Choose B + coalescing + 4 replicas.

Example 19 [SCB · pattern]

Turn the photo service of Example 15 into a monthly bill. CDN egress costs \$0.02 per GB, object storage \$0.02 per GB-month, an application server \$60 a month. Is bandwidth or storage the bigger line?

SOLUTION

Egress. Average (not peak) is what you are billed on.

$$480,000,000 \text{ views/day} \times 150,000 \text{ B} = 7.2 \times 10^{13} \text{ bytes/day} = 72,000 \text{ GB/day}$$

$$72,000 \times 30 = 2,160,000 \text{ GB/month}$$

$$2,160,000 \times \$0.02 = \$43,200 \text{ /month}$$

Storage. By the end of month 12 you are holding $1.92 \text{ TB/day} \times 365 = 700.8 \text{ TB}$.

$$700,800 \text{ GB} \times \$0.02 = \$14,016 \text{ /month, and rising every month}$$

Servers. 32 API servers:

$$32 \times \$60 = \$1,920 \text{ /month}$$

Line	Per month	Share
CDN egress	\$43,200	73%
object storage (end of year 1)	\$14,016	24%
application servers	\$1,920	3%
total	\$59,136	

What the bill tells you to work on. Not the servers. Three-quarters of the money is bytes leaving the building. So the highest-value engineering is:

1. **Serve smaller images.** Going from 150 KB to 100 KB per view is a 33% cut: $\$43,200 \times 0.67 = \$28,944$, saving \$14,256 a month. Modern formats routinely do that.
2. **Raise the CDN hit rate.** Origin egress is usually charged more than CDN egress; every point of hit rate moves bytes to the cheaper meter.
3. **Lifecycle old photos to cold storage.** Photos older than 90 days are viewed rarely. Cold tiers cost roughly a quarter as much per GB.

The interview sentence: “Cost follows bytes, not requests. I would spend my first engineering week on image size, because a 33% size reduction is worth more than deleting the entire server fleet.”

Answer: ≈ \$59 K/month; egress is 73% of it. Optimise image size first.

PRACTICE · TIER 2 target: 30 min

1. A messaging app has 4 million DAU sending 25 messages each per day. Rows are 400 B. Give messages/s average and peak (3x), GB/day and TB/year.
2. 150,000 drivers ping every 5 seconds. Writes per second?
3. Peak is 12,000 reads/s and one database does 4,000. What cache hit rate do you need?

4. A video service has 2 million DAU watching 20 minutes a day at 1.5 Mbps. Give the bytes delivered per day, the average and 3x peak egress in Gbps, and the CDN bill at \$0.02/GB.
5. A feed API returns 20 items of 300 B each plus 1 KB of envelope. At 25,000 peak QPS, what is the egress in Gbps?
6. A service is 95% reads. If you cache and get a 90% hit rate, by what factor does database read load drop?

ANSWER KEY

1. $4,000,000 \times 25 = 100,000,000/\text{day}$. $100,000,000 / 86,400 = 1,157/\text{s}$ average; $\times 3 = 3,472/\text{s}$ peak. $100,000,000 \times 400 = 40 \text{ GB/day}$; $40 \times 365 = 14,600 \text{ GB} = 14.6 \text{ TB/year}$.
2. $150,000 / 5 = 30,000 \text{ writes/s}$. Compare with “how many rides start per second” — telemetry dwarfs business traffic, so it belongs in a different store.
3. $1 - 4,000/12,000 = 1 - 0.333 = 66.7\%$. Easy to hit — this is a case where a modest cache removes the need for any replicas at all.
4. Seconds of video: $2,000,000 \times 20 \times 60 = 2.4 \times 10^9 \text{ s/day}$. Bits: $2.4 \times 10^9 \times 1.5 \times 10^6 = 3.6 \times 10^{15} \text{ bits/day}$. Bytes: $3.6 \times 10^{15} / 8 = 4.5 \times 10^{14} = 0.45 \text{ PB/day}$. Average: $3.6 \times 10^{15} / 86,400 = 4.17 \times 10^{10} \text{ bps} = 41.7 \text{ Gbps}$; peak $\times 3 = 125 \text{ Gbps}$. Bill: $450,000 \text{ GB/day} \times \$0.02 = \$9,000/\text{day} \approx \$270,000/\text{month}$.
5. Response = $20 \times 300 + 1,000 = 7,000 \text{ B}$. $25,000 \times 7,000 = 1.75 \times 10^8 \text{ B/s} = 175 \text{ MB/s} \times 8 = 1.4 \text{ Gbps}$.
6. The database sees only misses, so load drops from 100% to 10% of reads — a factor of **10**. Note it is $1/(1 - h)$, not $1/h$: at 99% it would be 100x.

Tier 3 — estimation at large scale

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

At this tier the arithmetic is the same. What changes is that the numbers start **forbidding** designs. A Tier-3 estimate ends with “and that is why the obvious approach is impossible”.

Example 20 [Google · pattern]

Video streaming egress. 30 million daily users watch 45 minutes a day at an average 2.5 Mbps. Compute bytes per day, average and peak egress, and the CDN bill at \$0.02/GB. Then say what the number forbids.

SOLUTION

Step 1 — seconds of video delivered per day.

$$30,000,000 \times 45 \text{ min} \times 60 = 81,000,000,000 \text{ seconds of video/day}$$

That is 81 billion seconds — about 2,570 years of video every day. Worth saying; it makes the scale land.

Step 2 — bits per day.

$$81 \times 10^9 \text{ s} \times 2.5 \times 10^6 \text{ bits/s} = 2.025 \times 10^{17} \text{ bits/day}$$

Step 3 — bytes per day.

$$2.025 \times \frac{10^{17}}{8} = 2.53 \times 10^{16} \text{ bytes/day} = 25.3 \text{ PB/day}$$

Step 4 — average egress rate.

$$2.025 \times \frac{10^{17}}{86,400} = 2.34 \times 10^{12} \text{ bits/s} = 2.34 \text{ Tbps}$$

Step 5 — peak. Evening peak, one region, 3x:

$$2.34 \times 3 = 7.03 \text{ Tbps at peak}$$

Step 6 — the bill.

$$25.3 \text{ PB/day} = 25,312,500 \text{ GB/day}$$

$$25,312,500 \times \$0.02 = \$506,250 \text{ per day}$$

$$\$506,250 \times 365 = \$184,781,250 \text{ per year}$$

What the number forbids.

1. **It forbids a commercial CDN as the only answer.** \$185 million a year for delivery alone is larger than most companies' entire revenue. This is precisely why large video companies build their own caching boxes and place them inside internet providers. Making that point is the whole reason the interviewer asked.
2. **It forbids a single origin region.** 7 Tbps at peak is 700 ten-gigabit links running flat out. No single building takes that. Delivery must be distributed and near users.
3. **It makes bitrate the most valuable engineering lever in the company.** Cutting the average bitrate from 2.5 to 2.0 Mbps is a 20% cut:

$$\$184.8 \text{ M} \times 0.8 = \$147.8 \text{ M}, \text{ a saving of } \$37 \text{ M a year}$$

A better codec is worth more than the entire infrastructure team.

The closing sentence. "Storage here is almost free by comparison — the **catalogue** is maybe a petabyte and it is written once. The constraint is egress, and egress is a physical and commercial problem, not a software one. So my design spends its effort on caching close to the user and on adaptive bitrate, not on the database."

Answer: 25.3 PB/day · 2.34 Tbps average · 7 Tbps peak · ≈ \$185 M/year on a commercial CDN

Example 21 [Amazon · pattern]

Feed fan-out. 400 million daily users. 100 million posts a day. The average author has 300 followers. If every post is copied into every follower's inbox, compute the write load. Then compute what one celebrity with 50 million followers does, and decide the strategy.

SOLUTION

Step 1 — inbox writes per day.

$$100,000,000 \text{ posts} \times 300 \text{ followers} = 3.0 \times 10^{10} \text{ inbox rows/day}$$

Thirty billion rows a day, from only 100 million user actions. The fan-out multiplied the write load by 300.

Step 2 — writes per second.

$$3.0 \times \frac{10^{10}}{86,400} = 347,222 \text{ writes/s average}$$

$$347,222 \times 3 = 1,041,667 \text{ writes/s at peak}$$

Step 3 — can the storage take it? A key-value store node absorbs about 5,000 durable writes a second.

$$\frac{1,041,667}{5,000} = 208 \text{ nodes}$$

With 1.5x headroom: $208 \times 1.5 = 312$ nodes, purely for inbox writes.

Step 4 — storage. An inbox row is ids only, about 60 bytes.

$$3.0 \times 10^{10} \times 60 = 1.8 \times 10^{12} \text{ bytes} = 1.8 \text{ TB/day}$$

Keep 30 days of inbox:

$$1.8 \times 30 = 54 \text{ TB} \times 3 \text{ (RF)} = 162 \text{ TB}$$

Storage is not the problem. Write **rate** is.

Step 5 — the celebrity. One post by an account with 50 million followers:

$$50,000,000 \text{ writes from one button press}$$

At the cluster's peak capacity of 1,041,667 writes/s, and assuming this post could use the entire cluster:

$$\frac{50,000,000}{1,041,667} = 48 \text{ seconds}$$

Forty-eight seconds during which the cluster does **nothing else**. And celebrities do not post alone — ten of them posting in the same minute is 500 million writes, over eight minutes of total cluster time, while 400 million ordinary users get nothing.

The decision: hybrid fan-out.

	push (fan-out on write)	pull (fan-out on read)
when	author has $< 10,000$ followers	author has $\geq 10,000$ followers
read cost	one range scan of your inbox	merge your inbox with k celebrity timelines
write cost	300 rows per post	1 row per post
who it fits	99.9% of accounts	the few thousand huge accounts

The arithmetic that proves the hybrid works. Suppose accounts above 10,000 followers produce 1% of posts but hold 60% of the follower edges. Then push traffic becomes:

$$3.0 \times 10^{10} \times (1 - 0.60) = 1.2 \times 10^{10} \text{ rows/day}$$

$$1.2 \times \frac{10^{10}}{86,400} \times 3 = 416,667 \text{ writes/s at peak}$$

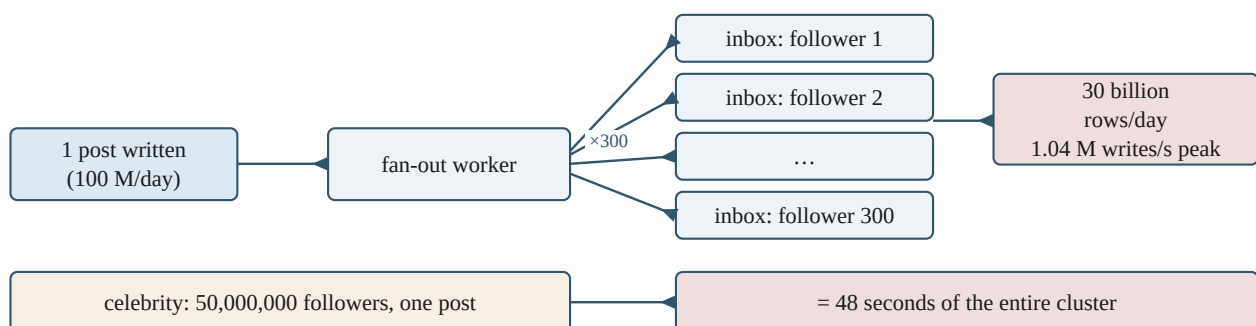
$$\frac{416,667}{5,000} \times 1.5 = 125 \text{ nodes}$$

$$312 \text{ nodes} \rightarrow 125 \text{ nodes} = 2.5 \times \text{cheaper}$$

And the read cost rises by a bounded amount: a user follows at most a handful of huge accounts, so a feed read becomes one inbox scan plus (say) up to 20 timeline reads, all of them cached, all of them in parallel. That is a latency cost of one extra round trip, about 1 ms, against a 2.5x hardware saving and the removal of a 48-second stall.

Decision: hybrid, with the threshold set by measurement, not by taste. Start at 10,000 followers, watch the p99 of feed reads, and move the threshold until reads and writes cost about the same.

Answer: 30 B rows/day, 1.04 M writes/s peak, 312 nodes. Hybrid cuts it to 125 and removes the 48 s stall.



Fan-out amplification. One button press on the left becomes N writes on the right — the number that breaks naive feed designs.

Example 22 [Microsoft · pattern]

How many shards? A dataset grows 36.5 TB a year and you plan for five years. One shard holds 2 TB of usable data. Peak write load is 1,041,667 writes/s. Choose the shard count and justify the number you pick.

SOLUTION

From storage.

$$36.5 \times 5 = 182.5 \text{ TB in five years}$$

$$\frac{182.5}{2} = 91.25 \rightarrow 92 \text{ shards}$$

From write throughput. One shard node does 5,000 durable writes/s:

$$\frac{1,041,667}{5,000} = 208.3 \rightarrow 209 \text{ shards}$$

Take the larger. Throughput needs 209, storage needs 92. Shard count is $\max(92, 209) = 209$. Add 1.5x headroom: $209 \times 1.5 = 313.5$.

Now round to a power of two: 512. Here is why that specific rounding, and it is worth saying:

1. **Splitting is halving.** If shard count is a power of two and you route by $\text{hash}(\text{key}) \bmod 2^k$, then doubling from 2^k to 2^{k+1} moves exactly half of each shard's keys, and each key moves to exactly one new shard. With 313 shards, growing to 314 re-maps almost every key in the cluster.
2. **mod on a power of two is a bit mask.** $h \bmod 512 = h$ and 511. Free.
3. **It leaves room.** 512 shards at 2 TB each is 1,024 TB of capacity — over five times what five years needs — and 512 at 5,000 writes/s is 2.56 million writes/s, about 2.5x peak.

Check it is not absurd. 512 shards is not 512 machines. A machine can host 4–8 shards.

$$\frac{512}{8} = 64 \text{ machines}$$

64 machines to hold 182 TB and take a million writes a second is entirely reasonable.

The alternative, named and rejected. Virtual nodes with consistent hashing let you use any count and add one node at a time. That is genuinely better for smooth growth. It costs a routing table that every client must know and keep fresh, plus the operational work of rebalancing ranges. **Decision: fixed 512 logical shards mapped onto a variable number of physical machines.** You get the cheap mod, you get rebalancing by moving whole logical shards (which is a file copy, not a key-by-key migration), and you can grow machines one at a time even though the shard count never changes. Consistent hashing solves a problem — arbitrary shard counts — that you chose not to have.

Answer: 92 by storage, 209 by throughput; take 209, add headroom, round to 512 logical shards on ≈ 64 machines

Example 23 [Uber · pattern]

Tail latency amplification. A search request fans out to 50 shards and must wait for all of them. Each shard answers within 20 ms 99% of the time. What fraction of requests take longer than 20 ms? What do you do about it?

SOLUTION

The probability. The request is fast only if **every** shard is fast. Assuming the shards are independent:

$$P(\text{all 50 fast}) = 0.99^{50}$$

Compute it step by step. $0.99^{10} = 0.9044$. Then $0.99^{50} = (0.99^{10})^5 = 0.9044^5 = 0.6050$.

$$P(\text{at least one slow}) = 1 - 0.6050 = 0.395 = \mathbf{39.5\%}$$

Read that again. Every single shard is a 99th-percentile-20-ms component, which sounds excellent, and yet **four requests in ten** are slow. The p99 of one shard has become roughly the p60 of the whole request.

tail.js — how fan-out turns a good p99 into a bad one

JS

```
const atLeastOneSlow = (n, pFast) => 1 - Math.pow(pFast, n);

console.log("shards   chance the whole request is slow (each shard p99 = 20 ms)");
for (const n of [1, 5, 10, 20, 50, 100])
  console.log(String(n).padStart(6), " ", (atLeastOneSlow(n, 0.99) *
100).toFixed(1) + "%");
```

```
console.log("\nsame fan-out, but each shard is p99.9 instead:");
for (const n of [10, 50, 100])
  console.log(String(n).padStart(6), " ", (atLeastOneSlow(n, 0.999) *
100).toFixed(1) + "%");
```

node tail.js

TEXT

```
shards    chance the whole request is slow (each shard p99 = 20 ms)
  1        1.0%
  5        4.9%
 10        9.6%
 20       18.2%
 50       39.5%
100       63.4%
```

```
same fan-out, but each shard is p99.9 instead:
 10        1.0%
 50        4.9%
100        9.5%
```

The three fixes, with numbers.

1. **Reduce the fan-out.** 50 shards to 10 takes the slow fraction from 39.5% to 9.6%. Route by a key that lets you touch fewer shards. This is the cheapest fix and it is a **data model** decision, made in step 4, not a latency patch.
2. **Improve each shard's p99 to p99.9.** At 50 shards that takes 39.5% down to 4.9% — an 8x improvement. This means fixing garbage-collection pauses, slow disks, and noisy neighbours. Expensive and slow, but it is the only fix that helps every query shape.
3. **Hedged requests.** Send to the shard; if no answer in 15 ms, send a second copy to a replica and take whichever returns first. Extra load is bounded by the fraction that are slow — under 5% more traffic — and the slow fraction collapses, because both copies must be slow for the request to be slow:

$$0.01 \times 0.01 = 0.0001 \text{ per shard, so } 1 - 0.9999^{50} = 0.5\%$$

Decision: do 1 and 3 now, 2 continuously. Cut the fan-out in the data model because it is free; add hedging because it is a small amount of code and buys an 80x improvement; and treat per-shard p99.9 as an ongoing engineering programme, not an interview answer.

Answer: 39.5% of requests are slow. Cut fan-out, hedge after 15 ms, and drive each shard to p99.9.

SHORTCUT

The fan-out rule you can quote instantly: if each of n components is fast with probability p , the whole request is fast with probability p^n . For n around $1/(1-p)$ you lose about a third of your requests. So “p99 per shard” is only safe up to about 10 shards. Beyond that you need p99.9, or hedging, or a smaller fan-out.

Example 24 [Goldman Sachs · pattern]

The retry storm. A client retries 3 times. It calls an API gateway that retries 3 times. The gateway calls a service that retries 3 times against the database. Normal traffic is 1,000 requests per second. What does the database see during a partial outage, and what is the correct retry policy?

SOLUTION

The amplification. Each layer multiplies.

$$3 \times 3 \times 3 = 3^3 = 27 \times$$

$$1,000 \text{ req/s} \times 27 = 27,000 \text{ req/s at the database}$$

Why this is a doom loop. The database was slow because it was at its limit. The retries make it 27 times more loaded, so it gets slower, so more requests time out, so more retries fire. The system cannot recover even after the original cause is gone, because the retries are now the cause. Engineers call this **metastable failure**: the system has two stable states, “working” and “melted”, and a small push moves it from one to the other with no path back except turning traffic off.

The correct policy, with the arithmetic for each rule.

Retry at one layer only	Pick the layer closest to the user that has the context to retry safely — usually the gateway. $3 \times 1 \times 1 = 3x$ instead of 27x. A 9x reduction from one rule.
Cap total attempts at 3 and budget them	Give the whole request a 2 s budget. If 1.4 s is gone, do not start a retry that needs 1 s. Retries that cannot finish are pure waste.
Exponential backoff with full jitter	$\text{delay} = \text{random}(0, \text{base} \times 2^{\text{attempt}})$. Without jitter, 10,000 clients that failed together retry together, producing a spike exactly as large as the original one.
A retry budget: at most 10% of traffic may be retries	Track the ratio. Above 10%, stop retrying entirely and fail fast. $1,000 \times 1.1 = 1,100$ req/s worst case, instead of 27,000. That is a 24x reduction against the naive design.
Circuit breaker	After N consecutive failures, stop calling for 30 seconds and fail immediately. This gives the database an idle window to catch up, which is the only thing that actually ends the storm.

The decision, stated plainly. Retry once, at the gateway, with jittered backoff, inside a 10% budget, behind a circuit breaker. That turns a 27x amplifier into a 1.1x one.

The trade-off, named honestly. Retrying less means more requests fail during a small, genuinely transient glitch — the user sees an error that a retry would have hidden. That is the price. It is worth paying, because the alternative is that a 30-second glitch becomes a 3-hour outage. **Choose a few visible failures over a metastable collapse.**

Answer: 27,000 req/s — 27x. Retry at one layer, jittered, inside a 10% budget, behind a breaker: 1.1x.

Example 25 [Adobe · pattern]

Cross-region replication. A region takes 200,000 writes per second, each 500 bytes on the replication stream. You replicate to two other regions. What bandwidth? What does the 250 ms round trip do to a synchronous design?

SOLUTION

Bandwidth per region pair.

$$200,000 \times 500 = 100,000,000 \text{ bytes/s} = 100 \text{ MB/s}$$

$$100 \times 8 = 800 \text{ Mbps}$$

To two regions.

$$800 \times 2 = 1.6 \text{ Gbps of cross-region traffic, continuously}$$

That is affordable — a couple of dedicated links — but note it runs 24 hours a day and inter-region traffic is billed at a much higher rate than traffic inside one region. Monthly volume:

$$100 \text{ MB/s} \times 86,400 \times 30 \times 2 = 5.18 \times 10^{14} \text{ bytes} = 518 \text{ TB/month}$$

At \$0.02 per GB that is $518,000 \times \$0.02 = \$10,360$ a month, just to keep copies in step.

Now the latency question — this is the real answer.

Synchronous replication means the write is not acknowledged until the remote regions have it. The floor is one round trip:

$$\text{write latency} \geq 250 \text{ ms}$$

Apply Little's Law to see what that does to concurrency:

$$L = 200,000 \text{ writes/s} \times 0.250 \text{ s} = 50,000 \text{ writes in flight}$$

Fifty thousand simultaneously open write transactions. No ordinary database holds that. Synchronous cross-continent replication at this write rate is **not slow** — **it is impossible**, and that is the sentence to say.

Asynchronous replication acknowledges locally (say 5 ms) and ships the change after.

$$L = 200,000 \times 0.005 = 1,000 \text{ in flight} \quad \checkmark$$

The cost is a **replication lag window**. At 100 MB/s of changes and a healthy link, lag is usually under a second, but during a network incident it grows. If the region dies with 3 seconds of lag:

$$200,000 \times 3 = 600,000 \text{ writes lost}$$

The decision, split by data type — this is what a senior answer looks like.

Data	Mode	Why
money movement, ledger	synchronous, but only within one region's 3 zones	≈ 1 ms between zones, so $L = 200$ in flight. Durable and fast. Cross-region is async — a bank accepts a recovery procedure, not a 250 ms write.
user profile, settings	asynchronous, last-write-wins on a time-stamp	a 2-second-old display name harms nobody
counters, view counts	asynchronous, merge by addition	order does not matter; the sum is the same
session tokens	not replicated at all	re-login on failover is cheaper than replicating them

Decision: asynchronous across regions, synchronous across zones within a region, and accept a bounded loss window that you measure and alarm on. The alternative — global synchronous writes — buys zero data loss and costs you the ability to serve traffic at all. Every large system that claims otherwise has quietly restricted synchronous writes to a small, low-rate subset of its data.

Answer: 1.6 Gbps, ≈ 518 TB/month. Sync across zones (1 ms), async across regions (250 ms is impossible at 200 K writes/s).

PRACTICE · TIER 3 target: 40 min

- 50 million posts a day, average 500 followers. Inbox writes per day, per second, and at 3x peak. How many 5,000-writes/s nodes with 1.5x headroom?
- A dataset is 300 TB and a shard holds 3 TB. Minimum shards? Round to a power of two.
- A request fans out to 30 shards, each fast 99% of the time. What fraction of requests are slow?
- Two layers each retry 4 times. Amplification factor? At 2,000 req/s of real traffic, what does the bottom layer see?
- 500,000 writes/s at 300 bytes replicate to one other region. Bandwidth in Gbps?
- A global service has a 2x peak factor, a single-country one has 3x. At 20 million requests a day, what is each peak QPS, and why does the global one get the smaller multiplier?

ANSWER KEY

- $50,000,000 \times 500 = 2.5 \times 10^{10}$ rows/day. $2.5 \times 10^{10} / 86,400 = 289,352/\text{s}$ average; $\times 3 = 868,056/\text{s}$ peak. Nodes: $868,056 / 5,000 = 173.6 \rightarrow 174$; $\times 1.5 = 261$ nodes.
- $300/3 = 100$ shards minimum; round up to **128**.
- 0.99^{30} . Using $0.99^{10} = 0.9044$: $0.9044^3 = 0.7397$. So $1 - 0.7397 = 0.260 = 26.0\%$ of requests are slow. At 30 shards a per-shard p99 is already not good enough.
- $4 \times 4 = 16x$. $2,000 \times 16 = 32,000$ req/s at the bottom layer.

5. $500,000 \times 300 = 150,000,000 \text{ B/s} = 150 \text{ MB/s} \times 8 = 1.2 \text{ Gbps}$.

6. $20,000,000 / 86,400 = 231.5 \text{ QPS}$ average. Global at $2x = 463 \text{ QPS}$. Single-country at $3x = 694 \text{ QPS}$. The global service gets the smaller multiplier because its users are spread across time zones: while India sleeps, Brazil is awake, so the daily curve is flatter. One country has one evening, and everybody arrives in it.

Interview drill — what the interviewer pushes on

The push	The answer that scores
“Where did that number come from?”	“I assumed 30 actions per user per day. If it is 10, every number below divides by 3 and I need 8 servers instead of 24. Tell me if you have a better figure and I will redo it in ten seconds.”
“Isn’t 3x peak too high / too low?”	“A flat day puts 4.2% of traffic in each hour. A single-country consumer app puts 12–15% in the evening hour, so 3x. For a global product I would use 2x; for a flash sale, 10x or more.”
“You rounded 86,400 to 100,000. Isn’t that wrong?”	“It makes my QPS about 14% low, and I said so. For choosing between 20 servers and 200 that does not matter. If we are sizing the actual order I will use 86,400.”
“Do you need a cache here?”	“Peak is (number) and one database replica does about (number). The ratio is (number). If it is under 1, no cache. Here it is (number), so I need a hit rate of $1 - \text{db capacity} / \text{peak} = (\text{number})\%$.”
“How much will this cost?”	“Egress is (GB/month) at about \$0.02 per GB = \$(number). Storage is (TB) at \$0.02 per GB-month = \$(number). Servers are (n) at \$60 = \$(number). The biggest line is (which), so that is where I would optimise.”
“Your storage number ignores indexes.”	“Correct. Add 30% for two secondary indexes: (number) becomes (number). It does not change the conclusion that this is (small/large), but the corrected number is (value).”
“What breaks first at 10x?”	“At 10x, peak goes from (number) to (number). The application tier just needs more machines. The first structural break is (component) at (its limit), because (reason). I would fix it by (change).”
“Why is bandwidth not your bottleneck?”	“Peak QPS times response size is (number) MB/s. A single 1 Gbps card does 125 MB/s. I am at (percent) of one card, so the bottleneck is request count, not bytes.”
“Is your fan-out number right?”	“One user action writes (n) rows because of (followers / replicas / indexes). So the API sees (number) writes a second and the storage sees (number times n). That multiplier is the whole reason I chose (design).”
“Can’t you just use a bigger machine?”	“Yes, up to a point, and it is cheaper than my time. One box tops out near 5,000 durable writes a second. I need (number). So vertical scaling works until (number) and then I shard.”
“How confident are you in these numbers?”	“Within about 2x. The assumption I am least sure about is (which), and if it is wrong by 5x then (what changes). Everything else survives being wrong by 2x.”
“Estimate it again, differently.”	“Sure — instead of DAU times actions, let me work backwards from storage: the database is (size) and rows are (bytes), so there are (n) rows, over (years) that is (rows/day), which agrees with my first number within (factor).”

SHORTCUT

The two-way check. Whenever you have time, estimate the same quantity from a different direction and see if the two agree. Forwards: users $\times$ actions $\rightarrow$ rows. Backwards: disk size $\div$ row size $\rightarrow$ rows. If the two

land within 2x of each other, your numbers are sound and you can say so. If they are 20x apart, one of your assumptions is wrong and you have just caught it yourself — which scores higher than never making the mistake.

TRAP

Three estimation habits that lose marks:

1. **Precision theatre.** “4,166.67 QPS” is worse than “about 4,200”. The extra digits claim an accuracy your assumptions do not have, and interviewers notice.
2. **No assumption list.** If you never say “I am assuming 30 actions per user”, the interviewer cannot correct you, and every later number looks like a guess.
3. **Numbers with no conclusion.** Computing 1.75 TB and then saying nothing is half a mark. “1.75 TB a year, so one database for five years, no sharding” is the full mark. Always end an estimate with the decision it just made.

ONE-PAGE REVISION CARD

The rule. One significant figure. Within 2x is good, 3x is acceptable, 10x is a fail. Always end with “so the binding constraint is ____”.

Time.

day	86,400 s	round to 10^5
month (30 d)	2.59 M s	
year	31.5 M s	
1 M/day	11.6 /s	multiply daily millions by 11.6
1 B/day	11,574 /s	

Bytes. $2^{10} \approx 10^3$, $2^{20} \approx 10^6$, $2^{30} \approx 10^9$, $2^{32} = 4.29 \times 10^9$, $2^{63} = 9.2 \times 10^{18}$. 1 KB $\times$ 1 M = 1 GB. 1 KB $\times$ 1 B = 1 TB. 1 MB $\times$ 1 M = 1 TB. Bytes are capital B, bits are small b: 1 Gbps = 125 MB/s.

Latency. L1 0.5 ns · memory 100 ns · 1 MB from memory 30 μ s · SSD random read 100 μ s · same-DC round trip 500 μ s · disk seek 10 ms · Mumbai $\leftrightarrow$ Singapore 60 ms · Mumbai $\leftrightarrow$ Virginia 250 ms. Memory is $\approx$ 5,000x faster than a network hop. Physics sets 250 ms cross-ocean; only a nearby copy fixes it.

One machine does. App server 1,000–2,000 req/s · Postgres reads 10–20 k/s · Postgres durable writes 3–5 k/s · Redis 100 k ops/s · 100 k WebSockets held · 1 Gbps card 125 MB/s.

JavaScript memory. Int32Array 4 B/elem · Array of ints 8 B/elem · object with 4 fields $\approx$ 64 B · Map entry with string key $\approx$ 61 B · Map<int, object> $\approx$ 85 B.

Peak factors. Global 2x · one country consumer 3x · one country work app 4–5x · event/sale 10–50x. Defence: flat is 4.2% per hour, real evening peak is 12–15%.

The five lines.

traffic DAU $\times$ actions $\div$ 86,400 $\times$ peak
storage rows/day $\times$ bytes/row $\times$ 365 $\times$ years $\times$ RF ($\times$ 1.3 for indexes)
bandwidth peak QPS $\times$ response bytes ($\times$ 8 for bits)
memory hot items $\times$ bytes/item
machines peak QPS $\div$ per-server QPS $\times$ 1.5

Four formulas that decide designs.

Little’s Law	$L = \lambda W$ — in flight = rate $\times$ latency. Sizes every pool.
cache relief	database load drops by $1/(1-h)$. 90% hit = 10x; 99% = 100x.
needed hit rate	$1 - \text{db capacity} / \text{peak QPS}$
fan-out tail	$P(\text{slow}) = 1 - p^n$. 50 shards at p99 = 39.5% slow.

Amplifiers to look for. Followers (1 post $\rightarrow$ 300 rows) · replication ($\times$ 3) · indexes (+30%) · retries (3^{layers}) · logs (often bigger than the data) · telemetry (pings can be 1,000x business traffic).

Things that gain marks. Stating assumptions before the arithmetic. Doing one division out loud. Saying “this is small, one server is enough” when it is true. Ending with the binding constraint. Estimating the same thing twice from two directions.

Things that lose marks. Precision theatre. No assumptions. Numbers with no conclusion. Forgetting the write amplifier. Quoting a hit rate without computing the miss traffic.

The recovery line. “Let me redo that division out loud.”

OOP & SOLID for Interviews

Classes that survive the next change

The idea in one page

An LLD round is 45 minutes long. The interviewer gives you a small system — a bill, a locker, a fee engine — and asks you to write the classes. Nobody runs your code. So what are they actually grading?

What they watch	What a pass looks like
Did you find the right nouns?	4 to 8 classes, each with one job you can say in one sentence
Where does each rule live?	Every rule sits in exactly one class, not copied in three
What happens when I add a feature?	You add a class or a line. You do not edit five old files
Can this be tested?	The rules class needs no database to run
Did you name the trade-off?	You named two options and picked one, with a reason

That last row is the one most students lose. Saying “it depends” scores zero. Saying “a map of small rule objects, because a new coupon then adds one line instead of editing a 40-branch `if`” scores full marks.

THE FOUR PILLARS, AND THE ONE TEST FOR EACH

Encapsulation — the data and the rules that guard it live inside one class, and the outside world cannot reach past the rules.

Test: can an outsider put the object into a state your class calls illegal? If yes, it is not encapsulated.

Abstraction — the caller is given a short list of things it can ask for, and is not shown how any of them work.

Test: can you replace the whole inside of the class and leave every caller untouched? If yes, the abstraction is good.

Inheritance — a child class **is a** kind of the parent, and can be used anywhere the parent is expected.

Test: read every promise the parent makes. Does the child keep all of them? If not, it should not inherit.

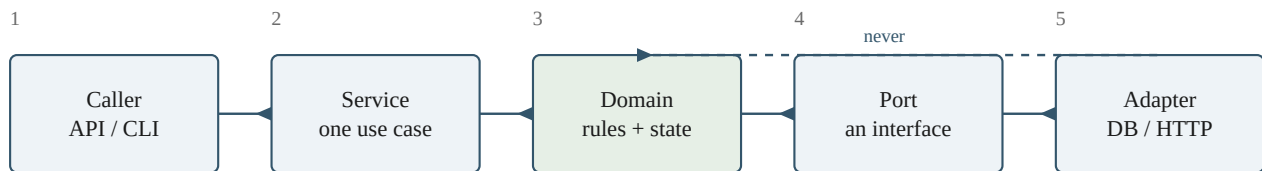
Polymorphism — one call site, many behaviours, chosen by the real type of the object at run time.

Test: can you add a new behaviour without touching the call site? If yes, it is real polymorphism, not a disguised `if`.

JavaScript has classes but no `interface` keyword, no `abstract` keyword, and no method overloading. This chapter teaches the concept in JavaScript, and then states plainly what the Java or C++ answer is — because a service-company interviewer will ask for the answer in that vocabulary, and “JavaScript does not have that” is not an accepted reply.

The shape of every LLD answer

Draw this on the board before you write a single class. It tells the interviewer you have a plan.



The five layers. Arrows point the way a call travels. The dashed arrow is the one that must NOT exist.

Layer 3 is where the marks are. It holds the rules, holds no connection object, and can be run in a test in under a millisecond. The dashed arrow says: the database must never reach back into your rules.

SOLID — FIVE RULES, FIVE SMELLS

Letter	The rule in one line	The smell that triggers it
S — Single responsibility	A class should have one reason to change.	The class name has “and” in it, or two teams edit the same file.
O — Open/closed	Open to add new behaviour, closed to editing old code.	A growing <code>if / else if</code> chain or <code>switch</code> on a type string.
L — Liskov substitution	A child must be usable wherever the parent is, with no surprise.	An overridden method that throws, or does nothing, or asks “am I really a X?”.
I — Interface segregation	Many small contracts beat one fat one.	Half the methods of an implementation throw “not supported”.
D — Dependency inversion	Rules depend on shapes; details plug into the shapes.	The rules file imports a database driver.

SHORTCUT

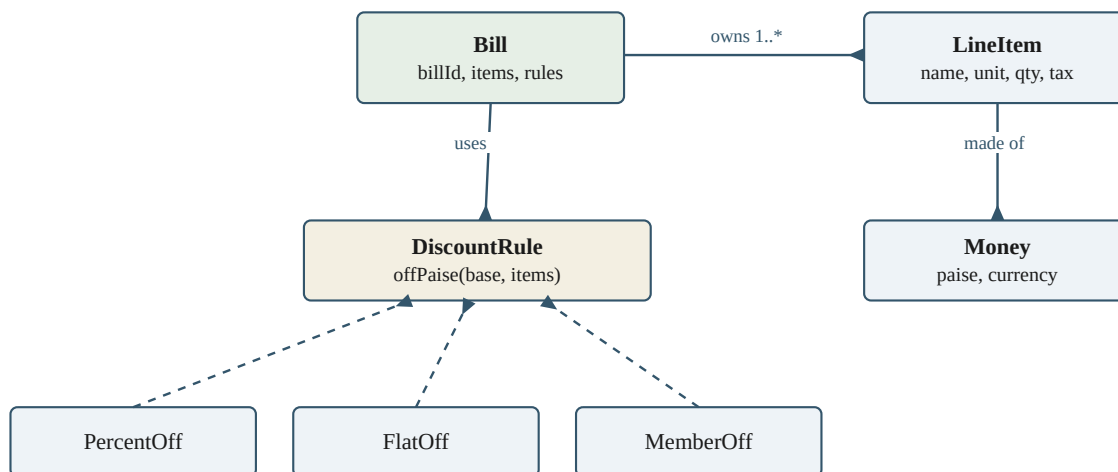
In an interview, never recite the five names. Point at the code and say the **smell**: “this switch grows every time marketing invents a coupon — that is the open/closed smell, so I will hold the coupons in a map of small objects.” One sentence, one fix, done.

UML-lite: the only notation you need

You do not need full UML. You need four arrows and a box. An interviewer who sees these knows you have drawn a class diagram before.

Relationship	Say it as	How to draw it fast
Inheritance	Car is a Vehicle	plain arrow, child → parent
Implements an interface	Csv is a kind of Format	dashed arrow, class → interface
Composition (owns)	Bill owns its LineItems; kill the bill, the lines die	solid arrow with 1..* on it
Association (uses)	Bill uses a DiscountRule it did not create	thin arrow, label uses

A class box has three floors: the name, the fields, the methods. Only write the fields and methods that matter to the question.



A class diagram for the food-court bill built later in this chapter. Dashed arrows mean 'is a kind of'.

Read it out loud: “A bill owns one or more line items. A line item is made of money. A bill uses discount rules; percent-off, flat-off and member-off are each a kind of discount rule.” That sentence is the whole design.

Warm-up

WARM-UP

build the reflex

Example 1

A Ticket class is needed for a cinema. Which of these belong as **fields** of Ticket, and which do not? seatNumber, showTime, pricePaise, theatreAddress, bookedBy, totalSeatsInHall.

SOLUTION

Ask one question per candidate: **does this change when the ticket changes?**

Field	Belongs?	Why
seatNumber	yes	different for every ticket
showTime	yes	a ticket is for one show
pricePaise	yes	what this buyer paid, frozen at purchase
theatreAddress	no	belongs to Theatre; every ticket would carry a copy
bookedBy	yes	who holds it
totalSeatsInHall	no	belongs to Hall; nothing to do with one seat

Answer: seatNumber, showTime, pricePaise, bookedBy. The other two are copies of another class's data.

Example 2

For each pair say **is-a** (inheritance) or **has-a** (composition): (a) Car and Engine; (b) SavingsAccount and Account; (c) Order and Address; (d) Circle and Shape; (e) Playlist and Song.

SOLUTION

The test: put the words in the sentence “every ___ is a ___”. If it sounds silly, it is has-a.

- (a) “every car is an engine” — silly. A car **has an** engine. → composition.
- (b) “every savings account is an account” — true. → inheritance.
- (c) “every order is an address” — silly. An order **has a** delivery address. → composition.
- (d) “every circle is a shape” — true. → inheritance.

- (e) “every playlist is a song” — silly. A playlist **has many** songs. → composition.

Answer: is-a: (b), (d). has-a: (a), (c), (e).

Example 3

Run this in your head, then check. What are the three lines printed?

dispatch.js — the part that surprises people

JS

```
class Fee {
  amount(orderPaise) { throw new Error('subclass must implement amount()'); }
  label() { return `${this.constructor.name} -> ${this.amount(100000)} paise`; }
}
class FlatFee extends Fee { amount(_) { return 2000; } }
class PercentFee extends Fee { amount(orderPaise) { return Math.round(orderPaise * 0.04); } }
class FreeFee extends Fee { amount(_) { return 0; } }

for (const f of [new FlatFee(), new PercentFee(), new FreeFee()]) {
  console.log(f.label());
}
```

SOLUTION

label() is written once, in the parent. It calls this.amount(...). At run time this is the real object, so a different amount runs each time. The parent never knew the children existed.

measured output of node dispatch.js

TEXT

```
FlatFee -> 2000 paise
PercentFee -> 4000 paise
FreeFee -> 0 paise
```

$100000 \times 0.04 = 4000$, so PercentFee prints 4000.

Answer: One loop, three behaviours. That is polymorphism, and label() is the call site that never changes.

Example 4

JavaScript trap. What does this print, and what would Java print?

Two methods with the same name

JS

```
class Router {
  go(a) { return 'one arg'; }
  go(a, b) { return `two args: ${a},${b}`; }
}
console.log(new Router().go('x'));
```

SOLUTION

In JavaScript a class body is a list of property assignments. The second go **replaces** the first. Only one go survives, and it is the two-argument one, called with b missing.

measured output

TEXT

```
two args: x,undefined
```


What the interviewer expects to hear about Java. Java has real overloading: the two methods have different **signatures**, both exist, and the compiler picks one by the argument list at compile time. That choice is made from the **declared** type, not the runtime type. Overriding is the opposite: chosen at run time from the real object.

TRAP

Overloading is resolved at **compile** time by the declared types. Overriding is resolved at **run** time by the real object. Say this sentence exactly. It is the most-asked one-liner in a service-company OOP round, and JavaScript cannot show it to you, because JavaScript has no overloading at all.

Answer: JavaScript prints two args: x, undefined. JavaScript has no overloading; Java does, and resolves it at compile time.

Example 5

Which pillar is broken in each line? (a) `order.status = 'PAID'` written from outside the `Order` class. (b) A `Report` class with a public method `getMySQLConnection()`. (c) `if (shape.type === 'circle') ... else if (shape.type === 'square') ...` (d) `class Ostrich extends Bird { fly() { throw new Error('cannot fly'); } }`

SOLUTION

- (a) **Encapsulation.** The status rule (which changes are legal) lives outside the class now. Fix: `order.markPaid()`.
- (b) **Abstraction.** The caller is shown the machinery. Fix: expose `save()`, hide the connection.
- (c) **Polymorphism.** The call site must be edited for every new shape. Fix: `shape.area()`.
- (d) **Inheritance / Liskov.** The child breaks the parent's promise. Fix: `Bird` and `FlyingBird`.

Answer: (a) encapsulation (b) abstraction (c) polymorphism (d) inheritance

Example 6

Abstract class or interface? Pick one for each, and say why in one line. (a) Every payment method must have `charge()`, and they share nothing else. (b) Every report has the same three-step publish flow, but step 2 differs per report. (c) A class needs to be both `Serializable` and `Comparable`.

SOLUTION

- (a) **Interface.** Nothing to share, only a shape to promise.
- (b) **Abstract class.** The shared flow is real code that lives in the parent; only step 2 is abstract. (This is the template method pattern — Chapter 4.)
- (c) **Interfaces.** A class can implement many interfaces, and in Java can extend only one class.

What the interviewer expects to hear. In Java: an interface holds no state and (before default methods) no code; a class implements many interfaces but extends one class; an abstract class may hold fields, a constructor, and finished methods. In JavaScript neither keyword exists — you promise a shape by throwing from the base method, or by checking `typeof obj.charge === 'function'`.

Answer: (a) interface (b) abstract class (c) interfaces

Example 7

Here are two wallets. One of them can be pushed into an illegal state from outside. Which one, and what exactly does the program print?

wallet.js

JS

```
class OpenWallet { // no encapsulation
  constructor() { this.paise = 0; }
}
```

```

class SafeWallet { // encapsulated with real private fields (#)
  #paise = 0;
  #spentToday = 0;

  add(amount) {
    if (amount <= 0) throw new Error('add must be > 0');
    this.#paise += amount;
  }
  spend(amount) {
    if (amount <= 0) throw new Error('spend must be > 0');
    if (amount > this.#paise) throw new Error('not enough money');
    this.#paise -= amount;
    this.#spentToday += amount; // the two fields can never drift apart
  }
  get balance() { return this.#paise; }
  get spentToday() { return this.#spentToday; }
}

const o = new OpenWallet();
o.paise = 5000;
o.paise = -900; // nothing stops this
console.log('OpenWallet balance =', o.paise);

const s = new SafeWallet();
s.add(5000);
s.spend(1200);
console.log('SafeWallet balance =', s.balance, 'spentToday =', s.spentToday);
try { s.spend(99999); } catch (e) { console.log('blocked:', e.message); }
s.paise = -900; // this does NOT touch #paise
console.log('outsider wrote s.paise =', s.paise);
console.log('real balance still =', s.balance);

```

SOLUTION

measured output of node wallet.js

TEXT

```

OpenWallet balance = -900
SafeWallet balance = 3800 spentToday = 1200
blocked: not enough money
outsider wrote s.paise = -900
real balance still      = 3800

```

Step by step for SafeWallet: add(5000) → balance 5000. spend(1200) → balance 5000 − 1200 = 3800 and spentToday 1200. spend(99999) is rejected because 99999 > 3800, so **both** fields stay untouched.

The last two lines are the JavaScript detail worth knowing. Writing `s.paise = -900` from outside does not fail and does not touch `#paise`. It silently creates a brand new, unrelated public property. The real balance is still 3800.

SHORTCUT

Private fields written with # are the only truly private members in JavaScript. A leading underscore (`_paise`) is a polite request, not a lock. Say this if asked how JavaScript does private: “#field is enforced by the language; `_field` is a naming convention only.”

Answer: OpenWallet can be corrupted. SafeWallet cannot — its balance stays 3800.

Example 8

Name the SOLID letter broken by each smell. (a) class `UserServiceAndEmailer` (b) `calculateFee()` has 31 else if branches, one per city. (c) class `ReadOnlyList` extends `List` { `add(x)` { `throw new Error('read only');` } } (d) interface `Vehicle` { `drive(); fly(); sail();` } (e) class `ReportService` { `constructor()` { `this.db = new MySQLDriver();` } }

SOLUTION

- (a) **S**. The name says two jobs. Two teams will fight over this file.
- (b) **O**. A 32nd city means editing a function that already works.
- (c) **L**. The child weakens a promise the parent made.
- (d) **I**. A truck must implement `fly()` and lie about it.
- (e) **D**. The rules class builds its own concrete driver, so it cannot be tested without MySQL.

Answer: (a) S (b) O (c) L (d) I (e) D

Tier 1 — low-level design

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 9 [TCS NQT · pattern]

Single responsibility, done on real code. This class works. Marketing, the billing team, the database team and the design team all edit it. Split it, and say how many reasons to change each new class has.

before — one class, four reasons to change

JS

```
class InvoiceGod {
  constructor(id, lines) { this.id = id; this.lines = lines; }
  total() { const s = this.lines.reduce((a, b) => a + b, 0); return s + Math.round(s * 0.18); }
  render() { return `Invoice ${this.id} payable=${this.total()}`; } // screen team
  save() { console.log('[sql] INSERT', this.id); } // db team
  email() { console.log('[smtp] sent', this.id); } // comms team
}
```

SOLUTION

Find the reasons to change, not the methods. Ask: “who files the ticket that edits this line?”

Line	Who files the ticket
0.18	finance, when GST changes
render()	the design team, when the layout changes
save()	the platform team, when the database changes
email()	the comms team, when the template changes

Four filers, four classes.

after — one reason to change each

JS

```
class Invoice { // the data and its own rule only
  constructor(id, linePaise) { this.id = id; this.linePaise = linePaise; }
}
```

```

class InvoiceTotal {                                // changes when the tax rule changes
  static GST_PERCENT = 18;
  of(inv) {
    const sum = inv.linePaise.reduce((a, b) => a + b, 0);
    return sum + Math.round(sum * InvoiceTotal.GST_PERCENT / 100);
  }
}
class InvoiceText {                                // changes when the layout changes
  constructor(totals) { this.totals = totals; }
  render(inv) {
    return `Invoice ${inv.id} lines=${inv.linePaise.length}
payable=${this.totals.of(inv)}`;
  }
}
class InvoiceStore {                               // changes when the database changes
  #rows = new Map();
  save(inv) { this.#rows.set(inv.id, inv); return this.#rows.size; }
}

const inv = new Invoice('INV-7', [12000, 45000, 3000]);
const totals = new InvoiceTotal();
console.log('sum   =', 12000 + 45000 + 3000);
console.log('gst   =', Math.round(60000 * 0.18));
console.log('total =', totals.of(inv));
console.log(new InvoiceText(totals).render(inv));
console.log('stored rows =', new InvoiceStore().save(inv));

```

measured output of node srp.js

TEXT

```

sum   = 60000
gst   = 10800
total = 70800
Invoice INV-7 lines=3 payable=70800
stored rows = 1

```

Check the arithmetic by hand: $12000 + 45000 + 3000 = 60000$ paise. $60000 \times 18 \div 100 = 10800$. $60000 + 10800 = 70800$ paise = Rs 708.00. ✓

TRAP

Splitting is not free. Four classes mean four files and four names to remember. Split when **two different people** have a reason to edit the same file — not because a class reached 50 lines. A 200-line class with one job is healthier than six 30-line classes with tangled jobs.

Answer: Four classes: Invoice (data), InvoiceTotal (money rule), InvoiceText (layout), InvoiceStore (storage). One reason to change each.

Example 10 [Infosys · pattern]

Open/closed, with the cost counted. A coupon function grows one if per campaign. Marketing ships 2 new coupons a month. Rewrite it so a new coupon never edits an old line, then count how many lines change over a year, both ways.

SOLUTION

before — every new coupon edits this function

JS

```
function offBefore(code, orderPaise) {
  if (code === 'STUDENT') return Math.round(orderPaise * 0.10);
  if (code === 'FESTIVE') return Math.min(Math.round(orderPaise * 0.20), 15000);
  return 0;
}
```

after — a registry of tiny rules

JS

```
const rules = new Map();
const register = (code, fn) => rules.set(code, fn);

register('NONE',    () => 0);
register('STUDENT', (p) => Math.round(p * 0.10));
register('FESTIVE', (p) => Math.min(Math.round(p * 0.20), 15000));
register('FIRST3',  (p) => (p >= 20000 ? 5000 : 0)); // new: nothing above
changed

function off(code, orderPaise) {
  const rule = rules.get(code) ?? rules.get('NONE');
  return Math.min(rule(orderPaise), orderPaise); // never below zero
payable
}

const order = 90000; // Rs 900.00 in
paise
for (const code of rules.keys()) {
  const d = off(code, order);
  console.log(code.padEnd(8), 'off=', String(d).padStart(6), 'pay=', order - d);
}
console.log('unknown code falls back:', off('WHATEVER', order));
```

measured output of node ocp.js

TEXT

```
NONE      off=      0 pay= 90000
STUDENT   off=   9000 pay= 81000
FESTIVE   off=  15000 pay= 75000
FIRST3    off=   5000 pay= 85000
unknown code falls back: 0
```

Hand-check FESTIVE on an order of 90000 paise: $90000 \times 0.20 = 18000$, but the cap is 15000, and $\min(18000, 15000) = 15000$. Payable = $90000 - 15000 = 75000$ paise = Rs 750.00. ✓

Now count the cost over one year. 2 coupons a month $\times$ 12 months = 24 new coupons.

	Lines added	Lines edited	Risk
if chain	24	24 (the function body)	each edit can break the 23 live coupons
registry	24	0	a broken new rule cannot touch the old ones

The function under test is edited 24 times in one version, and 0 times in the other. That is the whole argument for open/closed, and it is an argument about **regression risk**, not beauty.

Trade-off, and the decision. The if chain is 4 lines shorter and you can read every coupon in one screen. The registry costs one extra concept (a map of functions) and the rules are now spread out. **Decision: use the registry.** The tipping point is about 4 branches; below that the if chain wins on readability, above that the regression risk wins. This function was already at 3 and grows 24 a year, so it is far past the line.

Answer: Registry of small rule objects. Lines edited per new coupon drops from 24 a year to 0.

Example 11 [Capgemini · pattern]

Liskov substitution, caught at run time. GiftCard extends PrepaidCard looks harmless. Run the caller and see what happens.

```
lsp.js — the break JS

class PrepaidCard {
  #paise = 0;
  // promise 1: after load(x), balance is exactly x higher
  load(x) { this.#paise += x; }
  // promise 2: after pay(x) with x <= balance, balance is exactly x lower
  pay(x) { if (x > this.#paise) throw new Error('insufficient'); this.#paise -= x; }
  get balance() { return this.#paise; }
}

class GiftCard extends PrepaidCard {
  load(x) { /* "gift cards cannot be topped up" */ } // breaks promise 1
}

function topUpAndSpend(card) { // knows only the base class
  card.load(10000);
  try {
    card.pay(4000);
    console.log(card.constructor.name, 'balance =', card.balance);
  } catch (e) {
    console.log(card.constructor.name, 'CRASHED:', e.message);
  }
}

topUpAndSpend(new PrepaidCard());
topUpAndSpend(new GiftCard());
```

SOLUTION

measured output of node lsp.js (first two lines)

TEXT

```
PrepaidCard balance = 6000
GiftCard CRASHED: insufficient
```

Trace it. For PrepaidCard: $0 + 10000 = 10000$, then $10000 - 4000 = 6000$. ✓ For GiftCard: load does nothing, balance stays 0, so `pay(4000)` sees $4000 > 0$ and throws. The caller never asked for a gift card. It asked for “a prepaid card” and got a crash.

The fix is to stop lying about the hierarchy. Build the base around what is always true — a card can be spent — and let a subclass only **add**.

the fix: a subclass may only ADD, never weaken

JS

```
class Spendable {
  constructor(paise) { this._paise = paise; } // subclasses may use it
  pay(x) { if (x > this._paise) throw new Error('insufficient'); this._paise -= x; }
  get balance() { return this._paise; }
}

class Reloadable extends Spendable { // adds load(); changes
```

```

nothing else
  load(x) { this._paise += x; }
}

const gift = new Spendable(10000); // a gift card IS only
spendable
gift.pay(4000);
console.log('gift balance =', gift.balance, '| has load()?', typeof gift.load);

const wallet = new Reloadable(0); // spendable AND reloadable
wallet.load(10000);
wallet.pay(4000);
console.log('wallet balance =', wallet.balance, '| has load()?', typeof
wallet.load);

```

measured output

TEXT

```

gift balance = 6000 | has load()? undefined
wallet balance = 6000 | has load()? function

```

THE LISKOV CHECKLIST YOU CAN RUN IN YOUR HEAD

A child class is safe if, compared with the parent, it:

1. accepts **the same or wider** inputs (never narrower — no new “this argument is not allowed”),
2. returns **the same or narrower** outputs (never a surprise type, never null where the parent never returned null),
3. throws **no new kind** of exception the caller was not warned about,
4. leaves every promise about state that the parent made still true,
5. does not need the caller to know which subclass it holds.

Break any one of the five and you must use composition instead.

Answer: GiftCard breaks promise 1, so the caller crashes. Fix: Spendable base, Reloadable adds load().

Example 12 [Wipro · pattern]

Interface segregation. Every office device implements print, scan, fax, staple. A cheap printer can only print. Show what goes wrong, then fix it.

SOLUTION

isp.js — the fat contract and the fix

JS

```

// ----- BEFORE: one fat contract -----
class OfficeMachine { // every device must implement all four
  print(doc) { throw new Error('not supported'); }
  scan() { throw new Error('not supported'); }
  fax(doc) { throw new Error('not supported'); }
  staple(doc) { throw new Error('not supported'); }
}

class CheapPrinter extends OfficeMachine {
  print(doc) { return `printed ${doc}`; }
  // scan / fax / staple inherited -> they throw. Callers cannot tell.
}

// ----- AFTER: small contracts, mixed in only where true -----

```

```

const CanPrint = (Base) => class extends Base { print(doc) { return `printed ${doc}`; } };
const CanScan  = (Base) => class extends Base { scan()      { return 'scan-0001.png'; } };
const CanFax   = (Base) => class extends Base { fax(doc)    { return `faxed ${doc}`; } };

class Device {}
class HomePrinter extends CanPrint(Device) {}
class OfficeCombo extends CanFax(CanScan(CanPrint(Device))) {}

const supports = (d, job) => typeof d[job] === 'function';

for (const d of [new CheapPrinter(), new HomePrinter(), new OfficeCombo()]) {
  const jobs = ['print', 'scan', 'fax'].filter((j) => supports(d, j));
  console.log(d.constructor.name.padEnd(13), 'claims:', jobs.join(','));
}
console.log('HomePrinter can scan?', supports(new HomePrinter(), 'scan'));
console.log('CheapPrinter can scan?', supports(new CheapPrinter(), 'scan'), '<- a lie');

```

measured output of node isp.js

TEXT

```

CheapPrinter  claims: print,scan,fax
HomePrinter   claims: print
OfficeCombo   claims: print,scan,fax
HomePrinter can scan? false
CheapPrinter can scan? true <- a lie

```

Look at line 1 against line 2. The fat design makes CheapPrinter **claim** three abilities it does not have. Any code that asks “can you scan?” gets true and then crashes at the call. In the split design the question is answered honestly.

What the interviewer expects to hear. In Java you would declare three interfaces — Printer, Scanner, Fax — and class CheapPrinter implements Printer. The compiler then refuses to pass a CheapPrinter where a Scanner is wanted, so the lie is caught before the program runs. JavaScript has no interfaces, so the same idea is expressed with mixins plus a typeof check, and the lie is caught at run time instead.

Trade-off, and the decision. One fat base class is fewer files and one place to look. Three small contracts mean more names but no false claims. **Decision: split.** The moment any implementation is forced to write “not supported”, the fat interface has already cost you a production incident; the extra names are cheaper.

Answer: Split into CanPrint / CanScan / CanFax. The fat base makes classes claim abilities they do not have.

Example 13 [Accenture · pattern]

Dependency inversion, and the payoff you can measure. PlaceOrder writes to Postgres and sends an SMS. Make it testable without a database and without an SMS bill.

SOLUTION

The rule: **the use case names the shape it needs; the details plug in from outside.**

dip.js

JS

```

class PlaceOrder {
  constructor({ orders, notifier, clock }) {
    this.orders = orders; this.notifier = notifier; this.clock = clock;

```



```

    }
    run(userId, paise) {
      if (paise <= 0) throw new Error('amount must be > 0');
      const id = `o-${this.clock()}`;
      this.orders.save({ id, userId, paise });
      this.notifier.send(userId, `Order ${id} placed for ${paise} paise`);
      return id;
    }
  }
}

// ---- production adapters ----
class PostgresOrders { save(o) { console.log('[pg] INSERT', o.id, o.userId, o.paise); } }
class SmsNotifier { send(u, t) { console.log('[sms] ->', u, ':', t); } }

// ---- test doubles: no database, no SMS bill ----
class FakeOrders { constructor() { this.rows = []; } save(o) { this.rows.push(o); } }
class SpyNotifier { constructor() { this.sent = []; } send(u, t) { this.sent.push([u, t]); } }

new PlaceOrder({ orders: new PostgresOrders(), notifier: new SmsNotifier(), clock:
() => 1001 })
  .run('u-1', 45000);

const orders = new FakeOrders(), notifier = new SpyNotifier();
const id = new PlaceOrder({ orders, notifier, clock: () => 2002 }).run('u-2',
45000);
console.log('test: id =', id, '| rows =', orders.rows.length, '| messages =',
notifier.sent.length);
console.log('test: message text =', notifier.sent[0][1]);

```

measured output of node dip.js

TEXT

```

[pg] INSERT o-1001 u-1 45000
[sms] -> u-1 : Order o-1001 placed for 45000 paise
test: id = o-2002 | rows = 1 | messages = 1
test: message text = Order o-2002 placed for 45000 paise

```

Notice `clock` is injected too. A real `Date.now()` makes the test unrepeatable; a function returning 2002 makes the order id predictable, so the test can assert on it.

The payoff, in numbers. Suppose a test that really talks to Postgres and a real SMS sandbox takes 400 ms, and the in-memory version takes 0.4 ms. A suite of 600 such tests:

- with real adapters: $600 \times 400 \text{ ms} = 240\{, \}000 \text{ ms} = 240 \text{ seconds} = 4 \text{ minutes}$.
- with fakes: $600 \times 0.4 \text{ ms} = 240 \text{ ms} = 0.24 \text{ seconds}$.

$240 \div 0.24 = 1000$ times faster. A 4-minute suite gets run once before lunch. A quarter-second suite gets run on every save. That change in habit is the real return on dependency inversion.

TRAP

Injection is not “pass everything in”. If `PlaceOrder` also took `taxPercent`, `currency`, `retryCount` and `logger` through the constructor, the wiring code becomes the new god class. Inject the things that **touch the outside world** (database, network, clock, random) and hard-code the things that are part of the rule.

Answer: Inject orders, notifier and clock. Tests use in-memory doubles and run about 1000 times faster.

Example 14 [Cognizant · pattern]

Composition over inheritance, with the explosion counted. A report can be CSV, JSON or TSV; stored on disk or on S3; zipped or not. Build it with inheritance, count the classes. Then build it with composition and count again.

SOLUTION

With inheritance you need one subclass per combination: $3 \text{ formats} \times 2 \text{ stores} \times 2 \text{ zip options} = 12$ subclasses, named things like `JsonS3GzipReport`. Add a 4th format and you need $4 \times 2 \times 2 = 16$ — four new classes for one new idea.

With composition you need one small class per independent choice: $3 + 2 + 2 = 7$ classes, and you **combine** them at run time. Add a 4th format and you write **one** class; the count goes to 8 and covers all 16 combinations.

```
compose.js JS

class Csv { render(rows) { return rows.join(','); } }
class Json { render(rows) { return JSON.stringify(rows); } }
class Tsv { render(rows) { return rows.join('\t'); } }

class Disk { put(name, body) { return `disk/${name} (${body.length}B)`; } }
class S3 { put(name, body) { return `s3://reports/${name} (${body.length}B)`; } }

class NoZip { pack(s) { return s; } }
class FakeGz { pack(s) { return s.slice(0, Math.ceil(s.length / 2)); } } //
pretend 2x

class Report { // HAS-A format, HAS-A zip, HAS-A store
  constructor(format, zip, store) { this.format = format; this.zip = zip;
  this.store = store; }
  publish(name, rows) { return this.store.put(name,
  this.zip.pack(this.format.render(rows))); }
}

const rows = ['seat=A12', 'seat=B07', 'seat=C03'];
let n = 0;
for (const f of [new Csv(), new Json(), new Tsv()])
  for (const z of [new NoZip(), new FakeGz()])
    for (const s of [new Disk(), new S3()]) {
      n++;
      if (n <= 4 || n === 12) console.log(String(n).padStart(2), new Report(f, z,
s).publish('r', rows));
    }
console.log('combinations built =', n, 'from classes =', 3 + 2 + 2);
```

measured output of node compose.js

TEXT

```
1 disk/r (26B)
2 s3://reports/r (26B)
3 disk/r (13B)
4 s3://reports/r (13B)
12 s3://reports/r (13B)
combinations built = 12 from classes = 7
```

The CSV body is `seat=A12,seat=B07,seat=C03` = 26 bytes. The fake zip keeps $\lceil 26 \div 2 \rceil = 13$ bytes. ✓

Design	3 formats	4 formats	classes per new format
inheritance	12	16	4
composition	7	8	1

Trade-off, and the decision. Inheritance gives one concrete name per behaviour, which is easy to read in a stack trace. Composition needs the caller to assemble three pieces, which is one more line at every construction site. **Decision: composition**, and it is not close — the multiply in $3 \times 2 \times 2$ is what kills inheritance here. Use inheritance only when the axes do not multiply, that is, when there is genuinely one axis of variation.

SHORTCUT

Count the **axes of variation** out loud. One axis → inheritance is fine. Two or more axes that can be combined freely → composition, always. This sentence alone will carry you through most “inheritance vs composition” follow-ups.

Answer: 12 subclasses vs 7 small classes; adding a format costs 4 classes vs 1. Choose composition.

Example 15 [TCS Digital · pattern]

Value objects and money. Why is pricePaise an integer and not price = 125.50? Write a Money class that cannot be corrupted.

SOLUTION

money.js

JS

```
class Money {
  static of(paise, currency) { return new Money(paise, currency); }
  constructor(paise, currency) {
    if (!Number.isInteger(paise)) throw new Error('paise must be a whole number');
    this.paise = paise; this.currency = currency;
    Object.freeze(this); // no field can ever change again
  }
  plus(o) {
    if (o.currency !== this.currency)
      throw new Error(`cannot add ${o.currency} to ${this.currency}`);
    return new Money(this.paise + o.paise, this.currency); // a NEW object
  }
  times(n) { return new Money(Math.round(this.paise * n), this.currency); }
  get key() { return `${this.currency}:${this.paise}`; }
  equals(o) { return o instanceof Money && o.key === this.key; }
  toString() { return `${this.currency} ${this.paise / 100}.toFixed(2)}`; }
}
```

measured output of node money.js

TEXT

```
INR 125.50 + INR 44.50 = INR 170.00
a unchanged: INR 125.50
equal by value: true
18% GST on 60000 paise = INR 108.00
blocked: cannot add SGD to INR
0.1 + 0.2 = 0.30000000000000004 | equals 0.3? false
```

The last line is the answer to the question. A JavaScript number is a 64-bit float. It cannot hold 0.1 exactly, so $0.1 + 0.2$ lands on 0.30000000000000004 and the equality check is false. Money in floats drifts by fractions of a

paise, and a ledger that adds a million rows ends up visibly wrong. Store the smallest unit as an integer — paise, cents, satang — and divide only when printing.

Check the GST line: $60000 \times 0.18 = 10800$ paise = Rs 108.00. ✓

WHAT MAKES A CLASS A VALUE OBJECT

1. It has no identity of its own. Two Money (17000, "INR") are the same thing.
2. It is **immutable**: Object.freeze in the constructor, and every operation returns a new object.
3. Equality is by **value**, not by reference — give it a key getter and compare keys.
4. It validates itself in the constructor, so an invalid one can never exist.

Money, Date range, Coordinates, Email address, Seat number. Make these value objects and a whole family of bugs disappears.

Answer: Floats cannot hold money exactly ($0.1 + 0.2 \neq 0.3$). Store integer paise in a frozen Money value object.

Example 16 [Cognizant · pattern]

The identity trap that costs a booking. A seat is { row: 'A', number: 12 }. It is used as a key in a Map. Predict the output.

keybug.js

JS

```
const seatA = { row: 'A', number: 12 };

const m = new Map();
m.set(seatA, 'rishabh');
console.log('same object    ->', m.get(seatA));
console.log('equal copy    ->', m.get({ row: 'A', number: 12 }));

const obj = {};
obj[seatA] = 'rishabh';
obj[{ row: 'B', number: 7 }] = 'aisha';
console.log('object keys    ->', Object.keys(obj), '| stored count =',
Object.keys(obj).length);
```

SOLUTION

measured output of node keybug.js

TEXT

```
same object    -> rishabh
equal copy     -> undefined
object keys    -> [ '[object Object]' ] | stored count = 1
by value key   -> rishabh
frozen write   -> A
```

Two separate disasters here.

Disaster 1 — Map matches objects by identity. m.get({row: 'A', number: 12}) builds a brand new object. It has the same contents but it is a different object, so the Map says undefined. Every request that rebuilds the seat from JSON gets a miss, and the seat looks free when it is booked.

Disaster 2 — a plain object stringifies its keys. obj[seatA] becomes obj["[object Object]"]. So does obj[{row: 'B', number: 7}]. Two different seats collide onto one key and the second silently overwrites the first. One entry survives out of two.

The fix: give the value object one string identity, and key on that.

```
class Seat {
  constructor(row, number) { this.row = row; this.number = number; }
  Object.freeze(this); }
  get key() { return `${this.row}#${this.number}`; }
  equals(o) { return o instanceof Seat && o.key === this.key; }
}

const booked = new Map();
booked.set(new Seat('A', 12).key, 'rishabh');
console.log('by value key ->', booked.get(new Seat('A', 12).key));
```

What the interviewer expects to hear about Java. Java solves this with the equals / hashCode contract. If you override equals, you must override hashCode, and two objects that are equals must return the same hashCode. A HashMap first finds the bucket by hashCode, then compares with equals. Override only equals and the map looks in the wrong bucket, so it never finds the entry. And if you **mutate** a key after putting it in the map, its hash changes, the entry stays in the old bucket, and it becomes unreachable for ever — the map reports size 1 while every lookup returns null.

TRAP

The universal rule, in every language: **a key must be immutable**. Freeze the value object. If you cannot freeze it, copy it before using it as a key.

Answer: undefined for the equal copy, and 1 surviving key '[object Object]'. Key on a derived string; freeze the value object.

Example 17 [Infosys · pattern]

Replace a status switch with behaviour on the type. Orders have statuses NEW, PACKED, SHIPPED, DELIVERED, CANCELLED. Four different functions each switch on the status. Show the design that removes all four switches.

SOLUTION

Put the behaviour **next to** the status, not in the callers.

```
const STATUS = {
  NEW: { next: ['PACKED', 'CANCELLED'], canRefund: true, label: 'Order placed' },
  PACKED: { next: ['SHIPPED', 'CANCELLED'], canRefund: true, label: 'Packed' },
  SHIPPED: { next: ['DELIVERED'], canRefund: false, label: 'On the way' },
  DELIVERED: { next: [], canRefund: false, label: 'Delivered' },
  CANCELLED: { next: [], canRefund: false, label: 'Cancelled' },
};

class Order {
  #status = 'NEW';
  get status() { return this.#status; }
  get label() { return STATUS[this.#status].label; }
  get canRefund() { return STATUS[this.#status].canRefund; }
  moveTo(next) {
    if (!STATUS[this.#status].next.includes(next))
      throw new Error(`illegal move ${this.#status} -> ${next}`);
  }
}
```

```

    this.#status = next;
  }
}

```

Now `moveTo`, `label` and `canRefund` all read the same table. Adding a status `RETURNED` means adding one row and listing it in `DELIVERED.next`. No caller changes.

Count the saving. Suppose the status appears in 4 switches, each with 5 branches. Adding a status by hand means $4 \times 1 = 4$ edits, in 4 files, each of which must be found first. With the table it is 1 edit in 1 file. And a forgotten branch in a switch fails silently (falls through to `default`); a missing table row throws immediately.

Answer: One STATUS table holding next, canRefund, label. Four switches become four table reads.

Tier 1 design — the food-court bill

This is a full design, walked through all seven steps. Keep the steps in this order in a real interview; it is the order the interviewer is taking notes in.

Example 18 [TCS NQT · pattern]

Design the classes for a food-court bill. Items with tax, one or more discounts, and an equal split between friends. No database.

SOLUTION

Step 1 — Clarify. Ask these four, and know why each changes the design.

Question	Why it changes the design
Is tax per item or per bill?	Per item means tax lives on <code>LineItem</code> ; per bill means one number on <code>Bill</code> . Food courts tax per item (5% food, 18% packaged), so it goes on the item.
Can discounts stack?	If yes I need a list of rules and an order of application, not one field.
Is discount applied before or after tax?	Different totals. I will apply it after tax on the subtotal, and say so.
Do splits have to be exact to the paisa?	Yes — otherwise the total collected does not match the bill. This forces an explicit “who gets the leftover paisa” rule.

Step 2 — Scale estimate. A single food court, 40 counters, lunch rush 12:00 to 14:00. Say 900 bills in that 2-hour window.

$900 \text{ bills} \div (2 \times 3600 \text{ s}) = 900 \div 7200 = 0.125 \text{ bills per second.}$

Each bill has at most 20 items. So the peak work is $0.125 \times 20 = 2.5$ line-item computations per second. This is **nothing**. The design can be as object-heavy as it likes; there is no performance question here at all. Saying this out loud — “the scale is 0.125 writes per second, so I will optimise purely for clarity” — is worth marks, because it shows you check before you optimise.

Step 3 — API surface. For an LLD problem the “API” is the public methods.

Call	Arguments	Returns
<code>bill.add(item)</code>	a <code>LineItem</code>	the bill, so calls chain
<code>bill.apply(rule)</code>	any object with <code>offPaise(base, items)</code>	the bill
<code>bill.subtotal</code>	—	integer paise before tax
<code>bill.tax</code>	—	integer paise of tax
<code>bill.discount</code>	—	integer paise, never more than the subtotal
<code>bill.payable</code>	—	<code>subtotal + tax - discount</code>

<code>bill.splitEqually(n)</code>	<code>n</code> payers	an array of <code>n</code> integers that sums to payable
-----------------------------------	-----------------------	--

Step 4 — Data model. No database, so this is the field list.

Class	Fields	Key / invariant
LineItem	name, unitPaise, qty, taxPercent	frozen; subtotal and tax are derived, never stored
Bill	billId, private #items, private #discounts	key is billId; #items is private so nobody can push a raw object in
DiscountRule	—	any object with <code>offPaise(base, items)</code>

Step 5 — Diagram. See the class diagram earlier in this chapter: Bill owns 1..* LineItem, uses 0..* DiscountRule, and PercentOff / FlatOff / MemberOff are each a kind of DiscountRule.

Step 6 — Deep dive: the leftover paise. This is the genuinely hard part, and interviewers do push on it. $41920 \div 3 = 13973.33\dots$, which is not a whole number of paise. If every share is rounded the same way the collected total will not equal the bill.

- Round each share up: $\lceil 13973.33 \rceil = 13974$, and $13974 \times 3 = 41922$. You collect 2 paise too much.
- Round each share down: $\lfloor 13973.33 \rfloor = 13973$, and $13973 \times 3 = 41919$. You are 1 paise short.
- **The fix:** give everyone the floor, then hand the remainder to payer 1. $41920 - 13973 \times 3 = 41920 - 41919 = 1$, so payer 1 pays $13973 + 1 = 13974$. The shares are $[13974, 13973, 13973]$ and they sum to exactly 41920. ✓

bill.js — the whole design, runnable

JS

```
class LineItem {
  constructor(name, unitPaise, qty, taxPercent) {
    Object.assign(this, { name, unitPaise, qty, taxPercent });
    Object.freeze(this);
  }
  get subtotal() { return this.unitPaise * this.qty; }
  get tax()      { return Math.round(this.subtotal * this.taxPercent / 100); }
}

class Bill {
  #items = [];
  #discounts = [];
  constructor(billId) { this.billId = billId; }
  add(item)  { this.#items.push(item); return this; }
  apply(rule) { this.#discounts.push(rule); return this; }

  get subtotal() { return this.#items.reduce((s, i) => s + i.subtotal, 0); }
  get tax()      { return this.#items.reduce((s, i) => s + i.tax, 0); }
  get discount() {
    const base = this.subtotal;
    return Math.min(this.#discounts.reduce((s, r) => s + r.offPaise(base,
this.#items), 0), base);
  }
  get payable() { return this.subtotal + this.tax - this.discount; }

  splitEqually(n) {
    const each = Math.floor(this.payable / n);
    const shares = new Array(n).fill(each);
    shares[0] += this.payable - each * n;           // the leftover paise go to
payer 1
  }
}
```

```

    return shares;
  }
  print() {
    for (const i of this.#items)
      console.log(` ${i.name.padEnd(12)} ${i.qty} x ${i.unitPaise} =
${String(i.subtotal).padStart(6)} tax ${i.tax}`);
      console.log(' subtotal', this.subtotal, '| tax', this.tax, '| discount',
this.discount,
        '| PAYABLE', this.payable);
    }
  }
}

const FlatOff = (p) => ({ offPaise: () => p });
const PercentOff = (pct, capPaise) => ({ offPaise: (base) =>
Math.min(Math.round(base * pct / 100), capPaise) });

const bill = new Bill('B-1')
  .add(new LineItem('Dosa', 12000, 2, 5))
  .add(new LineItem('Filter Cof', 4000, 3, 5))
  .add(new LineItem('Ice Cream', 9000, 1, 18))
  .apply(PercentOff(10, 5000))
  .apply(FlatOff(2000));

bill.print();
console.log('split 3 ways:', bill.splitEqually(3), 'sum =',
  bill.splitEqually(3).reduce((a, b) => a + b, 0));

```

measured output of node bill.js

TEXT

```

Dosa          2 x 12000 = 24000 tax 1200
Filter Cof    3 x 4000 = 12000 tax 600
Ice Cream     1 x 9000 = 9000 tax 1620
subtotal 45000 | tax 3420 | discount 6500 | PAYABLE 41920
split 3 ways: [ 13974, 13973, 13973 ] sum = 41920

```

Every number checked by hand:

- Dosa: $12000 \times 2 = 24000$; tax = $\text{round}(24000 \times 5 \div 100) = 1200$.
- Coffee: $4000 \times 3 = 12000$; tax = $\text{round}(12000 \times 5 \div 100) = 600$.
- Ice cream: $9000 \times 1 = 9000$; tax = $\text{round}(9000 \times 18 \div 100) = 1620$.
- Subtotal = $24000 + 12000 + 9000 = 45000$. Tax = $1200 + 600 + 1620 = 3420$.
- PercentOff(10, 5000): $45000 \times 10 \div 100 = 4500$, and $\min(4500, 5000) = 4500$.
- FlatOff(2000): 2000. Total discount = $4500 + 2000 = 6500$, and $6500 < 45000$ so no clamp.
- Payable = $45000 + 3420 - 6500 = 41920$ paise = Rs 419.20. ✓

Step 7 — Trade-offs, failure modes, and 10x.

Choice	The other option	Decision and why
Discounts as a list of rule objects	one discountPaise field on the bill	List. The clarify step said discounts stack. One field cannot hold two campaigns, and the moment it has to, the field becomes a switch.
Derived getters (subtotal computed each time)	store the totals as fields	Derive. At 0.125 bills per second re-computation is free, and a stored total can go stale the instant an item is

		added. Store totals only after the bill is paid and frozen.
Discount applied after tax	before tax	After tax , and printed on the bill. Either is defensible; what loses marks is not saying which one you chose.
Leftover paise to payer 1	spread them round-robin	Payer 1. It is one line and always correct. Round-robin is fairer but needs state about who paid last — not worth it for a maximum of $n - 1$ paise.

Failure modes.

1. A discount larger than the bill. Guarded: `Math.min(..., base)` clamps it, so `payable` can never go negative.
2. A rule that throws. Today one bad rule kills the whole bill. At 10x I would wrap each `offPaise` call in a `try/catch`, count it as zero, and log the rule name.
3. `qty` of 0 or a negative price. Not guarded yet — add the checks to the `LineItem` constructor so a bad item can never exist.

At 10x (a chain of 400 food courts, central billing). The class design does not change; the wiring does. `Bill` becomes a domain object saved through a `BillStore` port, the discount rules move into a registry loaded from config, and `billId` gets a prefix per outlet so two outlets can never mint the same id.

Answer: `LineItem` (frozen, derives its own tax) + `Bill` (owns items, uses a list of rules) + rule objects. Leftover paise go to payer 1 so the shares sum exactly.

Tier 2 — the same OOP inside a real service

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

At Tier 2 the classes are the same. What changes is that the object now runs a few thousand times a second, several teams touch it, and the cost of a bad boundary is measured in incidents instead of in ugliness.

Example 19 [Grab · pattern]

Design the pricing module inside a food-delivery service. Base fare, distance fee, surge, packing, platform fee, and promotions, for an app with 4 million daily active users. Walk all seven steps.

SOLUTION

Step 1 — Clarify.

Question	Why it changes the design
Is the price shown on the list screen the same price that is charged?	If yes, I must freeze the quote and store it; if no, I need a re-price step at checkout and a rule for what to do when it moves.
How often do the fee rules change?	Daily means the rules must be config, not code. Quarterly means code is fine.
Who owns surge — this team or a pricing team?	If another team owns it, surge must be a port with a timeout and a fallback value, not an in-process calculation.
Must the same order re-price to the same number if the request is retried?	Yes. That forces a quote id and an idempotency rule, which is step 6.

Step 2 — Scale estimate. Show the arithmetic.

- 4,000,000 daily active users.
- Each opens the app 3 times a day: $4\{, \}000\{, \}000 \times 3 = 12\{, \}000\{, \}000$ sessions/day.

- Each session shows a list of 20 restaurant cards, and every card shows a delivery fee: $12\{, \}000\{, \}000 \times 20 = 240\{, \}000\{, \}000$ price calculations per day.
- Average: $240\{, \}000\{, \}000 \div 86\{, \}400 = 2\{, \}778$ price calculations per second.
- Peak is about 3x the average (lunch and dinner): $2\{, \}778 \times 3 = 8\{, \}334$ per second.

Now the important second calculation — **does the object design cost anything at that rate?** Measured on this machine:

bench.js — the real cost of a polymorphic call

JS

```
const N = 10_000_000;
function hardCoded(meters, seconds) { // no polymorphism at all
  return 3000 + Math.floor(meters * 12 / 1000) * 100 + seconds * 2;
}
class BikeFare { pause(m, s) { return 3000 + Math.floor(m * 12 / 1000) * 100 + s * 2; } }
class CarFare { pause(m, s) { return 5000 + Math.floor(m * 18 / 1000) * 100 + s * 3; } }
const pool = [new BikeFare(), new CarFare()];

let sink = 0;
let t0 = process.hrtime.bigint();
for (let i = 0; i < N; i++) sink += hardCoded(i % 9000, i % 700);
let t1 = process.hrtime.bigint();
for (let i = 0; i < N; i++) sink += pool[i & 1].pause(i % 9000, i % 700);
let t2 = process.hrtime.bigint();

const ms = (a, b) => Number(b - a) / 1e6;
console.log('direct call      :', ms(t0, t1).toFixed(0), 'ms for', N, 'calls');
console.log('method on object:', ms(t1, t2).toFixed(0), 'ms for', N, 'calls');
console.log('ns per method call =', (Number(t2 - t1) / N).toFixed(1));
```

measured output of node bench.js

TEXT

```
direct call      : 64 ms for 10000000 calls
method on object: 69 ms for 10000000 calls
ns per method call = 6.9
(sink = 206210975000 )
```

A polymorphic call costs about 7 nanoseconds. Put that against the day's work:

$240\{, \}000\{, \}000 \times 7 \text{ ns} = 1\{, \}680\{, \}000\{, \}000 \text{ ns} = 1.68 \text{ seconds of CPU for the whole day.}$

And against one database round trip of 2 ms: $2 \text{ ms} = 2\{, \}000\{, \}000 \text{ ns}$, and $2\{, \}000\{, \}000 \div 7 \approx 285\{, \}714$. **One** database call costs as much as 285,714 polymorphic calls.

SHORTCUT

Memorise this comparison. When an interviewer asks “isn't all this indirection slow?”, the answer is: “a virtual call is about 7 ns; one database round trip is 2 ms, which is 285,000 of them. I will spend the 7 ns and save the round trip.” That is a numbers answer, not an opinion.

Step 3 — API surface.

the two endpoints

TEXT

```
POST /v1/quotes
{ "userId":"u-77", "restaurantId":"r-12", "dropLat":12.97, "dropLng":77.59,
```

```

"items":[{"sku":"s-1","qty":2}], "promoCode":"FIRST3" }
->201 { "quoteId":"q-8f3a", "basePaise":24000, "lines":[
    {"type":"DISTANCE","paise":4000}, {"type":"SURGE","paise":12000},
    {"type":"PACKING","paise":600}, {"type":"PLATFORM","paise":500},
    {"type":"PROMO","paise":-5000}],
    "totalPaise":36100, "expiresAt":"2026-09-15T12:05:00Z" }

POST /v1/orders
{ "quoteId":"q-8f3a", "idempotencyKey":"u-77:cart-3:1757937600" }
->201 { "orderId":"o-55231", "chargedPaise":36100 }
-> 409 if the quote has expired, with a fresh quote in the body

```

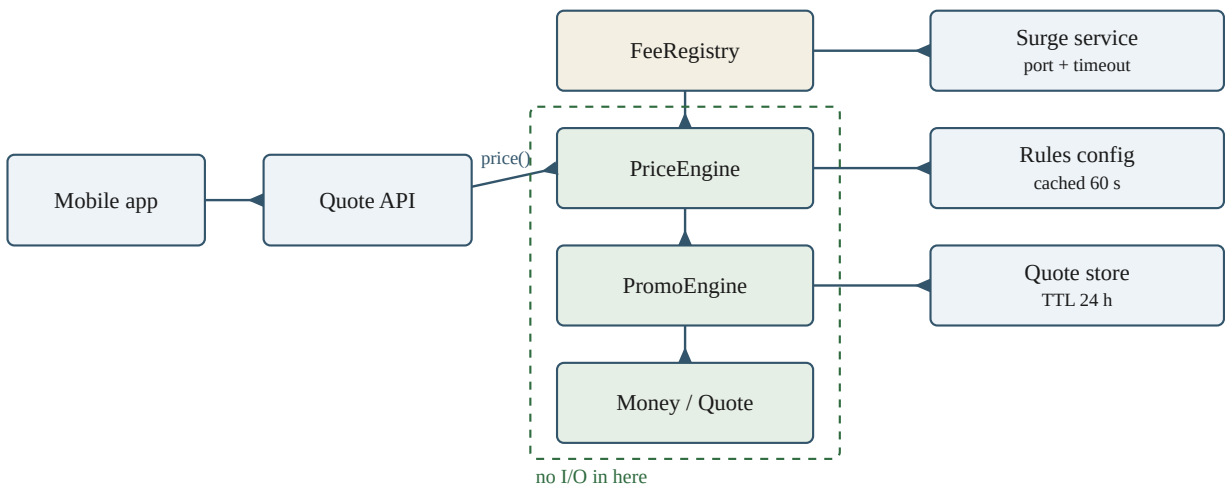
Two things to point at while drawing this. The quote returns **every line**, not just a total, because a customer who cannot see why they are paying Rs 361 will call support. And the quote has an `expiresAt`, which is what makes surge safe to show.

Step 4 — Data model.

Table	Key	Fields	Why
quotes	quote_id	user_id, restaurant_id, lines_json, total_paise, created_at, expires_at	written once, read once, then dead. Give it a 24-hour TTL.
fee_rules	(rule_type, city_id, version)	params_json, active_from, active_to	versioned, so an old quote can be explained months later
orders	order_id	quote_id, idempotency_key (unique), charged_paise, status	the unique index on idempotency_key is what stops a double charge

Sizing the quotes table: only about 1 quote in 20 becomes an order, and quotes are written at checkout, not per card. Say 2,000,000 quotes/day at 400 bytes each: $2\{\text{, } \}000\{\text{, } \}000 \times 400 = 800\{\text{, } \}000\{\text{, } \}000$ bytes = 0.8 GB/day. With a 24-hour TTL the table never grows past about 0.8 GB. Without the TTL it would be $0.8 \times 365 = 292$ GB a year for data nobody reads.

Step 5 — Architecture diagram.



The pricing module. Everything inside the dashed area is pure computation with no I/O — that is what makes it fast and testable.

Step 6 — Deep dive 1: strategy objects in code, or fee rules in a config table?

This is the real design question and the interviewer will push on it.

	Rules as code objects	Rules as rows in a table
--	-----------------------	--------------------------

New fee type	a deploy	an insert, live in 60 s
Change a number	a deploy	an update, live in 60 s
Can be unit tested	yes, fully	only the interpreter can be tested
A bad rule	caught by tests and types	reaches production instantly
Reading the code later	the rule is right there	you must query the database to know what runs

Decision: both, split by what actually changes. The **shape** of a fee (how distance fee is computed from kilometres) is code — a small class registered in `FeeRegistry`. The **numbers** inside it (paise per km, the cap, the city it applies to) are rows in `fee_rules`, cached for 60 seconds. This is the split that matches reality: finance changes numbers weekly and engineering adds fee **types** about twice a year. Putting numbers in code means a deploy per price change; putting shapes in the database means a home-made programming language inside your own tables, which is the worst outcome of the two.

registry.js — shapes in code, numbers injected

JS

```
class FeeRegistry {
  #byType = new Map();
  register(type, calculator) {
    if (this.#byType.has(type)) throw new Error(`duplicate fee type: ${type}`);
    for (const m of ['name', 'paise']) {
      if (typeof calculator[m] !== 'function')
        throw new Error(`${type} is missing ${m}()`); // fail at boot, not
    }
    this.#byType.set(type, calculator);
    return this;
  }
  quote(type, order) {
    const c = this.#byType.get(type);
    if (!c) throw new Error(`no fee calculator for ${type}`);
    return { type, name: c.name(), paise: c.paise(order) };
  }
  get types() { return [...this.#byType.keys()]; }
}

const reg = new FeeRegistry()
  .register('PLATFORM', { name: () => 'Platform fee', paise: () => 500 })
  .register('DISTANCE', { name: () => 'Distance fee', paise: (o) =>
    Math.ceil(o.km) * 800 })
  .register('SURGE', { name: () => 'Surge', paise: (o) =>
    Math.round(o.base * (o.surge - 1)) })
  .register('PACKING', { name: () => 'Packing', paise: (o) => o.items *
    200 });

const order = { base: 24000, km: 4.3, items: 3, surge: 1.5 };
let total = order.base;
for (const t of reg.types) {
  const q = reg.quote(t, order);
  total += q.paise;
  console.log(q.type.padEnd(9), q.name.padEnd(14), String(q.paise).padStart(6));
}
console.log('TOTAL'.padEnd(24), String(total).padStart(6), 'paise = Rs', (total /
100).toFixed(2));

try { reg.register('SURGE', { name: () => 'x', paise: () => 0 }); }
```

```
catch (e) { console.log('boot check:', e.message); }
try { reg.register('TIP', { name: () => 'Tip' }); }
catch (e) { console.log('boot check:', e.message); }
```

measured output of node registry.js

TEXT

```
PLATFORM Platform fee      500
DISTANCE Distance fee     4000
SURGE     Surge           12000
PACKING   Packing         600
base      24000
TOTAL     41100 paise = Rs 411.00
boot check: duplicate fee type: SURGE
boot check: TIP is missing paise()
```

Every line checked: distance = $\lceil 4.3 \rceil \times 800 = 5 \times 800 = 4000$. Surge = $\text{round}(24000 \times (1.5 - 1)) = 12000$. Packing = $3 \times 200 = 600$. Total = $24000 + 500 + 4000 + 12000 + 600 = 41100$ paise. ✓

The two boot check lines are the part worth pointing at. The registry refuses a duplicate type and refuses an incomplete calculator **at start-up**. That is how you get interface safety in a language with no interfaces: check the shape once, at boot, and crash loudly rather than at 2 a.m. under load.

Step 6 — Deep dive 2: the same request priced twice.

The app retries on a timeout. Surge moved between the two tries. Now the customer sees Rs 361 and is charged Rs 395.

The fix has two halves and both are object design:

1. **Freeze the quote.** PriceEngine returns an immutable Quote with a quoteId and an expiresAt. Checkout charges quote.totalPaise, never a recomputation. Price can only move when the customer asks for a new quote.
2. **Make the order idempotent.** POST /v1/orders carries an idempotencyKey built by the client from userId + cartId + minute. A unique index on that column means the second attempt fails the insert, and the handler returns the **existing** order instead of a new one. The database, not the application, is what guarantees this.

What if the quote has expired? Return 409 with a fresh quote in the body, and let the app show “the price changed, confirm again”. Silently charging the new price is the design that ends up on social media.

Step 7 — Trade-offs, failure modes, and 10x.

Choice	The other option	Decision
Fee shapes in code, numbers in config	everything in a rules table	Split. Numbers change weekly, shapes twice a year. A rules engine in the database is an unversioned language nobody can test.
Quote frozen with a TTL	re-price at checkout	Freeze, 5-minute TTL. Shown price must equal charged price. 5 minutes is long enough to check out and short enough that surge is not badly stale.
Surge fetched through a port with a 50 ms timeout and a fallback of 1.0	call it inline and fail the quote	Port with fallback. A pricing page that returns an error because a secondary service is slow loses far more money than one that under-charges surge for a minute.
Promotions computed in-process	a separate promo service call	In-process for now. At 8,334 quotes/s a second network hop adds 2 ms and a new failure mode. Revisit when promo rules need their own release cycle.

Failure modes.

1. **Surge service down.** Fallback 1.0, log it, raise an alarm on the fallback rate. Under-charging for ten minutes is survivable; a broken menu page is not.
2. **Config cache poisoned with a bad number.** Guard every fee with a sanity clamp (“no single fee above Rs 500”) and a boot-time schema check.
3. **Clock skew between the app and the server.** Always compare `expiresAt` on the server, never trust the client’s clock.
4. **A fee calculator throws.** Catch per fee, treat it as zero, tag the quote degraded: `true`, and alarm. A missing packing fee is better than a blank screen.

At 10x — 40 million DAU. Redo the arithmetic: $40\{, \}000\{, \}000 \times 3 \times 20 = 2\{, \}400\{, \}000\{, \}000$ calculations/day = $2\{, \}400\{, \}000\{, \}000 \div 86\{, \}400 = 27\{, \}778/s$ average, $\times 3 = 83\{, \}334/s$ at peak. At 7 ns each that is still only 16.8 seconds of CPU across the whole day, so the **objects** are still not the problem. What breaks first is the config fetch and the surge call, so at 10x I would push the rules into each instance’s memory at boot with a push-based invalidation instead of a 60-second poll, and batch the surge lookup for all 20 cards into one call instead of 20.

Answer: PriceEngine over a FeeRegistry of small fee objects; shapes in code, numbers in versioned config; a frozen Quote plus a unique idempotency key stops double pricing and double charging.

Example 20 [Shopee · pattern]

The concurrency bug that survives in a single-threaded language. Node runs your JavaScript on one thread, so shared state is safe. True or false? Prove it.

SOLUTION

False. One thread does not mean one request at a time. Every `await` is a place where the thread leaves your function and runs somebody else’s code.

```
race.js JS

const sleep = (ms) => new Promise((r) => setTimeout(r, ms));

class SeatMapBuggy {
  constructor() { this.free = 1; }
  async book(who) {
    const free = this.free;           // 1. read
    await sleep(5);                  // 2. a DB call -> another request runs
    HERE
    if (free > 0) { this.free = free - 1; return `${who}: booked`; }
    return `${who}: sold out`;
  }
}

class SeatMapFixed {
  constructor() { this.free = 1; this.lock = Promise.resolve(); }
  book(who) {
    // queue the whole read-modify-write
    const run = this.lock.then(async () => {
      const free = this.free;
      await sleep(5);
      if (free > 0) { this.free = free - 1; return `${who}: booked`; }
      return `${who}: sold out`;
    });
    this.lock = run.then(() => {}, () => {});
    return run;
  }
}
```

```
(async () => {
  const bad = new SeatMapBuggy();
  console.log(await Promise.all([bad.book('rishabh'), bad.book('aisha')]));
  const good = new SeatMapFixed();
  console.log(await Promise.all([good.book('rishabh'), good.book('aisha')]));
})();
```

measured output of node race.js

TEXT

```
[ 'rishabh: booked', 'aisha: booked' ] seats left = 0
[ 'rishabh: booked', 'aisha: sold out' ] seats left = 0
```

One seat. Two winners. Trace it:

Time	rishabh	aisha
t0	reads <code>free = 1</code>	—
t0	hits <code>await</code> , pauses	starts, reads <code>free = 1</code>
t0	—	hits <code>await</code> , pauses
t5	wakes, sees its own <code>free = 1</code> , books	—
t5	—	wakes, sees its own <code>free = 1</code> , books

The read and the write are separated by an `await`. Everything between them is a window.

The object-design lesson. A domain object that holds **mutable** state and also does I/O in the middle of a state change is unsafe in every language, threads or not. Two ways out:

1. **Make the object immutable** and let the database decide. `UPDATE seats SET free = free - 1 WHERE id = ? AND free > 0` returns 0 rows for the loser. One statement, no window.
2. **Serialise the whole read-modify-write**, as `SeatMapFixed` does with a promise chain.

Trade-off, and the decision. The promise-chain lock is 4 lines and needs no database feature, but it only protects **one process** — start a second Node instance and both can book the same seat. The conditional `UPDATE` is one line of SQL and is correct across every instance, but it puts the rule in the database where your unit tests cannot see it. **Decision: the conditional UPDATE is the real answer** for anything that is shared across instances — which a seat map always is. Use the in-process lock only for state that genuinely belongs to one process, such as a local rate-limit counter.

Answer: False. `await` opens a window between read and write. Fix with a conditional UPDATE in the database, not with an in-process lock.

Example 21 [Agoda · pattern]

Nine teams, one enum. A `BookingType` enum has 9 values. Somewhere in the codebase it is switch-ed on in 14 places, across 9 services. A tenth booking type is coming. Count the cost of the current design, then fix it.

SOLUTION

Count first. A new enum value that must behave correctly in every switch means visiting 14 switch sites. But the switches are spread across 9 services, so the real unit of work is a deploy per service:

- switch sites to edit: 14
- services to release, test and roll out: 9
- if each of the 9 services has its own review and deploy cycle of half a day: $9 \times 0.5 = 4.5$ engineer-days for one enum value, before any feature work.

Worse, a switch with no default that throws will **silently** fall through for the new value. You find out in production.

The fix has two halves.

1. **Put the behaviour with the data.** Anything that is a pure function of the booking type — the label, whether it is refundable, the cancellation window — moves into a single table that all services read. Then there is one place to add a row, not 14.
2. **Make the remaining switches loud.** Where a service genuinely must branch, the default case throws:

a switch that cannot fail silently

JS

```
function cancellationHours(type) {  
  const table = {  
    HOTEL: 24, FLIGHT: 4, BUS: 2, TRAIN: 4, CAR: 1,  
    ACTIVITY: 48, CRUISE: 72, PACKAGE: 72, INSURANCE: 0,  
  };  
  const hours = table[type];  
  if (hours === undefined) throw new Error(`unknown booking type: ${type}`);  
  return hours;  
}
```

Now a tenth type causes a loud, immediate, findable failure in every service that has not been updated — on the first request, in staging, with the type name in the message.

Trade-off, and the decision. A shared table means 9 services now depend on one schema; a bad row breaks everybody at once, and you have created a coupling point. Nine private copies mean no shared blast radius but guaranteed drift — three services will disagree about the cancellation window within a year. **Decision: share it,** but ship it as a **versioned** config artefact (a package or a signed config blob with a version number) rather than a live shared table, so a bad row can be rolled back per service instead of taking all nine down at once.

TRAP

“Just add a default: break” is the wrong instinct. A default that does nothing turns a crash into wrong data, and wrong data is discovered by a customer, weeks later, in money. Throw.

Answer: 14 switch sites across 9 services $\approx$ 4.5 engineer-days per enum value. Move behaviour into one versioned table; make every remaining default throw.

Tier 3 — OOP that survives a distributed system

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

At Tier 3 your objects stop living in one process. They are serialised, sent over a network, retried, stored for seven years, and read back by a service that was deployed eight months after yours. Every OOP rule you learned still applies, but the cost of breaking one is now counted in shards and outages.

Example 22 [Amazon · pattern]

Design the domain model for a payments ledger that must never lose or duplicate an entry, at 90 million transactions a day. Walk all seven steps, and be explicit about what the **objects** have to guarantee.

SOLUTION

Step 1 — Clarify.

Question	Why it changes the design
----------	---------------------------

Is a balance a stored number or a sum of entries?	This is the modelling decision. Stored number is fast to read and impossible to audit; sum of entries is auditable and slow to read. It decides everything downstream.
How long must entries be kept?	7 years for financial records changes storage from “a table” to “a table plus cold archive”, and changes the yearly storage arithmetic.
Can two services write the same account at the same time?	Yes. That forces an optimistic-lock version or a single-writer-per-account rule.
Is a partial transfer ever acceptable?	No. Debit and credit must land together, which forces the double-entry model below.

Step 2 — Scale estimate.

- 90,000,000 transactions/day.
- Average: $90\{, \}000\{, \}000 \div 86\{, \}400 = 1\{, \}042$ transactions per second.
- Peak 3x (salary day, sale day): $1\{, \}042 \times 3 = 3\{, \}126$ per second.
- **Double entry means every transaction writes 2 rows** (one debit, one credit): $90\{, \}000\{, \}000 \times 2 = 180\{, \}000\{, \}000$ rows/day, so $1\{, \}042 \times 2 = 2\{, \}084$ row-writes/s average and $6\{, \}252$ /s at peak.
- Row size about 200 bytes: $90\{, \}000\{, \}000 \times 200 = 18\{, \}000\{, \}000\{, \}000$ bytes = 18 GB/day (counting a transaction as one logical record of 200 B).
- Per year: $18 \text{ GB} \times 365 = 6\{, \}570 \text{ GB} = 6.57 \text{ TB/year}$.
- With 3 replicas: $6.57 \times 3 = 19.7 \text{ TB/year}$ of real disk.
- Over the 7-year retention: $6.57 \times 7 = 46 \text{ TB}$ of logical data, 138 TB with replicas.

That last number is the one that decides the design: 46 TB does not sit on one machine, so the account id must be a shard key from day one, and shard choice must be a property of the **domain object**, not an afterthought in the data layer.

Step 3 — API surface.

the write path	TEXT
<pre>POST /v1/transfers Idempotency-Key: 5f2c-...-9a { "from":"acc-1001", "to":"acc-2938", "paise": 250000, "reference":"rent-sep" } -> 201 { "transferId":"t-77c1", "status":"POSTED", "postedAt":"2026-09-15T09:12:03Z" } -> 200 (same body) if this Idempotency-Key was already posted -> 409 { "error":"INSUFFICIENT_FUNDS", "availablePaise": 180000 } GET /v1/accounts/acc-1001/balance -> 200 { "accountId":"acc-1001", "balancePaise": 4512000, "asOfEntryId": 918273645 }</pre>	

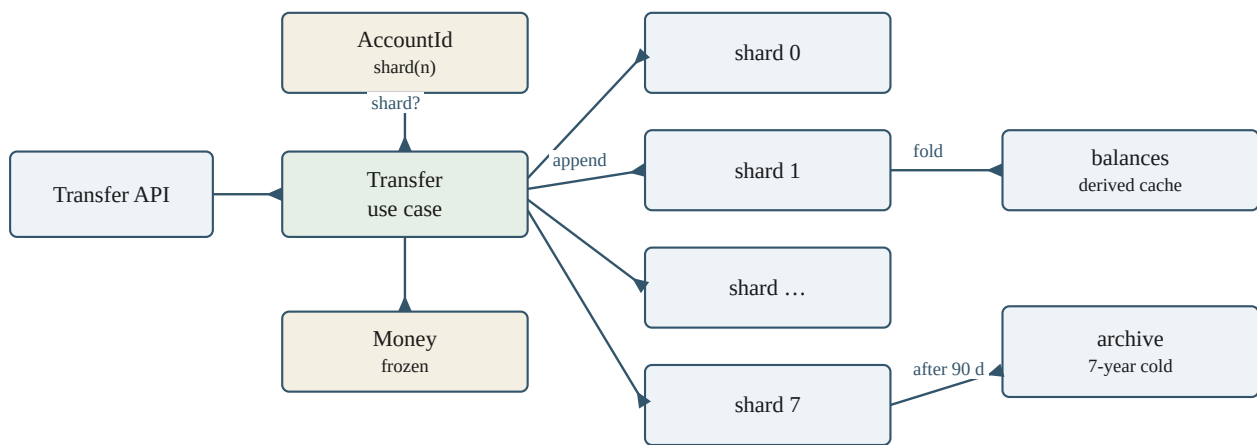
Note asOfEntryId on the balance. It turns “here is a number” into “here is a number and the exact point in the ledger it is true at”, which is what makes a balance checkable.

Step 4 — Data model.

Table	Key	Fields	Notes
entries	(account_id, entry_id)	transfer_id, direction (D/C), paise, created_at	append only. Never updated, never deleted. Sharded by account_id.
transfers	transfer_id	idempotency_key (unique), from_account, to_account, paise, status	the unique index is the anti-duplicate guarantee

balances	account_id	balance_paise, as_of_entry_id, version	a cache of the sum, not the truth
----------	------------	---	--

Step 5 — Architecture diagram.



Entries are the truth; balances are a derived cache. The account id chooses the shard, and it does so inside the domain object.

Step 6 — Deep dive 1: the shard key must live in the domain object.

If the shard is computed in the data layer, two services will compute it differently the day someone “improves” the hash, and half the entries for an account go to the wrong shard. Make it a method on the identifier, and make the hash one that is identical in every language and every version.

shard.js — a hash that never changes its mind

JS

```

function fnv1a(str) { // 32-bit FNV-1a: same answer in every
  language
  let h = 0x811c9dc5;
  for (let i = 0; i < str.length; i++) {
    h ^= str.charCodeAt(i);
    h = Math.imul(h, 0x01000193) >>> 0;
  }
  return h >>> 0;
}

class AccountId {
  constructor(value) { this.value = value; Object.freeze(this); }
  get shardKey() { return this.value; } // the ONLY thing hashed. Never
  the object.
  shard(n) { return fnv1a(this.shardKey) % n; }
}

const counts = new Array(8).fill(0);
for (let i = 0; i < 100000; i++) counts[new AccountId('acc-' + i).shard(8)]++;
console.log('spread over 8 shards:', counts.join(' '));
console.log('ideal per shard      :', 100000 / 8);

```

measured output of node shard.js

TEXT

```

acc-1001 hash = 1105070759 -> shard 7 of 8
acc-1002 hash = 1121848378 -> shard 2 of 8

```

```
acc-2938 hash = 1673079079 -> shard 7 of 8
acc-7777 hash = 1684264593 -> shard 1 of 8
acc-8123 hash = 1733756585 -> shard 1 of 8
spread over 8 shards: 12497 12498 12501 12497 12504 12501 12498 12504
ideal per shard      : 12500
```

100,000 keys over 8 shards should give $100,000 \div 8 = 12,500$ each. The measured spread is 12,497 to 12,504 — the worst shard is $(12,504 - 12,500) \div 12,500 = 0.00032 = 0.032\%$ above ideal. That is flat enough to ignore.

TRAP

Never shard on a language’s built-in hash of an object. In Java, `Objects.hash(...)` is computed from the fields and can differ between JVM versions and is **not** specified to be stable forever; in Python, string hashing is randomised per process by default, so the same key gives a different shard after a restart. Write the hash yourself, in the domain object, from an explicit string. FNV-1a and MurmurHash3 are both fine; what matters is that it is **your** code and it never changes.

Step 6 — Deep dive 2: balance as stored number, or as a sum of entries?

	Stored balance (mutate a row)	Sum of entries (append only)
Read a balance	1 row read, about 1 ms	sum of up to millions of rows — unusable without help
Write	UPDATE with a version check	INSERT only — no locks, no contention
Audit “why is it 45,120?”	impossible; the history is gone	replay the entries and see
A bug that double-applies	balance is permanently wrong	duplicate entry is visible and reversible
Concurrent writers	lock contention on a hot account	none: inserts never conflict

Decision: entries are the truth, balance is a derived cache. Keep entries append-only, and keep a balances row that stores the running total **plus** `as_of_entry_id`. Reads go to `balances` (1 ms). A nightly job recomputes from the entries and alarms on any mismatch, which turns a silent corruption into a ticket. This costs one extra table and one extra job, and it buys the one thing a ledger cannot live without: the ability to prove a number.

Why not pure sum-on-read? Do the arithmetic. A three-year-old salary account at 30 transactions a month has $30 \times 36 = 1,080$ entries — summing that on every read is fine. But a merchant settlement account at 90 transactions per **second** accumulates $90 \times 86,400 = 7,776,000$ entries a day. Summing 7.8 million rows for one balance read is not a design, it is an outage. The cache is not an optimisation; it is required.

Step 6 — Deep dive 3: the object that crosses the wire.

Your Transfer object is serialised, queued, retried, and read by a consumer running last month’s code. Three rules follow, and all three are Liskov in disguise — the new version must be usable everywhere the old one was:

1. **Only add optional fields.** A new **required** field breaks every old consumer at once. Count it: 9 consuming services, each needing a coordinated release, is a 9-way deploy with an ordering constraint — the classic distributed-systems trap.
2. **Never reuse a field name with a new meaning.** `amount` meaning rupees in v1 and paise in v2 is a 100x bug that no type system will catch.
3. **Never remove a field until the metrics say nobody reads it.** Ship the removal in two releases: stop writing, wait, then delete.

Step 7 — Trade-offs, failure modes, and 10x.

Choice	The other option	Decision
Append-only entries + cached balance	mutate a balance row	Append-only. A ledger that cannot be audited is not a ledger. The extra table

		and the nightly reconciliation job are the price.
Shard key inside <code>AccountId</code>	shard chosen in the repository	Inside the domain object. One definition, one file, testable, and impossible for two services to disagree about.
Hand-written FNV-1a	the language's built-in hash	Hand-written. Built-in hashes are not promised to be stable across versions or processes, and a shard map must be stable for seven years.
Idempotency enforced by a unique index	checked in application code	Unique index. Application checks have a window between read and write; a unique index has none. Let the database be the one that says no.
Both legs of a transfer in one transaction when the accounts share a shard	always use a distributed transaction	Same-shard transaction when possible, a saga otherwise. Most transfers are within one shard; paying the cost of a two-phase commit for all of them to handle the minority is the wrong trade.

Failure modes.

1. **Debit written, credit lost.** Both legs go in one database transaction when the accounts share a shard. When they do not, write a PENDING transfer first, then the two legs, then mark POSTED; a sweeper finishes or reverses anything stuck in PENDING for more than 30 seconds.
2. **A retried request that already succeeded.** The unique index on `idempotency_key` rejects it; the handler returns 200 with the original transfer.
3. **Balance cache drifts from the entries.** The nightly fold recomputes and alarms. Because entries are immutable, the truth is always recoverable.
4. **A hot account.** One merchant taking 90 transfers a second lands entirely on one shard. Fix by splitting the merchant into sub-accounts ($m-77/0 \dots m-77/9$) that hash apart and sum together. This is a **domain** fix, not an infrastructure one.

At 10x — 900 million transactions a day. $900\{, \}000\{, \}000 \div 86\{, \}400 = 10\{, \}417/s$ average, $\times 3 = 31\{, \}251/s$ peak, and $18\text{ GB} \times 10 = 180\text{ GB/day} = 65.7\text{ TB/year}$ of logical data. Two things change. First, 8 shards become 64 or more, so the shard count must never be baked into the id — store a **shard map** and route through it, or the day you resize, every account moves. Second, the balance fold moves from a nightly batch to a stream consumer, so `as_of_entry_id` is seconds behind rather than hours.

Answer: Append-only entries as truth, balances as a derived cache with `as_of_entry_id`, `AccountId.shard()` using a hand-written FNV-1a, and a unique idempotency index. 6.57 TB/year logical, 19.7 TB with 3 replicas.

Example 23 [Google · pattern]

Where does validation live? An `Email` value object validates in its constructor, so an invalid email cannot exist. A reviewer says: “our service reads 40 million rows a night from a legacy table, 0.3% of them have broken emails, and now the whole batch dies.” Who is right?

SOLUTION

Both are, about different things. The mistake is treating one rule as universal.

Count the damage first. $40\{, \}000\{, \}000 \times 0.003 = 120\{, \}000$ bad rows. If a bad row throws and the batch stops, the batch never finishes. If a bad row is silently accepted, 120,000 customers get no mail and nobody notices for a month.

The resolution is a boundary rule, not a class rule.

Where	What to do with bad input	Why
-------	---------------------------	-----

At the edge — an API request a human just made	reject with 400 and a clear message	the human can fix it now, and a bad value must never enter the system
At an import from a system you do not control	parse, do not throw: return either a valid Email or a reason	you cannot fix legacy data by refusing to read it
Inside the domain, after the edge	assume valid; no re-checks	if the edge did its job, re-validating everywhere is noise that hides the real rules

one class, two doors

JS

```
class Email {
  constructor(value) {
    // strict door: throws
    if (!Email.isValid(value)) throw new Error(`invalid email: ${value}`);
    this.value = value.toLowerCase();
    Object.freeze(this);
  }
  static isValid(v) { return typeof v === 'string' && /^[^@\\s]+@[^@\\s]+.[^@\\s]+$/i.test(v); }
  static parse(v) {
    // tolerant door: never throws
    return Email.isValid(v) ? { ok: true, email: new Email(v) }
      : { ok: false, reason: 'malformed', raw: v };
  }
}
```

The batch uses `Email.parse` and sends the 120,000 failures to a quarantine table with the reason. The API uses `new Email(...)` and returns 400. The invariant “an Email object is always valid” holds in both paths, which is the whole point of the value object.

Trade-off, and the decision. A single strict constructor is simpler and gives the strongest guarantee, but it makes the class unusable at any boundary where bad data is a fact of life. Two doors is one extra method and a rule people must remember. **Decision: two doors**, with a naming convention enforced in review — `new X(...)` throws, `X.parse(...)` returns a result. The quarantine table is not optional: data you drop without recording is data you cannot fix.

Answer: Both. Strict constructor at the edge, a tolerant `parse()` at imports, no re-validation inside. Quarantine the 120,000 bad rows with a reason.

Example 24 [Uber · pattern]

Inheritance across a network boundary. A base class `Vehicle` has `etaSeconds()`. `Bike`, `Car` and `Auto` override it. A new `PartnerCar` gets its ETA from a partner’s HTTP API. What breaks, and what do you do?

SOLUTION

What breaks. Every caller of `etaSeconds()` was written believing it is a cheap, in-memory, always-succeeds, microsecond call. `PartnerCar` makes it a network call. Measure the gap: an in-memory ETA is about 0.001 ms; a partner HTTP call is about 120 ms. $120 \div 0.001 = 120\{, \}000$ times slower. And it can fail, time out, or return nonsense.

This is a Liskov break of the loudest kind. The method signature is identical, so the compiler is happy, and the production behaviour is unrecognisable. Concretely, a list screen that ranks 30 nearby vehicles used to take $30 \times 0.001 = 0.03$ ms and now takes $30 \times 120 = 3\{, \}600$ ms if the calls are sequential. The screen times out.

The fix, in three moves.

1. **Make the cost visible in the type.** Rename the contract so a slow call cannot hide behind a fast one: `etaSeconds()` stays synchronous and in-memory; anything that can block becomes `async fetchEta()`. Different name, different shape, no surprise.

2. **Give the remote one an interface of its own.** PartnerCar implements RemoteEtaSource, which promises a timeout and a fallback. It is not a Vehicle subclass at all — it is a Vehicle **plus** a port.
3. **Batch and cache at the call site.** One call for 30 vehicles instead of 30 calls, with the result cached for 15 seconds. 3{, }600 ms becomes about 120 ms.

the shape that keeps the promise

JS

```
class Vehicle {
  etaSeconds(distanceM) { return Math.round(distanceM / this.speedMps()); } //
  always fast
  speedMps() { throw new Error('subclass must implement speedMps()'); }
}
class Bike extends Vehicle { speedMps() { return 6.5; } }
class Car extends Vehicle { speedMps() { return 9.0; } }

class PartnerEtaSource {
  // a PORT, not a subclass
  constructor(http, timeoutMs = 150, fallback = 900) {
    this.http = http; this.timeoutMs = timeoutMs; this.fallback = fallback;
  }
  async etaFor(vehicleIds) {
    // batched
    try { return await this.http.postWithTimeout('/eta', { vehicleIds },
this.timeoutMs); }
    catch { return new Map(vehicleIds.map((id) => [id, this.fallback])); }
  }
}
```

Trade-off, and the decision. Keeping PartnerCar extends Vehicle means one uniform list and no caller changes — very tempting, and it is why this bug is so common. Splitting it into a port means the ranking code must now handle two sources and merge them, which is real extra work. **Decision: split it.** A method whose cost varies by five orders of magnitude between subclasses is not one method. The uniformity you keep by leaving it alone is fake uniformity, and it is paid for with a timeout on the busiest screen in the product.

THE RULE FOR INHERITANCE ACROSS A BOUNDARY

A subclass may change **what** is computed. It must not change:

1. the **order of magnitude** of the cost (a microsecond method must not become a network call),
2. whether it can **fail** (an always-succeeds method must not start throwing),
3. whether it can **block** (a synchronous method must not start waiting on I/O).

Change any of those three and you need a new contract, not an override.

Answer: The override turns a 0.001 ms call into a 120 ms one — a 120,000x change the compiler cannot see. Make the remote source a port with a timeout, batch it, and cache it.

Interview drill

These are the follow-ups that come after your first answer. Each one has a short, decided reply. Learn the shape, not the words.

The interviewer pushes	The answer that scores
“Isn’t all this indirection slow?”	“A method call through an object is about 7 ns — I measured it. One database round trip is 2 ms, which is 285,000 of them. The indirection is free; the round trip is the cost.”
“You have 6 classes for what could be one function.”	“Agreed for one function. I split at the point where two different teams file tickets on the same file. Here the tax rule and the

	layout are owned by finance and design, so they get one class each. I would not split further.”
“Why not just use inheritance here?”	“There are three independent axes — format, storage, compression — and $3 \times 2 \times 2 = 12$ combinations. Inheritance needs 12 classes; composition needs 7 and one new class per new option. One axis, I would inherit.”
“Where do you validate?”	“At the edge, strictly, with a <code>400</code> . Inside the domain, not at all. At a legacy import, with a tolerant <code>parse()</code> that quarantines bad rows instead of killing the batch.”
“Singleton — good or bad?”	“Bad as a global that classes reach for themselves, because it hides a dependency and makes tests share state. Fine as one instance, injected once at start-up . The lifetime is fine; the global lookup is the problem.” (Chapter 4 works this through.)
“Your value object is immutable — isn’t that wasteful?”	“A Money object is two fields, about 40 bytes. At 8,334 quotes a second with 5 fee lines each, that is $8\{, \}334 \times 5 \times 40 \approx 1.7$ MB/s of short-lived garbage, which a generational collector handles in its cheapest path. I will pay that to make a whole class of aliasing bugs impossible.”
“What if two threads touch this object?”	“Then it must be immutable, or the state change must be one atomic operation. In Node, remember that <code>await</code> opens the same window a thread does — I showed a seat being sold twice on one thread.”
“How do you test this?”	“The rules class takes no database, so a test builds it with in-memory doubles and runs in under a millisecond. 600 such tests take 0.24 s instead of 4 minutes, which is the difference between running them on every save and running them never.”
“Add a new fee type. What changes?”	“One new class and one <code>register(...)</code> line. No existing file is edited, and the registry rejects a duplicate type or a missing method at boot.”
“Why is the shard key in the domain object?”	“So there is exactly one definition. If the data layer computes it, two services will disagree the day someone changes the hash, and entries for one account land on two shards. It is a hand-written FNV-1a because built-in hashes are not promised to be stable across versions.”
“What would you do differently at 10x?”	“Redo the arithmetic out loud first. $40 \text{ M} \times 3 \times 20 = 2.4$ billion calls a day $= 27\{, \}778/\text{s}$, $83\{, \}334/\text{s}$ at peak. The objects are still 16.8 seconds of CPU across the day, so they are not the problem — the config poll and the per-card surge call are. I would push config at boot and batch surge into one call.”
“You used a <code>switch</code> . Why?”	“Because it has 3 branches and is read in one screen. The rule I apply is: past about 4 branches, or any branch that a different team owns, it becomes a registry. This one is neither yet.”

Practice

PRACTICE • TIER 1 target: 35 min

1. A Student class has `name`, `rollNo`, `marks[]`, `printReportCard()`, `saveToFile()`, `emailParents()`. List the reasons to change, then split it. How many classes, and what is each one’s single job?

2. Write a `DateRange` value object with `start` and `end` (both day numbers). It must reject `end < start` in the constructor, be frozen, expose `days`, and have an `overlaps(other)` method. Show `overlaps` returning `true` and `false` on two examples, with the arithmetic.
3. This is claimed to be polymorphism. It is not. Say why, then fix it.

```
function area(s) { if (s.kind === 'circle') return 3.14159 * s.r * s.r; return s.w * s.h; }
```
4. A parking system has `Vehicle` with `wheels()` and `feePerHour()`. `Bicycle` returns `feePerHour() = 0`, and the billing code has `if (v instanceof Bicycle) skipBilling()`. Which SOLID letter is broken, and what is the fix?
5. Count the classes: a notification can be Email, SMS or Push; in English, Hindi or Thai; urgent or normal. With inheritance, how many subclasses? With composition, how many classes? Show both multiplications.
6. Write the `STATUS` table for a support ticket with `OPEN`, `WAITING`, `RESOLVED`, `CLOSED`, `REOPENED`, giving legal next states for each. Then say what `moveTo('CLOSED')` does from `OPEN`, and why.

PRACTICE · TIER 2 target: 40 min

1. A quiz app has 6,000,000 DAU. Each user attempts 4 quizzes a day, each quiz has 10 questions, and each question is scored server-side. Compute questions scored per day, average per second, and peak at 4x. Then, using 7 ns per polymorphic call, compute the total CPU seconds per day spent on scoring dispatch.
2. Your `Order` service exposes `order.total` as a stored field. A reviewer wants it computed from the lines every time. Give the read rate at which the stored field starts to win, assuming a line sum costs 0.2 μ s and orders average 6 lines. Then decide, and say what guards the stored field against going stale.
3. A `PaymentMethod` interface has `charge()`, `refund()`, `tokenize()`, `verify0tp()`, `scheduleRecurring()`. UPI cannot do `scheduleRecurring()`, and a gift card cannot do `refund()`. Redesign the contracts and say exactly what a caller asks before calling.
4. You inject 4 dependencies into a use case today. Six months later it has 11. Name the smell, and give the two fixes, with the rule for choosing between them.
5. Write the promise-chain lock from this chapter as a reusable `Mutex` class with `runExclusive(fn)`. Then say in one sentence why you would still not use it for a seat map across 6 service instances.

PRACTICE · TIER 3 target: 45 min

1. A social app stores 400,000,000 likes a day. Each like row is 64 bytes. Compute GB/day, TB/year, and TB/year with 3 replicas. Then decide: store likes as rows, or as a counter per post? Name both sides and choose, with the read rate that decides it.
2. Design `PostId` so that it carries its own shard. Requirements: stable for 10 years, computable in Node and in Python with the same answer, and even across 64 shards. Say which hash and why not the built-in one.
3. Your `OrderPlaced` event has a field `amount` in rupees. You must move to paise. Nine services consume it. Write the migration as a sequence of releases where no release breaks a consumer, and say how you know it is safe to delete the old field.
4. A base class method `price()` is overridden by a subclass to call a partner API. Give three separate things this breaks for existing callers, with an order-of-magnitude number for each.
5. A merchant account takes 90 transfers a second and all of them hash to one shard. Give a domain-level fix (not an infrastructure one), and compute the new per-shard rate if the fix splits the merchant 10 ways.

ANSWER KEY

Tier 1.

1. Four reasons to change: marks rule, report layout, file format, email template. Four classes: `Student` (data + marks), `ReportCard` (layout), `StudentStore` (file), `ParentNotifier` (email).
2. `DateRange(10, 14)` has `days = 14 - 10 + 1 = 5`. `overlaps` is `start <= o.end && o.start <= end`. `(10,14) vs (12,20): 10 ≤ 20 and 12 ≤ 14, both true → true`. `(10,14) vs (15,20): 10 ≤ 20 true, but 15 ≤ 14 false → false`.
3. It is a `switch` wearing a costume — the call site must be edited for every new shape. Fix: `class Circle { area() { ... } }, class Rect { area() { ... } }`, and the caller says `s.area()`.
4. `L`, and also `O`. The `instanceof` check in the billing code proves the subclass is not usable like the others. Fix: `feePerHour()` of 0 is fine, and billing must handle a zero fee normally — delete the `instanceof` branch entirely.
5. Inheritance: $3 \times 3 \times 2 = 18$ subclasses. Composition: $3 + 3 + 2 = 8$ classes. A fourth channel costs 6 subclasses or 1 class.
6. `OPEN -> [WAITING, RESOLVED]`, `WAITING -> [OPEN, RESOLVED]`, `RESOLVED -> [CLOSED, REOPENED]`, `CLOSED -> []`, `REOPENED -> [WAITING, RESOLVED]`. `moveTo('CLOSED')` from `OPEN` throws, because a ticket must be resolved before it is closed — otherwise “closed” hides unresolved work.

Tier 2.

1. $6\{, \}000\{, \}000 \times 4 \times 10 = 240\{, \}000\{, \}000$ scored/day. $240\{, \}000\{, \}000 \div 86\{, \}400 = 2\{, \}778/s$ average; $\times 4 = 11\{, \}112/s$ peak. CPU: $240\{, \}000\{, \}000 \times 7 \text{ ns} = 1.68$ seconds per day.
2. Computing costs $6 \times 0.2 = 1.2 \mu\text{s}$ per read, so even at 100,000 reads/s it is $100\{, \}000 \times 1.2 \mu\text{s} = 0.12 \text{ s}$ of CPU per second — 12% of one core. **Decision: compute it**, until the order is **paid**; then freeze the total onto the order, because at that moment it becomes a historical fact and must never change even if a price does. The guard against staleness is that nothing can edit a paid order’s lines.
3. Split into `Chargeable` (`charge`), `Refundable` (`refund`), `Tokenizable` (`tokenize`), `Recurring` (`scheduleRecurring`). The caller asks `typeof m.refund === 'function'` (or checks a declared `capabilities` set) before offering a refund button — and the capability list is what the UI reads, so an unsupported button never renders.
4. The smell is a use case doing too much (SRP), with the constructor as the evidence. Fix A: split the use case into two, each with its own dependencies. Fix B: bundle related dependencies behind one port (a `Ledger` port instead of `accounts`, `entries`, `balances`). Choose A if the 11 split cleanly into two disjoint groups; choose B if 4 of them always travel together.
5. `class Mutex { #tail = Promise.resolve(); runExclusive(fn) { const r = this.#tail.then(fn); this.#tail = r.then(() => {}, () => {}); return r; } }`. It guards one process only — with 6 instances, 6 mutexes each let one booking through, so 6 people can buy the same seat.

Tier 3.

1. $400\{, \}000\{, \}000 \times 64 = 25\{, \}600\{, \}000$ bytes = 25.6 GB/day. $\times 365 = 9\{, \}344 \text{ GB} = 9.34 \text{ TB/year}$; $\times 3 = 28.0 \text{ TB/year}$ with replicas. **Decision: store rows and keep a counter as a derived cache** — the same pattern as the ledger. Rows answer “did I like this?” and allow un-liking exactly once; the counter answers “how many?” in one read. Pure counters cannot answer the first question, and pure rows cannot answer the second at feed-render rates (a post with 2,000,000 likes cannot be counted per render).
2. `class PostId { get shardKey() { return this.value; } shard(n) { return fnv1a(this.value) % n; } }`. Use FNV-1a or MurmurHash3 written out in your own code. Not the built-in: Python randomises string hashing per process, so the shard changes after every restart, and Java’s `Objects.hash` is not promised to be stable across versions. Evenness was measured at 0.032% from ideal over 100,000 keys.
3. Release 1: add `amountPaise` and write **both** fields; consumers still read `amount`. Release 2: each consumer switches to `amountPaise` at its own pace; add a metric counting reads of `amount`. Release 3: when the metric has been zero for two weeks, stop writing `amount`. Release 4: delete the field from the schema. You know it is safe when the read metric — not a code search — has been zero across a full billing cycle.
4. (i) **Latency**: 0.001 ms becomes about 120 ms, roughly 120,000x. A 30-item list page goes from 0.03 ms to 3,600 ms if called in a loop. (ii) **Failure**: a method that could never throw now throws on timeout, so every call site needs error handling it does not have. (iii) **Blocking**: a synchronous method becomes asynchronous, so either the caller changes shape or the event loop stalls. Any one of these needs a new contract.
5. Split the merchant into sub-accounts `m-77/0 ... m-77/9`, chosen round-robin at write time and summed at read time. The hash of each sub-account id is different, so the writes spread. New rate per shard: $90 \div 10 = 9$ transfers per second. Reading the merchant balance becomes 10 reads that are summed — an acceptable price, and they can run in parallel.

ONE-PAGE REVISION CARD

The five layers of an LLD answer. Caller → Service → Domain → Port → Adapter. The domain layer holds the rules, holds no connection, and runs in a test in under a millisecond. The adapter must never reach back into the domain.

The four pillars and their one test.

Encapsulation	Can an outsider create an illegal state? If yes, it is broken.
Abstraction	Can you replace the whole inside and leave callers untouched?
Inheritance	Does the child keep every promise the parent made?
Polymorphism	Can you add a behaviour without touching the call site?

SOLID by smell — what you actually say out loud.

S	the class name has “and” in it, or two teams edit the same file
O	an if/switch chain that grows with the business
L	an override that throws, does nothing, or forces <code>instanceof</code> at the call site
I	an implementation that has to write “not supported”
D	the rules file imports a database driver

The Liskov checklist. Same or wider inputs · same or narrower outputs · no new exceptions · every state promise kept · caller never needs to know the subclass. Also, across a boundary: no change in **order of magnitude of cost**, no new **failure**, no new **blocking**.

Inheritance or composition? Count the axes of variation. One axis → inheritance is fine. Two or more that combine freely → composition, always, because the axes **multiply**: $3 \times 2 \times 2 = 12$ subclasses versus $3 + 2 + 2 = 7$ classes.

Numbers to memorise.

Polymorphic method call	about 7 ns (measured: 69 ms for 10,000,000 calls)
One database round trip	about 2 ms = 285,000 polymorphic calls
Test with in-memory doubles	about 0.4 ms ; with a real database about 400 ms — 1000x
A small value object	about 40 bytes
Seconds in a day	86,400 — every QPS estimate starts here
Peak multiplier	3x the average unless the problem says otherwise

The estimation drill, every time. $\text{DAU} \times \text{actions/user} = \text{actions/day} \div 86,400 = \text{average/s} \times 3 = \text{peak/s}$. Then $\text{rows/day} \times \text{bytes/row} = \text{bytes/day} \times 365 = \text{bytes/year} \times \text{replicas}$. Worked once: $4 \text{ M} \times 3 \times 20 = 240 \text{ M/day} \div 86,400 = 2,778/\text{s} \times 3 = 8,334/\text{s}$ peak.

Value object rules. No identity · frozen in the constructor · equality by value through a key getter · validates itself so an invalid one cannot exist. Money is **integer paise**, never a float — $0.1 + 0.2 = 0.30000000000000004$.

Keys must be immutable. A mutated key is unreachable for ever. In JavaScript a Map keyed by an object matches by **identity**, and a plain object stringifies every key to `[object Object]`. Key on a derived string. In Java: override `equals` and `hashCode` together, and never mutate a key in a `HashMap`.

Distributed extras. The shard key lives in the domain object, computed by a hash **you wrote** (FNV-1a), because built-in hashes are not stable across versions or processes. Events may only **add optional** fields. Idempotency is enforced by a unique index, never by an application check.

What loses marks. “It depends” with no choice. Reciting the five SOLID names instead of pointing at a smell. Optimising before doing the arithmetic. Splitting a class because it got long rather than because two teams edit it. Not saying what you would do at 10x.

Design Patterns That Get Asked

Six patterns answer nine out of ten LLD questions. Learn those six properly.

The idea in one page

A design pattern is not clever code. It is a **name for a shape** that keeps appearing. Knowing the name is worth something in an interview for one reason only: it lets you say in four words what would otherwise take four minutes.

There is exactly one question behind every pattern in this chapter:

THE ONLY QUESTION A PATTERN ANSWERS

“What varies here, and how do I let it vary without editing code that already works?”

Find the thing that changes. Put it behind a small interface. Inject it from outside. That single move is Strategy, Factory, Decorator, State and Observer wearing different hats.

The failure it prevents has a name too: the **type-code if-chain**. Every time you see

```
if (kind === "A") ... else if (kind === "B") ... else if (kind === "C") ...
```

somebody will add kind “D” next month by editing that function. That function has no reason to know about D. A pattern moves D into its own file.

Symptom to pattern — the table to memorise

What you see in the problem	Pattern	The one-line fix
several ways to do the same calculation	Strategy	inject an object with one method
behaviour changes as the object’s status changes	State	one class per status; the status handles the call
“create the right kind of X from a string”	Factory	a registry from key to constructor
optional behaviours that combine freely	Decorator	wrappers that all share one interface
one event, many uninterested-in-each-other reactions	Observer	subject publishes, listeners subscribe
exactly one of something, shared	Singleton	module-level instance — and usually inject it instead
a constructor with 8 arguments, half optional	Builder	chained setters, validate in <code>build()</code>
a third-party API whose shape is wrong for you	Adapter	a thin class that translates both ways
undo, redo, queue-the-action, retry-the-action	Command	each action is an object with <code>do/undo</code>
a request that several handlers might claim	Chain of Responsibility	linked handlers, first match wins
the same algorithm, one step different	Template Method	base class calls an abstract hook
control access, add caching or laziness in front	Proxy	same interface, stands in for the real thing

SHORTCUT

The pattern-naming move that scores. Do not say “I will use the Strategy pattern.” Say what varies first, then the name:

“The fare rule changes per city and will keep changing, so I will put it behind a `quote(ride)` interface and inject it — that is the Strategy pattern. Adding Chennai becomes adding a class, not editing `FareCalculator`.”

Varies → interface → injected → **then** the name. The name last. The name alone sounds memorised; the reasoning sounds like experience.

TRAP

Pattern-itis is a real rejection reason. A candidate who uses six patterns in a 40-minute design has built something nobody can read. Two patterns is the right number for one LLD problem: one for the thing that varies most, one for the structure. If you reach for a third, ask yourself whether a plain function would do. Usually it would.

What the interviewer expects to hear — Java and C++ vocabulary

Most pattern questions at service companies are **phrased** in Java or C++ vocabulary, because the classic catalogue was written in those languages. Answer in both.

THE TRANSLATION TABLE — SAY THE JAVA WORD, SHOW THE JS REALITY

Java / C++ word	What it means	The JavaScript reality
interface	a contract with no code	no such keyword; a base class whose methods <code>throw</code> , or a duck-typed object
abstract class	partly implemented, cannot be instantiated	a class whose base methods <code>throw new Error("implement me")</code>
implements vs extends	contract vs inheritance	only <code>extends</code>
method overloading	same name, different parameter types	does not exist; one function, check the arguments
private field	visible only inside the class	<code>#field</code> — real since ES2022

The sentence that satisfies a Java interviewer: “In Java I would write `interface FareStrategy { long quote(Ride r); }` and `FlatFare implements FareStrategy`. JavaScript has no interface keyword, so the contract is a base class whose methods `throw`. The design is identical; only the compiler’s help is missing.”

Do not apologise for JavaScript and do not pretend it has interfaces. State the concept, the Java form, and the JS form. That shows you understand the **idea** rather than one language’s syntax, which is what the question tests.

Warm-up

WARM-UP

build the reflex

Example 1 [warm-up]

Name the pattern and say what varies.

```
class Invoice {
  total(items, country) {
    if (country === "IN") return this.#gst(items);
    else if (country === "SG") return this.#gst9(items);
    else if (country === "TH") return this.#vat(items);
  }
}
```

SOLUTION

What varies: the tax rule, once per country.

The smell: a type-code if-chain. Adding Malaysia means editing Invoice, a class that has nothing to do with tax law. That breaks the open/closed rule — open to extension, closed to modification.

The pattern: Strategy.

```
class Invoice {  
    constructor(taxRule) { this.taxRule = taxRule; } // injected  
    total(items) { return this.taxRule.apply(items); }  
}
```

```
const inv = new Invoice(new IndiaGst());
```

Adding Malaysia is now class MalaysiaSst { apply(items) { ... } } — a new file, and Invoice is never opened again.

Answer: Strategy. The tax rule varies.

Example 2 [warm-up]

Strategy and State look identical — both are “an object with a method, held by a context”. What is the actual difference?

SOLUTION

Who chooses the next object.

	Strategy	State
who picks it	the caller , from outside	the current state , from inside
how often it changes	rarely; usually once at construction	constantly, as a result of calls
do the objects know each other?	no — FlatFare never mentions SlabFare	yes — Assigned creates Resolved
what it models	a choice	a lifecycle

Strategy: “price this ride with the slab rule.” You chose. The rule does not decide to become a different rule.

State: “the ticket is Assigned; calling resolve() makes it Resolved.” Nobody outside chose that. Assigned.resolve() chose it.

One sentence for the interview: “Same structure, opposite direction of control. Strategy is chosen from outside and stays; State changes itself from inside.”

Answer: Strategy is picked by the caller and stays; State replaces itself from inside.

Example 3 [warm-up]

A coffee costs a base price, and a customer may add: extra shot, oat milk, syrup, whipped cream, and a large size. Someone proposes a subclass for each combination. How many classes is that? Give the formula and the fix.

SOLUTION

Each option is independently on or off. With 5 independent options:

$$2^5 = 32 \text{ subclasses}$$

CoffeeWithExtraShotAndOatMilk, CoffeeWithExtraShotAndSyrup, and so on. Add a sixth option next month and it is $2^6 = 64$.

The general formula: n independent options $\rightarrow 2^n$ subclasses.

$$n = 3 \rightarrow 8 \quad n = 5 \rightarrow 32 \quad n = 10 \rightarrow 1,024$$

The fix: Decorator. n options need n wrapper classes, not 2^n subclasses.

$$10 \text{ options: } 1,024 \text{ subclasses} \rightarrow 10 \text{ decorators}$$

```
const drink = new Large(new Whip(new Syrup(new Coffee())));
```

Each wrapper adds exactly one behaviour and calls the one inside it.

Answer: 32 subclasses (2⁵). Decorator needs 5 classes instead.

Example 4 [warm-up]

Which pattern is `[3, 1, 2].sort((a, b) => a - b)` an example of? Name it and explain in one line.

SOLUTION

Strategy. `sort` owns the algorithm — comparisons, swapping, partitioning. It does not own **what “smaller” means**. That is the part that varies, so it is passed in.

`sort` is the context. The comparator is the strategy. A different one can be passed on every call and `sort` never changes.

The wider point: in JavaScript most strategies are just functions, because a one-method interface and a function are the same thing. Use a class when the strategy carries configuration (`new SlabFare(5000, 2, 1200)`); use a plain function when it does not.

Answer: Strategy. The comparison rule varies; the sorting algorithm does not.

Example 5 [warm-up]

Why is Singleton often called an anti-pattern? Give two concrete problems, and say when it is still correct.

SOLUTION

Problem 1 — it is a global variable in a costume. Any code anywhere can call `Config.get()`, so you cannot tell from a signature what a function touches. Change the singleton in one test and the next test sees the change, so tests must run in order, which they never reliably do.

Problem 2 — “one instance” is a lie in production. `static #instance` means one per **process**. Forty pods give forty instances:

$$40 \text{ pods} \times 1 \text{ "singleton" scheduler} = 40 \text{ schedulers}$$

If that singleton runs a nightly job for 2 million users:

$$40 \times 2,000,000 = 80,000,000 \text{ emails instead of } 2,000,000$$

When it is still correct. When a second instance would be a bug or a waste: a database **connection pool**, a metrics registry, a read-only config object loaded once at boot.

The interview answer: “Yes for a connection pool, because a second pool is a resource leak. No for anything with business logic, because it hides dependencies and it does not mean what people think across processes — one-per-cluster needs a lock or a leader election, not a static field.”

Answer: It is a global (untestable), and it means one per process, not one per cluster. Correct for pools and registries.

Example 6 [warm-up]

Factory and Builder both “create objects”. When do you use each?

SOLUTION

Factory answers “which class?” You have a string or an enum and you need the matching type.

```
ChannelFactory.create("sms") // -> SmsChannel
```

Builder answers “how do I assemble one object with many optional parts?” There is only ever one class; the difficulty is the ten constructor arguments.

```
new PizzaBuilder().size("large").add("paneer").extraCheese().build()
```

	Factory	Builder
the problem	many possible types	one type, many optional fields

the input	a key or a rule	a sequence of choices
returns	a different class each time	always the same class
validation	“is this a known kind?”	“is this combination legal?” in <code>build()</code>

They combine happily: a factory picks the builder, or a builder’s `build()` calls a factory for one of its parts. That is not a contradiction; they answer different questions.

Answer: Factory = which class. Builder = how to assemble one class with many options.

Example 7 [warm-up]

In the Observer pattern, what is the difference between **push** and **pull**, and which do you choose?

SOLUTION

Push: the subject sends the data with the notification.

```
bus.emit("order.placed", { id: 91, userId: "u1", amountPaise: 45000 });
```

Pull: the subject sends only “something changed”, and each listener fetches what it needs.

```
bus.emit("order.changed", { id: 91 }); // listener then calls orders.get(91)
```

	Push	Pull
wins	no extra round trip; the listener works even if the row is later deleted	small messages; each listener takes only what it needs
loses	big messages; every listener gets fields it does not want; the payload becomes a contract you cannot change	N listeners means N reads of the same row, all at the same instant

Decision: push the identifiers plus the few fields everybody needs; pull the rest. Here that means pushing { `orderId`, `userId`, `status`, `amountPaise`, `at` } — roughly 100 bytes — and letting the warehouse service fetch the 40-line item list itself. You avoid a fat contract and you avoid six services hammering one row.

Answer: Push the ids plus the common fields; pull the heavy details.

Example 8 [warm-up]

Here is a `Report` class. Name every pattern that is **missing**, then say which single one you would add first.

```
class Report {
  build(type, user) {
    let rows = db.query("SELECT ...");
    if (type === "pdf") { /* 40 lines */ }
    else if (type === "csv") { /* 30 lines */ }
    else if (type === "xlsx") { /* 50 lines */ }
    log("built " + type); metrics.count("report");
    return out;
  }
}
```

SOLUTION

Missing patterns:

1. **Strategy or Factory** for the format — the `if`-chain is the obvious smell.
2. **Decorator** for `log` and `metrics` — cross-cutting concerns, repeated in every method of every class.
3. **Template Method** — “fetch rows, render, finish” is the same for every format; only the middle step differs.
4. **Adapter** — if the PDF and XLSX libraries have different APIs, each needs a thin translation class so `Report` sees one shape.

Which one first: the format Strategy. It is the change that is **actually happening** — someone will ask for JSON next week, and today that means a fourth branch in a 120-line method. Logging and metrics are annoying but they are not growing.

| **The rule:** fix the axis that changes most often, not the one with the most famous pattern name.

Answer: Strategy/Factory, Decorator, Template Method, Adapter. Add the format Strategy first, because that is what keeps changing.

Example 9 [warm-up]

A vending machine ignores “select item” until money is in, and ignores “insert coin” while it is dispensing. A candidate writes `if (this.mode == "IDLE") ...` inside every method. What breaks, and what is the fix?

SOLUTION

What breaks — count it. With m methods and s modes, the `if`-checks are spread over $m \times s$ places. Four methods and five modes is 20 checks in 4 functions.

1. Adding a sixth mode means finding and editing all four methods. Miss one and you have a bug that only appears in one transition.
2. The legal transitions are nowhere written down. You have to read all four methods and reconstruct the machine in your head.
3. Two methods will disagree about what “DISPENSING” allows, and nobody will notice for a year.

The fix — State. One class per mode. `Idle.select()` simply refuses; `HasMoney.select()` does the work. Nobody asks “what mode am I in?” because the current state object **is** the answer.

The count changes from $m \times s$ scattered `ifs` to s classes with m methods each — the same number of behaviours, but each one has an address. And the base class refuses everything by default, so a transition you forgot to allow fails loudly instead of silently doing the wrong thing.

Answer: $m \times s$ scattered checks and no written transition table. Fix: one class per state, base class refuses by default.

PRACTICE · WARM-UP target: 12 min

1. Name the pattern: “an object that looks exactly like the real service but caches results and can refuse the call if the caller lacks permission.”
2. Name the pattern: “the same seven-step import job for CSV, XML and JSON; only step 3 differs.”
3. 7 independent optional features. How many subclasses if you use inheritance? How many classes with Decorator?
4. Which pattern lets you implement undo in a text editor, and what does each object store?
5. Give one case where Singleton is the right answer and one where it is not.
6. A method takes (a, b, c, d, e, f) where c, d, e, f are optional. Which pattern, and what does it add that a plain options object does not?

ANSWER KEY

1. Proxy. Same interface, stands in front of the real object, adds access control, caching or laziness.
2. Template Method. The base class owns the seven steps and calls an abstract `parse()` that each subclass fills in.
3. $2^7 = 128$ subclasses, or 7 decorator classes.
4. Command. Each object stores what it did and enough information to reverse it — a `Delete` command stores the position, the length, **and the text it removed**, because you cannot restore text you did not keep.
5. Right: a database connection pool (a second pool doubles the connections for no gain). Wrong: a `PricingService` singleton — it hides a dependency, makes tests order- dependent, and is one-per-process anyway.
6. Builder. Over a plain options object it adds (i) a `build()` step where you can **validate combinations** (“a small pizza cannot have 10 slices”), (ii) required fields enforced before construction, and (iii) an immutable result — the built object has no setters, so nobody mutates it later.

Tier 1 — the six patterns, with runnable code

Every snippet in this section runs as written with node. Type them once. Do not read them.

Pattern 1 — Strategy

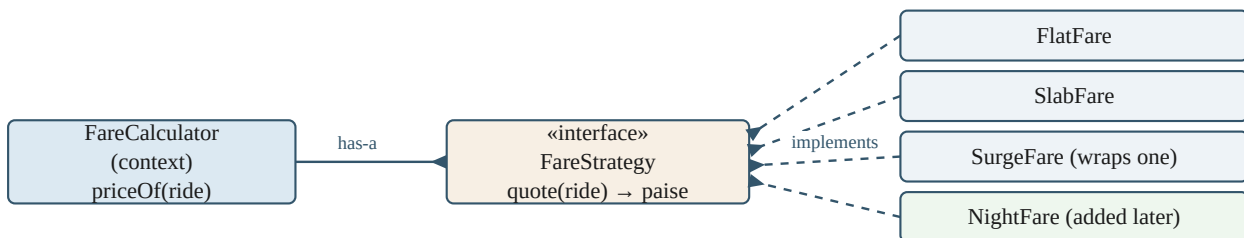
STRATEGY IN FOUR LINES

Problem: several interchangeable ways to do one job, and more will be added.

Structure: a **context** holds a **strategy** object. Every strategy has the same method name. The caller injects the one it wants.

Test that you need it: can you finish the sentence “the ___ rule will change”? If yes, that rule is a strategy.

Test that you do NOT need it: there is exactly one rule and there always will be. Then a plain method is correct and a strategy is ceremony.



Adding the green box changes no existing file. That is the whole point of the pattern.

Strategy. The context on the left never changes. Every new rule is a new box on the right.

strategy.js — three fare rules, one of which wraps another

JS

```
// ---- the plug: every strategy has the same method name and shape ----
class FlatFare {
  constructor(paisePerKm) { this.paisePerKm = paisePerKm; }
  quote(ride) { return ride.km * this.paisePerKm; }
  get name() { return "flat"; }
}

class SlabFare {
  constructor(basePaise, baseKm, perKmAfter) {
    this.basePaise = basePaise; this.baseKm = baseKm; this.perKmAfter = perKmAfter;
  }
  quote(ride) {
    if (ride.km <= this.baseKm) return this.basePaise;
    return this.basePaise + Math.ceil(ride.km - this.baseKm) * this.perKmAfter;
  }
  get name() { return "slab"; }
}

class SurgeFare {
  constructor(inner, multiplierBp) { // bp = basis points, 15000 = 1.5x
    this.inner = inner; this.multiplierBp = multiplierBp;
  }
  quote(ride) {
```

```

    return Math.round(this.inner.quote(ride) * this.multiplierBp / 10000);
  }
  get name() { return `surge(${this.inner.name}, ${this.multiplierBp / 10000}x)`; }
}

// ---- the context: knows nothing about any rule ----
class FareCalculator {
  constructor(strategy) { this.strategy = strategy; }
  setStrategy(s) { this.strategy = s; } // swappable at runtime
  priceOf(ride) {
    const paise = this.strategy.quote(ride);
    return { rule: this.strategy.name, paise, rupees: (paise / 100).toFixed(2) };
  }
}

const ride = { km: 12.4, minutes: 31 };
const calc = new FareCalculator(new FlatFare(1400));
console.log(calc.priceOf(ride));

calc.setStrategy(new SlabFare(5000, 2, 1200));
console.log(calc.priceOf(ride));

calc.setStrategy(new SurgeFare(new SlabFare(5000, 2, 1200), 15000));
console.log(calc.priceOf(ride));

```

node strategy.js

TEXT

```

{ rule: 'flat', paise: 17360, rupees: '173.60' }
{ rule: 'slab', paise: 18200, rupees: '182.00' }
{ rule: 'surge(slab, 1.5x)', paise: 27300, rupees: '273.00' }

```

Example 10 [TCS NQT · pattern]

Check the three numbers above by hand, then say what SurgeFare proves about the pattern.

SOLUTION

Flat. 12.4 km at 1,400 paise per km:

$$12.4 \times 1,400 = 17,360 \text{ paise} = \text{Rs.}173.60$$

Slab. Base Rs.50 covers the first 2 km, then Rs.12 per started kilometre:

$$12.4 - 2 = 10.4 \text{ km beyond the base}$$

$$\lceil 10.4 \rceil = 11 \text{ started kilometres}$$

$$5,000 + 11 \times 1,200 = 5,000 + 13,200 = 18,200 \text{ paise} = \text{Rs.}182.00$$

Surge. 1.5x on the slab price:

$$18,200 \times 15,000 / 10,000 = 18,200 \times 1.5 = 27,300 \text{ paise} = \text{Rs.}273.00$$

What SurgeFare proves. It takes a FareStrategy and **is** a FareStrategy. So it can wrap any rule — flat, slab, or a rule written next year — without knowing which. That is Decorator structure used inside a Strategy family, and it is the reason both patterns are on the same interface.

Because of it, surge is defined **once**. The alternative — FlatSurgeFare, SlabSurgeFare, NightSurgeFare — is the 2ⁿ explosion again.

Answer: Rs.173.60, Rs.182.00, Rs.273.00. SurgeFare is both a strategy and a decorator, so surge is written once.

TRAP

`ride.km * this.paisePerKm` returns a whole number here only because 12.4×1400 happens to be exact in binary floating point. It will not always be. Any strategy that computes money must end in `Math.round(...)` and return an integer number of paise. Money in a double is a rejection in a payments interview, every time.

Pattern 2 — Factory

THREE THINGS CALLED FACTORY — KNOW WHICH ONE THEY MEAN

1. **Simple factory** — one function or static method with a switch. Fine for 3 or 4 fixed types. Still has an if-chain, but it is in **one** place instead of twenty.
2. **Factory method** — a base class declares `create()` and each subclass returns its own product. Used when the creator itself varies.
3. **Abstract factory** — one object that creates a whole **family** of matching products (`WindowsButton` + `WindowsMenu`, or `MySQLConnection` + `MySQLDialect`). Rare in interviews but you should be able to define it.

The version to write in JavaScript: a registry. A Map from key to a builder function. Adding a kind is `register(...)` in the new file — the factory class is never edited.

factory.js — the registry factory, which nobody ever edits

JS

```
class Channel {
  send(msg) { throw new Error("subclass must implement send()"); }
}
class SmsChannel extends Channel {
  constructor(gateway) { super(); this.gateway = gateway; }
  send(m) { return { via: "sms", to: m.phone, gateway: this.gateway, cost: 12 }; }
}
class EmailChannel extends Channel {
  send(m) { return { via: "email", to: m.email, cost: 0 }; }
}
class PushChannel extends Channel {
  send(m) { return { via: "push", to: m.deviceToken, cost: 0 }; }
}
class WhatsAppChannel extends Channel {
  send(m) { return { via: "whatsapp", to: m.phone, cost: 35 }; }
}

// ---- registry factory: adding a channel never edits this class ----
class ChannelFactory {
  static #registry = new Map();
  static register(kind, builder) { ChannelFactory.#registry.set(kind, builder); }
  static create(kind, config = {}) {
    const build = ChannelFactory.#registry.get(kind);
    if (!build) throw new Error(`unknown channel: ${kind}`);
    return build(config);
  }
  static kinds() { return [...ChannelFactory.#registry.keys()]; }
}

ChannelFactory.register("sms", c => new SmsChannel(c.gateway ?? "primary"));
ChannelFactory.register("email", () => new EmailChannel());
ChannelFactory.register("push", () => new PushChannel());
ChannelFactory.register("whatsapp", () => new WhatsAppChannel());
```

```
const user = { phone: "+919000000000", email: "s@example.com", deviceToken: "dt-77" };
for (const kind of ChannelFactory.kinds())
  console.log(kind.padEnd(9), ChannelFactory.create(kind).send(user));

try { ChannelFactory.create("pigeon"); }
catch (e) { console.log("error ", e.message); }
```

node factory.js

TEXT

```
sms      { via: 'sms', to: '+919000000000', gateway: 'primary', cost: 12 }
email    { via: 'email', to: 's@example.com', cost: 0 }
push     { via: 'push', to: 'dt-77', cost: 0 }
whatsapp { via: 'whatsapp', to: '+919000000000', cost: 35 }
error    unknown channel: pigeon
```

Example 11 [Infosys · pattern]

Your notification service must pick a channel by user preference, but fall back if the chosen channel is unavailable, and it must never send WhatsApp to a user who has not opted in. Where does each of those three rules live? Be specific.

SOLUTION

Three rules, three homes. Putting all three in the factory is the mistake everyone makes.

Rule	Where it lives	Why not the factory
which class for the key "sms"	the factory	this is exactly what a factory is for
fallback order sms → push → email	a Strategy object, FallbackPolicy	the order changes per country and per message type; that is a business rule, not a construction rule
no WhatsApp without opt-in	a guard before the factory is called, in the dispatcher	a factory that refuses to build things is lying about its job, and consent is a legal rule that must be checked and logged , not silently skipped

The flow, in order:

dispatcher

```
-> consent check (legal gate, logged)
-> FallbackPolicy.order(user, messageType) -> ["sms", "push", "email"]
-> for each kind: ChannelFactory.create(kind).send(msg) until one succeeds
```

Why the ordering matters: it shows you can tell a **construction** concern from a **policy** concern. The factory answers “which class”. The policy answers “in what order, and may I at all”. Mix them and the factory needs the user object, the country, the consent table and the message type — at which point it is not a factory, it is the whole service.

Answer: Factory: key to class. Strategy: fallback order. Dispatcher guard: consent, before any construction.

Pattern 3 — Singleton

singleton.js — the classic form, what JS gives free, and why you should inject instead

JS

```
// --- 1. The classic textbook singleton, in JavaScript ---
class ConnectionPool {
  static #instance = null;
```

```

#conns; #inUse = 0;

constructor(size) {
  if (ConnectionPool.#instance)
    throw new Error("use ConnectionPool.get(), not new ConnectionPool()");
  this.#conns = Array.from({ length: size }, (_, i) => `conn-${i}`);
}
static get(size = 4) {
  if (!ConnectionPool.#instance) ConnectionPool.#instance = new
ConnectionPool(size);
  return ConnectionPool.#instance;
}
static reset() { ConnectionPool.#instance = null; } // ONLY for tests
acquire() {
  if (this.#inUse >= this.#conns.length) return null;
  return this.#conns[this.#inUse++];
}
release() { if (this.#inUse > 0) this.#inUse--; }
get stats() { return { size: this.#conns.length, inUse: this.#inUse }; }
}

const a = ConnectionPool.get(3);
const b = ConnectionPool.get(99); // size ignored: it already exists
console.log("same object?", a === b);
console.log("size is 3, not 99:", a.stats);
console.log("acquire:", a.acquire(), a.acquire(), a.acquire(), a.acquire());
try { new ConnectionPool(2); } catch (e) { console.log("blocked:", e.message); }

// --- 2. What a JS module already gives you, for free ---
// in pool.js: export const pool = new ConnectionPool(8);
// A module body runs once per process. That IS a singleton. No class tricks needed.

// --- 3. Why the pattern lies at scale ---
const pods = 40;
console.log(`\n"one" scheduler x ${pods} pods = ${pods} schedulers running the same
cron`);
console.log("a process-local singleton guarantees ONE PER PROCESS, never one per
cluster");

// --- 4. The testable alternative: pass it in, do not look it up ---
class ReportJob {
  constructor(pool) { this.pool = pool; }
  run() { return `report built on ${this.pool.acquire()}`; }
}
console.log(new ReportJob({ acquire: () => "fake-conn" }).run());

```

node singleton.js

TEXT

```

same object? true
size is 3, not 99: { size: 3, inUse: 0 }
acquire: conn-0 conn-1 conn-2 null
blocked: use ConnectionPool.get(), not new ConnectionPool()

```

```

"one" scheduler x 40 pods = 40 schedulers running the same cron
a process-local singleton guarantees ONE PER PROCESS, never one per cluster
report built on fake-conn

```

TRAP

Look at line 2 of the output: `ConnectionPool.get(99)` silently returned a pool of size 3. The argument was ignored because the instance already existed. This is a real and nasty bug class — the caller thinks it configured something and it did not. If a singleton takes configuration, either (i) make `get()` take no arguments and read config from one place, or (ii) throw if `get()` is called with different arguments than the first time. Never ignore the arguments silently.

Example 12 [Capgemini · pattern]

An interviewer asks: “Make your Singleton thread-safe.” In Java the answer is double-checked locking with a `volatile` field. What is the answer in JavaScript, and why is the question still worth a real reply?

SOLUTION

The JavaScript answer. One Node process runs your JavaScript on one thread, and nothing yields between the `if` and the assignment, so the naive lazy initialisation above is already safe — **in this exact form**.

Where it stops being safe: the moment initialisation is `async`.

```
static async get() {
  if (!this.#instance) this.#instance = await createPool(); // BUG
  return this.#instance;
}
```

The `await` yields. Ten concurrent callers all see `null`, all start `createPool()`, and you build ten pools. The fix is to cache the **promise**, not the result:

```
static get() {
  if (!this.#promise) this.#promise = createPool(); // assigned synchronously
  return this.#promise;                             // everyone awaits the same one
}
```

The assignment now happens before any `await`, so there is no window.

And with worker_threads: each worker has its own isolate and its own module instances, so a singleton is per worker, not per process.

What to say: “In Java, double-checked locking with a `volatile` field, because two threads can interleave between the null check and the assignment. In Node the synchronous version is safe — one JavaScript thread, no yield point in between — but as soon as construction becomes `async`, the `await` is a yield point and I get the same race. The fix there is to memoise the promise rather than the value.”

Answer: Cache the promise, not the value. Java needs double-checked locking; Node needs it only once `await` appears.

Pattern 4 — Observer

OBSERVER IN FOUR LINES

Problem: one thing happens; several unrelated parts of the system must react; the thing that happened must not know about any of them.

Structure: a **subject** keeps a list of **listeners** and calls them all. Listeners subscribe and unsubscribe.

The three details that separate a good answer from a bad one:

1. **Unsubscribe.** If `on()` returns nothing, listeners leak forever. Return a function that removes it.
2. **Error isolation.** One broken listener must not stop the other five. Wrap each call in `try/catch`.
3. **Iterate over a copy.** A listener may unsubscribe itself during the emit, which mutates the array you are looping over.

```

class EventBus {
  #handlers = new Map(); // event -> array of {fn, once}

  on(event, fn) { this.#add(event, fn, false); return () => this.off(event, fn); }
  once(event, fn) { this.#add(event, fn, true); return () => this.off(event, fn); }

  #add(event, fn, once) {
    if (!this.#handlers.has(event)) this.#handlers.set(event, []);
    this.#handlers.get(event).push({ fn, once });
  }

  off(event, fn) {
    const list = this.#handlers.get(event);
    if (!list) return false;
    const i = list.findIndex(h => h.fn === fn);
    if (i === -1) return false;
    list.splice(i, 1);
    return true;
  }

  emit(event, payload) {
    const list = this.#handlers.get(event);
    if (!list || list.length === 0) return { delivered: 0, failed: 0 };
    let delivered = 0, failed = 0;
    for (const h of [...list]) { // copy: a handler may unsubscribe itself
      try { h.fn(payload); delivered++; }
      catch (err) { failed++; console.log(" [isolated]", event, "->",
err.message); }
      finally { if (h.once) this.off(event, h.fn); }
    }
    return { delivered, failed };
  }
}

// --- the subject does not know who is listening ---
class OrderService {
  constructor(bus) { this.bus = bus; this.seq = 0; }
  place(userId, amountPaise) {
    const order = { id: ++this.seq, userId, amountPaise, status: "PLACED" };
    this.bus.emit("order.placed", order);
    return order;
  }
}

const bus = new EventBus();
bus.on("order.placed", o => console.log(" email : receipt for order", o.id));
bus.on("order.placed", o => console.log(" warehouse: pick list for order", o.id));
bus.on("order.placed", o => { throw new Error("loyalty service is down"); });
bus.once("order.placed", o => console.log(" welcome : first-order coupon (once only)"));

const svc = new OrderService(bus);
console.log("order 1:"); console.log(" result", svc.place("u1", 45000));
console.log("order 2:"); console.log(" result", svc.place("u1", 12000));

const stop = bus.on("order.placed", o => console.log(" audit : logged", o.id));
console.log("order 3:"); svc.place("u2", 9900);

```



```
stop();
console.log("order 4 (audit unsubscribed:)"); svc.place("u2", 100);
```

node observer.js

TEXT

```
order 1:
  email    : receipt for order 1
  warehouse: pick list for order 1
  [isolated] order.placed -> loyalty service is down
  welcome  : first-order coupon (once only)
  result { id: 1, userId: 'u1', amountPaise: 45000, status: 'PLACED' }
order 2:
  email    : receipt for order 2
  warehouse: pick list for order 2
  [isolated] order.placed -> loyalty service is down
  result { id: 2, userId: 'u1', amountPaise: 12000, status: 'PLACED' }
order 3:
  email    : receipt for order 3
  warehouse: pick list for order 3
  [isolated] order.placed -> loyalty service is down
  audit    : logged 3
order 4(audit unsubscribed):
  email    : receipt for order 4
  warehouse: pick list for order 4
  [isolated] order.placed -> loyalty service is down
```

Read the output carefully. The once welcome listener fires for order 1 and never again. The loyalty listener throws every time and never stops the other listeners. The audit listener appears for order 3 and disappears for order 4 because `stop()` was called. Those three behaviours are the whole difference between a toy Observer and one you would put in production.

Example 13 [Wipro · pattern]

In the code above, `OrderService.place()` returns **after** all four listeners have run. Is that right? What are the two options, and which do you choose?

SOLUTION

What is happening now: synchronous notification. The customer's HTTP response waits for all four listeners. At 40 ms each:

$$4 \times 40 = 160 \text{ ms added to every order}$$

The customer is paying for the warehouse's slowness.

Option A — keep it synchronous. Wins: a failed listener can still fail the whole order, ordering is guaranteed, and debugging is one stack trace. Loses: the caller's latency is the **sum** of all listeners, and one slow listener is everybody's problem.

Option B — notify asynchronously. Wins: `place()` returns in its own time and a slow listener cannot hurt the customer. Loses: a failed listener now fails **silently** after the customer got a 201, so you need retries, a dead-letter queue and monitoring, or work simply vanishes.

Decision: split by whether the listener affects the answer.

synchronous, inside the transaction	things that must be true before you say "order placed": reserve stock, write the ledger row. If these fail, the order must fail.
asynchronous, after commit	email, loyalty points, analytics, search indexing. The customer does not need them to have happened.

And the asynchronous ones must not be plain in-process listeners. They must be rows written in the **same database transaction** as the order — the outbox pattern — then picked up by a worker. Otherwise a crash between “committed the order” and “called the listener” loses the email with no record that it was ever owed.

Answer: Split them. Sync for anything that can fail the order; async via an outbox row for everything else.

Pattern 5 — Decorator

order A — idempotency inside the log, so retries are logged



order B — idempotency outermost, so a replayed request never reaches the log or the timer



Same four classes, different order, different behaviour. Always say which order you chose and why.

Decorator. The request travels right through the wrappers to the core, and the response travels back out through them. Order is part of the design.

decorator.js — four wrappers and a compose() helper

JS

```
// The thing being decorated: anything with .handle(req) -> response
class CreateOrder {
  handle(req) {
    if (req.amount <= 0) throw new Error("amount must be positive");
    return { orderId: "0-" + req.id, amount: req.amount };
  }
}

// Each decorator holds the next one and adds exactly one behaviour.
class WithLogging {
  constructor(next) { this.next = next; }
  handle(req) {
    console.log(" log > in ", JSON.stringify(req));
    const out = this.next.handle(req);
    console.log(" log < out", JSON.stringify(out));
    return out;
  }
}

class WithTiming {
  constructor(next) { this.next = next; }
  handle(req) {
    const t0 = process.hrtime.bigint();
    const out = this.next.handle(req);
    const us = Number(process.hrtime.bigint() - t0) / 1000;
    console.log(" timing took", us < 1000 ? "<1 ms" : us / 1000 + " ms");
    return out;
  }
}

class WithRetry {
  constructor(next, tries) { this.next = next; this.tries = tries; }
  handle(req) {
    let last;
    for (let i = 1; i <= this.tries; i++) {
      try { return this.next.handle(req); }
    }
  }
}
```

```

        catch (e) { last = e; console.log(`  retry  attempt ${i} failed: ${e.message}
`); }
      }
      throw last;
    }
  }
}
class WithIdempotency {
  constructor(next) { this.next = next; this.seen = new Map(); }
  handle(req) {
    if (this.seen.has(req.key)) {
      console.log("  idem    replayed stored response for key", req.key);
      return this.seen.get(req.key);
    }
    const out = this.next.handle(req);
    this.seen.set(req.key, out);
    return out;
  }
}

// compose(a, b, c)(core) === a(b(c(core)))  -- outermost listed first
const compose = (...wrappers) => core =>
  wrappers.reduceRight((inner, W) => new W(inner), core);

const handler = compose(WithLogging, WithTiming, WithIdempotency)(new CreateOrder());

console.log("first call:");
console.log(" ", handler.handle({ id: 1, key: "k-1", amount: 500 }));
console.log("same key again:");
console.log(" ", handler.handle({ id: 1, key: "k-1", amount: 500 }));

// A flaky core, wrapped in retry
let n = 0;
const flaky = { handle() { if (++n < 3) throw new Error("timeout"); return { ok:
n }; } };
console.log("retry demo:", new WithRetry(flaky, 5).handle({}));

// Why not subclasses: 2^n classes for n options
for (const k of [3, 5, 10])
  console.log(`${k} optional behaviours -> ${2 ** k} subclasses, or ${k}
decorators`);

```

node decorator.js

TEXT

```

first call:
log   > in  {"id":1,"key":"k-1","amount":500}
timing took <1 ms
log   < out {"orderId":"0-1","amount":500}
{ orderId: '0-1', amount: 500 }
same key again:
log   > in  {"id":1,"key":"k-1","amount":500}
idem   replayed stored response for key k-1
timing took <1 ms
log   < out {"orderId":"0-1","amount":500}
{ orderId: '0-1', amount: 500 }
retry  attempt 1 failed: timeout
retry  attempt 2 failed: timeout

```

```

retry demo: { ok: 3 }
3 optional behaviours -> 8 subclasses, or 3 decorators
5 optional behaviours -> 32 subclasses, or 5 decorators
10 optional behaviours -> 1024 subclasses, or 10 decorators

```

Example 14 [Cognizant · pattern]

In the run above, the second call with key k-1 was **still logged and still timed**, even though it did no work. Is that the right order? Give both orders and choose.

SOLUTION

Order A (what the code does): logging → timing → idempotency → core. The replay is logged and timed. You can see in your logs that a duplicate arrived.

Order B: idempotency → logging → timing → core. The replay returns instantly and never appears in the log or the latency metric.

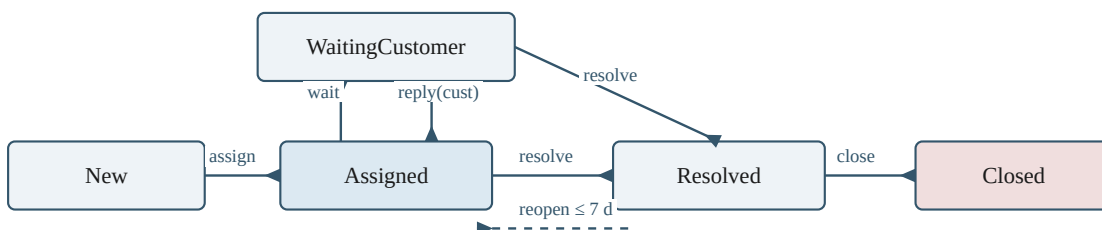
	A: idempotency inside	B: idempotency outside
duplicates visible in logs	yes	no
latency metric	includes replays, so it looks artificially fast	measures real work only
work saved	the core call	the core call, the log write, the timer
cost	a log line per duplicate — during a retry storm that is a lot of log	a blind spot: you cannot see duplicates at all

Decision: order A, with one change — log the replay at a lower level and count it as a metric. The reason is operational. Duplicate requests are a **signal**: a spike in replays means a client is retrying, which usually means something upstream is timing out. Making that invisible to save a log line is a bad trade. But include the replay count as its own counter so you can alert on it, and make sure the latency metric records replays separately so your p99 is not flattered by them.

The wider lesson: decorator order is not decoration. Swapping two wrappers changes what the system can see about itself. In an interview, always say the order out loud and give a reason for it.

Answer: Order A (idempotency innermost), but log replays at a lower level and count them separately.

Pattern 6 — State



`New.close()` (spam) also exists. `Closed` has no methods at all — it is terminal.

base class refuses every action
not listed on the current state

A support-ticket lifecycle as State. Every arrow is a method on a state class; everything not drawn is refused by the base class.

state.js — the base class refuses everything; each state allows only its own moves

JS

```

class TicketState {
  get name() { return this.constructor.name; }
  assign(t, agent) { return this.#no("assign"); }
}

```

```

waitForCustomer(t) { return this.#no("waitForCustomer"); }
reply(t, who)      { return this.#no("reply"); }
resolve(t)         { return this.#no("resolve"); }
close(t)           { return this.#no("close"); }
reopen(t)          { return this.#no("reopen"); }
#no(action) { return { ok: false, error: `cannot ${action} while ${this.name}` }; }
}

class New extends TicketState {
  assign(t, agent) { t.agent = agent; t.go(new Assigned()); return { ok: true }; }
  close(t)         { t.go(new Closed()); return { ok: true, note: "closed as
spam" }; }
}
class Assigned extends TicketState {
  waitForCustomer(t) { t.go(new WaitingCustomer()); return { ok: true }; }
  resolve(t)         { t.resolvedAt = t.clock; t.go(new Resolved()); return { ok:
true }; }
  assign(t, agent)   { t.agent = agent; return { ok: true, note: "re-assigned" }; }
}
class WaitingCustomer extends TicketState {
  reply(t, who) {
    if (who !== "customer") return { ok: false, error: "only the customer unblocks
this" };
    t.go(new Assigned()); return { ok: true };
  }
  resolve(t) { t.resolvedAt = t.clock; t.go(new Resolved()); return { ok: true }; }
}
class Resolved extends TicketState {
  close(t) { t.go(new Closed()); return { ok: true }; }
  reopen(t) {
    if (t.clock - t.resolvedAt > 7) return { ok: false, error: "reopen window
expired" };
    t.go(new Assigned()); return { ok: true };
  }
}
class Closed extends TicketState {} // terminal: every action refused by the base
class

class Ticket {
  constructor(id) {
    this.id = id; this.agent = null; this.clock = 0;
    this.resolvedAt = null; this.state = new New(); this.history = ["New"];
  }
  go(next) { this.state = next; this.history.push(next.name); }
  tick(days = 1) { this.clock += days; return this; }
  do(action, ...args) {
    const before = this.state.name;
    const r = this.state[action](this, ...args);
    console.log(
      (before + " ." + action + "()").padEnd(34),
      (r.ok ? "-> " + this.state.name : "REFUSED: " + r.error));
    return r;
  }
}

const t = new Ticket(41);
t.do("resolve"); // refused: not assigned yet

```

```

t.do("assign", "asha");
t.do("waitForCustomer");
t.do("reply", "agent");           // refused: wrong person
t.do("reply", "customer");
t.do("resolve");
t.tick(3).do("reopen");           // inside the 7-day window
t.do("resolve");
t.tick(9).do("reopen");           // window expired
t.do("close");
t.do("reopen");                   // terminal
console.log("history:", t.history.join(" -> "));

```

node state.js

TEXT

```

New .resolve()           REFUSED: cannot resolve while New
New .assign()             -> Assigned
Assigned .waitForCustomer() -> WaitingCustomer
WaitingCustomer .reply()  REFUSED: only the customer unblocks this
WaitingCustomer .reply()  -> Assigned
Assigned .resolve()       -> Resolved
Resolved .reopen()        -> Assigned
Assigned .resolve()       -> Resolved
Resolved .reopen()        REFUSED: reopen window expired
Resolved .close()         -> Closed
Closed .reopen()          REFUSED: cannot reopen while Closed
history: New -> Assigned -> WaitingCustomer -> Assigned -> Resolved -> Assigned ->
Resolved -> Closed

```

SHORTCUT

The “deny by default” base class is the most important line in the whole pattern. Because TicketState refuses every action, a state you forgot to code fails loudly with a clear message instead of silently doing something wrong. Closed has **no code at all** and is completely correct. Say this out loud: “the base class denies, so the only transitions that exist are the ones I wrote down.”

Example 15 [Accenture · pattern]

The reopen window is “7 days”. Notice that the code uses `t.clock`, a number passed in, not `Date.now()`. Why, and what would break if it used `Date.now()`?

SOLUTION

Why time is a parameter. Look at the run: `t.tick(3)` then `t.tick(9)` tested twelve days of lifecycle in under a millisecond.

If `reopen()` called `Date.now()` directly:

1. **You could not test the expiry** without waiting seven real days, or patching the global clock, which breaks every other test running at the same time.
2. **Boundaries would be untestable.** Is exactly 7.0 days allowed? With a clock parameter you write `t.tick(7)` and find out.
3. **Replay would be impossible.** Re-running last month’s events to debug an incident would use **today’s** clock and give different answers.
4. **Two servers with slightly different clocks would disagree**, so the user would see different answers depending on which server they hit.

The general rule, worth stating in every LLD round: “Time, randomness and id generation are **inputs**, not ambient facts. I pass a clock, a random source and an id generator into the object. Same inputs, same output, always testable.”

In production the HTTP layer reads the clock once at the edge and passes that one value all the way down. Everything below it is deterministic.

Answer: So the rule is testable, replayable, and identical on every server. Time is an input, not an ambient fact.

Six more you must be able to name

You will not be asked to implement these in a 40-minute round, but you **will** be asked to name them. Here is enough to answer well, with working code for the three that come up most.

Pattern	One line	The giveaway in the question
Builder	chained setters, validate in build()	“the constructor has ten arguments”
Adapter	translate a foreign API into yours	“we are switching payment providers”
Command	each action is an object with do and undo	“add undo”, “queue it”, “retry it”
Chain of Responsibility	linked handlers; first one to claim it wins	“run these checks in order”
Template Method	base class owns the steps, subclass fills one in	“same job, one step differs”
Proxy	same interface, stands in front of the real object	“add caching / access control / laziness”

builder.js — the value of Builder is the validation in build()

JS

```
class Pizza {
  constructor({ size, crust, toppings, extraCheese, cutInto }) {
    Object.assign(this, { size, crust, toppings, extraCheese, cutInto });
  }
  toString() {
    return `${this.size} ${this.crust} [${this.toppings.join(",")}]\` +
      `${this.extraCheese ? " +cheese" : ""} cut into ${this.cutInto}`;
  }
}

class PizzaBuilder {
  #d = { size: "medium", crust: "thin", toppings: [], extraCheese: false, cutInto:
8 };
  size(s) { this.#d.size = s; return this; }
  crust(c) { this.#d.crust = c; return this; }
  add(...t) { this.#d.toppings.push(...t); return this; }
  extraCheese(){ this.#d.extraCheese = true; return this; }
  slices(n) { this.#d.cutInto = n; return this; }
  build() {
    if (this.#d.toppings.length > 6) throw new Error("max 6 toppings");
    if (this.#d.size === "small" && this.#d.cutInto > 6)
      throw new Error("a small pizza cannot have more than 6 slices");
    return new Pizza({ ...this.#d, toppings: [...this.#d.toppings] });
  }
}

console.log(String(new PizzaBuilder().size("large").crust("stuffed")
  .add("paneer", "capsicum").extraCheese().slices(12).build()));
console.log(String(new PizzaBuilder().add("corn").build()));
try { new PizzaBuilder().size("small").slices(10).build(); }
catch (e) { console.log("rejected at build():", e.message); }
```

node builder.js

TEXT

```
large stuffed [paneer,capsicum] +cheese cut into 12
medium thin [corn] cut into 8
rejected at build(): a small pizza cannot have more than 6 slices
```

The line `toppings: [...this.#d.toppings]` is not decoration. Without it the built `Pizza` shares the array with the builder, so reusing the builder would silently modify a pizza you already made. `[...a]` is a shallow copy; for a nested array you would need `a.map(r => [...r])`. This is a JavaScript trap that has failed real candidates.

adapter.js — one provider translated into your own port

JS

```
class PaymentPort { charge(orderId, paise) { throw new Error("not implemented"); } }

// SDK A: rupees as a float, callbacks, its own error shape.
const gwA = { pay(ref, rupees, cb) {
  if (rupees > 10000) return cb({ code: 42, text: "limit exceeded" });
  cb(null, { txn: "A-" + ref, amount_rupees: rupees });
} };

class AdapterA extends PaymentPort {
  charge(orderId, paise) {
    // paise in, paise out – always
    return new Promise((ok, no) => gwA.pay(orderId, paise / 100, (e, r) =>
      e ? no(new Error(`payment failed [${e.code}]: ${e.text}`))
        : ok({ txnId: r.txn, paise: Math.round(r.amount_rupees * 100) })));
  }
}

const port = new AdapterA();
port.charge("ord-7", 45000).then(r => console.log(r));
port.charge("ord-8", 5_000_000).catch(e => console.log(e.message));
```

node adapter.js

TEXT

```
{ txnId: 'A-ord-7', paise: 45000 }
payment failed [42]: limit exceeded
```

A second provider with promises and minor units gets `AdapterB` with the same `charge(orderId, paise)` signature. Nothing above the adapters changes.

Example 16 [TCS Digital · pattern]

The adapter turns two different error shapes into one message. List three more things a good adapter must normalise, and say why each matters.

SOLUTION

1. **Units.** One SDK takes rupees as a float, the other minor units as an integer. Convert **at the boundary** so no business code ever sees a float rupee. The round trip must be exact: 45,000 paise → 450.0 rupees → 45,000 paise, which is why the code says `Math.round(r.amount_rupees * 100)` and not a bare multiplication.
2. **Idempotency key names.** One calls it `ref`, the other `idempotencyKey`. Your code has one concept — “the order id is the key” — and the adapter maps it. Let the SDK names leak and half your code passes the wrong field on the day you switch providers.
3. **Retryability.** “Limit exceeded” must never be retried; “gateway timeout” must be. The adapter maps each provider’s 40 error codes onto your own three: `PERMANENT`, `TRANSIENT`, `UNKNOWN`. The retry decorator above it then needs no provider knowledge.
4. **Timeouts.** The callback SDK has none. The adapter imposes **yours**, because a dependency with no timeout can hang your whole process.

The sentence that ties it together: “The adapter is the only file allowed to know the provider’s vocabulary. Everything above it speaks one language. That is what makes swapping providers a one-file change.”

Answer: Units, idempotency key names, error classification (permanent/transient), and timeouts.

command.js — undo and redo; the deleted text lives in the command

JS

```
class Insert {
  constructor(at, text) { this.at = at; this.text = text; }
  do(d) { return d.slice(0, this.at) + this.text + d.slice(this.at); }
  undo(d) { return d.slice(0, this.at) + d.slice(this.at + this.text.length); }
}
class Delete {
  constructor(at, len) { this.at = at; this.len = len; this.removed = ""; }
  do(d) { this.removed = d.substr(this.at, this.len); // keep it, or you cannot
  undo
    return d.slice(0, this.at) + d.slice(this.at + this.len); }
  undo(d) { return d.slice(0, this.at) + this.removed + d.slice(this.at); }
}
class Editor {
  constructor(text = "") { this.text = text; this.done = []; this.undone = []; }
  run(c) { this.text = c.do(this.text); this.done.push(c); this.undone = []; return
  this; }
  undo() { const c = this.done.pop(); if (!c) return this;
    this.text = c.undo(this.text); this.undone.push(c); return this; }
  redo() { const c = this.undone.pop(); if (!c) return this;
    this.text = c.do(this.text); this.done.push(c); return this; }
  show(t) { console.log(t.padEnd(16), JSON.stringify(this.text)); return this; }
}
new Editor("hello world").show("start")
  .run(new Insert(5, ",")).show("insert comma")
  .run(new Delete(0, 6)).show("delete")
  .undo().show("undo delete")
  .undo().show("undo insert")
  .redo().show("redo insert");
```

node command.js

TEXT

start	"hello world"
insert comma	"hello, world"
delete	" world"
undo delete	"hello, world"
undo insert	"hello world"
redo insert	"hello, world"

TRAP

Two details in Editor that interviewers look for:

1. Delete stores this.removed during do(). You cannot undo a delete unless you kept what you deleted. Candidates forget this constantly.
2. run() clears this.undone. After you undo three times and then type something new, the redo stack must be thrown away — otherwise redo would replay commands that apply to a document that no longer exists.

chain.js — Chain of Responsibility as a validation pipeline

JS

```

class Check {
  setNext(c) { this.next = c; return c; } // returns c, so you can chain
  handle(req) {
    const rejection = this.test(req); // null means "not my problem"
    if (rejection) return rejection;
    return this.next ? this.next.handle(req) : { ok: true };
  }
}

class NotEmpty extends Check {
  test(r) { return r.body.trim() ? null : { ok: false, code: 400, why: "empty body" }; }
}

class LengthLimit extends Check {
  constructor(max) { super(); this.max = max; }
  test(r) { return r.body.length <= this.max ? null : { ok: false, code: 413, why: `over ${this.max} characters` }; }
}

class NoLinks extends Check {
  test(r) { return /https?:\/\//.test(r.body) ? { ok: false, code: 422, why: "links not allowed yet" } : null; }
}

const head = new NotEmpty();
head.setNext(new LengthLimit(40)).setNext(new NoLinks());
for (const body of [" ", "x".repeat(50), "see https://spam.example", "hello there"])
  console.log(JSON.stringify(body).padEnd(22).slice(0, 22), head.handle({ body }));

```

node chain.js

TEXT

```

" " { ok: false, code: 400, why: 'empty body' }
"xxxxxxxxxxxxxxxxxxxxxxxxxxx { ok: false, code: 413, why: 'over 40 characters' }
"see https://spam.exam { ok: false, code: 422, why: 'links not allowed yet' }
"hello there" { ok: true }

```

Example 17 [Cognizant · pattern]

Look at the chain order: empty → length → links → rate limit. Is that order right? What happens if you put RateLimit first?

SOLUTION

Two competing principles. Cheapest first: `body.trim()` costs nanoseconds, a rate limit that hits Redis costs half a millisecond, so run the cheap ones first. **Protective first:** a rate limit exists to stop an attacker, and if it is last, 50,000 junk requests a second still run three checks each.

The numbers settle it. With the rate limit last, an attacker's 50,000 requests/s are rejected by local string checks and **never reach Redis**. With it first, they cost 50,000 Redis ops a second — half of one node — to reject traffic that three free checks would have killed anyway.

Decision: keep the current order inside the chain, and move the rate limit out of the chain altogether — to the load balancer or gateway. A rate limit is not a validation rule, it is a **protection** rule, and protection belongs as far from your application as you can push it. A rate-limited request should ideally never reach your process. Inside the chain, its correct position is last. Outside the chain is better still.

Answer: Cheapest-first is right inside the chain, so rate limit last. Better still: move it to the gateway.

PRACTICE · TIER 1 target: 30 min

1. Write a FareStrategy for “first 3 km free, then Rs.18 per started km, minimum fare Rs.60”. Price a 1 km ride and a 9.2 km ride, in paise.
2. Add a telegram channel to the registry factory. How many existing files do you edit?
3. Write the Assigned state’s close() so a ticket can be closed directly by a manager, but only if an agent is assigned. Where does the check go?
4. Write a WithTimeout(next, ms) decorator. Which of the four wrappers must it sit outside, and which inside?
5. You have Insert and Delete commands. Write Replace(at, len, text) using them. What must undo() do?
6. Name the pattern: “one object presents a simple placeOrder() that internally calls inventory, payments, and shipping.”

ANSWER KEY

1.

```
class FreeKmFare {
  quote(r) {
    const charged = Math.max(0, Math.ceil(r.km - 3));
    return Math.max(6000, charged * 1800);
  }
}
```

1 km: $[1 - 3] = 0$ charged km, so $0 \times 1800 = 0$, and the minimum lifts it to **6,000 paise (Rs.60)**. 9.2 km: $[9.2 - 3] = [6.2] = 7$ charged km, $7 \times 1800 = 12,600$ paise (**Rs.126.00**), above the minimum.

2. **Zero** existing files. You add telegram.js with the class and one ChannelFactory.register("telegram", ...) line. That is the entire value of a registry factory, and it is the sentence to say out loud.

3.

```
class Assigned extends TicketState {
  close(t) {
    if (!t.agent) return { ok: false, error: "assign an agent before closing" };
    t.go(new Closed()); return { ok: true };
  }
}
```

The check goes **inside Assigned.close()**, not in Ticket.do(). A rule about what is legal while assigned belongs to the Assigned class. Putting it in Ticket starts the if-chain again.

4. WithTimeout must sit **inside WithRetry** (so each attempt gets its own deadline, rather than one deadline covering all attempts) and **outside** the core. Relative to logging and timing it does not matter much, but put it inside WithLogging so the timeout is logged.

5. Replace runs Delete(at, len) then Insert(at, text) and keeps both. undo() must reverse them **in the opposite order**: undo the insert first, then undo the delete. Undoing a composite in the same order corrupts the document — this is the single most common bug in Command implementations.

6. Facade. One simple entry point in front of several subsystems. It is not on the “six that get asked” list, but you must be able to name it, and it is the correct answer whenever someone says “hide the complexity behind one call”.

Tier 2 — the same patterns inside one real service

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

In a Tier-2 round the pattern is not the answer — it is one paragraph of the answer. What is being tested is whether you know how each pattern **changes** when there is a database, a network, and more than one process.

WHAT HAPPENS TO EACH PATTERN WHEN IT CROSSES A PROCESS BOUNDARY

Pattern	In one process	Across services
Strategy	an injected object	a config row + a registry; needs versioning and a rollout
Observer	a list of functions	a queue topic; needs at-least-once, dedupe, DLQ, ordering
State	a field holding an object	a column + a conditional UPDATE; needs optimistic locking
Singleton	static #instance	a distributed lock or leader election — a static field means nothing
Decorator	wrapper classes	middleware, sidecars, or a service mesh
Factory	a Map of constructors	service discovery: which endpoint, not which class

Example 18 [Grab · pattern]

Strategy as configuration. Pricing rules must change without a deploy. Product wants to change the surge multiplier at 6 p.m. on a Friday. Design it, and name the danger.

SOLUTION

The naive version and why it fails. “Put the formula in the database and evaluate it.” Now your pricing engine is an interpreter for a little language, and a typo in a config row prices every ride at zero. That is untested, unreviewed code running in production.

The design that works: a registry of code, parameterised by config.

```
strategies (code, written by engineers)  "flat" -> FlatFare
                                         "slab" -> SlabFare
                                         "surge" -> SurgeFare(inner)

pricing_config row (edited by product, reviewed, versioned)
{ city: "BLR", version: 14, strategy: "surge",
  params: { inner: "slab", basePaise: 5000, baseKm: 2,
            perKmAfter: 1200, multiplierBp: 15000 },
  activeFrom: "2026-02-13T18:00:00+05:30" }
```

Product changes **numbers**; engineers change **behaviour**. Different pull requests, different reviewers.

Five rules that make it safe:

version every row	a quote records the config version it used, so a fare dispute six months later is reproducible
validate on write	when version 15 is saved, replay it against 1,000 recorded rides. If any price moves more than a threshold, refuse the save
bounds on every parameter	multiplierBp between 10,000 and 30,000, so a typed 150000 is rejected by the schema, not by a customer's bank statement
activeFrom, never “now”	changes are scheduled and visible in advance
cache 30 s, with a floor	if the config service is down, keep serving the last known good version. Never fail open on price

Why the cache. At 5 million rides a day:

$$5,000,000 / 86,400 = 57.9 \text{ quotes/s average, about } 174/\text{s at } 3x \text{ peak}$$

Reading the row per quote is 174 reads/s of a row that changes twice a week. With a 30-second cache on 40 servers: $40/30 = 1.33 \text{ reads/s}$, a **130x** reduction.

The danger, named plainly: config is code that skipped code review. Everything above exists to put the review back.

Answer: Registry of code + versioned, bounds-checked, replay-validated config rows, cached 30 s. Config is code without review; add the review back.

Example 19 [Shopee · pattern]

Observer becomes a queue. The in-process EventBus from Tier 1 worked because the listeners were in the same process. Now email, warehouse and loyalty are three separate services. What changes, and what new problems appear?

SOLUTION

What stays the same: the subject still does not know who listens. That is Observer, unchanged.

What changes — five things, and every one is a follow-up question.

1. Delivery is at-least-once	a consumer can crash after doing the work and before acknowledging, so the same message arrives twice. Every consumer keys on (consumerName, eventId) with a unique index and skips what it has already done.
2. Failures are invisible	in-process, a thrown listener printed a line. Across services a failing consumer just stops acknowledging. You need a retry policy, a dead-letter queue , and an alert on its depth.
3. Ordering is per partition	order.placed and order.cancelled must not be processed out of sequence. Partition by orderId so one order's events land on one partition.
4. The publish and the database write can disagree	commit the order, then fail to publish, and nobody is told. Fix: write the event into an outbox table in the same transaction , and let a relay publish from there.
5. The payload is a public contract	three teams parse it. Adding a field is safe; renaming or removing one breaks somebody. Only add; version the event type when you must break it.

The numbers. 200 million events a day, 6 consumer groups:

$$200,000,000 / 86,400 = 2,315 \text{ events/s published} \times 6 = 13,889 \text{ deliveries/s}$$

$$200,000,000 \times 6 = 1,200,000,000 \text{ deliveries/day}$$

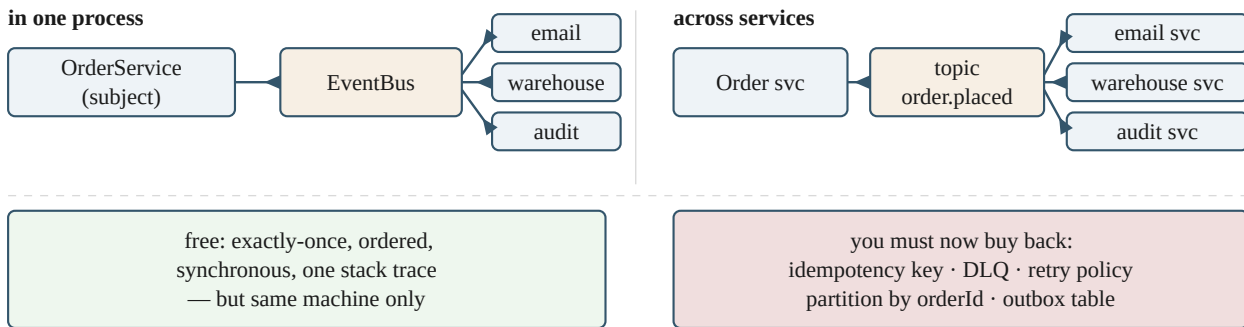
At a 0.1% permanent failure rate:

$$1,200,000,000 \times 0.001 = 1,200,000 \text{ dead letters/day} = 13.9 \text{ per second}$$

Fourteen a second is not something a human inspects, so the DLQ needs its own tooling: group by error signature, bulk replay once the bug is fixed, and alert on the **rate of change** rather than the absolute depth.

The summary sentence: “The pattern survives the network; the **guarantees** do not. In-process Observer gives me exactly-once, ordered, synchronous delivery for free. Across a broker I buy each of those back with idempotency keys, partitioning and a DLQ.”

Answer: Same pattern, but you must add idempotency, DLQ, per-key partitioning, an outbox, and an additive-only payload contract.



Observer in one process (left) and the same pattern across services (right). The shape is identical; everything below the dashed line is what the network costs you.

Example 20 [Agoda · pattern]

State in a database. The ticket machine from Tier 1 kept state in memory. Now there are 20 servers and the state is a column. Two requests arrive at the same millisecond. Write the code and show the race being caught.

SOLUTION

The bug you must not write.

```
const order = await db.read(id);           // status = CREATED
if (ALLOWED.get(order.status).includes("PAID"))
  await db.update(id, { status: "PAID" }); // read-modify-write race
```

Two requests both read CREATED, both pass the check, both write. The second write wins and the first caller was told “you paid” when it did not cause the payment.

The fix: put the old state in the WHERE clause.

```
UPDATE orders SET status = 'PAID', version = version + 1
WHERE id = ? AND status = 'CREATED' AND version = ?
```

The database decides. If it returns 0 rows, you lost the race. That is **optimistic locking**, and the version column is the whole mechanism.

persisted.js — a state machine whose guard is a conditional UPDATE

JS

```
// A table with one row per order. version = optimistic lock column.
class FakeTable {
  #rows = new Map();
  insert(row) { this.#rows.set(row.id, { ...row }); }
  read(id) { const r = this.#rows.get(id); return r ? { ...r } : null; }
  // UPDATE orders SET status=?, version=version+1
  // WHERE id=? AND status=? AND version=? <-- the whole trick is this WHERE
  compareAndSet(id, fromStatus, fromVersion, toStatus) {
    const r = this.#rows.get(id);
    if (!r || r.status !== fromStatus || r.version !== fromVersion) return 0; // 0 rows
    r.status = toStatus; r.version++;
    return 1;
  }
}

const ALLOWED = new Map(Object.entries({
  CREATED: ["PAID", "CANCELLED"],
  PAID:    ["SHIPPED", "REFUNDED"],
  SHIPPED: ["DELIVERED"],
}));
```

```

    DELIVERED: [],
    CANCELLED: [],
    REFUNDED: [],
  }));

class OrderStateMachine {
  constructor(table) { this.table = table; }
  transition(id, toStatus, snapshot) {
    const allowed = ALLOWED.get(snapshot.status) ?? [];
    if (!allowed.includes(toStatus))
      return { ok: false, code: 409, why: `${snapshot.status} -> ${toStatus} is not allowed` };
    const rows = this.table.compareAndSet(id, snapshot.status, snapshot.version, toStatus);
    if (rows === 0) {
      const now = this.table.read(id);
      if (now.status === toStatus)
        return { ok: true, idempotent: true, status: now.status }; // someone else did it
      return { ok: false, code: 409, why: `lost the race; row is now ${now.status}` };
    }
    return { ok: true, status: toStatus };
  }
}

const db = new FakeTable();
db.insert({ id: "0-1", status: "CREATED", version: 1 });
const sm = new OrderStateMachine(db);

console.log("1 illegal jump ", sm.transition("0-1", "DELIVERED", db.read("0-1")));

// Two workers read the SAME snapshot, then both try to move it. Classic race.
const snapA = db.read("0-1");
const snapB = db.read("0-1");
console.log("2 worker A pays ", sm.transition("0-1", "PAID", snapA));
console.log("3 worker B pays ", sm.transition("0-1", "PAID", snapB));
console.log("4 worker B cancels (stale snapshot)",
  sm.transition("0-1", "CANCELLED", snapB));
console.log("5 ship ", sm.transition("0-1", "SHIPPED", db.read("0-1")));
console.log("6 deliver ", sm.transition("0-1", "DELIVERED", db.read("0-1")));
console.log("7 refund after delivery", sm.transition("0-1", "REFUNDED", db.read("0-1")));
console.log("final row ", db.read("0-1"));

```

node persisted.js

TEXT

```

1 illegal jump  { ok: false, code: 409, why: 'CREATED -> DELIVERED is not allowed' }
2 worker A pays { ok: true, status: 'PAID' }
3 worker B pays { ok: true, idempotent: true, status: 'PAID' }
4 worker B cancels (stale snapshot) { ok: false, code: 409, why: 'lost the race; row is now PAID' }
5 ship          { ok: true, status: 'SHIPPED' }
6 deliver       { ok: true, status: 'DELIVERED' }
7 refund after delivery { ok: false, code: 409, why: 'DELIVERED -> REFUNDED is not

```

```
allowed' }
final row      { id: '0-1', status: 'DELIVERED', version: 4 }
```

Read lines 2, 3 and 4 together — they are the whole answer.

- Line 2: worker A wins the compare-and-set. One row changed.
- Line 3: worker B has a stale snapshot, so the `WHERE` matches nothing and zero rows change — but the row is **already** in the state B wanted, so B returns success marked idempotent. B asked for “make it PAID” and it is PAID.
- Line 4: B, still stale, asks to cancel. Now the answer must be a hard **409**, because the outcome B wanted did not happen and pretending otherwise would lose money.

That distinction — “zero rows, but the goal is met” versus “zero rows, and the goal is not met” — is what candidates miss. Returning 409 for both makes correct retries fail. Returning 200 for both makes cancellations silently vanish.

What it costs. Optimistic locking rejects writers instead of queueing them, so a very hot row (a flash-sale item) would thrash. **Decision: optimistic locking by default, because contention on one order row is near zero; a per-key queue only for the few rows known to be hot.**

Answer: Guard the transition with `WHERE status = ? AND version = ?`. Zero rows plus the goal already met = idempotent success; zero rows otherwise = 409.

Example 21 [DBS · pattern]

Factory plus Adapter for provider failover. Your service sends OTP messages through three SMS providers. Provider A is cheapest, B is more reliable, C is the emergency option. Design the selection, and say where each pattern sits.

SOLUTION

Four pieces, four patterns. Keeping them separate is the answer.

Piece	Pattern	What it does, and nothing else
ProviderRegistry	Factory	key "A" to a live client object
ProviderAAdapter	Adapter	translates A's fields, errors and timeouts into your <code>SmsPort</code>
RoutingPolicy	Strategy	given country, message type and health, returns the order to try
CircuitBreaker	State	per provider: CLOSED / OPEN / HALF_OPEN

```
send(otp)
-> RoutingPolicy.order(...)      -> ["A", "B", "C"]
-> for each p:
    if breaker[p].isOpen -> skip, no network call at all
    try Registry.create(p).send(otp) -> success, record it, return
    catch transient      -> record failure, try the next provider
    catch permanent      -> stop; another provider will reject it too
```

The detail that earns the mark: transient versus permanent. “Invalid phone number” is permanent — B will reject it too, so failing over turns one wasted call into three. “Gateway timeout” is transient. The **adapter** is the only place that knows which of provider A’s 40 error codes means which.

Why the breaker. If A dies and each call takes its full 30-second timeout at 2,000 OTPs a second, Little’s Law gives

$$L = \lambda W = 2,000 \times 30 = 60,000 \text{ calls in flight}$$

Your process dies long before that. With a breaker that fails in about 1 ms, $L = 2,000 \times 0.001 = 2$. Example 23 works this through in full.

Decision: route by cost when healthy, by reliability when degraded. Normal order is A, B, C (cheapest first). If A's breaker is OPEN, or its success rate over the last minute drops below 97%, the policy reorders to B, C, A. A user who cannot log in is a lost user; the SMS costs paise.

Answer: Factory = which client. Adapter = translate fields/errors/timeouts. Strategy = order to try. State = circuit breaker per provider, which turns 60,000 hung calls into 2.

PRACTICE · TIER 2 target: 35 min

1. Your pricing config is cached for 30 s on 40 servers. A bad config is published. What is the worst-case time until every server is serving the old good version again, and what do you add to make it faster?
2. An `order.placed` consumer processes the same event twice. Write the exact table and unique index that makes this harmless.
3. Events for one order are processed out of order (cancelled before placed). Give two fixes and choose one.
4. 80 million events a day with 5 consumer groups. Deliveries per day and per second?
5. A conditional UPDATE returns 0 rows. Give the two possible reasons and the correct HTTP status for each.
6. Where do you put a `WithTimeout` decorator relative to a circuit breaker, and why?

ANSWER KEY

1. Worst case is the full TTL, **30 seconds**, plus the time to publish the rollback. To make it faster add (i) a push channel — publish a “config changed” event so servers refresh immediately instead of waiting for the poll, and (ii) an instant kill switch that pins every server to a named known-good version. Keep the 30 s poll as the fallback for when the push channel is broken.

2.

```
CREATE TABLE processed_events (  
  consumer VARCHAR(64) NOT NULL,  
  event_id  VARCHAR(64) NOT NULL,  
  handled_at TIMESTAMP NOT NULL,  
  PRIMARY KEY (consumer, event_id)  
);
```

Insert the row **in the same transaction** as the work. A duplicate hits the primary key, the transaction rolls back, the message is acknowledged anyway. The key must include the consumer name, or the email service's row would silently suppress the warehouse service.

3. Fix A: partition the topic by `orderId`, so one order's events always go to one partition and are consumed in order. Fix B: put a sequence number in each event and have the consumer buffer or reject anything out of order. **Choose A** — it is one configuration line and it makes the problem impossible, whereas B is code in every consumer, forever. Use B only when events genuinely come from several producers that cannot share a partition key.

4. $80,000,000 \times 5 = 400,000,000$ deliveries/day. $400,000,000 / 86,400 = 4,630$ deliveries/s average; at 3x peak about **13,900/s**.

5. Reason 1: the row is already in the state you asked for — someone else did it, or this is your own retry. Return **200** with the current state; the caller's goal is met. Reason 2: the row moved to a **different** state. Return **409 Conflict** with the current state in the body so the caller can decide what to do.

6. `WithTimeout` goes **inside** the breaker. The breaker must see the timeout as a failure so it can count it and open. If the timeout were outside, the breaker would never learn that the dependency is slow, and it would stay closed while every call hung.

Tier 3 — patterns at large scale

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

At this tier the question is usually a trap: an interviewer names a pattern and waits to see whether you apply it without thinking. The winning answer almost always contains the words “that pattern does not mean what it means in one process”.

Example 22 [Google · pattern]

The singleton that is not single. A nightly job emails 2 million users. It is implemented as a singleton scheduler inside the API service, which runs on 40 pods. What happens, and what is the correct design?

SOLUTION

What happens.

$$40 \text{ pods} \times 1 \text{ scheduler each} = 40 \text{ schedulers}$$

$$40 \times 2,000,000 = 80,000,000 \text{ emails instead of } 2,000,000$$

Forty copies of every email. Your sending reputation is destroyed, your provider rate-limits you, and the bill is 40x. A static `#instance` field guarantees one instance **per process** and says nothing at all about a cluster.

Three designs, compared honestly.

Design	How it works	Cost and failure mode
A. Leader election	Pods compete for a lease key with a TTL; the holder runs the job and renews it	needs a consistent store, and a long garbage-collection pause can let the lease expire while the old leader still runs — so the job must be idempotent anyway
B. External scheduler	a managed cron makes one call or enqueues one message; pods are stateless workers	simplest, and usually right. The scheduler becomes a dependency you must monitor
C. Partition the work	all 40 pods run; pod i takes users where $\text{hash}(\text{userId}) \bmod 40 = i$	no coordination at all and 40x throughput; needs stable pod identity and a rebalance plan

Decision: B for correctness, C for throughput, idempotency underneath both.

B removes the problem rather than managing it — exactly one trigger exists because exactly one thing triggers. But one pod sending 2 million emails at 200/s takes

$$2,000,000 / 200 = 10,000 \text{ seconds} = 2.8 \text{ hours}$$

which will not fit a nightly window. So the triggered job does not send; it **fans out** 2 million messages onto a queue and all 40 pods consume:

$$2,000,000 / (200 \times 40) = 250 \text{ seconds} \approx 4 \text{ minutes}$$

Underneath everything: a unique index on (`campaignId`, `userId`). If the trigger fires twice, or two leases briefly overlap, the second send hits the index and does nothing. **Never rely on “only one runner” for correctness.** Coordination reduces duplicate **work**; the unique index prevents duplicate **effects**. You need both, and you must say which is doing which job.

Answer: 80 million emails. Use an external trigger, fan out to all pods for throughput, and make the effect idempotent with a unique index — coordination optimises, the index is correct.

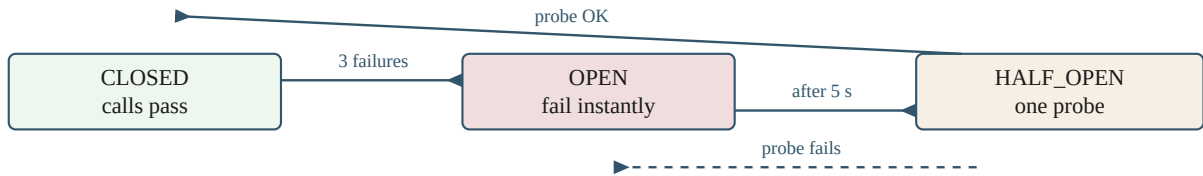
Example 23 [Amazon · pattern]

Circuit breaker as a State machine. Implement it, trace the three states, and give the numbers that prove it is worth having.

SOLUTION

The three states, and what each one does.

State	Behaviour	Leaves when
CLOSED	every call goes through; failures are counted	N consecutive failures
OPEN	every call fails instantly, no network touched	the cool-down timer expires
HALF_OPEN	exactly one probe call is allowed through	probe succeeds → CLOSED; probe fails → OPEN



Without HALF_OPEN, the cool-down would end by sending the full 2,000 requests/s at a dependency that may still be dead.

The circuit breaker is a three-state machine. HALF_OPEN exists so recovery costs one request, not a flood.

breaker.js — the state machine, driven by an injected clock

JS

```

class CircuitBreaker {
  constructor({ failuresToOpen = 3, openMs = 5000, halfOpenProbes = 1 } = {}) {
    this.failuresToOpen = failuresToOpen;
    this.openMs = openMs;
    this.halfOpenProbes = halfOpenProbes;
    this.state = "CLOSED";
    this.failures = 0;
    this.openedAt = 0;
    this.probesLeft = 0;
    this.phase = "CLOSED";
  }

  call(fn, now) {
    if (this.state === "OPEN") {
      if (now - this.openedAt >= this.openMs) this.#toHalfOpen();
      else return { ok: false, short: true, why: "circuit OPEN, failing fast" };
    }
    if (this.state === "HALF_OPEN" && this.probesLeft <= 0)
      return { ok: false, short: true, why: "circuit HALF_OPEN, probe in progress" };

    this.phase = this.state; // the state the call actually ran in
    if (this.state === "HALF_OPEN") this.probesLeft--;
    try {
      const value = fn();
      this.#onSuccess();
      return { ok: true, value };
    } catch (e) {
      this.#onFailure(now);
      return { ok: false, short: false, why: e.message };
    }
  }

  #onSuccess() {
    if (this.state === "HALF_OPEN") { this.state = "CLOSED"; }
    this.failures = 0;
  }

  #onFailure(now) {
    this.failures++;
    if (this.state === "HALF_OPEN" || this.failures >= this.failuresToOpen) {
      this.state = "OPEN"; this.openedAt = now;
    }
  }
}

```

```

}
#toHalfOpen() { this.state = "HALF_OPEN"; this.probesLeft =
this.halfOpenProbes; }
}

// A dependency that is down from t=0 to t=12000, then healthy.
let clock = 0;
const dep = () => { if (clock < 12000) throw new Error("upstream 503"); return
"pong"; };

const cb = new CircuitBreaker({ failuresToOpen: 3, openMs: 5000 });
const trace = [0, 100, 200, 300, 1000, 5400, 10500, 15600, 15700];
for (const t of trace) {
  clock = t;
  const before = cb.state;
  const r = cb.call(dep, t);
  console.log(String(t).padStart(6), "ms ", before.padEnd(9), "ran as",
    (r.short ? before : cb.phase).padEnd(9), "-> now",
cb.state.padEnd(9),
    r.ok ? "OK " + r.value : (r.short ? "short-circuited" : "real call
failed"));
}

```

node breaker.js

TEXT

0 ms	CLOSED	ran as CLOSED	-> now CLOSED	real call failed
100 ms	CLOSED	ran as CLOSED	-> now CLOSED	real call failed
200 ms	CLOSED	ran as CLOSED	-> now OPEN	real call failed
300 ms	OPEN	ran as OPEN	-> now OPEN	short-circuited
1000 ms	OPEN	ran as OPEN	-> now OPEN	short-circuited
5400 ms	OPEN	ran as HALF_OPEN	-> now OPEN	real call failed
10500 ms	OPEN	ran as HALF_OPEN	-> now OPEN	real call failed
15600 ms	OPEN	ran as HALF_OPEN	-> now CLOSED	OK pong
15700 ms	CLOSED	ran as CLOSED	-> now CLOSED	OK pong

The numbers that prove it is worth having. The dependency is dead, each call hangs for its 30-second timeout, and traffic is 2,000 requests a second.

Without a breaker, by Little's Law:

$$L = 2,000 \times 30 = 60,000 \text{ calls in flight}$$

Sixty thousand sockets and sixty thousand pending promises. The process runs out of file descriptors and dies — because **someone else's** service died.

With the breaker, after 3 failures every call returns in about 1 ms:

$$L = 2,000 \times 0.001 = 2 \quad 60,000 \rightarrow 2 = 30,000 \text{ times fewer}$$

What HALF_OPEN buys. Without it, the cool-down would end by releasing the full 2,000 requests a second at a dependency that may still be dead — which re-kills it and re-opens the breaker, a cycle that can run for hours. HALF_OPEN spends **one** request to ask “are you back?” The trace shows it: the probes at 5,400 ms and 10,500 ms cost one call each and fail; the probe at 15,600 ms succeeds and the circuit closes.

The trade-off, named and decided. A breaker fails requests that **might** have succeeded — during a 2-second blip it rejects traffic the dependency could have served. **Take that price.** The alternative is that a 2-second blip in one dependency takes down every service that calls it, which is how a small incident becomes a company-wide outage. Bounded, fast, visible failure beats unbounded queueing.

And say this last part: a circuit breaker is per process. Forty pods have forty breakers, each learning independently — which is better than a shared one, because a shared breaker is itself a single point of failure and a new dependency in the hot path.

Answer: CLOSED / OPEN / HALF_OPEN. It turns 60,000 hung calls into 2. HALF_OPEN makes recovery cost one request instead of a flood.

Example 24 [Microsoft · pattern]

Rolling out a new Strategy safely. You have a new pricing algorithm. Rolling it out to everyone at once is unacceptable. Design the rollout and prove the new rule is safe before it serves anyone.

SOLUTION

Three mechanisms, used in this order.

1. **Shadow mode.** Serve the old rule. Also run the new rule and throw its answer away, recording the difference. Nobody is affected; you get real production data.
2. **Percentage rollout.** Serve the new rule to a small, **stable** slice of users.
3. **Kill switch.** One config change returns everyone to the old rule in under a minute.

The detail that matters: bucketing must be deterministic. If you use `Math.random()`, a user sees the old price on one request and the new price on the next, which is a support nightmare and makes the experiment meaningless. Hash the user id instead — the same user always lands in the same bucket.

shadow.js — deterministic buckets, a percentage rollout, shadow comparison

JS

```
const strategies = new Map([
  ["v1_flat", r => Math.round(r.km * 1400)],
  ["v2_slab", r => 5000 + Math.max(0, Math.ceil(r.km - 2)) * 1200],
]);

// deterministic bucketing: the same user always lands in the same bucket
function bucketOf(userId, buckets = 100) {
  let h = 2166136261; // FNV-1a, 32-bit
  for (let i = 0; i < userId.length; i++) {
    h ^= userId.charCodeAt(i);
    h = Math.imul(h, 16777619) >>> 0;
  }
  return h % buckets;
}

function price(userId, ride, cfg, log) {
  const inTest = bucketOf(userId) < cfg.rolloutPercent;
  const chosen = inTest ? cfg.candidate : cfg.control;
  const served = strategies.get(chosen)(ride);
  if (cfg.shadow && !inTest) { // run it, throw the answer away
    const shadow = strategies.get(cfg.candidate)(ride);
    log.push({ userId, served, shadow,
      deltaBp: Math.round((shadow - served) * 10000 / served) });
  }
  return { userId, bucket: bucketOf(userId), strategy: chosen, paise: served };
}

const log = [];
const cfg = { control: "v1_flat", candidate: "v2_slab", rolloutPercent: 10, shadow: true };
for (const u of ["u-1001", "u-1002", "u-1003", "u-1004", "u-1005", "u-1006"])
  console.log(price(u, { km: 12.4 }, cfg, log));
```

```
const avg = Math.round(log.reduce((s, r) => s + r.deltaBp, 0) / log.length);
console.log(`shadow samples: ${log.length}, average difference ${avg} bp =
${(avg/100).toFixed(2)}%`);
```

node shadow.js

TEXT

```
{ userId: 'u-1001', bucket: 1, strategy: 'v2_slab', paise: 18200 }
{ userId: 'u-1002', bucket: 44, strategy: 'v1_flat', paise: 17360 }
{ userId: 'u-1003', bucket: 63, strategy: 'v1_flat', paise: 17360 }
{ userId: 'u-1004', bucket: 58, strategy: 'v1_flat', paise: 17360 }
{ userId: 'u-1005', bucket: 77, strategy: 'v1_flat', paise: 17360 }
{ userId: 'u-1006', bucket: 20, strategy: 'v1_flat', paise: 17360 }
shadow samples: 5, average difference 484 bp = 4.84%
```

Reading the output. u-1001 landed in bucket 1, under 10, so it gets the new rule. The other five are controls and went through the shadow comparison. The new rule is on average 484 basis points — 4.84% — dearer:

$$(18,200 - 17,360) \times 10,000 / 17,360 = 483.9 \approx 484 \text{ bp}$$

What you check before raising the percentage.

the mean	is the change what product asked for? 4.84% when they wanted 2% means the rule is wrong — and nobody was charged to find that out
the tail, not the mean	a mean of +4.84% can hide a p99.9 of +400%. Those are the cases that become screenshots
sign flips	any ride that got cheaper when it should have got dearer is a logic bug, not a tuning question
business metrics in the test arm	completed rides, cancellations, repeat rate. A price rise that is arithmetically perfect and loses 12% of rides is still a failure

The rollout ladder. At 5 million rides a day:

$$1\% = 50,000 \text{ rides} \quad 10\% = 500,000 \quad 50\% = 2,500,000$$

One day at 1%, two days at 10%, two days at 50%, then 100%. At 50,000 rides you get about 50 samples at p99.9 — barely enough to notice a problem, which is exactly why you do not start at 1% and stop looking.

The trade-off, decided. Shadow mode doubles pricing CPU for every control request. At 174 quotes a second and a sub-millisecond calculation that is free, so run it on everything. If the candidate made a network call, you would sample instead — shadow 5% of requests, not all of them.

Answer: Shadow first (no user affected), then deterministic hash buckets 1/10/50/100%, with a kill switch. Watch the tail and the business metric, not just the mean.

Example 25 [Uber · pattern]

A saga is a State machine with compensations. Placing an order touches payments, inventory and shipping — three services, so no single transaction. Design it, and compute how often you will have to compensate.

SOLUTION

Why the obvious answer is wrong. A distributed transaction (two-phase commit) holds locks in all three services for the length of the slowest one, and a coordinator that dies mid-commit leaves them held. At any real order rate this is unusable.

The saga. Break it into local transactions, each with a **compensating** action.

Step	Forward action	Compensation
1	reserve stock	release stock

2	authorise payment	void the authorisation
3	create shipment	cancel the shipment
4	capture payment	refund (a new transaction, not an undo)

It is a State machine. The saga's state is the column, the steps are the transitions, and the conditional UPDATE from Example 20 is still the guard. Failure at step k runs compensations for steps $k - 1$ down to 1, **in reverse order**.

How often? At 99.9% per step:

$$P(\text{all 4 succeed}) = 0.999^4$$

$0.999^2 = 0.998001$, and $0.998001^2 = 0.996006$, so

$$P(\text{at least one fails}) = 1 - 0.996006 = 0.003994 = 0.3994\%$$

At 1 million orders a day:

$$1,000,000 \times 0.003994 = 3,994 \approx 4,000 \text{ compensating flows per day}$$

Four thousand a day is far too many for humans, so compensation must be automatic, retried and monitored — yet small enough that the 0.1% of compensations that themselves fail is 4 cases a day, which a person can handle.

The degraded case. If one service drops to 99%:

$$0.99 \times 0.999^3 = 0.98703 \rightarrow 1.3\% \rightarrow 13,000 \text{ per day}$$

A 0.9 percentage-point drop in one dependency tripled the compensation load. The saga's own health is a dashboard, not an afterthought.

Four rules, each with a reason:

1. **Every step and compensation is idempotent**, keyed on (sagaId, step). Retries are certain, so they must be harmless.
2. **Compensations run in reverse order.** Releasing stock before voiding the payment can leave a customer charged for an item that was resold in between.
3. **Some steps can only be offset, not undone.** A captured payment is refunded. Order the saga so irreversible steps come **last**, and there is much less to compensate.
4. **After N failed compensation attempts, escalate to a human queue** with the full saga state. Silent failure in a money flow is the worst outcome available.

Decision: saga over two-phase commit. Two-phase commit buys atomicity and pays with availability. The saga gives up atomicity — there is a window where stock is reserved and payment is not yet taken — and buys availability, because each service commits locally and moves on. For e-commerce that window is invisible and the availability is worth money. **For a bank moving money between two accounts in one database I would use a real transaction instead** — the trade only makes sense when the services are genuinely separate.

Answer: Saga = State machine + reverse-order compensations. At 99.9% per step, 0.4% of 1 M orders = about 4,000 compensations/day, so automate them and irreversible steps go last.

Example 26 [Adobe · pattern]

When NOT to use a pattern. An interviewer says: “make this extensible.” Give three cases where the correct answer is “no pattern”, and the rule that tells you which case you are in.

SOLUTION

Case 1 — there is exactly one implementation and there always will be. A Strategy interface with one class is not extensibility, it is **indirection**: a reader opens two files to learn one thing, and the interface constrains nothing because it was designed from a single example. **Say:** “I would write it as a plain method. When the second rule appears, extracting it is a ten-minute refactor — and by then I will know what the interface should look like.”

Case 2 — the variation is data, not behaviour. Tax **rates** per country vary; the **calculation** does not. That is a lookup table, not twelve strategy classes. Twelve rows can be changed by a non-engineer, reviewed in a

spreadsheet and tested in bulk; twelve classes need a deploy each. Only a genuinely different **algorithm** — compound tax on top of a levy — earns a class.

Case 3 — the flexibility is for a requirement nobody has asked for. A plugin system built because “we might support another database one day” costs an abstraction layer that leaks anyway, and the day usually never comes. Meanwhile every feature is slower to write.

THE RULE OF THREE, AND THE COST SENTENCE

Write it concretely the first time. Copy it the second time. Extract the pattern the third time. By the third case you can see what actually varies, so the interface you extract is the right one.

Say the cost out loud too: “a pattern costs one more file, one more indirection, and one more thing for a new joiner to learn. Worth paying when the change is **already happening**; a loss when it is speculative.”

The answer that scores when an interviewer pushes “but make it extensible”: “I can. The seam would go here — `quote(ride)` on an injected object — and it is a small change whenever we need it. Today there is one rule, so I would leave it concrete and spend the time on the genuinely hard part, which is the concurrency on the booking row. If you would rather see the extensible version, I will write it now.”

That proves you know the pattern, proves you know its cost, makes a decision, and hands control back. It beats both “yes, here are five interfaces” and a flat “no”.

Answer: One implementation; variation that is data not behaviour; speculative flexibility. Rule of three: concrete, copy, then extract.

PRACTICE · TIER 3 target: 40 min

- 60 pods each run a “singleton” job that writes one summary row per customer, for 500,000 customers. How many rows are written? Give the one-line fix that makes it harmless even if the coordination fails.
- A dependency times out after 20 s. Traffic is 3,000 rps. Calls in flight without a breaker? With a breaker that fails in 1 ms?
- A saga has 6 steps, each 99.5% reliable. What fraction of sagas need compensation? At 2 million sagas a day, how many per day?
- A shadow run shows the candidate strategy is on average 1% dearer but its p99.9 is 8x the control. Ship it or not, and what do you do next?
- 3 retries at each of 3 layers, with a circuit breaker at only the middle layer. Roughly what amplification survives?
- An interviewer asks you to “use the Visitor pattern”. You have never needed it. What do you say?

ANSWER KEY

1. $60 \times 500,000 = 30,000,000$ rows instead of 500,000 — 60 copies of each. The one-line fix: a **unique index on (reportDate, customerId)**, with the writer using “insert, ignore duplicates”. Then even if all 60 pods run, exactly one row per customer survives. Coordination becomes an optimisation instead of a correctness requirement.

2. Without: $L = 3,000 \times 20 = 60,000$ calls in flight. With: $L = 3,000 \times 0.001 = 3$. A reduction of 20,000x.

3. 0.995^6 . Step by step: $0.995^2 = 0.990025$, $0.995^3 = 0.985075$, and $0.985075^2 = 0.970373$. So $1 - 0.970373 = 0.029627 = 2.96\%$ need compensation. At 2 million a day: $2,000,000 \times 0.029627 = 59,254$ sagas a day, about $59,254 / 86,400 = 0.69$ per second. Sixty thousand compensations a day means the compensation path is a **main** code path and must be as well tested as the forward path.

4. **Do not ship.** A 1% mean with an 8x tail means a small number of users are being charged enormously more, and those are the cases that become screenshots. Next step: pull the worst 100 shadow samples and look for the shared input — it is almost always one branch of the new rule (a very short ride, a zero-distance ride, a missing field) hitting a divide or a minimum that was never considered. Fix that branch, re-shadow, then start the percentage rollout.

5. The breaker stops the amplification **below** it once it opens. Before it opens you get the full $3 \times 3 \times 3 = 27x$. After it opens, the middle layer stops calling downward, so the bottom layer sees roughly nothing and the top two layers still amplify $3 \times 3 =$

9x against the middle layer’s fast failures. The real answer to say: “a breaker limits the **duration** of the amplification, not its peak. To limit the peak I need a retry budget — at most 10% of traffic may be retries — at every layer.”

6. “I have not used Visitor in production, so let me say what I understand it to be and you can correct me: it lets you add a new **operation** over a fixed set of node types without editing those types — each node has an `accept(visitor)` and the visitor has one method per node type. It fits compilers and document trees, where the type set is stable and the operations keep growing. It is a poor fit when new node **types** appear often, because every visitor then needs a new method. For this problem I do not think the type set is fixed, so I would not reach for it — but tell me if you are seeing a case I am missing.”

Honesty plus a correct definition plus a judgement beats bluffing. Interviewers ask about rare patterns precisely to see what you do when you do not know.

Interview drill — what the interviewer pushes on

The push	The answer that scores
“Which pattern would you use here?”	“First, what varies: (the thing). I would put it behind a (method name) interface and inject it. That is Strategy. Adding a new case becomes a new class instead of an edit to (existing class).”
“Strategy or State?”	“Strategy if the caller chooses it and it stays. State if the object replaces its own behaviour as a result of calls. Here (which) because (who decides).”
“Is Singleton bad?”	“It is a global, so it hides dependencies and makes tests order-dependent. And it means one per process , not one per cluster — 40 pods give 40 instances. I use it for a connection pool; for one-per-cluster I need a lease or an external trigger, plus a unique index so duplicates are harmless anyway.”
“How do you test this?”	“Each strategy is tested alone with a table of inputs and expected paise. The context is tested with a fake strategy. Time and randomness are injected, so the state machine’s 7-day rule is tested in microseconds.”
“What if I add a new payment provider?”	“One new adapter file plus one <code>register()</code> line. No existing file changes. The adapter is the only place that knows that provider’s field names, error codes and timeout behaviour.”
“Your Observer is synchronous. Is that okay?”	“For listeners that can fail the operation, yes — they belong inside the transaction. For the rest, no: I write an outbox row in the same transaction and a worker publishes it, so a crash between commit and notify loses nothing.”
“How do you change behaviour without a deploy?”	“Code stays in a registry; config chooses which entry and supplies bounded parameters. Versioned rows, validated against recorded traffic on write, activated on a schedule, cached 30 s with a kill switch. Config is code that skipped review, so I put the review back.”
“Two requests hit the same state transition at once.”	“The old state and version go in the WHERE clause. Zero rows updated means I lost the race. If the row is already in the state I wanted, I return 200 as idempotent; if it moved somewhere else, 409 with the current state.”
“That is a lot of patterns for one problem.”	“Agreed — I would ship two: (the one for the thing that varies most) and (the one for the structure). The others I named so you know I considered them; I would add each one on the day a second case actually appears.”
“Use pattern X.” (one you do not know)	“Here is what I understand it to be, here is the kind of problem it fits, and here is why I am not sure it fits this one. Correct me if I have it wrong.”

TRAP

Four ways candidates lose marks on pattern questions, in order of frequency:

1. **Naming without reasoning.** “I’ll use Factory” with no sentence about what varies. It reads as a memorised list.

2. **Six patterns in one design.** Nobody can read it, and it signals that you cannot prioritise.
3. **Reciting the classic catalogue's Java structure in JavaScript** — an `AbstractFactory` base class with a `createProductA()` that throws. Write the idiomatic version and **mention** the Java form.
4. **Refusing to decide.** “Both Strategy and State could work here” and then stopping. Pick one, give the reason, and name the condition that would change your mind.

ONE-PAGE REVISION CARD

The one question behind every pattern. “What varies here, and how do I let it vary without editing working code?” Find it, put it behind a small interface, inject it.

The naming move. Say what varies → the interface → injected → **then** the pattern name. Name last.

Symptom to pattern.

several ways to do one calculation	Strategy
behaviour changes with status	State
pick a class from a key	Factory (registry of builders)
optional behaviours that combine	Decorator (n classes, not 2^n)
one event, many reactions	Observer
exactly one, shared, in one process	Singleton — usually inject instead
ten constructor arguments	Builder (validate in <code>build()</code>)
a foreign API with the wrong shape	Adapter
undo / queue / retry an action	Command
checks in order, first match wins	Chain of Responsibility
same steps, one differs	Template Method
caching, laziness, access control in front	Proxy
one simple call over several subsystems	Facade

Strategy vs State. Same structure, opposite control. Strategy: chosen outside, stays. State: replaces itself from inside, and states know each other.

Decorator maths. n independent options → 2^n subclasses → use n decorators. $n = 5$: 32 vs 5. $n = 10$: 1,024 vs 10. Order matters — always say which order and why.

Observer’s three details. Return an unsubscribe function. Wrap each listener in try/catch. Iterate over a copy of the list.

State’s key line. The base class **refuses by default**, so a transition you did not write fails loudly. A terminal state needs no code at all.

Java vocabulary to say out loud. `interface` = contract with no code (JS: a base class whose methods throw). `abstract class` = partly implemented. JS has no overloading, no `implements`, but has real `#private` fields.

What the network costs each pattern.

Strategy	becomes versioned, bounds-checked config + a code registry; cache 30 s, keep a kill switch
Observer	becomes a topic: at-least-once, so idempotency keys, DLQ, partition by entity id, outbox
State	becomes a column: guard with <code>WHERE status = ? AND version = ?</code>
Singleton	means nothing across processes: external trigger or lease, plus a unique index
Decorator	becomes middleware; <code>WithTimeout</code> inside <code>WithRetry</code> , both inside the breaker

Numbers to quote.

40 pods, 1 “singleton” job	40 runs — $40 \times 2 \text{ M} = 80 \text{ M}$ emails
breaker, 2,000 rps, 30 s timeout	$L = 60,000$ in flight → 2 with a breaker
3 retries at 3 layers	$3^3 = 27\times$ amplification

saga, 4 steps at 99.9%	$1 - 0.999^4 = 0.4\% \rightarrow 4,000 \text{ compensations per million}$
200 M events, 6 consumers	1.2 B deliveries/day = 13,889/s

Zero rows from a conditional UPDATE. Already in the target state $\rightarrow$ 200, idempotent. Moved elsewhere $\rightarrow$ 409 with the current state. Never the same answer for both.

The rule of three. Concrete the first time. Copy the second. Extract the third — by then you can see what actually varies.

Things that gain marks. Saying what varies before naming anything. Two patterns, not six. Stating decorator order and why. Money as integer paise. Time as an injected parameter. Naming what your design cannot do.

Things that lose marks. Pattern names with no reasoning. Pattern-itis. A Java AbstractFactory transliterated into JavaScript. “Both could work” with no choice.

Classic LLD Problems

Parking lot, vending machine, Splitwise, seat booking, elevators, chess — the same six moves every time

The idea in one page

WHAT A LOW-LEVEL DESIGN ROUND ACTUALLY TESTS

The interviewer is not checking whether you know a parking lot. They are checking six things, in this order:

1. Scope	Can you cut the problem down to something buildable in 40 minutes?
2. Nouns	Can you turn the story into classes with clean responsibilities?
3. The one hard part	Every classic problem has exactly one. Did you find it?
4. Code	Can you write the core method fully, not in hand-waves?
5. Change	When they add a rule, does your design bend or break?
6. Concurrency	What happens when two users do the same thing in the same millisecond?

The one hard part, per problem. Memorise this column. It is what earns the offer.

Problem	The one hard part	The pattern that solves it
Parking lot	pricing and allocation keep changing	Strategy
Vending machine	behaviour depends on state	State
Splitwise	money must never leak; many split rules	Strategy + integer money
Seat booking	two people, one seat, same moment	atomic compare-and-set + TTL hold
Elevator bank	which car, out of many?	a cost function you can tune
Chess	“is this move legal?” is not local	make-move / test / unmake (memento)

The six moves. Do them in this order, out loud, every single time.

Move 1	Cut scope. Say what you are NOT building. “No payments gateway, no login, single building.” The interviewer will stop you if they disagree — that is the point.
Move 2	Nouns to classes, verbs to methods. Read the problem sentence by sentence and underline them.
Move 3	Draw the boxes. Who owns whom. Who knows about whom. Ten boxes maximum.
Move 4	Name the one hard part and say which pattern you are spending on it. One or two patterns, not six.
Move 5	Write the core method in full. The <code>allocate</code> , the <code>settle</code> , the <code>dispatch</code> . Not the getters.
Move 6	Say the concurrency story and one extension. “Two cars at the gate” and “now add electric charging bays.”

TRAP

The most common way to fail this round is to spend 25 minutes writing `getName()` and `setName()` for eight classes, then have four minutes left for the part that actually matters. Write the **hard** method first. Say out loud: “getters and setters assumed.”

SHORTCUT

Responsibility test in one sentence. For every class, finish this line: “This class is the only place that knows how to ____.” If you cannot finish it, the class is decoration. If you can finish it two different ways, split the class in two.

Money, time and identity — three rules that are never negotiable

GET THESE WRONG AND THE DESIGN IS WRONG, HOWEVER PRETTY THE CLASSES ARE

1. **Money is an integer of the smallest unit.** Rupees become paise. $0.1 + 0.2$ in JavaScript is 0.30000000000000004 . Never store money in a float.
2. **Time is a single monotonic number** you pass in, never `Date.now()` read inside a method. If time is a parameter, you can unit-test 24-hour parking without waiting a day.
3. **Every entity has an id you generate, not a field you hope is unique.** Number plates repeat across states. Seat “A5” repeats in every screen in the country.

Warm-up

WARM-UP

build the reflex

Example 1

A parking lot problem says: “A vehicle arrives at a gate, an attendant gives it a ticket, the vehicle occupies a spot, and on exit the driver pays a fee.” List the nouns and the verbs.

SOLUTION

Read it word by word and underline.

Nouns: vehicle, gate, attendant, ticket, spot, driver, fee.

Verbs: arrives, gives, occupies, exits, pays.

Now cut. “Attendant” and “driver” are humans who do not hold state that our program needs — fold them into Gate. “Fee” is not a thing that lives; it is a number computed from a ticket. So it becomes a method, or better, a strategy object.

Classes that survive: Vehicle, Gate, Ticket, Spot, FeeStrategy, and a ParkingLot that owns the spots.

Answer: 6 classes: Vehicle, Gate, Ticket, Spot, FeeStrategy, ParkingLot

Example 2

Should SpotSize be a class or an enum? Should FeeStrategy be an enum or a class?

SOLUTION

Ask one question: **does it carry behaviour that differs per value?**

SpotSize has three values (SMALL, MEDIUM, LARGE) and the only thing you ever do with them is compare them. No behaviour. So: enum (in JavaScript, a frozen object of constants).

FeeStrategy has values whose **behaviour** differs — flat rate computes differently from a slab rate. So: a class per strategy, with the same method name.

The rule: same data, different behaviour → class. Same behaviour, different label → enum.

Answer: SpotSize = enum, FeeStrategy = one class per rule

Example 3

Here is a bad design. Name the smell and fix it in one line.

```
class ParkingLot {
    fee(ticket) {
        if (this.city === "BLR") return ...
        else if (this.city === "DEL") return ...
        else if (this.city === "BOM") return ...
    }
}
```

SOLUTION

Smell: an if-chain over a **type code**. Every new city edits a class that has nothing to do with cities. That breaks the open/closed rule: open to extension, closed to modification.

Fix: ParkingLot holds a pricing object. Adding Chennai means adding a class, not editing ParkingLot.

```
const lot = new ParkingLot(spots, allocator, new SlabRate(40, 25, 300));
```

Answer: Type-code if-chain → inject a strategy object

Example 4

A vending machine ignores the “select item” button until money is in. Where does that rule live?

SOLUTION

Two wrong places and one right one.

Wrong 1: in the button handler (`if (machine.credit === 0) return;`). Then every new rule adds another `if` to the same handler.

Wrong 2: a mode string checked everywhere. Same problem, spread wider.

Right: the machine holds a **state object**, and `select()` is a method on that object. The Idle state’s `select()` simply refuses. Nobody asks “what mode am I in?” — the current state already knows.

Answer: In the Idle state class’s own `select()` method

Example 5

Three friends split a bill of Rs.10.00 equally. Show why `total / 3` is a bug and give the fix.

SOLUTION

Work in paise. Total = 1000 paise.

$1000 \div 3 = 333.33\dots$ — not a whole number of paise.

If each person owes 333, the sum is $333 \times 3 = 999$. One paisa has vanished. Over a million splits that is Rs.100 that the ledger cannot explain.

Fix: integer division plus a remainder spread.

$\text{base} = \lfloor 1000/3 \rfloor = 333$, remainder = $1000 - 333 \times 3 = 1$.

Give the first 1 person an extra paisa: shares are 334, 333, 333. Sum = 1000. Exact.

Answer: Floor-divide, then hand the leftover paise out one each. Sum must equal the total.

Example 6

Two people click “book seat C7” at the same instant. Your code is `if (seat.isFree) seat.book(user)`. What goes wrong?

SOLUTION

Trace the interleaving, line by line, with a clock ticking between lines.

t	Request 1	Request 2
1	reads <code>seat.isFree</code> → true	—

2	—	reads <code>seat.isFree</code> → true
3	writes <code>holder = u1</code>	—
4	—	writes <code>holder = u2</code>
5	returns “you got C7”	returns “you got C7”

Both were told yes. Only u2 holds the seat. This is a **read-modify-write race**, and it is the number-one bug in booking systems.

Fix: one atomic operation that reads and writes together — a conditional UPDATE, a Redis SET NX, or a unique index that rejects the second insert.

Answer: Both succeed; the seat is double-sold. Needs one atomic compare-and-set.

Example 7

In chess, why can you not decide “is this knight move legal?” by only looking at the knight?

SOLUTION

Because of the **pin**. A knight may be standing between your own king and an enemy rook. The knight’s own movement rule says the jump is fine. But making the jump exposes the king, so the move is illegal.

Legality is a property of the whole board after the move, not of the piece before it.

That is why the code is always: make the move, ask “is my king attacked?”, unmake the move. Three steps, never one.

Answer: Legality depends on the whole board after the move — make, test, unmake

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Tier 1 · Design a parking lot

Move 1 — cut the scope

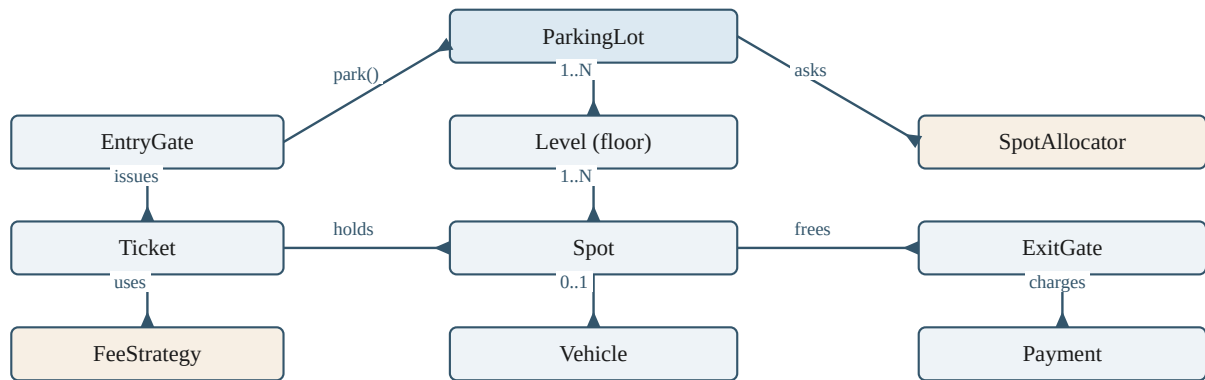
Say this out loud in the first 60 seconds:

“I will build one building with several floors. Three spot sizes. A vehicle takes the smallest free spot that fits, on the lowest floor. Pricing is hourly with a daily cap. I am **not** building: payment gateways, number-plate cameras, reservations, or multiple buildings. If you want any of those, tell me and I will make room.”

The questions to ask back, and why each one changes the design:

“Can a big vehicle take several small spots?”	If yes, a <code>Ticket</code> holds a list of spots, not one. That changes <code>Ticket</code> , <code>Spot.free</code> , and the allocator. Huge difference — ask first.
“Is pricing per started hour or per minute?”	Decides whether <code>fee()</code> uses <code>Math.ceil</code> or plain division. Also decides whether the daily cap is even reachable.
“Do I need to find a specific car later?”	If yes I need a <code>plate</code> → <code>ticket</code> index. If no, I skip it and save a map.
“How many spots?”	200 spots means a linear scan is fine. 200,000 spots means I need free-lists per (floor, size).

Move 2 and 3 — the boxes



Who owns whom. Solid arrows point from owner to owned, or from caller to callee.

The two shaded boxes on the sides are the **plug points**. Everything the interviewer will ask you to change lives in one of them.

Move 4 and 5 — the code

Sizes, entities, and the two plug points

JS

```

const SpotSize = { SMALL: 0, MEDIUM: 1, LARGE: 2 };

class Vehicle {
  constructor(plate, size) { this.plate = plate; this.size = size; }
}

class Spot {
  constructor(id, floor, size) {
    this.id = id; this.floor = floor; this.size = size;
    this.vehicle = null;
  }
  get free() { return this.vehicle === null; }
  fits(v) { return this.free && this.size >= v.size; } // a car may use a large
spot
}

class Ticket {
  constructor(id, vehicle, spot, inAt) {
    this.id = id; this.vehicle = vehicle; this.spot = spot;
    this.inAt = inAt; this.outAt = null;
  }
}

// ---- plug point 1: pricing ----
class FlatRate {
  constructor(rate) { this.rate = rate; }
  fee(minutes) { return Math.ceil(minutes / 60) * this.rate; }
}

class SlabRate {
  constructor(firstHour, perExtraHour, dayCap) {
    this.firstHour = firstHour; this.perExtraHour = perExtraHour; this.dayCap =
dayCap;
  }
  fee(minutes) {
    if (minutes <= 60) return this.firstHour;
  }
}

```

```

    const extra = Math.ceil((minutes - 60) / 60);
    return Math.min(this.firstHour + extra * this.perExtraHour, this.dayCap);
  }
}

// ---- plug point 2: which spot ----
class NearestFreeFirst {
  pick(spots, vehicle) {
    let best = null;
    for (const s of spots) {
      if (!s.fits(vehicle)) continue;
      if (best === null) { best = s; continue; }
      if (s.floor < best.floor) { best = s; continue; }
      if (s.floor === best.floor && s.size < best.size) best = s; // smallest that
fits
    }
    return best;
  }
}

```

The lot itself — park() and unpark() written in full

JS

```

class ParkingLot {
  constructor(spots, allocator, pricing) {
    this.spots = spots; this.allocator = allocator; this.pricing = pricing;
    this.open = new Map(); // ticketId -> Ticket
    this.nextTicket = 1;
    this.freeBySize = new Map(); // so isFull() is O(1), not a scan
    for (const s of spots)
      this.freeBySize.set(s.size, (this.freeBySize.get(s.size) || 0) + 1);
  }

  park(vehicle, atMinute) {
    const spot = this.allocator.pick(this.spots, vehicle);
    if (spot === null) return { ok: false, reason: "LOT_FULL" };
    spot.vehicle = vehicle;
    this.freeBySize.set(spot.size, this.freeBySize.get(spot.size) - 1);
    const t = new Ticket(this.nextTicket++, vehicle, spot, atMinute);
    this.open.set(t.id, t);
    return { ok: true, ticket: t };
  }

  unpark(ticketId, atMinute) {
    const t = this.open.get(ticketId);
    if (!t) return { ok: false, reason: "NO_SUCH_TICKET" };
    t.outAt = atMinute;
    const minutes = t.outAt - t.inAt;
    const amount = this.pricing.fee(minutes);
    t.spot.vehicle = null;
    this.freeBySize.set(t.spot.size, this.freeBySize.get(t.spot.size) + 1);
    this.open.delete(ticketId);
    return { ok: true, minutes, amount, spot: t.spot.id };
  }
}

```

Run the lot. Four spots: F0-S1 (floor 0, small), F0-M1 (floor 0, medium), F1-M1 (floor 1, medium), F1-L1 (floor 1, large). Pricing: Rs.40 for the first hour, Rs.25 per started hour after that, daily cap Rs.300. A bike parks at 09:47 and leaves at 14:12. What does it pay?

SOLUTION

Step 1 — allocation. A bike is SMALL. NearestFreeFirst scans:

- F0-S1: fits (size $0 \geq 0$), floor 0 — best so far.
- F0-M1: fits, floor 0, but size 1 is **not** smaller than 0 — keep F0-S1.
- F1-M1, F1-L1: floor 1 is worse. Keep F0-S1.

So the bike gets F0-S1.

Step 2 — duration. Convert clock times to minutes past midnight.

09:47 becomes $9 \times 60 + 47 = 540 + 47 = 587$

14:12 becomes $14 \times 60 + 12 = 840 + 12 = 852$

minutes = $852 - 587 = 265$

Step 3 — fee. $265 > 60$, so use the slab branch.

extra minutes = $265 - 60 = 205$

extra hours = $\lceil 205/60 \rceil = \lceil 3.4166... \rceil = 4$

fee = $40 + 4 \times 25 = 40 + 100 = 140$

Cap check: $\min(140, 300) = 140$.

Answer: Spot F0-S1, 265 minutes, Rs.140

node parking.js — actual output

JS

```
bike spot: F0-S1 SMALL
car  spot: F0-M1 MEDIUM
van  spot: F1-L1 LARGE
bike stayed 265 min, pay 140
24h stay pay: 300
```

Example 9 [Infosys · pattern]

A vehicle stays exactly 24 hours (1440 minutes). Show that the cap binds, and find the exact stay length at which the cap first binds.

SOLUTION

24-hour stay.

extra minutes = $1440 - 60 = 1380$

extra hours = $\lceil 1380/60 \rceil = \lceil 23 \rceil = 23$

uncapped fee = $40 + 23 \times 25 = 40 + 575 = 615$

fee = $\min(615, 300) = 300$. The cap binds.

Where does it first bind? We need $40 + 25h \geq 300$, where h is extra hours.

$25h \geq 260$

$h \geq 10.4$, so the first whole h is 11.

$h = 11$ means $\lceil \text{extra minutes} / 60 \rceil = 11$, which first happens at extra minutes = 601.

total minutes = $60 + 601 = 661$ minutes = 11 hours 1 minute.

Check $h = 10$: $40 + 250 = 290 < 300$. Check $h = 11$: $40 + 275 = 315$, capped to 300. Correct.

Answer: 24 h costs 300 (capped). The cap first binds at 11 h 1 min of stay.

Move 6 — the follow-ups they will actually ask

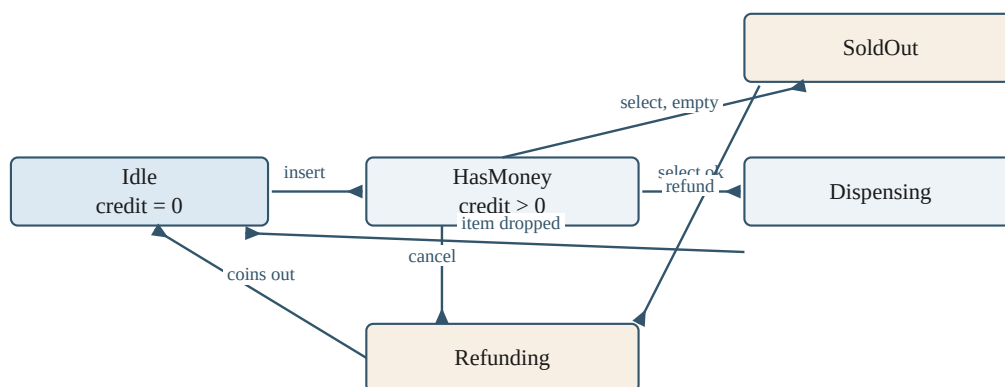
“Add electric charging bays.”	Spot gets a features: Set and fits() also checks the vehicle’s required features. The allocator does not change. One field, zero edits to ParkingLot.
“Weekend pricing is different.”	New class WeekendRate implementing fee(minutes). Zero edits anywhere else. This is why pricing was a plug point.
“200,000 spots — the scan is too slow.”	Replace NearestFreeFirst with FreeListAllocator that keeps one array of free spots per (floor, size) pair. pick becomes pop from the lowest non-empty list: $O(1)$ instead of $O(n)$. Zero edits to ParkingLot — same interface.
“Two cars at the gate at once.”	park() does a read-modify-write on spot.vehicle. With real threads, wrap the pick-and-claim in one lock per floor, or make the free-list a concurrent queue so pop is atomic. Do NOT lock the whole lot — that serialises 200,000 spots behind one mutex.

Time park() = $O(n)$ scanning, $O(1)$ with free lists; unpark() = $O(1)$ **Space** $O(n)$ spots + $O(\text{open tickets})$
 n = number of spots. Free lists trade about 8 bytes a spot for a 200,000x faster pick.

Tier 1 • Design a vending machine

The one hard part here is **state**. The same button does different things depending on what happened before. That is the definition of a state machine, and the State pattern is the only clean way to code one.

The machine drawn as states



Each box is a class. Each arrow is a method on that class that changes this.state.

THE STATE PATTERN, IN FOUR LINES

1. The machine holds the **data** (credit, stock, coin bank) and a pointer this.state.
2. Each state is a small class with the same method names: insert, select, cancel, tick.
3. A base State class makes every method say “you cannot do that now”. A concrete state overrides only the methods that are legal in it.
4. A transition is one line: m.state = m.DISPENSING. Nothing else in the program branches on the state.

Base state: the default answer is 'no'

JS

```

class State {
  constructor(name) { this.name = name; }
  insert(m, coin) { return `${this.name}: cannot take coins now`; }
}
  
```

```

select(m, code) { return `${this.name}: cannot select now`; }
cancel(m)       { return `${this.name}: nothing to cancel`; }
tick(m)         { return `${this.name}: idle tick`; }
}

class Idle extends State {
  constructor() { super("Idle"); }
  insert(m, coin) { m.credit += coin; m.state = m.HAS_MONEY; return `credit = ${m.credit}`; }
}

class Dispensing extends State {
  constructor() { super("Dispensing"); }
  tick(m) { m.state = m.IDLE; return "item dropped -> Idle"; }
}

class Refunding extends State {
  constructor() { super("Refunding"); }
  tick(m) { const r = m.credit; m.credit = 0; m.state = m.IDLE; return `refunded ${r} -> Idle`; }
}

```

The interesting state: every rejection path is explicit

JS

```

class HasMoney extends State {
  constructor() { super("HasMoney"); }
  insert(m, coin) { m.credit += coin; return `credit = ${m.credit}`; }
  cancel(m) { m.state = m.REFUNDING; return m.state.tick(m); }
  select(m, code) {
    const slot = m.slots.get(code);
    if (!slot) { m.state = m.REFUNDING; return `no such code ${code}; refunding`; }
    if (slot.qty === 0) { m.state = m.REFUNDING; return `${code} sold out; refunding`; }
    if (m.credit < slot.price) return `need ${slot.price - m.credit} more`;

    const change = m.makeChange(m.credit - slot.price);
    if (change === null) { m.state = m.REFUNDING; return `no change available; refunding`; }

    slot.qty -= 1;
    for (const c of change) m.bank.set(c, m.bank.get(c) - 1);
    m.credit = 0;
    m.state = m.DISPENSING;
    return `dispensing ${code}, change ${JSON.stringify(change)}`;
  }
}

```

The machine: data + a change-maker that respects the real coin bank

JS

```

class VendingMachine {
  constructor(slots, bank) {
    this.IDLE = new Idle();           this.HAS_MONEY = new HasMoney();
    this.DISPENSING = new Dispensing(); this.REFUNDING = new Refunding();
    this.state = this.IDLE;
  }
}

```

```

    this.credit = 0;
    this.slots = new Map(Object.entries(slots)); // code -> {price, qty}
    this.bank = new Map(Object.entries(bank).map(([k, v]) => [Number(k), v]));
  }
  // greedy over the coins we ACTUALLY HAVE, largest first
  makeChange(amount) {
    if (amount === 0) return [];
    const coins = [...this.bank.keys()].sort((a, b) => b - a); // numeric sort!
    const used = [];
    const left = new Map(this.bank);
    for (const c of coins) {
      while (amount >= c && left.get(c) > 0) {
        amount -= c; left.set(c, left.get(c) - 1); used.push(c);
      }
    }
    return amount === 0 ? used : null; // null = "I cannot pay this exactly"
  }
}

```

TRAP

`[...this.bank.keys()].sort()` with no comparator sorts **as text**. `[1, 5, 10]` becomes `[1, 10, 5]`, so the machine would try the 1-coin before the 10-coin and hand out a fistful of change. Numbers always need `(a, b) => a - b` (or `b - a` for descending). This is the single most common JavaScript bug in an interview.

Example 10 [Capgemini · pattern]

The machine holds coins $\{1 \times 5, 5 \times 4, 10 \times 3\}$. Slot A1 costs 25, A2 costs 40 but has `qty = 0`. Trace this button sequence: select A1, insert 10, insert 10, select A1, insert 10, select A1, tick, insert 50, select A2, tick.

SOLUTION

Step	State before	State after	Why
select A1	Idle	Idle	base <code>State.select</code> refuses — no money
insert 10	Idle	HasMoney	credit $0 \rightarrow 10$
insert 10	HasMoney	HasMoney	credit 20
select A1	HasMoney	HasMoney	$20 < 25$, asks for 5 more, no transition
insert 10	HasMoney	HasMoney	credit 30
select A1	HasMoney	Dispensing	$30 \geq 25$, change = 5
tick	Dispensing	Idle	item fell, reset
insert 50	Idle	HasMoney	credit 50
select A2	HasMoney	Refunding	<code>qty === 0</code>
tick	Refunding	Idle	50 returned

The change step, in full. Credit 30, price 25, so change owed = $30 - 25 = 5$.

Coins sorted descending: `[10, 5, 1]`.

- Try 10: $5 \geq 10$? No. Skip.
- Try 5: $5 \geq 5$ and bank has 4 fives. Take one. Now $5 - 5 = 0$ left, bank fives = 3.
- Try 1: $0 \geq 1$? No. Skip.

Amount left is 0, so return `[5]`. Success.

Answer: Ends back in Idle. Change handed out is a single 5-coin.

node vending.js — actual output

JS

```
Idle      --select(A1)--> Idle      | Idle: cannot select now
Idle      --insert(10)--> HasMoney   | credit = 10
HasMoney  --insert(10)--> HasMoney   | credit = 20
HasMoney  --select(A1)--> HasMoney   | need 5 more
HasMoney  --insert(10)--> HasMoney   | credit = 30
HasMoney  --select(A1)--> Dispensing | dispensing A1, change [5]
Dispensing --tick()--> Idle         | item dropped -> Idle
Idle      --insert(50)--> HasMoney   | credit = 50
HasMoney  --select(A2)--> Refunding  | A2 sold out; refunding
Refunding --tick()--> Idle         | refunded 50 -> Idle
```

Example 11 [Wipro · pattern]

The greedy change-maker can fail even when exact change is possible. Give a coin bank where it fails, and decide what to ship.

SOLUTION

Take a bank of $\{25 \times 1, 10 \times 3, 1 \times 0\}$ and change owed = 30.

Greedy, largest first:

- Try 25: $30 \geq 25$, take it. Left = 5. Bank: 25-coin gone.
- Try 10: $5 \geq 10$? No.
- Left is 5, not 0 $\rightarrow$ greedy returns null, “no change”.

But $10 + 10 + 10 = 30$ works. Greedy was wrong.

The exact answer is a bounded-coin dynamic program: $\text{dp}[a]$ = fewest coins to make a , filled for $a = 0 \dots \text{amount}$, decrementing each coin’s count. Cost is $O(\text{amount} \times \text{coin types})$.

Decision. Ship greedy, and here is why, stated as a trade-off with both sides named:

Greedy	Wins: 5 lines, $O(k \log k)$, obviously correct to a reviewer. Loses: can refuse change it could actually make.
Bounded DP	Wins: never refuses when a solution exists. Loses: 25 lines, an array of size <code>amount</code> , and a bug surface in a machine that must never hang.

Real machines use **denominations that are canonical** (1, 2, 5, 10, 20, 50). For a canonical set, greedy is provably optimal, so the failure case never arises. I would ship greedy plus one unit test asserting the coin set is canonical, and put the DP behind a feature flag for markets with odd coins. That keeps the hot path five lines long.

Answer: Greedy fails on non-canonical coin sets. Ship greedy + a canonical-coin assertion.

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Tier 2 · Design an expense-sharing app

Four friends share costs on a trip. Anyone can add an expense, say who paid, and say how to split it. The app must show, at any moment, who owes whom — and the app must never lose a paisa.

Move 1 — clarify

“How many split rules?”	Equal, exact amounts, percentages, shares. If the answer is “more later”, split rules must be a plug point, not an if-chain.
“Do I show pairwise debts or net balances?”	Pairwise (A owes B 300, B owes C 300) is what people remember. Net (A is +500) is what settles fastest. I will store net and derive settlements, because pairwise storage grows as n^2 and gets tangled after every expense.
“Can an expense be edited or deleted?”	If yes, I cannot store only running totals — I must keep every expense and be able to recompute. That is the difference between a Map of balances and an append-only event list.
“Multi-currency?”	If yes, an expense stores its own currency plus the rate at the time. Converting at read time gives different answers every day. I will say “single currency” and move on.

Move 2 — the money rules

THREE INVARIANTS. WRITE THEM ON THE WHITEBOARD BEFORE ANY CODE.

1. **Shares sum to the total.** For every expense, $\sum \text{share}_i = \text{total}$ exactly, in paise.
2. **Net balances sum to zero.** $\sum_i \text{net}_i = 0$ across all users, always. If it ever drifts, an expense was recorded wrongly.
3. **A settlement transfer is itself an expense** with payer = the debtor and a single share on the creditor. That way there is one code path, not two.

Split rules as three small pure functions — the plug point

JS

```
// Money is in PAISE (integers). Never floats.
const Equal = (total, users) => {
  const base = Math.floor(total / users.length);
  const rem = total - base * users.length; // leftover paise
  return users.map((u, i) => [u, base + (i < rem ? 1 : 0)]); // spread it, one each
};

const Exact = (total, users, amounts) => {
  const sum = amounts.reduce((a, b) => a + b, 0);
  if (sum !== total) throw new Error(`exact split ${sum} !== total ${total}`);
  return users.map((u, i) => [u, amounts[i]]);
};

const Percent = (total, users, pct) => {
  const sum = pct.reduce((a, b) => a + b, 0);
  if (sum !== 100) throw new Error(`percents sum to ${sum}, not 100`);
  const shares = pct.map(p => Math.floor(total * p / 100));
  let rem = total - shares.reduce((a, b) => a + b, 0);
  for (let i = 0; rem > 0; i = (i + 1) % users.length) { shares[i]++; rem--; }
  return users.map((u, i) => [u, shares[i]]);
};
```

The ledger: net balances, and the greedy settlement

JS

```
class Ledger {
  constructor() { this.net = new Map(); } // user -> (paid minus owed), in paise
  bump(u, d) { this.net.set(u, (this.net.get(u) || 0) + d); }
```



```

addExpense(payer, total, shares) {
  const sum = shares.reduce((a, [v]) => a + v, 0);
  if (sum !== total) throw new Error(`shares ${sum} != total ${total}`); //
invariant 1
  this.bump(payer, total);
  for (const [u, v] of shares) this.bump(u, -v);
}

// greedy: biggest creditor meets biggest debtor
settle() {
  const cred = [], debt = [];
  for (const [u, v] of this.net) {
    if (v > 0) cred.push([u, v]);
    else if (v < 0) debt.push([u, -v]);
  }
  cred.sort((a, b) => b[1] - a[1]);
  debt.sort((a, b) => b[1] - a[1]);
  const out = [];
  let i = 0, j = 0;
  while (i < cred.length && j < debt.length) {
    const pay = Math.min(cred[i][1], debt[j][1]);
    out.push([debt[j][0], cred[i][0], pay]);
    cred[i][1] -= pay; debt[j][1] -= pay;
    if (cred[i][1] === 0) i++;
    if (debt[j][1] === 0) j++;
  }
  return out;
}
}

```

Example 12 [Grab · pattern]

Four friends A, B, C, D.

1. A pays 1200 for dinner, split equally among all four.
2. B pays 900 for a cab, split equally among A, B, C only.
3. C pays 600 for a movie, split exactly: A 100, C 200, D 300.

Compute every net balance, check the zero invariant, and produce the settlement.

SOLUTION

Step 1 — turn each expense into shares (in rupees here; the code uses paise).

Expense 1: equal among 4. $1200 \div 4 = 300$ each, no remainder. Shares: A 300, B 300, C 300, D 300. Sum = 1200. ✓

Expense 2: equal among 3. $900 \div 3 = 300$ each. Shares: A 300, B 300, C 300. Sum = 900. ✓

Expense 3: exact. A 100, C 200, D 300. Sum = $100 + 200 + 300 = 600$. ✓

Step 2 — net = what you paid minus what you owed.

User	Paid	Owed	Net
A	1200	$300 + 300 + 100 = 700$	$1200 - 700 = +500$
B	900	$300 + 300 + 0 = 600$	$900 - 600 = +300$
C	600	$300 + 300 + 200 = 800$	$600 - 800 = -200$
D	0	$300 + 0 + 300 = 600$	$0 - 600 = -600$

Step 3 — the zero check.

$500 + 300 + (-200) + (-600) = 800 - 800 = 0$. ✓ The ledger is consistent.

Step 4 — settle, biggest meets biggest.

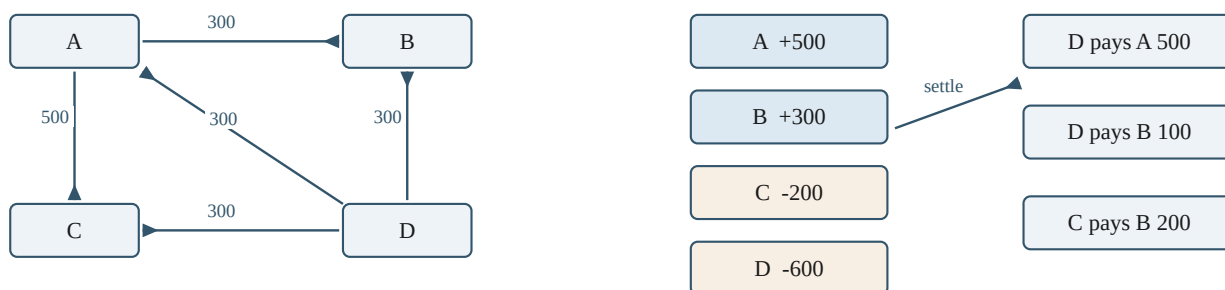
Creditors sorted: A 500, B 300. Debtors sorted: D 600, C 200.

- Round 1: D (600) meets A (500). Transfer $\min(600, 500) = 500$. D owes 100 more; A is settled. Move to creditor B.
- Round 2: D (100) meets B (300). Transfer $\min(100, 300) = 100$. D is settled; B still expects 200. Move to debtor C.
- Round 3: C (200) meets B (200). Transfer $\min(200, 200) = 200$. Both settled.

Three transfers: D → A 500, D → B 100, C → B 200.

Step 5 — check. Money out of D = $500 + 100 = 600$ = D's debt. ✓ Money into B = $100 + 200 = 300$ = B's credit. ✓

Answer: Nets: A +500, B +300, C -200, D -600. Settlement is 3 transfers.



node split.js — actual output

JS

```
net (rupees):
  A 500.00
  B 300.00
  C -200.00
  D -600.00
sum of net = 0
settlement:
  D pays A 500.00
  D pays B 100.00
  C pays B 200.00
percent split of 1000 paise 33/33/34: [ [ 'X', 330 ], [ 'Y', 330 ], [ 'Z', 340 ] ]
equal split of 1000 paise among 3: [ [ 'X', 334 ], [ 'Y', 333 ], [ 'Z', 333 ] ]
```

Example 13 [Shopee · pattern]

Is the greedy settlement the **minimum** number of transfers? Prove or disprove for the example above, and decide what to ship.

SOLUTION

Upper bound first. With n people who have non-zero balances, you never need more than $n - 1$ transfers: line everyone up and push the running imbalance along the line. Here $n = 4$, so at most 3. Greedy gave exactly 3.

Is 3 the minimum? A settlement needs fewer than $n - 1$ transfers only if the group splits into two or more **sub-groups whose balances each sum to zero**. Check every subset:

Subset	Sum	Zero?
{A, C}	$500 - 200 = 300$	no
{A, D}	$500 - 600 = -100$	no
{B, C}	$300 - 200 = 100$	no
{B, D}	$300 - 600 = -300$	no
{A, B, C}	$500 + 300 - 200 = 600$	no
{A, B, D}	$500 + 300 - 600 = 200$	no
{A, C, D}	$500 - 200 - 600 = -300$	no
{B, C, D}	$300 - 200 - 600 = -500$	no

No proper subset sums to zero, so the group cannot be split. Minimum = $n - 1 = 3$. Greedy is optimal **here**.

In general it is not. Finding the true minimum means finding the largest number of zero-sum subsets, which is the partition problem — NP-hard. A backtracking search is exponential.

settle.py — the exact minimum, exponential, for checking only

PYTHON

```
def min_transfers(net):
    debts = [v for v in net.values() if v != 0]

    def go(i, arr):
        while i < len(arr) and arr[i] == 0:
            i += 1
        if i == len(arr):
            return 0
        best = 10**9
        for j in range(i + 1, len(arr)):
            if arr[i] * arr[j] < 0:          # opposite signs can cancel
                arr[j] += arr[i]
                best = min(best, 1 + go(i + 1, arr))
                arr[j] -= arr[i]
        return best
    return go(0, debts)

case1 = {"A": 500, "B": 300, "C": -200, "D": -600}
print(case1, "-> minimum transfers =", min_transfers(case1))    # 3
```

The trade-off, both sides named, then a decision.

Greedy	Wins: $O(n \log n)$, ten lines, always correct (it always settles everyone), never more than $n - 1$ transfers. Loses: occasionally one transfer more than the theoretical best.
Exact search	Wins: provably minimal. Loses: exponential — a 20-person trip can take minutes, and the app must respond in 100 ms.

Decision: ship greedy. Real groups are 3 to 10 people, greedy is within one transfer of optimal almost always, and “one extra UPI payment” costs the user nothing while a 3-second spinner costs them the app. I would add the exact solver as a test-only oracle that asserts greedy is within 1 of optimal on random inputs.

Answer: Here greedy is optimal ($3 = n - 1$). In general it is not, but ship it anyway.

TRAP

`Equal(1000, [X, Y, Z])` returns `[334, 333, 333]` — **not** three equal numbers. If your UI shows “333.33 each”, your ledger and your screen disagree by one paisa and a user will screenshot it. Show the actual per-person integer, and always give the leftover to a **deterministic** person (sorted user id), not a random one, or the number changes each time the page reloads.

Tier 2 · Design seat booking for a cinema

Now the one hard part is not classes. It is **two users, one seat, one millisecond**.

Move 1 — clarify, then Move 2 — the numbers

“Can a user hold seats before paying?”	Yes — this is the whole design. A hold with a time-to-live is what makes payment possible without locking a seat forever.
“How long is the hold?”	5 minutes. Short enough that abandoned carts free up; long enough for a UPI payment.
“Can a booking be partly successful?”	No. Four seats or zero. That is an all-or-nothing rule and it decides whether I lock seats one by one or in one atomic step.
“Is the seat map per show or per screen?”	Per show. The same physical seat is free at 6 pm and sold at 9 pm.

SCALE ESTIMATE — SHOW EVERY DIVISION

Assumptions I state out loud: 12M daily active users; 3% of them book on a given day; each user views 8 pages; a booking row is about 400 bytes; 6,000 cinemas, 5 screens each, 5 shows a day, 200 seats a screen.

Browse traffic.

$12,000,000 \times 8 = 96,000,000$ page views a day

$96,000,000 \div 86,400 = 1,111$ requests per second (average)

Peak is roughly 5x the average for an entertainment app (evening spike): $1,111 \times 5 = 5,555$ RPS peak.

Booking traffic.

$12,000,000 \times 0.03 = 360,000$ bookings a day

$360,000 \div 86,400 = 4.2$ bookings per second (average) — tiny.

But bookings are not spread out. Say 60% land in a 3-hour evening window ($3 \times 3600 = 10,800$ seconds):

$360,000 \times 0.6 = 216,000$ bookings in that window

$216,000 \div 10,800 = 20$ bookings per second.

The number that actually matters. A blockbuster opens. 40,000 people hit **one show** in the first minute:

$40,000 \div 60 = 667$ lock attempts per second — all on the **same** handful of rows.

Average RPS is 4.2. The hot row sees 667. **Design for 667, not for 4.2.**

Storage.

Bookings: $360,000 \times 365 = 131,400,000$ rows a year.

$131,400,000 \times 400$ bytes = 52,560,000,000 bytes = 52.6 GB a year. One machine.

Seat inventory: $6,000 \times 5 \times 5 = 150,000$ shows a day.

$150,000 \times 200 = 30,000,000$ seat rows a day.

At 50 bytes a row: $30,000,000 \times 50 = 1,500,000,000$ bytes = 1.5 GB a day.

Keep 30 days hot: $1.5 \times 30 = 45$ GB. Also one machine — but a **hot** one.

Move 3 — the API surface

```

POST /v1/shows/{showId}/holds
  body  { seats: ["C7","C8"], userId: "u_991", idemKey: "5f1c-..." }
  200    { holdId: "h_88x", expiresAt: 1730000300, seats: ["C7","C8"] }
  409    { error: "SEAT_TAKEN", seats: ["C7"] }          <- name WHICH seat

POST /v1/holds/{holdId}/confirm
  body  { paymentRef: "pay_7712", idemKey: "5f1c-..." }
  200    { bookingId: "b_4410", seats: ["C7","C8"], total: 76000 }
  410    { error: "HOLD_EXPIRED" }

DELETE /v1/holds/{holdId}          -> 204, seats free immediately

```

idemKey is not decoration. A phone on a bad network retries the POST. Without an idempotency key the user gets two holds, or worse, two bookings and two charges. The server stores idemKey -> response for 24 hours and replays the stored response on a repeat.

Move 4 — the data model

Seat state lives in ONE row per (show, seat). That row is the lock.

SQL

```

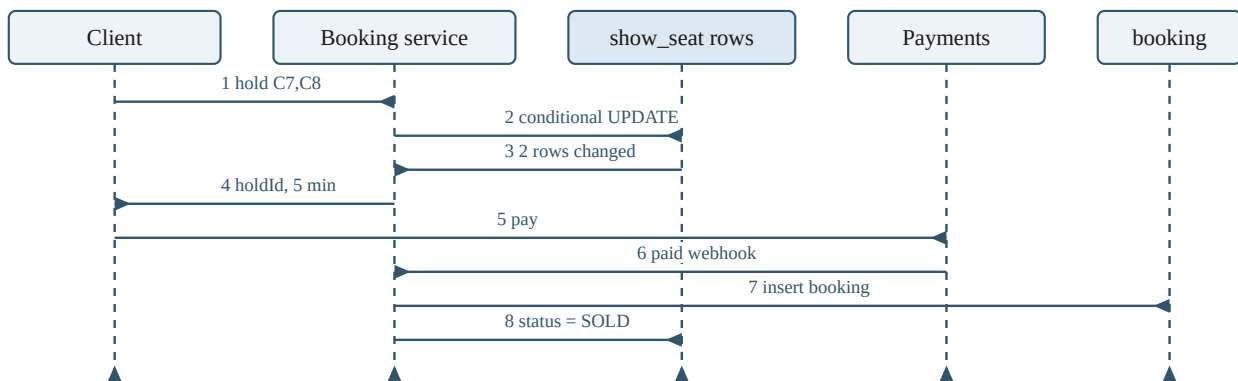
CREATE TABLE show_seat (
  show_id      BIGINT      NOT NULL,
  seat_label   VARCHAR(8)  NOT NULL,
  status       SMALLINT    NOT NULL,      -- 0 FREE, 1 HELD, 2 SOLD
  holder_id    BIGINT      NULL,
  hold_until   TIMESTAMP   NULL,
  price_paise  INT          NOT NULL,
  version      INT          NOT NULL DEFAULT 0,
  PRIMARY KEY (show_id, seat_label)
);
CREATE INDEX ix_seat_expiry ON show_seat (hold_until) WHERE status = 1;

CREATE TABLE booking (
  booking_id   BIGINT PRIMARY KEY,
  show_id      BIGINT NOT NULL,
  user_id      BIGINT NOT NULL,
  seats        TEXT    NOT NULL,          -- "C7,C8"
  total_paise  INT      NOT NULL,
  idem_key     VARCHAR(64) NOT NULL,
  created_at   TIMESTAMP NOT NULL,
  UNIQUE (idem_key)                       -- the retry guard, enforced by the database
);

```

The primary key (show_id, seat_label) is the whole trick. Because it is the primary key, the database gives you a row lock on it for free, and a conditional UPDATE on it is atomic without any application-level lock.

Move 5 — the flow



Hold, pay, confirm. The hold is taken BEFORE payment and expires on its own.

Move 6 — the deep dive: making the hold atomic

One statement. Read and write happen together, so no race exists.

SQL

```

UPDATE show_seat
  SET status = 1, holder_id = :user, hold_until = now() + interval '5 minutes',
      version = version + 1
 WHERE show_id = :show
   AND seat_label IN ('C7','C8')
   AND (status = 0 OR (status = 1 AND hold_until <= now()));
-- then check: did this statement change exactly 2 rows?
-- if not, ROLLBACK. All-or-nothing, enforced by row count.
  
```

The same idea in code, so you can see the race and the fix

JS

```

class SeatStore {
  constructor(seats) {
    this.state = new Map(seats.map(s => [s, { status: "FREE", holder: null, until: 0 }]));
  }
  // WRONG: read, decide, write. Two callers can both pass the read.
  naiveHold(seat, user, now) {
    const s = this.state.get(seat);
    if (s.status !== "FREE") return false;
    // <-- another request can run right here
    s.status = "HELD"; s.holder = user; s.until = now + HOLD_MS;
    return true;
  }
  // RIGHT: one atomic compare-and-set. The store decides, not the caller.
  casHold(seat, user, now) {
    const s = this.state.get(seat);
    const expired = s.status === "HELD" && s.until <= now;
    if (!(s.status === "FREE" || expired)) return false;
    s.status = "HELD"; s.holder = user; s.until = now + HOLD_MS;
    return true;
  }
  confirm(seat, user, now) {
    const s = this.state.get(seat);
    if (s.status !== "HELD" || s.holder !== user || s.until <= now) return false;
    s.status = "SOLD"; s.until = 0;
    return true;
  }
}
  
```

```
}  
}
```

node seathold.js — actual output

JS

```
naive, two racing holds -> [ true, true, 'u2' ] (both told YES, only u2 really has  
it)  
cas u1 -> true  
cas u2 -> false (correctly refused)  
u2 confirm -> false  
u1 confirm -> true  
after 6 min, u2 cas -> true  
u1 confirm late -> false
```

Example 14 [Agoda · pattern]

Why does the hold carry an expiry **timestamp** rather than a background job that “unlocks abandoned seats”? What breaks if you use the background job alone?

SOLUTION

The timestamp approach. The row says `hold_until = 12:05:00`. Any reader can decide for itself whether the hold is alive, by comparing with the current time. No job needs to have run.

The job-only approach. The job runs every 30 seconds and sets expired holds back to FREE.

What breaks:

1. **The job dies.** Nobody notices for 20 minutes. Every abandoned seat stays locked. The show looks sold out and you lose real money.
2. **The job is slow.** 30,000,000 seat rows a day. Scanning them all takes minutes, so the effective hold length is “5 minutes plus however long the job took”, which is not a number you can tell a user.
3. **The job races the user.** At 12:05:00 exactly, the job frees the seat while the user’s confirm is in flight. The user pays and gets nothing.

The fix is both, with the timestamp as the source of truth.

- The WHERE clause (`status = 0 OR (status = 1 AND hold_until <= now())`) means **any** incoming request can steal an expired hold. Correctness never depends on the job.
- The job still exists, but only as **housekeeping**: it rewrites expired rows to FREE so the “seats available” count on the browse page is honest. If it dies, bookings still work — only a counter goes stale.

That is the general rule: **make correctness depend on data, and make background jobs depend on correctness — never the other way round.**

Answer: Timestamp = source of truth, job = cosmetic cleanup. Correctness must not need a cron.

Example 15 [DBS · pattern]

667 lock attempts a second land on one show’s seat rows. The database starts timing out. Name two fixes, compare, and choose.

SOLUTION

Why it hurts. 667 requests all want the same 200 rows. Most pick popular seats, so maybe 80% target the same 20 middle rows. Each one takes a row lock, holds it for the length of the transaction, and the rest queue. Queueing 500 connections behind 20 rows exhausts the connection pool, and then **unrelated** queries fail too.

Fix A — a virtual waiting room. Before the seat map loads, put the user in a queue. Admit *N* users a second into the booking flow; everyone else sees a position number.

admit rate = 50 users/sec → 667 drops to 50, a 13x reduction.

- Wins: protects everything downstream, and the user sees an honest “you are number 4,120” instead of a spinner then a failure.
- Loses: a whole extra service, and a bad user experience for the 4,119 people behind.

Fix B — hold seats in Redis first, database second. `SET seat:{show}:{C7} u991 NX PX 300000` is a single-threaded atomic operation, roughly 100,000 ops/sec on one node. The database only sees writes for holds that **succeeded**, and only at confirm time.

667 attempts → maybe 200 succeed → maybe 60 actually pay → 60 database writes. That is a 11x reduction in database writes.

- Wins: cheap, fast, one dependency you already have.
- Loses: Redis is now in the correctness path. If Redis loses data, two people can hold one seat, and you find out at the door.

Decision: ship B, then add A only for known blockbusters.

Reasoning: B removes the load at its source with one moving part, and the Redis failure mode is containable — make the **database** the final authority at confirm time. The confirm still runs the conditional `UPDATE`, so even if Redis hands out two holds for C7, exactly one confirm will change a row and the other gets a clean 409 before any money moves. Redis is an optimisation, not the truth. A is a big build and only pays off on maybe ten days a year, so it waits until a launch actually hurts.

Answer: Redis SET NX hold in front, database conditional UPDATE as final authority. Waiting room only for launch days.

TRAP

If Redis is your **only** lock, a Redis restart sells the same seat twice and you cannot un-sell it. Always leave one authoritative check at the last moment before money moves — a unique constraint or a conditional `UPDATE` in the database. Fast locks are for shedding load; the slow lock is for being right.

ONE-PAGE REVISION CARD

The six moves, in order. Cut scope → nouns to classes → draw the boxes → name the one hard part and the pattern you spend on it → write the core method in full → say the concurrency story and one extension.

The one hard part, per problem — memorise this table.

Problem	The one hard part	What you reach for
Parking lot	pricing and allocation keep changing	Strategy objects, injected
Vending machine	behaviour depends on state	State pattern, explicit transitions
Splitwise	money must never leak	integer paise + a remainder rule
Seat booking	two people, one seat, one millisecond	atomic compare-and-set + TTL hold
Elevator bank	which car, out of many	a cost function you can tune
Chess	“is this move legal” is not local	make-move / test / unmake

The three non-negotiables.

1. **Money is an integer of the smallest unit.** Paise, not rupees. Never a float. Split remainders by an explicit rule (first payer takes the extra paise), and assert that the parts sum back to the whole.
2. **Time is a parameter, never `Date.now()` inside a method.** That is what makes a 24-hour parking fee testable in one millisecond.
3. **Ids are generated, not borrowed.** Number plates repeat across states; seat “A5” repeats in every screen in the country.

Concurrency — the one-line answers.

two cars, one spot	allocate inside one atomic step; the loser is told “full”, not given the same spot
two clicks, one seat	SET NX with a TTL in front for speed, and a conditional UPDATE in the database as the final authority
fast lock fails	the database check still holds. A fast lock sheds load; the slow lock is what makes you right
held seat, user vanishes	the hold carries an expiry timestamp , so it releases itself with no cleanup job needed
667 attempts/s on one row	that is contention, not throughput — queue behind one writer per show, or split the lock per seat

Numbers from this chapter.

- a food-court bill: 900 bills over 2 hours = $0.125 \text{ bills/s} \times 20 \text{ items} = 2.5 \text{ computations/s}$. Optimise for clarity, and **say** that you checked.
- Rs.10.00 split three ways = $333 + 333 + 334$ paise. The remainder is a decision, not a rounding accident.
- greedy settlement gives **at most** $n - 1$ transfers for n people, and it is not always the true minimum. Say both halves.

Sentences that score points.

- “Getters and setters assumed — let me write `allocate` first.”
- “This class is the only place that knows how to price a stay.”
- “The scale here is 0.125 writes per second, so there is no performance question; I will optimise the design purely for change.”
- “Redis is an optimisation, not the truth. The database has the last word before money moves.”

Traps, in the order they bite.

1. 25 minutes of boilerplate, 4 minutes for the hard method.
2. Six patterns where two would do. Abstraction you cannot name a second user for.
3. Floating-point money.
4. `Date.now()` buried inside a fee calculation.

5. An enum that needs a new switch arm in four files every time a rule changes.
6. Forgetting that two users exist.

Databases & Data Modelling

The schema is the design. Everything else is plumbing around it.

The idea in one page

You can draw ten boxes and still fail the round. You cannot write a correct schema and fail the round. The schema is where the interviewer finds out whether you actually understand the problem: which things exist, which things are the same thing, what must never be lost, and which query has to be fast.

This chapter teaches four things, in this order:

- 1. how a database **finds** a row (indexes, pages, B-trees, LSM trees),
- 2. how to **shape** the rows (keys, normalisation, and when to stop normalising),
- 3. how the database keeps two users from corrupting each other (transactions, isolation),
- 4. how all of that changes when the table has ten billion rows.

WHAT A DATABASE QUESTION IS REALLY TESTING

1. Keys	Did you pick a primary key that is stable, unique, and not guessable?
2. Access path	For every query you wrote, can you name the index that serves it?
3. Correctness	Two users, same row, same millisecond — what happens?
4. Shape	SQL or NoSQL, and can you defend it with the query pattern, not with taste?
5. Growth	At 100x rows, which query dies first, and what do you do about it?

The single sentence that earns the most marks in this round:

“Here is the query, and here is the index that makes it fast.”

Say it for every endpoint you wrote in Step 3. A candidate who lists five endpoints and five matching indexes has already out-scored a candidate who drew a beautiful diagram.

The numbers you must know by heart

read one row by primary key, in RAM	0.1 – 0.5 ms	read one row by index, from SSD	1 – 5 ms
indexed query returning 100 rows	5 – 15 ms	full scan of 1 million rows	0.2 – 1 s
random 4 KB SSD read	~0.1 ms	sequential SSD read	0.5 – 2 GB/s
one SQL node, simple reads	3,000 – 8,000 QPS	one SQL node, durable writes	200 – 2,000 TPS
B-tree page size	8 KB	B-tree fanout	~500 keys/page
row overhead per row	24 – 40 bytes	one DB connection	5 – 10 MB RAM

SHORTCUT

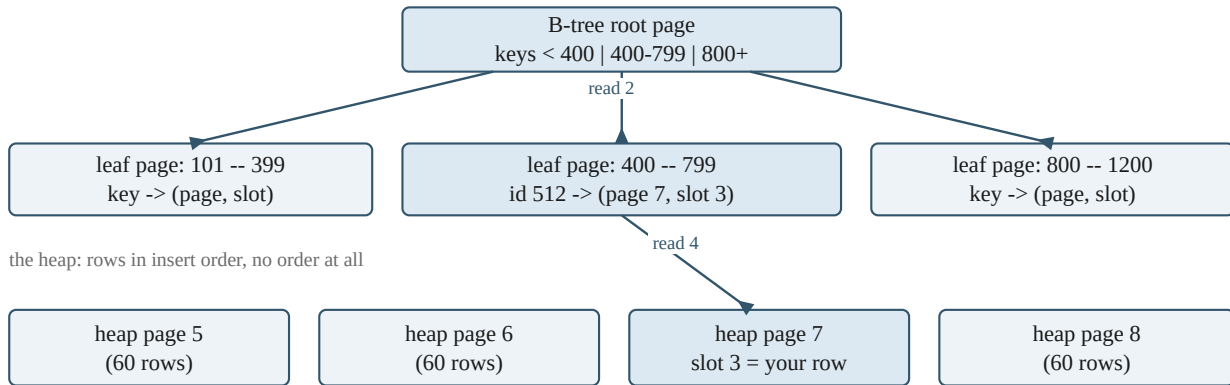
The B-tree height trick. A page is 8 KB. One key plus one child pointer is roughly 16 bytes. So one page holds about $8192/16 = 512$ entries; round it to 500.

rows covered by a tree of height $h = 500^h$

$500^1 = 500$. $500^2 = 250,000$. $500^3 = 125,000,000$. $500^4 = 62.5$ billion.

So **any table you will ever meet in an interview is 3 or 4 page reads deep**, and the top two levels are always in RAM. That is why “the index makes it fast” is not hand-waving: it is the difference between 3 reads and 12,500 reads on a 100-million-row table.

How the database actually finds your row



An index lookup: 3 reads down the tree, then 1 read into the heap. A full scan would read every heap page instead.

Read that picture as four disk reads: root, one internal level (not drawn), one leaf, then one heap page. The leaf does not hold the row. It holds a **pointer** to the row. That extra hop has a name — the **heap fetch** — and killing it is what a covering index does.

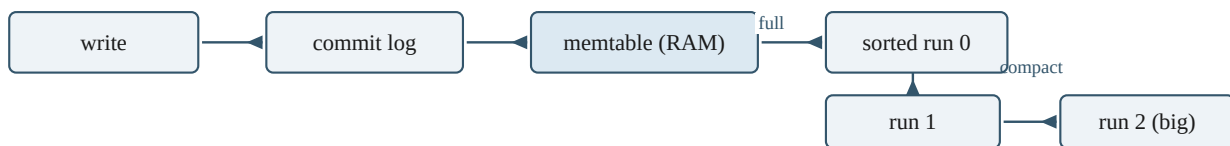
Two ways to store a table: B-tree and LSM

Every database you will be asked about is one of these two. Know which, and why.

B-tree (Postgres, MySQL/InnoDB, Oracle)



LSM tree (Cassandra, RocksDB, HBase, ScyllaDB)



B-tree: write in place, one seek per write. LSM: append to RAM, flush sorted runs, merge later.

	B-tree	LSM tree
write path	find page, rewrite page (random I/O)	append to RAM, flush sorted (sequential I/O)
write speed	limited by random I/O	very high; 10–50x more write throughput
read one row	3–4 reads, predictable	check memtable + N runs; Bloom filters help
range scan	excellent, the leaves are a linked list	good, but merges N runs on the fly
space	pages are ~70% full, so ~30% slack	compaction reclaims space in bursts

write amplification	low (1–3x)	high (10–30x) — measured below
the cost you pay	random writes	background compaction CPU and disk
pick it when	you need joins, transactions, secondary indexes	you need huge write rates and simple key access

Write amplification means: the user wrote 1 row, the disk wrote k rows. The LSM demo later in this chapter measures 10.01x on a real run. That is not a bug; it is the price of turning random writes into sequential ones. Say the number out loud in an interview and you sound like somebody who has operated one.

SQL or NoSQL — decide with the query pattern

THE DECISION TABLE. DO NOT SAY 'IT DEPENDS'.

If this is true about your data...	then
you need a multi-row transaction (money, inventory, bookings)	SQL
the same entity is read by many different filters (status, owner, date, tag)	SQL
you have real joins across 3+ entities on the read path	SQL
under ~10 TB and under ~10,000 write TPS	SQL — it will cope, do not over-engineer
one access pattern only, always by a known key	key-value / wide-column
writes are enormous and each row is independent (events, metrics, logs)	LSM wide-column
the document is always read whole and never joined	document store
the query is “find text like...” or “rank by relevance”	search index, alongside SQL
the query is “friends of friends of friends”	graph store, alongside SQL

The default answer is SQL. Start there, and move a specific table out only when you can say which query SQL cannot serve at your scale. “We used Mongo because it is web-scale” is the worst answer in this chapter.

TRAP

NoSQL is not “schemaless”. The schema still exists — it has just moved out of the database and into every piece of code that reads the data. In SQL, one `ALTER TABLE` fixes a bad field. In a document store, you now have four years of documents in six different shapes, and every reader must handle all six. Always say this sentence; it shows you have maintained something, not just started something.

ACID, in one line each, with the failure it prevents

Letter	Promise	What breaks without it
A — atomic	all the statements land, or none do	money leaves account A, the crash happens, it never arrives at B
C — consistent	the database’s own rules always hold after a commit	an order row points at a customer id that does not exist
I — isolated	concurrent transactions do not see each other’s half-done work	two buyers both read “1 left” and both buy it
D — durable	once you got “committed”, a power cut cannot take it back	the write was in RAM only; the server reboots; the order is gone

Warm-up

Example 1

A helpdesk app says: “A customer of a company raises a ticket. Agents reply with messages. A ticket has a status and one assigned agent.” Turn it into tables and pick each key.

SOLUTION

Underline the nouns: **customer, company, ticket, agent, message, status**.

Now ask of each noun: **does it have a life of its own?**

- **company** — yes, it exists before any ticket. Table. Key: `company_id`.
- **customer** — yes. Table. Key: `customer_id`. It belongs to a company, so it carries `company_id`.
- **agent** — yes. Table. Key: `agent_id`, carries `company_id`.
- **ticket** — yes. Table. Key: `ticket_id`, carries `company_id`, `customer_id`, `assignee_agent_id`.
- **message** — yes, many per ticket. Table. Key: `message_id`, carries `ticket_id`.
- **status** — no. It is one of a fixed small set of labels with no life of its own. It is a **column** on `ticket`, not a table.

The test for “table or column”: if the thing can exist when nothing points at it, it is a table. “Open” cannot exist on its own. Company can.

Answer: 5 tables (company, customer, agent, ticket, message); status is a column

Example 2

This table breaks first normal form. Say how, and fix it.

```
tickets(ticket_id, subject, tags)
  1001, "card declined", "billing,urgent,vip"
```

SOLUTION

The break. The tags column holds a **list** squeezed into one string. First normal form says every cell holds one single value.

Why it hurts, concretely. “Find all urgent tickets” becomes `WHERE tags LIKE '%urgent%'`. That cannot use a normal index, so it is a full scan. It also matches the tag not-urgent. And deleting the tag vip from 40,000 tickets means string surgery on every row.

The fix: a second table, one row per (ticket, tag) pair.

```
tickets(ticket_id, subject)
ticket_tags(ticket_id, tag)          PRIMARY KEY (ticket_id, tag)
  1001, "billing"
  1001, "urgent"
  1001, "vip"
```

Now “all urgent tickets” is an index seek on `ticket_tags(tag)`, and removing a tag is one DELETE.

Answer: A list in one cell. Split into a child table with key (ticket_id, tag).

Example 3

Take this table to third normal form, showing each step.

```
order_lines(order_id, line_no, product_id, product_name,
            product_category, qty, unit_paise, customer_city)
PRIMARY KEY (order_id, line_no)
```

SOLUTION

Step 1 — is it in 1NF? Yes. Every cell holds one value.

Step 2 — 2NF: no non-key column may depend on only PART of the key. The key is (order_id, line_no).

- customer_city depends on order_id alone. It does not care which line. **Violation.**
- product_name, product_category depend on product_id, which is not part of the key at all. Hold that thought for step 3.
- qty, unit_paise depend on the whole key. Fine.

Move customer_city up to the order:

```
orders(order_id, customer_id, customer_city, placed_at)
order_lines(order_id, line_no, product_id, product_name,
            product_category, qty, unit_paise)
```

Step 3 — 3NF: no non-key column may depend on another non-key column. In order_lines, product_category depends on product_name, which depends on product_id. That is a chain through non-key columns. **Violation.** And in orders, customer_city depends on customer_id. **Violation.**

```
products(product_id, product_name, product_category)
customers(customer_id, customer_city)
orders(order_id, customer_id, placed_at)
order_lines(order_id, line_no, product_id, qty, unit_paise)
```

Now the interview follow-up you must pre-empt. “Wait — if the customer moves city, the old order now shows the new city. Is that right?”

No. For an order, the shipping city is **historical fact**, not a lookup. So you deliberately de-normalise it back:

```
orders(order_id, customer_id, ship_city, ship_pincode, placed_at)
```

and the same for the price: order_lines.unit_paise stays a copy, because the price on the day of the order must never change when the catalog price changes.

The rule: normalise by default; copy a value only when it is a **snapshot of a fact at a moment**, and say that out loud.

Answer: Split to products/customers/orders/order_lines, then copy back ship_city and unit_paise as historical snapshots.

Example 4

A table has 100,000 rows. Column status has 3 possible values, roughly equal. Column tenant_id has 500 possible values, roughly equal. Which column deserves an index? Show the arithmetic.

SOLUTION

Selectivity = the fraction of the table one value picks out. Lower is better.

$$\text{selectivity}(\text{status}) = \frac{1}{3} = 0.333 = 33.3\% \rightarrow 33,300 \text{ rows}$$

$$\text{selectivity}(\text{tenant_id}) = \frac{1}{500} = 0.002 = 0.2\% \rightarrow 200 \text{ rows}$$

Now the cost. An index lookup that matches 33,300 rows must do 33,300 heap fetches, each possibly a separate page. A full scan of 100,000 rows reads maybe 1,700 pages (60 rows per 8 KB page). **The index is slower than the scan.** The planner knows this and will ignore your index.

200 rows is 200 heap fetches. That is far cheaper than 1,700 page reads. The index wins.

Rule of thumb: an index pays off below roughly 5–10% selectivity. Above that, the planner scans.

Answer: tenant_id (0.2%). An index on status alone (33%) will not be used.

Example 5

You have one index: (tenant_id, status), in that order. Which of these four queries can use it?

- A WHERE tenant_id = 7
- B WHERE tenant_id = 7 AND status = 'open'
- C WHERE status = 'open'
- D WHERE status = 'open' AND tenant_id = 7

SOLUTION

An index is a **sorted list of concatenated keys**. Sorted by `tenant_id` first, then by `status` inside each `tenant_id`. Think of a phone book sorted by surname, then first name.

- **A** — yes. `tenant_id` is the first column. This is “all the Sharmas”.
- **B** — yes. Both columns, in order. “Sharma, Anil”. One seek.
- **C** — **no**. You know the first name but not the surname. Every Anil is scattered across the whole book. Full scan.
- **D** — **yes**. The order you type the conditions in does not matter at all. The planner reorders them. D is the same query as B.

The rule has a name: the **left-prefix rule**. An index on (a, b, c) serves a , (a, b) , and (a, b, c) — never b alone, never c alone.

Answer: A, B and D use it. C cannot. Query order does not matter; index column order does.

Example 6

The query is `SELECT status, subject FROM tickets WHERE tenant_id = 7`, matching 200 rows. The index is `(tenant_id)`. How many reads? Now change the index to `(tenant_id) INCLUDE (status, subject)`. How many reads now?

SOLUTION

With the plain index.

- walk down the tree: 3 page reads,
- read the matching index leaves: say 2 pages (200 small entries fit easily),
- then **one heap fetch per row** to get `status` and `subject`: up to 200 page reads.

$$3 + 2 + 200 = 205 \text{ page reads}$$

With the covering index. The index leaf now carries `status` and `subject` itself. The query never needs the table.

$$3 + 2 + 0 = 5 \text{ page reads}$$

$$\frac{205}{5} = 41 \times \text{fewer reads}$$

This is called an **index-only scan**. The cost: the index is fatter, so writes are slower and the index takes more RAM.

Answer: 205 reads becomes 5 reads — 41x fewer. That is what “covering index” buys.

Example 7

A table has 80 million rows. Pages are 8 KB and hold about 500 index entries. How many page reads does a primary-key lookup need? How many does a full scan need if a heap page holds 50 rows?

SOLUTION

Index height. Find the smallest h with $500^h \geq 80,000,000$.

$$500^1 = 500$$

$$500^2 = 250,000$$

$$500^3 = 125,000,000 \geq 80,000,000 \quad \checkmark$$

So $h = 3$: three index pages, plus one heap page for the row.

$$\text{index lookup} = 3 + 1 = 4 \text{ page reads}$$

Full scan.

$$\frac{80,000,000}{50} = 1,600,000 \text{ heap pages}$$

$$\frac{1,600,000}{4} = 400,000 \times \text{more work}$$

At 0.1 ms per random read that scan is $1,600,000 \times 0.0001 = 160$ seconds; even at 1 GB/s sequential it is $1,600,000 \times 8192/10^9 = 13.1$ seconds. The index does it in under a millisecond.

Answer: 4 page reads by index; 1,600,000 pages for a scan — 400,000x more.

Example 8

Name the anomaly in each trace.

Trace 1: T1 writes qty=5 (not committed). T2 reads qty=5. T1 rolls back.

Trace 2: T1 reads qty=5. T2 writes qty=3 and commits. T1 reads qty=3.

Trace 3: T1 counts rows WHERE status='open' -> 40. T2 inserts a new open ticket and commits. T1 counts again -> 41.

Trace 4: T1 reads bal=100. T2 reads bal=100. T1 writes 90. T2 writes 80.

SOLUTION

- **Trace 1 — dirty read.** T2 saw a value that never existed. Only READ UNCOMMITTED allows this. Nobody runs that level on purpose.
- **Trace 2 — non-repeatable read.** The same row, read twice in one transaction, gave two answers. Stopped by REPEATABLE READ (a snapshot).
- **Trace 3 — phantom read.** Not a changed row — a **new** row appearing inside a range T1 already looked at. Stopped by SERIALIZABLE (and, for locking reads, by MySQL's gap locks).
- **Trace 4 — lost update.** T1's write of 90 is simply gone. This is the one that costs real money, and the most important thing to know is that **REPEATABLE READ does not always save you** — you need an atomic update, a SELECT ... FOR UPDATE, or a version check.

Answer: dirty read; non-repeatable read; phantom; lost update

Example 9

Give the column type for: a price, a “created at” timestamp, a user id, a country code, a “is deleted” flag. Say what goes wrong with the tempting wrong choice.

SOLUTION

Field	Use	Wrong choice and what it costs
price	BIGINT in paise/cents	FLOAT: $0.1 + 0.2 = 0.30000000000000004$. Money silently drifts. DECIMAL(12,2) is acceptable and exact; never FLOAT.
created at	TIMESTAMPTZ, stored UTC	TIMESTAMP without zone: the same row means two different moments in Mumbai and Singapore. Bugs appear only in October.
user id	BIGINT or UUID/ULID	INT: dies at 2,147,483,647 rows. That is not a hypothetical; it is a famous class of outage.
country code	CHAR(2)	free text: “IN”, “in”, “India”, “Bharat” all appear. Constrain it.
is deleted	TIMESTAMPTZ deleted_at NULL	BOOLEAN: you lose when , and you cannot expire old soft-deletes.

Answer: BIGINT paise, TIMESTAMPTZ, BIGINT/ULID, CHAR(2), nullable deleted_at

Example 10

Your primary key is a random UUID (version 4). A colleague says “switch to auto-increment, it is faster”. Explain the mechanism, then decide.

SOLUTION

Mechanism. In a clustered index (MySQL/InnoDB) or any B-tree, rows are stored **in key order**.

- With an **increasing** key, every new row goes at the right-hand end of the tree. The right-most page stays in RAM. One page is dirtied per insert.
- With a **random** key, each new row lands in a random page out of (say) 2 million pages. That page is probably not in RAM, so the insert costs a read **and** a write, and the page may have to **split** because it is already full.

Order-of-magnitude: a 100 GB table on a machine with 16 GB of RAM has about 16% of pages cached. Random inserts miss the cache 84% of the time. Sequential inserts miss ~0%.

But auto-increment has two real costs:

1. ids are guessable — `/orders/1002` tells a stranger you have about a thousand orders, and lets them try `/orders/1001`,
2. you cannot generate an id on the client or in a second data centre without coordination.

Decision: use a **time-sorted** id — ULID, UUIDv7, or Snowflake. The first 48 bits are a millisecond timestamp, so inserts stay at the right-hand end like auto-increment, and the random tail keeps them unguessable and generatable anywhere. If you must choose only between the two the colleague offered: keep auto-increment as the internal key and expose a random public id in the API.

Answer: Random UUID causes page splits and cache misses. Use ULID/UUIDv7 — sorted like an integer, unguessable like a UUID.

Example 11

tickets has 3 million rows. 120,000 are `status='open'`. The only hot query is `WHERE tenant_id = ? AND status = 'open' ORDER BY created_at DESC LIMIT 20`. Design one index for it and say why each column is where it is.

SOLUTION

Build the index in three parts, in this order:

1. **Equality columns first, most selective first.** `tenant_id` (0.2% selective) and `status`. Both are equality tests, so both can sit at the front.
2. **Then the ORDER BY column.** `created_at DESC`. If it is in the index in the right direction, the database reads 20 entries and stops — no sort at all.
3. **Then, optionally, the columns the query returns**, to make it covering.

```
CREATE INDEX ix_tickets_hot
ON tickets (tenant_id, status, created_at DESC)
INCLUDE (subject, assignee_agent_id);
```

Better still, since only 4% of rows are open, make it a **partial index**:

```
CREATE INDEX ix_tickets_open
ON tickets (tenant_id, created_at DESC)
INCLUDE (subject, assignee_agent_id)
WHERE status = 'open';
```

Size: 120,000 entries instead of 3,000,000 — 25x smaller, so it stays in RAM, and every non-open write does not touch it at all.

Answer: (`tenant_id, status, created_at DESC`); better, a partial index `WHERE status='open'`, 25x smaller.

Tier 1 • LLD: model it, then index it

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

This is the low-level design version of a database question. You are given a paragraph of English, a whiteboard, and 25 minutes. Six moves, same as any LLD.

THE SIX MOVES FOR A SCHEMA QUESTION

1. **Nouns to tables.** A noun is a table if it can exist alone.

2. **Pick each key.** Stable, unique, never re-used, never guessable in public.

3. **Draw the relationships** with their cardinality (1:1, 1:N, N:M). Every N:M is a table.

4. **Write the DDL** — types, NOT NULL, foreign keys, unique constraints.

5. **Write the five queries the product needs**, and one index for each.

6. **Say the concurrency story:** which two writes can collide, and how you stop them.

The worked model: a small library

The English: “Members borrow copies of books. A book has many physical copies. A member may hold at most 4 copies at once. A copy is on loan to at most one member. Late returns are fined by the day.”

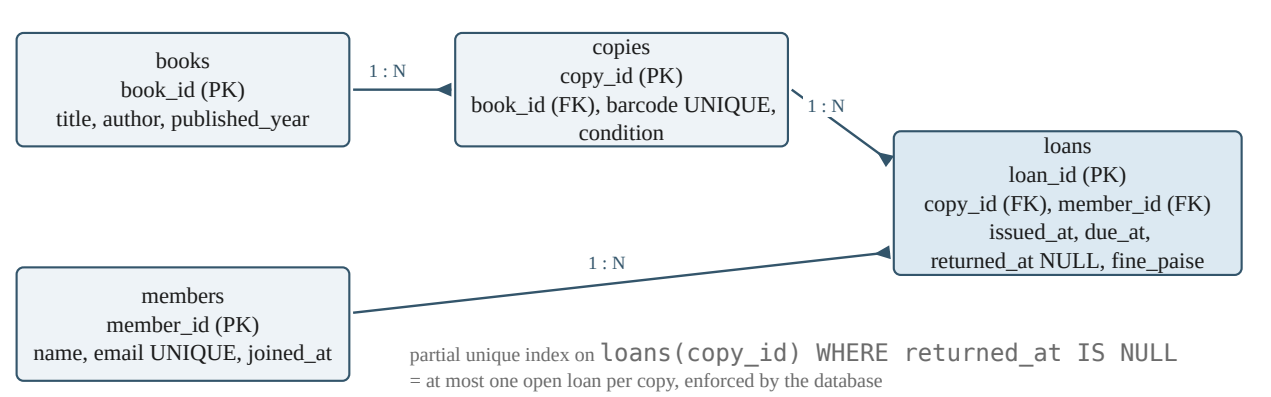
Move 1 and 2 — nouns and keys.

Noun	Table?	Key and why
book	yes	book_id. Not the ISBN — old books have none, and ISBNs get re-used across editions.
copy	yes	copy_id. A copy is a physical object with its own damage and location.
member	yes	member_id. Not the phone number — people change phones and share them.
loan	yes	loan_id. A loan is an event , not a link: it has a start, an end and a fine.
fine	no	computed from the loan’s dates and a rate. Store the amount charged on the loan, not a separate table.
librarian	no	out of scope; say so out loud.

SHORTCUT

The “event vs link” test. If a join table has only two foreign keys, it is a **link** and can stay a join table. The moment it grows a date, a state, or a price, it is an **event** and deserves its own id. loan has issued_at, due_at, returned_at, fine_paise — it is an event. Name it with a noun, never member_copy.

Move 3 — the picture.



Library schema. The arrow head is the 'many' end. The loan table is an event, not a link.

Move 4 — the DDL. This is what you write on the board. Types and constraints, no prose.

```

CREATE TABLE books (
  book_id      BIGINT PRIMARY KEY,
  title        TEXT          NOT NULL,
  author       TEXT          NOT NULL,
  published_year SMALLINT
);

CREATE TABLE copies (
  copy_id      BIGINT PRIMARY KEY,
  book_id      BIGINT NOT NULL REFERENCES books(book_id),
  barcode      TEXT          NOT NULL UNIQUE,
  condition    TEXT          NOT NULL DEFAULT 'good'
);

CREATE TABLE members (
  member_id    BIGINT PRIMARY KEY,
  name         TEXT          NOT NULL,
  email        TEXT          NOT NULL UNIQUE,
  joined_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE loans (
  loan_id      BIGINT PRIMARY KEY,
  copy_id      BIGINT          NOT NULL REFERENCES copies(copy_id),
  member_id    BIGINT          NOT NULL REFERENCES members(member_id),
  issued_at    TIMESTAMPTZ NOT NULL,
  due_at       TIMESTAMPTZ NOT NULL,
  returned_at  TIMESTAMPTZ,           -- NULL means "still out"
  fine_paise   BIGINT          NOT NULL DEFAULT 0,
  CHECK (due_at > issued_at),
  CHECK (returned_at IS NULL OR returned_at >= issued_at)
);

-- THE rule of the whole system, enforced by the database, not by code:
-- a copy can have at most one loan that has not been returned.
CREATE UNIQUE INDEX ux_one_open_loan_per_copy
ON loans (copy_id) WHERE returned_at IS NULL;

```

SHORTCUT

That last index is the single highest-scoring line in a library LLD. It makes double-lending **impossible at the storage layer**. No amount of racing application code can get past it: the second INSERT fails with a unique-violation and the API returns 409.

The general move: **turn your most important business rule into a constraint**. If you can only remember one thing from this chapter, remember that sentence.

Move 5 — the queries and their indexes.

The query the product needs	The index that serves it
what does member 42 currently hold?	(member_id) WHERE returned_at IS NULL
is copy 991 available?	ux_one_open_loan_per_copy — already there
which loans are overdue today?	(due_at) WHERE returned_at IS NULL
loan history of a member, newest first	(member_id, issued_at DESC)

how many copies of book 7 are free?

`copies(book_id) + the open-loan index`

The “4 books at a time” rule is **not** a constraint you can write in standard SQL on this shape. Options: (a) check inside the same transaction with `SELECT count(*) ... FOR UPDATE` on the member row, (b) keep a `held_count` column on members and add a `CHECK (held_count <= 4)` updated in the same transaction. Option (b) is better: it is $O(1)$, and the database enforces the limit even if a new code path forgets to check. Decide out loud, and say which you chose.

Move 6 — prove the index actually does something

Here is a runnable model of a table with one composite index. It counts **comparisons**, so you can see the gap rather than believe it.

mini-index.js — a table, a composite index, and a comparison counter

JS

```
class Table {
  constructor(name, rows = []) { this.name = name; this.rows = rows;
    this.indexes = new Map(); }

  insert(row) {
    const rid = this.rows.length; this.rows.push(row);
    for (const ix of this.indexes.values()) ix.add(row, rid);
    return rid;
  }

  scan(pred) { // full table scan: look at every row
    let cmp = 0; const out = [];
    for (const r of this.rows) { cmp++; if (pred(r)) out.push(r); }
    return { out, cmp };
  }
}

// a sorted composite index: [key..., rid] kept sorted, binary-searched
class SortedIndex {
  constructor(cols) { this.cols = cols; this.entries = []; }
  key(row) { return this.cols.map(c => row[c]); }
  static cmp(a, b) {
    for (let i = 0; i < Math.min(a.length, b.length); i++) {
      if (a[i] < b[i]) return -1;
      if (a[i] > b[i]) return 1;
    }
    return 0; // a shorter key that matches is a PREFIX
  }
  add(row, rid) {
    const e = { k: this.key(row), rid };
    let lo = 0, hi = this.entries.length;
    while (lo < hi) { const m = (lo + hi) >> 1;
      if (SortedIndex.cmp(this.entries[m].k, e.k) < 0) lo = m + 1; else hi = m; }
    this.entries.splice(lo, 0, e);
  }
  lookup(prefix) { // every entry whose leading columns equal prefix
    let cmp = 0;
    const cmpTo = (e) => SortedIndex.cmp(e.k, prefix);
    let lo = 0, hi = this.entries.length;
    while (lo < hi) {
      const m = (lo + hi) >> 1; cmp++;
      if (cmpTo(this.entries[m]) < 0) lo = m + 1; else hi = m;
    }
    const rids = [];
    const n = this.entries.length;
```

```

    for (let i = lo; i < n && cmpTo(this.entries[i]) === 0; i++) {
      cmp++; rids.push(this.entries[i].rid);
    }
    return { rids, cmp };
  }
}

```

mini-index.js — 100,000 rows, then the same query two ways

JS

```

const t = new Table("tickets");
const ix = new SortedIndex(["tenantId", "status"]);
t.indexes.set("ix_tenant_status", ix);

const STATUS = ["open", "pending", "solved"];
let seed = 42; // a fixed seed: same output every run
const rnd = () => (seed = (seed * 1103515245 + 12345) % 2147483648) / 2147483648;

for (let i = 0; i < 100000; i++) {
  t.insert({ id: i, tenantId: 1 + Math.floor(rnd() * 500),
    status: STATUS[Math.floor(rnd() * 3)], subject: "t" + i });
}

const a = t.scan(r => r.tenantId === 7 && r.status === "open");
const b = ix.lookup([7, "open"]);
console.log("rows in table      :", t.rows.length);
console.log("scan -> matched", a.out.length, "rows,", a.cmp, "comparisons");
console.log("index -> matched", b.rids.length, "rows,", b.cmp, "comparisons");
console.log("speed-up   :", (a.cmp / b.cmp).toFixed(1) + "x fewer comparisons");

const canServe = (ix, cols) =>
  cols.length <= ix.cols.length && cols.every((c, i) => c === ix.cols[i]);
console.log("\nWhich queries can ix(tenantId, status) serve?");
const QUERIES = [
  ["tenantId"], ["tenantId", "status"],
  ["status"], ["status", "tenantId"]];
for (const cols of QUERIES) {
  const ok = canServe(ix, cols);
  console.log("WHERE " + cols.join(" AND ").padEnd(22),
    ok ? "-> INDEX" : "-> FULL SCAN, the index cannot help");
}

const pct = (f) => (t.rows.filter(f).length / t.rows.length * 100);
console.log("nselectivity of status='open' :",
  pct(r => r.status === "open").toFixed(1) + "% (bad index on its own)");
console.log("selectivity of tenantId=7      :",
  pct(r => r.tenantId === 7).toFixed(2) + "% (good)");

```

\$ node mini-index.js — real output

TEXT

```

rows in table      : 100000
scan -> matched 80 rows, 100000 comparisons
index -> matched 80 rows, 97 comparisons
speed-up   : 1030.9x fewer comparisons

Which queries can ix(tenantId, status) serve?
WHERE tenantId      -> INDEX

```

```

WHERE tenantId AND status      -> INDEX
WHERE status                   -> FULL SCAN, the index cannot help
WHERE status AND tenantId      -> FULL SCAN, the index cannot help

selectivity of status='open' : 34.0% (bad index on its own)
selectivity of tenantId=7    : 0.25% (good)

```

Time scan $O(n)$; index lookup $O(\log n + k)$ for k matches **Space** index $O(n)$ entries, roughly 10--20% of the table size per index
 97 comparisons for 100,000 rows is $\log_2(100000)$ approx 17\$ for the seek plus 80 for the matches.

TRAP

The JavaScript trap hiding in this code. `SortedIndex.cmp` compares with `<` and `>`. If one key is a number and the other is a string, JavaScript gives you **neither** less nor greater — `1 < "open"` is false and `1 > "open"` is also false — so the comparator silently reports “equal” and your index returns garbage. A real database has typed columns and refuses the mismatch. In JS you must check the types yourself, or keep every index column the same type. The same family of bug: `[10, 9, 1].sort()` gives `[1, 10, 9]` because the default sort is **lexicographic**.

Example 12 [TCS NQT · pattern]

The library adds a rule: “A member may not borrow two copies of the **same book** at once.” Add it to the schema. Do not add application code.

SOLUTION

The rule is about `(member_id, book_id)`, but `loans` does not have `book_id` — it has `copy_id`. Two ways:

Option A — join at check time. Inside the borrow transaction, run `SELECT 1 FROM loans l JOIN copies c USING (copy_id) WHERE l.member_id=? AND c.book_id=? AND l.returned_at IS NULL`. Cost: a join on every borrow, and a race window between the `SELECT` and the `INSERT` unless you also lock.

Option B — copy book_id onto loans and constrain it.

```

ALTER TABLE loans ADD COLUMN book_id BIGINT NOT NULL REFERENCES books(book_id);
CREATE UNIQUE INDEX ux_one_copy_per_book_per_member
  ON loans (member_id, book_id) WHERE returned_at IS NULL;

```

Decision: B. The cost is one de-normalised column, and the risk is that `loans.book_id` could disagree with `copies.book_id`. Both are cheap to control: `book_id` is written once at insert and never updated, and a nightly check can assert agreement. The benefit is that the rule is now impossible to break — no race, no forgotten code path, one index entry instead of a join.

This is the same pattern as `ux_one_open_loan_per_copy`. **De-normalise the column you need in order to write the constraint.**

Answer: Copy book_id onto loans, then a partial unique index on (member_id, book_id) WHERE returned_at IS NULL.

Example 13 [Infosys · pattern]

The librarian wants “the 10 most borrowed books this month”. Write the query, then say what you would do when it becomes slow.

SOLUTION

The query.

```

SELECT c.book_id, count(*) AS borrows
FROM loans l
JOIN copies c ON c.copy_id = l.copy_id
WHERE l.issued_at >= date_trunc('month', now())

```

```
GROUP BY c.book_id
ORDER BY borrows DESC
LIMIT 10;
```

Why it gets slow. `count(*)` with `GROUP BY` must touch **every** loan in the month. There is no index that can skip rows: the answer depends on all of them. At 200 loans a day and one branch, that is 6,000 rows — fine. At 50,000 loans a day across 300 branches it is 1.5 million rows per month, and the query does a sort on top.

Three fixes, cheapest first.

1. **Index the filter:** `CREATE INDEX ON loans (issued_at) INCLUDE (copy_id)` turns the scan into a range read of only this month's rows. Do this first; it may be enough forever.
2. **Roll up on write:** a table `book_month_counts(book_id, month, borrows)` incremented by one `UPDATE` inside the borrow transaction. The report becomes an index read of 10 rows. Cost: one extra write per borrow, and a hot row per popular book.
3. **Materialised view refreshed nightly.** The report is a **yesterday** report anyway.

Decision: start with (1). Move to (3) when the report takes over a second, because a nightly refresh costs nothing on the write path. Only go to (2) if the product genuinely needs live counts, and then expect the hot-row contention from Example 15.

Answer: Index (`issued_at`) INCLUDE (`copy_id`) first; then a nightly materialised view. Live counters only if the product truly needs live.

Tier 1 · LLD: transactions and the lost update

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Every LLD round ends with the same question in some costume: **“two users do this at the same moment — what happens?”** Here is the whole answer, once, so you can reuse it for seats, stock, wallets and likes.

The four isolation levels, and what each one still allows

Level	Dirty read	Non-repeatable	Phantom	Lost update
READ UNCOMMITTED	possible	possible	possible	possible
READ COMMITTED	no	possible	possible	possible
REPEATABLE READ	no	no	no (snapshot)	still possible
SERIALIZABLE	no	no	no	no

Two facts that make you sound experienced. **One:** the default in PostgreSQL and in Oracle is `READ COMMITTED`; in MySQL/InnoDB it is `REPEATABLE READ`. So the level your code runs at is probably not the one you assumed. **Two:** `REPEATABLE READ` in PostgreSQL is **snapshot isolation**. It stops phantoms in your reads, but it still allows **write skew**: two transactions each read the same snapshot, each check a rule, each write a different row, and together they break the rule that neither broke alone. Only `SERIALIZABLE` stops that.

The lost update, measured three ways

lost-update.js — a store with three different write methods

JS

```
class Row {
  constructor(id, qty) { this.id = id; this.qty = qty; this.version = 1; }
}

class Store {
```



```

constructor() { this.rows = new Map(); this.conflicts = 0; }
put(r) { this.rows.set(r.id, r); }
read(id) { return { ...this.rows.get(id) }; } // a COPY = a snapshot

writeBlind(id, qty) { this.rows.get(id).qty = qty; } // the bug

// UPDATE t SET qty=?, version=version+1 WHERE id=? AND version=?
writeIfVersion(id, qty, version) {
  const r = this.rows.get(id);
  if (r.version !== version) { this.conflicts++; return false; }
  r.qty = qty; r.version++; return true;
}

// UPDATE t SET qty = qty-1 WHERE id=? AND qty>0 <- one atomic statement
decrementIfPositive(id) {
  const r = this.rows.get(id);
  if (r.qty <= 0) return false;
  r.qty--; r.version++; return true;
}
}

```

lost-update.js — the same 200 buyers, three times

JS

```

// 1. read-modify-write. Worst case: everybody reads, then everybody writes.
function lostUpdate(n) {
  const s = new Store(); s.put(new Row("sku1", 200));
  const snapshots = [];
  for (let i = 0; i < n; i++) snapshots.push(s.read("sku1"));
  for (const snap of snapshots) s.writeBlind("sku1", snap.qty - 1);
  return s.rows.get("sku1").qty;
}

// 2. optimistic concurrency control: version check, retry on conflict
function optimistic(n) {
  const s = new Store(); s.put(new Row("sku1", 200));
  let pending = n, rounds = 0, retries = 0;
  while (pending > 0) { // one round = all pending read, then all write
    rounds++;
    const snaps = [];
    for (let i = 0; i < pending; i++) snaps.push(s.read("sku1"));
    let won = 0;
    for (const snap of snaps) {
      if (snap.qty <= 0) { pending--; continue; }
      if (s.writeIfVersion("sku1", snap.qty - 1, snap.version)) won++;
      else retries++; // stale version -> read again and retry
    }
    pending -= won;
  }
  return { qty: s.rows.get("sku1").qty, rounds, retries };
}

// 3. one atomic UPDATE. No read at all, so nothing to lose.
function atomicUpdate(n) {
  const s = new Store(); s.put(new Row("sku1", 200));
  let sold = 0;
  for (let i = 0; i < n; i++) if (s.decrementIfPositive("sku1")) sold++;
}

```

```

    return { qty: s.rows.get("sku1").qty, sold };
}

console.log("start qty = 200, 200 buyers each buying 1");
console.log("1. read-modify-write :", "qty =", lostUpdate(200));
console.log("2. optimistic + retry:", optimistic(200));
console.log("3. atomic UPDATE      :", atomicUpdate(200));
console.log("4. oversell test, 250 buyers on 200 stock:", atomicUpdate(250));

```

\$ node lost-update.js — real output

TEXT

```

start qty = 200, 200 buyers each buying 1
1. read-modify-write : qty = 199
2. optimistic + retry: { qty: 0, rounds: 200, retries: 19900 }
3. atomic UPDATE      : { qty: 0, sold: 200 }
4. oversell test, 250 buyers on 200 stock: { qty: 0, sold: 200 }

```

Read the four lines carefully; each one is a whole answer.

- Line 1 is the disaster.** 200 units were sold and the counter went from 200 to 199. One unit was deducted. You have sold 199 units of stock you do not have.
- Line 2 is correct but expensive.** 19,900 retries. That is $199 + 198 + \dots + 1 + 0 = (199 \times 200)/2 = 19,900$ — every buyer that loses a round has to read and try again. Optimistic locking is **correct under contention and terrible under contention**.
- Line 3 is correct and cheap.** Zero retries, because there is no read to invalidate.
- Line 4 proves the guard works.** 250 buyers, 200 stock, exactly 200 sold, never —50.

THE FOUR FIXES, AND WHEN TO USE EACH

Fix	SQL	Use when
atomic update	UPDATE stock SET qty=qty-1 WHERE sku=? AND qty>0	the new value is a function of the old one . Always your first choice.
pessimistic lock	SELECT ... FOR UPDATE then update	you must read several rows, think, then write. Holds a lock: keep the transaction short.
optimistic (version)	UPDATE ... WHERE id=? AND version=? + retry	conflicts are rare . Great for user profiles, awful for one hot inventory row.
unique constraint	CREATE UNIQUE INDEX ... WHERE active	the rule is “at most one of these may exist”. The best fix of all, when it fits.

TRAP

UPDATE stock SET qty = qty - 1 WHERE sku = ? **without** AND qty > 0 looks atomic and is atomic — and still sells you into negative stock. Atomicity is not the same as correctness. The guard is the AND.

Example 14 [Accenture · pattern]

Two transactions transfer money between the same two accounts at the same time: T1 moves Rs.100 from A to B, T2 moves Rs.50 from B to A. Both lock rows with SELECT ... FOR UPDATE. What can go wrong, and what is the fix?

SOLUTION

Trace it.

t	T1 (A → B)	T2 (B → A)
1	locks row A	—
2	—	locks row B
3	asks for row B — waits for T2	—
4	—	asks for row A — waits for T1

Both wait forever. This is a **deadlock**. The database detects it (Postgres in about one second) and kills one transaction with a deadlock error. So nothing is corrupted — but one user sees a failure, and under load a large share of transfers fail.

The fix is one line: always lock rows in a fixed global order.

```
SELECT * FROM accounts
WHERE account_id IN (:from, :to)
ORDER BY account_id      -- <= the whole fix
FOR UPDATE;
```

Now both transactions ask for the lower id first. T2 waits at step 1 instead of half-way through, and then completes. No deadlock is possible, because a cycle in the wait graph needs two transactions holding locks in opposite order.

Say this out loud: “I sort the ids before locking, so the lock order is total and cycles cannot form.” It is a one-sentence answer that very few candidates give.

Answer: Deadlock. Fix: lock rows in a fixed order — ORDER BY account_id FOR UPDATE.

Example 15 [Cognizant · pattern]

A flash sale has one SKU with 10,000 units. 5,000 buyers per second hit `UPDATE stock SET qty=qty-1 WHERE sku='X' AND qty>0`. The row updates take 2 ms each while holding a row lock. What throughput do you actually get, and what do you do?

SOLUTION

The arithmetic. One row, one lock. The updates on that row are **serial**.

$$\text{max throughput on one row} = \frac{1}{0.002} \text{ s} = 500 \text{ updates per second}$$

Offered load is 5,000/s. So 4,500 requests a second queue up behind the lock. The queue grows without bound and the whole database’s connections are consumed waiting.

The fix: split the hot row into buckets. Keep 20 rows for the same SKU:

```
CREATE TABLE stock_bucket (
  sku VARCHAR(32), bucket SMALLINT, qty INT,
  PRIMARY KEY (sku, bucket)
);
-- 10,000 units spread as 500 per bucket, 20 buckets
```

Each buyer picks a bucket at random and decrements it:

```
UPDATE stock_bucket SET qty = qty - 1
WHERE sku = 'X' AND bucket = :b AND qty > 0;
```

$$20 \text{ buckets} \times 500 \text{ updates/s} = 10,000 \text{ updates per second}$$

Now 5,000/s fits with 2x headroom.

The cost you must name. A buyer can be told “sold out” while another bucket still has stock. Fix it by retrying 2 or 3 other buckets before giving up:

$$P(\text{all 3 tried buckets empty while stock remains})$$

is tiny until the sale is genuinely almost over. Also, “how much is left?” is now `SELECT sum(qty)` across 20 rows instead of one read — cheap.

Decision: bucket it. 20x throughput for the price of a 3-try retry loop and a `sum()`.

Answer: 500/s on one row. Split into 20 buckets → 10,000/s, with a 3-bucket retry on “empty”.

Example 16 [Wipro · pattern]

Explain write skew with a concrete example, and say which fix works.

SOLUTION

The rule: “at least one doctor must be on call at all times.”

Right now Asha and Bikram are both on call. Both feel unwell and both click “go off call” at the same instant, at REPEATABLE READ (snapshot isolation).

t	T1 (Asha)	T2 (Bikram)
1	counts on-call doctors → 2. OK to leave.	—
2	—	counts on-call doctors → 2. OK to leave.
3	sets Asha.on_call = false	—
4	—	sets Bikram.on_call = false
5	commit	commit

Nobody is on call. Neither transaction broke the rule **on the rows it wrote**, so no lock and no version check catches it. They wrote **different rows**. That is write skew.

Which fixes work?

- Optimistic version check on the row you write: **no**. Each row’s version was untouched.
- `SELECT ... FOR UPDATE` on the rows you **read**: **yes**, if you lock the whole on-call set (`SELECT ... WHERE on_call FOR UPDATE`). T2 then waits and re-reads 1, not 2.
- **SERIALIZABLE: yes**. Postgres SSI detects the read–write dependency cycle and aborts one transaction; you retry it.
- A constraint: **yes, and best**. Keep a counter row `oncall_count` with `CHECK (n >= 1)` and `UPDATE oncall_count SET n = n - 1`. Now the rule lives in one row, and one row cannot have write skew with itself.

Decision: materialise the invariant into a single row and constrain it. Fall back to **SERIALIZABLE** + retry when the invariant is too complex to materialise.

Answer: Two transactions, different rows, one broken rule. Fix: lock the read set, or **SERIALIZABLE**, or best, materialise the invariant into one constrained row.

Tier 2 · Mid-scale: the database for a helpdesk SaaS

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 17 [Agoda · pattern]

Design the data layer for a multi-tenant helpdesk product. Companies sign up; their customers raise tickets; their agents reply. Walk all seven steps.

SOLUTION

Step 1 — Clarify

Question I ask	Why the answer changes the design
----------------	-----------------------------------

How many tenants, and how skewed are they?	500 equal tenants is one schema. 20,000 tenants where the top one is 30% of all data means I need per-tenant limits and possibly a separate shard for the whales.
Must one tenant's data be physically separate?	If a contract says "separate database", the answer is a database per tenant and I lose cross-tenant reporting. If not, one schema with <code>tenant_id</code> everywhere is far cheaper.
How long must tickets be kept?	Forever means 10 TB a year and partitioning from day one. 18 months means the old partitions are simply dropped.
Is full-text search on message bodies required?	Yes means a search index next to the database, and a new consistency problem. No means the schema stays simple.
Are attachments in scope?	Yes means a blob store and signed URLs; the database only stores metadata. Never put a 300 KB file in a row.

My assumptions: 20,000 tenants, one shared schema with `tenant_id` on every table; tickets kept 24 months then archived; full-text search required; attachments go to object storage.

Step 2 — Scale estimate

Writes: new tickets. 20,000 tenants, average 150 new tickets per tenant per day.

$$20,000 \times 150 = 3,000,000 \text{ tickets/day}$$

$$\frac{3,000,000}{86,400} = 34.7 \text{ ticket inserts per second}$$

$$\text{peak} = 34.7 \times 3 = 104 \text{ per second}$$

Writes: messages. Each ticket collects about 4 messages over its life.

$$3,000,000 \times 4 = 12,000,000 \text{ messages/day}$$

$$\frac{12,000,000}{86,400} = 138.9 \text{ per second, peak } 138.9 \times 3 = 417$$

Reads. 200,000 agents are active daily and each loads about 250 pages.

$$200,000 \times 250 = 50,000,000 \text{ page views/day}$$

$$\frac{50,000,000}{86,400} = 578.7 \text{ page views per second}$$

$$\text{peak} = 578.7 \times 3 = 1,736 \text{ page views per second}$$

Each page view costs about 3 database queries (the list, the ticket, the messages):

$$1,736 \times 3 = 5,208 \text{ queries per second at peak}$$

5,208 QPS is **right at the ceiling** of a single good SQL node (3,000–8,000 simple reads). That single number decides the whole architecture: I need read replicas and a cache, and I need them at launch, not later. This is exactly what Step 2 exists to tell you.

Storage. A ticket row with all its columns is about 800 bytes. A message row, body included, averages 2,000 bytes.

$$\text{tickets} = 3,000,000 \times 800 = 2,400,000,000 \text{ bytes} = 2.4 \text{ GB/day}$$

$$\text{messages} = 12,000,000 \times 2,000 = 24,000,000,000 \text{ bytes} = 24 \text{ GB/day}$$

$$\text{total} = 2.4 + 24 = 26.4 \text{ GB/day}$$

$$\text{per year} = 26.4 \times 365 = 9,636 \text{ GB} = 9.64 \text{ TB}$$

$$\text{with indexes, + 40\% : } 9.64 \times 1.4 = 13.5 \text{ TB/year}$$

Two years retained is about 27 TB. That is too big for one comfortable node, so tickets and messages get **time partitions** and old partitions get archived.

Attachments. 10% of tickets carry one 300 KB file.

$$3,000,000 \times 0.10 = 300,000 \text{ files/day}$$

$$300,000 \times 300,000 \text{ bytes} = 90,000,000,000 = 90 \text{ GB/day}$$

$$90 \times 365 = 32,850 \text{ GB} = 32.85 \text{ TB/year} \rightarrow \text{object storage, not the database}$$

Step 3 — API surface

The five calls that generate 95% of the load

TEXT

```
GET /v1/tickets?status=open&assignee=me&limit=25&cursor=<opaque>
  200 { tickets:[{id,subject,status,priority,requester,updatedAt}], nextCursor }
  note tenant comes from the auth token, NEVER from a query parameter

GET /v1/tickets/{id}
  200 { id, subject, status, priority, requester, assignee, tags[], createdAt }

GET /v1/tickets/{id}/messages?limit=50&cursor=<opaque>
  200 { messages:[{id, authorId, authorKind, bodyHtml, createdAt, attachments[]}],
        nextCursor }

POST /v1/tickets
  body { subject, bodyHtml, requesterEmail, priority }
  header Idempotency-Key: <uuid> -- an email retry must not make two tickets
  201 { id }

PATCH /v1/tickets/{id}
  body { status?, assigneeId?, priority?, expectedVersion }
  200 { id, version } 409 if expectedVersion is stale
```

Two details worth saying out loud. **One:** `tenant_id` never appears in a URL or a body. It comes from the signed token. A tenant id the client can type is a data leak waiting for somebody to type a different number. **Two:** PATCH carries `expectedVersion`. Two agents editing one ticket is the most common collision in a helpdesk, and this turns a silent overwrite into a visible 409.

Step 4 — Data model

The core tables. Note `tenant_id` leading every primary key.

SQL

```
CREATE TABLE tickets (
  tenant_id BIGINT NOT NULL,
  ticket_id BIGINT NOT NULL,
  subject TEXT NOT NULL,
  status SMALLINT NOT NULL, -- 0 new 1 open 2 pending 3 solved 4 closed
  priority SMALLINT NOT NULL,
  requester_id BIGINT NOT NULL,
  assignee_id BIGINT, -- NULL = unassigned
  version INT NOT NULL DEFAULT 1,
  created_at TIMESTAMPTZ NOT NULL,
  updated_at TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (tenant_id, ticket_id)
) PARTITION BY RANGE (created_at);
```

```

CREATE TABLE messages (
  tenant_id BIGINT NOT NULL,
  ticket_id BIGINT NOT NULL,
  message_id BIGINT NOT NULL,
  author_id BIGINT NOT NULL,
  author_kind SMALLINT NOT NULL,      -- 0 customer 1 agent 2 system
  body_html TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (tenant_id, ticket_id, message_id)
) PARTITION BY RANGE (created_at);

CREATE TABLE attachments (
  tenant_id BIGINT NOT NULL,
  message_id BIGINT NOT NULL,
  blob_key TEXT NOT NULL,             -- s3://bucket/t/9/2026/09/ab12...
  bytes BIGINT NOT NULL,
  mime TEXT NOT NULL,
  PRIMARY KEY (tenant_id, message_id, blob_key)
);

```

Three decisions to defend.

(a) tenant_id is the first column of every primary key. It is not decoration. It means (i) one tenant's rows are physically next to each other, so a tenant's queries touch few pages; (ii) every index is automatically tenant-scoped; (iii) when you finally shard, the shard key is already chosen and no code changes. **Decision: always lead with the tenant.**

(b) Partition by created_at, monthly. 26.4 GB a day is 800 GB a month. A 24-month retention is 24 partitions. Dropping the oldest month is `DROP TABLE` — instant — versus a `DELETE` of 800 GB that would run for hours and bloat the table. **Decision: range partition on time, drop don't delete.**

(c) body_html lives in the row, attachments do not. A 2 KB text body in the row is fine. A 300 KB file in the row would make every `SELECT *` drag megabytes through the buffer pool and would multiply backup size by 4. **Decision: bytes to object storage, a key in the row.**

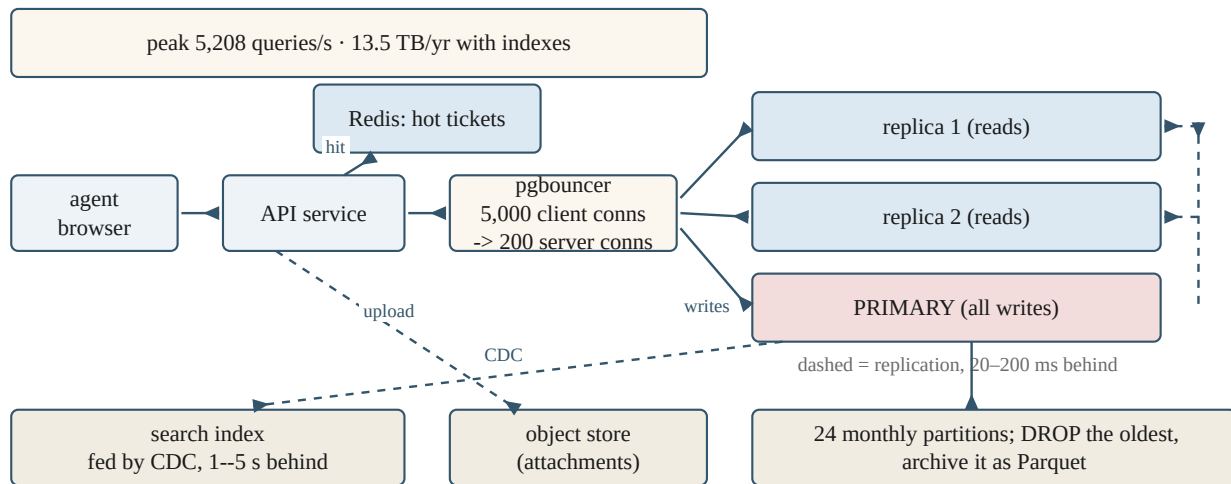
The indexes, one per hot query:

Query	Index
my open tickets, newest first	(tenant_id, assignee_id, updated_at DESC) WHERE status IN (0,1,2)
the tenant's queue, newest first	(tenant_id, updated_at DESC) WHERE status IN (0,1,2)
one ticket's messages in order	primary key (tenant_id, ticket_id, message_id) — already sorted
a customer's tickets	(tenant_id, requester_id, created_at DESC)
search the body text	not an index — a separate search service, see the deep dive

SHORTCUT

Notice all three list indexes are **partial**: `WHERE status IN (0,1,2)`. Solved and closed tickets are 90% of the table after a year and are almost never listed. Excluding them makes the index roughly 10x smaller, so it stays in RAM. A partial index is the cheapest big win in this whole chapter.

Step 5 — Architecture



Helpdesk data layer. Solid = the request path. Dashed = what happens after the commit.

Step 6 — Deep dive 1: tenant isolation without 20,000 databases

Three ways to keep 20,000 companies apart. You will be asked to compare them.

	database per tenant	schema per tenant	shared tables + <code>tenant_id</code>
isolation	strongest	strong	logical only
connections	20,000 pools — impossible	1 pool, 20,000 search paths	1 pool
migrations	20,000 runs, days	20,000 runs	1 run
cross-tenant report	impossible	painful	one query
noisy neighbour	contained	partly	needs rate limits
cost at 20,000	absurd	high	low

Decision: shared tables with `tenant_id` leading every key. At 20,000 tenants the other two are not engineering choices, they are operations disasters — 20,000 schema migrations per release. The isolation you give up must then be bought back deliberately:

1. **`tenant_id` comes only from the signed token**, never from user input.
2. **Row-level security in the database as a second lock.** If application code forgets a `WHERE tenant_id = ?`, the database adds it.

```
ALTER TABLE tickets ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON tickets
  USING (tenant_id = current_setting('app.tenant_id')::BIGINT);
```

1. **Per-tenant rate limits** so one tenant's runaway integration cannot eat the pool.
2. **The escape hatch:** the top 10 tenants by volume can be moved to their own database later, because `tenant_id` is already the leading key. Say this — it shows you planned the exit.

Step 6 — Deep dive 2: the list query that dies at page 500

The obvious list query:

```
SELECT ticket_id, subject, status, updated_at
FROM tickets
WHERE tenant_id = 9 AND status IN (0,1,2)
ORDER BY updated_at DESC
LIMIT 25 OFFSET 12475;          -- page 500
```

Why OFFSET dies. The database cannot jump to row 12,475. It must produce rows 1 to 12,475, throw them away, and return the next 25.

rows touched at page $p = 25 \times p$

Page	Rows the database touches	Rough time
1	25	1 ms
100	2,500	12 ms
500	12,500	60 ms
4,000	100,000	480 ms

And it is **wrong**, not only slow: a ticket updated between page 1 and page 2 shifts everything down, so page 2 repeats a row that page 1 already showed.

The fix: keyset (cursor) pagination. Order by something **totally ordered** and carry the last value forward.

```
SELECT ticket_id, subject, status, updated_at
FROM tickets
WHERE tenant_id = 9 AND status IN (0,1,2)
  AND (updated_at, ticket_id) < (:last_updated_at, :last_ticket_id)
ORDER BY updated_at DESC, ticket_id DESC
LIMIT 25;
```

Every page is now an index seek plus 25 entries — **constant cost, page 1 or page 4,000.**

	OFFSET	keyset
rows touched, page 500	12,500	25
cost as pages grow	linear	flat
duplicates when data changes	yes	no
can jump to “page 500”	yes	no

The cost, stated honestly: you lose “jump to page 500” and you lose a total page count. **Decision:** take keyset. A support agent has never once wanted page 500; they want “next” and a filter. Where the product truly needs numbered pages (an export screen), cap the offset at 10,000 rows and tell the user to filter.

TRAP

Keyset pagination is **only correct if the sort key is unique**. `ORDER BY updated_at DESC` alone will skip or repeat rows whenever two tickets share a millisecond — and at 104 inserts a second, they will. Always append the primary key as the tie-breaker, and compare as a tuple: `(updated_at, ticket_id) < (:a, :b)`.

Step 6 — Deep dive 3: search, and the consistency you must admit

“Find every ticket mentioning refund failed” is not an index problem. `LIKE '%refund%'` cannot use a B-tree, so it is a full scan of 24 TB.

Two options.

Option A — the database’s own full-text index. Postgres `tsvector` with a GIN index. One system, transactional: the search result can never disagree with the row, because they commit together. Cost: the GIN index on message bodies is roughly 30–50% of the text size — call it 4 TB — and every insert now pays the GIN update. It also gives you no ranking to speak of, no typo tolerance, and no faceting.

Option B — a separate search service, fed by change-data-capture from the write-ahead log. Cost: a second system to run, and the index is **eventually** consistent — typically 1–5 seconds behind.

Decision: B, a separate search index, because (i) it keeps 4 TB of GIN index off the transactional database that is already at 5,208 QPS, and (ii) helpdesk search genuinely needs ranking and typo tolerance, which the database does not give.

And then handle the lag honestly, which is the part candidates forget:

- after an agent creates a ticket, show it from the **database** on their next screen, not from search;

- label the search screen “results may be a few seconds behind”;
- for the one screen that must be exact (legal export), query the database with an explicit filter, not search.

Step 7 — Trade-offs, failure modes, and 10x

Decision	Chosen	Rejected	Why, in one line
tenancy	shared tables + <code>tenant_id</code>	database per tenant	20,000 migrations per release is not a release
pagination	keyset cursor	OFFSET	flat cost and no duplicates beat “jump to page 500”
search	external index via CDC	in-database GIN	keeps 4 TB and its write cost off the hot node
old data	monthly partitions, drop + archive	DELETE by date	DROP TABLE is instant; DELETE of 800 GB is an outage
ticket edits	version column, 409 on conflict	last write wins	two agents editing one ticket must not silently overwrite
attachments	object storage + key	bytes in the row	keeps rows small, backups small, buffer pool useful

Failure modes, and what actually happens.

Failure	Symptom	What I do
primary dies	all writes fail; reads from replicas still work	promote a replica (30–60 s), point the writer DNS at it; the app already serves reads
replication lag spikes to 10 s	an agent replies, refreshes, and their reply is missing	route “read your own write” to the primary for 5 s after a write, keyed by user id
one tenant imports 5 million tickets	the pool fills, every tenant is slow	per-tenant rate limit + a separate low-priority pool for bulk imports
a partition is not created for next month	inserts fail on the 1st at midnight	a job that creates partitions 3 months ahead, plus an alert if fewer than 2 exist
search index falls behind by an hour	agents cannot find new tickets	the list view still works from the database; page on CDC lag, not on index size

At 10x (200,000 tenants, 52,000 queries/s, 135 TB/year).

1. `tenant_id` is already the leading key, so **shard by `hash(tenant_id)` into 16 shards**. No schema change, only a router. This is why decision (a) mattered.
2. Move messages — 91% of the bytes — out of the SQL primary into a wide-column store keyed by (`tenant_id`, `ticket_id`), since messages are only ever read by that key. Tickets stay in SQL because they are filtered six different ways.
3. The list query is served from a read model kept in Redis per agent, rebuilt from CDC.
4. Archive to Parquet in object storage after 6 months instead of 24.

Example 18 [Grab · pattern]

A product manager asks for a `ticket_count` on every tenant’s dashboard, live. At peak, 104 tickets a second are created across 20,000 tenants. What do you build?

| SOLUTION

The naive answer: `SELECT count(*) FROM tickets WHERE tenant_id = ?`. For the largest tenant that is a count over 600,000 rows on every dashboard load. With a partial index it is an index-only scan of 600,000 entries — about 100 ms, every load. Unacceptable on a dashboard that polls.

Option A — a counter row per tenant, incremented in the same transaction.

writes per second on the counter table = 104

But they are spread over 20,000 tenants, so the **hottest single row** takes only the biggest tenant's share. If the biggest tenant is 5% of all tickets:

$104 \times 0.05 = 5.2$ updates per second on one row

At 2 ms per update that row can do 500/s. 5.2 is nothing. **No hot-row problem.**

Option B — recompute every 60 seconds into a summary table. Cost: one query per tenant per minute = $20,000 / 60 = 333$ counts per second. That is worse than the writes it replaces.

Option C — approximate count from table statistics. Free, but wrong by a few percent and useless per tenant.

Decision: Option A, a `tenant_stats(tenant_id, open_count, total_count)` row updated in the same transaction as the ticket insert and the status change. It is exact, it is $O(1)$ to read, and the arithmetic above proves the write contention is a non-issue at this scale.

What I say next, unprompted: “If one tenant ever grew to 50% of traffic, the row would take $104 \times 0.5 = 52$ updates a second — still under 500, so it holds. I would revisit at roughly 10x total volume, and then bucket the counter the way I would a flash-sale stock row.”

Answer: A per-tenant counter row updated in the same transaction. Peak contention is 5.2 updates/s on the hottest row — 100x below the limit.

Example 19 [DBS · pattern]

The team wants to add “tags” to tickets: each ticket has 0 to 10 tags, and agents filter by “has tag X”. Compare three designs and pick one. 3 million tickets a day.

SOLUTION

Design A — a text column tags holding a comma list. Rejected immediately: not 1NF, `LIKE '%vip%'` is a full scan and matches not-vip. See Warm-up 2.

Design B — a child table.

```
CREATE TABLE ticket_tags (  
    tenant_id BIGINT NOT NULL, ticket_id BIGINT NOT NULL, tag TEXT NOT NULL,  
    PRIMARY KEY (tenant_id, tag, ticket_id)      -- note the order!  
);
```

Key order is `(tenant_id, tag, ticket_id)` because the query is “tickets with tag X”, so tag must come before ticket_id.

Rows added per day: $3,000,000 \times 3$ tags average = 9,000,000 rows/day. At ~60 bytes: $9,000,000 \times 60 = 540$ MB/day = 197 GB/year. Acceptable. Filtering by two tags is a join or an intersect — still index seeks.

Design C — an array column with a GIN index.

```
ALTER TABLE tickets ADD COLUMN tags TEXT[];  
CREATE INDEX ix_tags ON tickets USING GIN (tags);  
-- WHERE tags @> ARRAY['vip']
```

No extra table, no join, and “has all of these tags” is one operator. Cost: a GIN index update on every ticket write, GIN's pending-list behaviour makes write latency spiky, and you cannot put a foreign key on a tag inside an array, so typos create new tags forever.

Decision: B, the child table. Two reasons I can defend: (i) a `tags(tenant_id, tag)` lookup table plus a foreign key stops typo-tags, which is the actual product problem after six months; (ii) writes stay predictable at 104 tickets/s peak, where GIN's spiky updates would sit directly on the ticket-creation path.

When I would switch to C: if the product needed “tickets having ALL of tags A, B and C” as its main filter, because the array containment operator does that in one index probe where the child table needs an N-way intersect.

Answer: Child table keyed (`tenant_id`, `tag`, `ticket_id`). Array + GIN only if “has all of these tags” becomes the main query.

Tier 3 · Large-scale: a telemetry store for 20 million devices

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 20 [Amazon · pattern]

20 million field devices (meters, sensors, scooters) each send one reading a minute, and each reading carries 8 metrics. Engineers query “device D over the last hour” constantly and “fleet average over the last month” occasionally. Design the storage. All seven steps.

SOLUTION

Step 1 — Clarify

Question	What it changes
Is a lost reading acceptable?	Yes (one missing point in 1,440 a day) means I can choose availability over consistency and use a quorum of 2 of 3. No would force synchronous replication and halve throughput.
How long is data kept at full resolution?	7 days at full resolution and 2 years rolled up is a 40x storage difference versus keeping everything raw.
Do readings arrive in order and on time?	No — devices go offline in tunnels and dump an hour of backlog at once. That means I must handle out-of-order writes and late arrivals, which rules out some append-only designs.
Is any query “give me all devices where temperature > 80 right now”?	If yes, I need a secondary index or a separate “latest value” store. If no, everything can be keyed by device.
Does a reading ever need to be corrected?	Overwrite by (<code>device</code> , <code>timestamp</code> , <code>metric</code>) is enough; no transactions, no multi-row atomicity.

Assumptions: losing a rare point is fine; 7 days raw, 2 years of 5-minute rollups; out-of-order arrivals up to 6 hours late; “current value of all devices” is a real query.

Step 2 — Scale estimate

Ingest rate.

$$\text{messages/s} = 20,000,000 \frac{\text{devices}}{60} \text{ s} = 333,333 \text{ messages/s}$$

$$\text{points/s} = 333,333 \times 8 \text{ metrics} = 2,666,667 \text{ points/s}$$

Per day.

$$\text{messages/day} = 20,000,000 \times 1,440 = 28,800,000,000 = 28.8 \text{ billion}$$

$$\text{points/day} = 28.8 \text{ billion} \times 8 = 230,400,000,000 = 230.4 \text{ billion}$$

Raw storage, if stored naively at 16 bytes per point:

$$230.4 \times 10^9 \times 16 = 3.686 \times 10^{12} \text{ bytes} = 3.69 \text{ TB/day}$$

Compressed. Time-series values compress hard: delta-of-delta on the timestamp and XOR-of-float on the value give roughly 2 bytes per point in practice.

$$230.4 \times 10^9 \times 2 = 460.8 \times 10^9 = 460.8 \text{ GB/day}$$

$$\text{per year} = 460.8 \times 365 = 168,192 \text{ GB} = 168.2 \text{ TB/year}$$

$$\text{with 3 replicas} = 168.2 \times 3 = 504.6 \text{ TB/year}$$

Compression is not a detail here — it is the difference between 168 TB and 1,346 TB a year. Say the ratio out loud: $16/2 = 8 \times$.

Ingest bandwidth. A message with 8 metrics plus headers is about 120 bytes.

$$333,333 \times 120 = 40,000,000 \text{ bytes/s} = 40 \text{ MB/s} = 320 \text{ Mbit/s}$$

Tiny. The bytes are not the problem — the **write rate** is.

How many storage nodes? One LSM node sustains about 300,000 point-writes per second while also compacting.

$$\text{with replication factor 3: } 2,666,667 \times 3 = 8,000,000 \text{ point-writes/s}$$

$$\frac{8,000,000}{300,000} = 26.7 \rightarrow 27 \text{ nodes}$$

$$\text{with 1.5x headroom for compaction and node loss} = 27 \times 1.5 = 40 \text{ nodes}$$

Do the replication multiplication **out loud**. Half of candidates size the cluster for 2.67 million writes a second and forget that replication factor 3 means the cluster actually performs 8 million. It is a 3x sizing error, and interviewers listen for it.

Step 3 — API surface

Three writes, three reads

TEXT

```
POST /v1/ingest                                <- devices, batched, protobuf
  body { deviceId, points:[ {ts, metric, value} x N ] }
  202 Accepted                                <- accepted, not stored: it goes to a log first.
  note the device retries on any non-2xx; writes are idempotent by
      (deviceId, ts, metric) so a retry overwrites with the same value

GET /v1/devices/{id}/series?metric=temp_c&from=...&to=...&step=60s
  200 { deviceId, metric, step, points:[[ts, value], ...] }
  note `step` is mandatory. Without it a 2-year query returns 1M points
      and nobody's browser survives it.

GET /v1/devices/{id}/latest
  200 { deviceId, ts, metrics:{ temp_c: 31.4, volts: 12.1, ... } }
  note served from a separate tiny store, not from the time series

POST /v1/query/aggregate
  body { filter:{ fleet:"scooters-bkk" }, metric:"battery_pct",
        from, to, step:"5m", agg:"avg" }
  202 { jobId }                               <- a month-wide fleet query is a JOB, not a request
GET /v1/query/jobs/{jobId}                    -> 200 { state, resultUrl }
```

SHORTCUT

Making the fleet-wide aggregate an **asynchronous job** rather than a synchronous GET is one of the highest-value design moves in this whole chapter. A month of 5-minute rollups across 20 million devices is

$$20,000,000 \times (30 \times 24 \times 12) = 20,000,000 \times 8,640 = 172.8 \text{ billion values}$$

That cannot be a request with a 30-second timeout. Making it a job means a slow query can never take down the read path that the dashboards depend on.

Step 4 — Data model

This is a wide-column (LSM) model. There are no joins and no ad-hoc filters — **you design the key for the query you will run, and you cannot run any other query.**

The physical layout. The partition key is the whole design.

TEXT

```
TABLE readings
  PARTITION KEY (device_id, day_bucket)      -- day_bucket = floor(ts / 86400)
  CLUSTERING KEY (metric, ts)                -- sorted inside the partition
  VALUE      value DOUBLE
  TTL        7 days
```

-> one partition = one device, one day = $1,440 \times 8 = 11,520$ points
-> "device D, metric temp_c, last hour" = ONE partition, ONE contiguous slice

```
TABLE rollup_5m
  PARTITION KEY (device_id, month_bucket)
  CLUSTERING KEY (metric, ts_5m)
  VALUE      min, max, sum, count  -- avg is sum/count, and it composes
  TTL        730 days
```

```
TABLE latest
  PARTITION KEY (device_id)
  VALUE      ts, metrics map<text,double>  -- rewritten every minute
```

```
TABLE fleet_index
  PARTITION KEY (fleet_id, day_bucket)
  CLUSTERING KEY (device_id)
  -> the ONLY way to answer "which devices are in fleet F"
```

Why (device_id, day_bucket) and not just device_id?

Key	Partition size after 7 days	Verdict
device_id	$11,520 \times 7 = 80,640$ points	works today; breaks the day someone raises retention or adds metrics — unbounded partitions are the classic wide-column mistake
(device_id, day_bucket)	11,520 points, ~23 KB compressed	chosen — bounded by construction, whatever the retention
(device_id, hour_bucket)	480 points	too small: a 24-hour query now reads 24 partitions instead of 1

The rule for a partition key, in one line: it must make the hot query read **one** partition, and it must be **bounded** in size no matter how long the system runs.

Prove it: what a bad partition key costs

Do not argue about partition keys. Simulate them.

partition.py — the same data, four different keys

PYTHON

```
import hashlib
from collections import Counter
```

```

DEVICES, PARTITIONS, HOUR = 2_000, 64, 3600
STEP = 900 # one reading every 15 minutes

def h(s):
    return int(hashlib.md5(s.encode()).hexdigest(), 16)

def load(label, keyfn):
    c = Counter()
    for d in range(DEVICES):
        for t in range(0, 24 * HOUR, STEP):
            c[h(keyfn(d, t)) % PARTITIONS] += 1
    avg = sum(c.values()) / PARTITIONS
    hot = max(c.values())
    print(f"{label:<22} used={len(c):>3} hottest={hot:>8,}"
          f" avg={avg:>5,.0f} hot/avg={hot/avg:>5.2f}x")

load("hour", lambda d, t: f"{t // HOUR}")
load("device_id", lambda d, t: f"dev{d}")
load("(device_id, day)", lambda d, t: f"dev{d}#{t // (24 * HOUR)}")
load("(device_id, 6h bucket)", lambda d, t: f"dev{d}#{t // (6 * HOUR)}")

```

\$ python3 partition.py — real output

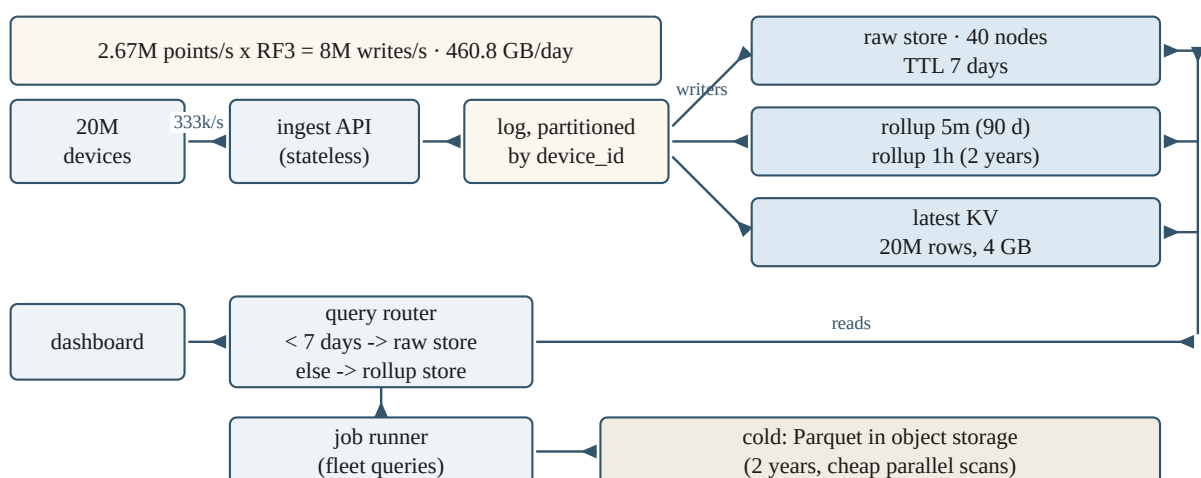
TEXT

hour	used= 19	hottest= 24,000	avg=3,000	hot/avg= 8.00x
device_id	used= 64	hottest= 4,320	avg=3,000	hot/avg= 1.44x
(device_id, day)	used= 64	hottest= 4,224	avg=3,000	hot/avg= 1.41x
(device_id, 6h bucket)	used= 64	hottest= 3,888	avg=3,000	hot/avg= 1.30x

Read the first line and feel the pain. Keying by **time** uses only 19 of 64 partitions, and the busiest is **8x** the average. In a real cluster that means one node takes 8x the write load of its neighbours while 45 nodes sit idle. Every device writing “now” writes to the same partition — that is the definition of a hot partition.

Every device-keyed variant spreads flat (1.3–1.4x, which is just normal hashing variance). Adding a **time bucket to a device key** costs nothing in balance and buys bounded partitions.

Step 5 — Architecture



Ingest goes through a log, so storage can stall without losing data. The router picks raw or rollup by time range.

Step 6 — Deep dive 1: why LSM, measured

The claim “LSM is better for write-heavy workloads” is worth nothing unless you can say what it costs. Here is a tiny LSM you can run.

lsm.js — memtable, levels, compaction, and a write-amplification counter

JS

```
class LSM {
  constructor({ memLimit = 1000, fanout = 10 } = {}) {
    this.mem = new Map(); // memtable: newest writes, in RAM
    this.levels = []; // levels[0] newest ... levels[n] oldest
    this.memLimit = memLimit; this.fanout = fanout;
    this.userWrites = 0; this.diskWrites = 0;
  }
  put(k, v) {
    this.userWrites++;
    this.mem.set(k, v);
    if (this.mem.size >= this.memLimit) this.flush();
  }
  flush() {
    const run = [...this.mem.entries()]
      .sort((a, b) => (a[0] < b[0] ? -1 : a[0] > b[0] ? 1 : 0));
    this.mem.clear();
    this.mergeInto(0, run);
    this.compact();
  }
  mergeInto(level, run) {
    const merged = LSM.merge(run, this.levels[level] ?? []); // run is newer
    this.diskWrites += merged.length; // merges rewrite bytes
    this.levels[level] = merged;
  }
  compact() {
    for (let i = 0; i < this.levels.length; i++) {
      const cap = this.memLimit * Math.pow(this.fanout, i + 1);
      if (this.levels[i].length <= cap) continue;
      const run = this.levels[i];
      this.levels[i] = [];
      this.mergeInto(i + 1, run);
    }
  }
  static merge(newer, older) { // 2-way merge; the newer key wins
    const out = []; let i = 0, j = 0;
    while (i < newer.length || j < older.length) {
      if (j >= older.length) out.push(newer[i++]);
      else if (i >= newer.length) out.push(older[j++]);
      else if (newer[i][0] < older[j][0]) out.push(newer[i++]);
      else if (newer[i][0] > older[j][0]) out.push(older[j++]);
      else { out.push(newer[i++]); j++; } // duplicate: the older copy is dropped
    }
    return out;
  }
  get(k) {
    let runsRead = 0;
    if (this.mem.has(k)) return { v: this.mem.get(k), runsRead };
    for (const run of this.levels) {
      if (run.length === 0) continue;
      runsRead++;
    }
  }
}
```



```

    let lo = 0, hi = run.length;
    while (lo < hi) { const m = (lo + hi) >> 1;
      if (run[m][0] < k) lo = m + 1; else hi = m; }
    if (lo < run.length && run[lo][0] === k) return { v: run[lo][1], runsRead };
  }
  return { v: undefined, runsRead };
}
}

const t = new LSM({ memLimit: 1000, fanout: 10 });
const key = (i) => "k" + String(i % 50000).padStart(6, "0");
for (let i = 0; i < 200000; i++) t.put(key(i), i);
console.log("keys the user wrote   :", t.userWrites);
console.log("rows rewritten to disk:", t.diskWrites);
console.log("write amplification   :",
  (t.diskWrites / t.userWrites).toFixed(2) + "x");
console.log("levels (rows each)    :", t.levels.map(r => r.length));
const hit = t.get("k000042"), miss = t.get("k999999");
console.log("read an existing key  :", hit.v, "| runs touched =", hit.runsRead);
console.log("read a missing key    :", miss.v, "| runs touched =", miss.runsRead);

```

\$ node lsm.js — real output

TEXT

```

keys the user wrote   : 200000
rows rewritten to disk: 2001000
write amplification   : 10.01x
levels (rows each)    : [ 2000, 50000 ]
read an existing key  : 150042 | runs touched = 2
read a missing key    : undefined | runs touched = 2

```

What those five lines tell you, in interview language:

1. **Write amplification is 10.01x.** The user wrote 200,000 rows; the disk wrote 2,001,000. This is the real cost of LSM. Budget for it: at 8 million point-writes a second, the disks are actually doing $8,000,000 \times 10 = 80$ million row-writes a second across 40 nodes, or 2 million per node. That is why the node estimate needed 1.5x headroom for compaction.
2. **A read touched 2 runs.** A B-tree would touch exactly 1 path. LSM reads are slower and **less predictable** than B-tree reads. That is the trade.
3. **A missing key touched 2 runs too.** A read for a key that is not there does the maximum work. That is precisely what a **Bloom filter per run** removes: it answers “definitely not here” in RAM, so a missing key costs zero disk reads.
4. **Duplicates are collapsed during merge.** The same key written 4 times occupies 4 slots until compaction, then 1. So disk usage rises and falls in a sawtooth, and you must provision for the peak of the sawtooth, not the average.

The decision, stated plainly: the workload is 2.67 million writes a second, 8 metrics per device, always read back by device. LSM wins the write side by an order of magnitude, and the read penalty (2 runs instead of 1 path, fixed by Bloom filters) does not matter because reads are 1,000x rarer than writes here. If the read/write ratio were reversed, this decision would flip to a B-tree store.

Step 6 — Deep dive 2: the query “temperature > 80 across the fleet”

This query has no device id. In a partition-keyed store it is a full cluster scan: 40 nodes each scanning everything. Three options.

Option A — a global secondary index on value. The index must be partitioned by value, so every write to any device sends an extra write to a **different** node. Write cost doubles (8 million → 16 million writes/s, meaning 40 → 80 nodes), and the index for a popular value range becomes its own hot partition. Rejected.

Option B — a local secondary index (one index per partition). Writes stay local and cheap. But a query with no device id must still ask all 40 nodes and merge — a scatter-gather across the entire cluster on every dashboard refresh. Rejected for the hot path; useful for the rare job.

Option C — a separate latest store plus alert evaluation on the write path.

- The **latest** table already holds one small row per device, 20 million rows total — roughly $20,000,000 \times 200$ bytes = 4 GB. That fits in memory on a handful of nodes.
- “Which devices are over 80 right now” becomes a scan of a 4 GB in-memory table, not 168 TB of history.
- Alerts are evaluated as the point arrives, in the writer, and **breaches** are written to a small **alerts** table that is tiny and fully indexable.

Decision: C. The insight to say out loud: **the fleet-wide question is about the present, not about history.** Do not build an index over 168 TB to answer a question about 4 GB. Keep a small, current, differently-shaped copy of the data and query that.

Step 6 — Deep dive 3: rollups, TTL and the sawtooth

Why rollups. A 30-day chart of one device at full resolution is $30 \times 1,440 = 43,200$ points per metric. Nobody’s screen has 43,200 pixels. Rolling up to 5-minute buckets gives

$$\frac{30 \times 24 \times 60}{5} = 8,640 \text{ points}$$

and to 1-hour buckets gives $30 \times 24 = 720$ points. Storage falls the same way:

5-minute rollup is 5 × fewer rows than 1-minute raw

$$460.8 \frac{\text{GB/day}}{5} = 92.2 \text{ GB/day, storing min/max/sum/count}$$

Since a rollup row holds 4 numbers instead of 1, the real saving is $5/4 = 1.25 \times$ on that table — which sounds disappointing until you remember the **raw** table is deleted after 7 days while the rollup lives 2 years.

$$\text{raw kept: } 460.8 \times 7 = 3,226 \text{ GB} \approx 3.2 \text{ TB}$$

$$\text{rollup kept: } 92.2 \times 4 \times 730 = 269,224 \text{ GB} \approx 269 \text{ TB}$$

Hold on — that is **worse** than keeping raw for 2 years? Check:

$$\text{raw for 2 years} = 460.8 \times 730 = 336,384 \text{ GB} = 336 \text{ TB}$$

$$\frac{269}{336} = 0.80$$

So the rollup saves only 20%. **That arithmetic changes the design**, and this is exactly the kind of moment an interviewer is waiting for. Fix it by rolling up **twice**:

$$\text{5-min rollup, kept 90 days} = 92.2 \times 4 \times 90 = 33,192 \text{ GB} = 33 \text{ TB}$$

$$\text{1-hour rollup, kept 730 days} = \left(\frac{460.8}{60} \right) \times 4 \times 730 = 22,426 \text{ GB} = 22 \text{ TB}$$

$$\text{total} = 3.2 + 33 + 22 = 58.2 \text{ TB versus } 336 \text{ TB raw} = 5.8 \times \text{smaller}$$

Decision: three tiers — raw 7 days, 5-minute 90 days, 1-hour 2 years. And say the number: 5.8x, not “much smaller”.

TRAP

Aggregates must compose. Store min, max, sum, count — never avg. You cannot average a set of averages when the buckets have different counts: the average of avg=10 (n=1) and avg=20 (n=999) is not 15, it is $(10 \times 1 + 20 \times 999)/1000 = 19.99$. Store sum and count, divide at read time. The same rule kills

median and p99: they do not compose at all, so you must store a sketch (t-digest, HLL for distinct counts), not a number.

Step 7 — Trade-offs, failure modes, and 10x

Decision	Chosen	Rejected	Why
engine	LSM wide-column	B-tree SQL	2.67M writes/s; B-tree random I/O cannot do it at any sane cost
partition key	(device_id, day_bucket)	device_id alone or time	time keys give 8x hot partitions (measured); device alone grows unbounded
consistency	quorum W=2, R=2 of RF=3 (AP-leaning)	W=3 strict	one lost point in 1,440 is invisible; an unavailable ingest endpoint is not
fleet queries	async jobs + a small latest table	global secondary index	a global index doubles the write cost and creates its own hot partitions
retention	3 tiers: raw 7d, 5m 90d, 1h 730d	raw for 2 years	5.8x less storage, and dashboards never want 43,200 points anyway
ingest	log first, storage second	write straight to storage	a compaction stall must slow the writers, never reject the devices

The CAP sentence you should say. “During a network partition I choose **availability**: the ingest path keeps accepting writes with W=2 on whichever replicas it can reach, and readers may briefly see a point missing. I make that choice because a device that cannot write drops the reading permanently, while a reader seeing 1,439 of 1,440 points sees a chart that looks identical. If this were a billing meter, I would flip to consistency and accept ingest failures, because a missing unit of electricity is money.”

Failure modes.

Failure	Symptom	What happens
one storage node dies	its partitions lose one replica	quorum of 2 still satisfied; hinted writes replay when it returns; no user impact
compaction storm on a node	p99 write latency on that node goes 10x	the log absorbs it: writers lag, devices never see an error; alert on log lag, not on latency
10,000 devices come out of a tunnel at once	a 6-hour backlog arrives in 60 seconds	the log flattens the burst; out-of-order points are fine because the key is (metric, ts) and writes are idempotent
a whole region is partitioned	half the devices cannot reach their nearest cluster	devices fail over to the next region's endpoint; data lands in the wrong region and is reconciled by device id later
a bad deploy writes a broken metric name	a new, unbounded set of metric names	cardinality limit per device (say 64 metrics), enforced at ingest; reject and count, do not store

The failure that has no clean fix, and you should say so: cardinality explosion. If a device starts emitting a metric name containing a timestamp or a request id, the number of distinct series grows without limit and every index and rollup grows with it. There is no storage trick that survives it. The only defence is a hard limit at

the ingest boundary and an alert on new-series-per-minute. Naming this unprompted is what separates a Tier-3 answer from a Tier-2 one.

At 10x (200 million devices, 26.7 million points/s).

1. Node count scales linearly: $26.7 \text{ M} \times 3 / 300,000 \times 1.5 = 400$ nodes. Linear scaling is the **point** of a partition key with no cross-partition queries.
2. Push the first rollout **to the device**: a device that sends one 1-minute average instead of 60 one-second readings cuts ingest 60x. At this scale the cheapest byte is the one never sent.
3. Move raw retention from 7 days to 24 hours and lean harder on rollups.
4. Split the `latest` table out to its own cluster; at 200 million rows it is 40 GB and deserves its own memory budget.

Example 21 [Google · pattern]

Your telemetry cluster has 40 nodes. One customer's 200,000 scooters all report at exactly `hh:mm:00` because their firmware uses a shared cron. What breaks, and how do you fix it without a firmware update?

SOLUTION

What breaks. The partition key is `(device_id, day_bucket)`, so the **partitions** are spread evenly — that part is fine. What is not spread is **time**: all 200,000 writes land in the same few hundred milliseconds of every minute.

$$200,000 \frac{\text{writes}}{0.5 \text{ s}} = 400,000 \text{ writes/s in the spike}$$

against a steady-state budget of

$$\frac{200,000}{60} = 3,333 \text{ writes/s}$$

That is a **120x** burst. The ingest API autoscaler cannot react in 500 ms, connection pools fill, and the writers time out — once a minute, forever.

Fix 1 — absorb it in the log. The log is append-only and sequential; 400,000 small appends a second is well within one partition set's capacity. The writers then drain at their own pace. The burst becomes **lag**, and lag is harmless here. This alone probably solves it.

Fix 2 — smear at the edge. The ingest API, on receiving a batch, does not need to write immediately. Hold each device's batch for

$$\text{delay} = \text{hash}(\text{device_id}) \bmod 30 \text{ seconds}$$

The spike flattens across 30 seconds:

$$\frac{200,000}{30} = 6,667 \text{ writes/s}$$

That is a 60x reduction, with a deterministic delay per device so ordering per device is preserved. Note this is a **server-side** change — no firmware update, which was the constraint.

Fix 3 — fix the numbers, not the system. Tell the customer to add `sleep $((RANDOM % 55))` to the cron on the next firmware cycle. Correct, free, and slow to arrive. Do it anyway, but do not wait for it.

Decision: 1 and 2 now, 3 scheduled. The interviewer is testing whether you noticed that an evenly-spread **key** does not imply evenly-spread **time**. Say that sentence.

Answer: A 120x per-minute write burst. Absorb it in the log, then smear by `hash(device_id) mod 30 s` at the ingest API.

Example 22 [Microsoft · pattern]

Compute how much RAM the Bloom filters need for the raw table, and decide whether they are worth it.

SOLUTION

How many keys? One Bloom filter entry per key per sorted run. The raw table holds 7 days:

$$\text{partitions} = 20,000,000 \text{ devices} \times 7 \text{ days} = 140,000,000 \text{ partitions}$$

A Bloom filter is built per **run** over the **partition keys** it contains (not over every point — that is the common mistake). Assume a steady state of about 8 runs per node.

Bits per key. The standard result: for a false-positive rate p , you need

$$\text{bits per key} = -1.44 \times \log_2(p)$$

$$p = 1\% \rightarrow -1.44 \times \log_2(0.01) = -1.44 \times (-6.64) = 9.6 \text{ bits} \approx 10 \text{ bits}$$

$$p = 0.1\% \rightarrow -1.44 \times \log_2(0.001) = -1.44 \times (-9.97) = 14.4 \text{ bits}$$

Memory at 1%:

$$140,000,000 \text{ keys} \times 10 \text{ bits} = 1,400,000,000 \text{ bits}$$

$$\frac{1,400,000,000}{8} = 175,000,000 \text{ bytes} = 175 \text{ MB across the cluster}$$

$$175 \frac{\text{MB}}{40} \text{ nodes} = 4.4 \text{ MB per node}$$

What it buys. Without filters, a read for a key not on this node touches all runs. The LSM demo measured 2 runs on a small tree; a production node with 8 runs would do 8 disk probes for every miss. With a 1% filter:

$$\text{disk probes per miss} = 8 \times 0.01 = 0.08$$

$$\frac{8}{0.08} = 100 \times \text{fewer disk reads on misses}$$

Decision: obviously yes, at 1%. 4.4 MB per node — less than a rounding error on a 64 GB machine — removes 99% of wasted disk reads. Going to 0.1% costs 1.44x the memory (6.3 MB/node) for another 10x fewer probes; take that too, because the memory is free at this size. The only reason not to raise it further is diminishing returns: at 0.01% you are spending 19 bits per key to avoid probes that already almost never happen.

Answer: ~10 bits/key at 1% → 175 MB cluster-wide, 4.4 MB per node, removing 99% of wasted probes. Clearly worth it.

Interview drill

These are the pushes that come after your first answer. Each answer is what a strong candidate says in about 30 seconds.

Push 1: “Why not just add an index on every column?”

Answer. Because every index is a second copy of that column that must be written on every insert, update and delete, and kept in RAM to be useful.

Concretely, on the helpdesk: 104 ticket inserts a second. With 4 indexes that is $104 \times (1 + 4) = 520$ writes a second instead of 104 — **5x the write cost**. Index storage is roughly 10–20% of table size each, so 8 indexes roughly doubles the database.

And most of them would never be used: the planner ignores any index above about 10% selectivity (Warm-up 4). So the extra indexes cost writes, RAM and backup size, and buy nothing.

My rule: **one index per hot query, and delete any index with zero scans after 30 days**. Both numbers are visible in `pg_stat_user_indexes`.

Push 2: “Your query is slow. Walk me through what you do, in order.”

Answer. Five steps, in this order, and I do not skip to step 5.

1. **EXPLAIN (ANALYZE, BUFFERS)**. I am reading three things: is it a Seq Scan or an Index Scan; is the estimated row count close to the actual; and how many buffers were read.
2. **Estimate far off actual** means the statistics are stale — **ANALYZE** the table. A planner that thinks 20 rows match when 200,000 do will pick a terrible plan.
3. **Seq Scan where I expected an index** means either no index matches the left-prefix rule, or the predicate is not sargable — `WHERE lower(email) = ?` cannot use an index on `email`; it needs an index on `lower(email)`.

4. **Index used but still slow** means too many heap fetches. Make the index covering.
5. Only then do I consider changing the query, de-normalising or caching.

The thing I do **not** do first is add a cache. A cache in front of a query that should have been 2 ms hides a bug and doubles the code paths.

Push 3: “SQL or NoSQL for this? Choose now.”

Answer. I choose by counting access patterns, not by preference.

One access pattern, always by a known key, enormous write rate — that is the telemetry store. Wide-column LSM.

Six access patterns over the same entity, plus a transaction — that is the helpdesk. SQL, and I will not move it until a specific query proves SQL cannot serve it at my scale.

The trap in this question is the word “or”. The real answer for almost every product is **both**: SQL for the entities people filter and transact on, and a second store for the one table that is 90% of the bytes. In the helpdesk that split is tickets (SQL) and messages (key-value by (`tenant_id`, `ticket_id`)) — and I showed the arithmetic: messages are $24/26.4 = 91\%$ of the daily bytes.

Push 4: “Two users edit the same ticket. What does the user see?”

Answer. Not “last write wins” — that silently destroys one agent’s work.

The PATCH carries `expectedVersion`. The update is `UPDATE tickets SET ..., version = version + 1 WHERE tenant_id=? AND ticket_id=? AND version=?`. If it touches 0 rows, somebody else got there first and I return **409 Conflict** with the current row in the body.

The UI then shows “Priya changed the status to Pending while you were typing” and offers merge or overwrite. That is an optimistic-locking design, and it is right here because two agents editing the same ticket is **rare** — Example 15 showed optimistic locking is exactly wrong when conflicts are common, as with flash-sale stock.

Push 5: “Why is your primary key (`tenant_id`, `ticket_id`) and not just `ticket_id`?”

Answer. Four reasons, and the fourth is the one that matters in two years.

1. **Locality.** One tenant’s rows sit in the same pages. A tenant’s list query touches a handful of pages instead of pages scattered across the whole table.
2. **Safety.** Every index is automatically tenant-scoped, so a missing `WHERE tenant_id = ?` cannot accidentally return another company’s ticket through an index-only scan.
3. **Smaller indexes.** Every secondary index starts with `tenant_id`, so each one is really 20,000 small trees rather than one huge one.
4. **The exit.** When the database outgrows one node, the shard key is already the leading column. Sharding becomes a routing change with **no schema migration**. Retrofitting a tenant column into a live 27 TB table is a multi-month project.

Push 6: “Your replica is 3 seconds behind. A user posts a reply and does not see it. Fix it.”

Answer. This is read-your-own-writes, and there are three fixes with different costs.

1. **Sticky-to-primary window.** After a write by user U, route U’s reads to the primary for the next 5 seconds. Cost: a small, bounded increase in primary reads. In our numbers, writes are 417/s at peak and each buys 5 seconds of stickiness, so at most $417 \times 5 = 2,085$ concurrent sticky users out of 200,000 — about 1%. Cheap.
2. **Write-through the client.** The POST response already contains the created message, so the UI renders it locally without re-reading. Free, and it handles the common case.
3. **LSN token.** The write returns the log position; the read sends it back; the replica waits until it has caught up to that position. Exact, but adds latency and complexity.

Decision: 2 first (free), 1 as the safety net (1% extra primary reads), 3 only for a screen where correctness is legally required.

Push 7: “How do you change a column type on a 27 TB table with no downtime?”

Answer. Never with `ALTER TABLE ALTER COLUMN TYPE` — that rewrites the whole table under an exclusive lock. Six steps, the expand–contract pattern:

1. Add the new column, nullable, with no default. Instant — it is a catalogue change only.
2. Deploy code that **writes both** columns and **reads the old** one.

3. Backfill the new column in batches of 10,000 rows with a pause between batches, so replication never falls behind.
4. Verify: a query that counts rows where the two disagree must return 0.
5. Deploy code that **reads the new** column.
6. Drop the old column, a week later, after a rollback window has passed.

The whole point is that every step is independently reversible. Adding NOT NULL or a default in step 1 is the classic mistake — on older engines that rewrites the table.

Push 8: “When would you de-normalise?”

Answer. Three cases, and only three.

1. **It is a historical snapshot, not a lookup.** `orders.ship_city` and `order_lines.unit_price` must not change when the customer moves or the price changes. This is not de-normalisation at all; it is correct modelling.
2. **To make a constraint writable.** `loans.book_id` was copied purely so that a unique index could enforce “one copy of a book per member”. A rule the database enforces beats a rule the code enforces.
3. **A join is measurably on the hot path.** Measured, not imagined: EXPLAIN shows the join, the query is in the top 3 by total time, and no index fixes it.

And I state the cost every time: two copies can disagree, so I need one writer, a reconciliation check, and a documented source of truth. If I cannot name who fixes the drift, I do not de-normalise.

PRACTICE · WARM-UP target: 15 min

1. A table has 4,000,000 rows. Pages hold 500 index entries. How many page reads for an index lookup? How many pages for a full scan at 40 rows per page?
2. Column city has 900 distinct values spread evenly over 2,000,000 rows. Compute the selectivity and say whether an index will be used.
3. You have INDEX (a, b, c). Which of these can use it? WHERE a=1 · WHERE b=2 · WHERE a=1 AND c=3 · WHERE c=3 AND b=2 AND a=1
4. 1,000 buyers do read-modify-write on a stock row starting at 1,000, all reading before any writes. What is the final quantity, and how many units were oversold?
5. A query returns 500 rows through a non-covering index. Count the page reads with and without an INCLUDE of the 2 returned columns (tree height 3, index leaves 4 pages).
6. 12,000,000 rows a day at 1,500 bytes each. Give GB/day, TB/year, and TB/year with indexes at +35%.
7. At a Bloom filter false-positive rate of 2%, how many bits per key? (bits = $-1.44 \log_2 p$)

ANSWER KEY

1. $500^2 = 250,000 < 4,000,000 \leq 500^3 = 125,000,000$, so height 3. Index lookup = $3 + 1 = * 4$ page reads. **Full scan = $4,000,000 / 40 = * 100,000$ pages.** Ratio $100,000 / 4 = 25,000 \times$.
2. $1/900 = 0.00111 = 0.111\%$, about 2,222 rows. Far below 5%, so **yes, the index is used.**
3. a=1 **yes** (left prefix). b=2 **no**. a=1 AND c=3 **partly** — it seeks on a and then filters on c; c is not usable as a seek because b is missing. c=3 AND b=2 AND a=1 **yes, fully** — written order is irrelevant.
4. Every buyer read 1,000, so every buyer writes 999. Final quantity **999**. Units actually sold: 1,000. Units the counter admits: 1. **Oversold by 999.**
5. Without INCLUDE: $3 + 4 + 500 = * 507$ page reads. **With INCLUDE: $3 + 4 + 0 = * 7$ page reads.** $507/7 = 72.4 \times$ fewer.
6. $12,000,000 \times 1,500 = 18,000,000,000 = * 18$ GB/day. $18 \times 365 = 6,570$ GB = *** 6.57 TB/year.** With indexes: $6.57 \times 1.35 = 8.87$ **TB/year.**
7. $-1.44 \times \log_2(0.02) = -1.44 \times (-5.644) = 8.13$, so **9 bits per key.**

PRACTICE · TIER 2 target: 40 min

1. Design the schema for a college course-registration system: students, courses, sections (a course taught at a time by a teacher), enrolments, and a seat cap per section. Give the DDL, the key of each table, the four indexes, and the **one constraint** that makes over-enrolment impossible. Then compute: 30,000 students, each registering for 5 sections in a 2-hour window on results day — what is the peak write rate, and what is the peak write rate on the single hottest section row if the most popular section has 400 applicants?
2. The helpdesk adds “merge two tickets into one”. Write the transaction, name every table it touches, and say what happens if it crashes half way. Then say which isolation level you run it at and why.
3. A reporting query joins tickets to messages to count messages per ticket for one tenant over one month. Using the chapter’s numbers (3M tickets/day, 4 messages each, 20,000 tenants), estimate the rows touched for an average tenant and decide between running it live, running it on a replica, and pre-aggregating.

ANSWER KEY

1. Tables: students(student_id PK), courses(course_id PK), sections(section_id PK, course_id FK, teacher_id, term, seat_cap, seats_taken), enrolments(student_id, section_id, enrolled_at, PRIMARY KEY (section_id, student_id)).

The primary key (section_id, student_id) is the “no double enrolment” constraint — a second insert fails. The seat cap is CHECK (seats_taken <= seat_cap) on sections, with UPDATE sections SET seats_taken = seats_taken + 1 WHERE section_id = ? AND seats_taken < seat_cap in the same transaction as the enrolment insert. If it touches 0 rows, the section is full and you roll back.

Indexes: (student_id, term) for “my timetable”; (course_id, term) for “sections of this course”; (teacher_id, term); the primary keys serve the rest.

Peak rate: $30,000 \times 5 = 150,000$ enrolments over $2 \times 3,600 = 7,200$ s = 20.8 per second average. Registration is not flat — use 5x for the first ten minutes: $20.8 \times 5 = 104$ **enrolments per second**.

Hottest row: 400 applicants for one section, arriving in the same first minute: $400/60 = 6.7$ **updates per second on one row**. At 2 ms per locked update the row supports 500/s, so it holds comfortably. **No bucketing needed** — and that is the answer to give, with the arithmetic, rather than reflexively bucketing.

2. merge(source, target): update messages SET ticket_id = target WHERE ticket_id = source; insert a system message into the target; update ticket_tags (ignore duplicates); set tickets.status = closed, merged_into = target on the source; bump both version columns; update tenant_stats. Five tables.

A crash half way: nothing is visible, because it is one transaction — atomicity means the partial state never existed. The dangerous version is doing it in five separate transactions, which can leave messages pointing at a closed ticket.

Isolation: READ COMMITTED is **not** enough, because an agent could add a message to the source ticket between the move and the close, orphaning it. Run it at SERIALIZABLE, or at READ COMMITTED with SELECT ... FOR UPDATE on **both** ticket rows first, in ticket_id order (Example 14) so two simultaneous merges cannot deadlock.

3. Per tenant per day: $3,000,000 / 20,000 = 150$ tickets and $150 \times 4 = 600$ messages. Over 30 days: $150 \times 30 = 4,500$ tickets and $600 \times 30 = 18,000$ **messages**.

18,000 rows is nothing — roughly 20 ms with the right index. **Run it live**, on a replica so a badly-filtered variant cannot hurt the primary. Pre-aggregation would be premature: it adds a write-path cost of 417 writes/s to save 20 ms on a report.

The answer changes for the largest tenant. If one tenant is 5% of all volume, that is $3,000,000 \times 0.05 \times 30 = 4.5$ million tickets and **18 million messages** a month — there, pre-aggregate. **So: live on a replica, with a row-count guard that switches the top 1% of tenants to a pre-aggregated table.**

PRACTICE · TIER 3 target: 50 min

1. Redesign the telemetry store for 5 million devices reporting every 10 seconds with 20 metrics each. Give: points/s, points/day, compressed GB/day at 2 bytes/point, node count at 300,000 point-writes/s with RF=3 and 1.5x headroom, and the partition key with its resulting partition size. Then say which of your numbers is most likely to be wrong and why.
2. The product adds “alert me when any device in fleet F has battery below 10% for 15 minutes”. Design the data path. Say exactly which store answers it, what you write on the ingest path, and what the failure mode is when the alert engine restarts.

3. Compare keeping 2 years of raw data in the LSM store against archiving to columnar files in object storage after 7 days. Use the chapter's numbers. Give the storage figures for both, name one query each option makes easy and one it makes hard, and decide.

ANSWER KEY

1. $5,000,000 / 10 = 500,000$ messages/s. $500,000 \times 20 = * 10,000,000$ points/s. **Per day: $10,000,000 \times 86,400 = 864,000,000,000 = * 864$ billion points/day.** Compressed: $864 \times 10^9 \times 2 = 1,728 \times 10^9$ bytes = $* 1,728$ GB/day = 1.73 TB/day; $\times 365 = 631$ TB/year, $\times 3$ replicas = **1,893 TB/year.** **Nodes: $10,000,000 \times 3 = 30,000,000$ point-writes/s; $30,000,000 / 300,000 = 100$; $\times 1.5 = * 150$ nodes.**

Partition key (`device_id`, `hour_bucket`): one partition is $(3,600 / 10) \times 20 = 7,200$ **points**, about 14 KB compressed. A **day** bucket would be $8,640 \times 20 = 172,800$ points — too big for one partition, which is why the bucket must shrink as the rate rises.

Most likely wrong number: 2 bytes per point. It assumes slowly-changing float values. Counters that jump, or values with genuine noise in the low bits, compress at 4–6 bytes, which would triple the storage bill. I would measure it on a day of real data before buying disks.

2. Ingest path: for each point, the writer updates the **latest** row **and** maintains a small per-device state `below_10_since` (set when the value first drops under 10, cleared when it rises). The alert fires when now — `below_10_since` ≥ 900 s. That is a comparison of two numbers on a row we are already writing — no scan, no index.

Which store answers the fleet question: `fleet_index` gives the device list; the alert state lives with **latest**. The 168 TB history is never touched.

Restart failure mode: the in-memory part of the alert engine loses `below_10_since`, so every device's timer restarts and alerts are **late by up to 15 minutes**, not lost. Fix by persisting `below_10_since` in the **latest** row itself, which makes the engine stateless and restart-safe. Say that: **put the state in the store, not in the process.**

3. Raw for 2 years in the LSM store: $460.8 \text{ GB/day} \times 730 = 336,384 \text{ GB} = * 336 \text{ TB} \times 3 \text{ replicas} = * 1,009 \text{ TB}$ of SSD. Archive after 7 days: $460.8 \times 7 \times 3 = * 9.7 \text{ TB of SSD, plus } 460.8 \times 723 = 333,158 \text{ GB} = * 333 \text{ TB}$ of object storage at roughly one tenth the price per GB and no replication to pay for separately.

LSM makes easy: “device D, any minute in the last 2 years”, in milliseconds. LSM makes hard: “average across 20 million devices for last March” — a full cluster scan. Columnar archive makes easy: exactly that fleet-wide scan, cheaply and in parallel. Columnar archive makes hard: a point lookup of one device-minute, which now means opening a large file.

Decision: archive after 7 days. The cost difference is roughly 100x on the expensive tier, and the query the archive makes hard (a single old point lookup) is rare, while the query it makes easy (fleet-wide history) is the one the LSM store could never serve. Keep the last 7 days hot because that is where every interactive query lives.

ONE-PAGE REVISION CARD

The five questions for any schema

1. What is the key — stable, unique, unguessable, and sorted if it can be?
2. For every query, which index serves it? (If you cannot name one, the query is a scan.)
3. Which two writes can collide, and what stops them?
4. What is the biggest table in two years, and what do you do with the old rows?
5. Which business rule can become a database constraint?

Numbers to memorise

PK read, cached	0.1–0.5 ms	PK read, SSD	1–5 ms
scan 1M rows	0.2–1 s	8 KB page holds	~500 index entries
B-tree height for 125M rows	3	index lookup	height + 1 reads
one SQL node, reads	3–8k QPS	one SQL node, writes	200–2,000 TPS
index is used below	~10% selectivity	one locked row	~500 updates/s
LSM write amplification	10–30x	Bloom bits at 1%	~10 per key

The formulas

- selectivity = rows matched / rows total; index pays below ~10%
- B-tree height : $500^h \geq \text{rows}$
- index lookup reads = $h + 1$; covering index reads = $h + \text{leaves}$
- OFFSET cost = $25 \times \text{page number}$; keyset cost = constant
- writes per insert = $1 + \text{number of indexes}$
- cluster write rate = user writes $\times$ replication factor
- Bloom bits per key = $-1.44 \log_2 p$
- max updates/s on one row = $1 / \text{lock hold time}$

Worked numbers from this chapter

- 100,000 rows: scan 100,000 comparisons vs index 97 — 1,031x
- 205 page reads become 5 with a covering index — 41x
- helpdesk: 3M tickets/day $\rightarrow$ 34.7/s, 50M page views $\rightarrow$ 5,208 queries/s peak
- helpdesk storage: 26.4 GB/day $\rightarrow$ 9.64 TB/yr $\rightarrow$ 13.5 TB/yr with indexes
- 200 buyers, read-modify-write: qty goes 200 $\rightarrow$ 199. Optimistic: 19,900 retries.
- telemetry: 20M devices/60 s = $333,333 \text{ msg/s} \times 8 = 2.67\text{M points/s}$
- telemetry: 230.4 B points/day, 460.8 GB/day compressed, 168.2 TB/yr, 504.6 TB with RF3
- nodes: $2.67\text{M} \times 3/300\text{k} \times 1.5 = 40$
- time-based partition key measured 8.00x hot; device-based 1.44x
- LSM measured write amplification 10.01x on 200,000 writes
- Bloom filters: 175 MB cluster-wide, 4.4 MB/node, 100x fewer probes on a miss

Order of decisions — never get this wrong normalise first $\rightarrow$ de-normalise only for a snapshot, a constraint, or a **measured** hot join. Write the source of truth in one transaction; everything else is derived.

The four concurrency fixes

1. atomic UPDATE ... WHERE guard — first choice, always
2. SELECT ... FOR UPDATE in sorted key order — when you must read, think, then write
3. version column + retry — when conflicts are rare
4. unique constraint — when the rule is “at most one may exist”. Best of all.

Sentences that score points

- “Here is the query, and here is the index that serves it.”
- “tenant_id leads every primary key, so sharding later is a routing change, not a migration.”
- “That is a partial index — 25x smaller, so it stays in RAM.”

- “Keyset pagination, not OFFSET: constant cost and no duplicates, at the price of page jumping.”
- “Replication factor 3 means the cluster really performs 8 million writes a second, not 2.67.”
- “Store min, max, sum, count — never avg. Averages do not compose.”
- “I turn the business rule into a constraint, so racing code cannot break it.”

Caching & CDNs

Keep the answer near the question

The idea in one page

THE ONE IDEA

A cache is a **second copy of an answer, stored somewhere closer or faster than the place that computed it.**

Every design question about caching is really three questions, always in this order:

1. **What do I put in?** (the key, the value, the size)
2. **When does it come out?** (eviction because the cache is full, expiry because the value got old)
3. **What breaks when it is wrong?** (stale data, a stampede on the database, one overheated key)

A student who can answer only the first question sounds like a beginner. A student who answers all three sounds like an engineer. This chapter is mostly about questions 2 and 3.

The numbers you must know by heart

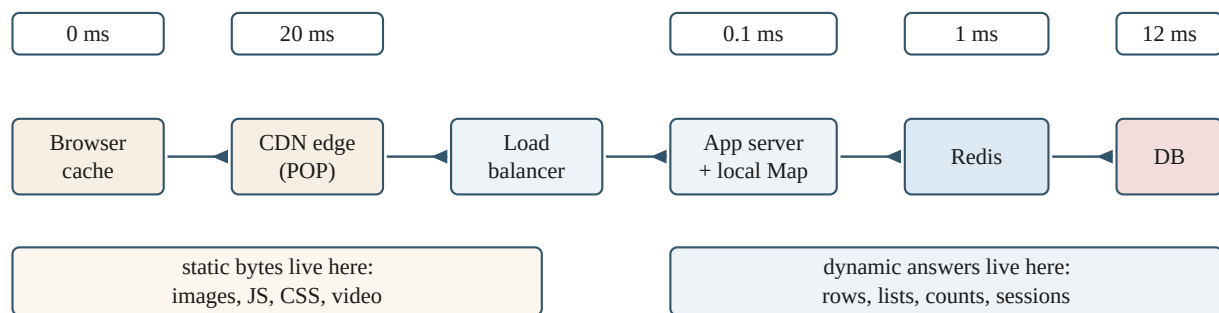
Caching only pays off because the gaps between these numbers are enormous. Memorise the column on the right; every estimate in this chapter uses it.

Where the answer comes from	Typical time	How many fit in 1 second
CPU L1 cache	1 nanosecond	1,000,000,000
Main memory (RAM) on the same box	100 nanoseconds	10,000,000
In-process cache (a JS Map)	~0.1 microseconds	~10,000,000
Redis in the same data centre	0.5 – 1 millisecond	~1,500 per connection
SSD read	0.1 millisecond	10,000
Indexed row from a SQL database	5 – 15 milliseconds	~100
Same-region network round trip	1 millisecond	1,000
India to US round trip	180 milliseconds	5
CDN edge in the user's own city	10 – 30 milliseconds	~50

SHORTCUT

The whole business case for caching in one line: **RAM is about 100 times faster than a database query, and a CDN edge is about 10 times closer than your origin.** If you remember nothing else, remember “100x” and “10x”.

Where caches live



The cache ladder. A request stops at the first layer that already has the answer.

Read that diagram left to right and ask at each box: **what does this layer know that the next one does not?** The browser knows one user. The CDN knows one city. Redis knows one region. The database knows the truth. Truth is the slowest thing in the picture.

The four caching patterns

Pattern	Read path	Write path	Pick it when
Cache-aside (lazy loading)	app checks cache, on miss reads DB and fills cache	app writes DB, then deletes the key	default choice; 90% of interviews
Read-through	app asks the cache; the cache library fetches from DB itself	same as cache-aside	you use a library/CDN that does the fetch for you
Write-through	normal cache read	write to cache and DB in one call, DB write is synchronous	reads must never see stale data and writes are rare
Write-behind (write-back)	normal cache read	write to cache only; flush to DB in batches later	huge write volume, losing a few seconds of data is acceptable

TRAP

Write-behind is the one that loses data. If the cache node dies before the flush, those writes are gone forever. Never propose write-behind for money, orders, or inventory. Propose it for view counts, “last seen at”, and analytics rollups – things where losing 3 seconds of updates costs nothing.

The five failures, and their fixes

Failure	What it looks like	Fix
Stale read	user edits a price, still sees the old one	delete the key on write, short TTL
Thundering herd (stampede)	one popular key expires, thousands of requests hit the DB at once	singleflight lock, or refresh before expiry
Hot key	one key gets 40% of all traffic, one Redis node melts	replicate the key with a suffix, or cache it in-process
Cache penetration	requests for keys that do not exist bypass the cache every time	cache the “not found” answer with a short TTL, or a Bloom filter
Cache avalanche	a million keys expire in the same second, or the cache cluster restarts	TTL jitter, warm-up on boot, a rate limit in front of the DB

Warm-up

WARM-UP

build the reflex

Example 1

A service gets 4,000 requests per second. The cache hit rate is 92%. A cache hit takes 1 ms. A cache miss takes 1 ms (to check the cache) plus 12 ms (to read the DB).

(a) How many requests per second reach the database? (b) What is the average response time?

SOLUTION

(a) A miss happens on $100\% - 92\% = 8\%$ of requests.

$4000 \times 0.08 = 320$ requests per second reach the database.

(b) Split the average into the two paths and weight each by how often it happens.

$$\text{avg} = 0.92 \times 1 \text{ ms} + 0.08 \times (1 + 12) \text{ ms}$$

$$\text{avg} = 0.92 + 1.04 = 1.96 \text{ ms}$$

Answer: 320 QPS to the database; 1.96 ms average response time.

Notice how lopsided this is. The 8% of requests that miss contribute 1.04 ms of the 1.96 ms average – **more than half the average latency comes from less than a tenth of the traffic**. That is why the tail (p99) of a cached service is so much worse than its average, and why interviewers ask about p99 and not the mean.

Example 2

The same service. The team improves the hit rate from 92% to 97%. Recompute both answers, and say what changed by how much.

SOLUTION

Database load: $4000 \times 0.03 = 120$ QPS.

Before it was 320 QPS. $\frac{320}{120} = 2.67$, so the database now does **2.67 times less work**.

Average latency:

$$\text{avg} = 0.97 \times 1 + 0.03 \times 13 = 0.97 + 0.39 = 1.36 \text{ ms}$$

Before it was 1.96 ms. The saving is $1.96 - 1.36 = 0.6$ ms, about 31% faster.

Answer: 120 QPS (2.67x less DB load); 1.36 ms average (31% faster).

SHORTCUT

Five percentage points of hit rate sounds small. It cut the database load by almost two thirds. **Cache work is measured on the miss side, not the hit side.** Going from 92% to 97% means the misses fell from 8 to 3 – and $\frac{8}{3} = 2.67$. Always do the arithmetic on misses.

Example 3

A cache holds 3 items and uses LRU eviction. Trace it for this sequence of keys and say what is in the cache after each step, plus the final hit count.

A, B, C, A, D, B, A, C

Every access is a get; on a miss, the item is loaded and inserted.

SOLUTION

Write the cache as a list, most-recently-used on the left.

Step	Key	Hit or miss	Cache after (MRU → LRU)
1	A	miss	A
2	B	miss	B, A
3	C	miss	C, B, A
4	A	hit	A, C, B
5	D	miss – evict B	D, A, C
6	B	miss – evict C	B, D, A
7	A	hit	A, B, D
8	C	miss – evict D	C, A, B

Hits at steps 4 and 7. That is 2 hits out of 8 accesses.

Answer: 2 hits, 6 misses; final cache is C, A, B (C most recent).

TRAP

The most common mistake in this trace is forgetting that a **hit also reorders** the list. At step 4, A was the oldest item; after the hit it became the newest, which is exactly why B (not A) got evicted at step 5. An LRU that does not reorder on read is not an LRU.

Example 4

Same sequence, same size 3, but now use **LFU** (evict the least frequently used; break ties by evicting the one inserted earlier). How many hits?

SOLUTION

Track a count for every key in the cache.

Step	Key	Result	Counts after
1	A	miss	A:1
2	B	miss	A:1 B:1
3	C	miss	A:1 B:1 C:1
4	A	hit	A:2 B:1 C:1
5	D	miss – tie between B and C at 1, B is older, evict B	A:2 C:1 D:1
6	B	miss – tie between C and D at 1, C is older, evict C	A:2 D:1 B:1
7	A	hit	A:3 D:1 B:1
8	C	miss – tie between D and B, D is older, evict D	A:3 B:1 C:1

Same answer here: 2 hits.

Answer: 2 hits. On this short trace LFU and LRU tie.

On a **short** trace the two policies often tie. The difference shows up on long traces: LFU protects a key that was popular for months, LRU throws it away after one quiet hour. LFU's weakness is the mirror image – a key that was hugely popular last year keeps its high count forever and refuses to leave. Real systems use **LFU with decay**, or the **TinyLFU / W-TinyLFU** idea: a tiny LRU window in front of an LFU main cache.

Example 5

You must cache 500,000 product records. Each record is about 4 KB of JSON.

(a) How much memory for all of them? (b) Traffic is skewed: 20% of products get 80% of the reads. How much memory for just those 20%? (c) Redis adds roughly 30% overhead for keys, pointers and its own structures. What do you actually provision for the 20% case?

SOLUTION

Use 1 KB = 1000 bytes for estimation. Interviewers expect round numbers, not exact binary ones.

(a) $500 \times 4 \text{ KB} = 2,000 \text{ KB} = 2 \text{ MB} = 2 \text{ GB}$

(b) 20% of 500,000 = 100,000 records.

$100,000 \times 4 \text{ KB} = 400,000 \text{ KB} = 400 \text{ MB}$

(c) $400 \text{ MB} \times 1.3 = 520 \text{ MB}$

Round up to a 1 GB instance so there is room to grow and room for replication buffers.

Answer: (a) 2 GB (b) 400 MB (c) provision about 520 MB, buy 1 GB.

SHORTCUT

The 80/20 rule is the reason caching works at all. You almost never need to cache everything. Caching the hot 20% gets you most of the benefit for a fifth of the memory. When an interviewer asks “how big should the cache be?”, the answer is never “the size of the database” – it is “the size of the working set”, and you estimate the working set from the traffic skew.

Tier 1 · Low-level design: build the cache

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Service-company interviews rarely ask you to design a CDN. They ask you to **write an LRU cache class**, then bend it: add TTL, make the eviction policy swappable, make it thread safe. This section is that ladder.

Example 6 [TCS NQT · pattern]

Design a class `LRUCache` with `get(key)` and `put(key, value)`, both in $O(1)$ average time. When the cache is full, the least recently used entry is removed. Also expose hit, miss and eviction counters.

SOLUTION

Step 1 – what does $O(1)$ force on us?

We need two things at once:

- find a key fast → that is a hash map
- know which key is oldest, and move a key to “newest” fast → that is a doubly linked list

The classic answer is “hash map + doubly linked list”, and in a Java or C++ interview you would hand-write both. In JavaScript you get the linked list for free: **a Map remembers insertion order**, and `delete` followed by `set` moves a key to the newest position.

Step 2 – the class.

An LRU cache in 25 lines, using Map insertion order

JS

```
class LRUCache {
  constructor(capacity) {
    this.cap = capacity;
    this.map = new Map();           // key -> value, oldest first
    this.hits = 0; this.misses = 0; this.evictions = 0;
  }
}
```



```

get(key) {
  if (!this.map.has(key)) { this.misses++; return undefined; }
  const val = this.map.get(key);
  this.map.delete(key);          // pull it out...
  this.map.set(key, val);        // ...and put it back as the newest
  this.hits++;
  return val;
}

put(key, val) {
  if (this.map.has(key)) this.map.delete(key);
  else if (this.map.size >= this.cap) {
    const oldest = this.map.keys().next().value; // first key = least recent
    this.map.delete(oldest);
    this.evictions++;
  }
  this.map.set(key, val);
}

get hitRate() {
  const total = this.hits + this.misses;
  return total === 0 ? 0 : this.hits / total;
}
keysNewestFirst() { return [...this.map.keys()].reverse(); }
}

```

The test that proves it works

JS

```

const c = new LRUCache(3);
c.put(1, 'one'); c.put(2, 'two'); c.put(3, 'three');
c.get(1);          // 1 is touched, so 2 is now the oldest
c.put(4, 'four');   // cache is full -> evict 2
console.log('newest -> oldest:', c.keysNewestFirst());
console.log('get(2) =', c.get(2));
console.log('hits', c.hits, 'misses', c.misses, 'evictions', c.evictions);

```

node lru.js

TEXT

```

newest -> oldest: [ 4, 1, 3 ]
get(2) = undefined
hits 1 misses 1 evictions 1

```

Time get and put are $O(1)$ average **Space** $O(\text{capacity})$

Map.delete and Map.set are both $O(1)$ average. keys().next() is $O(1)$ — it does not copy the key list.

TRAP

this.map.keys() returns an **iterator**, not an array. next().value gives you the first key in $O(1)$. If you write [...this.map.keys()][0] instead, you copy every key into a new array on **every eviction** and your $O(1)$ becomes $O(n)$. This exact line is what interviewers look at.

TRAP

In a Java or C++ interview you will be asked to write the doubly linked list by hand, because HashMap there has no insertion order (LinkedHashMap does, and saying so scores a point). Know both answers: **“in JS, Map order gives it to me; in Java I would use LinkedHashMap with accessOrder = true, or hand-roll a HashMap<K, Node> plus a doubly linked list.”**

Example 7 [Infosys · pattern]

Your reviewer says: “Good. Now the product team wants LFU instead of LRU on one endpoint, and maybe FIFO on another. Do not copy-paste the class three times.”

Redesign so the eviction rule can be swapped without touching the cache.

SOLUTION

Step 1 – name the thing that varies. Only one thing changes between LRU, LFU and FIFO: **who leaves next**. Storage, counters, capacity check – all identical.

Step 2 – pull that one thing behind a small interface. This is the **Strategy pattern** from Chapter 4. The policy never touches values; it only tracks keys.

Step 3 – the interface has exactly four methods. If you find yourself needing a fifth, you have leaked storage concerns into the policy.

Strategy pattern: the cache holds a policy it knows nothing about

JS

```
// ---- the plug: four small methods, nothing about storage ----
class EvictionPolicy {
  touched(key) {}           // key was read
  added(key)   {}           // key entered the cache
  removed(key) {}           // key left the cache
  victim()     { return null; } // who leaves next
}

// ---- plug 1: least recently used ----
class LruPolicy extends EvictionPolicy {
  constructor() { super(); this.order = new Map(); } // insertion order = recency
  touched(key) { this.order.delete(key); this.order.set(key, true); }
  added(key)   { this.order.set(key, true); }
  removed(key) { this.order.delete(key); }
  victim()     { return this.order.keys().next().value; }
}

// ---- plug 2: least frequently used ----
class LfuPolicy extends EvictionPolicy {
  constructor() { super(); this.count = new Map(); }
  touched(key) { this.count.set(key, this.count.get(key) + 1); }
  added(key)   { this.count.set(key, 1); }
  removed(key) { this.count.delete(key); }
  victim() {
    let best = null, low = Infinity;
    for (const [k, n] of this.count) if (n < low) { low = n; best = k; }
    return best;
  }
}

// ---- the cache never asks which plug it is holding ----
class Cache {
  constructor(capacity, policy) {
    this.cap = capacity; this.policy = policy; this.store = new Map();
  }
}
```

```

    this.hits = 0; this.misses = 0; this.evictions = 0;
  }
  get(key) {
    if (!this.store.has(key)) { this.misses++; return undefined; }
    this.policy.touched(key); this.hits++;
    return this.store.get(key);
  }
  put(key, val) {
    if (this.store.has(key)) { this.store.set(key, val); this.policy.touched(key);
return; }
    if (this.store.size >= this.cap) {
      const v = this.policy.victim();
      this.store.delete(v); this.policy.removed(v); this.evictions++;
    }
    this.store.set(key, val); this.policy.added(key);
  }
  has(key) { return this.store.has(key); }
}

```

Same cache, two different plugs, two different survivors

JS

```

const lru = new Cache(2, new LruPolicy());
lru.put('a', 1); lru.put('b', 2);
lru.get('a'); // a is recent, b is stale
lru.put('c', 3); // evicts b
console.log('LRU keeps a?', lru.has('a'), ' keeps b?', lru.has('b'));

const lfu = new Cache(2, new LfuPolicy());
lfu.put('a', 1); lfu.put('b', 2);
lfu.get('a'); lfu.get('a'); // a used 3 times, b used 1 time
lfu.put('c', 3); // evicts b
console.log('LFU keeps a?', lfu.has('a'), ' keeps b?', lfu.has('b'));

```

node policy.js

TEXT

```

LRU keeps a? true keeps b? false
LFU keeps a? true keeps b? false

```

Time LRU: $O(1)$. LFU as written: $O(1)$ for get/put, $O(n)$ for victim() **Space** $O(\text{capacity})$ for the store plus $O(\text{capacity})$ for the policy
Say the $O(n)$ out loud before the interviewer spots it, then say how you would fix it.

The follow-up you will get: make LFU $O(1)$

The interviewer's question: "Your victim() scans every key. At 100,000 keys that is 100,000 steps per eviction. Fix it."

The answer: keep keys grouped by count. Maintain

- countOf: Map<key, n> – the frequency of each key
- buckets: Map<n, Set<key>> – all keys that currently have frequency n
- minCount – the smallest n that has a non-empty bucket

touched(key): move the key from buckets[n] to buckets[n+1]. If buckets[minCount] became empty and n === minCount, then minCount++.

added(key): put it in buckets[1] and set minCount = 1.

victim(): return any key from buckets[minCount]. A Set gives you one in $O(1)$ with values().next().value.

Every operation is now $O(1)$. The trick is that `minCount` only ever goes up by one on a touch, and resets to 1 on an insert – so you never have to search for it.

Example 8 [Cappgemini · pattern]

Add **TTL** (time to live) to the cache: each entry expires after a number of milliseconds. Then explain why you must add **jitter** to the TTL, with numbers.

SOLUTION

Step 1 – store an expiry timestamp, not a countdown. A countdown needs a timer per key. A timestamp needs nothing: you compare it on read. This is called **lazy expiry**.

Step 2 – delete on read. If the entry is expired, remove it and report a miss. Do not return it.

Step 3 – jitter. Suppose 100,000 keys are loaded during a deploy at 10:00:00, all with a 5-minute TTL. At 10:05:00 **all 100,000 expire in the same millisecond**. The next second of traffic is a 100% miss rate, and every one of those misses becomes a database query. That is a **cache avalanche**.

The fix is one line: pick each TTL randomly inside a band around the base value.

TTL with lazy expiry, plus jitter

JS

```
const cacheGet = (k) => {
  const e = redis.get(k);
  if (!e) return undefined;
  if (Date.now() > e.expiresAt) { redis.delete(k); return undefined; } // TTL
  expired
  return e.val;
};

const cacheSet = (k, val, ttlMs) => redis.set(k, { val, expiresAt: Date.now() +
  ttlMs });

// TTL with jitter: never let a million keys expire in the same millisecond.
const jitter = (baseMs, spreadPct = 0.2) => {
  const spread = baseMs * spreadPct;
  return Math.round(baseMs - spread + Math.random() * 2 * spread);
};
```

The arithmetic that justifies jitter. Base TTL 300,000 ms (5 minutes), spread $\pm 20\%$. Expiries now land anywhere in a window of

$$2 \times 0.20 \times 300\{, \}000 = 120\{, \}000 \text{ ms} = 120 \text{ seconds}$$

So 100,000 keys expire spread over 120 seconds instead of all at once:

$$100\{, \} \frac{000}{120} \approx 833 \text{ expiries per second}$$

At a 12 ms database query each, that is $833 \times 0.012 = 10$ database queries running concurrently – completely survivable. Without jitter it was 100,000 at once.

Answer: Store `expiresAt`, check it on read; jitter every TTL by $\pm 20\%$ so a 100,000-key avalanche becomes a harmless 833 refreshes per second.

node aside.js — three jitter draws from a 300000 ms base

TEXT

```
three jitter draws from 300000: [ 281577, 266488, 266891 ]
```

Example 9 [Accenture · pattern]

Write the **cache-aside** read and write paths against a real store. Explain why the write path **deletes** the key instead of updating it.

SOLUTION

Step 1 – the read path. Check cache. On a miss, read the source, fill the cache, return.

Step 2 – the write path. Write the source **first**, then delete the key. Never the other way round.

Step 3 – why delete and not update? Two reasons, and you should give both.

Reason A: a race. Two writers, W1 setting price 100 and W2 setting price 200.

Time	W1	W2
t1	writes DB = 100	
t2		writes DB = 200
t3		sets cache = 200
t4	sets cache = 100	

The database says 200. The cache says 100. It stays wrong until the TTL expires. With **delete** instead of **set**, both writers delete; the next reader loads 200 from the DB. Delete is idempotent, set is not.

Reason B: wasted work. If a product is written 50 times an hour and read once a day, “update the cache on write” computes and stores 50 values that nobody reads. Delete computes nothing.

Cache-aside, both paths

JS

```
// ----- READ: cache-aside -----
async function getProduct(id) {
  const key = `product:${id}`;
  const hit = cacheGet(key);
  if (hit !== undefined) return { data: hit, from: 'cache' };

  const row = await db.read(id);           // miss -> go to the source
  cacheSet(key, row, jitter(300_000));     // 5 min, ±20%
  return { data: row, from: 'db' };
}

// ----- WRITE: change the DB, then DELETE the key (never update it) -----
async function setPrice(id, price) {
  const row = await db.write(id, price);
  redis.delete(`product:${id}`);          // invalidate, do not rewrite
  return row;
}
```

node aside.js

TEXT

```
1 [ 'db', 799 ]
2 [ 'cache', 799 ]
3 [ 'db', 649 ]
4 [ 'cache', 649 ]
db calls = 3
```

Read that output line by line. Call 1 misses and costs a DB read. Call 2 hits. Then the price changes and the key is deleted, so call 3 misses again and picks up the new price 649. Call 4 hits. Four reads and one write cost three database calls instead of five.

TRAP

Order matters and it is a favourite trap. If you **delete the key first and then write the DB**, a reader can slip in between: it misses, reads the OLD row from the DB, and writes that old row back into the cache – after your update. The cache is now stale forever. **Always: write the source of truth first, invalidate second.**

Even “write DB, then delete” has a rare hole. A reader that missed **before** your write can still be holding the old row in a local variable, and may write it into the cache **after** your delete. The fix at serious scale is **delayed double delete**: delete the key, do the write, wait a few hundred milliseconds, delete again. You should mention this only if the interviewer pushes – it signals you know that invalidation is genuinely hard.

Tier 2 · Mid-scale: a cached product catalog

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 10 [Shopee · pattern]

Design the read path for the product catalog of a shopping app. 8 million daily active users. Product pages must feel instant. The catalog itself changes rarely, but prices and stock change often.

Walk all seven steps.

SOLUTION

Step 1 – Clarify

Five questions, and what each one changes:

Question I ask	Why the answer changes the design
How stale may a price be?	If the answer is “zero”, I cannot cache the price at all and must split the object. If it is “up to 60 seconds”, one short TTL solves it.
How many products, and how big is one?	This sets the memory bill. 500k x 4 KB is one Redis box; 500M x 4 KB is a cluster.
Read to write ratio?	200:1 means cache-aside is obviously right. 2:1 would mean the cache is mostly churn and may not be worth it.
Is the traffic skewed, and how badly?	If the top 1% of products is 50% of traffic, I can cache far less memory – but I must plan for hot keys.
What is the p99 latency target?	A 50 ms p99 lets me go to the DB on a miss. A 10 ms p99 means misses must be rare, so I need a higher hit rate and pre-warming.

I will assume: price may be up to 60 seconds stale; 500,000 products at ~4 KB; reads to writes is 200 to 1; classic 80/20 skew; p99 target 50 ms.

Step 2 – Scale estimate

Reads. Each active user opens about 25 product pages a day.

$$8\{, \}000\{, \}000 \times 25 = 200\{, \}000\{, \}000 \text{ reads per day}$$

$$\text{average QPS} = 200\{, \}000\{, \} \frac{000}{86} \{, \}400 = 2\{, \}314.8 \approx 2\{, \}300 \text{ QPS}$$

Shopping traffic is spiky – evenings and sale events. Use a 4x peak factor.

$$\text{peak QPS} = 2\{, \}314.8 \times 4 = 9\{, \}259 \approx 9\{, \}300 \text{ QPS}$$

Writes. 500,000 products, each updated about twice a day (price, stock).

$$500\{, \}000 \times 2 = 1\{, \}000\{, \}000 \text{ writes per day}$$

$$1\{, \}000\{, \}\frac{000}{86}\{, \}400 = 11.6 \text{ QPS}$$

Read : write is 2314.8 : 11.6 $\approx$ 200 : 1. Cache-aside is clearly correct.

Memory.

$$\text{everything} = 500\{, \}000 \times 4 \text{ KB} = 2\{, \}000\{, \}000 \text{ KB} = 2 \text{ GB}$$

$$\text{hot } 20\% = 100\{, \}000 \times 4 \text{ KB} = 400 \text{ MB}$$

$$\text{with Redis overhead} = 400 \times 1.3 = 520 \text{ MB}$$

Bandwidth out of Redis at peak.

$$9\{, \}259 \text{ QPS} \times 4 \text{ KB} = 37\{, \}036 \text{ KB/s} \approx 37 \text{ MB/s} = 296 \text{ Mbit/s}$$

That is comfortable on a 1 Gbit/s network card, and it tells me one Redis node is enough for bandwidth. (Whether it is enough for a **hot key** is a different question – Step 6.)

Step 3 – API surface

Read API

TEXT

```
GET /v1/products/{id}
200 -> { "id":"p_1187", "title":"Steel Kettle 1.5L", "brand":"Nord",
  "images":[...], "attrs":{"...},
  "price":{" amount":64900, "currency":"IDR",
"as_of":"2026-09-15T04:10:02Z" },
  "stock":{" state":"in_stock", "as_of":"2026-09-15T04:10:02Z" } }
Cache-Control: public, max-age=30, stale-while-revalidate=120
ETag: "p_1187-v41"

GET /v1/products?ids=p_1,p_2,p_3      <- batch, up to 50 ids, one Redis MGET
200 -> { "products":[ ... ], "missing":["p_3"] }

PATCH /internal/products/{id}/price  <- called by the pricing service
body { "amount": 59900 }
204 No Content    (and the cache key is deleted)
```

Two details that score points. First, `as_of` timestamps: the client can see how stale the price is, which turns an invisible problem into a visible one. Second, the batch endpoint: a category page showing 40 products would otherwise fire 40 HTTP calls. One MGET of 40 keys costs one round trip instead of forty.

Step 4 – Data model

Store	Key	Value	TTL
Postgres products	id (PK)	title, brand, attrs, images JSONB, version int	–
Postgres prices	(product_id, region) (PK)	amount, currency, updated_at	–
Redis	product:{id}:v{version}	the full JSON blob	5 min $\pm$ 20%
Redis	price:{id}:{region}	just the amount	45 s $\pm$ 20%
Redis	product:{id}:miss	the literal string "404"	30 s

Three decisions worth defending out loud:

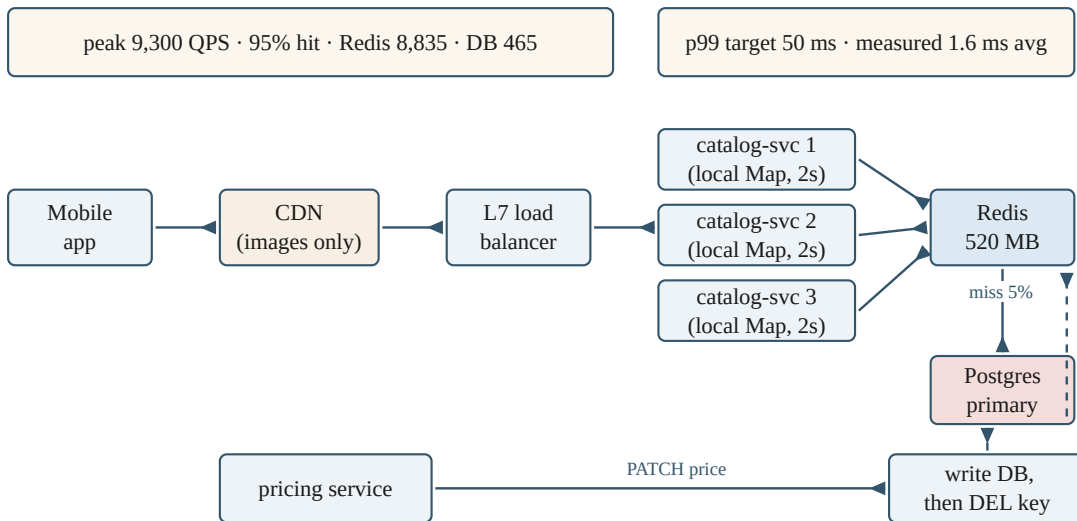
(a) Two keys, not one. The stable part of a product (title, images, attributes) changes maybe once a month. The price changes many times a day. If they share one key, every price change throws away 4 KB of stable data that

then has to be re-fetched. Splitting them means a price change invalidates 20 bytes. **Decision: split.** The cost is a second Redis round trip per page – but MGET fetches both keys in one trip, so the real cost is zero.

(b) The version in the key. Writing `product:p_1187:v41` instead of `product:p_1187` means an update does not have to delete anything: it just starts writing `v42`, and `v41` ages out by TTL. This makes invalidation across many regions trivial, because there is nothing to invalidate. **Decision: use versioned keys for the stable blob,** plain keys for price (where the value is tiny and delete is cheap).

(c) Caching the miss. A crawler asking for 10,000 product IDs that do not exist would otherwise put 10,000 queries a second on the database – **cache penetration**. Storing the string `"404"` for 30 seconds stops it dead.

Step 5 – Architecture



Read path (solid) and invalidation path (dashed) for the catalog service.

Step 6 – Deep dive 1: how big should the cache be?

Do not guess. Simulate the traffic skew and read the answer off a table. Here is a 30-line Python program that does it, and its real output.

```

hitrate.py — LRU hit rate against a Zipf workload PYTHON

# How big should the cache be? Simulate, do not guess.
import random, bisect
from collections import OrderedDict

random.seed(7)
N, REQ = 100_000, 400_000 # 100k products, 400k reads in the sample

# Zipf: product #1 is asked for about 10x more than product #10.
H = sum(1.0 / i for i in range(1, N + 1))
cum, s = [], 0.0
for i in range(1, N + 1):
    s += 1.0 / i
    cum.append(s / H)
draw = lambda: bisect.bisect_left(cum, random.random()) + 1
reqs = [draw() for _ in range(REQ)]

def lru_hit_rate(capacity):
    c, hits = OrderedDict(), 0
  
```



```

for k in reqs:
    if k in c:
        c.move_to_end(k); hits += 1
    else:
        c[k] = 1
        if len(c) > capacity:
            c.popitem(last=False)          # drop the least recently used
return hits / REQ

unique = len(set(reqs))
print(f"distinct keys touched: {unique} -> every one of them must miss once")
print(f"so the hit rate can never beat {(1 - unique / REQ) * 100:.1f}%")
PEAK = 9259
for cap in (1_000, 5_000, 10_000, 20_000, 50_000):
    hr = lru_hit_rate(cap)
    print(f"cache {cap:>6} keys = {cap * 4 / 1000:>5.0f} MB -> hit {hr*100:>4.1f}% "
          f"-> DB sees {round(PEAK * (1 - hr))>5} QPS at peak")

```

python3 hitrate.py

TEXT

```

distinct keys touched: 55840 -> every one of them must miss once
so the hit rate can never beat 86.0%
cache  1000 keys =    4 MB -> hit 50.7% -> DB sees  4567 QPS at peak
cache  5000 keys =   20 MB -> hit 66.4% -> DB sees  3113 QPS at peak
cache 10000 keys =   40 MB -> hit 73.2% -> DB sees  2479 QPS at peak
cache 20000 keys =   80 MB -> hit 79.8% -> DB sees  1870 QPS at peak
cache 50000 keys =  200 MB -> hit 85.9% -> DB sees  1305 QPS at peak

```

Three things to read off that table, and say out loud:

(1) Diminishing returns are brutal. Going from 1,000 to 5,000 keys (4 MB to 20 MB) buys 15.7 points of hit rate. Going from 20,000 to 50,000 keys (80 MB to 200 MB) buys only 6.1 points. Memory is linear; benefit is logarithmic.

(2) There is a ceiling you cannot pass. In this sample 55,840 distinct keys are touched, and **each one must miss at least once** – it has to be loaded before it can be hit. So

$$\text{best possible hit rate} = 1 - 55\{, \} \frac{840}{400} \{, \} 000 = 1 - 0.1396 = 86.0\%$$

Any team promising 99% on this workload is either measuring over a longer window (where each key gets more repeat reads) or is not measuring cold misses.

(3) A longer window changes the answer. The 86% ceiling is an artefact of only sampling 400,000 requests. In a day we serve 200 million requests against the same 500,000 products – 400 reads per product per day – so the cold-miss share becomes tiny and real hit rates of 95%+ are achievable.

Decision: provision 520 MB (the hot 20%, with overhead). It sits comfortably past the knee of the curve, leaves headroom, and costs one small Redis instance. **We are choosing the flat part of the curve on purpose: buying more memory past this point is paying linear money for logarithmic benefit.**

At 95% hit rate and 9,300 peak QPS:

$$\text{Redis QPS} = 9\{, \} 300 \times 0.95 = 8\{, \} 835$$

$$\text{DB QPS} = 9\{, \} 300 \times 0.05 = 465$$

A single Postgres primary with an index on the primary key handles 465 QPS without noticing. **Without the cache it would face 9,300 QPS and fall over.**

Average latency:

$$0.95 \times 1 \text{ ms} + 0.05 \times (1 + 12) \text{ ms} = 0.95 + 0.65 = 1.6 \text{ ms}$$

Step 6 – Deep dive 2: the thundering herd

The problem, with real numbers. The homepage banner product p_1187 is getting 1,200 QPS on its own during a flash sale. Its cache key expires. In the 12 ms it takes one database query to come back, how many more requests arrive and also miss?

$$1\{, \}200 \text{ QPS} \times 0.012 \text{ s} = 14.4 \text{ concurrent queries for the same row}$$

That alone is survivable. Now do it for the whole service at peak, when a deploy flushes the cache:

$$9\{, \}259 \text{ QPS} \times 0.012 \text{ s} = 111 \text{ concurrent database queries}$$

A typical Postgres connection pool is 50 connections. $111 > 50$, so half the requests queue, latency climbs, clients time out and **retry**, which adds more load. This is how a cache flush takes down a database.

The fix: singleflight. Let exactly one request per key go to the database. Everyone else waits on the same promise.

singleflight.js — 500 callers, one database query

JS

```
class SingleFlight {
  constructor() { this.inFlight = new Map(); } // key -> promise
  run(key, work) {
    if (this.inFlight.has(key)) return this.inFlight.get(key); // join the queue
    const p = work().finally(() => this.inFlight.delete(key)); // always clean up
    this.inFlight.set(key, p);
    return p;
  }
}

async function getProduct(id) {
  const key = `product:${id}`;
  if (cache.has(key)) return cache.get(key);
  return sf.run(key, async () => {
    if (cache.has(key)) return cache.get(key); // someone filled it while we waited
    const row = await loadFromDb(id);
    cache.set(key, row);
    return row;
  });
}
```

node singleflight.js

TEXT

```
all callers got the same value: true
DB queries fired: 1
DB queries without singleflight: 500
```

TRAP

Two lines in that code are not decoration.

`.finally(() => this.inFlight.delete(key))` – without it, a **failed** fetch leaves a rejected promise in the map forever, and every future caller gets the same old error. The cache becomes permanently broken for that key.

The **second** `cache.has(key)` check inside the worker – the “double-check”. Between joining and running, another flight may have already filled the cache.

Singleflight is per process. With 3 app servers you get 3 database queries per expiry, not 1. That is fine – 3 is not 111. If you truly need exactly one, take a short Redis lock (`SET lock:key token NX PX 5000`) and let the losers sleep 20 ms and re-read. Say this only if asked; the per-process version is the right default because it needs no extra network call.

The better fix, when you can use it: refresh before expiry. Instead of waiting for the key to die, serve the slightly-old value and refresh in the background:

Strategy	What the user sees on an expiry
Plain TTL	one user waits 12 ms; 110 others queue behind the DB
Singleflight	one user waits 12 ms; 110 others wait 12 ms on the same query, DB does 1 query
Stale-while-revalidate	nobody waits. Everyone gets the old value in 1 ms; one background job refreshes

Decision: use both. Serve stale for up to 120 seconds past expiry (`stale-while-revalidate=120` in the header, and the same rule inside Redis by storing a `softExpiresAt` next to `expiresAt`), and guard the background refresh with singleflight. Prices may be 60 seconds stale by requirement, so serving a 90-second-old price for one extra millisecond is inside budget. **We accept up to 2 minutes of staleness in exchange for never making a user wait on a database.**

Step 7 – Trade-offs, failures, and 10x

Decision	Chosen	Rejected	Why
Pattern	cache-aside	write-through	200:1 read ratio means most writes would cache data nobody reads; and write-through couples every write to cache availability
Invalidate	delete the key	update the key	delete is idempotent, so concurrent writers cannot interleave into a wrong final value
Price storage	separate short-TTL key	inside the product blob	a price change would otherwise throw away 4 KB of stable data
Local in-process cache	yes, 2-second TTL	Redis only	it absorbs hot keys for free; 2 seconds is short enough that price staleness stays inside the 60-second budget
Herd control	stale-while-revalidate + single-flight	distributed Redis lock	a lock adds a network round trip on the hot path; the cheaper pair already reduces 111 queries to 3

Failure modes, and what actually happens:

What fails	Blast radius	Our response
Redis node dies	hit rate goes to 0; DB faces 9,300 QPS against a 465 QPS budget – 20x overload	the 2-second local cache absorbs repeats; a concurrency limiter caps DB calls at 300 and sheds the rest with 503 Retry-After: 2 ; serve stale from local cache where we have it
Redis is slow, not dead (worst case)	every request pays the Redis timeout and the DB	set a hard 50 ms Redis timeout and treat a timeout as a miss; a circuit breaker skips Redis entirely after 5 timeouts
Postgres primary fails over	30 – 60 s of write errors; reads still served from cache	raise TTLs automatically during a DB outage – a stale catalog beats no catalog
A bad deploy flushes the cache	the 111-concurrent-query stampede above	never flush on deploy; use versioned keys so old and new coexist; warm the

		top 10,000 products at boot before taking traffic
One product goes viral	one Redis key gets 3,000 QPS	the local cache handles it; see Tier 3 for the general fix

What changes at 10x (80M DAU, 93,000 peak QPS)?

$$80\{, \}000\{, \}000 \times 25 = 2\{, \}000\{, \}000\{, \}000 \text{ reads/day}$$

$$2\{, \}000\{, \}000\{, \}000 \times \frac{1000}{86} = 23\{, \}148 \text{ QPS average}$$

$$23\{, \}148 \times 4 = 92\{, \}592 \text{ QPS peak}$$

Three things break, in this order:

1. **Redis bandwidth.** $92\{, \}592 \times 4 \text{ KB} = 370\{, \}368 \text{ KB/s} = 370 \text{ MB/s} = 2.96 \text{ Gbit/s}$. That is past a single 1 Gbit/s node. **Fix: shard Redis by key hash across 4 nodes** ($\frac{2.96}{4} = 0.74 \text{ Gbit/s}$ each), or add read replicas.
2. **Database misses.** $92\{, \}592 \times 0.05 = 4\{, \}630 \text{ QPS}$ to Postgres. One primary cannot do that on 4 KB rows. **Fix: read replicas, 3 of them, $4\{, \} \times \frac{630}{3} = 1\{, \}543 \text{ QPS}$ each.**
3. **Hot keys.** At 10x, a viral product is 10x hotter and the local cache alone will not save a single Redis shard. This is the Tier 3 problem.

Tier 3 · Large-scale: a global CDN for images

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 11 [Amazon · pattern]

The same shopping company now serves 60 million daily active users across India, Indonesia and Singapore. Each session loads about 40 images averaging 120 KB. Design the image delivery system. Then handle the two hard cases: a single hot object, and a mistaken upload that must be removed from every edge in under 60 seconds.

SOLUTION

Step 1 – Clarify

Question	Why it matters
Are images immutable once uploaded?	If yes, I can cache them for a year and never invalidate – this is the single biggest simplification available.
Who may see an image – everyone, or only some users?	Public images cache at the edge. Private images cannot, unless I use signed URLs.
How many distinct images, and how skewed is access?	Sets the edge disk size per POP.
What is the hard purge requirement?	A legal takedown in 60 seconds is a very different system from “it clears within a day”.
Which countries, and is there a data residency rule?	Residency can forbid caching certain bytes outside a country.

Assume: images are **immutable** (a new upload gets a new URL); public; 50 million distinct images; 60-second purge requirement for takedowns.

Step 2 – Scale estimate

Request rate.

$$60\{, \}000\{, \}000 \times 40 = 2\{, \}400\{, \}000\{, \}000 \text{ image requests/day}$$

$$2.4 \times \frac{10^9}{86} \{, \}400 = 27\{, \}778 \text{ QPS average}$$

$$27\{, \}778 \times 3 = 83\{, \}333 \text{ QPS peak}$$

Bytes.

$$2.4 \times 10^9 \times 120 \text{ KB} = 2.88 \times 10^{14} \text{ bytes/day} = 288 \text{ TB/day}$$

$$\text{bytes per second} = 288 \times \frac{10^{12}}{86} \{, \}400 = 3.33 \times 10^9 = 3.33 \text{ GB/s}$$

$$\text{bits per second} = 3.33 \text{ GB/s} \times 8 = 26.7 \text{ Gbit/s average}$$

$$\text{peak} = 26.7 \times 3 = 80 \text{ Gbit/s}$$

This number is the whole design. 80 Gbit/s out of one data centre is possible but expensive and fragile. 80 Gbit/s spread over 40 edge locations is 2 Gbit/s each, which is ordinary.

Origin load, if the CDN hits 90%.

$$80 \text{ Gbit/s} \times 0.10 = 8 \text{ Gbit/s peak out of origin}$$

$$83\{, \}333 \text{ QPS} \times 0.10 = 8\{, \}333 \text{ QPS to origin}$$

Storage.

$$50\{, \}000\{, \}000 \times 200 \text{ KB (original + thumbnails)} = 10 \times 10^{12} = 10 \text{ TB in object storage}$$

Hot 5% at each edge:

$$50\{, \}000\{, \}000 \times 0.05 \times 200 \text{ KB} = 500 \text{ GB per POP}$$

Half a terabyte of SSD per POP is a normal edge server.

Per-POP request load with 40 POPs:

$$27\{, \} \frac{778}{40} = 694 \text{ QPS average per POP, } 83\{, \} \frac{333}{40} = 2\{, \}083 \text{ QPS peak per POP}$$

The money argument, which wins the interview. Say origin egress costs \$0.09 per GB and CDN egress costs \$0.02 per GB.

$$\text{bytes per day} = 288\{, \}000 \text{ GB}$$

$$\text{all from origin} = 288\{, \}000 \times 0.09 = \$25\{, \}920 \text{ per day}$$

$$90\% \text{ CDN, } 10\% \text{ origin} = 288\{, \}000 \times 0.9 \times 0.02 + 288\{, \}000 \times 0.1 \times 0.09$$

$$= 5\{, \}184 + 2\{, \}592 = \$7\{, \}776 \text{ per day}$$

The CDN saves about \$18,144 a day, roughly \$6.6 million a year. Say that number. It turns an architecture choice into a business decision, which is what a senior interviewer is listening for.

Step 3 – API surface

Public read + private upload

TEXT

```
GET https://img.cdn.example.com/v1/{imageId}/{width}x{height}.webp
200 OK
Cache-Control: public, max-age=31536000, immutable
ETag: "sha256-9f2c..."
Vary: Accept
304 Not Modified (when If-None-Match matches)
```

```
POST /v1/images <- origin only, authenticated
multipart body: file
```

```
201 -> { "imageId":"im_8Kq2", "variants":["320x320","800x800","orig"],
        "url":"https://img.cdn.example.com/v1/im_8Kq2/800x800.webp" }
```

```
POST /internal/purge <- takedown, authenticated, rate limited
  body { "imageIds":["im_8Kq2"], "reason":"legal" }
202 -> { "purgeId":"pg_44", "pops":40, "eta_seconds":25 }
```

```
GET /internal/purge/{purgeId}
200 -> { "acked": 40, "pending": 0, "completed_at":"..." }
```

Why immutable in the Cache-Control header? It tells the browser not to even send a revalidation request when the user hits reload. Without it, a reload fires an If-None-Match request per image – 40 requests that all return 304 Not Modified. With it, 40 requests become zero. Because our IDs are content-addressed (the URL contains a hash), the image at a URL can never change, so this is safe.

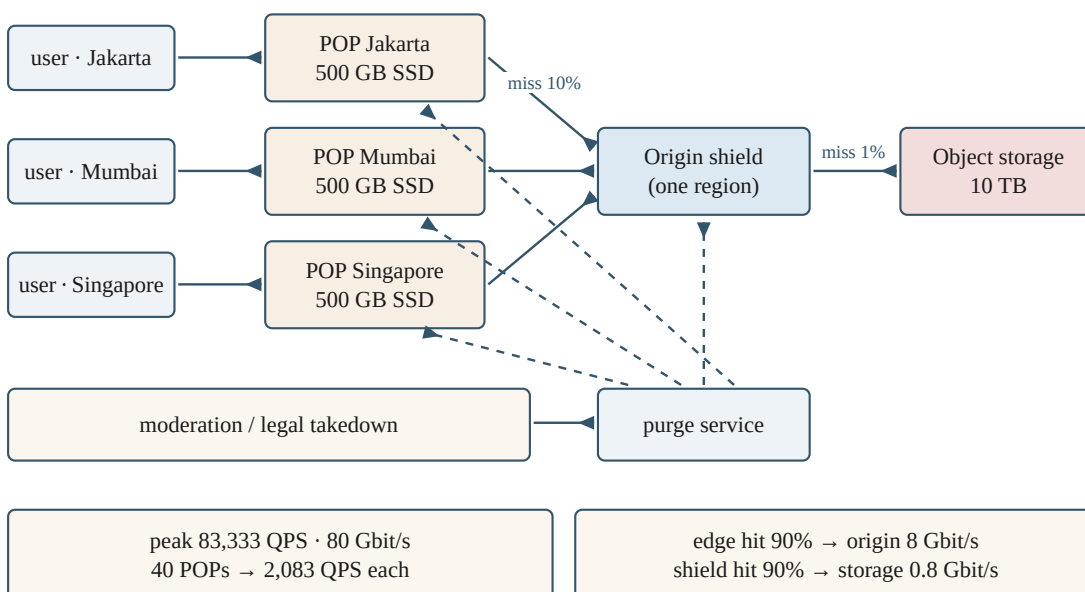
Step 4 – Data model

Store	Key	Holds
Object storage (S3-like)	images/{imageId}/{variant}.webp	the actual bytes; the source of truth
Postgres images	image_id (PK)	owner, sha256, width, height, uploaded_at, status
Postgres purges	purge_id (PK)	image ids, requested_at, acked_pops int
Edge disk (each POP)	the URL path	the bytes, plus last_used_at for LRU
Edge memory (each POP)	the URL path	the hottest ~2 GB

The key decision: the URL contains a content hash. imageId is derived from the SHA-256 of the bytes. Two consequences:

1. The same image uploaded twice gets the same ID – free deduplication.
2. An image at a given URL **can never change**, so we never need to invalidate for correctness, only for takedowns. Cache invalidation, the famously hard problem, is deleted from the design by choosing the right key.

Step 5 – Architecture



Three POPs, an origin shield, and object storage. Dashed lines are the purge fan-out.

The origin shield: why the middle box exists

Without a shield, every one of the 40 POPs independently misses on a **newly popular** image and calls object storage. One new viral image costs 40 origin fetches. With a shield, the first POP to miss pulls the object into the shield; the other 39 get it from the shield.

without shield = 40 origin fetches per new object

with shield = 1 origin fetch + 39 shield fetches

At a 10% edge miss rate, the shield sees $83\{, \}333 \times 0.10 = 8\{, \}333$ QPS. If the shield itself hits 90%, object storage sees:

$$8\{, \}333 \times 0.10 = 833 \text{ QPS}$$

and bandwidth:

$$80 \text{ Gbit/s} \times 0.10 \times 0.10 = 0.8 \text{ Gbit/s}$$

Decision: use a shield. Cost: one extra hop, about 20 ms, on 10% of requests, which moves the average by $0.10 \times 20 = 2$ ms. Benefit: object storage load drops 10x and the origin egress bill drops with it. Two milliseconds for a 10x reduction is an easy trade.

Step 6 – Deep dive 1: the hot key

The problem. A celebrity posts one product. That single image is now 25% of all image traffic.

$$83\{, \}333 \text{ QPS} \times 0.25 = 20\{, \}833 \text{ QPS for one object}$$

$$20\{, \}833 \times 120 \text{ KB} = 2.5 \times 10^9 \text{ bytes/s} = 20 \text{ Gbit/s for one object}$$

At the **edge** this is fine – it is spread over 40 POPs, so each POP serves $20\{, \}\frac{833}{40} = 521$ QPS of it from local SSD. Edges are built for exactly this.

The danger is in the **dynamic** cache, where the same object lives on exactly one Redis shard. There, one key means one CPU core on one machine.

Three fixes, and which to pick.

Fix	How it works	Cost
Local (in-process) cache	each app server keeps the hot value in its own Map for 1 – 2 seconds	staleness up to 2 s; zero network cost; needs no coordination
Key splitting / replication	store the value under <code>hot:{id}:0 .. hot:{id}:9</code> ; each reader picks a random suffix	10x the memory for that key; invalidation must delete 10 keys
Client-side hedging	read from 2 replicas, take the first answer	2x the read traffic – makes the hot-key problem worse, not better

Decision: local cache first, key splitting only if that is not enough.

The arithmetic for why local cache is usually enough. With 30 app servers each holding the value for 2 seconds, the **maximum** number of Redis reads for that key is:

$$30 \frac{\text{servers}}{2} \text{ seconds} = 15 \text{ reads per second}$$

We went from 20,833 QPS on one Redis key to 15. No key splitting required, no extra memory, no invalidation complexity. **The 2-second staleness is the entire price, and for an image URL – which is immutable – the staleness costs nothing at all.**

Key splitting into 10 replicas would give:

$$20\{, \}\frac{833}{10} = 2\{, \}083 \text{ QPS per replica key}$$

which is also fine, but it costs 10x memory for that key and turns one DEL into ten. Only reach for it when values must be fresher than a local cache allows.

TRAP

A local in-process cache is **not** a smaller Redis. Two differences bite people:

(1) **Every server has a different copy.** Two users refreshing at the same moment can see different values for up to the local TTL. Make the local TTL short (1 – 2 s) and only use it for data where that is acceptable.

(2) **Memory is per process, not per machine.** Node running 8 workers with a 500 MB local cache uses 4 GB, not 500 MB. Size it as `perProcess x workers`.

Step 6 – Deep dive 2: purge in under 60 seconds

A takedown must remove an object from 40 POPs within 60 seconds. Three designs; pick one.

Design	How	Time to clear	Verdict
Short TTL everywhere	set <code>max-age=60</code> on every object	60 s	Rejected. It destroys the hit rate: every object revalidates every minute, so origin traffic goes up roughly 100x. You pay for 50 million objects to solve a problem that affects a handful.
Purge broadcast	push a delete to all 40 POPs over a pub/sub fabric, collect acks	1 – 5 s	Chosen. Costs nothing in steady state; only runs when a takedown happens.
Generation counter	prefix every path with a global generation number; bump it to invalidate	instant	Rejected for takedowns – bumping the generation invalidates everything , a 288 TB re-fetch. It is the right tool for “deploy new site assets”, not for one image.

The chosen design, step by step:

1. Moderation calls `POST /internal/purge` with the image ids.
2. The purge service writes a row (`purge_id`, `ids`, `acked_pops = 0`) so the request survives a crash.
3. It publishes the ids on a fan-out topic that all 40 POPs subscribe to.
4. Each POP deletes the paths from memory and disk, then acks with its POP id.
5. The purge service counts acks. At 40/40 it marks the purge complete.
6. Any POP that has not acked within 10 seconds is **re-sent** the message. A POP that has been offline replays the purge log on boot before it starts serving.

The hole you must mention before the interviewer does: the bytes are also in **browser** caches, with `max-age=31536000`. You cannot purge those. The only real answer is to make the URL unreachable at the origin **and** rotate the signing key so old URLs stop verifying; already-downloaded copies in a user’s browser are simply gone from your control. Saying this plainly – “here is the part I cannot fix, and here is why” – scores better than pretending the purge is total.

Step 6 – Deep dive 3: consistent hashing, with measurements

Inside a POP there are several cache servers, and the same problem appears one level down: **which server holds which object?** If you use `hash(key) % N`, adding one server moves almost every key.

ring.js — a hash ring with virtual nodes

JS

```
// FNV-1a plus a mixing ("avalanche") tail. The tail matters – see the experiment below.
const hash32 = (s) => {
  let h = 0x811c9dc5;
  for (let i = 0; i < s.length; i++) {
    h ^= s.charCodeAt(i);
    h = Math.imul(h, 0x01000193) >>> 0; // >>> 0 keeps it unsigned 32-bit
  }
}
```



```

    }
    h ^= h >>> 16; h = Math.imul(h, 0x85ebca6b) >>> 0;
    h ^= h >>> 13; h = Math.imul(h, 0xc2b2ae35) >>> 0;
    h ^= h >>> 16;
    return h >>> 0;
};

class HashRing {
  constructor(nodes = [], vnodes = 150) {
    this.v = vnodes;
    this.ring = []; // sorted list of { pos, node }
    nodes.forEach(n => this.addNode(n));
  }
  addNode(node) {
    for (let i = 0; i < this.v; i++) this.ring.push({ pos: hash32(`${node}#${i}`),
node });
    this.ring.sort((a, b) => a.pos - b.pos); // numeric compare – never a
bare .sort()
  }
  removeNode(node) { this.ring = this.ring.filter(e => e.node !== node); }
  nodeFor(key) {
    const h = hash32(key);
    let lo = 0, hi = this.ring.length; // first ring point with pos >= h
    while (lo < hi) { const m = (lo + hi) >> 1; this.ring[m].pos < h ? lo = m + 1 :
hi = m; }
    return this.ring[lo % this.ring.length].node; // wrap past the top of the
circle
  }
}

```

node ring.js — 4 nodes, 100,000 keys

TEXT

```

vnodes= 1  33.8%  5.2%  31.2%  29.8%
vnodes= 10 20.2% 19.3% 19.4% 41.1%
vnodes=150 25.4% 25.3% 22.4% 26.9%
add a 5th node -> 19.9% of keys move (ideal 1/5 = 20%)
plain modulo   -> 80.2% of keys move

```

Read the output carefully; there are three lessons in five lines.

(1) One point per node is useless. With `vnodes=1`, one node got 5.2% of the keys and another got 33.8% – a 6.5x imbalance. Four random points on a circle simply do not cut it into four equal arcs.

(2) Virtual nodes fix it, and 150 is the standard number. At 150 points per node the spread is 22.4% to 26.9% – within a few points of the ideal 25%. The general rule: imbalance shrinks roughly as $\frac{1}{\sqrt{V}}$ where V is virtual nodes per real node. Going from 1 to 150 shrinks it by about $\sqrt{150} \approx 12$ times.

(3) This is the entire point of the ring. Adding a fifth node moved **19.9%** of keys – essentially the theoretical minimum of $\frac{1}{5} = 20\%$. Plain hash % N moved **80.2%**.

At our scale, “moved” means “must be re-fetched from the shield”. Adding one server to a POP:

consistent hashing: $500 \text{ GB} \times 0.199 = 99.5 \text{ GB}$ re-fetched

plain modulo: $500 \text{ GB} \times 0.802 = 401 \text{ GB}$ re-fetched

Decision: consistent hashing with 150 virtual nodes. It costs a binary search per lookup ($\log_2 600 \approx 10$ steps – nothing) and saves 300 GB of origin traffic **per server added, per POP**.

TRAP

Look at the `.sort((a, b) => a.pos - b.pos)`. A bare `.sort()` in JavaScript sorts **lexicographically**, so ring position 1000000000 would come before 999999999 and the ring would be silently wrong – not crashed, just wrong. Every key would go to the wrong node. This is the single most expensive one-character mistake in JavaScript system design.

Why the avalanche tail matters. The first version of this code used plain FNV-1a with no mixing tail. With 150 virtual nodes the split was 11.9% / 20.7% / 21.1% / **46.2%** – one node taking almost half the traffic. FNV-1a's final bits are not well mixed for short, nearly identical strings like `cache-3#0`, `cache-3#1`, so the virtual points clustered. Adding the three-step mixing tail fixed it completely. **When a hash ring is unbalanced, suspect the hash before you suspect the ring.**

Step 7 – Trade-offs, failures, and 10x

Decision	Chosen	Rejected	Why
Image URLs	content hash, immutable, 1-year TTL	mutable URLs with short TTL	immutability removes invalidation from the design entirely; the cost is that an “edit” is really an upload, which we accept
Origin protection	origin shield in one region	POPs talk to storage directly	a shield turns 40 origin fetches per new object into 1; costs 2 ms of average latency
Takedown	purge broadcast with acks	short TTL on everything	short TTLs would raise origin traffic 100x to solve a rare event
POP sharding	consistent hashing, 150 vnodes	hash % N	measured: 19.9% vs 80.2% of keys move when a server is added
Hot object	rely on 40-way edge spread + local cache	key splitting	edges already spread it 40 ways; splitting adds memory and invalidation cost for no gain

Failure modes:

What fails	Effect	Response
One POP goes down	its users are routed to the next nearest POP: latency goes from 20 ms to 60 ms; that POP's 2,083 peak QPS lands on a neighbour	size every POP with 40% headroom so a neighbour can absorb one failure; anycast routing moves users automatically
The shield region goes down	all 8,333 miss-QPS hit object storage directly – 10x its normal load	run two shields in different regions, POPs pick by health; object storage can take 8,333 QPS, it is just slower and dearer
Object storage is slow	misses time out, users see broken images	serve stale past expiry; a 6-hour-old image beats a broken one; circuit-break to a placeholder after 5 s
A purge message is lost	illegal content stays live at one POP	acks + retry every 10 s + replay the purge log on POP boot; alert if any POP is unacked after 60 s
Cache poisoning via a bad Vary	one user's private image served to others	never set Vary: Cookie on public paths; serve private media from a separate hostname with Cache-Control: private

At 10x (600M DAU):

$$600\{, \}000\{, \}000 \times 40 = 24 \times 10^9 \text{ requests/day}$$

$$24 \times \frac{10^9}{86} \{, \}400 = 277\{, \}778 \text{ QPS average; } \times 3 = 833\{, \}333 \text{ QPS peak}$$

$$\text{bandwidth} = 26.7 \times 10 = 267 \text{ Gbit/s average; peak} = 800 \text{ Gbit/s}$$

With 40 POPs that is $\frac{800}{40} = 20$ Gbit/s per POP – too much for one edge server. Three changes:

1. **More POPs, not bigger POPs.** Go to 200 POPs: $\frac{800}{200} = 4$ Gbit/s each. This also cuts user latency, because more POPs means a closer POP.
2. **Push image resizing to the edge.** Today we store 4 variants per image ($50\text{M} \times 200 \text{ KB} = 10 \text{ TB}$). At 10x with more device sizes that becomes unmanageable. Store **one** original and resize at the edge on first request, then cache the result. Storage drops from 10 TB to about $500 \times 10^6 \times 150 \text{ KB} = 75 \text{ TB}$ of originals instead of 4 × that in variants.
3. **Tiered edge caching.** A 500 GB POP at 10x traffic has a worse hit rate because the working set grew. Add a regional mid-tier between POP and shield: POP (hot 1%) → region (hot 20%) → shield → storage.

Interview drill

These are the pushes that come **after** your first answer. The answer is what a strong candidate says in 30 seconds.

Push 1: “Your cache is 95% effective. Why not just add a read replica instead?”

Answer. Because they solve different problems and the numbers are not close. A read replica gives me another 12 ms path with maybe 500 QPS of headroom; Redis gives me a 1 ms path with 100,000 QPS of headroom, for a fraction of the cost. Concretely: to absorb the same 8,835 peak QPS I would need about 18 replicas at 500 QPS each, versus one Redis node.

But I would use **both**. The replica is what the cache falls back to on a miss, and it is what protects the primary when Redis dies. They are layers, not alternatives.

Push 2: “What is your cache key? Be exact.”

Answer. `product:{id}:v{version}` for the blob, `price:{id}:{region}` for the price.

Three rules I follow for any cache key:

1. **Everything the response depends on must be in the key.** If the response changes by region, region goes in the key. If it changes by user tier, tier goes in the key. A key that forgets a dimension serves one user’s answer to another – that is a correctness bug, not a performance bug.
2. **Prefix by entity type** so I can find, count and mass-delete a family of keys.
3. **No user-controlled strings unescaped.** A product title in a key lets an attacker create colliding or unbounded keys.

Push 3: “A deploy flushed Redis at peak. Walk me through the next 60 seconds.”

Answer, with numbers.

Second 0: hit rate goes 95% → 0%. Database demand jumps from 465 QPS to 9,300 QPS – **20x over budget**.

Second 0 – 2: the 2-second in-process cache is also cold, so it does not help yet. The concurrency limiter in front of Postgres caps in-flight queries at 300. Everything above that gets 503 with `Retry-After: 2`. We shed load on purpose rather than letting the database queue and die.

Second 2 – 20: each app server’s local cache fills with the hottest products. Effective DB demand drops fast because the top 1,000 products are most of the traffic. Singleflight means each distinct key costs exactly one query per server, not one per request.

Second 20 – 60: Redis refills from those misses. Hit rate climbs back through 60%, 80%, 90%.

The real fix is prevention, and I would say so: never flush on deploy. Use versioned keys so the new code writes v42 while v41 is still serving, and warm the top 10,000 products before the new instances take traffic. A cache flush should never be a normal part of a deploy.

Push 4: “When would you NOT cache?”

I Answer – four cases, and I have hit all four.

1. **When the read to write ratio is near 1.** You pay the write cost to invalidate and almost never get a hit. Below roughly 5:1, measure before you cache.
2. **When staleness is not allowed at all.** A bank balance on a transfer screen, remaining seats at the moment of booking, a one-time password. Here you read the primary, and you make the primary fast with an index instead.
3. **When the value is enormous and the hit rate is low.** Caching 5 MB search results with a 20% hit rate spends a lot of memory to avoid a few queries.
4. **When the query is already fast.** A primary-key lookup on a small table that returns in 0.4 ms does not need a 0.5 ms Redis round trip in front of it. **A cache can be slower than the thing it caches.** That surprises people, so it is worth saying.

Push 5: “Redis or Memcached?”

Answer: Redis, and here is the deciding reason.

Memcached is genuinely good at one thing – a plain key-value string cache, multi-threaded, very low overhead per key. If my need were literally that and nothing else, it wins on memory efficiency.

But almost every real design needs at least one of: sorted sets (leaderboards, rate limit windows), atomic increments with expiry, pub/sub for invalidation fan-out, or persistence so a restart does not start cold. Redis has all four. The moment I need one of them, running both systems is worse than running Redis alone.

Decision: Redis. I accept the single-threaded command loop – one core per shard – and plan for it by sharding, which I have to do for memory anyway.

Push 6: “How do you measure whether the cache is working?”

Answer – four dashboards, not one.

1. **Hit rate, per key prefix.** One global number hides everything. `product : *` at 97% and `search : *` at 12% is a completely different story from “the cache is at 90%”.
2. **Origin QPS.** This is the number the cache exists to reduce. If hit rate is up but origin QPS is flat, the extra hits were on keys nobody was querying anyway.
3. **p99 latency, split by hit and miss.** The average lies. p99 is dominated by misses.
4. **Eviction rate.** If evictions are high while hit rate is fine, the cache is thrashing and is about to fall off a cliff when traffic grows 20%. This is the leading indicator.

PRACTICE · WARM-UP target: 12 min

1. A service does 6,000 QPS with an 88% hit rate. Hits cost 1 ms, misses cost 1 ms plus a 15 ms database read. Find the database QPS and the average latency.
2. The team wants the database at or below 300 QPS at the same 6,000 QPS of traffic. What hit rate is required?
3. A cache holds 4 items, LRU. Trace A B C D A E B A C and give the hit count and the final contents.
4. 2 million user sessions, 1.5 KB each. Compute total memory, and the provisioned size with 30% overhead.
5. A key with a 10-minute TTL is read 500 times a second. On expiry the database takes 20 ms. How many concurrent database queries fire without singleflight?

ANSWER KEY

1. Misses = 12%. DB QPS = $6000 \times 0.12 = 720$. Average = $0.88 \times 1 + 0.12 \times 16 = 0.88 + 1.92 = 2.80$ ms.
2. Allowed miss rate = $\frac{300}{6000} = 0.05 = 5\%$. So the required hit rate is **95%**. (Note this cuts DB load from 720 to 300, a 2.4x reduction, for 7 points of hit rate.)
3. Steps: A miss [A]; B miss [B,A]; C miss [C,B,A]; D miss [D,C,B,A]; A **hit** [A,D,C,B]; E miss, evict B [E,A,D,C]; B miss, evict C [B,E,A,D]; A **hit** [A,B,E,D]; C miss, evict D [C,A,B,E]. **2 hits, 7 misses**; final contents C, A, B, E (C newest).
4. $2 \times 10^6 \times 1.5 \text{ KB} = 3 \times 10^6 \text{ KB} = 3 \text{ GB}$. With overhead: $3 \times 1.3 = 3.9 \text{ GB}$. Provision a 4 GB instance.
5. $500 \times 0.020 = 10$ concurrent queries for the same key. With singleflight: 1 per process.

PRACTICE · TIER 2 target: 35 min

1. Design the cache layer for a “recently viewed products” list: 8M DAU, each user’s list holds 20 product ids, each user views 25 products a day. Give the key, the value type, the TTL, the memory, and the write QPS. Decide between Redis lists and a JSON blob, and justify the choice.
2. A category page shows 40 products. Today the service makes 40 separate Redis calls per page at 0.6 ms each. At 2,000 page-views per second, compute the Redis QPS and the time spent in Redis per page. Then redesign it and compute the new numbers.
3. Your `search:{query}` cache has a 12% hit rate and uses 8 GB. Decide whether to keep it, and say exactly what you would change. Show the arithmetic behind your decision.

ANSWER KEY

1. Key `recent:{userId}`, value a Redis list of 20 ids ($20 \times 12 \text{ bytes} = 240 \text{ B}$; with overhead call it 400 B).

Memory: $8\{, \}000\{, \}000 \times 400 \text{ B} = 3.2 \times 10^9 = 3.2 \text{ GB}$. With 30% overhead, provision 4.2 GB.

Writes: $8\{, \}000\{, \}000 \times 25 = 200\{, \}000\{, \}000$ per day $= 200 \times \frac{10^6}{86} \{, \}400 = 2\{, \}315$ QPS average, peak $4x = 9\{, \}260$ QPS.

TTL: 30 days, refreshed on every write.

Decision: Redis list, not a JSON blob. LPUSH + LTRIM 0 19 is one atomic round trip and writes 12 bytes. A JSON blob forces read-modify-write: 2 round trips, 400 bytes written each time, **and** a lost-update race between two devices of the same user. At 9,260 peak write QPS the blob approach doubles the Redis op rate to 18,520 for no benefit.

2. Today: $2\{, \}000 \times 40 = 80\{, \}000$ Redis QPS; time in Redis per page $= 40 \times 0.6 = 24 \text{ ms}$ (if sequential).

Redesign with MGET of 40 keys: **2,000 Redis QPS** (one call per page) and about **0.9 ms** per page (one round trip; a 40-key MGET is barely slower than a 1-key GET because the cost is dominated by the network hop).

That is a **40x** drop in Redis operations and a **26x** drop in page latency from one change.

3. **Decision: keep it, but shrink it and split the key space.**

A 12% hit rate is not automatically bad – what matters is what those hits cost to produce. A search query costs, say, 80 ms of database work. At 500 searches/second, 12% hits save $500 \times 0.12 = 60$ searches/second $\times 80 \text{ ms} = 4.8$ seconds of database time per second – the cache is doing the work of roughly 5 dedicated database cores. That is worth keeping.

What I change: (a) 8 GB for 12% means most entries are never read again – long-tail queries. Cache only queries seen 2+ times in the last hour (a small count-min sketch in front of the cache), which should drop memory to ~1 GB while keeping most of the hits. (b) Split into `search:hot:*` with a 10-minute TTL and `search:cold:*` with 60 seconds, so the tail expires quickly. (c) Alarm on hit rate per prefix, not globally.

PRACTICE · TIER 3 target: 45 min

1. A video thumbnail service serves 200M requests/day at 45 KB average. Compute daily bytes, average and peak (4x) bandwidth in Gbit/s, and the daily cost difference between 0% and 92% CDN hit rate at \$0.02/GB (CDN) and \$0.09/GB (origin).
2. You have 6 Redis shards. One key is receiving 35,000 QPS and its shard is at 100% CPU. The value is a counter that must be accurate within 1 second. Local caching for 2 seconds is therefore not allowed. Design a fix, show the arithmetic, and state the new staleness.
3. Your CDN hit rate dropped from 94% to 61% overnight with no traffic change and no deploy. List five possible causes, and for each say the **one** metric or log field that would confirm it.

ANSWER KEY

1. Bytes: $200 \times 10^6 \times 45 \text{ KB} = 9 \times 10^{12} \text{ B} = 9 \text{ TB/day}$.

Average: $9 \times \frac{10^{12}}{86} \{, \}400 = 1.042 \times 10^8 \text{ B/s} = 0.104 \text{ GB/s} \times 8 = 0.833 \text{ Gbit/s}$. Peak $4x = 3.33 \text{ Gbit/s}$.

Cost at 0% CDN: $9\{, \}000 \text{ GB} \times 0.09 = \$810/\text{day}$. At 92%: $9\{, \}000 \times 0.92 \times 0.02 + 9\{, \}000 \times 0.08 \times 0.09 = 165.6 + 64.8 = \$230.40/\text{day}$. **Saving \$579.60/day, about \$211,554/year.**

2. A counter that must be accurate within 1 second rules out a 2-second local cache, so use **key splitting on the write side and a fan-in on the read side**.

Write path: shard the counter into 12 keys `count:{id}:0 .. count:{id}:11`, each on a different shard (put the suffix in the hash tag so Redis Cluster spreads them). Each increment picks a random suffix. Write load per key: $35\{, \} \frac{000}{12} = 2\{, \} 917$ QPS – comfortable.

Read path: the true total is the sum of 12 keys. Do not sum on every read. Run one background job per app server that does a 12-key MGET, sums, and stores the result in a local variable **every 500 ms**. Reads then serve that local number at zero Redis cost.

New staleness: up to **500 ms**, inside the 1-second budget. Redis read load for the fan-in with 30 servers: $30 \times 2 = 60$ MGETs per second. We went from 35,000 QPS on one shard to 2,917 QPS on each of 12, plus 60 reads.

3. Five causes and the confirming signal:

Cause	What confirms it
A new query string is being appended to URLs (analytics tag), so every URL is unique	the <code>cache_key</code> field in edge logs – look for <code>?utm_</code> or <code>?t=</code> suffixes; check distinct-URL count
An origin change started sending <code>Cache-Control: no-store</code> or <code>private</code>	the response header logged at the edge on a miss-fill
Vary header widened (e.g. <code>Vary: Cookie</code> or <code>Vary: User-Agent</code>)	the <code>Vary</code> value in edge logs; distinct variants per URL exploded
A POP lost disk capacity or was rebuilt, so its working set is cold	eviction rate and <code>cache_disk_free</code> per POP; hit rate will be bad at one POP, not all
A crawler is walking the long tail of 50M objects	unique-URL rate and the user-agent / ASN breakdown in edge logs

ONE-PAGE REVISION CARD

The three questions for any cache

1. What goes in? (key, value, size)
2. When does it come out? (eviction = full; expiry = old)
3. What breaks when it is wrong? (stale, herd, hot key, penetration, avalanche)

Numbers to memorise

in-process Map	$0.1 \mu s$	Redis same DC	0.5 – 1 ms
SSD read	0.1 ms	indexed SQL row	5 – 15 ms
same-region RTT	1 ms	India ↔ US RTT	180 ms
CDN edge	10 – 30 ms	RAM vs DB	~100x faster

The formulas

- $DB \text{ QPS} = \text{total QPS} \times (1 - \text{hit rate})$
- $\text{avg latency} = h \times t_{\text{hit}} + (1 - h) \times (t_{\text{hit}} + t_{\text{miss}})$
- $\text{concurrent misses on expiry} = \text{QPS on that key} \times \text{DB latency in seconds}$
- $\text{cache memory} = \text{working set} \times \text{value size} \times 1.3$
- $\text{max hit rate} = 1 - \frac{\text{distinct keys}}{\text{requests in the window}}$
- $\text{bandwidth (Gbit/s)} = \frac{\text{bytes/day}}{86400} \times \frac{8}{10^9}$

Worked numbers from this chapter

- $8M \text{ DAU} \times 25 \text{ reads} = 200M/\text{day} = 2,315 \text{ QPS avg, } 9,259 \text{ peak (4x)}$
- $95\% \text{ hit} \rightarrow \text{Redis } 8,835 \text{ QPS, DB } 465 \text{ QPS, avg } 1.6 \text{ ms}$
- $60M \text{ DAU} \times 40 \text{ images} = 2.4B/\text{day} = 27,778 \text{ QPS, } 288 \text{ TB/day, } 26.7 \text{ Gbit/s (80 peak)}$
- $\text{CDN at } 90\% \text{ saves } \$18,144/\text{day on that traffic}$
- $\text{consistent hashing (150 vnodes): } 19.9\% \text{ of keys move when adding a node; hash \% N: } 80.2\%$

The five failures and their one-line fixes

1. stale $\rightarrow$ delete on write (never update), short TTL
2. thundering herd $\rightarrow$ singleflight + stale-while-revalidate
3. hot key $\rightarrow$ in-process cache for 1 – 2 s; key splitting if it must be fresher
4. penetration $\rightarrow$ cache the 404 for 30 s
5. avalanche $\rightarrow$ TTL jitter $\pm 20\%$, warm on boot, never flush on deploy

Order of operations on a write – never get this wrong write the source of truth **first**, then delete the cache key. Never the reverse.

Sentences that score points

- “Let me do the miss-side arithmetic – 92% to 97% cuts DB load 2.67x.”
- “I’ll split the price into its own short-TTL key so a price change does not throw away 4 KB.”
- “Immutable content-hashed URLs mean I never invalidate for correctness, only for takedowns.”
- “Here is the part I cannot fix: bytes already in a user’s browser cache.”

Load Balancing, Proxies & API Design

Everything between the user's finger and your code — and the contract your code promises back.

The idea in one page

Between a user's phone and your function there are four boxes, and every one of them is a place where a design interview goes wrong. This chapter names them, sizes them with real arithmetic, and then writes the contract your service hands back: the endpoints, the status codes, the pagination, the retry rules.

THE FOUR BOXES, AND WHAT EACH ONE IS ALLOWED TO KNOW

Box	Sees	Can therefore do
L4 load balancer	IP address and TCP port only	pick a server, forward packets. Extremely fast (millions of connections), knows nothing about your API. Cannot retry, cannot read a path.
L7 reverse proxy	the whole HTTP request: method, path, headers, body	route /images to one pool and /api to another, terminate TLS, retry a failed GET, compress, cache, add request ids.
API gateway	the request and who sent it	authenticate, rate limit per customer, check the quota, validate the schema, record usage for billing.
the service	the business problem	nothing about load balancing. A service that knows which replica it is has a bug.

The single sentence that earns marks: **“I terminate TLS at the L7 proxy, authenticate and rate limit at the gateway, and my service never sees a client IP except in a header.”**

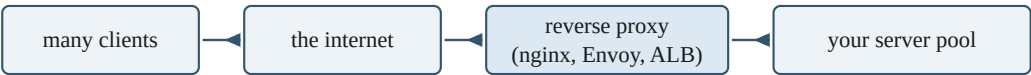
Forward proxy and reverse proxy are not the same word twice

Forward proxy — stands with the CLIENT



The **server** sees the proxy's IP, not the client's. Used for office filtering, outbound caching, hiding who is asking.

Reverse proxy — stands with the SERVER



The **client** sees one address for many servers. Used for load balancing, TLS termination, caching, and hiding how many servers you have.

A forward proxy works for the client. A reverse proxy works for the server. Same machine, opposite job.

A load balancer is a reverse proxy with a particular job. “Reverse proxy” describes where it stands; “load balancer” describes what it decides. Nginx, Envoy and HAProxy are all three things at once, which is why the words get mixed up.

The numbers you must know by heart

TCP handshake (same region)	1 RTT, ~1 ms	TLS 1.3 full handshake	1 RTT
TLS 1.2 full handshake	2 RTT	TLS session resumption	0 RTT
TLS CPU cost, one handshake	1–2 ms	requests per kept-alive conn	20–100
one L7 proxy node	20–50k RPS	one L4 load balancer	millions of conns
one app node, JSON API	1–3k RPS	DNS TTL for failover	30–60 s
health check interval	2 s, 3 strikes	connection drain time	30–60 s
safe steady-state utilisation	60–70%	autoscaler reaction time	2–5 min

The one law that explains every load-balancing decision

A server is a queue. As you push utilisation towards 100%, the **waiting** time explodes — not gently, but as $1/(1 - \rho)$. This is why you never size a fleet at 90%.

utilisation.py — why you never run a server at 90%

PYTHON

```
S = 20.0 # ms of real work per request
print(" util   avg wait   total time   multiplier")
for rho in (0.5, 0.6, 0.7, 0.8, 0.9, 0.95, 0.99):
    wait = S * rho / (1 - rho) # average queueing delay
    total = S + wait
    print(f" {rho:4.0%}   {wait:7.1f}ms   {total:8.1f}ms   {total / S:6.1f}x")
```

\$ python3 utilisation.py — real output

TEXT

```
util   avg wait   total time   multiplier
50%    20.0ms     40.0ms      2.0x
60%    30.0ms     50.0ms      2.5x
70%    46.7ms     66.7ms      3.3x
80%    80.0ms    100.0ms     5.0x
90%   180.0ms   200.0ms    10.0x
95%   380.0ms   400.0ms   20.0x
99%  1980.0ms  2000.0ms  100.0x
```

SHORTCUT

Read the jump from 70% to 90%. You added 29% more traffic and latency went up **3x**. That is the whole reason for the “provision at 60–70%” rule, and it is the answer to “why not just run fewer, busier servers?”

$$\text{latency multiplier} = \frac{1}{1 - \rho}$$

Memorise three points: 50% is 2x, 80% is 5x, 90% is 10x. If you can say those three numbers, you never have to argue about headroom again.

Warm-up

WARM-UP

build the reflex

Example 1

Name the load-balancing algorithm for each situation, in one line each. (a) five identical servers, all requests cost the same; (b) servers of two different sizes; (c) requests vary wildly — some take 5 ms, some take 5 seconds; (d) each user's session data lives in one server's memory; (e) a cache tier where the same key must reach the same node.

SOLUTION

- **(a) Round robin.** Nothing to be clever about. Cheapest possible decision: one counter.
- **(b) Weighted round robin.** Give the 8-core box weight 2 and the 4-core box weight 1.
- **(c) Least connections.** With wildly different costs, counting **requests** is meaningless; counting **work currently in flight** is not. A server stuck on three 5-second requests stops receiving new ones automatically.
- **(d) Sticky sessions** (a cookie or a source-IP hash). And say the cost out loud: sticky sessions break your ability to remove a server, and they make load uneven. The better answer is to make the servers stateless and put the session in Redis.
- **(e) Consistent hashing.** A plain hash % N reshuffles almost every key when N changes; consistent hashing moves roughly $1/N$ of them.

Answer: round robin · weighted RR · least connections · sticky (but fix the state instead) · consistent hashing

Example 2

Your service peaks at 5,000 requests per second. One node comfortably serves 800 RPS. How many nodes do you run? Show every step.

SOLUTION

Step 1 — the raw number.

$$\frac{5,000}{800} = 6.25 \rightarrow 7 \text{ nodes}$$

Step 2 — but 7 nodes at peak means each node is at

$$\frac{5,000}{7} = 714 \text{ RPS, which is } \frac{714}{800} = 89\% \text{ utilisation}$$

and the utilisation law says 89% is a 9x latency multiplier. Too hot.

Step 3 — size for 65% instead.

$$\text{capacity needed} = \frac{5,000}{0.65} = 7,692 \text{ RPS}$$

$$\frac{7,692}{800} = 9.6 \rightarrow 10 \text{ nodes}$$

Step 4 — N+1 for a node failure. If one node dies, the other 9 must carry 5,000:

$$\frac{5,000}{9} = 555 \text{ RPS each} = 69\% \text{ utilisation} \checkmark$$

Step 5 — spread across 2 availability zones, 5 per zone. If one whole zone fails, 5 nodes carry 5,000:

$$\frac{5,000}{5} = 1,000 \text{ RPS each} > 800 \rightarrow \text{overloaded}$$

So to survive a zone failure you need 7 per zone, 14 total.

The answer depends on what you promise to survive, and saying that sentence is the point of the question.

Answer: 7 to merely serve it; 10 for healthy utilisation; 14 (7 per zone) to survive losing an entire zone.

Example 3

A health check runs every 2 seconds and marks a node dead after 3 consecutive failures. A node dies at $t=0$. How long do users see errors, and how many requests fail at 5,000 RPS across 10 nodes?

SOLUTION

Detection time. The worst case is that the node dies just after a successful check:

up to 2 s (wait for the next check) + 3 × 2 s (three strikes) = 8 s

The typical case is 3 × 2 = 6 s.

Requests lost. The dead node was receiving its share:

$$\frac{5,000}{10} = 500 \text{ RPS}$$

$$500 \times 6 = 3,000 \text{ failed requests}$$

How to make it smaller — and what each fix costs.

- Check every 0.5 s with 2 strikes: detection falls to 1 s, so 500 failed requests. Cost: 4x the health-check traffic, and a higher chance of ejecting a healthy-but-slow node.
- **Passive** health checks: the proxy watches real traffic and ejects a node after 5 consecutive 5xx responses. Detection is now **sub-second** and costs nothing extra, because it uses requests you were sending anyway.
- **Retry the failed request on another node.** The user sees nothing at all. This is the real fix, and it is why an L7 proxy is worth more than an L4 one.

Decision: passive ejection plus a retry on a different node for idempotent requests. Active checks stay, but as the slower safety net.

Answer: 6–8 s of errors, about 3,000 failed requests. Fix with passive ejection + retry elsewhere.

Example 4

Which HTTP methods are safe? Which are idempotent? Why does a load balancer care?

SOLUTION

Method	Safe?	Idempotent?	Meaning
GET	yes	yes	changes nothing. Free to retry, free to cache.
HEAD	yes	yes	like GET with no body.
PUT	no	yes	“make the resource equal to this”. Doing it twice leaves the same state.
DELETE	no	yes	deleting twice leaves it deleted. The second call returns 404 or 204; either is fine.
POST	no	no	“create another one”. Twice means two.
PATCH	no	usually not	{“qty”: 5} is idempotent; {“qty”: “+1”} is not.

Why the proxy cares. A proxy may automatically retry a request when a backend times out or returns a connection error. Retrying a GET is free. Retrying a POST can charge a card twice. So a correctly configured proxy retries safe methods automatically and retries POST **only** when the client sent an Idempotency-Key — or never.

Answer: Safe: GET, HEAD. Idempotent: GET, HEAD, PUT, DELETE. POST is neither, so it must never be auto-retried without an idempotency key.

Example 5

Give the right status code, in one line each: the resource does not exist; the client sent bad JSON; the client is not logged in; the client is logged in but not allowed; the client sent 200 requests in a minute against a 60-per-minute limit; your database is down; a POST succeeded and created something; a PATCH succeeded but you have nothing to return.

SOLUTION

Situation	Code	Note
resource missing	404 Not Found	use it for “you may not see it” too, so you do not leak existence
bad JSON / bad field	400 Bad Request	422 Unprocessable Content if the JSON parsed but a value is invalid

not logged in	401 Unauthorized	badly named: it means unauthenticated
logged in, not allowed	403 Forbidden	the one that actually means unauthorised
over the rate limit	429 Too Many Requests	must carry a Retry-After header
database down	503 Service Unavailable	503 says “try later”; 500 says “I am broken”
created	201 Created	with a Location header pointing at the new resource
nothing to return	204 No Content	and truly no body — not {}

The rule that matters most: 4xx means **the client can fix it**, 5xx means **only you can**. Getting that backwards makes every client retry forever on a bug they could have fixed.

Answer: 404 · 400/422 · 401 · 403 · 429 + **Retry-After** · 503 · 201 + **Location** · 204

Example 6

An API returns 20 items per page. A client asks for ?page=2 while new items keep arriving at the top. Show exactly what goes wrong, with rows.

SOLUTION

Ten items exist, newest first: e10 e9 e8 e7 e6 e5 e4 e3 e2 e1, page size 5.

Page 1 (OFFSET 0 LIMIT 5) returns e10 e9 e8 e7 e6.

Now e11 arrives. The list is e11 e10 e9 e8 e7 e6 e5 e4 e3 e2 e1.

Page 2 (OFFSET 5 LIMIT 5) returns items 6 to 10 of the **new** list: e6 e5 e4 e3 e2.

The client has now seen e6 twice and has never seen e11. With deletions the opposite happens: items get **skipped** entirely.

The fix: a cursor. Page 1 returns `nextCursor = encode({createdAt: e6.createdAt, id: "e6"})`, and page 2 asks for “everything strictly older than that”. New arrivals at the top change nothing, because the question was never “skip 5 items” — it was “continue from e6”.

Answer: Page 2 repeats e6 and skips e11. Offsets count positions; cursors remember a place.

Example 7

A tier allows 60 requests per minute. A client sends 60 requests in the first second, then waits. Is that allowed? Compare a fixed window, a sliding window and a token bucket.

SOLUTION

- **Fixed window (per minute).** Allowed — the counter only cares about the total inside the minute. Now the **boundary problem**: the client can send 60 at 11:59:59 and 60 more at 12:00:00. That is **120 requests in one second** while never breaking “60 per minute”.
- **Sliding window.** Not allowed after the first 60: any 60-second span is counted, so the boundary trick fails. Costs more memory (a log of timestamps, or two counters blended).
- **Token bucket (capacity 60, refill 1/s).** Allowed once — the bucket starts full, so a burst of 60 goes through, and then the client is limited to 1 per second. **This is usually what you want**, because it permits a genuine burst (an app starting up and loading ten screens) while capping the long-run rate.

Decision: token bucket, stated as “60 per minute, burst 60”. Then be explicit in the docs, because a client that does not know the burst size cannot pace itself.

Answer: Fixed window: yes, and it allows 120 in a second at the boundary. Sliding window: no. Token bucket: yes once, then 1/s. Choose the token bucket.

Example 8

A service handles 3,000 RPS. Each request takes 250 ms. How many requests are in flight at any moment? If each in-flight request holds a thread using 1 MB of stack, how much memory is that?

SOLUTION

Little's law. For any stable system:

items in the system = arrival rate $\times$ time each item spends inside

$$3,000 \text{ req/s} \times 0.250 \text{ s} = 750 \text{ requests in flight}$$

Memory.

$$750 \times 1 \text{ MB} = 750 \text{ MB of thread stacks}$$

Why this is the most useful formula in the chapter. It converts a latency problem into a capacity problem. If a downstream call gets slower — 250 ms becomes 1 second — then

$$3,000 \times 1.0 = 3,000 \text{ in flight} \rightarrow 3 \text{ GB of stacks}$$

with no change in traffic at all. The server runs out of threads and starts refusing **healthy** requests. That is how one slow dependency takes down a whole service, and Little's law is how you predict it before it happens.

Answer: 750 in flight, 750 MB. Four times the latency means four times the memory at the same traffic.

Example 9

Your API does 200,000 RPS. Keep-alive gives you 50 requests per connection. How many new TLS handshakes per second? At 1.5 ms of CPU each, how many CPU cores does TLS alone need?

SOLUTION

$$\text{new connections/s} = \frac{200,000}{50} = 4,000 \text{ per second}$$

$$\text{CPU seconds per second} = 4,000 \times 0.0015 = 6$$

Six CPU-seconds of work arrive every second, so **6 cores are busy doing nothing but TLS handshakes.**

Now turn on session resumption, where a returning client skips the expensive part. If 80% of connections resume:

$$\text{full handshakes} = 4,000 \times 0.20 = 800 \text{ per second}$$

$$800 \times 0.0015 = 1.2 \text{ cores}$$

$$\frac{6}{1.2} = 5 \times \text{less CPU, for one configuration flag.}$$

And the trap: if keep-alive is off, every request opens a connection:

$$200,000 \times 0.0015 = 300 \text{ cores}$$

That is the difference between one machine and a rack.

Answer: 4,000 handshakes/s = 6 cores; 1.2 cores with 80% resumption; 300 cores with no keep-alive.

Example 10

A client calls service A, which calls B, which calls C. Each sets a 3-second timeout and retries twice. C hangs. How long does the user wait, and how many requests does C receive?

SOLUTION

Work upward from C.

C's caller is B. B tries, waits 3 s, retries, waits 3 s, retries, waits 3 s:

$$\text{B spends } 3 \times 3 = 9 \text{ s per attempt, and sends 3 requests to C}$$

A does the same to B, three times:

$$\text{A spends } 3 \times 9 = 27 \text{ s, and B sends } 3 \times 3 = 9 \text{ requests to C}$$

The client does the same to A:

$$\text{the user waits } 3 \times 27 = 81 \text{ s}$$

C receives $3 \times 9 = 27$ requests for one user action

That is retry amplification: $3^3 = 27$. Three layers of “harmless” double-retry turned one click into 27 requests on the already-dying service.

The two fixes, both required.

1. **A timeout budget.** The client’s deadline (say 3 s) travels down in a header. Each hop passes on what is left. When the budget is gone, nobody retries. The user waits 3 s, not 81.
2. **Retry only at ONE layer** — the edge — and use a **retry budget**: allow retries only while they are under 10% of traffic. Never retry in the middle of a chain.

Answer: 81 seconds, 27 requests on C. Fix: pass a deadline down the chain, and retry at one layer only, under a 10% budget.

Example 11

Why is `POST /createUser`, `POST /getUser`, `POST /deleteUser` a bad API? Rewrite it.

SOLUTION

What is wrong.

1. **Every call is a POST**, so nothing is cacheable, nothing is safely retryable, and a proxy cannot tell a read from a write.
2. **Verbs in the path.** The URL names an **action**, so the vocabulary grows forever: `createUser`, `createUserV2`, `createUserWithPhoto`. With nouns, `POST /users` already covers all three.
3. **No hierarchy.** “The orders of user 7” has no obvious spelling.

The rewrite.

```
POST /v1/users          201 + Location: /v1/users/1187
GET  /v1/users/1187      200
GET  /v1/users?limit=20&cursor=... 200
PATCH /v1/users/1187    200 (partial update)
PUT   /v1/users/1187    200 (full replace, idempotent)
DELETE /v1/users/1187    204
GET   /v1/users/1187/orders?limit=20&cursor=...
```

The honest exception. Some operations are genuinely not nouns: “send the password reset email”, “cancel the order”, “retry the payment”. Forcing those into nouns produces worse APIs than admitting it. The accepted pattern is a sub-resource that **is** the action’s record:

```
POST /v1/orders/887/cancellation 201 { cancellationId, status }
POST /v1/users/1187/password-resets 202
```

Now the action is a thing, it has an id, you can look it up later, and it can carry an idempotency key.

Answer: Nouns and HTTP methods, not verbs in paths. For genuine actions, make the action a sub-resource that has its own id.

Tier 1 · LLD: build the load balancer

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

A very common low-level design round: “write a load balancer”. Same six moves as any LLD — cut scope, nouns to classes, draw, name the hard part, write the core method, handle concurrency and change.

THE SCOPE YOU CUT, OUT LOUD, IN 30 SECONDS

In: a pool of backends, pluggable selection strategies, health tracking, a circuit breaker, and request counting.

Out: TLS, HTTP parsing, config reloading, and the actual socket code. “I will model it synchronously with a tick-based simulation so the behaviour is testable.”

The one hard part: selection must stay correct while the pool changes underneath it, and a sick backend must be removed **before** it hurts users.

Move 2 and 3 — the classes

Backend	one server: id, weight, capacity, in-flight count, healthy flag, counters
Strategy	the interface: <code>pick(pool) -> Backend</code> . One class per algorithm.
LoadBalancer	owns the backends, filters to the healthy pool, delegates to the strategy, tracks in-flight
CircuitBreaker	per backend: CLOSED / OPEN / HALF_OPEN, with a cooldown

SHORTCUT

Notice Strategy is the **strategy pattern**, and it is the correct answer here for a reason you can state: the set of algorithms is open (someone will invent another one) but the **interface** is closed — every algorithm answers exactly “which backend?”. That is precisely the shape the strategy pattern exists for. Say that sentence instead of just naming the pattern.

Move 5 — the code

lb.js — backends and four strategies

JS

```
class Backend {
  constructor(id, { weight = 1, capacity = 100 } = {}) {
    this.id = id; this.weight = weight; this.capacity = capacity;
    this.inFlight = 0; this.healthy = true;
    this.served = 0; this.failed = 0;
    this.cw = 0; // current weight, for smooth weighted RR
  }
}

class Strategy { pick(pool) { throw new Error("implement pick()"); } }

class RoundRobin extends Strategy {
  constructor() { super(); this.i = 0; }
  pick(pool) { const b = pool[this.i % pool.length]; this.i++; return b; }
}

// weights 5,1,1 must give A B A C A A A ... not A A A A A B C
class SmoothWeighted extends Strategy {
  pick(pool) {
    const total = pool.reduce((s, b) => s + b.weight, 0);
    let best = pool[0];
    for (const b of pool) { b.cw += b.weight; if (b.cw > best.cw) best = b; }
    best.cw -= total;
    return best;
  }
}

class LeastConnections extends Strategy {
  pick(pool) { return pool.reduce((a, b) => (b.inFlight < a.inFlight ? b : a)); }
}
```

```

}

// power of two choices: sample 2 at random, take the emptier one
class PowerOfTwo extends Strategy {
  constructor(rnd) { super(); this.rnd = rnd; }
  pick(pool) {
    if (pool.length === 1) return pool[0];
    const a = pool[Math.floor(this.rnd() * pool.length)];
    let b = pool[Math.floor(this.rnd() * pool.length)];
    if (a === b) b = pool[(pool.indexOf(a) + 1) % pool.length];
    return a.inFlight <= b.inFlight ? a : b;
  }
}

class LoadBalancer {
  constructor(backends, strategy) {
    this.backends = backends; this.strategy = strategy;
  }
  get pool() { return this.backends.filter(b => b.healthy); }
  route() {
    const pool = this.pool;
    if (pool.length === 0) return { status: 503, backend: null };
    const b = this.strategy.pick(pool);
    b.inFlight++;
    if (b.inFlight > b.capacity) { // the backend is full: shed, do not queue
      b.inFlight--; b.failed++;
      return { status: 503, backend: b };
    }
    b.served++;
    return { status: 200, backend: b };
  }
  done(b) { if (b) b.inFlight = Math.max(0, b.inFlight - 1); }
}

```

lb.js — the experiment: three backends, one of them 5x slower

JS

```

let seed = 20260915;
const rnd = () => (seed = (seed * 48271) % 2147483647) / 2147483647;

function simulate(name, makeStrategy) {
  seed = 20260915; // same randomness for every run
  const backends = [new Backend("a", { capacity: 60 }),
    new Backend("b", { capacity: 60 }),
    new Backend("c", { capacity: 60 })];
  const lb = new LoadBalancer(backends, makeStrategy());
  const holdTicks = { a: 1, b: 1, c: 5 }; // c is 5x slower
  const busy = [];
  let ok = 0, rejected = 0;

  for (let tick = 0; tick < 5000; tick++) {
    for (let i = busy.length - 1; i >= 0; i--) // finish what is done
      if (--busy[i].left === 0) { lb.done(busy[i].b); busy.splice(i, 1); }

    for (let r = 0; r < 40; r++) { // 40 new requests per tick
      const res = lb.route();
      if (res.status !== 200) { rejected++; continue; }
    }
  }
}

```



```

    ok++;
    busy.push({ b: res.backend, left: holdTicks[res.backend.id] });
  }
}
console.log(name.padEnd(18), "ok:", String(ok).padStart(6),
  " 503s:", String(rejected).padStart(6), " ",
  backends.map(b => b.id + "=" + b.served).join(" "));
}

console.log("3 backends, capacity 60 in-flight each; backend c is 5x slower\n");
simulate("round robin",      () => new RoundRobin());
simulate("least connections", () => new LeastConnections());
simulate("power of two",     () => new PowerOfTwo(rnd));

console.log("\nsmooth weighted round robin, weights a=5 b=1 c=1:");
const w = [new Backend("a", { weight: 5 }), new Backend("b", { weight: 1 }),
  new Backend("c", { weight: 1 })];
const sw = new SmoothWeighted();
const picks = Array.from({ length: 14 }, () => sw.pick(w).id);
console.log("  first 14 picks ->", picks.join(" "));

```

\$ node lb.js — real output

TEXT

3 backends, capacity 60 in-flight each; backend c is 5x slower

round robin	ok: 193334	503s: 6666	a=66667	b=66667	c=60000
least connections	ok: 200000	503s: 0	a=91997	b=89999	c=18004
power of two	ok: 200000	503s: 0	a=90913	b=90877	c=18210

smooth weighted round robin, weights a=5 b=1 c=1:
 first 14 picks -> a a b a c a a a b a c a a

Four things that output proves, and each one is an interview answer.

1. **Round robin dropped 6,666 requests — 3.3% of all traffic.** It kept sending one third of requests to a backend that could not keep up, and that backend filled and started refusing. Round robin is correct only when every request costs the same **and** every server is the same speed.
2. **Least connections dropped nothing** and automatically gave the slow backend 18,004 / 200,000 = 9% of traffic instead of 33%. Nobody configured that. It is a consequence of counting in-flight work rather than requests.
3. **Power of two choices matched it** — 18,210 versus 18,004 — while looking at only **two** backends per decision instead of all of them. With 3 backends that saves nothing. With 2,000 backends it is the difference between an $O(n)$ scan and $O(1)$ per request, and it is why every large proxy uses it.
4. **Smooth weighting matters.** Weights 5,1,1 produce a a b a c a a a b a c a a — the small backends are spread through the sequence. The naive implementation would send a a a a a b c, giving a a five-request burst every seven requests.

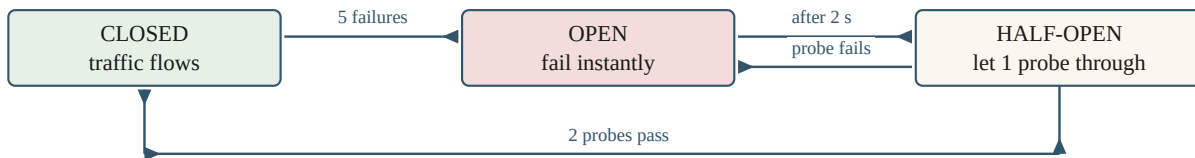
Time round robin $O(1)$ · least connections $O(n)$ · power of two $O(1)$ · smooth weighted $O(n)$ **Space** $O(n)$ for the pool

At $n = 2,000$ backends and 50,000 RPS, an $O(n)$ pick costs 100 million comparisons a second. That is the whole argument for power of two choices.

TRAP

`pool.reduce((a, b) => b.inFlight < a.inFlight ? b : a)` throws on an empty array. The `LoadBalancer.routeGuard(if (pool.length === 0) return 503)` is not decoration — it is the difference between returning 503 and crashing the proxy the moment every backend goes unhealthy at once. That is exactly the moment you least want the proxy to crash.

The circuit breaker — stop calling a service that is already down



Three states. The cooldown is what stops a retry storm from re-killing a recovering service.

breaker.js — the state machine

JS

```
class CircuitBreaker {
  constructor({ failureThreshold = 5, cooldownMs = 2000,
    probeSuccesses = 2 } = {}) {
    this.state = "CLOSED"; this.fails = 0; this.probeOk = 0; this.openedAt = 0;
    this.failureThreshold = failureThreshold;
    this.cooldownMs = cooldownMs; this.probeSuccesses = probeSuccesses;
  }
  allow(now) {
    if (this.state === "CLOSED") return true;
    if (this.state === "OPEN") {
      if (now - this.openedAt >= this.cooldownMs) { // cooldown over: probe
        this.state = "HALF_OPEN"; this.probeOk = 0; return true;
      }
      return false; // fail fast, no call made
    }
    return true; // HALF_OPEN: probes pass through
  }
  onSuccess() {
    if (this.state === "HALF_OPEN") {
      if (++this.probeOk >= this.probeSuccesses) {
        this.state = "CLOSED"; this.fails = 0;
      }
    } else this.fails = 0;
  }
  onFailure(now) {
    if (this.state === "HALF_OPEN") {
      this.state = "OPEN"; this.openedAt = now; return;
    }
    if (++this.fails >= this.failureThreshold) {
      this.state = "OPEN"; this.openedAt = now;
    }
  }
}
```

breaker.js — a backend that is broken from t=0 to t=6000 ms

JS

```

const brokenUntil = 6000;
const cb = new CircuitBreaker({ failureThreshold: 5, cooldownMs: 2000,
                                probeSuccesses: 2 });
let sentToBackend = 0, refusedFast = 0;
const log = [];

for (let now = 0; now <= 9000; now += 500) {
  const before = cb.state;
  if (!cb.allow(now)) {
    refusedFast++; log.push([now, before, "-- refused, no call made"]); continue;
  }
  sentToBackend++;
  const ok = now >= brokenUntil;
  if (ok) cb.onSuccess(); else cb.onFailure(now);
  log.push([now, cb.state === before ? before : before + " -> " + cb.state,
            ok ? "200 OK" : "500 error"]);
}
console.log(" ms      state                what happened");
for (const [t, st, r] of log)
  console.log(String(t).padStart(5), " ", st.padEnd(18), r);
console.log("\ncalls that reached the sick backend  :", sentToBackend);
console.log("calls refused instantly by the breaker:", refusedFast);

```

\$ node breaker.js — real output

TEXT

ms	state	what happened
0	CLOSED	500 error
500	CLOSED	500 error
1000	CLOSED	500 error
1500	CLOSED	500 error
2000	CLOSED -> OPEN	500 error
2500	OPEN	-- refused, no call made
3000	OPEN	-- refused, no call made
3500	OPEN	-- refused, no call made
4000	OPEN	500 error
4500	OPEN	-- refused, no call made
5000	OPEN	-- refused, no call made
5500	OPEN	-- refused, no call made
6000	OPEN -> HALF_OPEN	200 OK
6500	HALF_OPEN -> CLOSED	200 OK
7000	CLOSED	200 OK
7500	CLOSED	200 OK
8000	CLOSED	200 OK
8500	CLOSED	200 OK
9000	CLOSED	200 OK

calls that reached the sick backend : 13
calls refused instantly by the breaker: 6

Read the trace line by line — it contains every idea in the pattern.

- **0 to 1,500 ms:** the breaker is CLOSED, so all five calls go through and all five fail. This is the cost of the pattern: **you always pay the threshold in real failures** before it engages.
- **2,000 ms:** the fifth consecutive failure trips it to OPEN.
- **2,500 to 3,500 ms:** three calls are refused **instantly**, with no network call at all. The user gets a fast 503 instead of a 30-second timeout, and the sick service gets peace to recover.

- **4,000 ms:** the 2-second cooldown has expired, so exactly **one** probe is let through. It fails, and the breaker snaps straight back to OPEN. Note what did **not** happen: a thundering herd of every waiting client retrying at once.
- **6,000 ms:** the backend recovers. The next probe succeeds.
- **6,500 ms:** the second consecutive probe succeeds, so the breaker closes and normal traffic resumes.

The number to quote: 6 of 19 calls never touched the sick backend. Under real load — say 5,000 RPS — the OPEN state would have removed roughly $5,000 \times 3.5 \text{ s} = 17,500$ pointless requests from a service that was already struggling.

Example 12 [TCS NQT · pattern]

Your proxy retries any failed request twice. During an outage, traffic is 1,000 RPS. How much load does the failing backend see, and what do you change?

SOLUTION

$$\text{attempts per request} = 1 + 2 \text{ retries} = 3$$

$$\text{load on the failing backend} = 1,000 \times 3 = 3,000 \text{ RPS}$$

You have tripled the load on the one machine that is already failing. This is called a **retry storm**, and it is how a small blip becomes an outage.

Fix 1 — exponential backoff with full jitter. Not “wait 100 ms, then 200 ms” (every client then retries at exactly the same moments, which is another spike), but a random wait in the whole window:

```
function fullJitter(attempt, baseMs = 100, capMs = 20000, rnd = Math.random) {
  return Math.floor(rnd() * Math.min(capMs, baseMs * 2 ** attempt));
}
// attempts 0..5 produced: 0, 119, 83, 193, 1241, 1049 ms
```

The word **full** matters: picking uniformly in $[0, \text{window}]$ spreads the retries across the whole window, where picking the window’s end would bunch them at one instant.

Fix 2 — a retry budget. Allow retries only while they stay under 10% of total traffic.

$$1,000 \times 1.1 = 1,100 \text{ RPS, not } 3,000$$

When the backend is failing **everywhere**, retrying cannot help anybody, so the budget shuts retries off automatically.

Fix 3 — the circuit breaker above, which stops calls entirely rather than slowing them.

Decision: all three, in that order of importance. The budget is the one most teams forget, and it is the one that converts a 3x amplification into 1.1x.

Answer: 3,000 RPS — 3x amplification. Fix with full-jitter backoff, a 10% retry budget, and a circuit breaker.

Example 13 [Capgemini · pattern]

Why does “power of two choices” work so well, when picking the **single** least-loaded backend is obviously better? Explain with 2,000 backends.

SOLUTION

Why not just pick the least loaded? Two reasons.

1. **Cost.** Scanning 2,000 backends per request at 50,000 RPS is $50,000 \times 2,000 = 100,000,000$ comparisons a second, on every proxy node.
2. **The herd.** If you run 20 proxy nodes, they all see the same “least loaded” backend at the same moment and all send to it. The emptiest server becomes the fullest server instantly. This is a real failure mode, and it has a name: **herd behaviour on stale state**.

Why two random choices is enough. Picking one at random gives an expected maximum load that grows like $\log n / \log \log n$. Picking the better of **two** random choices drops it to $\log \log n / \log 2$. Put numbers on it for $n = 2,000$:

$$\log_2(\log_2 2000) = \log_2(10.97) = 3.5$$

versus $\log_2(2000) / \log_2 \log_2(2000) = 10.97 / 3.46 = 3.2$ — the point is not the exact constants but the **shape**: one extra sample turns a logarithmic imbalance into a double-logarithmic one. Doubling to three or four choices buys almost nothing more.

And the measurement from this chapter's own simulation: with a 5x-slow backend, least connections gave it 18,004 requests and power of two gave it 18,210 — a difference of 1.1%, for $O(1)$ work instead of $O(n)$ and with no herd.

Decision: power of two choices for any pool above about 20 backends. Plain least connections below that, where the scan is free and the herd is small.

Answer: It is $O(1)$ instead of $O(n)$, it cannot herd, and it lands within about 1% of true least-connections (measured: 18,210 vs 18,004).

Tier 1 · LLD: the four API pieces every service needs

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Piece 1 — a rate limiter with one number per client

The token bucket needs two numbers per client (tokens left, last refill time). GCRA — the **generic cell rate algorithm** — needs one, which matters when “per client” means 40 million entries in Redis.

api.js — GCRA: store one timestamp, get burst plus rate

JS

```
class Gcra {
  // rate = requests per second on average; burst = how many may arrive together
  constructor(rate, burst) {
    this.T = 1000 / rate;           // ms that one request "costs"
    this.tau = this.T * (burst - 1); // how far ahead of now a client may run
    this.tat = new Map();           // theoretical arrival time, one per client
  }
  check(key, now) {
    const tat = Math.max(this.tat.get(key) ?? now, now);
    if (tat - now > this.tau) { // this client is too far ahead
      const wait = Math.ceil(tat - this.tau - now);
      return { allowed: false, retryAfterMs: wait, remaining: 0 };
    }
    this.tat.set(key, tat + this.T);
    return { allowed: true, retryAfterMs: 0,
      remaining: Math.floor((this.tau - (tat - now)) / this.T) };
  }
}
```

\$ node api.js — GCRA at 2 req/s with burst 5

TEXT

```
GCRA: 2 req/s, burst 5
t= 0ms req 1 -> 200 X-RateLimit-Remaining: 4
t= 0ms req 2 -> 200 X-RateLimit-Remaining: 3
t= 0ms req 3 -> 200 X-RateLimit-Remaining: 2
t= 0ms req 4 -> 200 X-RateLimit-Remaining: 1
t= 0ms req 5 -> 200 X-RateLimit-Remaining: 0
t= 0ms req 6 -> 429 Retry-After: 500ms
t= 0ms req 7 -> 429 Retry-After: 500ms
```

```
t= 0ms req 8 -> 429 Retry-After: 500ms
... wait 1500 ms ...
t=1500ms req 9 -> 200 X-RateLimit-Remaining: 2
t=1500ms req 10 -> 200 X-RateLimit-Remaining: 1
t=1500ms req 11 -> 200 X-RateLimit-Remaining: 0
```

Check the arithmetic yourself: at 2 req/s, one request costs $T = 500$ ms. After 1,500 ms of waiting the client has earned $1,500 / 500 = 3$ requests back, and the output shows exactly three allowed. The `Retry-After: 500ms` on the first rejection is also exactly T : the time until one request is affordable again.

SHORTCUT

Always return three headers with a 429, and say so unprompted:

```
X-RateLimit-Limit: 120
X-RateLimit-Remaining: 0
Retry-After: 1
```

Without `Retry-After`, every client guesses — and their guesses cluster, producing the exact spike you were limiting. A rate limiter without `Retry-After` creates the problem it was installed to fix.

Piece 2 — a concurrency limiter, which is not a rate limiter

api.js — cap the work in flight, not the arrival rate

JS

```
class ConcurrencyLimiter {
  constructor(max) { this.max = max; this.inFlight = 0; this.shed = 0; }
  tryAcquire() {
    if (this.inFlight >= this.max) { // 503, immediately
      this.shed++; return false;
    }
    this.inFlight++; return true;
  }
  release() { this.inFlight--; }
}

const cl = new ConcurrencyLimiter(100);
for (const [rps, latency] of [[1000, 0.05], [1000, 0.20], [4000, 0.05]]) {
  const inFlight = rps * latency;
  console.log(`${rps} rps x ${latency}s = ${inFlight} in flight`,
    inFlight > cl.max ? `-> over the cap of ${cl.max}: shed the excess`
      : `-> fits under the cap`);
}
```

\$ node api.js — output

TEXT

```
1000 rps x 0.05s = 50 in flight -> fits under the cap
1000 rps x 0.2s = 200 in flight -> over the cap of 100: shed the excess
4000 rps x 0.05s = 200 in flight -> over the cap of 100: shed the excess
```

Read rows 1 and 2. **The traffic did not change.** Only the latency did — a downstream dependency got slower — and the service went from comfortable to overloaded. A rate limiter counting requests per second would have seen nothing wrong. A **concurrency** limiter sees it immediately, because it measures the thing that actually runs out: threads, sockets, memory.

Use both: a rate limiter to be fair between customers, and a concurrency limiter to protect the process from itself.

Piece 3 — cursor pagination

```

const encode = (o) => Buffer.from(JSON.stringify(o)).toString("base64url");
const decode = (s) => JSON.parse(Buffer.from(s, "base64url").toString());

function page(rows, limit, cursor) {
  // rows sorted by (createdAt DESC, id DESC) - a TOTAL order, no ties
  let start = 0;
  if (cursor) {
    const c = decode(cursor);
    start = rows.findIndex(r => r.createdAt < c.createdAt ||
      (r.createdAt === c.createdAt && r.id < c.id));
    if (start < 0) start = rows.length;
  }
  const slice = rows.slice(start, start + limit);
  const last = slice[slice.length - 1];
  return {
    items: slice.map(r => r.id),
    nextCursor: slice.length === limit && last
      ? encode({ createdAt: last.createdAt, id: last.id }) : null
  };
}

```

```

keyset pagination, limit 5
page 1: e20,e19,e18,e17,e16  cursor: eyJjcmVhdGVkQXQiOj...
(a new row e99 is inserted at the top here)
page 2: e15,e14,e13,e12,e11  cursor: eyJjcmVhdGVkQXQiOj...
page 3: e10,e9,e8,e7,e6      cursor: eyJjcmVhdGVkQXQiOj...

OFFSET pagination, limit 5
page 1: e20,e19,e18,e17,e16
(same new row e99 inserted)
page 2: e16,e15,e14,e13,e12  <- e16 is shown twice

```

TRAP

The cursor must encode a **total** order. {createdAt} alone is not enough: two rows created in the same millisecond make the comparison ambiguous, and the page either repeats one or skips one. Always append the primary key, and compare as a pair: (createdAt, id) < (cursorCreatedAt, cursorId).

Also: **the cursor is opaque**. Base64 it, do not document its contents, and treat it as a value you produced. The moment a client parses it, you can never change it.

Piece 4 — idempotency

```

class IdempotencyStore {
  constructor(ttlMs = 24 * 3600 * 1000) {
    this.m = new Map(); this.ttlMs = ttlMs;
  }
  // "new" | "in_progress" | "done" (with the saved response) | "conflict"
  begin(key, fingerprint, now) {

```

```

const e = this.m.get(key);
if (!e || now - e.at > this.ttlMs) {
  this.m.set(key, { state: "in_progress", fingerprint, at: now });
  return { state: "new" };
}
if (e.fingerprint !== fingerprint) return { state: "conflict" }; // 422
return e.state === "done" ? { state: "done", response: e.response }
  : { state: "in_progress" }; // 409, retry later
}
finish(key, response, now) {
  const e = this.m.get(key);
  this.m.set(key, { ...e, state: "done", response, at: now });
}
}

```

\$ node api.js — output

TEXT

```

idempotency, key = k_a1
1st POST : { state: 'new' }
retry    : { state: 'done', response: { status: 201, id: 'job_5512' } }
same key, different body: { state: 'conflict' }

```

Three details that separate a real implementation from a toy.

1. **The fingerprint.** Store a hash of the request body with the key. If a client reuses a key with a **different** body, that is a client bug, and returning the old response would be worse than an error. Return 422 and say why.
2. **The in_progress state.** Without it, two retries arriving 5 ms apart both see “no entry” and both create the order. The row must be written **before** the work starts, using an insert that fails if the key exists.
3. **The TTL.** 24 hours is the usual choice: long enough to cover every client retry policy, short enough that the table does not grow forever.

Tier 2 · Mid-scale: a public parcel-tracking API

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 14 [Shopee · pattern]

A logistics company wants a public API. Merchants create shipments, their buyers track parcels, and partner apps integrate. Design it — all seven steps.

SOLUTION

Step 1 — Clarify

Question I ask	Why the answer changes the design
Who calls this — browsers, or partner servers?	Browsers mean CORS, short-lived tokens and a CDN. Partner servers mean API keys, per-key quotas and webhooks. The answer changes authentication, caching and the whole rate-limit design.
How fresh must a tracking status be?	If 60 seconds of staleness is fine, I can put a CDN in front and drop read load by 90%. If it must be exact, every read hits the service.

Do partners poll, or do they want to be pushed to?	Polling is simple but wasteful. Webhooks cut the read traffic enormously but bring delivery retries, signatures and a whole failure surface.
Is the tracking number secret?	If a tracking number is guessable, anyone can enumerate other people's parcels. That decides whether the endpoint is public or needs a second factor such as the destination pincode.
What happens if the same scan event is sent twice?	Handheld scanners retry over bad networks constantly. This decides whether scan ingestion needs an idempotency key — it does.

Assumptions: partner servers and buyer browsers both; 60 seconds of staleness is acceptable; both polling and webhooks; tracking numbers include a check digit and a random suffix; scan ingestion is idempotent.

Step 2 — Scale estimate

Reads. 12 million parcels are in transit at any time; a buyer or merchant checks each about 6 times a day.

$$12,000,000 \times 6 = 72,000,000 \text{ reads/day}$$

$$\text{average} = \frac{72,000,000}{86,400} = 833.3 \text{ RPS}$$

Tracking is checked in bursts (people check after work, and after a “your parcel is out for delivery” message). Use a 4x peak factor.

$$\text{peak} = 833.3 \times 4 = 3,333 \text{ RPS}$$

Writes. 3 million parcels are shipped per day, each producing about 9 scan events (collected, hub-in, hub-out, ... , delivered).

$$3,000,000 \times 9 = 27,000,000 \text{ writes/day}$$

$$\text{average} = \frac{27,000,000}{86,400} = 312.5 \text{ RPS}$$

$$\text{peak} = 312.5 \times 3 = 937.5 \approx 938 \text{ RPS}$$

Total at peak.

$$3,333 + 938 = 4,271 \text{ RPS}$$

Nodes. One node serves about 2,000 RPS of this workload.

$$\frac{4,271}{2,000} = 2.14 \rightarrow 3 \text{ nodes}$$

But three nodes in two zones cannot survive a zone failure. Spread 3 per zone across 2 zones = 6 nodes; if a whole zone dies the remaining 3 carry

$$\frac{4,271}{3} = 1,424 \text{ RPS each} = 71\% \text{ of capacity } \checkmark$$

Bandwidth. A tracking response is about 1.8 KB.

$$3,333 \times 1,800 = 5,999,400 \text{ bytes/s} \approx 6 \text{ MB/s} = 48 \text{ Mbit/s}$$

Trivial. The bytes are never the problem for a JSON API; the request rate is.

Storage. A scan event row is about 200 bytes.

$$27,000,000 \times 200 = 5,400,000,000 = 5.4 \text{ GB/day}$$

$$5.4 \times 365 = 1,971 \text{ GB} = 1.97 \text{ TB/year}$$

Small enough for one database with monthly partitions.

Step 3 — API surface

The public contract

TEXT

```
GET /v1/parcels/{trackingNumber}
  auth  API key (partner) OR tracking number + pincode (public browser)
  200    { trackingNumber, status, statusUpdatedAt, estimatedDelivery,
          origin:{city}, destination:{city}, lastEvent:{code, city, at} }
  headers Cache-Control: public, max-age=60
          ETag: "p_77Q2-e9"
  404    unknown OR not yours – deliberately the same response for both

GET /v1/parcels/{trackingNumber}/events?limit=50&cursor=<opaque>
  200    { events:[ {eventId, code, city, at, note} ], nextCursor }

POST /v1/shipments
  header Idempotency-Key: <uuid>                <- REQUIRED, not optional
  body    { reference, from:{...}, to:{...}, weightGrams, service }
  201     { trackingNumber, labelUrl }    Location: /v1/parcels/IN77Q2K91X

POST /v1/webhooks
  body    { url, events:["parcel.delivered","parcel.exception"], secret }
  201     { webhookId }

POST /internal/scans                                <- scanners and hub systems only
  header Idempotency-Key: <scannerId>-<seq>
  body    { trackingNumber, code, at, hubId }
  202     Accepted
```

Five rules I follow and can defend:

version in the path	/v1/. Two characters. Header versioning is theoretically nicer and practically unusable from a browser address bar or a curl example in the docs.
404 for “not yours”	returning 403 tells an attacker the tracking number exists. Same body, same code, for unknown and forbidden.
idempotency on every create	a merchant’s retry must not print two labels for one parcel.
cursors, never page numbers	scan events arrive constantly; offsets would repeat rows.
202 for scans	the scan is accepted into a queue, not written synchronously. Saying 201 would be a lie about durability of the processing .

Step 4 — Data model

Five tables, and the one unique index that makes scans idempotent

SQL

```
CREATE TABLE parcels (
  tracking_number TEXT PRIMARY KEY, -- IN77Q2K91X: prefix + random + check digit
  merchant_id     BIGINT      NOT NULL,
  status          SMALLINT    NOT NULL,
  status_at       TIMESTAMPTZ NOT NULL,
  dest_pincode    TEXT        NOT NULL,
  eta             TIMESTAMPTZ,
  version         INT          NOT NULL DEFAULT 1
);
CREATE INDEX ix_parcels_merchant ON parcels (merchant_id, status_at DESC);
```

```

CREATE TABLE scan_events (
  tracking_number TEXT NOT NULL,
  event_id BIGINT NOT NULL,
  code SMALLINT NOT NULL,
  hub_id INT NOT NULL,
  at TIMESTAMPTZ NOT NULL,
  scanner_seq TEXT NOT NULL, -- "<scannerId>-<seq>"
  PRIMARY KEY (tracking_number, event_id)
) PARTITION BY RANGE (at);

-- a scanner retrying over a bad network must NOT create a second event
CREATE UNIQUE INDEX ux_scan_dedupe ON scan_events (scanner_seq);

CREATE TABLE api_keys (
  key_hash TEXT PRIMARY KEY, -- store the HASH, never the key
  partner_id BIGINT NOT NULL,
  tier SMALLINT NOT NULL, -- 0 free, 1 pro, 2 enterprise
  revoked_at TIMESTAMPTZ
);

CREATE TABLE webhooks (
  webhook_id BIGINT PRIMARY KEY,
  partner_id BIGINT NOT NULL,
  url TEXT NOT NULL,
  secret_enc BYTEA NOT NULL,
  events TEXT[] NOT NULL,
  state SMALLINT NOT NULL -- 0 active 1 backing-off 2 disabled
);

CREATE TABLE webhook_deliveries (
  delivery_id BIGINT PRIMARY KEY,
  webhook_id BIGINT NOT NULL,
  event_id BIGINT NOT NULL,
  attempts SMALLINT NOT NULL DEFAULT 0,
  next_try_at TIMESTAMPTZ NOT NULL,
  state SMALLINT NOT NULL, -- 0 pending 1 delivered 2 dead
  UNIQUE (webhook_id, event_id) -- one delivery row per event per webhook
);

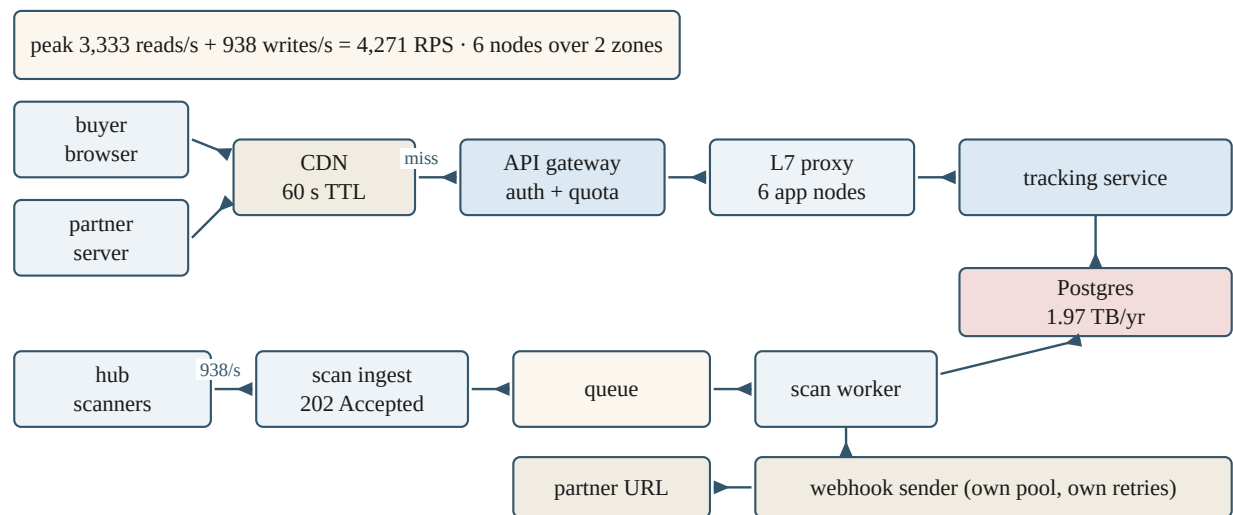
```

Two decisions worth defending out loud.

(a) ux_scan_dedupe on scanner_seq. The scanner generates <scannerId>-<sequenceNumber> and retries with the **same** value. The second insert fails with a unique violation, which the ingest code catches and turns into 202 Accepted. The deduplication is done by the database, so no race between two retries can slip through. This is the same move as the library’s “one open loan per copy” index: **turn the rule into a constraint**.

(b) api_keys.key_hash, never the key. An API key is a password. Store sha256(key) and compare hashes. If the table leaks, nobody can call your API with it. Candidates almost never say this, and it is free marks.

Step 5 — Architecture



Read path through the CDN and gateway; write path through a queue. The webhook sender is separate so a slow partner cannot slow ingestion.

Step 6 — Deep dive 1: rate limiting that survives contact with reality

The tiers.

Tier	Partners	Limit	= RPS each	Sum
free	3,600	60 / min	1	3,600
pro	380	600 / min	10	3,800
enterprise	20	6,000 / min	100	2,000
	4,000			9,400 RPS

Check the arithmetic:

$$3,600 \times 1 + 380 \times 10 + 20 \times 100 = 3,600 + 3,800 + 2,000 = 9,400$$

Now the sentence almost nobody says: my fleet serves 4,000 RPS at healthy utilisation, but the sum of the limits I have promised is 9,400 RPS.

$$\text{oversubscription} = \frac{9,400}{4,000} = 2.35 \times$$

I am **2.35x oversubscribed**. That is normal — airlines and ISPs do the same — but it must be a **decision**, not an accident, and it carries three obligations:

1. **A global limiter as well as a per-key one.** If total traffic crosses 4,000 RPS, start shedding the **lowest tier first**. A free-tier 429 is a broken sample script; an enterprise 429 is a broken business.
2. **A concurrency cap per node**, because the failure mode is not “too many requests”, it is “requests got slow” (Piece 2).
3. **Alert on the oversubscription ratio itself.** When sales sign 200 more pro partners, the ratio becomes $(9,400 + 200 \times 10) / 4,000 = 11,400 / 4,000 = 2.85$, and somebody has to decide whether to buy nodes.

Where the limiter runs. Not in the app — with 6 nodes, a local limit of 60/min per key would let a client do $6 \times 60 = 360/\text{min}$ by getting unlucky across nodes. It runs in the **gateway**, backed by a shared counter in Redis.

$$\text{Redis ops at peak} = 4,271 \text{ RPS} \times 1 \text{ op} = 4,271 \text{ ops/s}$$

which is under 5% of one Redis node. The GCRA algorithm from Piece 1 fits this perfectly: one GETSET-style operation, one small value per key.

$$\text{memory} = 4,000 \text{ keys} \times 100 \text{ bytes} = 400 \text{ KB}$$

Decision: GCRA in the gateway on shared Redis, plus a per-node concurrency cap, plus a global shed-by-tier rule. The failure mode of Redis being down is “fail open” — allow the request — because a rate limiter must never become the reason the API is down.

Step 6 — Deep dive 2: webhooks, which are an API you call

Polling costs 833 RPS of mostly-unchanged answers. A webhook fires only on change:

changes/day = 27,000,000 scans → but only 2 of 9 codes are interesting

$$27,000,000 \times \left(\frac{2}{9}\right) = 6,000,000 \text{ webhook events/day}$$

$$\frac{6,000,000}{86,400} = 69.4 \text{ deliveries per second}$$

That is 12x less traffic than polling — but now **you** are the client, and every mistake a bad API client makes is now yours to avoid.

THE FIVE RULES OF SENDING A WEBHOOK

1. **Sign the body.** X-Signature: sha256=<hmac(secret, timestamp + "." + body)>, with the timestamp inside the signed material so an old capture cannot be replayed. Without this, anyone who learns the URL can post fake “delivered” events.
2. **At-least-once, with an id.** Include eventId and tell partners to ignore duplicates. You cannot promise exactly-once across a network you do not control; promising it is a lie that breaks their accounting.
3. **Timeout hard — 5 seconds — and retry with backoff:** 1 s, 10 s, 1 min, 10 min, 1 h, 6 h. Six attempts across about 7 hours, then dead-letter.
4. **Isolate slow partners.** One partner whose endpoint takes 5 seconds must not consume the sender pool. Give each webhook its own small concurrency budget, and use the circuit breaker from earlier: after 20 consecutive failures, mark the webhook backing-off and stop trying for an hour.
5. **Give them a replay endpoint.** POST /v1/webhooks/{id}/replay?from=... lets a partner recover from their own outage without emailing support. This single endpoint removes most webhook support tickets.

The arithmetic that decides the sender pool size. Little’s law again:

$$\text{in flight} = 69.4 \text{ deliveries/s} \times 0.4 \text{ s average} = 27.8$$

So about 28 concurrent HTTP calls in steady state — one small worker handles it. But size for the bad day: if 10% of partners go slow and take the full 5-second timeout,

$$69.4 \times 0.1 \times 5 = 34.7 \text{ in flight on the slow path alone}$$

so provision roughly $28 + 35 = 63$ concurrent slots, with the slow ones **capped per partner** so they cannot take all 63.

Step 6 — Deep dive 3: caching a tracking page at the edge

The 60-second Cache-Control is worth an explicit number.

$$\text{requests/day} = 72,000,000$$

$$\text{distinct parcels checked} = 12,000,000$$

If each parcel is checked 6 times a day and the cache holds an answer for 60 seconds, the cache only helps when two checks fall inside the same minute. Bursty checking makes that common; assume a measured 70% hit rate.

$$\text{origin RPS at peak} = 3,333 \times (1 - 0.70) = 1,000 \text{ RPS}$$

$$\frac{3,333}{1,000} = 3.3 \times \text{less origin traffic}$$

That changes the node count:

$$\frac{1,000 + 938}{2,000} = 0.97 \rightarrow 1 \text{ node of work}$$

which, with the same zone-failure rule, becomes 2 per zone = 4 nodes instead of 6.

The cost, stated honestly: a buyer may see a status up to 60 seconds old. For “out for delivery” that is fine. For the **final** “delivered” event it is annoying. So: shorten the TTL to 10 seconds once the parcel enters “out for delivery”, and purge the key when the delivered event is processed. Say both halves — the saving **and** the staleness you chose to accept.

Step 7 — Trade-offs, failure modes, and 10x

Decision	Chosen	Rejected	Why in one line
scan writes	queue, return 202	write synchronously, 201	a database blip must never make a hub scanner retry a truck-load of parcels
dedupe	unique index on scanner_seq	check-then-insert in code	two retries 5 ms apart both pass a check; only a constraint is race-free
rate limiting	GCRA in the gateway on shared Redis	per-node counters	6 nodes would silently allow 6x the published limit
Redis unavailable	fail open	fail closed	the limiter must never be the reason the API is down
notifications	webhooks + polling both	webhooks only	some partners cannot accept inbound calls; polling is their only option
read freshness	60 s CDN TTL, 10 s when out for delivery	no caching	3.3x less origin traffic for a staleness nobody notices until the last hop

Failure modes.

Failure	Symptom	What I do
one app node dies	passive ejection after 5 errors	the proxy retries the GET on another node; users see nothing. Writes are already in the queue.
a whole zone dies	half the nodes gone	3 remaining nodes at 71% (computed above). DNS/ALB stops sending to the dead zone in 30–60 s.
queue backs up	scans appear minutes late	tracking shows a stale status but never a wrong one. Alert on queue age, not queue length.
one partner polls 50x their quota	429s for them only	per-key limiting contains it by construction. If they persist, revoke the key.
a partner’s webhook URL hangs for 30 s	sender pool fills	5 s timeout + per-partner concurrency cap + circuit breaker after 20 failures
Redis (limiter) dies	no limiting	fail open, alert loudly. The concurrency cap per node is the backstop that keeps nodes alive.

At 10x (120 million parcels in flight, 33,330 reads/s peak).

1. The CDN does more of the work, not less: at 10x the traffic on the same 12 million hot parcels, the hit rate rises. Measure it before buying nodes.
2. Split reads and writes into separate services and separate pools. Today they share nodes; at 10x a scan-ingest spike would hurt buyer reads.

3. Move `scan_events` out of Postgres — at 10x it is 19.7 TB/year and is only ever read by tracking number, which is exactly the wide-column access pattern.
4. Regionalise: put read replicas and a gateway in each major market, keep writes in one region. Tracking reads tolerate replica lag; label creation does not.

Example 15 [Grab · pattern]

A partner complains: “your API returned 200 OK but the shipment was created twice.” Your code has an idempotency key. Find the bug.

SOLUTION

There are exactly three places this can break. Walk them in order.

Bug A — the key is stored after the work, not before.

```
// WRONG
const existing = await store.get(key);
if (existing) return existing.response;
const shipment = await createShipment(body); // <- two retries are both here
await store.put(key, { response: shipment });
```

Two retries 5 ms apart both find nothing, both create. The window is the whole duration of `createShipment`.

Fix: claim the key **first**, with an insert that fails if it exists.

```
// begin() is an INSERT ... ON CONFLICT DO NOTHING
const claim = await store.begin(key, fingerprint, now);
if (claim.state === "done") return claim.response;
if (claim.state === "in_progress") return { status: 409, retryAfter: 1 };
```

Bug B — the key is scoped globally instead of per partner. If two partners happen to send the same UUID, one sees the other’s shipment. The key must be `(partner_id, idempotency_key)`.

Bug C — the client generates a new key per attempt. This is the most common cause and it is **not your bug** — but it is your documentation’s bug. A client that calls `uuid()` inside the retry loop defeats the whole mechanism. The docs must show the key being generated **before** the first attempt and reused for every retry, and the API should reject a POST `/v1/shipments` that arrives with no key at all, so the mistake is impossible to make silently.

How I would find which one it is: log `idempotency_key`, `partner_id` and `request_fingerprint` on every create. Two rows with the same key means A or B. Two rows with different keys and the same fingerprint means C.

Answer: Claim the key before doing the work (insert-or-fail), scope it per partner, and require the key so a client cannot silently omit it. The “different key per retry” case is a docs bug.

Example 16 [Agoda · pattern]

Your p50 is 30 ms and p99 is 900 ms. The gateway timeout is 1 second. Traffic doubles and everything falls over. Explain why, with Little’s law.

SOLUTION

Before. Average latency with a 99/1 split:

$$0.99 \times 30 + 0.01 \times 900 = 29.7 + 9 = 38.7 \text{ ms}$$

At 2,000 RPS:

$$\text{in flight} = 2,000 \times 0.0387 = 77.4$$

Comfortable against, say, a 200-slot worker pool.

After doubling to 4,000 RPS, if latency stayed the same:

$$4,000 \times 0.0387 = 154.8 \rightarrow \text{still under 200}$$

So on paper it should be fine. It is not, and here is why.

The utilisation law bites. At 2,000 RPS the fleet was at, say, 50% utilisation, so the queueing multiplier was $1/(1 - 0.5) = 2$. At 4,000 RPS it is at 100% — the multiplier goes to infinity. Real latency does not stay at 38.7 ms; it climbs.

Suppose average latency rises to 300 ms:

$$\text{in flight} = 4,000 \times 0.300 = 1,200$$

against 200 slots. **Six times oversubscribed.** Requests queue for slots, which raises latency further, which raises the in-flight count further. This loop is called **congestion collapse**, and it never recovers on its own.

Then the timeout makes it worse. Requests that queue for more than 1 second are abandoned by the gateway — but the **server is still working on them**. You are now spending 100% of your CPU producing responses that nobody will read.

The three fixes, in order of effect.

1. **Shed load at the door.** The concurrency limiter: if in-flight is at 200, return 503 immediately. 503 in 1 ms is enormously better than 200 OK in 4 seconds, because the client can retry elsewhere and the server stays healthy.
2. **Drop work whose deadline has passed.** Before starting a request, check whether the caller's deadline is already gone; if so, discard it without doing the work.
3. **Add nodes so peak utilisation is 65%,** which is the whole point of the earlier arithmetic.

The sentence to say: “The system did not fail because of the extra traffic; it failed because it had no way to say no.”

Answer: Utilisation crossed 100%, so latency rose, so in-flight requests (Little's law) exploded past the worker pool. Fix by shedding at the door, dropping expired work, and sizing for 65%.

Tier 3 · Large-scale: the global edge at one million requests per second

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 17 [Google · pattern]

Design the edge — everything from the user's DNS lookup to your service — for a product serving 1 million requests per second across four regions. All seven steps.

SOLUTION

Step 1 — Clarify

Question	What it changes
Must a user always reach the nearest region, or the fastest healthy one?	Nearest is DNS geo-routing, simple and static. Fastest healthy needs anycast plus live health data, and it is what makes failover invisible.
Is any request sticky to a region (session state, a write leader)?	If yes, a region failover is a correctness problem, not just a capacity one. If no — fully stateless — failover is only arithmetic.
What is the p99 latency target?	200 ms lets me terminate TLS at a regional edge. 50 ms forces edge points of presence close to users, which is a much bigger build.
What must survive: one node, one zone, or one whole region?	This is the single most expensive question in the design. Surviving a region costs 33% more hardware — computed below.

Are there long-lived connections (WebSocket, gRPC streams)?

Long connections make deploys hard: draining a node means waiting out connections that may last hours, so I need connection lifetime limits.

Assumptions: fastest healthy region via anycast; fully stateless requests; p99 target 150 ms; must survive the loss of one entire region; mostly short HTTP requests with keep-alive.

Step 2 — Scale estimate

Per region, in the normal case.

$$\frac{1,000,000}{4} = 250,000 \text{ RPS per region}$$

L7 proxy nodes. One tuned L7 node handles about 25,000 RPS.

$$\frac{250,000}{25,000} = 10 \text{ nodes per region}$$

But we promised to survive losing a region. The other three must carry everything:

$$\frac{1,000,000}{3} = 333,333 \text{ RPS per surviving region}$$

$$\frac{333,333}{25,000} = 13.3 \rightarrow 14 \text{ nodes per region}$$

$$14 \times 4 = 56 \text{ nodes total}$$

$$\text{the cost of region redundancy} = \frac{56 - 40}{40} = 40\% \text{ more hardware}$$

Name that number out loud. “Surviving one region costs 40% more edge hardware” is a business decision, not an engineering one, and showing you know its price is the point.

Connections in flight. Little’s law, with an average request lifetime of 60 ms:

$$1,000,000 \times 0.060 = 60,000 \text{ requests in flight globally}$$

$$\frac{60,000}{56} = 1,071 \text{ concurrent requests per node}$$

Fine for an event-driven proxy; impossible for a thread-per-request one at 1 MB of stack each (1,071 MB per node).

TLS cost. With keep-alive at 50 requests per connection:

$$\text{new connections/s} = \frac{1,000,000}{50} = 20,000 \text{ globally}$$

$$\text{per region} = \frac{20,000}{4} = 5,000 \text{ per second}$$

$$\text{CPU at 1.5 ms per full handshake} = 5,000 \times 0.0015 = 7.5 \text{ cores per region}$$

With 80% session resumption:

$$5,000 \times 0.20 \times 0.0015 = 1.5 \text{ cores per region}$$

Bandwidth. An average response of 1.5 KB:

$$1,000,000 \times 1,500 = 1.5 \times 10^9 \text{ bytes/s} = 1.5 \text{ GB/s}$$

$$\times 8 = 12 \text{ Gbit/s globally} = 3 \text{ Gbit/s per region}$$

Notice which of those numbers is the binding constraint. Not bandwidth (3 Gbit/s per region is one network card). Not TLS (1.5 cores). It is **request rate**: 25,000 RPS per node is what sets the fleet size. Say which number binds and why — it proves the estimate was thinking, not ritual.

Step 3 — The edge contract

An edge has an API too. It is just not the product's API.

What every backend must implement, and what the edge adds

TEXT

Every backend MUST serve:

```
GET /_health/live      200 if the process is alive      (restart me if not)
GET /_health/ready     200 if it should get traffic     (route to me or not)
POST /_lb/drain        stop passing readiness, finish open requests
```

Every request the edge forwards CARRIES:

```
X-Request-Id:         <uuid>          made at the edge if absent, logged everywhere
X-Forwarded-For:      <client ip>     appended, never trusted from the client
X-Forwarded-Proto:    https
X-Request-Deadline:   <unix ms>       the budget; every hop must honour it
traceparent:          <w3c trace>     so one request is one trace across services
```

Every response the edge adds:

```
Server-Timing: edge;dur=2, origin;dur=41
X-Cache: HIT | MISS | BYPASS
```

Edge policy, per route:

```
timeout              2s connect, 10s total
retries              GET/HEAD/PUT/DELETE: 1 retry on connect-error or 502/503/504
                    POST: never, unless Idempotency-Key is present
retry budget         retries may not exceed 10% of requests to that pool
outlier ejection     5 consecutive 5xx -> eject for 30s, doubling each time
```

TRAP

/_health/live and /_health/ready must be **different**. live answers “is the process broken?” — if it fails, restart the container. ready answers “should I get traffic right now?” — it can go false while the process is perfectly healthy, during warm-up or draining.

Teams that use one endpoint for both get a very specific disaster: a database blip makes health checks fail, the orchestrator decides every container is dead, and it restarts the **entire fleet** at once. The application was fine. The health check killed it.

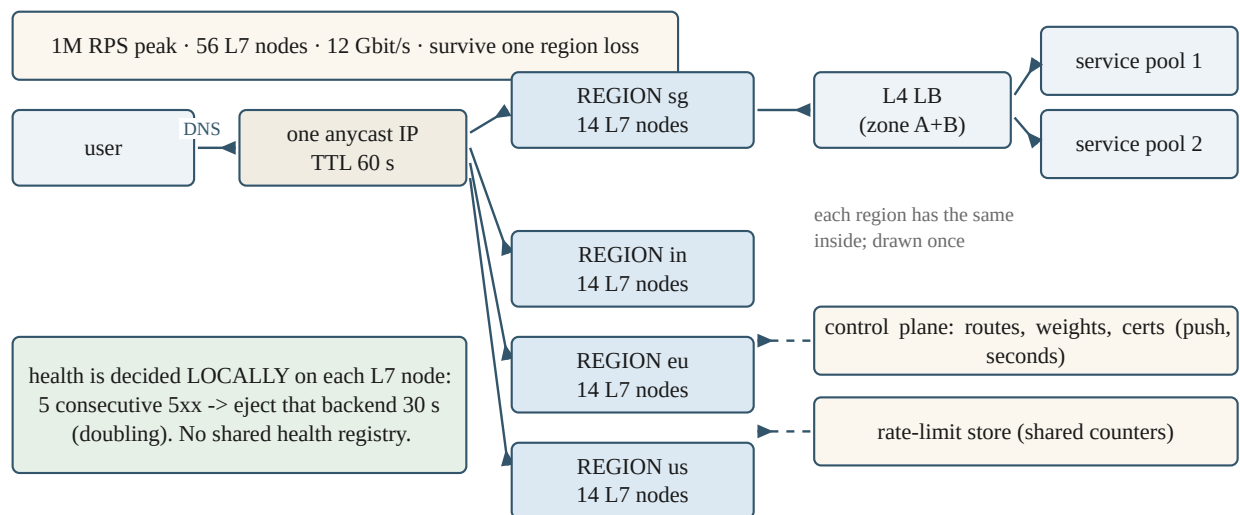
Step 4 — The data model of an edge

An edge is mostly stateless, but not entirely. Three pieces of state, with very different freshness needs:

State	What it is	Freshness	Where it lives
routing config	route patterns, backend pools, weights, timeouts	seconds	pushed from a control plane; each node keeps the last good copy on disk
health state	which backends are up, per node	milliseconds	local to each node , from its own observations — never shared
rate-limit counters	per API key	milliseconds	a shared store, with a local pre-filter
TLS certificates	per domain	hours	pushed, cached, auto-renewed

The decision worth defending: health state is local, not shared. A shared health registry sounds better — “every node knows everything” — and is worse for two reasons. **One:** it is a single point of failure that can declare your entire fleet dead. **Two:** health is not global. Node 7 may genuinely be unable to reach backend B because of a broken link between them, while node 8 reaches it fine. A local view is **more correct**, not less. Active checks from a central place are a useful **slow** signal; the fast signal is each node watching its own traffic.

Step 5 — Architecture



One anycast address, four regions. Health is decided locally at each L7 node, not centrally.

Step 6 — Deep dive 1: what actually happens when a region dies

Walk the clock. This is the answer an interviewer is really asking for.

t = 0 s	Region sg loses power. 250,000 RPS of users are mid-request; they get connection resets.
t = 0–10 s	Anycast withdraws the route (BGP). New packets to the same IP are delivered to the next closest region. Users who retry immediately already succeed. Users on a pinned TCP connection wait for their timeout.
t = 10 s	in and eu each gain roughly 125,000 RPS. in was at $250,000 / (14 \times 25,000) = 71\%$; it is now at $375,000 / 350,000 = 107\%$ — over capacity .
t = 10–60 s	Load shedding engages: the concurrency limiter returns 503 for traffic above capacity, protecting latency for the rest. Roughly 7% of requests are shed. This is the design working, not failing.
t = 60 s	The three surviving regions were provisioned for 333,333 RPS each — that is why we bought 14 nodes instead of 10 — so once traffic settles evenly, all of it fits: $333,333 / 350,000 = 95\%$.
t = 2–5 min	Autoscaling adds nodes if configured, bringing utilisation back under 70%.

Two honest admissions to make unprompted.

One: at $t = 60$ s the surviving regions are at 95% utilisation, and the utilisation law says that is a 20x latency multiplier. Surviving a region at **full quality** would need

$$\frac{1,000,000}{3 \times 0.65 \times 25,000} = 20.5 \rightarrow 21 \text{ nodes per region}$$

$$21 \times 4 = 84 \text{ nodes} = 110\% \text{ more than the } 40 \text{ we need for a normal day}$$

Decision: buy 14, not 21. Accept degraded latency for the first few minutes of a region-loss event, because it happens perhaps once a year and paying 110% extra every day to make it invisible is not worth it. State the trade with the numbers; do not pretend it is free.

Two: traffic does not redistribute evenly. Anycast sends **sg**'s users to whichever region is closest **by network topology**, which may be **in** for almost all of them. Plan for the worst split (one region absorbs everything) by making the load-shedding rule per region, not global.

Step 6 — Deep dive 2: deploys without dropping a request

56 nodes, deployed one at a time, is slow. Deployed all at once, it is an outage. The arithmetic decides the batch size.

Capacity during a rolling deploy. With 14 nodes per region and batch size b :

$$\text{capacity during the deploy} = \frac{14 - b}{14}$$

Batch	Capacity left	Utilisation at 250k RPS	Verdict
1	92.9%	76.9%	safe, but 14 rounds
2	85.7%	83.3%	acceptable, 7 rounds
3	78.6%	90.9%	10x latency multiplier — no
7	50%	143%	outage

$$\text{utilisation at batch } b = \frac{250,000}{(14 - b) \times 25,000}$$

Check batch 2: $250,000 / (12 \times 25,000) = 250,000 / 300,000 = 83.3\%$. Correct.

Decision: batch size 2, seven rounds per region. And the sequence for each node matters more than the batch size:

1. **POST /_lb/drain** — the node starts failing **ready**, so the L4 layer stops sending **new** connections. It is still serving the ones it has.
2. **Wait the drain window.** How long? Long enough for in-flight requests to finish: p99.9 request time ≈ 2 s, so wait 30 s to be safe. For long-lived connections, send **Connection: close** on responses so clients reconnect elsewhere naturally.
3. **Then** stop the process. Now zero requests are lost.
4. Start the new version; it fails **ready** while it warms up (JIT, connection pools, caches). **Only then** does it get traffic.

TRAP

Skipping step 2 is the single most common cause of “we see 502s during every deploy”. The orchestrator says the pod is terminating and the load balancer has not yet noticed — there is a gap of a few seconds in which traffic is still being sent to a process that is exiting. The fix is boring and reliable: fail readiness **first**, wait longer than you think, **then** exit.

Step 6 — Deep dive 3: choosing the algorithm at 2,000 backends

Behind the 56 L7 nodes sit thousands of service instances. The simulation from Tier 1 said least-connections and power-of-two-choices perform within 1% of each other. At this scale the tie is broken by two other facts.

Fact 1 — cost per decision.

$$\text{least connections at 2,000 backends} = 1,000,000 \times 2,000 = 2 \times 10^9 \text{ comparisons/s}$$

$$\text{power of two} = 1,000,000 \times 2 = 2 \times 10^6 \text{ comparisons/s}$$

A factor of 1,000.

Fact 2 — each proxy node has a different view. With 56 nodes, “least loaded” is computed from 56 separate, slightly stale pictures. They agree often enough to all pick the same backend at the same moment — and then that backend is the **most** loaded one, a millisecond later. Power of two choices cannot herd, because the two candidates are random.

Decision: power of two choices, plus a bounded-load guard. The guard is one line: never send to a backend whose in-flight count exceeds $c \times \text{average}$, with $c = 1.25$. If both random picks are over the bound, sample again. This gives you P2C’s $O(1)$ cost with a hard ceiling on how bad any single backend’s queue can get.

And one exception I would carve out: requests that must reach a specific instance — a cache shard, a WebSocket session — use consistent hashing with bounded loads instead, keyed on the session or shard id. Different question, different algorithm; say which one you are answering.

Step 7 — Trade-offs, failure modes, and 10x

Decision	Chosen	Rejected	Why
region routing	anycast	DNS geo-routing	failover in ~10 s (BGP) versus 60+ s (DNS TTL), and clients cache DNS badly
region redundancy	14 nodes/region (N+1 region)	10 nodes/region	40% more hardware buys survival of a full region loss
failover quality	degraded for minutes	21 nodes/region for full quality	110% extra cost every day to make a once-a-year event invisible is bad value
backend selection	power of two + bounded load	least connections	1,000x cheaper per decision and immune to cross-node herding
health decisions	local per node	a shared health registry	a shared registry is a single point of failure that can declare the fleet dead
deploys	batch of 2, drain 30 s first	batch of 3+	batch 3 puts the region at 91% utilisation — a 10x latency multiplier
overload	shed at the door with 503	queue everything	a fast 503 is recoverable; congestion collapse is not

Failure modes.

Failure	Symptom	What happens
one L7 node dies	1/14 of a region's capacity gone	L4 health check ejects it in ~2 s; region goes from 71% to 77% utilisation. Invisible.
one backend goes slow, not down	p99 climbs, no errors	the hardest case. Passive ejection only counts 5xx, and a slow backend returns 200. Fix: eject on latency outliers too — eject a backend whose p99 is over 3x the pool median.
a retry storm after a blip	3x traffic on a recovering service	retry budget caps retries at 10% of traffic; circuit breakers open; full-jitter backoff spreads what remains
control plane pushes a bad route	every node breaks at once	the scariest failure in this design. Mitigate: canary the config to 1 node for 60 s before the fleet, validate the config on each node before applying, and keep the last good copy on disk so a node can boot without the control plane.
certificate expires	total outage on one domain	alert at 30 days, auto-renew at 30 days, page at 7. This is a boring, extremely common, entirely preventable outage.
an anycast region is up but its backends are down	users routed into a black hole	the edge must withdraw its own anycast announcement when its backends fail health checks. An

		edge that is healthy but has nothing to serve must stop attracting traffic.
--	--	---

At 10x (10 million RPS).

1. 56 nodes becomes 560 by the same arithmetic — edge scales linearly because it is stateless. The thing that does **not** scale linearly is the control plane pushing config to 560 nodes; move from push to pull-with-jitter so 560 nodes do not all fetch at once.
2. Add edge points of presence: terminate TLS within 30 ms of the user even when the service is 200 ms away. This turns 2 RTT of handshake from 400 ms into 60 ms.
3. Move static and cacheable responses to the CDN entirely, so the 10x applies only to the dynamic remainder. Measure what fraction that is before buying 560 nodes.
4. At 10 million RPS, per-key rate-limit counters become the bottleneck: switch to a local token bucket per node with periodic reconciliation, accepting a small overshoot in exchange for zero shared-store round trips on the hot path.

Example 18 [Amazon · pattern]

One backend in a 500-instance pool responds in 4 seconds instead of 40 ms, but always returns 200. Your ejection rule counts 5xx responses. What happens, and how do you fix it?

SOLUTION

What happens. The 5xx-based ejection never fires, because there are no 5xx responses. So the slow instance stays in the pool and keeps receiving traffic.

With **round robin** it receives 1/500 of traffic, and 0.2% of all requests take 4 seconds. If the pool serves 100,000 RPS, that is 200 requests per second at 4 seconds — and those 200 requests **hold resources**. Little’s law:

$$200 \times 4 = 800 \text{ requests permanently in flight on one instance}$$

It will hit its connection limit and then start actually failing.

With **least connections or P2C**, the effect is much smaller and this is a real argument for those algorithms: the slow instance accumulates in-flight requests, so it is chosen less and less. It self-limits. But it never reaches zero, so a slice of users still waits 4 seconds.

The p99 arithmetic. Even at 0.2% of requests, this instance alone puts a 4-second answer at the 99.8th percentile. Your p99.9 SLO is dead, and every dashboard looks fine.

The fixes, in order.

1. **Eject on latency, not only on errors.** Compute the pool’s median p99 continuously and eject any instance whose p99 exceeds 3x it. This is the actual fix and it is what every serious proxy calls “outlier detection”.
2. **Hedged requests for read-only calls.** If a GET has not responded in the pool’s p95 (say 80 ms), send a second copy to a different instance and take whichever answers first. Cost:

$$\text{extra load} = 5\% \text{ of reads}$$

Benefit: the 4-second tail disappears entirely, because the hedge answers in 40 ms. Cap hedging at 5% of traffic so it cannot amplify during a general slowdown.

3. **A per-request deadline** so nothing can take 4 seconds in the first place — it becomes a fast failure that the retry logic can handle.

Decision: all three. Outlier ejection removes the cause, hedging hides the tail while detection happens, and deadlines bound the damage. Say why hedging must be capped: without a cap, a slow **pool** would cause every request to be duplicated, doubling load on something that is already struggling.

Answer: Error-based ejection never fires. Add latency-based outlier ejection (3x pool p99), capped hedged reads at 5%, and per-request deadlines.

Example 19 [Microsoft · pattern]

Your autoscaler adds nodes when CPU exceeds 70%, and takes 3 minutes to boot one. Traffic doubles in 30 seconds. Compute what happens and design around it.

SOLUTION

The setup. 14 nodes per region at 250,000 RPS, so $250,000 / 14 = 17,857$ RPS per node against a 25,000 capacity = 71% utilisation.

t = 0 to 30 s: traffic doubles to 500,000 RPS.

$$\frac{500,000}{14 \times 25,000} = \frac{500,000}{350,000} = 143\% \text{ of capacity}$$

t = 30 s: the autoscaler notices and asks for more nodes. How many does it need?

$$\frac{500,000}{0.65 \times 25,000} = 30.8 \rightarrow 31 \text{ nodes}$$

It must add 17 nodes.

t = 30 s to 3 min 30 s: the nodes are booting. For **three minutes**, the region is at 143% of capacity. Without protection, this is congestion collapse: queues grow, latency grows, in-flight count grows, and the nodes fall over one by one.

$$\text{requests arriving during the gap} = 500,000 \times 180 \text{ s} = 90,000,000$$

$$\text{requests the fleet can serve} = 350,000 \times 180 = 63,000,000$$

$$\text{excess} = 27,000,000 \text{ requests with nowhere to go}$$

The design, in four parts.

1. **Shed immediately, do not queue.** The concurrency limiter returns 503 for everything above 350,000 RPS. 63 million requests are served correctly at normal latency; 27 million get a fast 503 and a Retry-After. Compare that to the alternative, where **all 90 million** get a slow timeout. Shedding is not failure; it is choosing who gets a good answer.
2. **Scale on the right signal, earlier.** CPU is a lagging indicator. Scale on **in-flight requests** or **queue depth**, which move within milliseconds. And set the threshold at 60%, not 70%, buying

$$(0.70 - 0.60) \times 14 \times 25,000 = 35,000 \text{ RPS of extra warning}$$

— worth roughly 30 extra seconds of runway at this growth rate.

3. **Keep warm capacity.** Pre-booted nodes sitting at zero traffic cost money but respond in seconds instead of minutes. Two warm nodes per region is

$$\frac{2}{14} = 14\% \text{ extra cost}$$

and covers a 14% traffic jump instantly.

4. **Attack the 3 minutes.** Most of a 3-minute boot is image pull and application warm-up. Pre-pulled images and a smaller startup path routinely get this to 40 seconds, which shrinks the danger window by 4.5x. This is usually the cheapest fix of the four and the one nobody mentions.

Decision: shed first (correctness), scale on in-flight (speed), keep 2 warm nodes (insurance), and fix the boot time (root cause). The order matters: without shedding, the other three only change how long the outage lasts.

Answer: 143% of capacity for 3 minutes, 27 million excess requests. Shed at the door, scale on in-flight rather than CPU, keep 2 warm nodes, and cut boot time from 3 min to 40 s.

Interview drill

Push 1: “L4 or L7? Pick one and defend it.”

Answer. Both, in layers, and I can say what each one buys.

L4 in front: it handles millions of connections on modest hardware, survives traffic floods, and has nothing to parse, so there is very little that can go wrong with it. It is what receives the anycast address.

L7 behind it: it is the only layer that can read a path, so it is the only layer that can route `/api` and `/static` to different pools, retry an idempotent request on a different backend, add a request id, and eject a backend on the **content** of its responses.

If forced to choose one: L7. Everything in this chapter that saves users — retries, outlier ejection, header propagation, per-route timeouts — requires reading the request. L4 alone would mean every failure is visible to the user.

The concrete cost of L7: it must terminate TLS, which is 1.5 cores per region in our numbers, and it can handle 25,000 RPS per node instead of millions of connections. Worth it.

Push 2: “You have sticky sessions. Now you cannot deploy. What now?”

Answer. Sticky sessions are a symptom; the disease is state in the process.

The real fix: move the session out — a signed token in a cookie for small state, or Redis for large state. Then any node can serve any request, deploys are trivial, and the load balancer goes back to being free to choose.

If I cannot change the application today, the interim plan is:

1. switch stickiness from source-IP hashing to a **cookie**, so a shared office NAT does not pin thousands of users to one node;
2. use consistent hashing so adding or removing a node moves $1/N$ of sessions, not all of them;
3. during deploys, drain a node for the full session timeout (say 30 minutes) rather than 30 seconds — which means deploys take hours, and that pain is the argument that finally gets the real fix funded.

And the honest exception: long-lived connections (WebSocket, server-sent events) are inherently sticky, and that is fine. There, plan for connection lifetime limits — force a reconnect every hour — so a deploy never has to wait for a connection that lasts all day.

Push 3: “Rate limit by what — IP, user, or API key?”

Answer. By API key where there is one, by user id where there is a login, and by IP only as the outermost, loosest net — and I will say why IP is nearly useless on its own.

IP is wrong because thousands of users share one NAT address on a mobile network or in an office. An IP limit strict enough to stop an attacker will block a whole college campus. And an attacker with a botnet has more IPs than your limit has meaning.

The layered answer I actually deploy:

1. **per API key or user:** the real limit, generous (60/min free tier).
2. **per IP:** a very loose net, maybe 600/min, to catch unauthenticated abuse on login endpoints.
3. **global per endpoint:** protects the backend regardless of who is calling.
4. **per expensive operation:** search and export get their own, much smaller budget, because one search costs what fifty reads cost.

The last one is the one candidates miss. **Limit by cost, not by count:** charge a search 10 tokens and a read 1 token from the same bucket.

Push 4: “Your API returns 200 with an error inside the body. Comment.”

Answer. It is wrong, and here is the specific damage.

Everything in the path — the proxy, the CDN, the client library, the monitoring — reads the status code. A 200 that means failure gets **cached**, is never **retried**, never trips a circuit breaker, and never appears on an error dashboard. You have hidden your own outage from yourself.

What I return instead: the right status code, plus one consistent error body:

```
{ "error": { "code": "parcel_not_found",
            "message": "No parcel with that tracking number.",
            "requestId": "01J9Z4K2QW",
            "docs": "https://api.example.com/errors/parcel_not_found" } }
```

Four rules: a **stable machine-readable code** (clients switch on it; never on the message text), a human message, the `requestId` so support can find the log line, and a docs link.

The one genuine exception: batch endpoints. `POST /v1/parcels/batch` with 50 items where 3 fail is legitimately 207 Multi-Status or a 200 with per-item results — because the request as a whole succeeded. Say the exception; it shows you know the rule rather than repeating it.

Push 5: “How do you version an API without breaking anyone?”

Answer. I version only when I must, and I define “must” precisely.

Breaking changes (need a new version): removing a field, renaming a field, making an optional request field required, narrowing a type, changing the meaning of a value, adding a new required error case.

Non-breaking (no new version): adding a response field, adding an optional request field, adding a new endpoint, adding a new enum value **if the docs always said “ignore values you do not know”**.

The mechanism: /v1/ in the path, and both versions live at once, served by the same code with a translation layer at the edge. I do **not** fork the service.

The part people forget — the exit plan. A version you cannot delete is a version you pay for forever. So: publish a sunset date the day v2 launches, return the `Sunset` and `Deprecation` headers on every v1 response, measure v1 usage per API key, and email the top callers directly. Turn it off in stages — an hour of “brownout” a week before the date is worth ten emails.

Push 6: “A single customer sends 100x their normal traffic. Walk me through it.”

Answer. Four layers catch it, in order, and each one has a number.

1. **Their per-key limit** (60/min) rejects the excess at the gateway with 429 and a `Retry-After`. Everyone else is unaffected. **This should be the end of the story.**
2. **If they are an enterprise key** with a 6,000/min limit, 100x is 600,000/min = 10,000 RPS against a fleet sized for 4,271. Now the global limiter engages and sheds by tier — free first, enterprise last.
3. **The per-node concurrency cap** is the backstop: whatever gets through, no node accepts more work than it can hold, so nothing collapses.
4. **Bulkheads:** expensive endpoints run on their own pool, so a customer hammering search cannot take down tracking reads.

Then the human part, which is half the answer: I look at the request pattern before revoking anything. A 100x spike is usually a retry loop in their code caused by **our** 500s, not an attack. Killing their key would be punishing them for our bug. So: contain it automatically, then look at whether their retries started right after one of our errors.

Push 7: “Why do you cap retries at one, not three?”

Answer. Because of the arithmetic in Example 12 and one more fact.

The arithmetic: three attempts means 3x load on a service that is already failing, and across three service hops it is $3^3 = 27x$.

The extra fact: the second retry almost never helps. If a request failed twice against two **different** backends, the problem is not that backend — it is the request, the dependency, or the whole pool. A third attempt is load without information.

So my configuration is: 1 retry, on a **different** backend, only for idempotent methods, only on connect errors and 502/503/504 (never on a timeout after the request was sent — the work may have happened), under a 10% retry budget, with full jitter.

And the number that proves it: with a 10% budget, a total outage takes traffic from 1,000 RPS to 1,100 RPS. Without a budget, it takes it to 3,000. The budget is the difference between a service that can recover when you fix it and one that cannot.

Push 8: “What does your load balancer do when EVERY backend is unhealthy?”

Answer. This is the question that finds out whether I have run one.

The naive behaviour — eject them all, then have an empty pool — is the worst possible outcome, because the pool is empty only when the health check is wrong. A network blip between the proxy and the backends makes every check fail while every backend is perfectly fine. Ejecting all of them turns a partial problem into a total outage that the proxy caused.

So the rule is “panic mode”: if more than 50% of backends are unhealthy, **stop honouring health status entirely** and load balance across all of them. The reasoning: once half the fleet looks dead, the health signal is more likely to be broken than the fleet.

Concretely, in this design:

- under 50% unhealthy: route only to healthy backends;
- over 50% unhealthy: ignore health, spread across everything, keep shedding at the concurrency cap, and page a human;
- pool genuinely empty: return 503 with `Retry-After`, and **never** crash the proxy — the `pool.length === 0` guard in the Tier 1 code exists for exactly this second.

PRACTICE · WARM-UP target: 15 min

1. Peak is 12,000 RPS. One node does 900 RPS. How many nodes for 65% utilisation? How many to also survive losing one of two zones?

- At 85% utilisation, what is the latency multiplier? At 70%? How much more traffic took you from 70% to 85%?
- 400,000 RPS, 25 requests per keep-alive connection, 1.2 ms of CPU per full handshake, 75% session resumption. Give new connections per second and CPU cores for TLS.
- A service does 8,000 RPS with 180 ms average latency. How many requests are in flight? If latency triples, what is the new number?
- A client sends 30 requests instantly against a limit of “10 per second, burst 10”. How many succeed, and what Retry-After does request 11 get?
- Three services chained, each with 2 retries, innermost one hanging. How many requests does the innermost service receive per user action?
- A health check runs every 3 s with 2 strikes. Worst-case detection time? At 20,000 RPS across 25 nodes, how many requests fail?

ANSWER KEY

- $12,000 / 900 = 13.3 \rightarrow 14$ bare minimum. For 65%: $12,000 / 0.65 = 18,462$ RPS of capacity, and $18,462 / 900 = 20.5$, so **21 nodes**. Zone survival: one zone must carry 12,000 alone at a safe level, $12,000 / (0.65 \times 900) = 20.5 \rightarrow 21$ per zone, so **42 nodes**.
- $1 / (1 - 0.85) = 6.7$ so **6.7x**; $1 / (1 - 0.70) = 3.3$ so **3.3x**. Traffic increase: $0.85 / 0.70 = 1.214$, so **21% more traffic doubled the latency**.
- $400,000 / 25 = 16,000$ **new connections per second**. Full handshakes: $16,000 \times 0.25 = 4,000$ /s. CPU: $4,000 \times 0.0012 = 4.8$, so **4.8 cores**. (Without resumption it would be $16,000 \times 0.0012 = 19.2$ cores.)
- $8,000 \times 0.180 = 1,440$ **in flight**. Tripled: $8,000 \times 0.540 = 4,320$. Same traffic, 3x the resources.
- 10 succeed** (the burst), 20 get 429. At 10 per second one request costs $1000 / 10 = 100$ ms, so request 11 gets **Retry-After: 100ms**.
- $3 \times 3 \times 3 = 27$ **requests** for one user action.
- Worst case $3 + 2 \times 3 = 9$ **seconds**. Per-node share $20,000 / 25 = 800$ RPS; typical detection $2 \times 3 = 6$ s, so $800 \times 6 = 4,800$ **failed requests** (7,200 in the worst case).

PRACTICE · TIER 2 target: 40 min

- Design the public API for a “restaurant table booking” service: search availability, hold a table for 5 minutes, confirm, cancel. Give 5 endpoints with payloads, say which need idempotency keys and why, give the rate-limit tiers, and give the status code for “the table was taken while you were holding the form open”. Then compute: 2 million searches a day with a 5x evening peak — what is the peak search RPS, and how many nodes at 1,200 RPS each with 65% utilisation?
- Your API has one endpoint that is 40x more expensive than the others (a report). It is 1% of requests. Compute what share of your total CPU it consumes, then design the protection around it. Give three specific mechanisms and the numbers that justify each.
- Take the parcel-tracking design and replace polling entirely with webhooks. Compute the traffic saved, then list everything that gets **harder**, and say whether you would actually do it.

ANSWER KEY

- Endpoints: GET `/v1/availability?restaurantId=&date=&partySize=&limit=&cursor=` (no key — it is a read); POST `/v1/holds` body `{slotId, partySize}` $\rightarrow$ 201 `{holdId, expiresAt}` **needs an idempotency key** — a retry must not consume two tables; POST `/v1/bookings` body `{holdId, contact}` $\rightarrow$ 201 `{bookingId}` **needs one** — it is a create; DELETE `/v1/holds/{holdId}` $\rightarrow$ 204 (idempotent by nature); DELETE `/v1/bookings/{bookingId}` $\rightarrow$ 204.

“The table was taken while you held the form open”: **409 Conflict**, with the error code `slot_no_longer_available` and a suggested alternative in the body. Not 400 (the client’s request was well-formed) and not 404 (the slot exists).

Tiers: anonymous search 30/min per IP; logged-in user 120/min; partner key 1,200/min. Holds are separately limited at 5 per user per 10 minutes, because a hold consumes inventory — a clear case of **limiting by cost, not by count**.

Peak: $2,000,000 / 86,400 = 23.1$ RPS average; $\times 5 = 115.7$ **RPS peak**. Nodes: $115.7 / (0.65 \times 1,200) = 0.148 \rightarrow$ 1 node of work, so **2 nodes for redundancy**, and 4 if a zone must survive. The honest answer here is “this does not need a big fleet” — say so rather than inventing scale.

2. If the report costs 40x and is 1% of requests, then out of 100 requests the cost is $99 \times 1 + 1 \times 40 = 139$ units.

$$\text{report share} = \frac{40}{139} = 28.8\% \text{ of all CPU}$$

1% of requests, 29% of the machine.

Three mechanisms: (a) **Its own pool (bulkhead)**. Size it from the arithmetic: 29% of the work gets its own nodes, so a report spike cannot touch the other 71%. (b) **A separate, much smaller rate limit** — 5/min per key rather than 600/min — justified by the 40x cost. Or one shared bucket where a report costs 40 tokens. (c) **Make it asynchronous**: `POST /v1/reports → 202 {jobId}`, then poll or webhook. This removes it from the request path entirely, so a slow report cannot consume a request slot (Little’s law: a 20-second report at 5 RPS is $5 \times 20 = 100$ permanently in flight).

3. Traffic saved: polling is 72M reads/day. Real status changes are about $27,000,000 \times (2/9) = 6,000,000/\text{day}$.

$$\frac{72,000,000}{6,000,000} = 12 \times \text{less traffic}$$

and the peak read rate falls from 3,333 RPS to $6,000,000/86,400 = 69.4$ per second of outbound deliveries.

What gets harder: you now run an HTTP **client** fleet with retries, backoff, signing, secret rotation, dead-letter handling and per-partner isolation; partners must expose a public HTTPS endpoint (many cannot); debugging moves to **their** logs, which you cannot see; every delivery is at-least-once so partners must deduplicate; a partner’s outage becomes your queue backlog; and you need a replay endpoint or you will drown in support tickets.

Decision: no — keep both. Offer webhooks as the efficient path and keep polling for partners who cannot receive calls, with a cheap conditional-GET path (`If-None-Match → 304 Not Modified`) so polling costs bytes but not database work. Removing polling saves 12x traffic you were already serving cheaply from a CDN, and buys a support burden.

PRACTICE · TIER 3 target: 50 min

1. Design the edge for 4 million RPS across 6 regions, surviving the loss of any **two** regions simultaneously. Give: RPS per region normally, RPS per surviving region, nodes per region at 30,000 RPS each with 65% target utilisation, total nodes, and the percentage cost of two-region redundancy versus none. Then say whether you would actually buy it.
2. Your p99 is 90 ms but p99.9 is 6 seconds. Traffic is 200,000 RPS. Compute how many requests per second are in the p99.9 tail and how many are in flight because of them. Design the three mechanisms that would remove that tail and say what each costs.
3. A config push breaks routing on all 300 edge nodes at once. Design the release process that makes this impossible, with specific time windows and the blast radius at each stage.

ANSWER KEY

1. Normal: $4,000,000 / 6 = 666,667$ **RPS per region**. Losing two: $4,000,000 / 4 = 1,000,000$ **RPS per surviving region**. Nodes: $1,000,000 / (0.65 \times 30,000) = 51.3 \rightarrow 52$ **nodes per region**, and $52 \times 6 = 312$ **nodes**. With no redundancy: $666,667 / (0.65 \times 30,000) = 34.2 \rightarrow 35$ per region, $35 \times 6 = 210$ nodes.

$$\text{cost of surviving two regions} = \frac{312 - 210}{210} = 48.6\%$$

Would I buy it? **No, not as stated**. Two simultaneous region failures are overwhelmingly likely to be **correlated** — a bad deploy, a bad config, an expired certificate — and extra hardware does not help with any of those. I would buy single-region redundancy ($4,000,000/5 = 800,000$ per region $\rightarrow 41$ nodes each $\rightarrow 246$ total, **17% extra**) and spend the difference on the things that actually cause correlated failure: staged config rollout, canary deploys, and certificate automation. **Say the reasoning, not just the number.**

2. p99.9 tail: $200,000 \times 0.001 = 200$ **requests per second at 6 seconds**. In flight because of them: $200 \times 6 = 1,200$ **requests permanently occupied** — versus the healthy traffic's $199,800 \times 0.09 = 17,982$. So **0.1% of requests hold 6.3% of all in-flight capacity**.

Three mechanisms: (a) **Latency-based outlier ejection** (eject a backend whose p99 is 3x the pool median). Cost: you may eject a healthy backend during a general slowdown, so it needs the same “panic mode” guard as health checks — never eject more than 50% of the pool. (b) **Hedged requests at the p95** for idempotent reads. Cost: 5% extra load; benefit: the 6-second tail collapses to roughly the p95. Must be capped so a slow **pool** does not double its own traffic. (c) **A hard per-request deadline** below 6 s — say 1 s. Cost: those 200 requests per second now **fail** instead of succeeding slowly. That is the right trade, because a 6-second answer was already useless to the user and was holding capacity hostage.

3. Stages, with blast radius at each:

1. **Validate locally on the node before applying.** A config that does not parse or that removes all backends from a route is rejected, and the node keeps its last good copy. Blast radius: 0.
2. **Canary to 1 node for 5 minutes.** Watch 5xx rate, p99 and request rate on that node against the fleet median. Blast radius: $1/300 = 0.33\%$ of traffic.
3. **1% of the fleet (3 nodes) for 10 minutes.** Blast radius 1%.
4. **One region for 15 minutes.** Blast radius $1/6 = 16.7\%$, and it is a region the anycast layer can withdraw in 10 seconds if it goes bad.
5. **The rest, one region at a time, 10 minutes apart.**
6. **Automatic rollback** on any stage where the 5xx rate exceeds 2x the baseline for 60 seconds — rollback must not require a human to be awake.

Total rollout: about 90 minutes. That feels slow until you compare it with the alternative, which is a 300-node outage. And the non-negotiable rule: **the last good config is on every node's disk**, so a node that reboots while the control plane is broken still serves traffic.

ONE-PAGE REVISION CARD

The four boxes and what each may know L4 (IP + port) → L7 (the whole HTTP request) → gateway (request + who sent it) → service (the business problem, and nothing about load balancing).

Numbers to memorise

one L7 proxy node	20–50k RPS	one app node, JSON	1–3k RPS
TLS full handshake CPU	1–2 ms	requests per kept-alive conn	20–100
safe utilisation	60–70%	health check	2 s, 3 strikes
drain before shutdown	30–60 s	DNS TTL for failover	30–60 s
anycast failover	~10 s	autoscaler boot	2–5 min
retry budget	10% of traffic	hedge cap	5% of reads

The formulas

- latency multiplier = $1/(1 - \rho)$ — 50% is 2x, 80% is 5x, 90% is 10x
- in flight = RPS × latency (s) (Little's law)
- nodes = peak RPS / (target utilisation × RPS per node)
- surviving-region RPS = total / (regions – failures survived)
- retry amplification = (attempts)^{hops}
- new connections/s = RPS / requests per connection
- TLS cores = full handshakes/s × handshake CPU seconds
- sum of limits = $\sum(\text{partners in tier} \times \text{tier RPS})$; compare it with capacity

Worked numbers from this chapter

- round robin with one 5x-slow backend: 6,666 of 200,000 requests dropped (3.3%); least-connections and power-of-two dropped 0
- P2C gave the slow backend 18,210 requests; least-connections gave it 18,004 — 1.1% apart
- circuit breaker: 6 of 19 calls never touched the sick backend
- 3 hops x 3 attempts = 27 requests and 81 seconds for one user action
- tracking API: 72M reads/day → 833 RPS avg, 3,333 peak; 27M scans → 938 peak
- 4,271 peak RPS → 3 nodes per zone x 2 zones; a zone loss leaves each at 71%
- sum of published limits 9,400 RPS against 4,000 of capacity = 2.35x oversubscribed
- edge: 1M RPS, 4 regions; 10 nodes/region normally, **14** to survive a region (+40%)
- 60,000 requests in flight globally; 1,071 per node
- TLS: 20,000 new conns/s → 7.5 cores/region, or 1.5 with 80% resumption
- deploy batch of 2 of 14 leaves 85.7% capacity = 83.3% utilisation; batch of 3 = 91%, too hot

API rules that score points

- nouns and HTTP methods; for real actions, a sub-resource with its own id
- /v1/ in the path; add fields freely, never remove or rename
- cursors, never offsets; the cursor is opaque and encodes a **total** order
- Idempotency-Key required on every create; claim the key **before** doing the work
- 429 always carries Retry-After; 4xx = the client can fix it, 5xx = only you can
- one error shape everywhere: stable code, human message, requestId, docs link
- never return 200 with an error inside

Sentences that score points

- “I terminate TLS at the L7 proxy, rate limit at the gateway, and my service never load balances.”
- “Least connections is $O(n)$ and herds; power of two choices is $O(1)$ and cannot.”
- “Health is decided locally on each node — a shared health registry can declare the whole fleet dead.”
- “Over 50% unhealthy, I enter panic mode and ignore health, because the checker is now the likelier fault.”
- “A fast 503 is recoverable. Congestion collapse is not.”

- “I promised 9,400 RPS of limits against 4,000 of capacity — that is a deliberate 2.35x oversubscription.”
- “Surviving a full region costs 40% more hardware. Surviving it at full latency costs 110%. I chose 40%.”
- “Fail readiness first, wait 30 seconds, then exit. That is why we have no 502s on deploy.”

Queues, Streams & Async Processing

Do the slow work later, and prove you did it exactly once

The idea in one page

A user taps **Place order**. Five things must happen:

Step	Time	Does the user care right now?
charge the card	300 ms	yes — they must know it worked
write the order row	20 ms	yes — they need an order id
send the confirmation e-mail	800 ms	no — 10 seconds later is fine
update the analytics warehouse	150 ms	no
warm the recommendation cache	50 ms	no

Do all five inside the request and the user waits $300 + 20 + 800 + 150 + 50 = 1320$ ms.

Do the first two inside the request, and **drop a note in a queue** for the other three. Dropping a note costs about 2 ms each. The user now waits $300 + 20 + 2 + 2 + 2 = 326$ ms.

$1320/326 = 4.05$. The page got about **4 times faster** and nothing was deleted — the slow work still runs, just not while a human stares at a spinner.

THE FIVE WORDS YOU MUST BE ABLE TO DEFINE

Queue. A line of messages. A consumer takes a message, works on it, and the message is **deleted**. One message goes to exactly one worker. Example use: “send this e-mail”.

Topic (a stream / log). An append-only list of messages kept on disk for a set time (say 7 days). Nothing is deleted when it is read. Many different consumers can read the same message for their own reasons. Example use: “an order was placed” — billing, search and analytics all want to know.

Offset. A number that says “I have read up to here”. Each consumer group keeps its own offset. This is why a stream can be replayed: set the offset back and read again.

Partition. A topic is cut into P independent logs. Order is guaranteed **inside** one partition only, never across partitions. Messages with the same **partition key** always land in the same partition.

Consumer group. A set of worker processes that share the partitions of a topic. Each partition is owned by exactly one member of the group at a time. So: **max useful workers in one group = number of partitions**. Memorise this line. It decides your partition count.

Queue or stream? Decide with this table

Question	Queue (SQS / RabbitMQ style)	Stream (Kafka style)
after reading, the message is	gone	still there until retention ends
same message to 3 teams?	needs 3 copies / fan-out	free — 3 consumer groups
can I replay yesterday?	no	yes, rewind the offset
ordering	usually none, or a FIFO queue	strict inside a partition
throughput per unit	thousands/s per queue	tens of MB/s per partition

operational weight	light, usually managed	heavy — brokers, replicas, rebalances
best fit	jobs, tasks, retries	events, audit, analytics, change capture

SHORTCUT

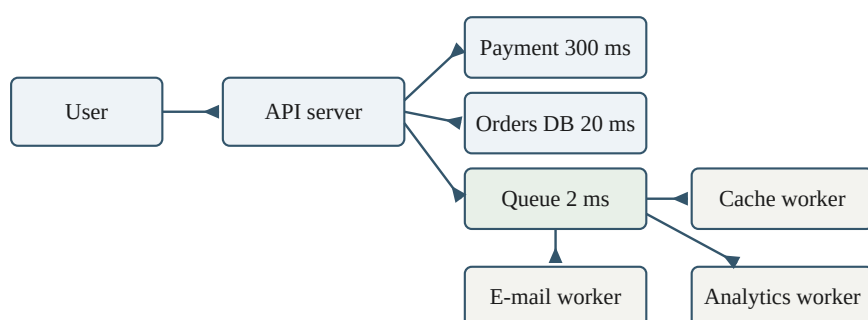
The one-line rule. If the message is an **order to do something** to one worker (“resize this image”), use a queue. If the message is a **fact that already happened** that several teams want (“user 42 placed order 99”), use a stream.

Delivery semantics — the three phrases interviewers grade you on

Name	How it is built	What goes wrong	When to pick it
At most once	ack the message before doing the work	work is lost on a crash	metrics, logs, anything you can lose
At least once	ack after the work succeeds	work runs twice on a crash	almost everything — the default
Exactly once	at-least-once plus a dedup or a transaction	costs latency and state	money movement, stock counts

TRAP

“Exactly once delivery” over a network is impossible. The sender can never tell the difference between “you did not get my message” and “you got it but your reply was lost”, so it must resend. What is possible is **exactly once effect**: deliver many times, but make the **effect** happen once. You get that from an idempotent consumer, not from the broker. Say it in those words in the interview.



User-visible latency = $300 + 20 + 3 \times 2 = 326$ ms. The grey work finishes within seconds.

Synchronous path vs. async path. The user only waits for the solid boxes.

The five failure words

Word	Plain meaning
Backpressure	producers are faster than consumers; you must slow the producer or drop, not silently pile up forever
Consumer lag	how many messages sit between the newest offset and my offset
Poison pill	one bad message that crashes the consumer every time it is retried, blocking the whole partition
DLQ (dead letter queue)	a side queue where a message goes after n failed tries, so the main line keeps moving
Head-of-line blocking	message 1 is stuck, so messages 2..1000 behind it wait, even though they are fine

Every question in this chapter is written by the author as a **pattern** — the shape of problem these companies ask. None of them is a transcript of a real interview. Every code block was run with `node` before it was printed here, and the output shown is the real output.

Warm-up — the mechanics

WARM-UP

build the reflex

Example 1 [warm-up]

A producer writes 5,000 messages per second. One consumer processes 3,000 per second. The system runs for 10 minutes. How many messages are waiting at the end?

SOLUTION

Messages in: $5\{, \}000 \times 600 = 3\{, \}000\{, \}000$.

Messages out: $3\{, \}000 \times 600 = 1\{, \}800\{, \}000$.

Waiting = $3\{, \}000\{, \}000 - 1\{, \}800\{, \}000 = 1\{, \}200\{, \}000$.

Shortcut: the gap grows at $5\{, \}000 - 3\{, \}000 = 2\{, \}000$ per second, and $2\{, \}000 \times 600 = 1\{, \}200\{, \}000$.

Answer: 1.2 million messages of lag

Example 2 [warm-up]

Same system. You add consumers until the group can handle 8,000 messages per second. The producer stays at 5,000 per second. How long until the lag reaches zero?

SOLUTION

Drain rate = out — in = $8\{, \}000 - 5\{, \}000 = 3\{, \}000$ per second.

Time = $1\{, \}200\{, \}000 / 3\{, \}000 = 400$ seconds = 6 minutes 40 seconds.

Answer: 400 s, about 6 min 40 s

Here is the same thing as a program, so you can change the numbers and check yourself.

lag-sim.js — build up the lag, then drain it

JS

```
function simulateLag({ produce, consume, seconds }) {
  let lag = 0;
  for (let t = 0; t < seconds; t++) {
    lag += produce(t) - consume(t);
    if (lag < 0) lag = 0; // cannot consume what is not there
  }
  return lag;
}

const lag10min = simulateLag({ produce: () => 5000, consume: () => 3000, seconds:
600 });
console.log('lag after 10 min at 5000 in / 3000 out =', lag10min.toLocaleString());

let drain = 0, l = lag10min;
while (l > 0) { l += 5000 - 8000; drain++; } // still 5000 arriving every second
console.log('seconds to drain once capacity is 8000/s =', drain);
console.log('check: 1,200,000 / (8000 - 5000) =', 1200000 / (8000 - 5000));
```

```
lag after 10 min at 5000 in / 3000 out = 1,200,000
seconds to drain once capacity is 8000/s = 400
check: 1,200,000 / (8000 - 5000) = 400
```

TRAP

Students write “8,000 per second, so $1\{, \}200\{, \}000/8\{, \}000 = 150$ s”. Wrong — new messages keep arriving during the drain. Always subtract the incoming rate first. The `l += 5000 - 8000` line in the program is exactly that subtraction.

Example 3 [warm-up]

A topic has 12 partitions. A consumer group has 20 processes. How many are actually working? What if the group has 5 processes?

SOLUTION

A partition is owned by exactly one member of a group.

20 processes: 12 get one partition each. The other 8 are **idle** — they hold no partition and read nothing. You paid for 8 machines that do nothing.

5 processes: 12 partitions shared by 5. Split is $12 = 3 + 3 + 2 + 2 + 2$. Every process works, but the two holding 3 partitions carry 50% more load than those holding 2.

Answer: With 20: only 12 work, 8 idle. With 5: all 5 work, unevenly (3,3,2,2,2).

SHORTCUT

Pick a partition count that is **divisible by** the consumer counts you expect to run: 12 splits evenly among 1, 2, 3, 4, 6, 12 workers. 64 and 128 are popular for the same reason. Choosing a prime like 13 is a small, silly, permanent tax.

Example 4 [warm-up]

Retry with exponential backoff: first retry after 1 s, then double each time, 6 retries total. What is the total wait before the message gives up?

SOLUTION

Waits: 1, 2, 4, 8, 16, 32 seconds.

Sum = $1 + 2 + 4 + 8 + 16 + 32 = 63$ seconds.

General form: with base b and n retries, total = $b(2^n - 1)$. Check: $1 \times (2^6 - 1) = 63$. Matches.

Answer: 63 seconds

Example 5 [warm-up]

Why do we add **jitter** (a random amount) to the backoff?

SOLUTION

Suppose a database goes down and 10,000 in-flight messages all fail at time t .

Without jitter, all 10,000 retry at $t + 1$, then all at $t + 3$, then all at $t + 7$. The database gets hit by 10,000 requests in the same millisecond, falls over again, and the cycle repeats forever. This is a **thundering herd**.

With jitter, each retry happens at a random point in $[0, 2^k]$ seconds. The same 10,000 retries now spread over a window, and the database sees a smooth load it can survive.

Full jitter in one line:

```
const sleepMs = Math.floor(Math.random() * base * 2 ** k);
```

Answer: To spread the retries out so the recovering service is not killed again

Example 6 [warm-up]

A consumer stores a dedup key for every message it has processed, so it can skip repeats. Rate is 5,000 messages per second. Each key costs 100 bytes in Redis (the key string plus overhead). How much memory does a 24-hour dedup window need? Is that sensible?

SOLUTION

Keys per day = $5,000 \times 86,400 = 432,000$.

Memory = $432,000 \times 100 \text{ B} = 43,200,000 \text{ B} = 43.2 \text{ GB}$.

That is a big, expensive Redis cluster whose only job is remembering the past.

Now try a **1-hour** window: $5,000 \times 3,600 = 18,000$ keys; $18,000 \times 100 = 1,800,000 \text{ B} = 1.8 \text{ GB}$. That fits on one node.

Decision: 1-hour window, and make the **database write itself** idempotent (a unique constraint) as the real safety net. The cache is an optimisation, not the guarantee.

Answer: 43.2 GB for 24 h — too much. Use a 1-hour window (1.8 GB) plus a unique key in the DB.

Warm-up drill: name the semantic

What the consumer does	Semantic	Why
ack() then work()	at most once	crash after ack loses the work
work() then ack()	at least once	crash before ack replays the work
work() then ack(), where work is an insert with a unique key	exactly once effect	the replay inserts nothing the second time
work() then ack(), where work is balance += 10	at least once, and wrong	the replay adds 10 again
work() then ack(), where work is balance = 250	at least once, and safe	setting the same value twice is harmless

SHORTCUT

Idempotent means: doing it twice gives the same result as doing it once. $x = 5$ is idempotent. $x += 5$ is not. When you can, turn increments into “write the computed absolute value” or “insert a row with a unique key”.

Tier 1 — LLD: build the job queue yourself

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

The Tier-1 interviewer does not want Kafka. They want classes. The question sounds like: “Design a background job system. I should be able to submit a job, retry it if it fails, and move it to a dead letter queue after 3 failures.”

Step 1 — the objects on the table

Read the sentence and underline the nouns. Each noun that has **state** becomes a class.

Noun	Class	What it owns
job	Job	id, payload, attempt count, state, run-after time

queue	InMemoryJobQueue	push, poll, ack, nack
retry rule	ExponentialBackoff / NoRetry	should we retry? how long to wait?
worker	Worker	a loop: poll, run, ack or nack
handler	a function per job type	the actual business code
dead letters	DeadLetterStore	jobs that gave up

TRAP

The number one Tier-1 mistake: putting the retry maths **inside** Worker as an if ladder. Then every new rule (fixed delay, exponential, retry only on network errors) edits Worker. That breaks the Open–Closed Principle. Make the retry policy a separate object with a fixed shape and pass it in — the Strategy pattern. This single move is worth several marks.

Step 2 — the code, in JavaScript

JavaScript has no interface keyword. The interviewer still wants to hear the word. Say: “In Java this would be an interface RetryPolicy. In JS I express the same contract as a duck-typed object with `shouldRetry(job, cause)` and `delayMs(attempt)`, and any object with those two methods can be passed in.” That answer scores in both rounds.

job.js — the message and its state

JS

```
const JobState = Object.freeze({
  READY: 'READY', RUNNING: 'RUNNING', DONE: 'DONE', DEAD: 'DEAD',
});

class Job {
  constructor(id, type, payload, priority = 1) {
    this.id = id; // unique: this is what dedup keys on
    this.type = type; // 'send_email', 'resize_image'
    this.payload = payload;
    this.priority = priority; // 0 high, 1 normal, 2 low
    this.attempts = 0; // how many times we have tried
    this.runAfter = Date.now(); // do not run before this clock time
    this.state = JobState.READY;
  }

  recordFailure(delayMs) {
    this.attempts += 1;
    this.state = JobState.READY;
    this.runAfter = Date.now() + delayMs;
  }
}
```

retry.js — Strategy pattern, so a new rule is a new class

JS

```
class PermanentError extends Error {} // bad input: retrying cannot help

class ExponentialBackoff {
  constructor(baseMs, maxAttempts, rng = Math.random) {
    this.baseMs = baseMs;
    this.maxAttempts = maxAttempts;
    this.rng = rng; // injected so tests are deterministic
  }
}
```

```

shouldRetry(job, cause) {
  if (cause instanceof PermanentError) return false;
  return job.attempts < this.maxAttempts;
}

delayMs(attemptNumber) {
  // attemptNumber starts at 1
  const ceiling = this.baseMs * 2 ** (attemptNumber - 1); // base * 2^(k-1)
  return Math.floor(this.rng() * ceiling); // full jitter
}

class NoRetry {
  shouldRetry() { return false; }
  delayMs() { return 0; }
}

```

$2 ** (\text{attemptNumber} - 1)$ is 2^{k-1} . For $k = 1, 2, 3, 4$ it gives 1, 2, 4, 8. In JS you can safely go to 2^{53} with a normal number, so unlike Java there is no 1L to remember. Past 2^{53} you would need BigInt — but a backoff that long is a bug anyway.

queue.js — the heap comes from the shared toolkit

JS

```

const { MinHeap } = require('../shared/js/toolkit'); // JS has no built-in heap

class InMemoryJobQueue {
  constructor(policy, dlq) {
    // ordered by runAfter ascending, so delayed jobs sort to the back
    this.ready = new MinHeap((x, y) => x.runAfter - y.runAfter);
    this.policy = policy;
    this.dlq = dlq;
    this.seen = new Set();
  }

  push(job) {
    if (this.seen.has(job.id)) return false; // dedup on the producer side
    this.seen.add(job.id);
    this.ready.push(job);
    return true;
  }

  poll(now = Date.now()) {
    const head = this.ready.peek();
    if (!head) return null;
    if (head.runAfter > now) return null; // the earliest job is not due yet
    const job = this.ready.pop();
    job.state = JobState.RUNNING;
    return job;
  }

  ack(job) { job.state = JobState.DONE; }

  nack(job, cause) {
    if (!this.policy.shouldRetry(job, cause)) {
      job.state = JobState.DEAD;
      this.dlq.store(job, cause);
    }
  }
}

```

```

    return;
  }
  const delay = this.policy.delayMs(job.attempts + 1);
  job.recordFailure(delay);
  this.ready.push(job);
}

msUntilNextDue(now = Date.now()) {
  const head = this.ready.peek();
  if (!head) return Infinity;
  return Math.max(0, head.runAfter - now);
}
}

class DeadLetterStore {
  constructor() { this.dead = []; }
  store(job, cause) { this.dead.push({ job, reason: String(cause &&
cause.message) }); }
  get size() { return this.dead.length; }
}

```

TRAP

JS trap 1 — `sort()` is lexicographic. If you had used a plain array and written `this.ready.sort()`, then run-after times `[10, 9, 1]` sort to `[1, 10, 9]`, because `sort()` compares them as **text**. Numbers always need a comparator: `(a, b) => a - b`.

JS trap 2 — there is no built-in priority queue. Java has `PriorityQueue`, C++ has `priority_queue`, JS has nothing. Use the `MinHeap` from the shared toolkit. Re-sorting an array on every push is $O(n \log n)$ per insert instead of $O(\log n)$ — 100 times slower at $n = 10^5$.

worker.js — the loop. Look hard at the last five lines.

JS

```

class Worker {
  constructor(queue, handlers) {
    this.queue = queue;
    this.handlers = handlers;    // Map<string, async (job) => void>
    this.running = true;
  }

  stop() { this.running = false; }

  async run() {
    while (this.running) {
      const job = this.queue.poll();
      if (!job) {
        const wait = Math.min(50, this.queue.msUntilNextDue());
        await new Promise(r => setTimeout(r, wait));
        continue;
      }
      const handler = this.handlers.get(job.type);
      if (!handler) {
        this.queue.nack(job, new PermanentError(`no handler: ${job.type}`));
        continue;
      }
      try {

```

```

    await handler(job);      // do the work FIRST
    this.queue.ack(job);     // then ack -> at least once
  } catch (err) {
    this.queue.nack(job, err);
  }
}
}
}
}

```

The test that proves all four paths work

JS

```

const dlq = new DeadLetterStore();
const policy = new ExponentialBackoff(10, 3, () => 0.999); // fixed rng, tiny delays
const q = new InMemoryJobQueue(policy, dlq);

const delivered = [];
let failCount = 0;
const handlers = new Map([
  ['send_email', async (job) => { delivered.push(job.id); }],
  ['flaky',      async ()      => { failCount++; throw new Error('smtp timeout'); }],
  ['bad_input',  async ()      => { throw new PermanentError('email missing @'); }],
]);

q.push(new Job('j1', 'send_email', '{}'));
q.push(new Job('j1', 'send_email', '{}')); // duplicate id -> ignored
q.push(new Job('j2', 'flaky', '{}'));
q.push(new Job('j3', 'bad_input', '{}'));
q.push(new Job('j4', 'unknown_type', '{}'));

const w = new Worker(q, handlers);
const loop = w.run();
await new Promise(r => setTimeout(r, 400));
w.stop(); await loop;

console.log('delivered      =', delivered);
console.log('flaky attempts =', failCount);
console.log('dlq size       =', dlq.size);
console.log('dlq reasons    =', dlq.dead.map(d => `${d.job.id}:${d.reason}`));

```

Real output

TEXT

```

delivered      = [ 'j1' ]
flaky attempts = 4
dlq size       = 3
dlq reasons    = [
  'j4:no handler: unknown_type',
  'j3:email missing @',
  'j2:smtp timeout'
]

```

Read the output line by line. delivered has **one** j1, not two — the seen set rejected the duplicate push. flaky attempts = 4 is one first try plus three retries, which is exactly maxAttempts = 3. bad_input went straight to the DLQ after **one** try, because PermanentError short-circuits shouldRetry. The unknown type also went to the DLQ instead of crashing the worker.

Example 7 [TCS NQT · pattern]

Point at the exact two lines in `Worker.run()` that decide the delivery semantic. What would you change to make it **at most once**, and why would that be a bad idea for e-mail?

SOLUTION

The two lines are:

```
await handler(job);    // work
this.queue.ack(job);   // ack
```

Work happens first, ack second. If the process dies between them, the job is still in the queue and will run again. That is **at least once**.

To get **at most once**, swap them:

```
this.queue.ack(job);
await handler(job);
```

Now a crash after ack loses the job forever.

For e-mail, losing an order confirmation is worse than sending it twice. A customer who gets two identical e-mails is mildly annoyed. A customer who gets none opens a support ticket and may believe the order failed.

Decision: keep at-least-once (work then ack), and make the handler idempotent by recording `sent_email(job_id)` in a table with a unique key before sending.

Example 8 [Infosys · pattern]

In Java this queue would need `synchronized` on `poll()`. Node is single-threaded — so is the JS version automatically safe? Prove your answer with code.

SOLUTION

Half safe, and the half that is not safe is the dangerous half.

A JS function that contains **no** `await` runs start to finish without interruption. Our `poll()` has no `await`, so two callers can never interleave inside it. No lock needed.

But the moment you add an `await` in the middle of a look-then-take sequence, another task runs in the gap. Here is the bug, made to happen on purpose:

The await in the middle is the whole bug

JS

```
class BrokenQueue {
  constructor(items) { this.items = items; }
  async pollAndLog() {
    const head = this.items[0];           // 1. LOOK
    if (!head) return null;
    await new Promise(r => setTimeout(r, 0)); // 2. YIELD — another task runs
    here
    return this.items.shift();           // 3. TAKE — may not be `head` any
    more
  }
}

const q = new BrokenQueue(['A', 'B']);
const [x, y] = await Promise.all([q.pollAndLog(), q.pollAndLog()]);
console.log('two concurrent polls got:', x, y, '| remaining =', q.items);

const q2 = new BrokenQueue(['A']);
const [p, r] = await Promise.all([q2.pollAndLog(), q2.pollAndLog()]);
console.log('one item, two pollers :', p, r, '| remaining =', q2.items);
```

Real output

TEXT


```
two concurrent polls got: A B | remaining = []
one item, two pollers : A undefined | remaining = []
```

Look at the second line. There was **one** item. Both pollers saw `head = 'A'` at step 1. Both then called `shift()`. One got 'A', the other got undefined. In a real queue that undefined becomes a `TypeError` inside the worker, and the worker dies.

The rule to say out loud: “In Node the unit of atomicity is the stretch of code between two `await`s. Keep check-then-act in one such stretch, or protect it with an explicit `async mutex`.”

Answer: Safe only because `poll()` has no `await`. Add one `await` between the check and the take and you get the same race Java needs synchronized for.

Example 9 [Accenture · pattern]

The seen set in `InMemoryJobQueue` grows forever. In a 30-day run at 200 jobs per second, how many ids does it hold, and roughly how much heap is that if a string id plus Set overhead is 120 bytes? Fix the design.

SOLUTION

$\text{Ids} = 200 \times 86\{, \}400 \times 30$.

Step 1: $200 \times 86\{, \}400 = 17\{, \}280\{, \}000$ per day.

Step 2: $17\{, \}280\{, \}000 \times 30 = 518\{, \}400\{, \}000$ ids.

Heap = $518\{, \}400\{, \}000 \times 120 \text{ B} = 62\{, \}208\{, \}000\{, \}000 \text{ B} \approx 62.2 \text{ GB}$.

Node’s default old-space heap is about 4 GB. The process dies long before 62 GB. This is an **unbounded memory leak**, and interviewers look for it.

Fix — three options, then a decision.

1. **Bounded LRU set** holding the last N ids, say $N = 2$ million ($\approx 240 \text{ MB}$). Duplicates older than the window slip through.
2. **Time-windowed set**: drop ids older than 1 hour. That is $200 \times 3\{, \}600 = 720\{, \}000$ ids $\approx 86 \text{ MB}$.
3. **Push the dedup to the store that matters**: a `UNIQUE` index on `job_id` in the jobs table. Costs one index lookup per insert, never leaks, and survives a restart.

Decision: option 3 as the correctness guarantee, option 2 as a cheap front filter so we do not hit the database for obvious repeats. Worker memory is now capped at about 86 MB and correctness no longer depends on worker memory at all.

The 1-hour windowed set, ~86 MB instead of 62 GB

JS

```
class WindowedSet {
  constructor(windowMs) { this.windowMs = windowMs; this.m = new Map(); }
  has(id) { this.#evict(); return this.m.has(id); }
  add(id) { this.#evict(); this.m.set(id, Date.now()); }
  #evict() {
    const cutoff = Date.now() - this.windowMs;
    // Map keeps INSERTION order, so the oldest entries are at the front.
    for (const [id, t] of this.m) { if (t >= cutoff) break; this.m.delete(id); }
  }
  get size() { return this.m.size; }
}
```

That eviction loop is only correct because `Map` iterates in insertion order and every `add` uses the current time, so times are non-decreasing. A plain object gives no such guarantee — this is **JS trap 6**, `Map` versus object, and here it is load-bearing.

Answer: 518.4 million ids, about 62 GB. Cap it with a 1-hour window and rely on a `UNIQUE` index in the database.

Example 10 [Capgemini · pattern]

Add **priority** to this design. High-priority jobs (password-reset e-mails) must be picked before low-priority ones (weekly digest), but a flood of high-priority jobs must not starve low ones for more than about 60 seconds. Write the comparator and prove it works.

SOLUTION

Naive idea: sort by (priority, runAfter). This starves low-priority jobs completely. If high-priority jobs never stop arriving, a low job never reaches the head. Forever.

Fix: ageing. Give every job an **effective time** that slides earlier the longer it waits.

Comparator with ageing — bounded starvation

JS

```
const BOOST_MS = [60_000, 20_000, 0]; // index = priority: 0 high, 1 normal, 2 low

const effectiveTime = (j) => j.runAfter - BOOST_MS[j.priority];
const byUrgency = (a, b) => effectiveTime(a) - effectiveTime(b);

const now = 1_000_000;
const jobs = [
  { id: 'low-waited-61s', priority: 2, runAfter: now - 61_000 },
  { id: 'high-just-now', priority: 0, runAfter: now },
  { id: 'normal-waited-5s', priority: 1, runAfter: now - 5_000 },
  { id: 'low-just-now', priority: 2, runAfter: now },
];
console.log('order =', [...jobs].sort(byUrgency).map(j => j.id));
console.log('effective times =',
  jobs.map(j => `${j.id}:${effectiveTime(j) - now}`).join(' '));
```

Real output

TEXT

```
order = [ 'low-waited-61s', 'high-just-now', 'normal-waited-5s', 'low-just-now' ]
effective times = low-waited-61s:-61000 high-just-now:-60000
                  normal-waited-5s:-25000 low-just-now:0
```

Read the proof in the numbers. A brand-new high job has effective time $-60\{, \}000$. A low job that has waited 61 s has effective time $-61\{, \}000$, which is **smaller**, so it sorts first and finally runs. A low job that just arrived is at 0 and waits, correctly.

Worst-case extra wait for a low job is therefore about 60 seconds — exactly what was asked for.

Trade-off named and decided. Strict priority gives the lowest possible latency for urgent jobs but allows infinite starvation. Ageing adds at most 60 s of delay to an urgent job in the worst case, but bounds the low-priority wait.

We pick ageing, because a weekly digest that never sends is a bug, while a password e-mail arriving in 60 s instead of 0 s is still acceptable.

The LLD scorecard

Principle	Where it shows up above
Single Responsibility	the retry policy decides waiting; Worker only loops; DeadLetterStore only stores
Open–Closed	a new retry rule is a new class, not an edit to Worker
Liskov	any object with push/poll/ack/nack can replace InMemoryJobQueue — Worker never inspects its type
Interface Segregation	a handler is one function; handlers are not forced to implement retry() or serialize()

Dependency Inversion	Worker receives the queue and the handler map in its constructor, so tests inject fakes — that is why the test above runs with no broker
Strategy pattern	ExponentialBackoff vs NoRetry, swapped at construction
Registry over switch	Map<string, handler> instead of switch (job.type)

PRACTICE · TIER 1 target: 25 min

1. Add a `visibilityTimeout` to `InMemoryJobQueue`: if a worker polls a job and does not ack within 30 seconds, the job becomes pollable again. Which field and which methods change?
2. Rewrite `poll()` so that it **does** contain an `await` (say, a database read). Show the race that appears, and write the four-line async mutex that fixes it.
3. Write a retry policy that retries only on a network error, at most 5 times, with a fixed 2-second delay.
4. `DeadLetterStore` currently just stores. Add `replay(jobId, queue)`. What must you reset on the `Job` before pushing it back?

ANSWER KEY

1. Add `leaseUntil` to `Job`. `poll()` sets `job.leaseUntil = now + 30_000` and keeps the job in a separate `inFlight` map; a sweeper moves anything whose `leaseUntil` has passed back into `ready`. `ack()` deletes it from `inFlight`.
2. The race is the one in Example 8: two pollers read the same head across the `await`. Fix: `this.lock = this.lock.then(() => this.#pollInner()); return this.lock;` — a promise chain is a one-slot mutex, because each call waits for the previous chain link.
3. `shouldRetry(job, cause)` returns `cause.code === 'ECONNRESET' || cause.code === 'ETIMEDOUT'` combined with `job.attempts < 5`; `delayMs()` returns 2000 always.
4. Reset `attempts` to 0, set `state` back to `READY`, and set `runAfter` to `Date.now()`. Otherwise the policy immediately declares it dead again and it bounces straight back into the DLQ.

Tier 2 — mid-scale: a notification service

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

The question. “Our shopping app needs to send order updates to users by e-mail and push notification. Design it.” We now walk the seven steps. Do not skip one, even when you are short of time — say the step name out loud and give it one sentence.

Step 1 — Clarify

Ask this	Suppose the answer is	What it changes
How many users, how many notes each per day?	40 M DAU, 3 per user per day	fixes every number below
Which channels?	e-mail and push now; SMS later	forces a channel interface
How fast must a note arrive?	under 60 s for order updates	rules out a batch job; keeps a queue
Is duplicate delivery acceptable?	no — users complain	forces an idempotency key
Do users have quiet hours or preferences?	yes, per channel	adds a preference lookup on the path
Can we drop marketing notes under load?	yes	gives us a priority lane
Must we show delivery status?	yes, 30-day history	adds a status store, adds storage

SHORTCUT

In Tier 2 the marks are in the **why**, not the **what**. “How many users” is worth nothing. “How many users, because that decides whether this is one queue or thirty-two partitions” is worth the mark.

Step 2 — Scale estimate (show every division)

Traffic.

$40\{\},\}000\{\},\}000 \text{ DAU} \times 3 \text{ notes} = 120\{\},\}000\{\},\}000 \text{ notes per day.}$

Average rate = $120\{\},\}000\{\},\}000 / 86\{\},\}400 = 1\{\},\}388.9 \approx 1\{\},\}390 \text{ notes/s.}$

Peak is spiky — most orders happen in the evening. Take $4 \times \text{average}$: $1\{\},\}390 \times 4 = 5\{\},\}560 \text{ notes/s at peak.}$

Storage of delivery status. One status row is about 250 bytes (id, user, channel, template, state, timestamps).

Per day: $120\{\},\}000\{\},\}000 \times 250 \text{ B} = 30\{\},\}000\{\},\}000 \text{ B} = 30 \text{ GB/day.}$

30-day history: $30 \times 30 = 900 \text{ GB.}$ One well-sized database. No sharding needed yet.

Queue depth if all workers die for 10 minutes.

Messages piled up = $1\{\},\}390 \times 600 = 834\{\},\}000.$

At 400 bytes per message: $834\{\},\}000 \times 400 \text{ B} = 333\{\},\}600\{\},\}000 \text{ B} \approx 334 \text{ MB.}$ A managed queue holds that without noticing. Good — a 10-minute outage is survivable.

How many worker processes?

An SMTP call to the e-mail provider takes about 200 ms. One **concurrent slot** therefore does $1/0.2 = 5$ sends per second.

Slots needed at peak = $5\{\},\}560 / 5 = 1\{\},\}112 \text{ concurrent sends.}$

A Node process handles network-waiting work with Promise concurrency, not threads. At 200 in-flight sends per process (they are almost all just waiting on a socket, so the CPU is nearly idle):

$1\{\},\}112 / 200 = 5.56 \rightarrow 6 \text{ processes.}$

Add one for headroom and one so a rolling deploy never drops below 6: **8 processes.**

THE ESTIMATE IN ONE BLOCK — COPY THIS SHAPE

- $40 \text{ M DAU} \times 3 = 120 \text{ M notes/day}$
- $120 \text{ M} / 86,400 = 1,390/\text{s average}$
- $\times 4 \text{ peak} = 5,560/\text{s}$
- $250 \text{ B status row} \times 120 \text{ M} = 30 \text{ GB/day, } \times 30 \text{ days} = 900 \text{ GB}$
- $10\text{-min outage backlog} = 1,390 \times 600 = 834 \text{ K msgs} = 334 \text{ MB}$
- $200 \text{ ms per send} \rightarrow 5 \text{ sends/s per slot} \rightarrow 5,560 / 5 = 1,112 \text{ slots}$
- $1,112 / 200 \text{ slots per process} = 5.56 \rightarrow 6, \text{ round to } 8 \text{ for headroom}$

Example 11 [Shopee · pattern]

Marketing wants a campaign: one push to all 40 M users, to go out “as fast as possible”. Your peak capacity is 5,560 sends/s. How long does the campaign take, and what breaks?

SOLUTION

Time = $40\{\},\}000\{\},\}000 / 5\{\},\}560 = 7\{\},\}194 \text{ seconds.}$

$7\{\},\}194 / 60 = 119.9 \text{ minutes} \approx 2 \text{ hours.}$

What breaks: for those 2 hours the workers are 100% busy on marketing. Order-update notes, which promised under 60 s, queue behind the campaign — classic head-of-line blocking. A user who cancels an order gets the confirmation an hour later.

Fix, with both sides named.

- **Option A — one queue with a priority field.** Cheap, but a single slow marketing batch still holds the connection pool; a priority field inside one queue does not stop resource starvation.
- **Option B — two separate queues and two separate worker pools,** split transactional : marketing = 70 : 30. Transactional latency is protected by hard isolation. Costs a second pool, and 30% of capacity sits idle when no campaign runs.

Decision: Option B. The 60-second promise is a product guarantee; idle capacity is only money. With 30% of 5,560 = 1{,}668 sends/s for marketing, the campaign takes $40{,}000/1{,}668 = 23{,}981 \text{ s} \approx 6.7$ hours — and we tell marketing to schedule it overnight, which is what they wanted anyway.

Answer: About 2 hours on a shared pool, and it starves order updates. Split the pools 70/30; the campaign then takes 6.7 h overnight.

Step 3 — API surface

Public API — note the idempotency header

HTTP

```
POST /v1/notifications
Idempotency-Key: ord-99213-shipped-v1
Content-Type: application/json

{
  "user_id": "u_4412",
  "template": "order_shipped",
  "channels": ["push", "email"],
  "priority": "transactional",
  "params": { "order_id": "99213", "eta": "2026-03-04" }
}

202 Accepted
{ "notification_id": "n_8f31c2", "state": "queued" }
```

Read the status, and manage preferences

HTTP

```
GET /v1/notifications/n_8f31c2
200 OK
{
  "notification_id": "n_8f31c2",
  "user_id": "u_4412",
  "attempts": [
    { "channel": "push", "state": "delivered", "at": "2026-03-03T10:02:11Z" },
    { "channel": "email", "state": "bounced", "at": "2026-03-03T10:02:14Z",
      "reason": "mailbox_full" }
  ]
}

PUT /v1/users/u_4412/preferences
{ "email": { "marketing": false, "transactional": true },
  "push": { "quiet_hours": ["22:00", "07:00"], "timezone": "Asia/Kolkata" } }
```

202 Accepted, not 200 OK. We have accepted the job, not delivered the message. Using 200 here is a small lie that confuses every caller later. Interviewers notice.

Step 4 — Data model

notifications — the status store

SQL

```

CREATE TABLE notifications (
  id          CHAR(16)          PRIMARY KEY,
  idempotency_key VARCHAR(128) NOT NULL,
  user_id     VARCHAR(32)       NOT NULL,
  template    VARCHAR(64)       NOT NULL,
  priority     TINYINT          NOT NULL,      -- 0 transactional, 1 marketing
  params_json  JSON             NOT NULL,
  created_at   DATETIME(3)      NOT NULL,
  UNIQUE KEY uk_idem (idempotency_key),      -- this line IS the dedup guarantee
  KEY idx_user_time (user_id, created_at)
);

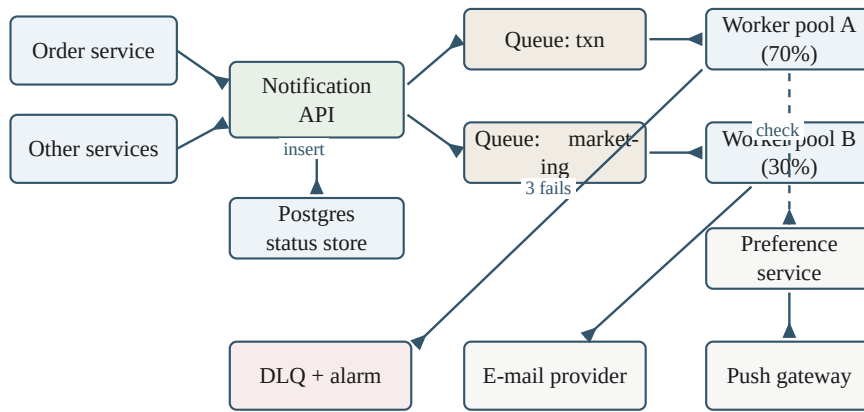
CREATE TABLE delivery_attempts (
  id          BIGINT          AUTO_INCREMENT PRIMARY KEY,
  notification_id CHAR(16)    NOT NULL,
  channel      VARCHAR(16)    NOT NULL,      -- 'email' | 'push' | 'sms'
  attempt_no   SMALLINT       NOT NULL,
  state        VARCHAR(16)    NOT NULL,      -- queued|sent|delivered|bounced|dead
  provider_id  VARCHAR(64),    -- id returned by the e-mail provider
  reason       VARCHAR(128),
  updated_at   DATETIME(3)    NOT NULL,
  UNIQUE KEY uk_attempt (notification_id, channel, attempt_no),
  KEY idx_state_time (state, updated_at)
);

```

Why each key was chosen.

Choice	Reason
UNIQUE (idempotency_key)	a retried POST hits the unique index and we return the existing id — dedup that survives a restart, unlike a Redis set
attempts in their own table	one notification can have 2 channels x 4 tries = 8 rows; keeping them in one row would need arrays and a rewrite of the row on every try
(state, updated_at) index	the sweeper query “find everything stuck in sent for over 10 minutes” must not scan the table
(user_id, created_at) index	the notification-history screen
monthly partition on created_at	dropping a partition is instant; a DELETE WHERE created_at < ... over 30 GB/day is a nightmare

Step 5 — Architecture



Two queues, two pools: a marketing flood can never delay an order update, because they do not share a single connection pool.

Notification service. The API never talks to a provider directly.

Step 6 — Deep dive: making duplicates impossible

The hard part is **not** the queue. It is this: the order service is also at-least-once, so it may call `POST /v1/notifications` twice for the same event. Then our own queue is at-least-once, so the worker may run twice. Two independent sources of doubles.

The dedup ladder — four gates, cheapest first.

Gate	Where	Catches	Cost
1. Redis SET NX on the idempotency key, 1-hour TTL	API	the fast retry storm	0.3 ms, and it may miss
2. UNIQUE (idempotency_key)	Postgres	every duplicate POST, forever	one index write
3. UNIQUE (notification_id, channel, attempt_no)	Postgres	a worker replaying the same attempt	one index write
4. Provider-side key	e-mail provider	our retry after their timeout	free, if the provider supports it

Gates 2 and 3 are the **guarantees**. Gate 1 is only a cost saver. Never present a cache as a correctness guarantee — that is the trap.

create-notification.js — dedup done right

JS

```

class DuplicateKeyError extends Error {}

async function createNotification(req, idemKey) {
  // Gate 1: cheap filter. A miss is fine; a hit saves a database round trip.
  const cached = await redis.get(`idem:${idemKey}`);
  if (cached) return { status: 202, notification_id: cached, deduped: 'cache' };

  const id = nextId();
  try {
    await db.insertNotification(id, idemKey, req); // Gate 2: UNIQUE
  } catch (err) {
    if (!(err instanceof DuplicateKeyError)) throw err;
  }
}
  
```

```

    const existing = await db.findByIdByIdemKey(idemKey); // someone beat us to it
    await redis.setex(`idem:${idemKey}`, 3600, existing);
    return { status: 202, notification_id: existing, deduped: 'db' }; // SAME id
  }
  await redis.setex(`idem:${idemKey}`, 3600, id);
  await queue.publish(req.priority, { id, ...req });
  return { status: 202, notification_id: id, deduped: null };
}

```

The test: same key four times, and three callers racing

JS

```

const req = { user_id: 'u_4412', template: 'order_shipped', priority:
'transactional' };
console.log(await createNotification(req, 'ord-99213-shipped-v1')); // fresh
redis.m.clear(); // cache wiped
console.log(await createNotification(req, 'ord-99213-shipped-v1')); // caught by
the DB
console.log(await createNotification(req, 'ord-99213-shipped-v1')); // caught by
cache
console.log('messages actually published =', queue.published.length);

redis.m.clear(); db.rows.clear(); queue.published.length = 0;
const out = await Promise.all([1,2,3].map(() => createNotification(req, 'race-
key')));
console.log('race ids =', out.map(o => o.notification_id),
'| published =', queue.published.length);

```

Real output

TEXT

```

{ status: 202, notification_id: 'n_0001', deduped: null }
{ status: 202, notification_id: 'n_0001', deduped: 'db' }
{ status: 202, notification_id: 'n_0001', deduped: 'cache' }
messages actually published = 1
race ids = [ 'n_0003', 'n_0003', 'n_0003' ] | published = 1

```

Three callers raced with an empty cache. All three got the **same** id, and exactly **one** message was published. That is what “idempotent endpoint” means, and now you have seen it rather than been told it.

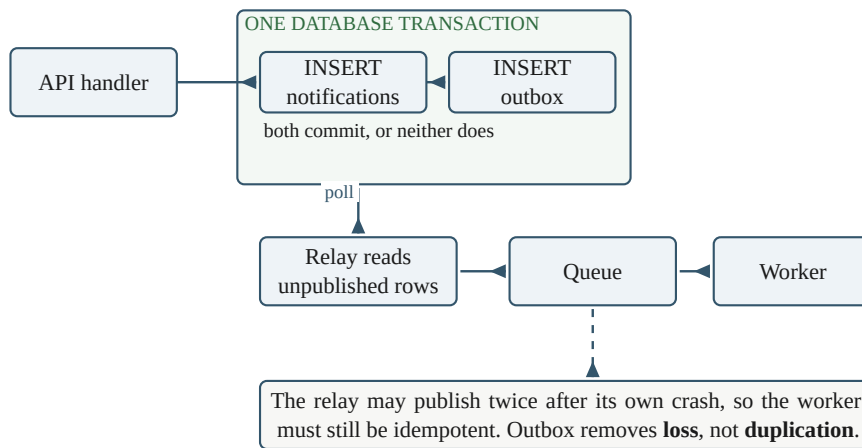
TRAP

Look closely at the order inside the happy path: **insert into the database, then publish to the queue**. If the process dies between those two awaits, the row exists but nothing was queued and the note is never sent. Silent loss.

Publishing first is worse: a worker may read the message before the row is committed and find nothing.

The clean fix is the **transactional outbox**, next. For Tier 2 an acceptable answer is: “insert the row with state = 'queued', publish, and run a sweeper every 30 seconds that re-publishes anything still queued and older than 60 seconds.” That turns silent loss into a 60-second delay, which we can live with.

The transactional outbox, drawn



Outbox: one transaction writes both the row and the message, so they cannot disagree.

The outbox table and the relay query

SQL

```

CREATE TABLE outbox (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  topic       VARCHAR(64) NOT NULL,
  msg_key     VARCHAR(64) NOT NULL,
  payload     JSON        NOT NULL,
  created_at  DATETIME(3) NOT NULL,
  published   BOOLEAN     NOT NULL DEFAULT FALSE,
  KEY idx_unpublished (published, id)
);

-- the relay, every 200 ms:
SELECT id, topic, msg_key, payload
  FROM outbox
 WHERE published = FALSE
 ORDER BY id
 LIMIT 500;
-- ... publish each one to the broker ...
UPDATE outbox SET published = TRUE WHERE id IN (...);

-- and a nightly job:
DELETE FROM outbox WHERE published = TRUE AND created_at < NOW() - INTERVAL 2 DAY;
  
```

Example 12 [Grab · pattern]

The relay publishes a batch of 500, then crashes before the UPDATE. What happens on restart? Give the number of duplicate messages, and say why the system is still correct.

SOLUTION

On restart the relay runs the same SELECT. Those 500 rows still have `published = FALSE`, so it publishes all 500 **again**.

Duplicates = 500 messages, in the worst case, per relay crash.

The system is still correct because the worker is idempotent: each duplicate carries the same `notification_id`, and gate 3 — UNIQUE (`notification_id`, `channel`, `attempt_no`) — rejects the second write. The user receives one e-mail.

The cost, stated honestly: 500 extra broker writes and 500 extra unique-index violations. At a crash rate of once a week, that is noise. If the relay crashed every minute we would shrink the batch to 50 and update published in smaller chunks, trading throughput for a smaller duplicate blast radius.

Answer: 500 duplicate publishes; harmless, because the unique key on (notification_id, channel, attempt_no) absorbs them.

Step 7 — Trade-offs, failure modes, and 10x

Choice	Side A	Side B	Decision and why
Managed queue vs self-run Kafka	Managed: no ops, priced per message	Kafka: cheap at volume, gives replay, heavy ops	Managed. At 1,390/s the bill is small and we have no streaming team. Revisit above 20,000/s.
One queue with priority vs two queues	One: simpler, one pool	Two: hard isolation, some idle capacity	Two. The 60-second promise must not depend on marketing behaving.
Postgres vs a wide-column store for status	Postgres: joins, transactions, familiar	Cassandra: cheaper at 30 GB/day	Postgres with monthly partitions. 900 GB fits comfortably on one box; do not buy a distributed database for 900 GB.
Retry by sleeping in the worker vs a delay queue	Sleep: simple, but the concurrency slot is held for up to 32 s	Delay queue: the slot is freed immediately	Delay queue. With only 1,112 slots in total, a held slot is real lost capacity.
Sync provider call vs webhook callback	Sync: we know the result now, but 200 ms per send	Webhook: the provider calls us back, slot freed in 20 ms	Webhook for e-mail (they support it), sync for push (the gateway answers in 15 ms anyway).

Failure modes and the response.

Failure	Symptom	Response
E-mail provider down	every send throws, retries pile up	circuit breaker opens after 50 failures in 10 s; stop calling; keep the queue; fail over to the secondary provider
Poison message	one worker crashes repeatedly on the same id	3 failures then DLQ; alarm if DLQ depth > 100
Queue backlog grows	lag over 5 minutes of traffic	autoscale pool A on lag , not on CPU — CPU is low because the work is all waiting
Database down	status writes fail	keep sending; buffer status locally and replay. Losing a status row is far better than losing a notification.
Bad template deployed	wrong text sent to millions	canary: send a new template to 0.1% of traffic for 10 minutes before the rest

At 10x (400 M DAU, 1.2 B notes/day).

$1\{, \}200\{, \}000\{, \}000/86\{, \}400 = 13\{, \}889/\text{s}$ average; $\times 4 = 55\{, \}556/\text{s}$ peak.

Slots = $55\{, \}556/5 = 11\{, \}111$; processes = $11\{, \}111/200 = 55.6 \rightarrow 56$, call it 70 with headroom.

Status storage = $300 \text{ GB/day}, \times 30 = 9 \text{ TB}$. Now one Postgres box is finished.

At 10x we shard the status store by user_id and move the queue to a partitioned log, because at 55,556/s the per-message price of a managed queue overtakes the cost of a small streaming team. That is the honest 10x answer: the design does not survive unchanged, and saying so is a plus, not a minus.

PRACTICE · TIER 2 target: 35 min

1. A user has quiet hours 22:00–07:00. An order-shipped note arrives at 23:10. What do you do with it, and where is that decision made — API, queue, or worker? Defend the choice.
2. Recompute the process count if provider latency drops from 200 ms to 50 ms. Show the division.
3. The DLQ has 40,000 messages after a provider outage. Write the replay plan, including the rate you replay at and why.
4. Design the API for “send to a segment of 2 million users”. Should the API expand the segment, or should a worker? Give both sides, then decide.

ANSWER KEY

1. Transactional notes ignore quiet hours (the user wants to know their order shipped); marketing respects them. The decision belongs in the **worker**, not the API, because quiet hours depend on the user’s local time **at delivery**, and the message may have sat in the queue for a while. A marketing note in quiet hours is re-queued with a delay until 07:00 local.
2. 50 ms per send $\rightarrow 1/0.05 = 20$ sends/s per slot. Slots = $5\{, \}560/20 = 278$. Processes = $278/200 = 1.39 \rightarrow 2$. Keep 3 so one can fail.
3. Replay at a **capped** rate, not full speed — the provider has only just recovered. A replay pool limited to 500/s gives $40\{, \}000/500 = 80$ seconds. Also check each message’s age: an order-shipped note 6 hours late may be worse than useless, so drop it with a log line rather than send it.
4. **API expands**: one call must write 2 M rows before returning — it times out. **Worker expands**: the API writes one campaign row and returns in 5 ms; a fan-out worker reads the segment in pages of 10,000 and publishes 2 M messages. **Decision: the worker**. The API must stay a constant-time call; anything whose cost grows with the segment size belongs behind the queue.

Tier 3 — large scale: the event backbone

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

The question. “Every service in the company should be able to publish events and every other service should be able to read them. Build that. We do about 5 billion events a day. Some of them move money.”

Step 1 — Clarify

Ask	Answer we will assume	Why it matters
Events per day and average size?	5 B/day, 1 KB	sets partitions and brokers
Retention?	7 days hot, 90 days in object storage	sets disk
Ordering needed?	per entity (per account), not global	partition key = entity id; global order is impossible at this size
Who reads?	about 4 major consumer groups	sets the read bandwidth
Is any event money?	yes, <code>payment.captured</code>	forces exactly-once effect
Latency target?	p99 under 2 s end to end	rules out 30-second micro-batching
Multi-region?	two regions, active-active for reads	forces a replication story

Step 2 — Scale estimate

Events per second.

$5\{, \}000\{, \}000\{, \}000/86\{, \}400 = 57\{, \}870$ events/s average.

Peak = $3 \times 57\{, \}870 = 173\{, \}611$ events/s.

Bytes per second at peak. Using 1 KB = $1\{, \}000$ B:

$173 \times 10^6 \times 10^3 \text{ B} = 173 \times 10^9 \text{ B/s} = 173.6 \text{ MB/s}$ written.

Partition count — the two constraints.

Constraint A, **write throughput**. One partition safely takes about 10 MB/s.

$173.6/10 = 17.36 \rightarrow 18$ partitions minimum.

Constraint B, **consumer parallelism**. One consumer process handles about 2,000 events/s (it does real work, not just counting).

$173 \times 10^6 / 2 \times 10^3 = 86.8 \rightarrow 87$ consumer processes needed.

And a consumer group can never have more **working** members than partitions. So we need at least 87 partitions.

Constraint B is almost 5 times stricter than constraint A. **Take the maximum, then round up to a power of two for clean splits: 128 partitions.**

TRAP

Almost everyone computes only constraint A, says “18 partitions”, and is then asked “you have 18 partitions and need 87 consumers — now what?” and has no answer. Partition count is very hard to change later, because increasing it changes which partition a key hashes to and **breaks per-key ordering**. Compute both constraints, take the max, add room.

Here is the “hard to change later” claim, measured rather than asserted:

partitioning.js — what happens when you grow P

JS

```
function fnv1a(str) { // small stable hash, no library needed
  let h = 0x811c9dc5;
  for (let i = 0; i < str.length; i++) {
    h ^= str.charCodeAt(i);
    h = Math.imul(h, 0x01000193) >>> 0; // >>> 0 keeps it an unsigned 32-bit int
  }
  return h >>> 0;
}
const partitionOf = (key, P) => fnv1a(key) % P;

const keys = ['acct_7781', 'acct_77', 'acct_888', 'acct_101', 'acct_42'];
console.log('P=128 :', keys.map(k => `${k}->${partitionOf(k, 128)}').join(' '));
console.log('P=256 :', keys.map(k => `${k}->${partitionOf(k, 256)}').join(' '));

let moved = 0;
for (let i = 0; i < 100000; i++) {
  if (partitionOf('acct_' + i, 128) !== partitionOf('acct_' + i, 256)) moved++;
}
console.log(`keys that change partition when 128 -> 256: ` +
  `${moved}/100000 = ${(moved / 1000).toFixed(1)}%`);
```

Real output

TEXT

```
P=128 : acct_7781->90 acct_77->109 acct_888->93 acct_101->29 acct_42->73
P=256 : acct_7781->90 acct_77->237 acct_888->221 acct_101->157 acct_42->73
keys that change partition when 128 -> 256: 49891/100000 = 49.9%
```

Half the keys move. For every one of those keys, its old events sit in one log and its new events in another, and no consumer can see them in order ever again. **That** is why you get the partition count right on day one.

Storage.

Raw per day = $5 \times 10^6 \times 10^3 \times 10^3 \text{ B} = 5 \times 10^9 \times 10^3 \text{ B} = 5 \text{ TB}$.

With replication factor 3: $5 \times 3 = 15$ TB/day.

7-day hot retention: $15 \times 7 = 105$ TB.

Brokers. Give each broker 20 TB of usable disk.

By storage: $105/20 = 5.25 \rightarrow 6$ brokers.

But check the **network**, which usually binds first:

- Producers in: 173.6 MB/s.
- Replication: each message is copied to 2 followers = $173.6 \times 2 = 347.2$ MB/s.
- Total broker inbound = $173.6 + 347.2 = 520.8$ MB/s.
- Consumers out: 4 groups each read everything = $173.6 \times 4 = 694.4$ MB/s.

With 12 brokers:

In per broker = $520.8/12 = 43.4$ MB/s.

Out per broker = $694.4/12 = 57.9$ MB/s.

Total per broker = $43.4 + 57.9 = 101.3$ MB/s = $101.3 \times 8 = 810$ Mbps.

A 10 Gbps NIC is 10,000 Mbps, so we sit at about 8% of the card. That leaves room for the doubled traffic of a replica rebuild after a broker dies.

Disk per broker = $105/12 = 8.75$ TB. Comfortable.

Decision: 12 brokers, 128 partitions, replication factor 3. Storage alone says 6 brokers; we take 12 for network headroom and smaller blast radius per node.

TIER-3 ESTIMATE, ONE BLOCK

- 5 B/day / 86,400 = 57,870/s avg; x3 peak = 173,611/s
- x 1 KB = 173.6 MB/s in
- partitions: $\max(173.6 / 10 = 18, 173,611 / 2,000 = 87) = 87 \rightarrow$ round to **128**
- storage: 5 TB/day raw, x3 replicas = 15 TB/day, x7 days = **105 TB**
- brokers: $105 / 20 = 5.25$ by disk; take **12** for network headroom
- per broker: $(520.8 + 694.4) / 12 = 101.3$ MB/s = 810 Mbps on a 10 Gbps NIC

Step 3 — API surface

Publish — the envelope is fixed for the whole company

HTTP

POST /v1/events

Content-Type: application/json

```
{
  "event_id":    "evt_01HQ8ZK4F2",    // unique, generated by the producer
  "type":       "payment.captured",
  "version":    3,                    // schema version
  "key":        "acct_7781",          // partition key: ordering is per account
  "occurred_at": "2026-03-03T10:02:11.481Z",
  "producer":   "payments-api",
  "trace_id":   "4bf92f3577b34da6",
  "data": { "amount_minor": 249900, "currency": "INR", "order_id": "99213" }
}
```

202 Accepted

```
{ "partition": 41, "offset": 90210411 }
```

Consumer-side contract, as code

JS

```

subscribe({
  topic: 'payments',
  groupId: 'ledger',
  // handler MUST be idempotent: the same batch can arrive twice
  handler: async (batch) => { /* ... */ },
  autoCommit: false, // we commit ourselves, after the handler returns
  maxPollIntervalMs: 300_000, // 5 min: longer than this and the group evicts us
});

```

Fixing one envelope for the whole company (event_id, type, version, key, occurred_at, trace_id) is the single highest-value decision in this design. Without it every consumer writes its own parser, and a schema change breaks six teams silently.

Step 4 — Data model: the log itself

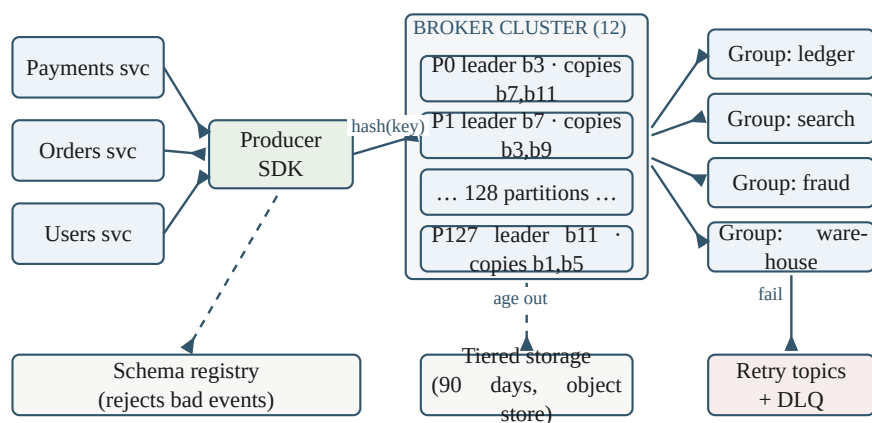
Field	Type	Note
topic	string	one per event family: payments, orders, users
partition	int 0..127	hash(key) mod 128
offset	int64	strictly increasing inside one partition ; never global
key	bytes	decides the partition; same key -> same partition -> ordered
value	bytes	the envelope above, serialized
timestamp	int64 ms	store both the broker time and the producer time
headers	map	trace_id, schema_version, retry_count

Consumer offsets live in a compacted topic of their own:

Key	Value
(group_id, topic, partition)	(offset, commit_time, member_id)

Compaction keeps only the newest value per key, so this topic stays small forever even though it is written to constantly.

Step 5 — Architecture



Each group keeps its own offset, so the warehouse can be 3 hours behind while fraud stays 200 ms behind. One log, four speeds, no copies.

One topic, 128 partitions, three copies of each, four independent consumer groups.

Step 6a — Deep dive: exactly-once for money

payment . captured must post to the ledger exactly once. Here is the chain of places a duplicate can be born, and the fix for each.

Where a duplicate is born	How	Fix
Producer	publish times out, the SDK resends	producer sequence numbers: the broker remembers (producer_id, seq) and drops a repeat
Broker	leader fails after writing but before acking; producer resends	the same sequence-number dedup, plus acks=all
Consumer	handler succeeds, process dies before the commit; the batch replays	this is the one you must solve in your own code

The consumer fix, in order of strength:

Technique	How it works	Strength	Cost
Natural idempotency	write an absolute value, not a delta	perfect when it applies	rarely applies to money
Unique key on the effect	insert into ledger_entries with UNIQUE(event_id)	perfect	one index, one row per event
Offset in the same transaction	write the business row and the consumed offset in one DB transaction	perfect	offsets now live in your DB, not the broker
Redis dedup set	SET NX event_id	best effort only	cheap, but a Redis failure silently lets a duplicate through

The pattern that actually works: a unique key on the effect

SQL

```
CREATE TABLE ledger_entries (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  event_id    CHAR(26)    NOT NULL,      -- the id from the envelope
  account_id  VARCHAR(32) NOT NULL,
  delta_minor BIGINT      NOT NULL,
  created_at  DATETIME(3) NOT NULL,
  UNIQUE KEY uk_event (event_id)         -- the whole guarantee is this one line
);

CREATE TABLE consumed_offsets (
  group_id VARCHAR(64), topic VARCHAR(64), part INT, off BIGINT,
  PRIMARY KEY (group_id, topic, part)
);
```

consumer.js — business write and offset write in ONE transaction

JS

```
function onBatch(db, batch) {
  const tx = db.begin();
  for (const rec of batch) {
    const e = rec.value;
    const n = tx.insertIgnoreLedger(e.event_id, e.data.account_id,
e.data.amount_minor);
    if (n === 0) metrics.duplicate_skipped++;    // already applied on an earlier
pass
  }
  const last = batch[batch.length - 1];
  tx.setOffset(GROUP, last.topic, last.partition, last.offset + 1);
  tx.commit();    // the business effect and the offset become durable TOGETHER
}
```

The test: run the SAME batch twice, as a crash would

JS

```
const db = new FakeDb();
const batch = [mk(100, 'evt_A', 1000), mk(101, 'evt_B', 2500), mk(102, 'evt_C',
400)];

onBatch(db, batch);
console.log('after first pass : balance =', db.balanceOf('acct_7781'),
'offset =', db.offsets.get('ledger|payments|41'));

onBatch(db, batch);    // the broker redelivers the WHOLE batch after a crash
console.log('after replay      : balance =', db.balanceOf('acct_7781'),
'offset =', db.offsets.get('ledger|payments|41'));
console.log('duplicates skipped =', metrics.duplicate_skipped);

let naive = 0;    // what a non-idempotent consumer would have done
for (const r of batch) naive += r.value.data.amount_minor;
console.log('naive "balance += amount" after two passes =', naive * 2);
```

Real output

TEXT

```
after first pass : balance = 3900 offset = 103
after replay      : balance = 3900 offset = 103
```



```
duplicates skipped = 3
naive "balance += amount" after two passes = 7800
```

The balance is 3,900 after one pass and still 3,900 after the replay. The naive version would have said 7,800. That gap — 3,900 rupees of invented money per replayed batch — is what `UNIQUE(event_id)` is buying you.

Example 13 [Amazon · pattern]

The consumer commits offsets to the **broker** every 5 seconds, in the background, while it is still processing. Show a concrete sequence where money is lost, and one where money is doubled. Then say which is worse and why.

SOLUTION

Let the batch be offsets 100..199.

Money doubled — commit happens after the work, but the process dies first.

1. $t = 0$: the consumer fetches 100..199.
2. $t = 1$: it writes 100 ledger rows (offsets 100..149 applied).
3. $t = 2$: the process is killed. The last committed offset is still 99.
4. $t = 3$: a new member takes the partition and starts at 100.
5. Offsets 100..149 are applied **a second time**. Fifty accounts are credited twice.

Money lost — the auto-commit timer fires too early.

1. $t = 0$: the consumer fetches 100..199.
2. $t = 5$: the 5-second timer fires and commits offset 200, because the **fetch** position has moved on — even though the handler has only reached offset 130.
3. $t = 6$: the process is killed.
4. $t = 7$: a new member starts at 200. Offsets 131..199 are **never processed**. Sixty-nine payments vanish.

Which is worse? Loss is worse. A duplicate credit is visible in the ledger and a reconciliation job can find two entries carrying one `event_id` and reverse one. A lost event leaves **no trace anywhere** — there is nothing to reconcile against, and you learn about it from an angry customer weeks later.

Decision: turn auto-commit off, commit manually after the handler returns, and store the offset in the same transaction as the effect. Then both sequences above become impossible, because the offset can be neither ahead of nor behind the work.

Answer: Auto-commit can commit past unprocessed records (loss) or lag behind applied ones (doubles). Loss is worse. Use manual commit inside the business transaction.

Step 6b — Deep dive: backpressure and the retry ladder

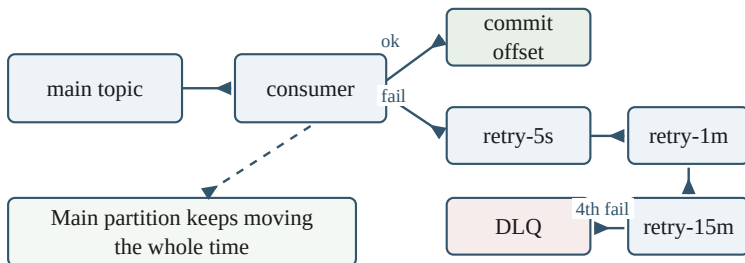
A consumer that cannot keep up has four honest options and one dishonest one.

Option	Effect	When it is right
Slow the producer (block on publish)	the pain moves upstream to a service that can shed load	internal producers you control
Add consumers	works only up to the partition count	the first thing to try
Shed load: drop low-value events	some data is lost, on purpose, with a metric	metrics, click streams
Spill to a slower cheaper store and catch up later	latency rises, nothing is lost	analytics
Silently buffer in memory	the consumer runs out of heap and dies, losing everything in flight	never

TRAP

An unbounded in-memory buffer is not backpressure. It is a delayed crash. In Node this is especially easy to write by accident: `stream.on('data', d => queue.push(d))` never applies backpressure, because the array grows as fast as the socket delivers. Use `await` (`const chunk of stream`) or `pipeline()`, which pause the socket when you stop consuming. Every queue inside your process must have a fixed size, and a full queue must **block or reject**, never grow.

The retry ladder. Never retry a failed message in a tight loop on the main partition — that is head-of-line blocking, and one bad account stops 40,000 good ones.



Each retry topic has its own consumers, so a 15-minute wait costs nothing on the hot path.
The DLQ is replayed by hand after the bug is fixed.

Retry topics: a slow or broken message leaves the main partition immediately.

Example 14 [Google · pattern]

A schema bug makes 0.5% of `payment.captured` events unparseable. Traffic is 173,611 events/s at peak. The bug lives for 40 minutes. How many events reach the DLQ? At a replay rate of 5,000/s, how long does the cleanup take? What is the risk during the replay?

SOLUTION

Bad events per second = $173,611 \times 0.005 = 868.06 \approx 868/\text{s}$.

Duration = $40 \times 60 = 2,400$ seconds.

Total in the DLQ = $868 \times 2,400 = 2,083,200$ events.

Replay time = $2,083,200 / 5,000 = 416.6$ seconds ≈ 7 minutes.

The risk during the replay — and this is the real answer. These are 40-minute-old money events. If the ledger consumer applies them by `occurred_at`, fine. But if any downstream consumer assumes “a newer offset means a newer event”, the replay injects 2 million old events **after** newer ones for the same account, and a naive last-write-wins balance update rolls accounts backwards.

Three protections:

1. Replay into a **separate** topic that only the ledger consumer reads, not the shared topic every team reads.
2. Make the ledger append-only — rows in `ledger_entries`, balance computed as a sum — so order does not matter at all. Addition is commutative; last-write-wins is not.
3. Cap the replay at 5,000/s, about 3% of peak, so the ledger database is not pushed over while it is still serving live traffic.

Answer: 2,083,200 events; about 417 s (7 min) to replay. The main risk is out-of-order application; fix it by making the ledger append-only and replaying on a private topic.

Example 15 [Microsoft · pattern]

Your consumer group has 87 members on 128 partitions. A “stop the world” rebalance pauses **every** member for 8 seconds. You deploy by restarting one member at a time. How long is the group paused in total, and what do you change?

SOLUTION

Restarts = 87. Each restart triggers **two** rebalances: one when the member leaves, one when it rejoins.

Rebalances = $87 \times 2 = 174$.

Pause = $174 \times 8 = 1,392$ seconds = 23.2 minutes of total stoppage.

During those 23 minutes, at 173,611 events/s, the backlog grows by $1,392 \times 173,611 = 241,666,512$ events — about 242 million. Draining that alone takes many minutes after the deploy has finished.

Three fixes, in order of value.

1. **Static membership.** Give each member a fixed instance id. A restart inside the session timeout (say 5 minutes) triggers **no rebalance at all** — the member reclaims its own partitions. This alone removes almost all 174 rebalances.
2. **Cooperative (incremental) rebalancing.** Only the partitions that actually move are paused, instead of all 128. A one-member change moves about $128/87 \approx 1.5$ partitions, so roughly 1% of the group pauses instead of 100%.
3. **Deploy in batches.** Restarting 9 members at a time gives $87/9 = 9.67 \rightarrow 10$ batches, so 20 rebalances instead of 174 — a $174/20 = 8.7 \times$ reduction, at the cost of losing 10% of capacity at once.

Decision: static membership first — it is a configuration change and it removes the problem. Cooperative rebalancing second. Batching third. Do not accept a 23-minute deploy pause as normal.

Answer: 174 rebalances x 8 s = 1,392 s = 23.2 min paused, about 242 M events of backlog. Fix with static membership, then cooperative rebalancing.

Step 6c — Ordering: what you can and cannot promise

Promise	Possible?	How, or why not
All events for one account in order	yes	partition key = <code>account_id</code>
All events in the whole topic in order	no, at this scale	that needs 1 partition = a 10 MB/s ceiling; we need 173.6 MB/s
Order across two topics	no	no shared clock, no shared log
Order after a key changes partition	no	changing the partition count re-maps keys, as the 49.9% measurement showed
Detect out-of-order at the consumer	yes	carry a per-key sequence in the envelope and drop anything with <code>seq <= lastSeenSeq</code>

SHORTCUT

When an interviewer asks for global ordering, do not say “impossible” and stop. Say: “Global order caps us at one partition and 10 MB/s, and we need 173.6 MB/s — 17 times more. So I will give **per-account** order, which is what the business actually needs, and add a per-key sequence number so consumers can **detect** any gap.” That sentence is the whole answer.

Step 7 — Trade-offs, failure modes, 10x

Choice	Side A	Side B	Decision
<code>acks=1</code> vs <code>acks=all</code>	<code>acks=1</code> : about 2 ms to publish, loses data if the leader dies before replication	<code>acks=all</code> : about 9 ms, survives a leader loss	<code>acks=all</code> for money topics, <code>acks=1</code> for click streams. Do not use one setting everywhere; the cost is

			7 ms and the benefit is not losing payments.
Replication factor 2 vs 3	RF2: 33% less disk, one failure away from data loss	RF3: survives two failures, costs 15 TB/day	RF3. With 12 brokers a single-node failure during a rolling restart is routine; RF2 turns that routine event into a data-loss event.
128 vs 512 partitions	128: fewer files, faster rebalance, caps consumers at 128	512: room for 512 consumers, far more metadata and open files	128 now. We need 87. Going to 512 quadruples rebalance time to buy capacity we will not use for two years — and a brand-new topic is always available later.
Broker-managed offsets vs offsets in the consumer's own database	Broker: simple, built-in tooling	Own DB: allows exactly-once together with the business write	Own DB for money consumers, broker for the rest. Pay the complexity only where it buys correctness.
Active-active vs active-passive across regions	Active-active: both regions publish, but the same key can be written in two places and ordering is broken	Active-passive: one region publishes, the other mirrors	Active-passive for writes, active-active for reads. Cross-region write ordering has no cheap correct answer; reading from a local mirror gets 90% of the benefit for 10% of the risk.

Failure modes.

Failure	What happens	What you do
Broker dies	its leader partitions are re-elected from the in-sync copies in seconds	require 2 in-sync replicas so a write is never acked by a lone survivor
Disk fills	the broker stops accepting writes, producers block	alarm at 70% and enforce retention by size as well as by time
Slow consumer	lag grows; retention may delete data it has not read	alarm on lag in seconds , not in messages; 105 TB of retention is your margin
Zombie producer	an old process that was replaced keeps publishing	epoch fencing on the producer id; the broker rejects the stale epoch
Whole region down	publishes fail	producers buffer to local disk for 15 minutes, then fail fast to the caller
Schema change breaks consumers	parse errors everywhere	the registry enforces backward compatibility: a field may be added, never removed and never re-typed

At 10x — 50 billion events/day.

$50 \times 1,000 \times 1,000 / 86,400 = 578 \times 704/s$ average; peak $\times 3 = 1,736 \times 111/s$.

Bytes = $1,736 \times 111 \times 1,000 = 1,736$ MB/s ≈ 1.74 GB/s.

Consumers needed = $1,736 \times 111 / 2 \times 1,000 = 868$. Partitions must exceed 868, so choose **1,024**.

Storage = 50 TB/day raw, $\times 3 = 150$ TB/day, $\times 7 = 1,050$ TB = 1.05 PB hot.

Brokers by disk at 20 TB each = $1,050 / 20 = 52.5 \rightarrow 53$; by network we would want more — say **80**.

What changes at 10x, honestly:

1. Seven days of hot retention becomes unaffordable. Move to **1 day hot plus tiered object storage**: hot drops from 1.05 PB to $150 \times 1 = 150$ TB, and older data lives in object storage at roughly a tenth of the price.
2. One cluster becomes a blast-radius problem. Split into **per-domain clusters** — payments, click stream, logs — so a click-stream flood cannot slow payments down.
3. At 1,024 partitions rebalances get slow. Static membership stops being optional.
4. Compression stops being optional too. A modern compressor gets roughly 4:1 on JSON, turning 1.74 GB/s into about 0.44 GB/s on the wire and on disk. That one flag is worth more than 20 extra brokers.

Interview drill — what the interviewer pushes on

They ask	You answer
“Guarantee exactly-once.”	“The broker cannot. I get exactly-once effect : at-least-once delivery, plus a unique key on the effect, plus the offset committed in the same transaction as the business write.”
“Why not just use a database table as a queue?”	“It works to a few hundred jobs per second — <code>SELECT . . . FOR UPDATE SKIP LOCKED</code> is a real pattern. It breaks when polling load and cleanup pressure start hurting the main database. At 1,390/s I would still consider it; at 173,611/s never.”
“How many partitions?”	“The max of two constraints: throughput (MB/s divided by about 10 MB/s per partition) and consumer parallelism (peak rate divided by per-consumer rate). Here $\max(18, 87) = 87$, rounded up to 128.”
“A consumer is 4 hours behind. What do you do?”	“First check whether retention will delete unread data — that is the real deadline. Then find out whether it is slow code or too few members. If members already equal partitions I cannot scale out, so I must make the handler faster or spin up a temporary parallel group writing into a side store.”
“Kafka or SQS?”	“Replay or multiple independent readers -> Kafka. Independent tasks with no replay and no ops team -> SQS. For this notification service, SQS; for the event backbone, Kafka.”
“Your DLQ has 2 million messages. Now what?”	“Do not replay blindly. Group by error reason first — usually three causes cover 95%. Fix the code, replay at a capped rate into a private topic, and drop anything whose business value has expired.”
“Does a queue reduce total work?”	“No. It moves work off the user’s critical path and makes it retryable. Total CPU is the same or slightly more.”
“When would you NOT use a queue?”	“When the caller needs the result in order to continue (a payment authorisation), when the work takes under 10 ms anyway, or when the added failure modes — dedup, ordering, DLQ, lag alarms — cost more than the 20 ms you saved.”
“Node is single-threaded — can it even do this?”	“Yes, because this work is I/O-bound, not CPU-bound. One process holds 200 in-flight sends while the CPU idles. If the handler were CPU-heavy — image resizing, for example — I would move it to <code>worker_threads</code> or a separate service, because then one process really does give me one core.”

TRAP

The most common Tier-3 failure is answering “we use Kafka” to every question. Kafka is a **log**. It is a poor fit for per-message delays, per-message acks, priority, and a million low-rate queues. If the question asks “retry this one message in 15 minutes”, the honest answer is a delay queue or a retry-topic ladder, not Kafka alone.

PRACTICE · TIER 3 target: 45 min

1. A topic has 64 partitions, 173,611 events/s at peak, and one key (`acct_super`) carries 8% of all traffic. Compute the load on its partition, compare with the average, propose a fix, and state what the fix costs.
2. Design a delayed-job service: “run this task at exactly 09:00 next Tuesday”, with 50 million scheduled tasks live at any time. Which parts of this chapter apply and which do not?
3. Your team wants to move the click-stream topic from at-least-once to at-most-once to save money. Compute what you save and what you lose, then decide.
4. A consumer group of 40 members reads 128 partitions. A deploy costs 8 rebalances of 6 s each. Compute the pause and propose a target.
5. Explain to a junior engineer, in five sentences, why a queue does not make the system faster.

ANSWER KEY

1. Average per partition = $173\{, \}611/64 = 2\{, \}712.7$ events/s. The hot key's partition gets $173\{, \}611 \times 0.08 = 13\{, \}889$ /s from that key alone, plus its share of the rest: $173\{, \}611 \times 0.92/64 = 2\{, \}495.9$ /s. Total $\approx 16\{, \}385$ /s, which is $16\{, \}385/2\{, \}712.7 \approx 6.0 \times$ the average. One consumer cannot keep up. **Fix:** salt the key — use `acct_super#0 .. acct_super#7` to spread over 8 partitions. **Cost:** you lose ordering for that account, so this is acceptable only if that account's events are commutative (an append-only ledger, yes; a state machine, no).
2. **Applies:** idempotency, DLQ, at-least-once. **Does not apply:** a log is the wrong store — you cannot ask a log “what is due at 09:00”. Use a store sorted by due time (a database index on `run_at`, or one Redis sorted set per minute bucket) and a poller that moves due tasks into a normal work queue. 50 M rows with an index on `run_at` is easy; the queue then only ever sees tasks that are due **now**.
3. **Save:** the ack round trip and the dedup store — at 5 B events/day, maybe 10–15% of consumer cost. **Lose:** on every consumer crash you lose the in-flight batch. At a batch of 500 and 20 crashes a month that is $500 \times 20 = 10\{, \}000$ lost events per month out of $5 \times 10^9 \times 30 = 1.5 \times 10^{11}$ — a loss rate of about 6.7×10^{-8} . **Decision: accept it for the click stream.** The analytics numbers move by far less than their own sampling error. Never accept it for payments.
4. A deploy as described costs $8 \times 6 = 48$ s. But restarting all 40 members one at a time actually causes $40 \times 2 = 80$ rebalances = 480 s. Target: static membership, so a restart inside the session timeout causes **zero** rebalances — under 10 s in total.
5. “The work still has to be done, and it still costs the same CPU. A queue only changes **who waits**. The user stops waiting; a background worker waits instead. That is why the page feels four times faster while the servers do slightly more work than before. The real prizes are that the slow part can now be retried, batched, and scaled on its own.”

ONE-PAGE REVISION CARD

Definitions. Queue = one consumer, message deleted after work. Stream = many consumer groups, message kept for a retention period, each group holds its own offset. Partition = an independent ordered log. **Max working consumers in a group = partition count.**

Delivery. At-most-once = ack then work (loses). At-least-once = work then ack (duplicates) — the default. Exactly-once **delivery** does not exist; exactly-once **effect** does, via a unique key on the effect plus the offset written in the same transaction.

Numbers to memorise.

- one partition ≈ 10 MB/s of writes
- one network-bound concurrency slot $\approx 1 / (\text{latency in seconds})$ jobs/s; 200 ms $\rightarrow 5/s$
- about 200 in-flight I/O-bound operations per Node process
- backoff total with base b and n retries $= b(2^n - 1)$; base 1 s, 6 tries = 63 s
- lag drain time = lag / (consumer rate — producer rate)
- doubling the partition count moves about 50% of keys (measured: 49.9%)

The partition formula. $P = \max(\text{peak MB/s} / 10, \text{peak events/s} / \text{per-consumer rate})$, rounded up to a power of two. Worked here: $\max(173.6/10, 173\{, \}611/2\{, \}000) = \max(18, 87) = 87 \rightarrow 128$.

The estimate skeleton. DAU x actions = per day $\rightarrow /86\{, \}400 = \text{average/s} \rightarrow \times \text{peak factor} \rightarrow \times \text{bytes} = \text{MB/s} \rightarrow \times \text{retention} = \text{storage} \rightarrow / \text{per-node capacity} = \text{node count}$. Always show the division.

JavaScript traps that bite in this chapter.

1. `sort()` is lexicographic. Times and offsets need $(a, b) \Rightarrow a - b$.
2. No built-in priority queue. Use the toolkit `MinHeap`.
3. Atomicity in Node is the stretch between two `awaits`. Check-then-act must live in one such stretch, or take an explicit `async mutex`.
4. Map keeps insertion order; a plain object does not promise it. The windowed-dedup eviction loop depends on that.
5. `stream.on('data', d => arr.push(d))` is not backpressure. Use `for await` or `pipeline()`.

Checklist before you say “done”.

1. Named the delivery semantic out loud.
2. Showed where duplicates are absorbed (a unique key, not a cache).
3. Gave a DLQ with a retry count and an alarm on depth.
4. Said what happens when consumers fall behind (backpressure, not silent buffering).
5. Chose a partition key and said what ordering it does and does not give.
6. Named both sides of every trade-off and then **decided**.
7. Said what breaks at 10x and what you would change.

Three sentences that score marks.

1. “The broker gives me at-least-once; my unique key on `event_id` gives me exactly-once effect.”
2. “Partition count is set by consumer parallelism, not by throughput, and it is very hard to change later — about half the keys move when you double it.”
3. “An unbounded in-memory buffer is not backpressure, it is a delayed crash.”

Replication, Sharding & Consistency

Copies keep you alive. Splits keep you fast. Consistency is the promise you make to the reader.

The idea in one page

One database machine can do three things badly at scale. It can **die** and take your data with it. It can **fill up**. It can **run out of CPU**. Each problem has exactly one family of answers, and beginners mix them up constantly.

THREE WORDS, THREE DIFFERENT JOBS

Word	What it does	What it does NOT do
Replication	Keeps N complete copies of the same data. Survives a dead machine. Adds read capacity.	Does not add write capacity. Does not add storage capacity. Every copy is the full data.
Sharding	Splits the data into pieces. Adds write capacity and storage capacity.	Does not add safety. A shard with one copy is one dead disk away from lost data.
Consistency model	The sentence you can honestly tell a reader about what they will see.	Not a tuning knob you turn to “high”. It is a promise with a price.

You almost always need all three, and in this order:

1. Replicate first. Three copies. This is not optional; it is how you sleep at night.
2. Cache and add read replicas. This buys you 10x reads for almost no design cost.
3. Shard last, when one machine genuinely cannot hold the data or take the writes.

Sharding is the expensive one. It breaks joins, breaks transactions, breaks **ORDER BY** across the whole table, and it is very hard to undo. Do it when you must, not when you can.

TRAP

The single most common wrong sentence in a design interview: “we will shard the database so it can handle more traffic”. Traffic is usually **reads**. Reads are fixed by a cache and read replicas — no sharding needed. Sharding is for **writes** and for **size**. Say which one you are solving, every time.

The vocabulary — say these words correctly

SAY IT EXACTLY LIKE THIS

- **Leader (primary)**. The one copy that accepts writes. Also called master in old books.
- **Follower (replica, secondary)**. Copies that apply the leader’s log. Usually read-only.
- **Replication lag**. How far behind a follower is, measured in **seconds** or in **writes**.
- **Synchronous replication**. The leader waits for a follower’s ack before answering the client. No data loss on failover; slower writes.
- **Asynchronous replication**. The leader answers immediately. Fast writes; a crash loses whatever was still in flight.
- **Semi-synchronous**. Wait for **one** follower, not all. The usual real answer.
- **Failover**. Promoting a follower to leader after the leader dies.
- **Split brain**. Two nodes both think they are leader. Both accept writes. You now have two truths and no way to merge them.
- **Shard (partition)**. One slice of the data, holding a disjoint set of rows.

- **Shard key (partition key).** The field that decides which shard a row lives on.
- **Quorum.** A count of nodes that must agree. N replicas, W to write, R to read.
- **Hot shard / hot key.** One shard or one row taking far more than its fair share.
- **Rebalancing / resharding.** Moving data because the shard count changed.

What “consistent” actually means — six levels, weakest first

“Consistent” is not one thing. It is a ladder. Every step up costs latency or availability. Learn to name the step you are standing on.

Level	The promise to the reader	Typical use
Eventual	If writes stop, every replica converges — eventually. Until then, anything.	view counts, “likes”, search index
Read-your-writes	You always see your own last write. You may not see mine.	profile edit, “my orders”
Monotonic reads	You never go backwards in time. Once you saw v5, you never see v4.	a feed you scroll
Consistent prefix	You see writes in causal order. No reply before its question.	chat, comment threads
Linearizable	Every read sees the latest committed write, as if there were one machine.	inventory count, username claim
Serializable	Whole transactions behave as if run one after another.	bank transfer, seat booking

Linearizable is about one object at a time (“this counter behaves like one counter”). **Serializable** is about whole transactions over many objects. A store can be one and not the other. Saying the right word here is a genuine senior signal.

CAP, stated correctly

Most candidates say “CAP means pick two of three”. That sentence loses marks because it is not true. You do not get to choose whether a network partition happens. It happens.

CAP, THE HONEST VERSION

P (partition) is not a choice. It is weather.

When a partition splits your cluster in two, and a client talks to the minority side, you have **exactly two options**:

1. **CP**: refuse to answer. Return an error or block. The data stays correct, the service is down for that client.
2. **AP**: answer with what you have. The service stays up, the answer may be stale or may later conflict with the other side.

There is no third door. So CAP is really: “**during a partition, do you prefer an error or a stale answer?**”

And the part the interviewer is actually waiting for — **PACELC**:

if P then $(A \text{ or } C)$, E lse $(L \text{ or } C)$

“Else” means: when the network is **fine** (which is 99.9% of the time), you still trade **Latency** against **Consistency**. Waiting for a quorum in another region costs real milliseconds even when nothing is broken. That “else” branch is where your users actually live.

SHORTCUT

The sentence that scores the CAP mark, every time: “Partitions are not optional, so the real question is what I do during one. For the **shopping cart** I choose AP — showing a slightly old cart beats showing an error, and I merge by union so nothing is lost. For the **inventory counter** I choose CP — I would rather return 503 than sell the last phone twice. Different data in the same product can sit on different sides of CAP.”

The numbers card — memorise these

REPLICATION AND SHARDING NUMBERS

- 1 day = 86,400 s. **1 million/day** $\approx$ 11.6/s.
- Same-datacentre network round trip: **0.5 ms**. Across a region: **1–2 ms**.
- Singapore $\leftrightarrow$ Mumbai round trip: **60 ms**. Singapore $\leftrightarrow$ Virginia: **230 ms**.
- One SSD disk read: **0.1 ms**. One fsync: **1–2 ms**.
- One mid-size SQL node: **5,000 writes/s**, **20,000 indexed reads/s**.
- One Redis node: **100,000 simple ops/s**.
- Disk restore/rebuild speed: **200 MB/s**. This is what sets your shard **size**.
- Comfortable shard size: **200 GB to 1 TB**. Bigger and a rebuild takes hours.
- **Quorum rule:** $W + R > N$ gives you a guaranteed overlap of at least one node.
- **Write availability:** you survive $N - W$ node failures for writes, $N - R$ for reads.
- Consistent hashing moves $1/(N + 1)$ of keys when you add the $(N + 1)$ -th node. Modulo hashing moves about $N/(N + 1)$ — nearly everything.

Warm-up

WARM-UP

build the reflex

Example 1 [warm-up]

A leader takes 10,000 writes/s for 60 seconds during a sale. Its follower can only apply 8,000 writes/s. After the sale the leader drops to 5,000 writes/s. How big does the backlog get, and how long until the follower catches up?

SOLUTION

Step 1 — the gap while the sale runs.

$$10,000 - 8,000 = 2,000 \text{ writes/s of backlog growth}$$

Step 2 — backlog at the end of 60 seconds.

$$2,000 \times 60 = 120,000 \text{ writes behind}$$

Step 3 — drain rate after the sale. The follower still applies 8,000/s, but 5,000/s of new work keeps arriving.

$$8,000 - 5,000 = 3,000 \text{ writes/s of catch-up}$$

Step 4 — time to catch up.

$$120,000 / 3,000 = 40 \text{ seconds}$$

So for 100 seconds in total, any read served from that follower can be stale — at the worst moment, by $120,000 / 10,000 = 12$ seconds of real time.

Answer: 120,000 writes of backlog, 40 seconds to drain, up to 12 s of staleness

TRAP

Notice the follower **never fell over**. It just fell behind. That is the danger: a lagging replica looks perfectly healthy in every dashboard except the one that measures lag. Alarm on lag in **seconds**, not on CPU.

Example 2 [warm-up]

You have $N = 5$ replicas. For each pair below, say whether a read is guaranteed to see the newest committed write, and how many node failures the system survives.

W	R	$W + R$	?
3	3		
5	1		
1	1		
4	2		

SOLUTION

The rule is $W + R > N$. If the write quorum and the read quorum must overlap in at least one node, then that node has the newest value, and the reader can pick the highest version.

W	R	$W + R$	Sees newest?	Survives
3	3	$6 > 5$	yes — overlap of $6 - 5 = 1$ node	writes: $5 - 3 = 2$ down; reads: $5 - 3 = 2$ down
5	1	$6 > 5$	yes — every node has it, so any one node answers	writes: $5 - 5 = 0$ down (fragile!); reads: 4 down
1	1	$2 \leq 5$	no — the one writer and the one reader can be different nodes	writes: 4 down; reads: 4 down
4	2	$6 > 5$	yes	writes: 1 down; reads: 3 down

Read the second row again. $W = 5$ looks “safest” and is in fact the **worst** choice for availability: a single node reboot for a security patch stops all writes.

Answer: Rows 1, 2 and 4 are strongly consistent; row 3 is not. $W = 3, R = 3$ is the balanced pick.

Example 3 [warm-up]

A dataset is 20 TB. One node holds 2 TB comfortably. Peak write load is 60,000 writes/s and one node handles 5,000 writes/s. How many shards, and how many machines at replication factor 3?

SOLUTION

Two constraints. Compute both, take the larger.

Constraint A — size.

$$20 \text{ TB} / 2 \text{ TB per node} = 10 \text{ shards}$$

Constraint B — write throughput.

$$60,000 / 5,000 = 12 \text{ shards}$$

Shards needed = $\max(10, 12) = 12$.

Machines. Each shard needs 3 copies:

$$12 \times 3 = 36 \text{ database instances}$$

Those 36 instances do not need 36 physical machines — you can pack 2 per machine, giving 18 machines, as long as **no two copies of the same shard sit on the same machine or the same rack**. That last clause is the whole point of replication.

Answer: 12 shards, 36 instances, packed onto ≥ 18 machines with anti-affinity

Example 4 [warm-up]

You have 10 cache nodes and route keys with $\text{hash}(\text{key}) \% 10$. You add an eleventh node. What fraction of keys move? What if you use consistent hashing instead?

SOLUTION

Modulo. A key stays put only if $h \bmod 10 = h \bmod 11$. For a uniform hash that happens for roughly $1/11$ of keys.
$$\text{moved} \approx 1 - 1/11 = 10/11 = 90.9\%$$

Consistent hashing. The new node claims one arc of the ring. Only the keys inside that arc move, and only from the one node that owned the arc.

$$\text{moved} \approx 1/11 = 9.09\%$$

Why it matters in real money. Suppose the cache holds 50 GB and serves a 95% hit rate. Moving 90.9% of keys means $50 \times 0.909 = 45.5$ GB of cache goes cold at once, and every one of those keys becomes a database read until it is refilled. Moving 9.09% means $50 \times 0.0909 = 4.5$ GB goes cold. That is a 10x difference in the size of the stampede you hand to your database.

Answer: Modulo: 90.9% move. Consistent hashing: 9.1% move.

Example 5 [warm-up]

Your logical data is 4 TB and grows 40% a year. You run replication factor 3, and the storage engine needs 50% free space for compaction. How much raw disk do you buy for year one, and for year three?

SOLUTION

Year 1.

$$4 \text{ TB} \times 3 \text{ (RF)} = 12 \text{ TB of replicated data}$$

$$12 \times 1.5 \text{ (compaction headroom)} = 18 \text{ TB of raw disk}$$

Year 3. Grow the logical size first.

$$4 \times 1.4 = 5.6 \text{ TB (end of year 1)}$$

$$5.6 \times 1.4 = 7.84 \text{ TB (end of year 2)}$$

$$7.84 \times 1.4 = 10.976 \approx 11.0 \text{ TB (end of year 3)}$$

Then apply the same two multipliers:

$$11.0 \times 3 \times 1.5 = 49.5 \text{ TB of raw disk}$$

Answer: 18 TB now, 49.5 TB by end of year three

TRAP

Candidates quote the **logical** size and stop. The interviewer is waiting for $\times \text{RF} \times \text{headroom}$. Forgetting the 1.5x is how real clusters run out of disk at 2 a.m., because a log-structured store needs free space to merge files, and a full disk means it cannot even delete data.

Example 6 [warm-up]

16 shards. Peak load 40,000 writes/s. One enterprise customer generates 12% of all writes and all of their rows land on one shard. How loaded is that shard compared with the average?

SOLUTION

Average shard.

$$40,000/16 = 2,500 \text{ writes/s}$$

The hot shard gets the big customer plus its normal share of everyone else.

from the big customer = $40,000 \times 0.12 = 4,800$ writes/s

its share of the rest = $40,000 \times 0.88/16 = 2,200$ writes/s

total = $4,800 + 2,200 = 7,000$ writes/s

Ratio.

$7,000/2,500 = 2.8 \times$ the average

If one node handles 5,000 writes/s, this shard is over capacity while the other 15 sit at half load. You cannot fix that by adding shards: 32 shards would give the hot one $4,800 + 40,000 \times 0.88/32 = 4,800 + 1,100 = 5,900$ s. Still over. The big customer must be **split**, not the cluster.

Answer: 7,000 writes/s, which is 2.8x the average — adding shards does not fix it

Example 7 [warm-up]

A write takes 2 ms to commit locally. You want it replicated synchronously to a second region 60 ms round trip away. Each API server holds 200 concurrent database connections. What is the maximum write throughput per server, before and after?

SOLUTION

Before (async, local only). Time per write = 2 ms = 0.002 s.

$200/0.002 = 100,000$ writes/s per server

After (sync across regions). Time per write = 2 + 60 = 62 ms = 0.062 s.

$200/0.062 = 3,225.8 \approx 3,226$ writes/s per server

The cost.

$100,000/3,226 = 31 \times$ less throughput

To keep 100,000 writes/s you would need $100,000/3,226 = 31$ servers instead of 1, **or** far more connections, which just moves the queue somewhere else.

Answer: 100,000/s drops to 3,226/s — a 31x loss of throughput for cross-region sync

This one number is why nobody replicates synchronously across continents for ordinary data. It is also why the systems that **do** (global databases with consensus) charge a lot and are used only for the small, precious slice of data that truly needs it.

Example 8 [warm-up]

A user edits her profile, then the app immediately reloads the profile page. Replication lag is 50 ms most of the time but occasionally 3 seconds. What breaks, and give three fixes.

SOLUTION

What breaks. The write goes to the leader. The reload is a read, and a read balancer sends it to a follower. If that follower has not applied the write, the user sees her **old** name and thinks the save failed. She saves again. Now you have duplicate work and an angry user.

Fix 1 — read from the leader for a window. After a write, pin that user's reads to the leader for, say, 5 seconds. Cost: the leader takes extra read load, but only from users who just wrote — a tiny fraction.

Fix 2 — read your own writes by version. The write returns a version number ($v=8124$). The client sends `If-Version-At-Least: 8124` with the read. A follower that is behind either waits or replies "ask the leader". Cost: one extra field everywhere.

Fix 3 — do not read at all. The write response **contains** the new profile. The client renders that and never issues the read. Cost: nothing. This is the best fix and candidates almost never say it.

Decision: Fix 3 first because it is free, Fix 1 as the general safety net for pages you cannot control. Fix 2 only if you already carry versions for another reason — otherwise it touches every endpoint.

Answer: Read-your-writes is violated. Best fix: return the new value from the write; back it up with leader-pinning for 5 s.

PRACTICE · WARM-UP target: 12 min

1. $N = 7$. You want to survive 2 node failures for both reads and writes, and still read the newest write. Give a (W, R) pair and prove it.
2. Logical data 12 TB, RF 3, 40% compaction headroom, one node holds 3 TB. How many nodes?
3. A follower is 45 minutes behind at 9 a.m. and gains 400 writes/s of ground while the leader takes 1,200 writes/s. When does it catch up?
4. You shard by hash(`user_id`) into 8 shards. A celebrity user has 2% of all reads. Is that a hot shard problem? Show the arithmetic.
5. Cross-region round trip is 230 ms. A checkout does 4 sequential database calls. Compare total latency for a local leader versus a remote leader.

ANSWER KEY

1. Survive 2 failures means $W \leq 5$ and $R \leq 5$. Newest write means $W + R > 7$, so $W + R \geq 8$. Take $W = 4, R = 4$: $4 + 4 = 8 > 7$ ✓, survives $7 - 4 = 3$ failures for both. Even better than asked.
2. $12 \times 3 = 36$ TB replicated; $36 \times 1.4 = 50.4$ TB raw; $50.4/3 = 16.8 \rightarrow 17$ nodes.
3. 45 min = 2,700 s of lag at 1,200 writes/s = 3,240,000 writes behind. Catch-up rate 400/s. $3,240,000/400 = 8,100$ s = 2.25 hours. It catches up at about 11:15 a.m. — **if** the leader's rate does not rise at lunchtime, which it will.
4. Average shard read share = $100\%/8 = 12.5\%$. The celebrity adds 2% to one shard, giving it $2 + 98/8 = 2 + 12.25 = 14.25\%$, which is $14.25/12.5 = 1.14 \times$ the average. **Not** a hot shard problem. A hot **key** problem, yes — fixed by caching that one user's row, not by resharding. Reserve the word "hot shard" for ratios above about 2x.
5. Local: 4×2 ms = 8 ms. Remote: $4 \times 230 = 920$ ms, just under a second, and that is before any application work. Fix: make the calls in one round trip (a stored procedure or a batched API), which brings the remote case to 230 ms.

Tier 1 — build the pieces yourself

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Service-company rounds rarely say "design a sharded cluster". They say "write a consistent hashing class" or "how would you code a quorum read". These five pieces are the ones that actually get asked. Every one of them runs.

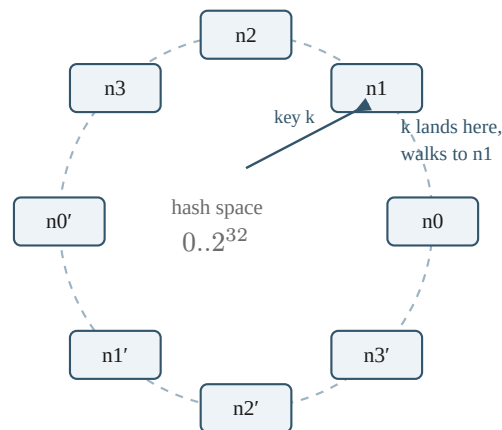
Piece 1 — a consistent hash ring

Example 9 [Infosys · pattern]

Write a class that maps keys to N nodes so that adding or removing a node moves as few keys as possible. Then **measure** how many keys move.

SOLUTION

The idea. Imagine the hash space $0..2^{32} - 1$ bent into a circle. Every node is placed at several points on that circle. To find a key's node, hash the key and walk **clockwise** to the first node point you meet.



Add a node: it lands somewhere on the circle and steals **one arc** from **one** neighbour.

Every other key keeps its owner. **n0'** and **n0** are two **virtual points** for the same machine.

The ring. Each node owns the arc that ends at its point. A key walks clockwise to its owner.

Why virtual nodes? With one point per machine, the arcs come out wildly uneven — one machine can own 20 times what another owns. Giving each machine 150 points averages the unevenness away. The measurement below shows both.

ring.js — runs as written

JS

```
function fnv1a(str) {
  let h = 0x811c9dc5;
  for (let i = 0; i < str.length; i++) {
    h ^= str.charCodeAt(i);
    h = Math.imul(h, 0x01000193) >>> 0;    // imul keeps it a 32-bit multiply
  }
  // avalanche: without this, "n0#1" and "n0#2" land next to each other
  h ^= h >>> 16; h = Math.imul(h, 0x7feb352d) >>> 0;
  h ^= h >>> 15; h = Math.imul(h, 0x846ca68b) >>> 0;
  h ^= h >>> 16;
  return h >>> 0;
}

// first index whose key >= target
function lowerBound(arr, target, key = x => x) {
  let lo = 0, hi = arr.length;
  while (lo < hi) {
    const mid = (lo + hi) >> 1;
    if (key(arr[mid]) < target) lo = mid + 1; else hi = mid;
  }
  return lo;
}

class ConsistentHashRing {
  constructor(vnodes = 150) {
    this.vnodes = vnodes;
    this.ring = [];           // sorted array of { hash, node }
    this.nodes = new Set();
  }

  addNode(name) {
    if (this.nodes.has(name)) return;
    this.nodes.add(name);
  }
}
```

```

    for (let i = 0; i < this.vnodes; i++) {
      this.ring.push({ hash: fnv1a(`${name}#${i}`), node: name });
    }
    // numeric comparator, never bare sort()
    this.ring.sort((a, b) => a.hash - b.hash);
  }

  removeNode(name) {
    if (!this.nodes.delete(name)) return;
    this.ring = this.ring.filter(e => e.node !== name);
  }

  // clockwise walk = binary search + wrap
  locate(key) {
    if (this.ring.length === 0) return null;
    let i = lowerBound(this.ring, fnv1a(key), e => e.hash);
    // past the last point: wrap to the first
    if (i === this.ring.length) i = 0;
    return this.ring[i].node;
  }

  locateN(key, n) { // the n replicas for this key
    const out = [];
    if (this.ring.length === 0) return out;
    const h = fnv1a(key);
    const start = lowerBound(this.ring, h, e => e.hash) % this.ring.length;
    for (let s = 0; s < this.ring.length && out.length < n; s++) {
      const nd = this.ring[(start + s) % this.ring.length].node;
      // skip vnodes of a node already chosen
      if (!out.includes(nd)) out.push(nd);
    }
    return out;
  }
}

```

the measurement — this is the part that earns the mark

JS

```

const KEYS = Array.from({ length: 100000 }, (_, i) => `user:${i}`);

const ring = new ConsistentHashRing(150);
for (let i = 0; i < 10; i++) ring.addNode(`n${i}`);
const before = KEYS.map(k => ring.locate(k));
ring.addNode('n10');
const after = KEYS.map(k => ring.locate(k));

let moved = 0;
for (let i = 0; i < KEYS.length; i++) if (before[i] !== after[i]) moved++;
console.log('consistent: moved', (100 * moved / KEYS.length).toFixed(2) + '%',
  ' ideal 1/11 =', (100 / 11).toFixed(2) + '%');

let movedMod = 0;
for (const k of KEYS) if (fnv1a(k) % 10 !== fnv1a(k) % 11) movedMod++;
console.log('modulo : moved', (100 * movedMod / KEYS.length).toFixed(2) + '%');

// how evenly are the keys spread over the 11 nodes?
const count = new Map();

```



```

for (const k of KEYS) {
  const n = ring.locate(k);
  count.set(n, (count.get(n) || 0) + 1);
}
const v = [...count.values()].sort((a, b) => a - b);
console.log('vnodes=150: min', v[0], 'max', v[v.length - 1],
  'ideal', Math.round(KEYS.length / 11));

// the same measurement with ONE point per machine
const ring1 = new ConsistentHashRing(1);
for (let i = 0; i < 11; i++) ring1.addNode(`n${i}`);
const c1 = new Map();
for (const k of KEYS) {
  const n = ring1.locate(k);
  c1.set(n, (c1.get(n) || 0) + 1);
}
const v1 = [...c1.values()].sort((a, b) => a - b);
console.log('vnodes=1 : min', v1[0], 'max', v1[v1.length - 1],
  'spread', (v1[v1.length - 1] / v1[0]).toFixed(1) + 'x');

console.log('replicas for user:42 ->', ring.locateN('user:42', 3));

```

actual output

TEXT

```

consistent: moved 8.25% ideal 1/11 = 9.09%
modulo : moved 91.05%
vnodes=150: min 8246 max 10870 ideal 9091
vnodes=1 : min 301 max 19410 spread 64.5x
replicas for user:42 -> [ 'n7', 'n10', 'n9' ]

```

Read the last three lines carefully. With 150 virtual nodes the busiest machine holds $10,870/8,246 = 1.32$ times the quietest. With **one** virtual node per machine, the busiest holds $19,410/301 = 64.5$ times the quietest — one machine melts while another is idle. Virtual nodes are not a detail; they are the whole trick.

TRAP

JS trap. `this.ring.sort()` with no comparator sorts the 32-bit hashes as **text**. `[10, 9, 1]` becomes `[1, 10, 9]`. Your binary search then returns garbage and the ring silently sends keys to the wrong node. Always `(a, b) => a - b`.

Interviewers sometimes ask “why not just `hash(key) % N`?” The answer in one sentence: “because N changes, and modulo makes almost every key change owner when it does, which turns one added node into a full cache flush and a database stampede.”

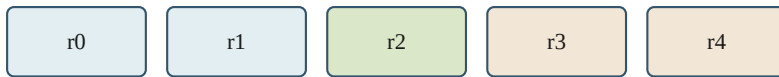
Piece 2 — a quorum store

Example 10 [Capgemini · pattern]

Code a key-value store over N replicas with tunable W and R . Show that $W + R > N$ really does prevent a stale read, and that $W + R \leq N$ really does allow one.

SOLUTION

write quorum $W = 3$



overlap: r2

$$W + R = 6 > N = 5$$

$$\text{overlap} = 6 - 5 = 1$$

survives 2 dead nodes

for reads **and** writes

If W were 3 and R were 2, the sum is 5, which is not greater than 5 — the two ranges could sit side by side with no shared node, and the reader would return an old value with a clear conscience.

$N = 5, W = 3, R = 3$. The two quorums must share at least $6 - 5 = 1$ node. That node holds the newest value.

quorum.js — a replica and the coordinator above it

JS

```
class Replica {
  constructor(id) { this.id = id; this.data = new Map(); this.up = true; }
  put(key, value, version) {
    if (!this.up) throw new Error(`${this.id} down`);
    const cur = this.data.get(key);
    if (!cur || version > cur.version) this.data.set(key, { value, version });
    // an older version arriving late is simply ignored
    return true;
  }
  get(key) {
    if (!this.up) throw new Error(`${this.id} down`);
    return this.data.get(key) || null;
  }
}

class QuorumStore {
  constructor(n, w, r) {
    if (w + r <= n) console.warn(`W+R=${w+r} <= N=${n}: stale reads`);
    this.replicas = Array.from({ length: n }, (_, i) => new Replica(`r${i}`));
    this.N = n; this.W = w; this.R = r;
    this.clock = 0;
  }

  write(key, value) {
    // stands in for a real timestamp or counter
    const version = ++this.clock;
    let ok = 0; const failed = [];
    for (const rep of this.replicas) {
      try { rep.put(key, value, version); ok++; } catch { failed.push(rep.id); }
    }
    if (ok < this.W) return { ok: false, acks: ok, need: this.W };
    return { ok: true, acks: ok, version, hintedFor: failed };
  }

  read(key) {
    const seen = [];
    for (const rep of this.replicas) {
      try { seen.push({ id: rep.id, v: rep.get(key) }); } catch { /* down */ }
      // stop as soon as R replicas answered
      if (seen.length === this.R) break;
    }
    if (seen.length < this.R) {
```

```

    return { ok: false, replies: seen.length, need: this.R };
  }

  let best = null;
  for (const s of seen) {
    if (s.v && (!best || s.v.version > best.version)) best = s.v;
  }

  // read repair: quietly fix any replica in this quorum that was behind
  for (const s of seen) if (best && (!s.v || s.v.version < best.version)) {
    const r = this.replicas.find(x => x.id === s.id);
    try { r.put(key, best.value, best.version); }
    catch {}
  }
  if (!best) return { ok: true, value: null, version: 0 };
  return { ok: true, value: best.value, version: best.version };
}
}

```

actual output

TEXT

```

--- N=5, W=3, R=3 (W+R=6 > 5) ---
write: { ok: true, acks: 5, version: 1, hintedFor: [] }
write, 2 nodes down: { ok: true, acks: 3, version: 2, hintedFor: ['r0','r1'] }
read  with 2 nodes down: { ok: true, value: 250, version: 2 }
write with 3 nodes down: { ok: false, acks: 2, need: 3 }
read  with 3 nodes down: { ok: false, replies: 2, need: 3 }

--- N=5, W=1, R=1 (W+R=2 <= 5): stale read is possible ---
W+R=2 <= N=5: stale reads
read sees: { ok: true, value: null, version: 0 }

```

Line by line:

- With 2 of 5 nodes dead, writes and reads both still work, and the read returns the **new** value (250). That is $W + R > N$ doing its job.
- With 3 dead, $W = 3$ can no longer be met. The store **refuses the write**. It did not quietly accept it on 2 nodes and hope. That refusal is the CP choice, made by arithmetic.
- In the second block, $W = 1$ and $R = 1$: the write landed only on r0, r0 then went away, and the reader asked r1, which had never heard of the key. The value did not just look old — **it was gone**.

hintedFor is the hook for hinted handoff: a live node holds the write on behalf of the dead one and replays it when that node returns. Say the phrase; it is the standard answer to “what happens to the two nodes that missed the write?”

Read repair is why an eventually-consistent store converges without a background job doing all the work: every read quietly fixes the replicas it touched. Popular keys self-heal in milliseconds. Cold keys need the background repair process — that is what **anti-entropy** and **Merkle trees** are for. Name-drop both; one sentence each is enough.

Piece 3 — version vectors: a real conflict versus a stale copy

Example 11 [Cognizant · pattern]

Two regions accept a write to the same key at the same moment. How does the store know whether one write simply **came later**, or whether the two writes genuinely **conflict**? Code it.

| SOLUTION

Why a timestamp is not enough. Two servers' clocks differ. If Singapore's clock is 40 ms ahead of Mumbai's, then a write made in Mumbai **after** the Singapore one still carries a smaller timestamp, and last-write-wins throws away the newer change. Silently. Forever.

The fix. Each writer keeps its own counter. A value carries the whole map {writer → counter}. Now comparison is mechanical.

COMPARING TWO VERSION VECTORS A AND B

- Every counter in A is $\geq$ B's, and at least one is bigger → **A is newer**.
- The mirror image → **B is newer**.
- A is bigger somewhere **and** B is bigger somewhere else → **CONCURRENT**. This is a true conflict. Somebody has to decide what to do.
- All equal → same version.

vv.js — the whole idea in 30 lines

JS

```
class VersionVector {
  constructor(map = new Map()) { this.v = new Map(map); }
  bump(node) {
    const c = new Map(this.v);
    c.set(node, (c.get(node) || 0) + 1);
    // immutable: a new version, not a mutation
    return new VersionVector(c);
  }
  get(node) { return this.v.get(node) || 0; }
  keys() { return new Set([...this.v.keys()]); }
  merge(o) {
    const c = new Map(this.v);
    for (const k of o.v.keys()) c.set(k, Math.max(this.get(k), o.get(k)));
    return new VersionVector(c);
  }
  toString() {
    const parts = [...this.v.entries()].sort().map(([k, n]) => `${k}:${n}`);
    return '{ ' + parts.join(' ') + ' }';
  }
}

function compare(a, b) {
  const all = new Set([...a.keys(), ...b.keys()]);
  let aBigger = false, bBigger = false;
  for (const k of all) {
    if (a.get(k) > b.get(k)) aBigger = true;
    if (b.get(k) > a.get(k)) bBigger = true;
  }
  if (aBigger && bBigger) return 'CONCURRENT'; // a real conflict
  if (aBigger) return 'A_NEWER';
  if (bBigger) return 'B_NEWER';
  return 'EQUAL';
}
```

actual output

TEXT

```
sg = {sg:2}  in = {in:1 sg:1}
sg vs base  -> A_NEWER
sg vs in    -> CONCURRENT
```

```
merged      -> {in:1 sg:2}
LWW keeps   -> [ 'shoes', 'hat' ]
union keeps -> [ 'hat', 'shoes', 'socks' ]
```

The last two lines are the punchline. A user added socks in Singapore and hat in Mumbai to the same cart. Last-write-wins keeps one and **deletes the other item the user chose**. Merging by union keeps both. For a cart, union is obviously right.

HOW TO RESOLVE A CONCURRENT PAIR --- PICK ONE AND SAY WHY

Strategy	Good for	Cost
Last-write-wins	values where the newest genuinely replaces the old: “display theme = dark”	silently loses data; depends on clocks you do not control
Merge function (union, max, sum)	sets and counters: carts, tags, seen-lists, like counts	you must write the function, and it must be commutative
Keep both, ask the user	documents, text a human wrote	product work: a UI for “you have two versions”
Refuse to have conflicts	money, inventory, username claims	one leader per key, and the latency that comes with it

Decision for a shopping cart: merge by union, plus a tombstone for removals so that “I deleted the hat” is itself a write that can win, rather than being undone by the union.

TRAP

Version vectors grow. One entry per **writer**, forever. If your writer id is the server process, a cluster that has ever run 4,000 pods carries 4,000 entries on every value. Use a small, stable set of writer ids — one per **replica** or one per **region**, not one per process — and prune entries that have not changed in a long time.

Piece 4 — the shard router

Example 12 [TCS NQT · pattern]

Write three routers — range, hash, and directory — and show, with real keys, what each one gets right and wrong.

SOLUTION

```
router.js JS

class RangeRouter {
  // bounds sorted by upTo
  constructor(bounds) { this.bounds = bounds; }
  route(key) {
    for (const b of this.bounds) if (key <= b.upTo) return b.shard;
    return 'overflow';
  }
}

class HashRouter {
  constructor(n) { this.n = n; }
  route(key) {
    let h = 0;
    for (const c of key) h = (h * 31 + c.charCodeAt(0)) >>> 0;
    return `s${h % this.n}`;
  }
}
```

```

    }
}

// an editable lookup table in front of a default
class DirectoryRouter {
    constructor() { this.map = new Map(); this.fallback = new HashRouter(4); }
    pin(key, shard) { this.map.set(key, shard); }
    route(key) { return this.map.get(key) ?? this.fallback.route(key); }
}

```

actual output, with keys that start with a date

TEXT

```

range routing (all of today lands on ONE shard):
  order_2026_09_15_a -> s0
  order_2026_09_15_b -> s0
  order_2026_09_15_c -> s0
  order_2026_09_16_a -> s1
  order_2026_09_16_b -> s1
hash routing (today is spread):
  order_2026_09_15_a -> s0
  order_2026_09_15_b -> s1
  order_2026_09_15_c -> s2
  order_2026_09_16_a -> s1
  order_2026_09_16_b -> s2
directory: tenant:mega_corp -> s_dedicated | tenant:small_shop -> s2

```

Router	Wins at	Loses at	Use when
Range	range scans: “all orders in September” is one shard, one scan	every new write goes to the last shard. A date-ordered key makes a permanent hot shard.	time-series you mostly read by range, and you accept the write hotspot
Hash	even spread of writes; no hotspot from ordered keys	a range scan must ask every shard and merge	the default. Pick this unless you have a scan-shaped workload
Directory	you can move one noisy tenant to its own shard without touching anyone else	the directory itself must be highly available and cached everywhere	multi-tenant systems where tenants differ in size by 1000x

Decision for a general OLTP service: hash routing, with a directory override table for the handful of keys that misbehave. You get an even spread by default and an escape hatch for the one customer who is 12% of your traffic — which is Example 6, solved.

SHORTCUT

The trick that makes resharding survivable: **never hash directly to a shard.** Hash to a large fixed number of **virtual buckets** — say 1,024 — and keep a small map from bucket to shard.

$$\text{bucket} = \text{hash}(\text{key}) \bmod 1024, \quad \text{shard} = \text{bucketMap}[\text{bucket}]$$

The bucket of a key **never changes**, for the life of the system. Growing from 16 to 32 shards is now a change to a 1,024-entry map plus the copying of 512 buckets. No key is ever rehashed. Every serious sharded system does this.

Piece 5 — leases and fencing tokens (how failover does not corrupt data)

Example 13 [Accenture · pattern]

Node A is the leader. A pauses for 12 seconds (garbage collection, a slow disk, a paused VM). The cluster promotes B. Then A wakes up, still believing it is the leader, and writes. Stop the corruption.

SOLUTION

Why a simple lock fails. A held a lock. The lock timed out. But A never found out — it was frozen, not disconnected. When A resumes it does not check; it just writes. No amount of “but the lock expired” helps, because the write is already on its way to the disk.

The fix: a fencing token. Every time leadership changes, the token goes **up**. The storage layer remembers the highest token it has seen and refuses anything lower. Now a late write from an old leader is rejected **by the storage**, not by the sender’s good manners.

```
lease.js JS

class LeaseService {
  constructor(ttlMs) {
    this.ttlMs = ttlMs;
    this.holder = null;
    this.expiresAt = 0;
    this.token = 0;
  }

  acquire(node, now) {
    if (this.holder !== null && now < this.expiresAt && this.holder !== node) {
      const msLeft = this.expiresAt - now;
      return { granted: false, holder: this.holder, msLeft };
    }
    // a NEW leader gets a NEW, higher token
    if (this.holder !== node) this.token += 1;
    this.holder = node;
    this.expiresAt = now + this.ttlMs;
    return { granted: true, token: this.token, expiresAt: this.expiresAt };
  }

  renew(node, now) {
    const dead = this.holder !== node || now >= this.expiresAt;
    if (dead) return { granted: false };
    this.expiresAt = now + this.ttlMs;
    return { granted: true, token: this.token };
  }
}

class FencedStorage {
  constructor() { this.highestSeen = 0; this.log = []; }
  write(token, what) {
    if (token < this.highestSeen) {
      return { ok: false, why: `stale token ${token} < ${this.highestSeen}` };
    }
    this.highestSeen = token;
    this.log.push(`${what} (token ${token})`);
    return { ok: true };
  }
}
```

actual output — A's late write is rejected

TEXT

```

t=0      A acquires: { granted: true, token: 1, expiresAt: 10000 }
t=0      A writes  : { ok: true }
t=12000 B acquires: { granted: true, token: 2, expiresAt: 22000 }
t=12000 B writes  : { ok: true }
t=12001 A writes  : { ok: false, why: 'stale token 1 < 2' }
log: [ 'A: set x=1 (token 1)', 'B: set x=2 (token 2)' ]

```

THREE RULES FOR SAFE FAILOVER

1. **Lease, not lock.** A lease expires by itself. The holder must keep renewing. If the holder freezes, the lease dies on its own with no help from anybody.
2. **Fencing token, always increasing.** Storage rejects anything below the highest token it has seen. This is what actually prevents corruption.
3. **Lease TTL > 2 x renew interval, and TTL > worst expected pause.** If TTL is 10 s, renew every 3 s. A 4-second GC pause must not cost you leadership — but a 12-second one must.

TRAP

Split brain is not prevented by “the other node should have known”. It is prevented by arithmetic: a leader must be elected by a **majority** ($\lfloor N/2 \rfloor + 1$ nodes), so two majorities cannot exist at once, and by a fencing token so that even a confused old leader cannot land a write. With 5 nodes the majority is 3, so a 2-node minority can never elect anybody. **This is why consensus clusters have an odd number of members.**

PRACTICE · TIER 1 target: 30 min

1. Extend ConsistentHashRing with `removeNode` and measure what fraction of keys move when you remove 1 node of 11. Predict the answer before you run it.
2. In QuorumStore, add **sloppy quorum**: if a replica is down, write to the next healthy node on the ring instead, tagged with the intended owner. Then write `handoff()` to replay those writes when the owner returns.
3. Write `resolveCart(a, b)` that takes two carts with version vectors, returns the merged cart when they are CONCURRENT, and returns the newer one otherwise. Handle removals with tombstones.
4. Convert HashRouter to the 1,024-bucket scheme. Then write `split(fromShard, toShard, n)` that moves n buckets and prints which key ranges moved.
5. In LeaseService, add a `now` that you control, and write a test proving that a node whose renew is delayed by 11 s with a 10 s TTL loses leadership and is then fenced.

ANSWER KEY

1. Removing 1 of 11 moves about $1/11 = 9.1\%$ of keys — all of them from the removed node, spread over the other 10. Removal is the mirror image of addition.
2. Sloppy quorum trades consistency for write availability: the write is durable but is **not** on a node a normal read would ask. You must run `handoff`, and until you do, $W + R > N$ no longer guarantees an overlap. Say that cost out loud; it is the whole point.
3. CONCURRENT → union of items, sum of quantities, and a removed item stays removed if its tombstone version is newer than its add. Not CONCURRENT → take the newer whole cart. Never blend a newer and an older cart field by field.
4. Moving n buckets moves exactly $n/1024$ of the keys and **no key changes bucket**. That is the property you are buying.
5. The renew at $t = 11,000$ must fail because `now >= this.expiresAt (10,000)`. The node must then stop serving **immediately**, before it even learns who the new leader is. Code that keeps serving “until it hears otherwise” is the bug.

Tier 2 — mid-scale: shard a marketplace order database

Example 14 [Shopee · pattern]

A marketplace has outgrown a single Postgres box for orders. Design the sharded order store. Buyers check “my orders” constantly; sellers need a dashboard; flash sales create 10x spikes.

SOLUTION**Step 1 — Clarify**

Question I ask	Why the answer changes the design
How many orders per day, and what is the flash-sale peak?	Decides shard count and whether the peak or the average sizes the cluster.
What are the top three queries?	The shard key must make the most frequent query a single-shard query.
How long must orders stay online?	Sets total size, which is usually what actually forces sharding.
Do sellers need live dashboards or 5-minute-old ones?	Live means cross-shard fan-out on the hot path. Stale means a separate read model.
Is there a cross-order transaction? (a basket paid as one payment)	If yes, those orders must share a shard, or we need a saga.
Can an order ever change buyer?	If the shard key can change, a row must move shards — a much harder design.

Answers I will assume, said out loud: 6 million orders/day; peak is 10x average during a 2-hour flash sale; top queries are (a) one order by id, (b) my last 20 orders, (c) a seller’s orders today; 3 years of orders stay online, older go to cold storage; seller dashboards may lag 60 seconds; a basket is one order, so no cross-order transaction; an order’s buyer never changes.

Step 2 — Scale estimate**Writes.**

$$6,000,000/86,400 = 69.4 \text{ orders/second average}$$

$$69.4 \times 10 = 694 \text{ orders/second at flash-sale peak}$$

Each order also writes about 3 item rows and 1 payment row, so the true row rate is $694 \times 5 = 3,470$ rows/second at peak.

Reads. Buyers poll order status. Assume 20 status reads over an order’s life:

$$6,000,000 \times 20 = 120,000,000 \text{ reads/day}$$

$$120,000,000/86,400 = 1,388.9 \approx 1,389 \text{ reads/second average}$$

$$1,389 \times 3 = 4,167 \text{ reads/second at peak}$$

Storage. One order plus its items and payment row: about 1,000 bytes.

$$6,000,000 \times 1,000 = 6,000,000,000 \text{ B} = 6 \text{ GB/day}$$

$$6 \times 365 = 2,190 \text{ GB/year} = 2.19 \text{ TB/year}$$

$$2.19 \times 3 = 6.57 \text{ TB online}$$

At replication factor 3: $6.57 \times 3 = 19.71 \text{ TB}$ of replicated data.

Now the shard count — and this is the interesting part.

shards needed for writes = $3,470/5,000 = 0.69 \rightarrow 1$

shards needed for reads = $4,167/20,000 = 0.21 \rightarrow 1$

shards needed for size = $6.57 \text{ TB} / 0.5 \text{ TB per shard} = 13.1 \rightarrow 16$

THE ONE-LINE SUMMARY FOR THE BOARD

694 orders/s peak · 4,167 reads/s peak · 6.57 TB online · **16 shards of 411 GB each.**

We are not sharding for throughput. One machine could take this load. We are sharding for **size and blast radius**. Say that sentence; it is the difference between a candidate who read a blog and one who has operated a database.

Why size, not QPS, is the real reason

$6,570 \text{ GB} / 16 = 410.6 \approx 411 \text{ GB per shard}$

Restore speed from backup is about 200 MB/s.

one 6.57 TB database : $6,570,000 \text{ MB} / 200 = 32,850 \text{ s} = 9.1 \text{ hours}$

one 411 GB shard : $410,600 \text{ MB} / 200 = 2,053 \text{ s} = 34 \text{ minutes}$

Nine hours of downtime versus 34 minutes, and in the sharded case only 1/16 of your buyers are affected while it happens. **That** is the argument for sharding at this scale, and it is an argument about operations, not about QPS.

Step 3 — API surface

five endpoints; note which ones carry the shard key

TEXT

```
POST /v1/orders
  header Idempotency-Key: 5f1c-...          // retry-safe: no duplicate orders
  body   { buyerId, sellerId, items:[{sku,qty,priceCents}], addressId, promo }
  201    { orderId, state: "CREATED", totalCents }
  note   orderId ENCODES the shard: "16-3f2a9c..." = bucket 16. See step 4.

GET  /v1/orders/{orderId}
  200   { orderId, buyerId, sellerId, state, items[], totalCents, updatedAt }
  note  single shard: the id tells the router where to go. ~2 ms.

GET  /v1/buyers/{buyerId}/orders?limit=20&cursor=<opaque>
  200   { orders:[...], nextCursor }
  note  single shard: buyerId IS the shard key. The query we optimised for.

GET  /v1/sellers/{sellerId}/orders?day=2026-09-15&limit=50&cursor=<opaque>
  200   { orders:[...], nextCursor, asOf: "2026-09-15T10:02:00Z" }
  note  served from the SELLER READ MODEL, not the order shards. May lag 60 s.

POST /v1/orders/{orderId}/transitions
  body   { to: "PAID" | "SHIPPED" | "CANCELLED", reason }
  200    { orderId, state }
  409    { error: "illegal_transition", from: "CANCELLED", to: "SHIPPED" }
```

Step 4 — Data model and the shard key

The shard key is the whole design. Four candidates:

Candidate	What it makes fast	What it breaks	Verdict
-----------	--------------------	----------------	---------

order_id (random)	even spread; “get one order” is single-shard	“my orders” hits all 16 shards — 16 calls for the most common screen	no
created_at (range)	“all orders today” is one scan	every write goes to the newest shard: a permanent 100% hotspot	no
seller_id	seller dashboard is single-shard	one flash-sale seller puts most of the day’s writes on one shard	no
buyer_id (hashed)	“my orders” is single-shard; buyers are many and roughly equal in size	seller queries need a second read path	yes

Decision: hash buyer_id into 1,024 buckets; map buckets to 16 shards. Buyers are the natural unit of isolation — there are millions of them, none is 1% of traffic, and the highest-frequency query is scoped to exactly one.

the tables, with the keys marked

TEXT

```
orders
  order_id      char(26)    PK.  Format: <bucket:4><ulid:22>
  -- bucket is stored once and NEVER changes for the life of the row
  bucket        smallint    = hash(buyer_id) % 1024      SHARD KEY INPUT
  buyer_id     bigint       the logical shard key
  seller_id     bigint
  state         enum         CREATED|PAID|SHIPPED|DELIVERED|CANCELLED|REFUNDED
  total_cents   bigint       integers only. Never a float for money.
  created_at    timestamp
  updated_at    timestamp
  version       int          optimistic lock for state transitions
  -- "my last 20 orders", single shard, one scan
  INDEX (buyer_id, created_at DESC)

order_items                                           -- SAME shard as its order
  order_id      char(26)    PK part 1
  line_no       smallint    PK part 2
  sku, qty, price_cents, seller_id

order_events                                           -- append-only, SAME shard
  order_id, seq, from_state, to_state, actor, at
```

SHORTCUT

Put the bucket inside the id. `order_id = "0417" + ulid()` means any service holding an order id can route to the right shard with a substring, **with no lookup at all**. This one trick removes an entire directory service from the design. Say it; interviewers love it because it is cheap and obviously correct.

The seller read model is a separate store, written by a stream job:

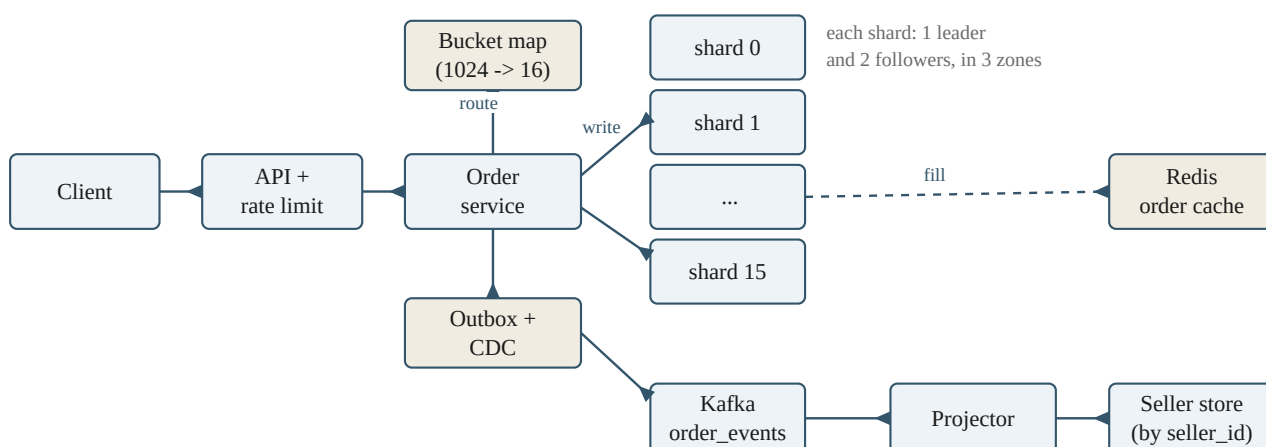
a second copy of the data, shaped for the other query

TEXT

```
seller_orders      (sharded by seller_id, in its own cluster)
  seller_id        bigint      SHARD KEY
  day              date        SORT KEY part 1
  order_id         char(26)     SORT KEY part 2
  state, total_cents, buyer_masked, updated_at
  -- one row per order, written by a consumer of the order_events stream
```

This is denormalisation on purpose. The same order exists twice, in two shapes, on two shard keys. That is the standard answer to “a sharded system cannot serve two different access patterns from one table” — **it cannot, so you keep two tables.**

Step 5 — Architecture



Solid = on the order’s critical path. Dashed = asynchronous, so a slow projector never slows a checkout.

Write path across the top. The seller read model is fed asynchronously, so it never slows an order down.

Trace one order creation:

1. `POST /v1/orders` arrives with an Idempotency-Key.
2. The order service computes `bucket = hash(buyerId) % 1024`, looks up `bucketMap[bucket] = shard 7`, and opens a transaction on shard 7’s leader.
3. Inside **one** transaction on **one** shard it writes: the `orders` row, the `order_items` rows, the first `order_events` row, and an outbox row. All four are on the same shard because they all key off the same `buyer_id`. This is a plain local ACID transaction — no two-phase commit anywhere.
4. Commit. Return 201 with the order id. Total: about 4 ms.
5. Separately, a CDC reader tails the outbox and publishes to Kafka. The projector builds the seller read model. If that pipeline is 60 seconds behind, no buyer notices.

Trace one “my orders”: the buyer id gives the bucket gives the shard. One query, `WHERE buyer_id = ? ORDER BY created_at DESC LIMIT 20`, served by an index on one shard, from a **follower**, in about 2 ms.

Step 6a — Deep dive: the cross-shard query you cannot avoid

Some queries genuinely need every shard: “find the order with this payment reference”, support tools, fraud sweeps. Scatter-gather is the only answer. Understand its cost.

WHY SCATTER-GATHER IS SLOWER THAN ANY SINGLE SHARD

You wait for the **slowest** of 16 answers, not the average.

If each shard independently answers within 20 ms with probability 0.99, then

$$P(\text{all 16 within 20 ms}) = 0.99^{16} = 0.851$$

So $1 - 0.851 = 14.9\%$ of scatter-gather requests are slower than 20 ms, even though only 1% of single-shard requests are.

With 64 shards it gets worse:

$$0.99^{64} = 0.526$$

Nearly **half** of all requests hit at least one slow shard.

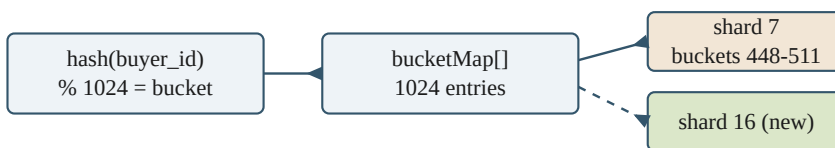
What we do about it:

1. **Do not do it on the hot path.** Buyer and seller screens are single-shard by design. The fan-out is for support and batch tools only.
2. **Give it a hedge.** If a shard has not answered in 15 ms, send the same query to that shard's follower too and take whichever returns first. This cuts the tail hard and costs about 5% extra read load.
3. **Give it a secondary index table.** Payment reference → order id, stored in its own small table sharded by payment reference. Then “find by payment ref” becomes **two** single-shard lookups instead of 16 parallel ones.

Decision: build the payment-reference index table. Two lookups at 2 ms each beats a 16-way fan-out with a 15% slow tail, and the index is 26 bytes per order — $6,000,000 \times 26 = 156 \text{ MB/day}$, $156 \times 365/1000 = 56.9 \text{ GB/year}$. Cheap.

Step 6b — Deep dive: growing from 16 shards to 32 with no downtime

Because keys hash to **buckets** and buckets map to shards, no key is ever rehashed. We move whole buckets.



The four phases, in order:



Rollback is available at every phase up to the flip: the old shard still holds a complete, current copy.

After the flip, keep the old copy read-only for 24 hours before deleting it. Deletion is the only irreversible step.

Split shard 7. Move 32 of its 64 buckets to the new shard 16. Keys never change bucket, so nothing is rehashed.

The numbers. Moving 32 of 1,024 buckets is $32/1024 = 3.1\%$ of the data:

$$6,570 \text{ GB} \times 0.031 = 203.7 \text{ GB to copy}$$

At 200 MB/s that is $203,700/200 = 1,018 \text{ s} \approx 17 \text{ minutes}$ of copying, done in the background while the system serves traffic normally.

TRAP

The flip must be atomic across every process that routes. If pod A thinks bucket 500 is on shard 7 and pod B thinks it is on shard 16, two buyers' orders land in two places and your “my orders” screen loses rows. Publish the bucket map with a **version number**, have every router refuse to serve with a stale version, and flip by bumping the version in one place (a config store with a watch). Dual-writing during phase 3 is exactly what makes a non-atomic flip survivable.

Step 6c — Deep dive: the flash sale, and where the hotspot really is

A seller starts a flash sale. 60% of the peak write load — $694 \times 0.6 = 416 \text{ orders/second}$ — is for that one seller's products.

Does that create a hot shard? No. We shard by `buyer_id`, and those 416 orders/s come from 416 **different buyers** spread over 1,024 buckets. Each shard sees about $416/16 = 26$ extra orders/second. Nothing notices.

So where is the pain? On the **stock counter** for one SKU. That is a single row that every one of those 416 buyers wants to decrement, and it lives in exactly one place. 416 transactions per second all locking the same row means they serialise: if the lock is held for 2 ms, the maximum possible rate on that row is $1/0.002 = 500/\text{s}$. We are at 416. One slow disk flush and we are over.

THREE FIXES FOR A SINGLE HOT ROW, WITH COSTS

1. **Split the counter.** Replace `stock = 1000` with 20 rows of `stock_shard(sku, i, qty=50)`. A buyer picks `i = random(0..19)` and decrements that row. Capacity becomes $20 \times 500 = 10,000/s$. **Cost:** “how many left?” is now a sum over 20 rows, and near the end some sub-counters are empty while others are not, so you must retry on a different `i` before declaring sold out.
2. **Reserve in Redis, settle in the database.** Decrement an in-memory counter (100,000 ops/s), and write the durable row asynchronously. **Cost:** if Redis dies you may oversell by the size of the in-flight window. Acceptable for T-shirts, not for concert seats.
3. **Queue the sale.** All flash-sale orders go into a single-partition queue keyed by SKU and are processed in order at whatever rate the row allows. **Cost:** buyers wait, and you must show them an honest “you are number 812 in the queue” screen.

Decision: fix 1 (split the counter into 20) as the default, plus fix 3 for sales of fewer than 100 units. Splitting is simple and gives 24x headroom. When the item count is tiny the sub-counter arithmetic gets fiddly and fairness matters more than throughput, so the queue wins there.

Step 7 — Trade-offs, failure modes, 10x

Choice	Option A	Option B	Decision
Shard key	<code>order_id</code> , perfect spread	<code>buyer_id</code> , hashed	B — the top query becomes single-shard
Routing	hash directly to 16 shards	hash to 1,024 buckets, map to shards	B — resharding stops being a rewrite
Seller queries	scatter-gather over 16 shards	a second store sharded by <code>seller_id</code>	B — 60 s of lag is cheaper than a 15% slow tail
Shard count	4 big shards	16 medium shards	B — 34-minute restore beats 9 hours; blast radius is 1/16
Replication	async only	semi-sync: wait for 1 of 2 followers	B — bounded data loss on failover for 1 ms of extra write latency
Cross-shard order	two-phase commit	one shard per order + saga if ever needed	B — 2PC blocks when the coordinator dies; we designed the need away
ID format	random UUID	bucket prefix + ULID	B — routing with no lookup, and ULIDs sort by time

What fails	What the user sees	What we do
One shard’s leader dies	1/16 of buyers get errors for 20 s	Semi-sync means a follower has every committed write. Promote it, bump the fencing token, repoint the bucket map. Lost data: none committed.

A follower lags 5 minutes	Some buyers see an old order state	Health check on lag seconds , not CPU. Pull a lagging follower out of the read pool automatically at 10 s.
The projector stops	Seller dashboards freeze at a timestamp	The asOf field in the API means the UI can say “as of 10:02”. Kafka retains the events, so the projector catches up with no data loss.
Bucket map flip is half-applied	Some orders written to the wrong shard	Routers refuse to serve on a stale map version. Dual-write during migration means the old shard still has everything, so recovery is a re-copy, not a data-loss event.
Flash sale on one SKU	Checkout slows for that item only	Split counter (20 sub-rows) gives 10,000/s of headroom against 416/s of demand.
Whole availability zone dies	Nothing, if replicas are placed correctly	Each shard’s 3 replicas live in 3 different zones. A zone loss costs every shard one replica and no shard its quorum.

At 10x (60 million orders/day):

- Writes become $60,000,000/86,400 = 694/s$ average, $6,944/s$ at flash peak, and $6,944 \times 5 = 34,720$ rows/second. Now throughput **does** matter: $34,720/5,000 = 6.9 \rightarrow 7$ shards minimum on write capacity alone.
- Storage becomes 21.9 TB/year and 65.7 TB over 3 years. $65,700/411 = 160$ shards at the same shard size. Round to **128 or 256** (powers of two keep bucket splits clean), with 1,024 buckets still the routing unit — at 256 shards that is only 4 buckets per shard, so **raise the bucket count to 8,192 before you get there**. Bucket count is the one number that is genuinely painful to change later, so over-provision it on day one: 8,192 buckets cost nothing today.
- The genuinely new problem at 10x is **archival**. 65.7 TB is mostly orders nobody will ever read again. Move anything older than 90 days and in a terminal state to cold object storage, keyed by order id, and leave a 40-byte stub row behind. If 85% of rows qualify, the online set drops to $65.7 \times 0.15 = 9.9$ TB, and the shard count drops back to $9,900/411 = 24$. **One archival policy saved you 130 shards**. That is a better answer than any amount of clever sharding.

Example 15 [Grab · pattern]

Your shard key is `buyer_id`. Product now wants “show me every order from my **household**”, where a household is up to 6 buyer accounts that can be linked and unlinked at any time. What breaks and what do you do?

SOLUTION

What breaks. Six buyer ids hash to six buckets, which can sit on up to six different shards. “Household orders” becomes a 6-way scatter-gather with a merge and a re-sort, on what the product team thinks is a normal screen.

Option A — fan out at read time. Query up to 6 shards in parallel, merge by `created_at`, paginate in the application. Works immediately, no data moves. **Cost:** the tail. With per-shard 99th percentile at 20 ms, $P(\text{all 6 fast}) = 0.99^6 = 0.941$, so 5.9% of these requests are slow. And cursor pagination across a merge is genuinely fiddly — the cursor must carry a position **per shard**.

Option B — shard by household_id instead. Every household is one shard, one query. **Cost:** households are created and **changed** after the fact. Linking two accounts would mean physically moving one person’s entire order history between shards. A shard key that can change is the one thing you must never choose.

Option C — a household read model. Same trick as the seller dashboard: a projector consumes order events and writes `household_orders`, sharded by `household_id`. Linking a new member triggers a backfill job for that one household. **Cost:** one more store, 60 s of lag, and a backfill on link.

Decision: C, with A as the fallback for the first 90 days. Ship A because it needs no new infrastructure and households are rare enough that 5.9% slow requests on a rarely-used screen is survivable. Build C once usage proves the screen matters. **Never B** — the moment a shard key can change, every design gets ten times harder, and “unlink” would leave orphaned rows on the wrong shard forever.

PRACTICE · TIER 2 target: 35 min

1. Redo the shard-count arithmetic for 20 million orders/day, 5 years online, 1.4 KB per order, and a target shard size of 600 GB. Show every division.
2. You must add “search orders by phone number” for support staff. Compare a 16-way scatter-gather against a phone $\rightarrow$ order_id index table: latency, storage, and write cost. Decide.
3. Your bucket count is 1,024 and you now need 300 shards. Explain exactly what goes wrong and what the migration to 8,192 buckets looks like.
4. Semi-sync replication waits for 1 of 2 followers. Compute the write latency if the local follower is 0.5 ms away and the remote one is 60 ms away, and say how much data a failover can lose.
5. Design the dual-write verification for step 3 of the resharding plan: what do you compare, how often, and what do you do on a mismatch?

ANSWER KEY

1. $20,000,000 \times 1,400 = 28,000,000,000 \text{ B} = 28 \text{ GB/day}; \times 365 = 10,220 \text{ GB} = 10.22 \text{ TB/year}; \times 5 = 51.1 \text{ TB}; 51,100/600 = 85.2 \rightarrow$ **128 shards** (round up to a power of two). Writes: $20,000,000/86,400 = 231.5/\text{s}$ average, 2,315/s at 10x peak, $\times 5$ rows = 11,575 rows/s, needing only $11,575/5,000 = 2.3 \rightarrow 3$ shards. Size wins again, by 40x.
2. Index table: 2 lookups $\times 2 \text{ ms} = 4 \text{ ms}$, 30 B per order, and one extra write per order. Scatter-gather: 16 parallel queries, $0.99^{16} = 0.851$ so 14.9% are slow, no extra storage, no extra writes. **Decision: index table.** Support tools are used by humans who wait, but a 15% slow rate on a tool used 500 times a day is 75 slow lookups a day and the storage cost is $20,000,000 \times 30 = 600 \text{ MB/day}$. Cheap enough.
3. 1,024 buckets over 300 shards is 3.4 buckets per shard, so the **smallest unit you can move is 0.33% of the data** and shard sizes cannot be balanced to better than $\pm 15\%$. Migration: add a second level — `bucket8k = hash(key) % 8192`, and `bucket1k = bucket8k % 1024`, so every old bucket splits into exactly 8 new ones with no key changing shard. Deploy the new map, verify it is identical in effect, then start moving the finer buckets.
4. Semi-sync waits for the **first** ack, which is the local follower at 0.5 ms. Write latency $\approx 2 + 0.5 = 2.5 \text{ ms}$. A failover to the **local** follower loses nothing. A failover to the **remote** one, which may be 60 ms behind, loses up to $60 \text{ ms} \times 3,470 \text{ rows/s} = 208 \text{ rows}$. Mitigation: only ever promote a follower whose lag is under 100 ms, and if none qualifies, stay down and wait rather than silently losing 208 orders.
5. Compare row counts per bucket every 60 s, and a checksum of (`order_id`, `updated_at`, `version`) for a random 1% sample every 5 minutes. On mismatch: stop the flip, do not roll back writes (the old shard is still authoritative), log the differing ids, re-copy just those buckets. Alarm if mismatches exceed 0 — for this comparison the only acceptable number is zero.

Tier 3 — large scale: a four-region profile store

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 16 [Google · pattern]

800 million users. Every service call reads the user’s profile and settings. Users travel. Design a store replicated across Singapore, Mumbai, Frankfurt and Virginia. Decide where it sits on CAP, and defend it.

SOLUTION

Step 1 — Clarify

Question I ask	Why it changes the design
What exactly is in a “profile”?	Display data is AP-safe. A username is a global uniqueness claim and is not.

Read-to-write ratio?	If it is 80:1, this is a read-path problem and the write path can be slow.
Do users move between regions often?	Rarely → pin a home region. Often → every region must serve everybody.
How stale may a read be, in seconds?	Sets whether local reads are allowed at all.
What happens if two regions write the same field at once?	Decides last-write-wins versus a merge function versus a single leader.
Is there any field that must be globally unique?	If yes, that one field needs consensus and the rest do not.

Answers assumed: profile = display name, avatar URL, language, 40-ish settings, plus **username** (globally unique) and **email** (globally unique). 200 million DAU, 40 profile reads per active user per day, 0.5 writes. **5 seconds of staleness is acceptable for display fields. Username and email must never be issued twice.** A user's home region rarely changes.

Step 2 — Scale estimate

Reads.

$$\begin{aligned}
 200,000,000 \times 40 &= 8,000,000,000 \text{ reads/day} \\
 8,000,000,000 / 86,400 &= 92,592.6 \approx 92,593 \text{ reads/second average} \\
 92,593 \times 3 &= 277,779 \text{ reads/second at peak}
 \end{aligned}$$

Writes.

$$\begin{aligned}
 200,000,000 \times 0.5 &= 100,000,000 \text{ writes/day} \\
 100,000,000 / 86,400 &= 1,157.4 \approx 1,157 \text{ writes/second average} \\
 1,157 \times 3 &= 3,471 \text{ writes/second at peak}
 \end{aligned}$$

Read-to-write ratio: $92,593 / 1,157 = 80.0$. **Eighty to one.** This is a read-path design and we should spend our whole latency budget there.

Storage. 2 KB per user.

$$800,000,000 \times 2,048 = 1,638,400,000,000 \text{ B} = 1.64 \text{ TB logical}$$

Three replicas per region, four regions:

$$1.64 \times 3 \times 4 = 19.7 \text{ TB of raw replicated data}$$

That is small. **The problem here is not size. It is latency and coordination.**

Nodes for reads. At 20,000 reads/s/node, per region at peak:

$$277,779 / 20,000 = 13.9 \rightarrow 16 \text{ nodes per region for the store}$$

But most reads should never reach the store at all — see the cache arithmetic in step 6.

Shards. $1.64 \text{ TB} / 0.5 \text{ TB per shard} = 3.3 \rightarrow 4$ shards is enough for **size**, but $277,779 / 20,000 = 13.9$ shards is what **throughput** needs. Take 16.

THE BOARD SUMMARY

92,593 reads/s average · 277,779 at peak · 1,157 writes/s · 80:1 read-heavy · 1.64 TB logical · 16 shards x 3 replicas x 4 regions.

Small data, enormous read rate, four regions. Therefore the design is about **where a read is allowed to be answered**, and nothing else.

Step 3 — API surface

```

GET /v1/profiles/{userId}?consistency=local|quorum
  200 { userId, displayName, avatarUrl, lang, settings{}, version, asOf }
  note local = read from this region's replicas. ~2 ms. may be ~1 s stale
        quorum = read a majority across regions. ~120 ms. never stale
        DEFAULT is local. The caller opts in to the slow, correct one.

PATCH /v1/profiles/{userId}
  body { displayName?, avatarUrl?, lang?, settings? , ifVersion: 8124 }
  200 { version: 8125, asOf }
  409 { error: "version_conflict", current: {...} }
  note returns the FULL new profile: the client needs no read-after-write

POST /v1/usernames
  body { userId, username }
  201 { username, userId }
  409 { error: "taken" }
  note this endpoint is LINEARIZABLE and costs ~200 ms. It is rare, so fine.

GET /v1/usernames/{username}
  // served from a local cache; a stale hit is harmless here
  200 { userId }
  404

```

SHORTCUT

Putting consistency in the API is a senior move. It says out loud that consistency is a per-call decision with a price, not a global setting. Then give the default: `local`, because 80 of every 81 calls are reads of display data where one second of staleness is invisible.

Step 4 — Data model, split by CAP requirement

```

-- STORE 1: profiles. AP. Leaderless, quorum-tunable, in all 4 regions.
profiles
  user_id      bigint      PARTITION KEY (hashed)
  display_name text
  avatar_url   text
  lang         text
  settings     map<text,text>
  -- version vector: {"sg":12,"in":3,"de":0,"us":7}
  vv          map<text,int>
  -- for humans and debugging, NOT for conflict resolution
  updated_at   timestamp
  home_region  text        -- where this user usually is

-- STORE 2: unique claims. CP. One consensus group per shard, majority writes.
username_claims
  username     text        PRIMARY KEY (lowercased, normalised)
  user_id      bigint
  claimed_at   timestamp
  -- written ONLY with compare-and-set: "insert if absent". Never LWW.

```

email_claims

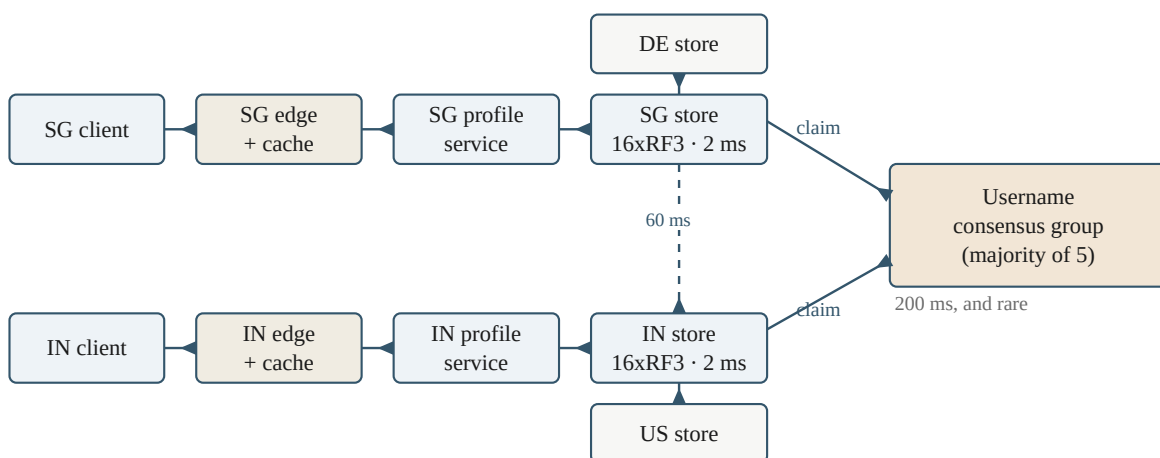
-- same shape, same rules

THE SPLIT IS THE ANSWER TO THE CAP QUESTION

- **Profiles: AP.** During a partition, Frankfurt keeps serving and keeps accepting writes. A German user sees her own change instantly; a Singapore reader may be a few seconds behind. Conflicts are resolved per field by version vector, with LWW on scalars and union on the settings map.
- **Username claims: CP.** During a partition, a minority region **cannot** issue a username. It returns 503 `try_again`. We would rather a user wait 30 seconds than discover that two people own @asha.

Same product. Same request, almost. **Different answers, because the cost of being wrong is different.** This is the whole content of CAP as an interview topic.

Step 5 — Architecture



Solid = synchronous, on the user's critical path. Dashed = asynchronous replication between regions. A read never crosses an ocean. A username claim always does. 80 of every 81 calls take the fast path.

Reads stay inside a region. Profile writes replicate asynchronously. Only a username claim leaves the region synchronously.

Step 6a — Deep dive: making 277,779 reads/second cheap

The store at 16 nodes per region can serve $16 \times 20,000 = 320,000$ reads/s, which covers peak — but only just, and only if the load is perfectly even. Put a cache in front and the arithmetic stops being scary.

Cache sizing. Assume in any 10-minute window, 30 million distinct users are active in a region.

$$30,000,000 \times 2,048 \text{ B} = 61,440,000,000 \text{ B} = 61.4 \text{ GB}$$

Split over 8 Redis nodes: $61.4/8 = 7.7 \text{ GB}$ each. Comfortable.

Hit rate and what reaches the store. At a 95% hit rate:

$$277,779 \times 0.05 = 13,889 \text{ reads/second reach the store}$$

$$13,889/20,000 = 0.69 \rightarrow 1 \text{ node's worth of work}$$

We keep 16 nodes for **storage** and **failure tolerance**, not for read throughput. Now a bad day — a cache node loss, say — has 16x headroom instead of 1.15x.

The cold-cache case, which is the one that actually breaks systems. If the whole cache tier restarts, all 277,779 reads/s hit a store sized to comfortably do 320,000. It survives, but with no margin. Two defences, both cheap:

1. **Request coalescing.** 4,000 simultaneous requests for the same missing key become **one** store read; the rest wait on the same in-flight promise.

2. **Staggered restart.** Never restart more than 2 of 8 cache nodes at once, and warm each one from the most-read-1-million list before putting it in rotation: $1,000,000 \times 2,048 = 2.05$ GB, which loads in well under a minute.

Step 6b — Deep dive: read-your-writes across 60 ms of ocean

A user in Mumbai changes her display name. Replication to Singapore takes 60 ms, sometimes 2 s. She reloads. What does she see?

Approach	How	Cost	Verdict
Return the new value	PATCH responds with the complete new profile	nothing	always do this
Sticky region	her session pins to her home region for all reads and writes	a user who travels gets 200 ms reads until the pin moves	yes , with a 24 h pin
Version token	the write returns <code>version: 8125</code> ; reads send <code>minVersion: 8125</code> ; a behind replica forwards to the leader	one header everywhere, plus a forward path	yes , for the 3 screens that matter
Quorum read always	every read waits for a majority across regions	$92,593$ reads/s $\times$ 120 ms of extra latency, and a 4x cost increase	no

Decision: all three of the first three, in that order. They are cumulative and cheap. The fourth costs 120 ms on 8 billion daily reads to fix a problem that affects the 100 million daily **writers** for about one second each. That is a 60:1 bad trade and you should say the ratio out loud.

Step 6c — Deep dive: why last-write-wins on wall clocks loses data

Two servers write the same field. LWW keeps the one with the larger timestamp.

THE FAILURE, IN NUMBERS

Singapore's clock reads 10:00:00.100. Mumbai's clock reads 10:00:00.060 — it is 40 ms behind, which is **normal** for NTP-synchronised machines and can be far worse.

1. At true time T , Singapore writes `lang = en`, stamped 10:00:00.100.
2. At true time $T + 20$ ms — genuinely **later** — Mumbai writes `lang = hi`, stamped 10:00:00.080.
3. LWW compares 100 versus 80 and keeps `en`.

The newer write was silently deleted. No error, no log line, no conflict. The user set her language to Hindi and the system quietly set it back.

What we do instead:

1. **Per-field version vectors.** Only fields that were **actually** written concurrently are conflicts. Two regions editing two different settings is not a conflict at all, and field-level vectors make that free.
2. **For the settings map, merge by union with per-key vectors.** Losing a setting is worse than keeping a stale one.
3. **For true scalars where a merge makes no sense (display name), LWW is acceptable**, but use a **logical** timestamp (the version vector's own counters plus a region tiebreak), never the wall clock.
4. **For anything where a lost write costs money or identity, do not resolve at all — prevent it.** That data belongs in store 2 with consensus.

TRAP

“We will just use NTP and keep clocks within 10 ms” is not a fix, it is a hope. NTP fails silently, virtual machines get paused and resume with a jumped clock, and leap seconds exist. Systems that **do** use clocks for ordering (Google’s TrueTime is the famous one) do not trust a timestamp — they carry an explicit **uncertainty interval** and deliberately **wait** until the interval has passed before committing. Waiting on purpose is the price of trusting a clock. Say that; it shows you know why the naive version fails.

Step 6d — Deep dive: the username claim, which is the only CP thing here

A username claim must be linearizable: if @asha is issued once, it is never issued again, even during a partition, even if two regions get the request in the same millisecond.

THE CLAIM PATH, STEP BY STEP

1. The username is **normalised** first: lowercase, strip zero-width characters, map confusable characters (rn looks like m, Cyrillic a looks like Latin a). Normalise **before** hashing, or two different byte strings claim “the same” name.
2. Hash the normalised name to one of 16 consensus groups. Each group is 5 nodes spread over the 4 regions (2 + 1 + 1 + 1).
3. The claim is a **compare-and-set**: “create this key only if absent”. The group’s leader proposes it; a majority of 3 of 5 must accept.
4. Latency: the leader must hear from 2 more nodes. The second-nearest is typically 60–180 ms away, so a claim costs roughly **200 ms**.
5. Availability: a region cut off from the others holds at most 2 of 5 nodes, cannot reach 3, and therefore **cannot issue a username**. It returns 503. That is the CP choice, working exactly as designed.

Is 200 ms acceptable? A user claims a username once, at sign-up, on a screen where they are already typing. 200 ms is invisible there. Compare with profile reads: 8 billion a day, where 200 ms would be catastrophic. **Same system, two answers, and the reason is the request rate and the cost of being wrong.**

Step 7 — Trade-offs, failure modes, 10x

Choice	Option A	Option B	Decision
Profile CAP side	CP: quorum across regions	AP: local quorum, async cross-region	B — 80:1 reads; 1 s of staleness on a display name harms nobody
Username CAP side	AP with conflict resolution	CP with consensus	A is not even coherent — two owners of one name cannot be merged. B
Conflict resolution	wall-clock LWW	per-field version vectors	B — clock skew silently deletes newer writes
Read default	quorum, always correct	local, opt-in quorum	B — and put the knob in the API so callers choose per call
Cache	none, the store is fast	8-node regional cache at 95% hit	B — it turns 1.15x headroom into 16x for a few GB of RAM

Settings merge	replace the whole map	union with per-key vectors	B — replacing a map loses a setting the other region just added
Region routing	nearest region by latency	home region, repinned after 24 h	B — stable routing gives free read-your-writes for 99% of users

What fails	What the user sees	What we do
One region is cut off	Profiles: fully working, slightly stale. Usernames: 503 in that region.	This is the design working. Announce it, do not “fix” it — the alternative is duplicate usernames.
Cross-region link is slow (60 ms → 900 ms)	Nothing, for reads. Cross-region convergence takes 1 s instead of 60 ms.	Alarm on replication lag in seconds. Async replication absorbs this completely; it is exactly why we chose it.
Clocks skew by 5 minutes	Nothing	Because we resolve with version vectors, not clocks. If we had used LWW, five minutes of one region’s writes would vanish.
A consensus group loses its leader	Username claims pause for 5 s in every region	Election takes one or two election timeouts. Keep the timeout at 1–2 s, and the fencing token stops the old leader’s late writes.
Cache tier restarts	p99 read goes from 3 ms to 15 ms for 60 s	Coalescing plus staggered warm restart. The store has 16x headroom because we sized it for storage.
A “hot” celebrity profile	Nothing	It is one cache key in every region — the best case. Add a 5-second in-process cache on the service if a single Redis key exceeds 50,000 ops/s.
Two regions rename the same user at once	One name wins for display; nothing is lost in settings	Version vectors mark it CONCURRENT; scalars take the region-tiebreak winner, the settings map takes the union.

At 10x (2 billion DAU-equivalent reads × 10 = 80 billion reads/day):

- $80,000,000,000/86,400 = 925,926$ reads/s average, 2,777,778 at peak.
- No cache tier answers 2.8 million requests/s cheaply from 8 nodes. The fix is not a bigger cache; it is **moving the read closer**: an in-process LRU inside every profile-service pod, holding the 100,000 hottest users. At a 60% in-process hit rate, the Redis tier sees $2,777,778 \times 0.4 = 1,111,111/s$, and at Redis’s 95% hit rate the store sees $1,111,111 \times 0.05 = 55,556/s$, which is $55,556/20,000 = 2.8 \rightarrow 3$ nodes of real work. **Three cache layers, each removing an order of magnitude.**
- The new hard problem is **invalidation**. With an in-process cache in 3,000 pods, a profile edit must reach 3,000 processes. Broadcasting every edit is $3,471 \times 3,000 = 10,413,000$ messages/second at peak, which is absurd. **Decision: do not invalidate — expire.** A 5-second TTL on the in-process layer bounds staleness to 5 seconds with zero messages, and we already agreed that 5 seconds is acceptable for display fields. The three screens that need better use the version-token path from step 6b and skip the in-process layer entirely.

- Storage is still tiny (16.4 TB logical at 10x users). **Say that out loud** — it stops the interviewer from expecting a storage answer and keeps the conversation on latency, which is where the real design is.

Example 17 [Amazon · pattern]

Your AP profile store is asked to also hold a per-user **credit balance** that services deduct from. A product manager says “just put it in the profile, it is one more field”. Explain, with arithmetic, why that is wrong, and give the design that works.

SOLUTION

Why it is wrong. A balance is not a value you overwrite; it is a value you **modify based on what you last read**. Under AP replication two regions can both read 100, both deduct 30, and both write 70. The user spent 60 and lost 30.

How often would that actually happen? Suppose a user spends twice a day and replication convergence takes 1 second. The window in which a second deduction can see a stale balance is 1 second out of the gap between the two spends. If the two spends are independent within a day:

$$P(\text{second spend lands inside the 1 s window}) \approx 1/86,400 = 1.16 \times 10^{-5}$$

With 200 million daily active users spending twice a day, that is

$$200,000,000 \times 1.16 \times 10^{-5} = 2,320 \text{ lost deductions per day}$$

At an average 50 rupees each, that is 116,000 rupees a day of money given away, every day, silently. “Rare” is not the same as “acceptable” when you multiply by 200 million.

What works.

1. **Move balances into store 2** (the CP store), sharded by `user_id`, one consensus group per shard. A deduction is a compare-and-set on (`balance`, `version`), retried on conflict. Cost: 200 ms for a cross-region write, and unavailability in a cut-off region.
2. **Or make the operation commutative.** Store the balance as an append-only **ledger** of signed entries, and define the balance as their sum. Two concurrent deductions are then two entries, and the sum is correct with no coordination at all. Cost: you must still refuse a spend that would go below zero, and **that** check needs coordination — unless you pre-allocate.
3. **Or pre-allocate.** Each region holds a **reservation** of the user’s balance — say 40% each for the two regions the user actually uses. A spend inside your region’s reservation needs no coordination at all. Running low triggers a slow, coordinated top-up.

Decision: the ledger (option 2) plus a reservation (option 3), with the balance floor enforced by the reservation. Spends are then local and fast in the normal case, the ledger means no deduction is ever lost, and coordination happens only when a region’s reservation runs low — which for a typical user is a few times a month, not twice a day. Option 1 is the honest fallback if the product cannot tolerate a user being told “insufficient balance” while 60% of their money is reserved in a region they are not in.

Example 18 [Microsoft · pattern]

Your 5-node consensus group is spread 2-1-1-1 over four regions. The link between Singapore (2 nodes) and everywhere else fails. Walk through exactly what happens, and then explain why a 2-2-1 layout would have been worse.

SOLUTION

What happens with 2-1-1-1.

1. Majority is $\lfloor 5/2 \rfloor + 1 = 3$.
2. Singapore’s side has 2 nodes. $2 < 3$, so it **cannot** elect a leader and cannot commit anything. If the old leader was in Singapore, it loses its lease within one TTL and steps down.
3. The other side has $1 + 1 + 1 = 3$ nodes. $3 \geq 3$, so it elects a leader and keeps committing. The service is **available** everywhere except for clients that can only reach Singapore.
4. Writes committed by the majority before the split are safe. Writes the Singapore leader had proposed but not committed are discarded when it rejoins — which is correct, because they were never acknowledged to any client.

5. Fencing: the new leader's term is higher, so the old Singapore leader's late messages are rejected on arrival.

Why 2-2-1 is worse. Now suppose the split puts the two 2-node regions on one side and everything else on the other. One side has $2 + 1 = 3$ and can still work — fine. But the **other** common failure, losing the single 1-node region entirely, leaves $2 + 2 = 4$ nodes which is still a majority, so that case is fine too. The real problem with 2-2-1 is different: **any two-region outage** can leave $2 + 1 = 3$ or $2 + 2 = 4$, but the combination of “lose the 1-node region” **and** “one node in a 2-node region reboots for patching” leaves $2 + 1 = 3$ — exactly at the edge, with zero spare. With 2-1-1-1 the same double event leaves $2 + 1 + 1 = 4$ or $1 + 1 + 1 = 3$ depending on which pieces are lost, and crucially **no single region holds more than 2 of 5**, so no single region's loss can ever take you below

3.

The rule to state: place consensus members so that **no single failure domain holds a majority, and no single failure domain's loss drops you below a majority**. With 5 nodes and 4 regions, 2-1-1-1 is the only layout that satisfies both. With 3 regions, use 5 nodes as 2-2-1 and accept that losing a 2-node region leaves you with exactly 3 — then **add a sixth node? No**. Six is worse than five: majority becomes 4, so you still only tolerate 2 failures, and you have paid for an extra node and an extra round trip. **Consensus group sizes should be odd. Always.**

PRACTICE · TIER 3 target: 45 min

1. Recompute the whole read path if the cache hit rate is 85% instead of 95%. How many store nodes per region do you now need at peak, and what does that do to the cost argument?
2. A partition lasts 20 minutes. Mumbai took 1,157 writes/s locally the whole time. How many writes must converge when the link returns, how long does convergence take at 3,000 writes/s of repair bandwidth, and what does the user see meanwhile?
3. Design the migration that moves `credit_balance` out of the AP profile store and into the CP store with zero lost money and zero downtime. List the phases and the verification.
4. Your consensus group's leader is in Virginia but 70% of username claims originate in Singapore. Compute the latency each way and decide whether to move the leader, shard differently, or leave it.
5. Write the exact sentence you would say when an interviewer asks “is your system CP or AP?” — in under 40 words, covering both stores.

ANSWER KEY

1. Store load = $277,779 \times 0.15 = 41,667/\text{s}$, so $41,667/20,000 = 2.1 \rightarrow 3$ nodes of **read work**, against 16 nodes provisioned. Headroom falls from 16x to 5.3x. Still fine — which is the real lesson: when the store is sized by **storage and replication** rather than by QPS, a 10-point swing in hit rate is a non-event. Design so that your scariest number is not the one you can least predict.
2. $1,157 \times 20 \times 60 = 1,388,400$ writes to ship. At 3,000/s of repair capacity **on top of** the live 1,157/s, the spare is $3,000 - 1,157 = 1,843/\text{s}$, so $1,388,400/1,843 = 753 \text{ s} \approx 12.6$ minutes. Meanwhile users outside Mumbai see up to 20 minutes of staleness on profiles edited in Mumbai, and Mumbai users see staleness on everyone else's. Nobody sees an error. That is AP behaving as designed.
3. Phases: (1) dual-write every balance change to both stores, CP store authoritative for **nothing** yet; (2) backfill by replaying the ledger into the CP store and compare totals per user; (3) flip reads to the CP store, keep dual-writing; (4) flip writes, keep the AP field updated for rollback; (5) after 7 clean days, stop writing the AP field; (6) delete it. Verification: a daily job summing both stores per user and alarming on **any** non-zero difference. Money tolerates no sampling — check every row.
4. Singapore $\rightarrow$ Virginia is 230 ms one way region-to-region round trip, so a claim from Singapore costs roughly $230 + 60 = 290$ ms (reach the leader, then the leader reaches a majority). Moving the leader to Singapore makes Singapore claims 120 ms and Virginia claims 290 ms. Weighted: leader in Virginia = $0.7 \times 290 + 0.3 \times 120 = 239$ ms; leader in Singapore = $0.7 \times 120 + 0.3 \times 290 = 171$ ms. **Decision: move the leader to Singapore** — 68 ms saved on average, no design change, just leader placement. Better still, shard usernames by region-of-origin prefix so each shard's leader sits where its traffic is.
5. “Both, on purpose. Profiles are AP — during a partition every region keeps serving and writing, and conflicts merge by version vector. Username and balance claims are CP — a minority region returns 503 rather than issue a name twice.”

Interview drill — what the interviewer pushes on

They ask	You answer
“Why not just shard from day one?”	“Because sharding costs me joins, cross-shard transactions, global ordering and easy schema changes, and it is very hard to reverse. I shard when one machine cannot hold the data or take the writes. At 6 million orders a day I sharded for restore time , not for QPS, and I said so.”
“Pick a shard key for a chat app.”	“ <code>conversation_id</code> , hashed. Every message read and write is scoped to one conversation, so that is a single-shard operation. Not <code>user_id</code> , because a two-person chat would then live on two shards and every message would be a cross-shard write.”
“What is wrong with <code>hash(key) % N</code> ?”	“ <i>N</i> changes. Going from 10 to 11 nodes moves about 91% of keys instead of 9%. I hash into 1,024 fixed buckets and map buckets to shards, so a key’s bucket never changes for the life of the system.”
“Your follower is 20 minutes behind. Now what?”	“First, take it out of the read pool — health checks must watch lag in seconds, not CPU. Then find out which of three causes it is: a single long transaction on the leader, replication being single-threaded while the leader writes in parallel, or the follower’s disk being slower than the leader’s. Only the third is fixed by hardware.”
“Is your system CP or AP?”	“Per data type. Display data is AP because staleness is invisible and downtime is not. Unique claims and money are CP because two owners of one username cannot be merged. Saying ‘the system is AP’ would be a design error, not a summary.”
“Exactly-once across shards?”	“Not with a distributed transaction if I can avoid one. I make the operation land on a single shard by choosing the shard key well; when I genuinely cannot, I use a saga with idempotent steps and compensations, because a two-phase commit blocks every participant when the coordinator dies.”
“How do you prevent split brain?”	“Arithmetic, not etiquette. A leader needs a majority, so two majorities cannot exist. A leader holds a lease it must renew, so a frozen leader loses it without needing to be told. And every write carries a fencing token that storage rejects if it is lower than the highest seen.”
“Why is the consensus group size always odd?”	“Because 6 nodes tolerate the same 2 failures as 5 — majority goes from 3 to 4 — so the sixth node costs money and latency and buys nothing.”
“Can you just turn on synchronous replication everywhere?”	“You can, and a write to a region 60 ms away then takes 62 ms instead of 2 ms. With 200 connections per server that is 3,226 writes/s instead of 100,000 — 31 times less throughput. I use semi-sync to a local follower, which costs about 0.5 ms and still bounds data loss on failover.”
“Two regions wrote the same field. Who wins?”	“First I ask whether it is a real conflict: version vectors tell me CONCURRENT apart from merely stale. If it is real, the resolution depends on the field — union for sets, sum for counters, region-tiebreak for scalars, and for money I prevent the conflict instead of resolving it.”
“What breaks first as you add shards?”	“Anything that touches all of them. Scatter-gather tail latency: at 16 shards with a 1% slow rate, 14.9% of fan-out requests are slow; at 64 shards, 47%. Also schema migra-

tions, backups, and the number of connections every service holds.”

TRAP

The classic Tier-3 trap: the interviewer says “now make it strongly consistent globally” and waits to see whether you just say yes. The right answer names the price first. “I can, with a consensus group per shard. Writes go from 2 ms to about 200 ms, throughput per server drops roughly 30x, and a region cut off from the majority stops accepting writes entirely. For usernames that is the right price. For 8 billion profile reads a day it is not. Which data did you mean?”

PRACTICE · TIER 3 target: 40 min

1. A social product shards by `user_id`. Product adds “group chat with up to 500 members”. Choose the shard key for messages, and prove your choice beats the two alternatives with arithmetic on the number of shards touched per send and per read.
2. Design the lag alarm. What do you measure, what threshold, and what does the system do automatically at each level?
3. You have 32 shards and must run a schema migration adding a column with a backfill. Write the plan so that no shard is ever locked for more than 1 second and the migration can be stopped at any point.
4. Compute the expected number of **simultaneously unavailable** shards if each node has a 0.1% chance of being down at any moment, you run 128 shards at RF 3, and a shard is unavailable when 2 of its 3 replicas are down.
5. Your store offers $W=1$, $R=1$ for speed. Write the one-paragraph explanation you would give a product manager about what they are actually buying and what they are giving up.

ANSWER KEY

1. **Shard by `conversation_id`**. Send: 1 shard (write one message row). Read the last 50 messages: 1 shard. Alternative A, shard by `sender_id`: send is 1 shard, but reading a conversation must gather from up to 500 shards — with 0.99 per-shard fast probability, $0.99^{500} = 0.0066$, so **99.3%** of reads hit a slow shard. Dead. Alternative B, fan-out on write to each member’s inbox: send touches up to 500 shards, so at 100 group messages/s that is 50,000 shard writes/s. Reading is then 1 shard. **Decision: `conversation_id`**, with a small per-user index of “conversations I am in” (one row per membership) so the chat list is also a single-shard read.
2. Measure **seconds of lag** (the leader’s newest commit timestamp minus the follower’s last applied timestamp), not bytes and not CPU. Thresholds: >2 s warn; >10 s automatically remove from the read pool; >60 s page a human; $>\text{retention-window}/2$ is a data-loss emergency because the leader may drop log segments the follower still needs. Every one of those is an automatic action except the last.
3. Add the column as nullable with a default **at the metadata level only** (no table rewrite). Backfill in batches of 1,000 rows per shard with a sleep between batches, shard by shard, never more than 4 shards in flight. Application code must tolerate both null and filled for the whole migration — write the new column, read with a fallback. Stop = stop the backfill job; nothing is inconsistent because nothing read it as required yet. Only after 100% is backfilled and verified do you add the NOT NULL constraint.
4. $P(\text{one node down}) = 0.001$. $P(2 \text{ specific of } 3 \text{ down}) = 0.001^2 = 10^{-6}$, and there are $\binom{3}{2} = 3$ such pairs, so $P(\geq 2 \text{ down}) \approx 3 \times 10^{-6}$ (the all-three case is negligible). Expected unavailable shards = $128 \times 3 \times 10^{-6} = 3.84 \times 10^{-4}$. So on average about 0.0004 shards are unavailable at any moment — roughly 34 seconds per day across the whole fleet. That is the argument for RF 3: at RF 2 a **single** node failure loses a shard, giving $128 \times 2 \times 0.001 = 0.256$ expected unavailable shards, which is 670 times worse.
5. “You are buying about 2 ms instead of 6 ms on a write, and about 2 ms instead of 5 ms on a read. You are giving up the guarantee that a read sees the latest write at all: with five copies, one writer and one reader, they can simply be different machines. Concretely, a user can save a setting, reload, and see the old value — not for a second, but until the background repair catches up, which can be minutes. For a view counter that is fine. For anything a user will complain about, it is not, and the fix costs 4 ms.”

ONE-PAGE REVISION CARD

The three words. Replication = copies (safety + read capacity, **not** write capacity). Sharding = splits (write capacity + storage, **not** safety). Consistency model = the promise you make the reader. You need all three, in that order, and you shard **last**.

Why you shard. Size and blast radius far more often than QPS. 6.57 TB restores in 9.1 hours; a 411 GB shard restores in 34 minutes and takes only 1/16 of users with it.

The quorum rule. $W + R > N$ guarantees an overlap of $W + R - N$ nodes, so a read sees the newest write. You survive $N - W$ failures for writes and $N - R$ for reads. $N = 5$, $W = 3$, $R = 3$ is the balanced default. $W = N$ looks safe and is the most fragile choice on the table.

Consistent hashing. Adding the $(N + 1)$ -th node moves $1/(N + 1)$ of keys; modulo moves $N/(N + 1)$. Measured for 10 → 11: **8.25% versus 91.05%**. Use 100–200 virtual nodes per machine — with 1, the busiest node held 64.5x the quietest.

The bucket trick. Never hash to a shard. Hash to 1,024 (or 8,192) fixed buckets, and keep a versioned bucket → shard map. A key's bucket never changes, so resharding copies buckets instead of rehashing keys. Put the bucket in the id and routing needs no lookup at all.

Choosing a shard key — four tests.

1. Does it make the **most frequent** query single-shard?
2. Is it evenly distributed, with no entity above 2x the average?
3. Can it **never change** for a row? (If it can, stop. Choose again.)
4. Do the things that must be transactional share it?

CAP, said correctly. Partitions are not a choice. During one you pick an error (CP) or a stale answer (AP). PACELC adds: even with no partition, you trade latency against consistency. **Different data in one product sits on different sides.** Profiles AP, usernames CP.

Failover safety — three rules. Majority election (so two leaders cannot both win). Leases, not locks (so a frozen leader loses it without being told). Fencing tokens that always increase (so storage rejects a late write from an old leader). Consensus groups are always an **odd** size: 6 tolerates the same 2 failures as 5.

Numbers to memorise.

- 1 million/day $\approx 11.6/s$; 1 billion/day $\approx 11,574/s$
- same-DC round trip 0.5 ms; SG ↔ Mumbai 60 ms; SG ↔ Virginia 230 ms
- one SQL node 5,000 writes/s, 20,000 indexed reads/s; Redis 100,000 ops/s
- restore speed 200 MB/s → keep shards between 200 GB and 1 TB
- sync-to-a-far-region costs 31x throughput: 100,000/s → 3,226/s at 200 connections
- scatter-gather tail: $0.99^{16} = 0.851$, so 14.9% slow at 16 shards; $0.99^{64} = 0.526$
- raw disk = logical $\times$ RF $\times$ 1.5 for compaction headroom

Conflict resolution ladder. Prevent (single leader / consensus) > merge function (union, sum, max) > keep both and ask > last-write-wins. Never LWW on a wall clock: a 40 ms skew silently deletes the newer write.

Read-your-writes, cheapest first. (1) The write response contains the new value — free. (2) Pin the session to one region or the leader for a few seconds. (3) Version token on the read. (4) Quorum read everywhere — almost never worth it at an 80:1 read ratio.

JavaScript traps in this chapter.

1. `ring.sort()` with no comparator sorts 32-bit hashes as text and silently corrupts routing.
2. A weak hash without an avalanche step puts `n0#1` and `n0#2` next to each other, which defeats virtual nodes entirely.
3. Map keeps insertion order and real key types; a plain object stringifies keys, which breaks numeric bucket ids.
4. Money is integer cents. A float balance drifts and no ledger will ever balance again.

Checklist before you say “done”.

1. Said whether you are solving reads, writes, or size — and sharded only for the last two.
2. Named the shard key and passed all four tests on it out loud.
3. Gave N , W , R and showed $W + R > N$.
4. Said which data is CP and which is AP, and why the cost of being wrong differs.
5. Explained resharding **before** being asked, using buckets.
6. Named the hot key or hot shard and gave a fix with its cost.
7. Covered failover: majority, lease, fencing token.
8. Gave both sides of every trade-off and then **decided**.

Three sentences that score marks.

1. “I am sharding for restore time and blast radius, not for QPS — one machine could take this write load.”
2. “Partitions are not a choice, so the question is what I do during one; profiles answer stale, username claims answer 503.”
3. “A lease expires by itself and a fencing token is checked by the storage, so a frozen leader cannot corrupt anything even though it never found out it was replaced.”

HLD Case Studies I

The low-level pieces — base62, key blocks, token bucket, sliding window, dispatcher — then a URL shortener walked end to end

The idea in one page

You have learnt the parts. Chapters 6 to 10 gave you databases, caches, load balancers, queues, sharding and replication. This chapter puts them together.

Five designs. Each one is walked end to end using the same seven steps from Chapter 1. The steps never change. Only the numbers and the hard part change.

THE SEVEN STEPS — SAY THEM OUT LOUD BEFORE YOU DRAW ANYTHING

#	Step	Minutes
1	Clarify. Ask 4 to 6 questions. Write the answers on the board.	5
2	Scale estimate. DAU → QPS → storage/year → bandwidth. Show the division.	5
3	API surface. Three to six real endpoints with real payloads.	5
4	Data model. Tables, fields, and the chosen key (partition key + sort key).	5
5	Architecture diagram. Boxes and arrows. Read path and write path separately.	8
6	Deep dive on the one or two genuinely hard parts.	10
7	Trade-offs, failure modes, what changes at 10x.	7

Total 45 minutes. If you are 20 minutes in and still clarifying, you will fail.

What each design in this chapter teaches you

Design	The real lesson	The trap
URL shortener	Key generation without coordination	Thinking hashing is enough
Rate limiter	Shared counters under concurrency	Counting per-server, not globally
Notifications	Fan-out to unreliable third parties	No dedupe, no retry budget
News feed	Push vs pull, and the celebrity problem	One fan-out strategy for everyone
Chat	Stateful connections and message ordering	Using request/response for a stream

The numbers you must have in your head

THE ESTIMATION CARD

- 1 day = 86,400 seconds. Round it to 10^5 when you are in a hurry.
- **1 million per day ≈ 11.6 per second.** This single fact does most of the work.
- 1 billion per day ≈ 11,574 per second.
- Peak is **2x to 3x** the average for a global product, **5x to 10x** for one country or one event (a sale, a match, a festival).
- 1 KB × 1 million = 1 GB. 1 KB × 1 billion = 1 TB.
- 1 Mbps = 0.125 MB/s. So 3 Mbps = 0.375 MB/s.

- A single MySQL box: 5,000 simple writes/s, 20,000 indexed reads/s.
- A single Redis box: 100,000 simple ops/s.
- One machine holds 100,000 open WebSocket connections comfortably.

SHORTCUT

The divide-by-86,400 shortcut. Do not reach for a calculator. Write the daily count in millions, then multiply by 11.6.

- $10\text{M/day} = 10 \times 11.6 = 116/\text{s}$.
- $200\text{M/day} = 200 \times 11.6 = 2,320/\text{s}$.
- $2,000\text{M/day} = 2000 \times 11.6 = 23,200/\text{s}$.

Check: $2,000,000,000/86,400 = 23,148$. The shortcut is within 0.3%. Good enough for a whiteboard, and you did it in two seconds.

TRAP

Never say “it depends” and stop. That is the single most common way to lose a design round. Say “it depends on X. I will assume X is true because Y, so I choose Z. If X were false I would choose W instead.” That is one sentence longer and it is the whole difference between a hire and a no-hire.

Warm-up

WARM-UP

build the reflex

Example 1

A service has 40 million daily active users. Each user makes 25 requests per day. What is the average QPS, and the peak at 3x?

SOLUTION

Step 1 — total requests per day. $40,000,000 \times 25 = 1,000,000,000$ requests/day.

Step 2 — divide by seconds in a day. $1,000,000,000/86,400 = 11,574$ requests/second.

Step 3 — peak. $11,574 \times 3 = 34,722$ requests/second.

Answer: 11,600 QPS average, 34,700 QPS peak

Example 2

Each stored row is 250 bytes. You write 10 million rows per day. How much storage do you need for 5 years?

SOLUTION

Per day: $10,000,000 \times 250 = 2,500,000,000$ bytes = 2.5 GB/day.

Per year: $2.5 \times 365 = 912.5$ GB = 0.9125 TB.

Five years: $0.9125 \times 5 = 4.56$ TB.

Add a replication factor of 3 for durability: $4.56 \times 3 = 13.7$ TB of raw disk.

Answer: 4.6 TB of data, 13.7 TB of raw disk at RF 3

Example 3

Base62 uses the characters 0-9, a-z, A-Z. How many distinct strings of exactly 7 characters exist? At 10 million new strings per day, how long until you run out?

SOLUTION

Step 1 — count. $62^7 = 3,521,614,606,208 \approx 3.52 \times 10^{12}$. About 3.5 trillion.

Step 2 — days. $3,521,614,606,208 / 10,000,000 = 352,161$ days.

Step 3 — years. $352,161 / 365 = 965$ years.

For comparison, 6 characters gives $62^6 = 56,800,235,584 \approx 5.68 \times 10^{10}$, which is $56,800,235,584 / 10,000,000 = 5,680$ days = 15.6 years. Still fine, but 7 is the safe answer because it survives a 50x growth in traffic.

Answer: $62^7 \approx 3.52$ trillion; 965 years of runway

Example 4

A token bucket has capacity 5 and refills 1 token per second. It starts full. Requests arrive at $t = 0$ (seven of them at once), then one at $t = 2.5$ s. Trace it.

SOLUTION

Event	Tokens before	Decision	Tokens after
$t = 0$ req 1	5.0	allow	4.0
$t = 0$ req 2	4.0	allow	3.0
$t = 0$ req 3	3.0	allow	2.0
$t = 0$ req 4	2.0	allow	1.0
$t = 0$ req 5	1.0	allow	0.0
$t = 0$ req 6	0.0	deny	0.0
$t = 0$ req 7	0.0	deny	0.0
$t = 2.5$ req 8	$0 + 2.5 \times 1 = 2.5$	allow	1.5

The bucket let a **burst** of 5 through instantly. That is the point of a token bucket: it allows bursts up to the capacity, but the long-run rate is pinned to the refill rate.

Answer: 5 allowed, 2 denied, then allowed again at $t = 2.5$ s with 1.5 tokens left

Example 5

8 million posts are created per day. The average author has 200 followers. If every post is copied into every follower's inbox, how many inbox rows are written per day and per second?

SOLUTION

Rows per day: $8,000,000 \times 200 = 1,600,000,000$ rows/day.

Per second: $1,600,000,000 / 86,400 = 18,518$ rows/second average.

At 3x peak: $18,518 \times 3 = 55,555$ rows/second.

Now the thing that matters: the **post** rate is only $8,000,000 / 86,400 = 92.6$ posts/second. The fan-out multiplies the write load by 200. A design that looks tiny on the write API is huge behind it.

Answer: 1.6 billion rows/day, 18,500/s average, 55,500/s peak

Example 6

A redirect response is about 500 bytes on the wire. You serve 11,574 redirects per second. What is the outbound bandwidth?

SOLUTION

$11,574 \times 500 = 5,787,000$ bytes/second = 5.787 MB/s.

In bits: $5.787 \times 8 = 46.3$ Mbps.

That is nothing. A single 1 Gbps network card handles it 20 times over. This is worth saying out loud in the interview, because it tells the interviewer you know the bottleneck here is **request count**, not bandwidth.

Answer: 5.8 MB/s, about 46 Mbps

PRACTICE · WARM-UP target: 8 min

1. 25M DAU, 12 requests each per day. Average QPS? Peak at 2x?
2. Rows are 1.2 KB. 4 million rows/day. Storage for 3 years at RF 3?
3. A leaky bucket drains 10 requests/second and holds 50. A burst of 90 arrives at once. How many are dropped?
4. 500,000 concurrent WebSocket connections, 100,000 per machine. How many machines, and how many if you want to survive losing one whole availability zone out of three?

ANSWER KEY

1. $25 \times 12 = 300\text{M/day}$; $300,000,000/86,400 = 3,472/\text{s}$; peak 6,944/s.
2. $4,000,000 \times 1200 = 4.8 \text{ GB/day}$; $\times 365 = 1,752 \text{ GB/yr}$; $\times 3 \text{ yrs} = 5,256 \text{ GB} = 5.26 \text{ TB}$; $\times 3 \text{ RF} = 15.8 \text{ TB}$.
3. Bucket holds 50, so 50 are queued, 40 are dropped immediately. The 50 drain over $50/10 = 5$ seconds.
4. $500,000/100,000 = 5$ machines. To survive one zone of three you must run the full load on two zones, so provision $5/(2/3) = 7.5 \rightarrow 8$ machines, spread 3/3/2.

Tier 1 — the low-level pieces first

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Before you design a system, you must be able to write the two or three classes at its centre. Service-company rounds often ask **only** this part. Product-company rounds ask the big picture but will still say “code the key generator”.

Piece 1 — a base62 encoder

Example 7 [Infosys · pattern]

Write a function that turns a number into a short string using digits, lowercase and uppercase letters. Write the reverse too.

SOLUTION

62 characters means base 62. The algorithm is the same one you use for binary or hex: repeatedly take the remainder, then divide.

base62.js — run with: node base62.js

JS

```
const ALPHABET = "0123456789abcdefghijklmnopqrstuvwxyz" +
  "ABCDEFGHIJKLMNOPQRSTUVWXYZ";

function toBase62(n) {
  if (n === 0) return "0";
  let s = "";
  while (n > 0) {
    s = ALPHABET[n % 62] + s; // remainder picks the character
    n = Math.floor(n / 62);   // integer divide, then repeat
  }
}
```



```

    }
    return s;
}

function fromBase62(s) {
    let n = 0;
    for (const ch of s) n = n * 62 + ALPHABET.indexOf(ch);
    return n;
}

console.log(toBase62(0));           // 0
console.log(toBase62(61));          // Z
console.log(toBase62(62));          // 10
console.log(toBase62(123456789));   // 8m0Kx
console.log(fromBase62("8m0Kx"));    // 123456789

```

Hand-trace toBase62(123456789):

$123456789 \bmod 62 = 45 \rightarrow \text{ALPHABET}[45] \text{ is } x; 123456789 \div 62 = 1991238$

$1991238 \bmod 62 = 36 \rightarrow K; 1991238 \div 62 = 32116$

$32116 \bmod 62 = 0 \rightarrow 0; 32116 \div 62 = 518$

$518 \bmod 62 = 22 \rightarrow m; 518 \div 62 = 8$

$8 \bmod 62 = 8 \rightarrow 8; 8 \div 62 = 0$, stop.

Read the characters in the order they were prepended: 8m0Kx. Matches.

Time $O(\log_{62} n)$ — 7 loops for a 7-character code **Space** $O(1)$ beyond the output string

TRAP

`ALPHABET.indexOf(ch)` is a linear scan of 62 characters. That is fine for a demo but in a hot loop build a Map from character to value once, outside the function. Also: the decode side is only safe while n stays below 2^{53} . $62^9 = 1.35 \times 10^{16}$ already exceeds it, so a 9-character code needs BigInt.

Piece 2 — the key generator that does not need a lock

Example 8 [Capgemini · pattern]

Ten application servers all create short links. No two may ever produce the same code. You may not take a lock on every request. Design it, then code it.

SOLUTION

The idea: hand out blocks, not numbers. A central counter (one row in a database, or a ZooKeeper sequence) gives a server a **range** of 1,000 IDs in one round trip. The server then serves 1,000 requests from memory with zero coordination. When the block runs out it asks for the next one.

Cost of coordination drops by 1,000x. If the counter is hit 116 times/second without blocks, with blocks of 1,000 it is hit $116/1000 = 0.116$ times per second — about once every 9 seconds.

shortener.js — run with: `node shortener.js`

JS

```

class CounterRange {
    constructor(start, size) { this.next = start; this.end = start + size; }
    take() {
        if (this.next >= this.end) return null; // block exhausted
        return this.next++;
    }
}

```

```

class RangeAllocator { // stands in for one row in a DB / ZooKeeper
  constructor(blockSize = 1000) { this.cursor = 0; this.blockSize = blockSize; }
  allocate() {
    const start = this.cursor; // in reality: an atomic
    this.cursor += this.blockSize; // UPDATE ... SET c = c + 1000
    return new CounterRange(start, this.blockSize);
  }
}

class ShortenerService {
  constructor(allocator) {
    this.allocator = allocator;
    this.range = allocator.allocate();
    this.store = new Map(); // code -> longUrl
    this.byUrl = new Map(); // owner|longUrl -> code (dedupe)
  }
  #nextId() {
    let id = this.range.take();
    if (id === null) { // ran out: get a fresh block
      this.range = this.allocator.allocate();
      id = this.range.take();
    }
    return id;
  }
  shorten(owner, longUrl) {
    const dedupeKey = owner + "|" + longUrl;
    if (this.byUrl.has(dedupeKey)) return this.byUrl.get(dedupeKey);
    const code = toBase62(this.#nextId()).padStart(7, "0");
    this.store.set(code, longUrl);
    this.byUrl.set(dedupeKey, code);
    return code;
  }
  resolve(code) { return this.store.get(code) ?? null; }
}

```

Now run two servers against one allocator, with a tiny block size of 3 so you can see the blocks:

the test and its real output	JS
<pre> const alloc = new RangeAllocator(3); const a = new ShortenerService(alloc); // app server A: gets ids 0,1,2 const b = new ShortenerService(alloc); // app server B: gets ids 3,4,5 a.shorten("u1", "https://example.com/p1"); // 00000000 a.shorten("u1", "https://example.com/p2"); // 00000001 a.shorten("u1", "https://example.com/p1"); // 00000000 <- same url, same code b.shorten("u2", "https://example.com/p3"); // 00000003 <- B's block a.shorten("u1", "https://example.com/p4"); // 00000002 <- A's block ends a.shorten("u1", "https://example.com/p5"); // 00000006 <- A takes a NEW block a.resolve("00000001"); // https://example.com/p2 a.resolve("zzzzzzz"); // null </pre>	

Read the output carefully. Server A produced 0, 1, 2 then jumped to 6. It never collided with B's 3, 4, 5. No lock was taken on any individual request.

The gap from 3 to 5 is **wasted** if server A dies while holding a block. That is fine. Wasting 1,000 keys out of 3.5 trillion, once per server crash, costs nothing. Say this in the interview before the interviewer asks — it shows you thought about it.

Piece 3 — a token bucket

Example 9 [TCS NQT · pattern]

Code a rate limiter that allows a burst but holds the long-run rate at r requests per second.

SOLUTION

Do not store a list of timestamps. Store **two numbers**: how many tokens are left, and when you last looked. Refill lazily, at read time.

tokenbucket.js — run with: `node tokenbucket.js`

JS

```
class TokenBucket {
  constructor(capacity, refillPerSec) {
    this.capacity = capacity;
    this.refillPerSec = refillPerSec;
    this.tokens = capacity;
    this.lastMs = 0;
  }
  allow(nowMs, cost = 1) {
    const elapsed = (nowMs - this.lastMs) / 1000;
    // refill first, but never above capacity
    this.tokens = Math.min(this.capacity, this.tokens + elapsed *
this.refillPerSec);
    this.lastMs = nowMs;
    if (this.tokens >= cost) { this.tokens -= cost; return true; }
    return false;
  }
  retryAfterMs(cost = 1) { // what you put in the Retry-After header
    if (this.tokens >= cost) return 0;
    return Math.ceil(((cost - this.tokens) / this.refillPerSec) * 1000);
  }
}
```

Real output for capacity 5, refill 1/s:

output

JS

```
burst of 7 at t=0 : true true true true true false false
retry-after      : 1000 ms
at t=2.5s        : true true false
after 60s idle   : true, tokens left 4  <- capped at 5, not 60
```

Three things to point at:

1. Only **two** numbers per user are stored. 16 bytes, not a list.
2. Refill is lazy. Nothing runs in the background. There is no timer.
3. The cap stops an idle user from banking 60 tokens over a minute of silence.

Time $O(1)$ per check **Space** $O(1)$ per key — two numbers

Piece 4 — sliding window counter

Example 10 [Wipro · pattern]

A fixed one-minute window lets a user send 2x the limit — 100 at 11:59:59 and 100 at 12:00:00. Fix it without storing every timestamp.

SOLUTION

Keep the **previous** window's count and the **current** window's count. Blend them by how far into the current window you are.

$$\text{estimate} = \text{prev count} \times (1 - \text{fraction into window}) + \text{current count}$$

sliding.js — run with: node sliding.js

JS

```
class SlidingWindowCounter {
  constructor(limit, windowMs) {
    this.limit = limit; this.windowMs = windowMs;
    this.curStart = 0; this.cur = 0; this.prev = 0;
  }
  allow(nowMs) {
    const w = Math.floor(nowMs / this.windowMs) * this.windowMs;
    if (w > this.curStart) {
      // if we skipped a whole window, the previous one is genuinely empty
      this.prev = (w - this.curStart === this.windowMs) ? this.cur : 0;
      this.cur = 0;
      this.curStart = w;
    }
    const intoWindow = (nowMs - this.curStart) / this.windowMs; // 0.0 .. 1.0
    const estimate = this.prev * (1 - intoWindow) + this.cur;
    if (estimate < this.limit) { this.cur++; return true; }
    return false;
  }
}
```

Trace with limit 5 per 10 seconds. Five requests land at $t = 9\text{s}$ — all allowed, the sixth at $t = 9.5\text{s}$ is denied. Now the window rolls over at $t = 10\text{s}$.

At $t = 11\text{s}$ we are $1/10 = 0.1$ into the new window.

Estimate = $5 \times (1 - 0.1) + 0 = 4.5$. $4.5 < 5$, so **allow**, and cur becomes 1.

Second request at $t = 11\text{s}$: estimate = $4.5 + 1 = 5.5$. Not less than 5, so **deny**.

At $t = 19\text{s}$ we are 0.9 into the window: estimate = $5 \times 0.1 + 1 = 1.5$. **Allow**.

That matches the program's output exactly:

output

JS

```
t=9s   allowed: 5 of 5
t=9.5s 6th    : false
t=11s  allowed: true   (estimate 4.5)
t=11s  again  : false  (estimate 5.5)
t=19s  allowed: true   (estimate 1.5)
```

Time $O(1)$ **Space** $O(1)$ per key — three numbers

Exact sliding window needs a sorted set of timestamps: $O(\log n)$ and $O(\text{limit})$ memory.

Algorithm	Memory per key	Allows bursts?	Accuracy
Fixed window	1 counter	yes, 2x at the seam	poor
Sliding window log	one timestamp per request	no	exact
Sliding window counter	3 numbers	slightly	within 1%

Token bucket	2 numbers	yes, by design	exact on long-run rate
Leaky bucket	queue + 2 numbers	no — smooths output	exact

Decision. Use **token bucket** for user-facing APIs: bursts are normal human behaviour and you want to allow them. Use **sliding window counter** when a hard “no more than N per minute” promise was made to a partner. Use **leaky bucket** only when the downstream cannot absorb bursts at all, such as an SMS gateway with a fixed contracted rate.

Piece 5 — the notification dispatcher

Example 11 [Cognizant · pattern]

Design classes for a system that sends the same message by push, SMS or email, tries channels in order, respects opt-outs, and never sends the same notification twice.

SOLUTION

This is the **Strategy** pattern from Chapter 4. One interface, three implementations, and a dispatcher that does not know or care which one it is holding.

notifier.js — run with: node notifier.js

JS

```
class Channel {
  get name() { throw new Error("override"); }
  costPaise() { return 0; }
  async send(user, rendered) { throw new Error("override"); }
}

class PushChannel extends Channel {
  get name() { return "push"; }
  async send(user, r) {
    return user.deviceToken ? { ok: true, via: "push" }
      : { ok: false, why: "no device token" };
  }
}

class SmsChannel extends Channel {
  get name() { return "sms"; }
  costPaise() { return 12; } // money! this drives the design
  async send(user, r) {
    return user.phone ? { ok: true, via: "sms" } : { ok: false, why: "no phone" };
  }
}

class EmailChannel extends Channel {
  get name() { return "email"; }
  async send(user, r) {
    return user.email ? { ok: true, via: "email" } : { ok: false, why: "no
email" };
  }
}

class Template {
  constructor(id, body) { this.id = id; this.body = body; }
  render(data) { return this.body.replace(/\{(\w+)\}/g, (_, k) => data[k] ?? ""); }
}

class Notifier {
  constructor(channels, templates) {
    this.channels = new Map(channels.map(c => [c.name, c]));
    this.templates = templates;
    this.seen = new Set(); // idempotency keys already handled
  }
}
```

```

}
async notify(user, templateId, data, order, idemKey) {
  if (this.seen.has(idemKey)) return { skipped: "duplicate" };
  this.seen.add(idemKey);
  const rendered = this.templates.get(templateId).render(data);
  const log = [];
  for (const chName of order) {
    if (user.optOut.includes(chName)) { log.push(chName + ": opted out");
continue; }
    const ch = this.channels.get(chName);
    const res = await ch.send(user, rendered);
    log.push(chName + ": " + (res.ok ? "sent" : "failed - " + res.why));
    if (res.ok) return { text: rendered, log, pause: ch.costPaise() };
  }
  return { text: rendered, log, pause: 0, delivered: false };
}
}

```

Real output, for a user with no device token but a phone number:

```

output JS
{ text: 'Your code is 482913. It expires in 5 minutes.',
  log: [ 'push: failed - no device token', 'sms: sent' ],
  pause: 12 }

```

and when the same idempotency key arrives again:

```

output JS
{ skipped: 'duplicate' }

```

Four design points the interviewer is listening for:

1. Adding WhatsApp means adding **one class**. Nothing else changes. Open for extension, closed for modification.
2. `costPaise()` is on the channel, so the router can prefer free channels. That is a business rule expressed in the type system.
3. `seen` is a Set here; in production it is a Redis SET key value NX EX 86400.
4. The fallback loop stops at the first success. The order is data, not code — a campaign can pass ["push", "email"] and never spend money on SMS.

PRACTICE · TIER 1 target: 25 min

1. Extend TokenBucket so a single request can cost more than one token (an expensive search costs 5, a cheap read costs 1). Show the trace for capacity 10, refill 2/s, and requests of cost 5, 5, 5 at $t = 0$ and $t = 1$.
2. Write LeakyBucket with `push(nowMs)` returning `true` if the request was queued and `false` if the queue was full, plus `drain(nowMs)` removing at the fixed rate.
3. Add a `WhatsAppChannel` to the notifier with `costPaise() = 4` and a router method that picks the cheapest channel the user actually has, instead of a fixed order.
4. The ShortenerService currently dedupes by owner and URL in memory. Say exactly what breaks when there are ten servers, and give two fixes with the cost of each.

ANSWER KEY

1. Change `allow(now, cost)` to subtract `cost`. At $t = 0$: 10 tokens, cost 5 $\rightarrow$ allow (5 left), cost 5 $\rightarrow$ allow (0 left), cost 5 $\rightarrow$ deny. At $t = 1$: refill $2 \times 1 = 2$, tokens 2, cost 5 $\rightarrow$ deny, `retryAfter` = $(5 - 2)/2 = 1.5$ s.
2. Keep `queue` (an array) plus `lastMs`. `drain` removes `floor((now - lastMs)/1000 * rate)` items. `push` first drains, then appends only if `queue.length < capacity`.
3. `router(user)` filters channels the user can receive (`deviceToken`, `phone`, `email`) then sorts by `costPaise()` ascending. Push (0) and email (0) come first, WhatsApp (4), SMS (12) last.
4. Each server has its own `byUrl` map, so the same URL shortened on two servers gets two codes. Fix A: move the dedupe map into Redis — one extra network hop (1 ms) per write, which at 116 writes/s is free. Fix B: accept the duplicate. Two codes pointing at the same URL is **harmless** — the redirect still works. Choose B unless the product promises “one link per URL”, because B costs nothing.

Design 1 — a URL shortener

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 12 [Shopee · pattern]

Design a service that turns a long link into a short one and redirects. Support custom aliases, link expiry, and per-link click counts.

SOLUTION

Step 1 — Clarify

Question I ask	Why the answer changes the design
How many new links per day?	Decides the key length and whether one database is enough.
What is the read-to-write ratio?	If reads dominate, the whole design becomes a cache design.
Do links ever expire?	Expiry means a TTL and a background cleaner, or a lazy delete on read.
Can users pick their own alias?	Custom aliases need a uniqueness check — a different write path.
Do we need click analytics in real time?	Real time forces a stream. “Within 5 minutes” lets you batch.
Must the same long URL always give the same short one?	If yes, you need a global dedupe index, which is a second database.

Answers I will assume, stated out loud: **10 million new links/day**, **read:write is 100:1**, **links expire after a default 2 years**, **custom aliases allowed**, **click counts may lag by 5 minutes**, **no global dedupe required**.

Step 2 — Scale estimate

Writes.

$$10,000,000/86,400 = 115.7 \approx 116 \text{ writes/second}$$

Peak at 3x: $116 \times 3 = 348$ writes/second.

Reads. 100:1 means $10,000,000 \times 100 = 1,000,000,000$ redirects/day.

$$1,000,000,000/86,400 = 11,574 \text{ reads/second}$$

Peak at 3x: $11,574 \times 3 = 34,722$ reads/second.

Storage. One row: short code 7 B, long URL up to 2 KB but 180 B typical, owner id 8 B, created 8 B, expires 8 B, flags 4 B. Round to **250 bytes** with row overhead.

$$10,000,000 \times 250 = 2,500,000,000 \text{ B} = 2.5 \text{ GB/day}$$

$$2.5 \times 365 = 912.5 \text{ GB/year} \approx 0.91 \text{ TB/year}$$

Five years: $0.91 \times 5 = 4.6 \text{ TB}$. At replication factor 3: $4.6 \times 3 = 13.7 \text{ TB raw}$.

Key space. 7 base62 characters give $62^7 = 3,521,614,606,208$ codes.

$$3,521,614,606,208 / 10,000,000 = 352,161 \text{ days} = 965 \text{ years}$$

Bandwidth. Redirect response 500 B: $11,574 \times 500 = 5.79 \text{ MB/s}$ outbound. Trivial.

Cache. Suppose 20 million distinct links are hit on a given day and the hottest 20% serve most traffic. Cache 20 million rows: $20,000,000 \times 250 = 5,000,000,000 \text{ B} = 5 \text{ GB}$. That fits in one Redis node's memory with room to spare.

THE ONE-LINE SUMMARY YOU PUT ON THE BOARD

348 writes/s peak · 34,722 reads/s peak · 0.91 TB/year · 5 GB hot cache · 965 years of keys. **Read-heavy, tiny data, no bandwidth problem.** Therefore: cache first, database second.

Step 3 — API surface

the four endpoints that matter

TEXT

```
POST /v1/links
  body  { "url": "https://example.com/a/very/long/path?x=1",
          "alias": "sale24",           // optional
          "expiresAt": "2027-01-01T00:00:00Z" }
  header Idempotency-Key: 9f2c-...    // safe to retry
  201    { "code": "8m0Kx2p", "shortUrl": "https://sho.rt/8m0Kx2p" }
  409    { "error": "alias_taken" }

GET /{code}
  302    Location: https://example.com/a/very/long/path?x=1
          Cache-Control: private, max-age=0      // see the note on 301 below
  404    when the code does not exist
  410    when the link existed but has expired

GET /v1/links/{code}/stats?from=2026-09-01&to=2026-09-15
  200    { "total": 48213, "byDay": [...], "topCountries": [...] }

DELETE /v1/links/{code}
  204
```

TRAP

301 versus 302. A 301 is a permanent redirect. Browsers cache it forever, so your click counter stops counting after the first visit and you can never change or delete the link. A 302 is temporary: the browser asks again every time. You pay in QPS and you get analytics and the ability to revoke. **Decision: use 302.** We are read-heavy but our reads are cheap (a cache hit) and analytics is a product requirement. If analytics were dropped and cost mattered more, 301 would cut read traffic by perhaps 80%.

Step 4 — Data model

Field	Type	Note
code	char(7)	partition key. Random-looking, so writes spread evenly.

long_url	varchar(2048)	the destination
owner_id	bigint	null for anonymous links
created_at	timestamp	
expires_at	timestamp	null means never. Used as a TTL by the store.
is_custom	bool	custom aliases cannot be recycled

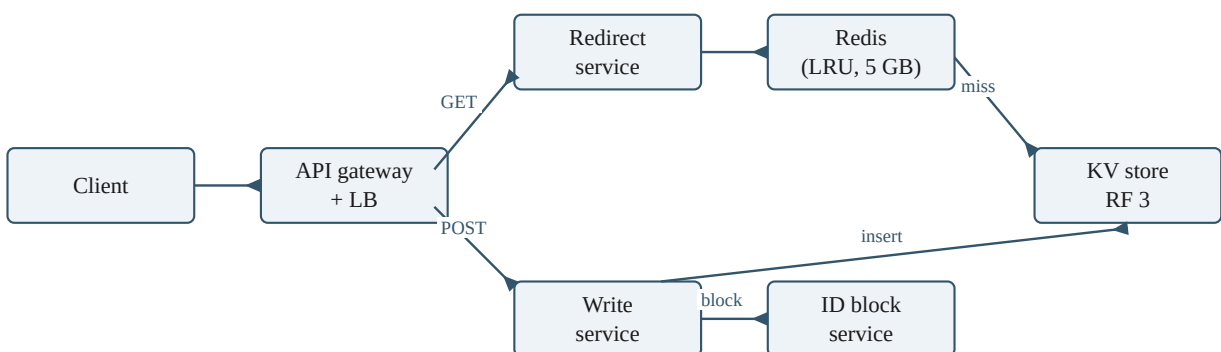
Second table, written by a stream job, never on the hot path:

Field	Type	Note
code	char(7)	partition key
day	date	sort key — so a date-range query is one contiguous scan
clicks	counter	incremented in batches
country	map	country code → count

Which database? The access pattern is: given one key, fetch one row. There are no joins, no range scans, no transactions across rows. That is the exact shape a key-value store is best at. **Decision: a wide-column / KV store such as Cassandra or DynamoDB, partitioned on code.** A relational database would also work at this size, but sharding it by hand at 10x is work we can avoid by choosing right the first time.

Why not hash the URL? `md5(url)` truncated to 7 characters looks clever and fails. With 3.5 trillion slots and 10 million inserts per day, the birthday bound says collisions start appearing after roughly $\sqrt{3.5 \times 10^{12}} \approx 1.9$ million inserts — that is **within the first day**. You would need a read-before-write on every insert to detect them, which is exactly the coordination we were trying to avoid. The counter-block scheme from Example 8 has **zero** collisions by construction.

Step 5 — Architecture



Read path along the top, write path along the bottom. The two share nothing but the store.

Trace one redirect:

1. GET `/8m0Kx2p` reaches the gateway.
2. The gateway routes it to a redirect service pod. No session, no state — any pod will do.
3. The pod asks Redis for key `u:8m0Kx2p`. **Hit (98% of the time):** return 302 in 1 ms.
4. **Miss:** read the KV store (5 ms), write it back into Redis with a TTL, return 302.
5. Either way, push a click event onto Kafka and return. The return does **not** wait for Kafka's ack — analytics is allowed to lose an event, the redirect is not allowed to be slow.

Trace one create:

1. POST `/v1/links` with an Idempotency-Key.
2. The write service checks its in-memory ID block; takes the next ID, or fetches a fresh block of 1,000 if empty.

3. Encode to base62, pad to 7 characters, insert into the KV store with `IF NOT EXISTS` (cheap, because a collision is impossible — it is a belt-and-braces check).
4. Return 201. Do **not** warm the cache; a brand-new link is usually not read immediately, and if it is, the first read costs 5 ms once.

Step 6 — Deep dive

Deep dive A: the cache is the system.

At 34,722 peak reads/second, the KV store would need to serve all of them if the cache were empty. Cassandra at 20,000 reads/s/node would need $34,722/20,000 = 1.74 \rightarrow 2$ nodes just for reads, before replication. With a 98% hit rate the store only sees $34,722 \times 0.02 = 694$ reads/second. That is one node, half asleep.

So the interesting question is **what happens when the cache is cold** — a Redis restart, or a new region coming online. All 34,722 reads/s hit the store at once. Two defences:

1. **Request coalescing.** If 500 requests for the same missing key arrive in the same millisecond, only one goes to the store; the other 499 wait on the same promise. This turns a thundering herd into a single read.
2. **Warm-start.** Before routing traffic to a restarted cache node, replay the top 1 million codes from a daily-generated list. $1,000,000 \times 250 \text{ B} = 250 \text{ MB}$, which loads in well under a minute.

Deep dive B: custom aliases without a lock.

Generated codes cannot collide. Custom aliases **can** — two users both want `sale24`. The fix is a conditional write, not a read-then-write:

the only safe order

TEXT

```
WRONG: if (!exists(alias)) { insert(alias) }      <- two writers both see "not
exists"
RIGHT: INSERT ... IF NOT EXISTS                  <- the store decides,
atomically
        on failure -> 409 alias_taken
```

Reserve a separate namespace so a custom alias can never be handed out by the counter: generated codes are always exactly 7 characters; custom aliases must be 4 to 6 characters or 8 or more. Now the two key spaces cannot overlap, and you never need to check one against the other.

Deep dive C: counting clicks without slowing redirects.

A naive `UPDATE links SET clicks = clicks + 1` on every redirect adds 34,722 writes/second to a store that was doing 116. It also makes every popular link a single hot row.

Instead: the redirect pod keeps a local Map of `code -> count` and flushes it to Kafka every 10 seconds. With 200 pods and, say, 50,000 distinct codes per pod per flush, that is $200 \times 50,000/10 = 1,000,000$ counter-deltas per second sent as **batched** messages — a few thousand Kafka messages per second, not a million. A stream job aggregates by (`code`, `day`) and writes one row per code per hour.

Cost of this choice: counts lag by up to 10 seconds plus the stream lag, and a pod crash loses at most 10 seconds of counts for that pod. We accepted a 5-minute lag in Step 1, and losing 10 seconds of clicks out of a day is a $10/86,400 = 0.012\%$ error. Fine.

Step 7 — Trade-offs, failures, 10x

Choice	Option A	Option B	Decision
Code generation	hash the URL	counter + base62	B — zero collisions, no read-before-write
Redirect status	301 permanent (cheap)	302 temporary (countable)	B — analytics and revocation are required
Store	relational	key-value	B — the access pattern is pure key lookup
Click counting	synchronous increment	batch to a stream	B — 300x fewer store writes for 10 s of lag

Cache write policy	write-through on create	lazy fill on first read	B — most new links are never read
What fails	What the user sees	What we do	
Redis node dies	Latency rises from 1 ms to 6 ms	Consistent hashing spreads the loss; only $1/N$ of keys are cold. Requests coalesce so the store is not flooded.	
Whole cache tier dies	Slow but working	Store is sized for 100% miss at average (11,574/s), not peak; shed 20% of traffic with 429s during the warm-up if needed.	
ID block service dies	Creates fail, redirects fine	Each write pod holds up to 1,000 unused IDs — roughly 1,000 / 348 per second $\approx$ 3 seconds of headroom at peak. Raise the block to 10,000 for 29 seconds. Then fail writes with 503 and let clients retry.	
KV node dies	Nothing	RF 3 with quorum reads. Two replicas are enough.	
A single link goes viral	Nothing	It is one cache key served from memory; that is the best case, not the worst.	

At 10x (100 million new links/day, 10 billion redirects/day):

- Reads become $10,000,000,000/86,400 = 115,740/s$ average, 347,222/s at peak.
- The cache grows to 50 GB. One Redis node no longer holds it: shard by code across 8 nodes of 8 GB each.
- Storage becomes 9.1 TB/year. Still small. Add nodes for throughput, not capacity.
- The genuinely new problem is **geography**. At 347k reads/s worldwide, a user in Jakarta should not be redirected by a server in Virginia. Move redirect pods and cache to 5 regions, replicate the KV store asynchronously, and accept that a link created in one region may be unknown in another for a few seconds. Handle the miss by falling back to the home region once. The alternative — synchronous global replication — would add 150 ms to every **write** to save a rare 150 ms on a read. Wrong trade. **Decision: async replication with a cross-region fallback read.**

Design 2 — a rate limiter for a public API

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 13 [Agoda · pattern]

Design the rate limiter that sits in front of a public API. 2 billion requests a day, 40 million distinct callers, and the limit must hold across every gateway machine at once.

SOLUTION

Step 1 — Clarify

Question I ask	Why the answer changes the design
What is limited — a user, an API key, an IP, or one endpoint?	It decides the key, and therefore how many keys exist and how much memory they need.
One global limit, or a limit per plan?	Per plan means the limit is data , looked up per request, so it needs its own cache.
Must the limit be exact, or is a 10% overshoot acceptable?	Exact costs one round trip per request. 10% buys you a local cache and 20x less traffic.

What happens if the limiter store is down — allow or block?	This is a product decision, not an engineering one, and it must be made before the outage.
Are bursts normal?	Yes → token bucket (Piece 3). “Never more than N per minute” → sliding window (Piece 4).
Does the limiter run inside the gateway or as its own service?	A separate service adds a network hop to every request in the company.

Answers I will assume, said out loud: **limit per API key per endpoint group; three plans — free 60/min, paid 600/min, partner 6,000/min; up to 10% overshoot is acceptable; bursts are allowed; the limiter is a library inside the gateway process, with a shared store behind it; if the store is unreachable we fail open with a local fallback.**

Step 2 — Scale estimate

Checks per second. Every request is checked, so the check rate **is** the request rate.

$$2,000,000,000/86,400 = 23,148 \text{ checks/second}$$

Peak at 3x: $23,148 \times 3 = 69,444$ checks/second.

How many keys.

$$2,000,000,000/40,000,000 = 50 \text{ requests per caller per day}$$

Traffic is not spread evenly over 24 hours. Take an eighth of the day’s callers as active in the busiest hour: $40,000,000/8 = 5,000,000$ live keys.

Memory. A token bucket is two numbers — tokens left and last-seen time, 16 bytes. With the key string and store overhead, call it **100 bytes** per key.

$$5,000,000 \times 100 = 500,000,000 \text{ B} = 500 \text{ MB}$$

That fits in one Redis node’s memory easily.

Throughput is the real constraint, not memory. One Redis box does about 100,000 simple ops/second.

$$69,444/100,000 = 69\% \text{ of one box}$$

69% leaves no headroom and no failover. Shard by key across 4 nodes:

$$69,444/4 = 17,361 \text{ ops/second per node} = 17\% \text{ loaded}$$

Latency. The API promises a p99 of 80 ms. A same-datacentre Redis round trip is about 0.5 ms.

$$0.5/80 = 0.6\% \text{ of the latency budget}$$

Acceptable — but we will spend far less than that in Step 6.

THE ONE-LINE SUMMARY YOU PUT ON THE BOARD

69,444 checks/s peak · 5 million live keys · 500 MB of state · 4 store shards at 17% · 0.5 ms added to an 80 ms budget.

Memory is trivial. Throughput and the atomicity of one check are the whole problem.

Step 3 — API surface

A rate limiter has almost no API of its own. Its interface is a **header contract**, and that contract is what your users actually integrate against.

the header contract, plus the two admin endpoints

TEXT

Every response from the gateway carries:

X-RateLimit-Limit: 600 requests allowed in the window

```
X-RateLimit-Remaining: 417          tokens left right now
X-RateLimit-Reset:      1789412460  unix seconds when it is full again
```

When the caller is over the limit:

```
HTTP 429 Too Many Requests
Retry-After: 3                      seconds — computed, never guessed
{ "error": "rate_limited", "scope": "key:ak_31f2", "limit": 600,
  "window": "1m", "retryAfterMs": 2840 }
```

```
GET /v1/limits/{apiKey}
200 { "plan": "paid", "rules": [ {"group": "search", "limit": 600, "windowSec": 60},
                                {"group": "write", "limit": 60, "windowSec": 60} ] }
```

```
PUT /v1/limits/{apiKey}
body { "plan": "partner", "overrides": { "search": 12000 } }
200 { "appliedAt": "2026-09-15T04:10:00Z", "propagationSec": 60 }
```

TRAP

Retry-After must be computed, not invented. A fixed `Retry-After: 60` makes every blocked client wake up at the same second and hit you together — you have built a synchronised herd out of your own error responses. Compute it from the bucket, as `retryAfterMs()` in Piece 3 does, and then add a random 0 to 20% jitter so two clients blocked in the same millisecond do not return in the same millisecond.

Step 4 — Data model

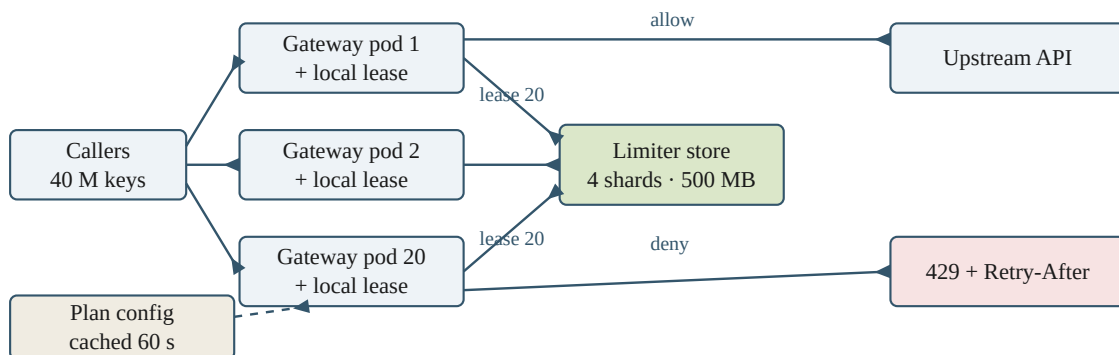
Key	Value	Note
<code>rl:{apiKey}:{group}</code>	hash: tokens, lastMs	the only hot key. TTL 3600 s, so idle callers cost nothing.
<code>plan:{apiKey}</code>	string: plan name + overrides	read once per minute per gateway pod, then cached in process
<code>rules:{plan}:{group}</code>	string: limit, windowSec	changes a few times a year; a 60-second cache is plenty

Which store? The access pattern is: one key, read-modify-write, 69,444 times a second, with a hard requirement that the read and the write are one indivisible step. That is exactly what a single-threaded in-memory store with server-side scripting is for. **Decision: Redis, sharded by `apiKey`, with the whole check written as one Lua script.**

A relational database would give the same atomicity with `SELECT ... FOR UPDATE`, but at 69,444 row locks a second on the same few hot rows it would spend all its time in lock contention, and a MySQL box does 5,000 writes/s, so you would need $69,444 / 5,000 = 14$ nodes to do what 4 Redis nodes do at 17% load.

Redis scripting is Lua, and that is not a JavaScript decision we can talk our way out of — it is what the server runs. The rule from the language policy applies: the **concept** is a token bucket, which you write in JavaScript to think about it, and the **deployment** is four lines of Lua because atomicity has to happen inside the store.

Step 5 — Architecture



Every arrow into the store is a lease of 20 tokens, so 20 requests cost one round trip. Under the limit that is 20x fewer store calls.

The check happens inside the gateway pod. The store is touched once per lease, not once per request. Plan config is cached in process for 60 seconds.

Trace one request:

1. A request arrives at gateway pod 7 with `Authorization: Bearer ak_31f2`.
2. The pod resolves the plan from its in-process cache (refreshed every 60 s). Zero network.
3. The pod looks in its own memory for an unused lease of tokens for that key.
4. **Lease available:** decrement it and forward upstream. Cost: one map lookup, 0.01 ms.
5. **No lease:** run the Lua script on the right shard, which refills the bucket and hands back up to 20 tokens in one round trip. Cost: 0.5 ms, once per 20 requests.
6. **Store returns zero tokens:** return 429 with a computed `Retry-After`, and stop asking that shard for 200 ms so a blocked caller does not turn into a store-hammering loop.

Step 6 — Deep dive

Deep dive A: counting per-server is the classic wrong answer.

You have 20 gateway pods. Two tempting shortcuts, both broken:

Shortcut	What happens	Error
Each pod keeps the full limit	A caller spread evenly across pods gets $20 \times 600 = 12,000/\text{min}$ instead of 600.	20x too many
Each pod keeps limit $/N$	$600/20 = 30$ per pod. A caller on one keep-alive connection is pinned to one pod and gets 30 instead of 600.	95% too few

$1 - 30/600 = 95\%$. Both failures are large, and both are invisible in testing with one pod. **The count has to be shared. The only question left is how often you touch the shared thing.**

Deep dive B: a check is not a read followed by a write.

Here is the bug, made visible. Two versions of the same limiter, both against the same fake store, 50 concurrent requests, limit 10.

ratelimit-race.js — run with: `node ratelimit-race.js`

JS

```

class FakeRedis {
  constructor() { this.m = new Map(); }
  async get(k) { await tick(); return this.m.get(k) ?? 0; } // a network hop
  async set(k, v) { await tick(); this.m.set(k, v); } // another one
  // ONE round trip that reads, decides and writes with nothing in between
  async incrIfBelow(k, limit) {
    await tick();
    const cur = this.m.get(k) ?? 0;
    if (cur >= limit) return { allowed: false, count: cur };
    this.m.set(k, cur + 1);
  }
}
  
```

```

    return { allowed: true, count: cur + 1 };
  }
}
const tick = () => new Promise(r => setImmediate(r));

async function unsafe(redis, key, limit) {
  const n = await redis.get(key);      // <- the gap opens here
  if (n >= limit) return false;
  await redis.set(key, n + 1);          // <- everyone writes the same value
  return true;
}

async function safe(redis, key, limit) {
  return (await redis.incrIfBelow(key, limit)).allowed;
}

async function race(fn, label) {
  const redis = new FakeRedis();
  const results = await Promise.all(
    Array.from({ length: 50 }, () => fn(redis, "u:7", 10));
  console.log(`${label}: limit 10, 50 concurrent -> allowed ` +
    `${results.filter(Boolean).length}, stored ${redis.m.get("u:7")}`);
}

await race(unsafe, "read-modify-write");
await race(safe, "atomic check-and-incr ");

```

actual output

JS

```

read-modify-write: limit 10, 50 concurrent -> allowed 50, stored 1
atomic check-and-incr : limit 10, 50 concurrent -> allowed 10, stored 10

```

Read that first line twice. Every one of the 50 requests was allowed against a limit of 10, **and the stored counter says 1**. All 50 read 0, all 50 wrote 1. The limiter did not slow anything down and did not know it had failed. In production this only appears under load, which is precisely when you need it.

The fix is not a lock. It is doing the whole check **inside the store**:

the token bucket of Piece 3, moved into the store

TEXT

```

-- KEYS[1] = rl:{apiKey}:{group}  ARGV = now_ms, capacity, refill_per_sec, want
local b      = redis.call('HMGET', KEYS[1], 'tokens', 'last')
local tokens = tonumber(b[1]) or tonumber(ARGV[2])
local last   = tonumber(b[2]) or tonumber(ARGV[1])
local elapsed = (tonumber(ARGV[1]) - last) / 1000
tokens = math.min(tonumber(ARGV[2]), tokens + elapsed * tonumber(ARGV[3]))
local give = math.min(tonumber(ARGV[4]), math.floor(tokens))
tokens = tokens - give
redis.call('HMSET', KEYS[1], 'tokens', tokens, 'last', ARGV[1])
redis.call('EXPIRE', KEYS[1], 3600)
return give

```

It is the same arithmetic as the TokenBucket class in Piece 3, character for character. The only thing that changed is **where it runs**. Redis executes one script start to finish before touching another command, so there is no gap for a second request to slip into.

Deep dive C: the lease — Piece 2's trick, applied to tokens.

At 69,444 checks/second, one store call per check is 69,444 ops/s. The key-block idea from Example 8 works here without modification: instead of asking for one token, ask for **twenty** and spend them locally.

lease-limiter.js — run with: node lease-limiter.js

JS

```
class CentralBucket { // ONE Redis key, touched by one atomic script
  constructor(capacity, refillPerSec) {
    this.capacity = capacity; this.refillPerSec = refillPerSec;
    this.tokens = capacity; this.lastMs = 0; this.calls = 0;
  }
  lease(nowMs, want) { // hand out up to `want` tokens in one round
    trip
    this.calls++;
    const elapsed = (nowMs - this.lastMs) / 1000;
    this.tokens = Math.min(this.capacity, this.tokens + elapsed *
this.refillPerSec);
    this.lastMs = nowMs;
    const given = Math.min(want, Math.floor(this.tokens));
    this.tokens -= given;
    return given;
  }
}

class GatewayLimiter { // lives inside each gateway process
  constructor(central, leaseSize = 20) {
    this.central = central; this.leaseSize = leaseSize;
    this.held = 0; this.quietUntil = -1;
  }
  allow(nowMs) {
    if (this.held === 0 && nowMs >= this.quietUntil) {
      this.held = this.central.lease(nowMs, this.leaseSize);
      if (this.held === 0) this.quietUntil = nowMs + 200; // empty: stop asking
    }
    if (this.held === 0) return false;
    this.held--;
    return true;
  }
}

function run(total, label) {
  const central = new CentralBucket(600, 600); // 600/s, burst 600
  const gws = Array.from({ length: 20 }, () => new GatewayLimiter(central, 20));
  let allowed = 0, denied = 0;
  for (let i = 0; i < total; i++) {
    const nowMs = (i / total) * 1000; // spread over one second
    gws[i % 20].allow(nowMs) ? allowed++ : denied++;
  }
  console.log(`${label}: allowed ${allowed}, denied ${denied}, ` +
    `central touched ${central.calls} -> ${((total / central.calls).toFixed(1))x
fewer`);
}
run(400, "under the limit (400/s)");
run(2000, "over the limit (2000/s)");
```

Twenty gateway pods, run twice. The demo uses a scaled-up limit of 600 per **second** with a burst of 600, so that one simulated second shows the whole behaviour:


```
under the limit (400/s): allowed 400, denied 0,    central touched  20 -> 20.0x
fewer
over the limit (2000/s): allowed 1199, denied 801, central touched 410 ->  4.9x
fewer
```

Three things to point at, and the third is the one that scores:

1. Under the limit the store is touched 20 times instead of 400. That is the $69,444/20 = 3,472$ ops/second version of our peak, which is 3% of a single Redis box.
2. 1,199 allowed in the overload run is **correct**, not a bug: a full bucket of 600 plus 600 refilled during that second is 1,200.
3. Over the limit the saving shrinks to 4.9x, because once the bucket is dry the leases come back partial and pods ask more often. **The lease helps most when you need it least.** That is why the 200 ms quiet period exists — without it, the one caller who is being blocked generates more store traffic than all the callers who are being served.

WHAT THE 10% OVERSHOOT ACTUALLY BUYS, IN ONE LINE

A pod holding an unused lease of 20 that crashes destroys those 20 tokens; a pod holding a lease across a second boundary spends them slightly late. With 20 pods and leases of 20, the worst-case error is $20 \times 20 = 400$ tokens against the paid plan's limit of 600 per minute — a 67% overshoot, far outside the 10% we promised. Set the lease to $600/(20 \times 10) = 3$ for a 10% bound, or keep leases of 20 only for the partner plan at 6,000/min. **Decision: lease size = $\max(1, \text{limit} / (N \times 10))$, computed per plan.** The free plan at 60/min gets a lease of 1 and pays the full round trip; it is 3% of traffic and nobody notices.

Step 7 — Trade-offs, failures, 10x

Choice	Op- tion A	Op- tion B	Decision
Where the count lives	per gate- way pod	shared store	B — per-pod is 20x too high or 95% too low, and both hide in testing
The check itself	read, de- cide, write	one atomic script in the store	B — the race is silent and only appears under load
Algorithm	sliding win- dow counter	token bucket	B — bursts are normal for an API client; we promised a rate, not a shape
Store traffic	one call per re- quest	lease a block of to- kens	B — 20x fewer calls under the limit, for a bounded 10% overshoot
Limiter placement	its own ser- vice	a li- brary in the gate- way	B — a separate service adds a hop to every request in the company
When the store is down	fail closed	fail open	B — see below; a limiter outage must not become an API outage

		with a local cap	
--	--	------------------	--

The fail-open decision, in full. If the limiter store is unreachable, blocking everything turns a limiter incident into a total outage — the limiter is now the least reliable thing in the request path and it can take down a healthy API. Allowing everything invites abuse for the length of the outage. **Decision: fail open, but not blindly.** Each pod falls back to a purely local token bucket at limit $/N$ per key. With 20 pods that caps the worst case at the true limit, penalises nobody in the normal case, and needs no store at all. The two exceptions are hard-coded to fail **closed**: the login endpoint and the OTP endpoint, where unlimited traffic is not a cost problem but a security one.

What fails	What the caller sees	What we do
One store shard dies	A quarter of keys lose their count	Those keys fall back to the local bucket at limit $/N$. The other three quarters are unaffected, because the shard key is the API key.
The whole store tier dies	Nothing, briefly	Fail open with local caps. Alarm immediately: we are now enforcing an approximation, and that must never be allowed to be the quiet normal state.
A gateway pod crashes holding leases	Nothing	Up to 20 tokens per key are lost. At a 600/s limit that is 3% of one second. Leases expire by refill anyway; no cleanup code exists or is needed.
One API key sends 50,000 requests/second	That key gets 429s	It is one hot store key on one shard. Add a per-key circuit breaker: after 60 seconds over the limit, stop calling the store for it entirely and deny locally for 5 minutes.
A plan change is made	Takes effect within 60 s	Config is cached per pod for 60 seconds. Instant propagation would mean reading config on every request — 69,444 extra reads/second to make a yearly event faster.
Clock skew between pods	Small overshoot	The refill arithmetic uses elapsed time from the store's own clock inside the script, not the pod's clock. Pods never write a timestamp.

At 10x (20 billion requests/day):

- Checks become $20,000,000,000/86,400 = 231,481/s$ average and $694,444/s$ at peak.
- Without leases that is $694,444/100,000 = 7$ Redis nodes saturated. With leases of 20 it is $694,444/20 = 34,722$ ops/s — **one node's worth of work**. Run 3 for failure tolerance, not for throughput. This is the whole payoff of Deep dive C.
- Live keys become 50 million, so state becomes $50,000,000 \times 100 = 5$ GB. Still small, but now it must be sharded for memory as well as for throughput.
- The genuinely new problem is **geography**. With gateways in 5 regions, a globally exact limit needs a cross-region round trip of 150 ms on every lease, which is 300x the local cost. **Decision: split the limit between regions in proportion to each region's traffic over the last 5 minutes, recomputed every minute, and enforce locally.** A caller who moves region can exceed their limit for up to one minute. We accept that: the alternative is putting a 150 ms intercontinental hop in front of every request in the company to make an approximate number slightly less approximate.

Design 3 — a notification system

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 14 [Grab · pattern]

Design the service that sends every push, SMS and email in the company. 500 million notifications a day, three third parties that all fail sometimes, and a marketing team that wants to send 20 million messages in ten minutes.

SOLUTION

Step 1 — Clarify

Question I ask	Why the answer changes the design
What classes of notification exist?	OTP, transactional and marketing have different deadlines. One queue cannot serve all three.
What is the latency promise for each class?	An OTP under 10 seconds is a hard product requirement. Marketing is “today”.
What is the biggest single send?	The peak here is self-inflicted . A 20 million blast is scheduled by us, not by users.
Who owns the opt-out, and is it per channel or global?	It is a legal requirement, so it must be checked at send time, not at campaign time.
Do the providers give delivery receipts?	Yes → a whole second inbound stream, and “sent” stops meaning “delivered”.
Do we pay per message?	SMS costs money. That single fact reorders every channel decision in the system.

Assumed and said out loud: **500 million notifications/day; 70% push, 25% email, 5% SMS; largest blast 20 million in 10 minutes; opt-out is per channel per category; receipts arrive by webhook; SMS costs 12 paise a message**, as in Piece 5.

Step 2 — Scale estimate

Average rate.

$$500,000,000/86,400 = 5,787 \text{ notifications/second}$$

The blast.

$$20,000,000/600 \text{ s} = 33,333 \text{ notifications/second}$$

$$33,333/5,787 = 5.76 \times \text{the average}$$

Note what kind of peak this is. Nobody surprised us. **We** pressed the button. So the answer is not “provision for 33,333/s” — it is “shape it”, which is much cheaper.

Channel volumes.

$$500,000,000 \times 0.70 = 350,000,000 \text{ push} \quad \times 0.25 = 125,000,000 \text{ email}$$

$$500,000,000 \times 0.05 = 25,000,000 \text{ SMS}$$

What the SMS costs.

$$25,000,000 \times 12 \text{ paise} = 300,000,000 \text{ paise} = 3,000,000 \text{ rupees/day}$$

$$3,000,000 \times 365 = 1,095,000,000 \text{ rupees} = 109.5 \text{ crore per year}$$

Say that number out loud. It is the largest number in the design and it is not a latency, a byte or a QPS — it is a bill. Moving 20% of non-OTP SMS to push saves $17,000,000 \times 0.2 \times 12 = 40,800,000$ paise = 408,000 rupees a day, about 14.9 crore a year, which is why `costPaise()` sits on the channel class in Piece 5.

SMS capacity is contracted, not elastic. Two providers at 500 messages/second each = 1,000/s total, and that number is in a contract you cannot exceed by asking nicely. OTP volume is 8 million/day:

$$8,000,000/86,400 = 92.6 \text{ OTP/second average} \times 5 = 463 \text{ at peak}$$

$$1,000 - 463 = 537 \text{ SMS/second left for everything else}$$

Storage. One notification row (id, user, template, channel, state, timestamps, provider reference) is about 400 bytes.

$$500,000,000 \times 400 = 200,000,000,000 \text{ B} = 200 \text{ GB/day}$$

Seven days: $200 \times 7 = 1,400 \text{ GB} = 1.4 \text{ TB}$. Thirty days: 6 TB. **Decision: keep 7 days of full rows and roll everything older into one counter row per (template, day).** Nobody has ever asked “did this exact promotional email send 25 days ago?” and paid 4.6 TB for the answer.

Dedupe memory. A 24-hour idempotency window holds every key:

$$500,000,000 \times 70 \text{ B} = 35,000,000,000 \text{ B} = 35 \text{ GB}$$

A one-hour window holds $500,000,000/24 = 20,833,333 \text{ keys} \times 70 = 1.46 \text{ GB}$. The whole retry ladder finishes in 85 seconds, so an hour covers every retry with 42x margin. **Decision: a one-hour Redis window for speed, plus a unique index on (user_id, dedupe_key) in the notification log for the long tail.** 1.46 GB instead of 35 GB, and the database still makes the guarantee.

THE BOARD SUMMARY

5,787/s average · a **self-inflicted** 33,333/s blast · 1,000 SMS/s of contracted capacity · 200 GB/day · 109.5 crore a year of SMS.

This system’s hard parts are a queue-priority problem and a money problem, not a throughput problem.

Step 3 — API surface

one call to send, one to schedule millions, one to be told what happened

TEXT

```
POST /v1/notifications
  header Idempotency-Key: otp-9f13-1789412400
  body { "userId": 88214, "templateId": "otp_login",
        "data": { "code": "482913", "mins": 5 },
        "class": "otp",
        "channels": ["sms", "push"],
        "expiresAt": "2026-09-15T04:20:00Z" }
  202 { "notificationId": "ntf_01J8...", "state": "QUEUED" }
  200 { "notificationId": "ntf_01J8...", "state": "SENT", "replayed": true }
  422 { "error": "opted_out", "channel": "sms" }

POST /v1/campaigns
  body { "templateId": "sale_sep", "audienceId": "aud_412", "class": "marketing",
        "channels": ["push", "email"], "spreadOverMin": 120,
        "quietHours": { "from": "21:00", "to": "09:00", "tz": "user" } }
  202 { "campaignId": "cmp_77", "estimated": 20000000, "startsAt": "..." }

GET /v1/notifications/{id}
  200 { state: "DELIVERED", channel: "sms", attempts: 2,
        providerRef: "sm_77aa", deliveredAt: "..." }
```

```
POST /v1/webhooks/providers/{provider}      (inbound, from the provider)
  body   { providerRef, status:"delivered"|"bounced"|"blocked", at }
  200    always – never make a provider retry because our database was slow
```

TRAP

expiresAt is not decoration. An OTP that arrives 40 minutes late is worse than one that never arrives: the user has already asked for a second code, and now two codes are valid. Every message in the queue carries a deadline, and a worker that picks up an expired message **drops it and records why**. Without this, a provider outage turns into a flood of stale, confusing messages the moment it recovers.

Step 4 — Data model

four tables and three queues

TEXT

```
notifications                                -- 7 days hot, then rolled into counters
  notification_id  ulid      PK              -- time-ordered, so the partition is the day
  user_id         bigint    SHARD KEY INPUT
  template_id, class, channel
  state          enum      QUEUED|SENT|DELIVERED|BOUNCED|FAILED|DROPPED
  attempts       int
  dedupe_key      text
  provider_ref    text      -- how the receipt finds its way back
  created_at, sent_at, settled_at
  UNIQUE (user_id, dedupe_key)              -- the guarantee, not the cache
  INDEX (provider_ref)                    -- the webhook's only lookup

preferences                                    -- checked at SEND time, never at campaign time
  user_id, category, channel, allowed BOOL, updated_at
  PRIMARY KEY (user_id, category, channel)

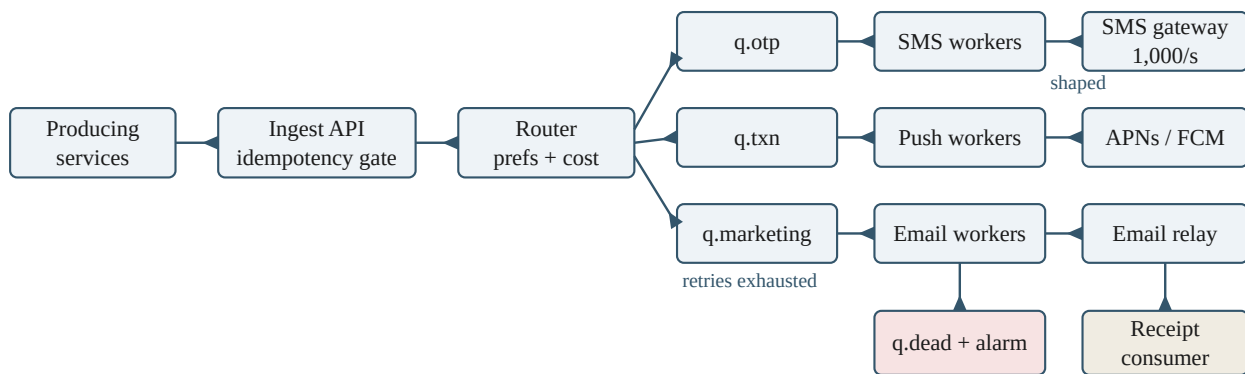
devices
  user_id, device_id, platform, token, last_seen_at, invalid_at
  -- invalid_at set when a provider says the token is dead. Never retried.

templates
  template_id, locale, channel, body, version  -- rendered by Piece 5's Template

-- queues (Kafka topics or SQS queues, one per class)
q.otp                partitions 12    workers reserved 20%
q.transactional      partitions 24    workers reserved 30%
q.marketing          partitions 48    workers reserved 50%
q.dead              everything that exhausted its retries
```

Why three queues and not one with a priority field? Because a priority field inside one queue still makes the OTP wait behind 20 million marketing messages that were **already delivered to the queue**. You cannot re-order a log after the fact. Separate queues with separately reserved workers is the only version that actually works, and Step 6 shows the arithmetic.

Step 5 — Architecture



The SMS arrow is the only one that is rate-shaped, because it is the only one with a contracted ceiling and a per-message cost. Receipts arrive minutes later and update the row by `provider_ref`. “SENT” is a claim; “DELIVERED” is evidence.

One ingest path, three queues, one worker pool per channel. Receipts come back on their own path and are the only thing that turns SENT into DELIVERED.

Trace one OTP:

1. The login service calls `POST /v1/notifications` with `Idempotency-Key: otp-9f13-1789412400`.
2. The ingest API claims the key with `SET key NX EX 3600`. If the claim fails, it returns the stored result and does nothing else. This is Piece 5’s seen set, in Redis.
3. The router reads preferences, drops channels the user opted out of, and orders the rest by `costPaise()`. For class `otp` it keeps the order the caller asked for, because a free channel that the user will not see is not cheaper.
4. It writes one row and publishes to `q.otp`.
5. An SMS worker picks it up, checks `expiresAt`, passes the notification id to the provider as the provider’s own idempotency reference, and records `provider_ref`.
6. Minutes later a webhook arrives with that reference. The receipt consumer moves the row from SENT to DELIVERED, or to BOUNCED — and a bounce on a push token sets `invalid_at` on the device so we never spend another attempt on it.

Step 6 — Deep dive

Deep dive A: one queue kills the OTP, and here is the number.

queues.js — run with: `node queues.js`

JS

```

function waitSeconds(aheadOfYou, drainRate) { return aheadOfYou / drainRate; }

const BLAST = 20_000_000; // a campaign fired at 11:00
const DRAIN = 5_787; // messages/second the whole worker fleet can push

console.log("ONE shared queue");
console.log(" OTP queued behind the blast waits",
  (waitSeconds(BLAST, DRAIN) / 60).toFixed(1), "minutes");

console.log("THREE queues, workers reserved per class");
const reserved = { otp: 0.20, transactional: 0.30, marketing: 0.50 };
for (const [cls, share] of Object.entries(reserved)) {
  console.log(` ${cls.padEnd(14)} drain ${ (DRAIN * share).toFixed(0).padStart(5) } / s`);
}
console.log(" OTP queued behind 500 other OTPs waits",
  waitSeconds(500, DRAIN * reserved.otp).toFixed(2), "seconds");
console.log(" the same blast now takes",
  (waitSeconds(BLAST, DRAIN * reserved.marketing) / 3600).toFixed(2), "hours");

```

```

ONE shared queue
  OTP queued behind the blast waits 57.6 minutes
THREE queues, workers reserved per class
  otp          drain  1157/s
  transactional drain  1736/s
  marketing     drain  2894/s
  OTP queued behind 500 other OTPs waits 0.43 seconds
  the same blast now takes 1.92 hours

```

57.6 minutes against a 10-second promise. And the arithmetic is embarrassingly simple: $20,000,000/5,787 = 3,456$ seconds. The fix costs nothing except deciding to do it before the campaign, rather than during it.

The second line matters just as much: the blast now takes 1.92 hours instead of 58 minutes. **We made the unimportant thing slower on purpose.** That sentence, said out loud, is the whole idea of quality of service.

Deep dive B: shaping a 33,333/s blast into a 1,000/s pipe.

The SMS gateway is contracted at 1,000 messages/second across two providers. OTP peak takes 463/s of that, leaving 537/s. If a campaign puts one million SMS into the queue:

$$1,000,000/537 = 1,862 \text{ seconds} = 31 \text{ minutes of solid SMS}$$

during which every OTP is competing for the same pipe.

Three mechanisms, all built from Tier 1 pieces:

1. **A token bucket per provider**, exactly Piece 3, with capacity 500 and refill 500/s. The worker calls `allow()` before every send and sleeps on `retryAfterMs()` when it is empty. This is the promise we made in the contract, enforced in our own code rather than discovered through the provider's 429s.
2. **A reservation**. The OTP worker pool has its own bucket of 500/s; the marketing pool gets what is left. A blast can never eat an OTP's capacity, because it is drawing from a different bucket.
3. **A spread**. `spreadOverMin: 120` in the campaign API means the scheduler releases $20,000,000/120 = 166,667$ messages per minute = 2,778/s instead of 33,333/s. The campaign finishes two hours later and costs a tenth of the peak capacity.

Decision: marketing never uses SMS unless the user has no valid push token, and every campaign is spread by default. The cost side decides it: the same message costs 12 paise by SMS and 0 by push.

Deep dive C: retries that stop.

```

const attempts = [0, 1, 4, 16, 64]; // seconds after the first try
console.log("retry schedule (seconds):", attempts.join(", "),
  "| total window", attempts.reduce((a, b) => a + b, 0), "s");
let live = 10_000_000; // messages that failed once, per day
const failAgain = 0.30; // 30% of retries fail again
for (let i = 1; i < attempts.length; i++) {
  live = Math.round(live * failAgain);
  console.log(`  attempt ${i + 1}: ${live.toLocaleString()} messages ` +
    `(${(live / 86400).toFixed(1)}/s)`);
}
console.log(`  dead-letter queue: ${live.toLocaleString()} per day`);

```

```

retry schedule (seconds): 0, 1, 4, 16, 64 | total window 85 s
  attempt 2: 3,000,000 messages (34.7/s)
  attempt 3: 900,000 messages (10.4/s)

```

attempt 4: 270,000 messages (3.1/s)
 attempt 5: 81,000 messages (0.9/s)
 dead-letter queue: 81,000 per day

Start from a 2% first-attempt failure rate — $500,000,000 \times 0.02 = 10,000,000$ failures a day — and assume 30% of each retry fails again. The retry traffic **shrinks by 70% per round**, so the extra load is 34.7/s on top of 5,787/s, which is 0.6%. Retries are cheap when they back off and stop.

THE FOUR RULES OF A RETRY BUDGET

1. **Exponential, with jitter.** 1, 4, 16, 64 seconds, each multiplied by a random 0.5 to 1.5. Without jitter, every message that failed during a 30-second outage retries at the same instant and re-creates the outage.
2. **A fixed number of attempts, then the dead-letter queue.** 81,000 a day land there. That is 0.016% of volume and it is a number a human can look at.
3. **Never retry a permanent failure.** An invalid device token, an unsubscribed email, a blocked number: these fail identically every time. Mark the device `invalid_at` and stop. Retrying them is how a 2% failure rate quietly becomes a 20% one.
4. **Respect the deadline over the schedule.** If `expiresAt` passes mid-ladder, drop the message and record DROPPED. A retry that succeeds after the code has expired is a bug that looks like a success.

Step 7 — Trade-offs, failures, 10x

Choice	Option A	Option B	Decision
Queueing	one queue per priority field	one queue per class	B — a priority field cannot re-order messages already in the log
Blast handling	provision for 33,333 s	spread it over 30 minutes	B — the peak is ours; shaping is free and capacity is not
Dedupe window	24 h in Redis, 35 GB	1 h in Redis + a unique index	B — the ladder is 85 s, so an hour is 42x margin at 4% of the memory
Send call	synchronous, in the caller's request	202 and a queue	B — a provider taking 800 ms must never hold a user's HTTP thread
Delivery truth	trust the provider's 200 OK	wait for the receipt	B — “accepted by the gateway” is not “arrived on the phone”

		web-hook	
Channel order	fixed per template	cheapest channel the user can receive	B — it is the 14.9 crore a year decision from Step 2
Preferences check	at campaign build time	at send time	B — a user who opts out during a 2-hour spread must stop receiving immediately

What fails	What the user sees	What we do
One SMS provider is down	Nothing	The router fails over to the second provider. Capacity halves to 500/s, so OTP keeps its reservation and marketing SMS pauses automatically — the reservation is the degradation plan.
Both SMS providers are down	OTP does not arrive	Fall back to the next channel in the order: push, then email. An OTP by email is worse than by SMS and much better than nothing. Alarm at the first 30 seconds.
A provider accepts everything and delivers nothing	Silence	This is why receipts exist. Alarm when the ratio of DELIVERED to SENT for a provider drops below its 7-day baseline by more than 20%.
The same notification is enqueued twice	One message	The unique index on (user_id , dedupe_key) rejects the second row. Redis is the fast path; the index is the guarantee.
A campaign targets the wrong audience	Millions of wrong messages	A kill switch that drains q.marketing without sending, plus a rule that any campaign over 1 million recipients sends to a 1% sample first and waits 10 minutes.
The receipt webhook floods us	Nothing	Return 200 immediately and process asynchronously. Never make a provider retry because our database was slow — their retry policy is not ours to control.

At 10x (5 billion notifications/day):

- Average becomes $5,000,000,000/86,400 = 57,870/\text{second}$, and the largest blast becomes 200 million in 10 minutes = $333,333/\text{second}$ if unshaped. Shaping is no longer an optimisation; it is the only version that exists.
- Storage becomes 2 TB/day. The 7-day window is 14 TB. Partition by day and drop partitions.
- SMS becomes 250 million/day = 3 crore rupees **a day**. At that price the company builds its own aggregator relationships and negotiates per-country rates, and the routing decision becomes per-country as well as per-cost.
- The genuinely new problem is **provider capacity**, not ours. 57,870/s of push is fine — APNs and FCM are built for it. But no SMS aggregator will sell you 10,000/s in one country. **Decision: route SMS across a pool of 8 aggregators with a token bucket per aggregator per country, and treat the pool like a connection pool — health-checked, weighted by recent delivery rate, and drained rather than switched when one degrades.**

PRACTICE · TIER 2 target: 40 min

1. The shortener's cache holds 20 million links in 5 GB. A competitor's link is spammed and one code receives 40% of all redirects. Compute the load on that single cache node and say whether anything breaks.
2. Rework the rate limiter for a **sliding window counter** instead of a token bucket. What changes in the store, in the memory per key, and in what a client can get away with?
3. A partner is on 6,000 requests/minute and complains that they see 429s at 5,200. Give three possible causes, and the one measurement that tells them apart.
4. Design the quiet-hours rule for notifications: a user in one time zone, a campaign spread over 2 hours, and a deadline. What happens to messages that fall inside quiet hours?
5. A template bug puts the wrong name in 4 million already-queued emails. Write the response: what you stop, what you cannot stop, and what you send afterwards.

ANSWER KEY

1. 40% of 34,722 peak redirects/s = 13,889/s all landing on **one** Redis key on **one** node. A Redis box does 100,000 ops/s, so that node is at 14% — it is fine. Nothing breaks, because a hot **read** key in memory is the best case in the whole system. What would break is the click counter if it were a synchronous increment on one row: 13,889 writes/s to a single row is a guaranteed hot-shard failure, which is exactly why Design 1 batches it.
2. Store three numbers per key instead of two, so memory goes from 100 B to 120 B per key — 5 million keys is still 600 MB. The store script changes from a refill calculation to the blend in Piece 4. What the client loses is the **burst**: a token bucket lets a paid caller fire 600 requests in one second and then wait; a sliding window spreads them. Pick the window only when you promised a partner a hard ceiling.
3. Causes: (a) their traffic is bursty and the bucket capacity is smaller than their burst; (b) leases held by crashed pods are destroying tokens; (c) they are being counted across two endpoint groups that share a key. The measurement that separates them: **log given** from the lease script per key per second and compare its sum to 6,000. If the sum is 6,000 the limit is fine and the burst is the problem; if it is 5,200 tokens are being lost.
4. Convert the user's local quiet window to UTC at send time, not at campaign time, because time zones and the user's own setting can both change mid-campaign. A message that lands inside quiet hours is **rescheduled** to the window's end with jitter — not dropped, because it is marketing and the deadline is generous. If a message's **expiresAt** falls before the quiet window ends, drop it; a deadline always beats a schedule.
5. Stop: pause **q.marketing** and drain it without sending — that recovers everything not yet picked up by a worker. Cannot stop: whatever the email relay has already accepted, which is irreversible the moment the relay says 200. Measure that number exactly from the **SENT** count before you say anything. Afterwards: one correction email to the affected list only, never to the whole audience, and add a canary — any campaign over 1 million sends to 1% first and waits 10 minutes.

Design 4 — a news feed

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 15 [Amazon · pattern]

Design the feed a user sees when they open the app. 200 million daily active users out of 600 million registered, 8 million posts a day, an average of 200 followers — and 500 accounts with two million followers each.

SOLUTION

Step 1 — Clarify

Question I ask	Why the answer changes the design
How many followers does the biggest account have?	This one number decides whether a single fan-out strategy can work at all.
Chronological or ranked?	Ranked adds a scoring step and means the feed cannot be a simple append-only list.
How far back can a user scroll?	It caps the stored inbox. Unlimited scroll and a 200-item cap are different systems.
How fresh must a post be?	Seconds means push on write. Minutes lets you batch, which is 10x cheaper.
How many registered users are active on a given day?	It tells you what fraction of your fan-out writes will never be read.
Can a post be deleted, or a follow undone?	If yes, you can never treat a written inbox row as final.

Assumed and said out loud: **200 million DAU of 600 million registered; 8 million posts/day; average 200 followers; the largest 500 accounts average 2 million followers and post 3 times a day; the feed is ranked; scroll is capped at 200 posts; a post should appear within 60 seconds; deletes and unfollows are handled at read time.**

Step 2 — Scale estimate

Reads. 200 million users opening the feed 8 times a day:

$$200,000,000 \times 8 = 1,600,000,000 \text{ feed reads/day}$$

$$1,600,000,000 / 86,400 = 18,518 \text{ reads/second average} \quad \times 3 = 55,555 \text{ at peak}$$

Writes look tiny.

$$8,000,000 / 86,400 = 92.6 \text{ posts/second}$$

And then fan-out multiplies them by 200.

$$8,000,000 \times 200 = 1,600,000,000 \text{ inbox rows/day}$$

$$1,600,000,000 / 86,400 = 18,518 \text{ row writes/second} \quad \times 3 = 55,555 \text{ at peak}$$

Stop and look at those two numbers. **The write path and the read path are the same size — 18,518 each.** That almost never happens. It happens here because one API call at 92.6/second turns into 18,518 writes/second behind it, and saying that out loud is how you show you understand fan-out.

Now the 500 big accounts. If their posts were fanned out the same way:

$$500 \times 3 \times 2,000,000 = 3,000,000,000 \text{ rows/day}$$

$$3,000,000,000 / (1,600,000,000 + 3,000,000,000) = 65.2\% \text{ of all fan-out writes}$$

Five hundred accounts out of six hundred million would generate 65% of the entire write load of the system. That is the celebrity problem, in one division.

How much of the fan-out is wasted?

$$200,000,000 / 600,000,000 = 33\% \text{ of followers are active today}$$

So 67% of every row we push is never read by anybody.

Inbox memory. One inbox entry is post id 8 B + author id 8 B + score 4 B + timestamp 8 B plus overhead ≈ 50 bytes. Cap at 200 entries per user:

$$200,000,000 \times 200 \times 50 = 2,000,000,000,000 \text{ B} = 2 \text{ TB}$$

$$2,000,000,000,000 / 100 \text{ GB per node} = 20 \text{ nodes}$$

That 2 TB is the **worst case**. The average inbox is much shorter: $1,600,000,000 / 200,000,000 = 8$ new rows per active user per day, so a 200-entry cap holds 25 days. Cap by 14 days as well and the typical inbox is $14 \times 8 = 112$ entries:

$$200,000,000 \times 112 \times 50 = 1,120,000,000,000 \text{ B} = 1.12 \text{ TB}$$

Post storage. One post row (text, author, media references, counters) is about 1 KB.

$$8,000,000 \times 1,000 = 8,000,000,000 \text{ B} = 8 \text{ GB/day} \times 365 = 2.92 \text{ TB/year}$$

A hot cache of the last 3 days is $8 \times 3 = 24 \text{ GB}$, which is one node.

THE BOARD SUMMARY

55,555 feed reads/s peak · 55,555 inbox writes/s peak · 2 TB of inboxes · 24 GB of hot posts · **500 accounts would be 65% of the write load** · 67% of pushed rows are never read.

This is not a throughput problem. It is a “who do you push to” problem.

Eight new posts per active user per day is worth pausing on. A strictly chronological feed of 8 items would be empty by the second scroll, which is exactly why real feeds are **ranked and padded** — with older posts the user has not seen and with recommended posts from accounts they do not follow. The ranking is not a nice-to-have bolted on top; it is what makes a feed of 8 items into a feed you can scroll.

Step 3 — API surface

one read endpoint that matters, and a cursor that is not a page number

TEXT

```
GET /v1/feed?limit=20&cursor=eyJzIjoyLjE0LCJwIjoicF8zM...
200 { "items": [ { "postId":"p_8812", "authorId":"u_71",
                  "text":"...", "media":[...], "score":2.14,
                  "createdAt":"2026-09-15T04:02:11Z" } ],
      "nextCursor":"eyJzIjowLjk4...", "asOf":"2026-09-15T04:11:02Z" }
note cursor = last (score, postId) seen, NOT an offset. Offsets break
the moment a new post arrives between page 1 and page 2.

POST /v1/posts
header Idempotency-Key: 8c11-...
body { "text":"...", "mediaIds":[...], "audience":"followers" }
201 { "postId":"p_8812", "fanout":"queued", "visibleWithinSec":60 }

DELETE /v1/posts/{postId}
204 note this writes a tombstone. It does NOT delete inbox rows.

POST /v1/follows { "targetId":"u_71" } 201
DELETE /v1/follows/{targetId} 204
note also a tombstone-style change: old rows stay, the read filters them.
```

TRAP

Never paginate a feed with offset and limit. Between page 1 and page 2 a new post arrives at the top, every item shifts down by one, and the user sees item 20 twice and never sees item 21. Use a cursor made of the last item’s (score, postId) and ask for “strictly after this”. It is the same amount of code and it is correct.

Step 4 — Data model

three stores, because three very different jobs

TEXT

```
-- IN MEMORY, the hot path
inbox:{userId}      sorted set    member = postId, score = rank score
                                capped at 200, TTL 14 days
hot:{authorId}      list          last 50 post ids of a BIG account
                                read at request time, never pushed

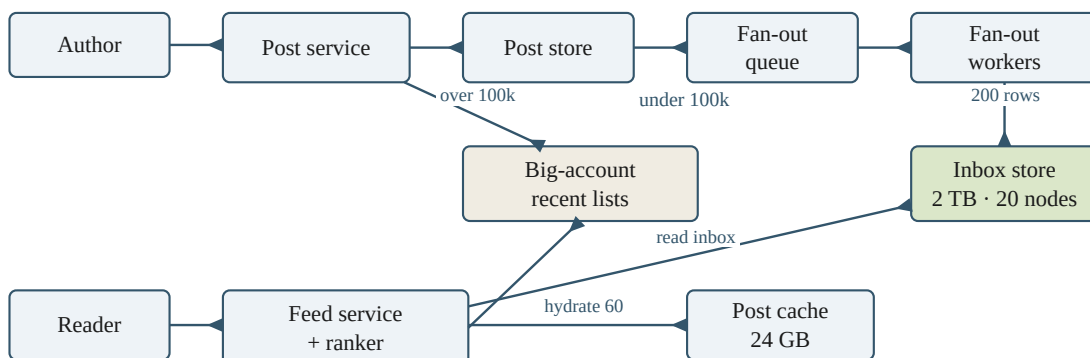
-- POST STORE, key-value, partitioned on post_id
posts
  post_id    ulid      PARTITION KEY  (time-ordered, so recent posts cluster)
  author_id  bigint
  body, media_ids, created_at
  deleted_at timestamp null          -- the tombstone; rows are never removed
  counters   map        -- likes/comments, updated by a stream job

-- GRAPH STORE, the two directions are two tables on purpose
follows_out  follower_id PARTITION KEY, followee_id SORT KEY  -- "who I follow"
follows_in   followee_id PARTITION KEY, follower_id SORT KEY  -- "who follows me"
note: fan-out reads follows_in; the read path reads follows_out.
      Duplicating the edge costs 2x storage on a tiny table and removes
      a scan from both hot paths.
```

Why a sorted set and not a list? Because the feed is ranked, and a sorted set gives you “top 20 by score” and “everything after this score” in one operation, plus an automatic way to trim to 200. A list would force a full read and a sort on every request.

Why is the biggest account’s list stored separately? Because it is read, not pushed. One key, 50 post ids, read by however many followers open the app. It is a cache entry, not a fan-out target.

Step 5 — Architecture



The threshold at 100,000 followers is the whole design. Below it, push. Above it, write once and let readers pull. A deleted post is a tombstone checked during hydrate. No inbox row is ever deleted by anything except the 200-entry cap.

Write path along the top, read path along the bottom. A post from a normal account is pushed into 200 inboxes. A post from a big account is written once and pulled at read time.

Trace one post by a normal account:

1. `POST /v1/posts` writes one row to the post store and returns 201 in about 20 ms. The user is done.
2. A message goes on the fan-out queue with the post id and the author id.
3. A worker reads `follows_in` for that author — 200 rows — filters to followers seen in the last 7 days, and pushes an entry into each of their sorted sets.

- 67% of those followers are not active today. We push anyway for the active-7-day set, which is a compromise: filtering to **today's** actives would be cheaper but a user returning tomorrow would find an empty feed.

Trace one post by a big account:

- The same 201 in 20 ms.
- The fan-out service sees `followerCount > 100,000` and pushes **nothing**. It prepends the post id to `hot: {authorId}` and stops. One write instead of two million.

Trace one feed read:

- Read the reader's inbox sorted set: top 200 by score, one operation.
- Read `follows_out` for big accounts only — at most 50 of them for a typical user — and multi-get their `hot: lists`.
- Merge the two lists, drop tombstoned and unfollowed authors, take the top 60 candidates.
- Multi-get those 60 post bodies from the post cache.
- Score, sort, return 20 with a cursor.

Step 6 — Deep dive

Deep dive A: the hybrid, and where the threshold goes.

	Push on write (fan-out)	Pull on read
Cost of one post	F writes, where F is the follower count	1 write
Cost of one feed read	1 read	1 read per big account followed
Latency when reading	lowest — the answer is already built	higher — a merge per read
Waste	67% of rows are never read	none
Worst case	2,000,000 writes for one post	a user following 500 big accounts

The threshold is where the burst stops being survivable, not where the totals cross. A 2-million-follower post at a fan-out rate of, say, 200,000 rows/second across the worker fleet takes $2,000,000/200,000 = 10$ seconds — and for those 10 seconds the queue is full of one person's post while everybody else's waits. Three such posts in the same minute and the 60-second freshness promise is broken for all 600 million users.

Decision: push below 100,000 followers, pull at or above it. Two numbers justify it:

- It removes 3 billion of 4.6 billion daily writes — 65% of the load — by changing the behaviour of 500 accounts.
- A typical user follows at most a few dozen big accounts, so the pull side costs one multi-get of under 50 keys, which is 4 ms.

feed-merge.js — run with: `node feed-merge.js`

JS

```
const HOUR = 3600_000;
const now = 10 * HOUR;

// pushed at write time, newest first, capped at 200 entries
const inbox = [
  { postId: "p9", authorId: "friend_a", ts: now - 0.5 * HOUR },
  { postId: "p7", authorId: "friend_b", ts: now - 3 * HOUR },
  { postId: "p4", authorId: "friend_a", ts: now - 9 * HOUR },
];

// NOT pushed: one cached list per big account, read at request time
const celebrityRecent = new Map([
  ["star_x", [{ postId: "c3", authorId: "star_x", ts: now - 1 * HOUR },
    { postId: "c1", authorId: "star_x", ts: now - 7 * HOUR }]],
  ["star_y", [{ postId: "c9", authorId: "star_y", ts: now - 2 * HOUR }]],
]);

const deleted = new Set(["p7"]); // tombstones, checked on read
```

```

const affinity = new Map([["friend_a", 3.0], ["friend_b", 2.0],
                          ["star_x", 1.0], ["star_y", 0.4]]);

function score(post, now) {
  const ageHours = (now - post.ts) / HOUR;
  const recency = 1 / (1 + ageHours);           // 1.0 fresh, 0.5 at 1 h, 0.1 at
9 h
  return +(recency * (affinity.get(post.authorId) ?? 1)).toFixed(4);
}

function buildFeed(inbox, follows, now, pageSize) {
  const pulled = follows.flatMap(id => celebrityRecent.get(id) ?? []);
  return [...inbox, ...pulled]
    .filter(p => !deleted.has(p.postId))          // tombstone filter, not 30M
deletes
    .map(p => ({ ...p, s: score(p, now) }))
    .sort((a, b) => b.s - a.s)                   // numeric comparator, always
    .slice(0, pageSize);
}

for (const p of buildFeed(inbox, ["star_x", "star_y"], now, 5)) {
  console.log(p.postId.padEnd(3), p.authorId.padEnd(9), "score", p.s);
}

```

actual output

JS

```

p9 friend_a score 2
c3 star_x   score 0.5
p4 friend_a score 0.3
c9 star_y   score 0.1333
c1 star_x   score 0.125

```

c3 is only one hour old and outranks p4, which is nine hours old from a closer friend. p7 does not appear at all: it was deleted, and the filter caught it without touching a single inbox row.

Deep dive B: spending the 200 ms read budget.

Step	Budget	Why it costs that
TLS, auth, routing	5 ms	fixed cost of being on the internet
Read inbox sorted set	2 ms	one range read of 200 entries = 10 KB from memory
Multi-get big-account lists	4 ms	up to 50 keys in one pipelined round trip
Filter tombstones and unfollows	1 ms	a set lookup per candidate, in process
Multi-get 60 post bodies	6 ms	60 KB from the post cache, one round trip
Score and sort 60 items	2 ms	60 multiplications and one sort
Serialise 20 items	3 ms	about 20 KB of JSON
Server total	23 ms	leaves 177 ms for the network and the phone

23 ms against a 200 ms promise looks comfortable, and it is — **while the caches are warm**. The number that actually needs defending is the cold one. A post-cache miss on all 60 candidates costs 60 key-value reads; issued in parallel that is one round trip of about 8 ms, not 60 of them. **This is the single most important implementation detail on the read path: never loop with await inside it.** Sixty sequential 5 ms reads is 300 ms and a broken promise.

Deep dive C: deletes, unfollows, and why you never delete an inbox row.

A user with 2 million followers deletes a post. If inbox rows had been pushed, removing them would be 2 million deletes — the same burst we just spent Deep dive A avoiding, now happening at the worst possible moment, because the user is watching and expects it to be instant.

Decision: never delete inbox rows. Filter at read time.

1. Deleting a post sets `deleted_at`. The hydrate step drops anything with it set.
2. Unfollowing writes the edge removal; the read path checks the candidate's author against `follows_out` and drops what is no longer followed.
3. A blocked user is the same check with a different set.
4. The stale rows cost 50 bytes each and vanish by themselves within 14 days or 200 posts, whichever comes first.

What this costs. Every read does a few set lookups it would not otherwise need — about 1 ms of the 23 ms budget — and the inbox is slightly larger than its useful content. **What it buys.** Deletion is one write, always, regardless of follower count, and it takes effect for every reader on their very next request. That is a much better guarantee than a 2-million-row cleanup job could ever give.

Step 7 — Trade-offs, failures, 10x

Choice	Option A	Option B	Decision
Fan-out	push for every one	hybrid at 100k followers	B — 500 accounts would otherwise be 65% of all writes
Inbox storage	rows in a database	capped sorted sets in memory	B — 55,555 writes/s and 55,555 reads/s of 50-byte rows is a memory workload
Inbox contents	full post bodies	post ids only	B — 50 B per entry instead of 1 KB; 2 TB instead of 40 TB
Pagination	offset and limit	a (<code>score</code> , <code>postId</code>) cursor	B — offsets duplicate and skip items whenever the feed changes
Deletes	remove every inbox row	tombstone, filter on read	B — one write instead of two million, and it is instant
Who gets pushed to	all followers	followers active in 7 days	B — 67% of all rows are never read; 7 days keeps returning users happy

Ranking	chronological	score with recency and affinity	B — 8 new posts a day is not a scrollable chronological feed
---------	---------------	---------------------------------	---

What fails	What the user sees	What we do
The fan-out queue backs up	New posts appear late	Freshness degrades, nothing breaks — the read path does not depend on the queue. Shed the marketing-like work first: stop pushing to 7-day-actives and push only to 24-hour-actives until the lag clears.
One inbox node dies	1/20 of users get an empty feed	Rebuild that node's inboxes from <code>follows_out</code> plus recent posts — it is a cache, not a source of truth. Meanwhile serve those users a pure pull-based feed, slower but correct.
The post cache is cold	Feeds load in 120 ms instead of 23 ms	Every hydrate becomes a parallel multi-get against the post store. It is 8 ms, not 300 ms, because the reads are issued together . Coalesce duplicate misses.
A big account crosses 100,000 followers	Nothing	The threshold is evaluated per post, not cached per account, so the switch happens on the very next post. Their existing pushed rows stay and simply age out.
A post goes viral	Nothing	It is one entry in the post cache, read by millions. The best case, not the worst — just like the viral link in Design 1.
A user follows 5,000 big accounts	Slow feed	Cap the pull side at the 100 most-recently-interacted-with big accounts. Beyond that the marginal post never reaches the top 20 anyway.

At 10x — and note that 10x here is not 10x the humans:

- Assume the same 200 million users open the feed 80 times a day instead of 8 (short video changes behaviour, not headcount) and posting rises to 80 million a day.
- Reads become $200,000,000 \times 80 / 86,400 = 185,185/s$ average, 555,555/s at peak.
- Fan-out becomes $80,000,000 \times 200 / 86,400 = 185,185$ rows/second, 555,555 at peak. Twenty inbox nodes become roughly 60, sized for **write throughput** now rather than memory.
- The inbox does not get bigger, it gets **faster-moving**: $80,000,000 \times 200 / 200,000,000 = 80$ rows per user per day, so a 200-entry cap now holds $200 / 80 = 2.5$ days instead of 25. That quietly changes the product — a user who opens the app twice a week no longer sees what they missed. **Decision: raise the cap to 600 entries (6 TB of inboxes) for weekly-active users and keep 200 for daily-actives.**
- The genuinely new problem at 10x is **ranking cost**, not fan-out. Scoring 60 candidates with a formula is 2 ms; scoring 600 candidates with a learned model is not. **Decision: keep the cheap formula as a first-stage filter that cuts 600 candidates to 60, and run the expensive model only on those 60.** Two-stage ranking is how you buy a better feed without buying 10x the machines.

Design 5 — a chat system

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 16 [Microsoft · pattern]

Design one-to-one and group messaging. 100 million daily active users sending 40 messages each, groups of up to 500, delivered and read receipts, and up to four devices per account.

SOLUTION

Step 1 — Clarify

Question I ask	Why the answer changes the design
One-to-one only, or groups? How big is the largest group?	It multiplies every delivery. 1:1 is 1 copy; a 500-member group is 499.
Does the server keep history, or does it live on the device?	This is the biggest storage decision in the design, by a factor of 100.
How many devices per account?	More than one means the read cursor is per device , not per user. Easy to get wrong.
Delivered and read receipts?	Each one is another message class flowing the other way.
End-to-end encrypted?	If yes, the server cannot search, preview, or rank — and cannot help you recover a lost device's history.
Must message order be identical for everyone in a chat?	Yes → one writer per chat assigns a sequence. Client time-stamps can never do this.

Assumed and said out loud: **100 million DAU sending 40 messages a day; 30% of messages go to groups averaging 25 members**, groups capped at 500; **the server keeps one year of history; up to 4 devices per account; delivered and read receipts required; transport-encrypted, not end-to-end** — and I say why that is a product choice, not a technical one, because end-to-end would remove server-side search and multi-device history recovery.

Step 2 — Scale estimate

Messages sent.

$$100,000,000 \times 40 = 4,000,000,000 \text{ messages/day}$$

$$4,000,000,000/86,400 = 46,296 \text{ sends/second} \quad \times 3 = 138,888 \text{ at peak}$$

Deliveries — the number that is 8x bigger than the one you were given.

$$0.70 \times 1 + 0.30 \times 24 = 0.7 + 7.2 = 7.9 \text{ deliveries per message}$$

$$4,000,000,000 \times 7.9 = 31,600,000,000 \text{ deliveries/day}$$

$$31,600,000,000/86,400 = 365,741 \text{ /second} \quad \times 3 = 1,097,222 \text{ at peak}$$

Connections — Little's Law, with users online 90 minutes a day:

$$100,000,000 \times 90/1,440 = 6,250,000 \text{ concurrent on average}$$

$$6,250,000 \times 2 = 12,500,000 \text{ at peak}$$

$$12,500,000/100,000 \text{ per box} = 125 \text{ boxes of load}$$

To survive losing one availability zone of three: $125/(2/3) = 188$ boxes, spread 63/63/62.

Per box at peak, which is the check that tells you the fleet is the right size:

$$1,097,222/188 = 5,836 \text{ pushes/second per box}$$

$$5,836 \times 400 \text{ B} = 2,334,400 \text{ B/s} = 2.33 \text{ MB/s} = 18.7 \text{ Mbps}$$

Both comfortable. A box holding 100,000 sockets and writing 5,836 small frames a second is doing easy work.

Storage — and the decision that saves 8x. One message row (ids, sender, sequence, timestamp, 200 bytes of body) is about 300 bytes.

If you store one copy **per recipient**:

$$31,600,000,000 \times 300 = 9,480,000,000,000 \text{ B} = 9.48 \text{ TB/day}$$

$$9.48 \times 365 = 3,460 \text{ TB} = 3.46 \text{ PB/year} \times 3 \text{ RF} = 10.4 \text{ PB}$$

If you store one copy **per chat** and give each device a cursor:

$$4,000,000,000 \times 300 = 1,200,000,000,000 \text{ B} = 1.2 \text{ TB/day}$$

$$1.2 \times 365 = 438 \text{ TB/year} \times 3 \text{ RF} = 1.31 \text{ PB}$$

$3.46/0.438 = 7.9 \times$ less — exactly the fan-out factor, which is the point.

The routing table. Which box holds which user's socket:

$$12,500,000 \times 50 \text{ B} = 625,000,000 \text{ B} = 625 \text{ MB}$$

One Redis node, with a TTL so a dead connection cleans itself up.

THE BOARD SUMMARY

138,888 sends/s peak · **1,097,222 deliveries/s peak** · 12.5 million open sockets on 188 boxes · 438 TB/year · 625 MB of routing state.

Chat pulls where the feed pushes. Design 4 wrote one row per reader because most readers never come back. Chat writes one row per **chat** because every recipient is going to read it, and there are only a handful of them. Say that comparison out loud — it is the same question answered differently for a good reason.

Step 3 — API surface

a WebSocket for the stream, plain HTTP for history and setup

TEXT

```
WS    /v1/stream                                (one per device, always open)
  up   { t:"send", chatId, clientMsgId:"01J8X...", text, sentAtMs }
  down { t:"ack",  chatId, clientMsgId, seq: 40912, serverTsMs }
  down { t:"msg",  chatId, seq, senderId, text, serverTsMs }
  down { t:"receipt", chatId, seq, userId, kind:"delivered"|"read" }
  up   { t:"cursor", chatId, seq: 40912 }          -- "this device has seen up to
here"
  up   { t:"typing", chatId }                      -- fire and forget, never stored
  up   { t:"hb" } every 30 s                       -- heartbeat

GET   /v1/chats/{chatId}/messages?after=40880&limit=200
200   { messages:[...], nextAfter: 41080 }
      note the ONLY history call. A reconnecting device sends its cursor and
      gets exactly what it missed, in order, with no duplicates.

POST  /v1/chats                                { type:"group", memberIds:[...], title } 201
POST  /v1/chats/{id}/members                    201
GET   /v1/chats?updatedAt=...                    200 -- the
chat list
```

SHORTCUT

clientMsgId is generated by the sender, not the server. It is what makes a retry safe: the phone that did not get an ack re-sends the identical frame, the server recognises the id and returns the **original** sequence number instead of creating a second message. Without it, every flaky train tunnel produces duplicate messages, and users notice duplicates far more than they notice delay.

Step 4 — Data model

one message row per chat, one cursor per device

TEXT

```
messages
  chat_id    bigint    PARTITION KEY    -- all of a chat lives together
  seq        bigint    SORT KEY        -- assigned by the chat's owner, gapless
  message_id ulid
  sender_id  bigint
  client_msg_id text          -- the sender's idempotency key
  body       blob          -- ~200 B
  created_at timestamp
  UNIQUE (chat_id, client_msg_id)    -- the retry guarantee
  -- partitioned by month; older months move to cold storage

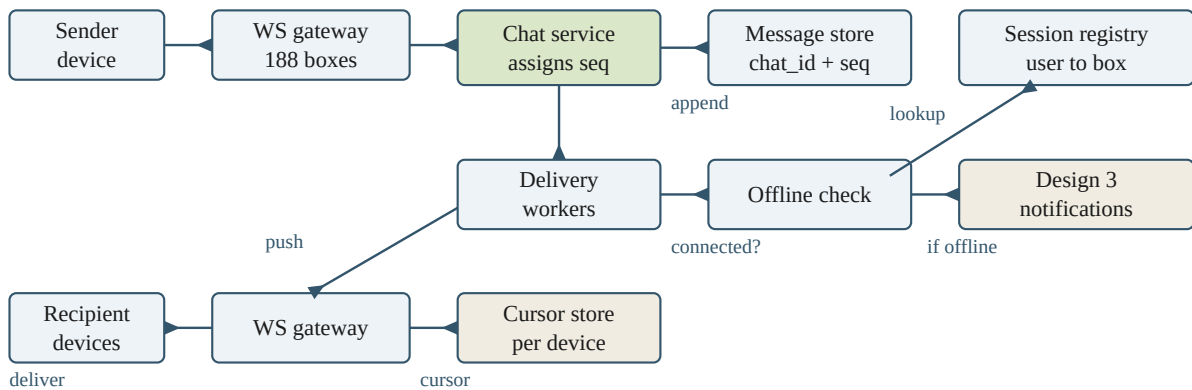
chat_members
  chat_id, user_id, role, joined_at_seq -- joined_at_seq caps what a new
                                         member can read. Joining a group
                                         does not hand you its history.

device_cursor
  user_id, device_id PRIMARY KEY
  chat_id, last_seq, updated_at          -- one row per device per chat
  note: per DEVICE, not per user. A phone and a laptop are at different places.

-- IN MEMORY
route:{userId}    set    deviceId -> gatewayBoxId    TTL 90 s
seq:{chatId}      int    the chat's current sequence (owned by one writer)
presence:{userId} string ONLINE | LAST_SEEN ts      TTL 90 s
```

Why partition by chat_id? Because every read is “give me this chat after this sequence”, and every write is “append to this chat”. Both are single-partition. The cost is that one enormous, very busy group is one hot partition — but a 500-member group at 20 messages a minute is $20/60 = 0.33$ writes/second, which is nothing. Group **size** is a delivery problem, not a storage problem.

Step 5 — Architecture



A reconnecting device sends only its cursor. The gateway replays what it missed from the message store — no per-user inbox exists. If no device for a recipient is connected, the delivery worker hands the message to the notification system of Design 3.

The sequence is assigned in exactly one place. Everything downstream — storage, delivery, receipts, offline push — reads it and never invents it.

Trace one group message:

1. The sender's device writes a `send` frame with a `clientMsgId` it generated itself.
2. Its gateway box forwards it to the chat service instance that **owns** that `chat_id`. Owning it is what makes the next step safe.
3. The chat service increments `seq: {chatId}` and appends one row. If the `clientMsgId` already exists, it skips the append and returns the original sequence. Either way the sender gets an `ack` with a number.
4. The delivery worker reads the 25 members, looks each up in the session registry, and for every connected device writes the frame to the box holding that socket.
5. Members with no connected device go to the notification system from Design 3, as a push with `class: "transactional"` and a short `expiresAt` — a chat notification that arrives an hour late is noise.
6. Each receiving device sends back a delivered receipt, and later a read receipt when the chat is on screen. Both are ordinary messages flowing the other way, and both are batched.

Step 6 — Deep dive

Deep dive A: order comes from one writer, never from a clock.

chat-order.js — run with: `node chat-order.js`

JS

```

class ChatPartition {
  // ONE writer owns a chat_id, so seq is safe
  constructor() { this.seq = new Map(); this.log = []; this.seen = new Map(); }

  append(chatId, senderId, clientMsgId, text, clientTs) {
    const key = chatId + "|" + clientMsgId;
    if (this.seen.has(key)) return this.seen.get(key); // the retry gets the SAME
    row
    const n = (this.seq.get(chatId) ?? 0) + 1;
    this.seq.set(chatId, n);
    const row = { chatId, seq: n, senderId, clientMsgId, text, clientTs };
    this.log.push(row);
    this.seen.set(key, row);
    return row;
  }

  since(chatId, cursor) {
    return this.log.filter(r => r.chatId === chatId && r.seq > cursor);
  }
}

const p = new ChatPartition();

```

```
// Asha's phone clock is 90 seconds FAST. Bo's is correct.
p.append("c1", "asha", "m1", "are you coming?", 1_000_090_000);
p.append("c1", "bo", "m2", "coming now", 1_000_005_000);
p.append("c1", "asha", "m3", "great", 1_000_120_000);
p.append("c1", "bo", "m2", "coming now", 1_000_005_000); // network retry

console.log("by client clock (WRONG):");
for (const r of [...p.log].sort((a, b) => a.clientTs - b.clientTs))
  console.log(" ", r.senderId.padEnd(5), r.text);

console.log("by server sequence (RIGHT):");
for (const r of [...p.log].sort((a, b) => a.seq - b.seq))
  console.log(" ", String(r.seq).padEnd(2), r.senderId.padEnd(5), r.text);

console.log("rows stored:", p.log.length, "- the retry did not create a fourth");
console.log("Bo reconnects with cursor 1, gets:",
  p.since("c1", 1).map(r => r.seq).join(", "));
```

actual output

JS

```
by client clock (WRONG):
  bo    coming now
  asha  are you coming?
  asha  great
by server sequence (RIGHT):
  1 asha  are you coming?
  2 bo    coming now
  3 asha  great
rows stored: 3 - the retry did not create a fourth
Bo reconnects with cursor 1, gets: 2, 3
```

Three things are proved by that output:

1. A 90-second clock skew — utterly normal on real phones — makes the answer appear **before** the question. Client timestamps are for display, never for ordering.
2. The duplicate send produced no fourth row, because `(chat_id, client_msg_id)` is unique.
3. A reconnecting device asks for “after 1” and gets exactly 2 and 3. No inbox, no per-user copy, no deduplication needed on the client.

TRAP

One writer per chat is a real constraint, not a diagram detail. If two chat-service instances can both append to chat `c1`, they will both compute sequence 41, and two different messages will share a number. Route by `chat_id` — consistent hashing, or a partition assignment held in the coordination service — so that at any moment exactly one instance owns a chat. When ownership moves, the new owner reads `MAX(seq)` from the store before accepting anything, and the old owner must have stopped first. This is the same “single writer” rule that makes Kafka partitions and database shards work.

Deep dive B: at-least-once delivery that feels exactly-once.

You cannot have exactly-once delivery over a network. What you can have is at-least-once delivery plus an identifier that makes duplicates harmless. Three places need it, and they are three different identifiers:

Duplicate source	What stops it	Where it lives
The sender retries after a lost ack	<code>client_msg_id</code> , unique per chat	a unique index on <code>(chat_id, client_msg_id)</code>

A delivery worker retries after a lost push	seq — the device ignores anything at or below its cursor	the device, in one integer comparison
Two delivery workers pick up the same message	the push is idempotent by (device_id, chat_id, seq)	nothing is stored; the device discards the second copy

The three ticks, and what each really means. sent is “the server assigned a sequence”. delivered is “at least one of the recipient’s devices acknowledged the frame”. read is “the chat was on screen”. They are three different facts and users understand the difference, so do not collapse them. Receipts are **batched**: a device that receives 40 messages sends one receipt naming the highest sequence, not 40 receipts. Without batching, receipts would triple the delivery load for no benefit — $1,097,222 \times 2 = 2,194,444$ extra frames/second at peak.

Deep dive C: presence and typing, the two features that can melt the fleet.

Naive presence: every device heartbeats every 30 seconds and the server fans the result out.

$$12,500,000/30 = 416,667 \text{ heartbeats/second}$$

and if each one were broadcast to that user’s contacts, multiply by a hundred. That is larger than the entire message workload, for a green dot.

Decision: the gateway already knows who is connected — derive presence from the connection, publish only changes, and fan out only to people who currently have that chat open.

1. The socket opening is the ONLINE event. The socket closing is the OFFLINE event. No separate heartbeat is needed for presence at all; the 30-second heartbeat exists only to detect dead sockets on the server side, and it never leaves the gateway box.
2. A state **change** is published, not a state. A user who stays online for an hour generates two events, not 120.
3. Fan-out is limited to sessions with that chat in the foreground. A user with 400 contacts generates presence traffic to the two people currently looking at their chat.
4. Typing indicators are fire-and-forget: never stored, never retried, never queued, and rate-limited to one per 5 seconds per chat with the token bucket from Piece 3. A dropped typing indicator costs nothing; a queued one is worse than useless, because it arrives after the message it was predicting.

Step 7 — Trade-offs, failures, 10x

Choice	Decision
Storage shape	One row per chat tip-indicator
Ordering	Clientphone clocks are wrong, often by minutes time-stamps sequence from one writer
Transport	With 2.5 million sockets is 188 boxes; long-polling the same users is far worse token bucket poll

Cursor granularity	Per — a phone and a laptop are genuinely at different places in the chat device
Delivery guarantee	Exactly — exactly-once over a network does not exist; make duplicates harmless instead + ids
Presence	Heart beat 416,667 events/s versus a few thousand broadcast from thread- cost- nec- tion, pub- lish changes
Receipts	Receives 2.19 million frames/second at peak per thes- high- est se- quence
Encryption	Radio — this product, and I say the cost out loud: no server-side search, no history on a new device. For the product is privacy-first, choose A and lose those features. only

What fails	What the user sees	What we do
One gateway box dies	100,000 users reconnect	Clients back off a random 0 to 30 s, so reconnects arrive at $100,000/30 = 3,333/\text{second}$ instead of all at once. Each one replays from its cursor, so nothing is lost.
A whole zone dies	4.17 million reconnect	The same mechanism, but 30 seconds is not enough: $4,166,666/30 = 138,889/\text{s}$ would be a second outage. Widen the backoff to 300 s $\rightarrow$ 13,889/s. The backoff window must be sized for the largest failure, not the likeliest one.
The chat-service owner of a chat crashes	That chat stalls for a few seconds	A new owner is assigned, reads MAX(seq) from the store, and resumes. Senders retry with the same clientMsgId , so nothing duplicates.
A device is offline for a week	A long catch-up	It sends one cursor and pages history 200 at a time. Cap the replay at the last 1,000 messages per chat and let the rest load on scroll.

A 500-member group gets a burst	Nothing	500 pushes for one message is 500 socket writes across up to 188 boxes — a rounding error against 1.1 million/second. Group size was never the scaling problem.
A user is added to a group with 5 years of history	They see nothing before they joined	<code>joined_at_seq</code> caps the read. This is a privacy decision as much as a storage one, and it must be explicit.
The push provider is slow	Offline users are notified late	It is Design 3's problem, with Design 3's answer: its own queue, its own retry budget, and an <code>expiresAt</code> so a stale chat notification is dropped instead of delivered.

At 10x (40 billion messages/day, the same 100 million users messaging far more):

- Sends become $40,000,000,000/86,400 = 462,963/s$ average and $1,388,889/s$ at peak.
- Deliveries become $40,000,000,000 \times 7.9/86,400 = 3,657,407/s$ average and $10,972,222/s$ at peak.
- Users are online 3 hours a day rather than 90 minutes: $100,000,000 \times 180/1,440 = 12,500,000$ average, 25 million at peak, which is $25,000,000/100,000 = 250$ boxes of load and 375 provisioned.
- Per box that is $10,972,222/375 = 29,259$ pushes/second and $29,259 \times 400 = 11.7$ MB/s = 94 Mbps. The bandwidth is fine; 29,259 individual socket writes a second is not comfortable. **Decision: coalesce per socket — buffer up to 50 ms and send one frame containing several messages.** A 50 ms delay is invisible in a chat and it cuts the number of writes by roughly the number of messages per burst.
- Storage becomes 12 TB/day and 4.38 PB/year. **Decision: 30 days on fast storage, the rest in object storage behind the same history API, and a hard one-year server-side retention with the device keeping anything older.**
- The genuinely new problem at 10x is **geography**. A group with members in three continents cannot have its single writer near everybody. **Decision: give each chat a home region chosen by where the majority of its members are, forward writes there, and serve reads from a local async replica.** A member on the far side pays 150 ms extra to **send** and nothing to read. The alternative — a global consensus round per message — would put 150 ms on every message for everyone to make a rare case slightly fairer.

Interview drill — what the interviewer pushes on

They ask	You answer
“Why not just hash the URL?”	“Because hashes collide and detecting a collision is a read on the write path. With 3.5 trillion slots and 10 million inserts a day, the birthday bound puts the first collision inside the first day. A counter handed out in blocks of 1,000 has zero collisions by construction and one coordination call per 1,000 links.”
“Where does the rate-limit count live?”	“In a shared store, never per pod. Per-pod with the full limit lets 20 pods allow 20x the limit; per-pod with limit-over-N gives a pinned client 5% of what they paid for. Both mistakes are invisible with one pod in a test.”
“Show me the race in a rate limiter.”	“Read, decide, write is three steps with a gap. Fifty concurrent requests against a limit of 10 all read zero and all write one — 50 allowed, and the counter says 1. The fix is one atomic script in the store, which is the same token-bucket arithmetic moved to where the data is.”

“Your API does 70,000 checks a second. Is that 70,000 Redis calls?”	“No. Each gateway pod leases a block of tokens, the same trick as the id blocks in the shortener. Under the limit that is 20x fewer calls. Over the limit the saving drops to about 5x, which is why a pod that gets an empty lease goes quiet for 200 ms.”
“What happens when the rate limiter’s store is down?”	“Fail open, with each pod falling back to a local bucket at limit-over-N so the worst case is still roughly the true limit — except for login and OTP, which fail closed in code. A limiter outage must never become an API outage.”
“A campaign sends 20 million messages. What happens to an OTP?”	“With one queue it waits 20,000,000 divided by 5,787, which is 58 minutes against a 10-second promise. With one queue per class and reserved workers it waits 0.43 seconds, and the campaign takes 1.92 hours instead — I made the unimportant thing slower on purpose.”
“Your SMS provider is contracted at 1,000 a second. Now what?”	“A token bucket per provider enforces it in my code instead of discovering it through their 429s, OTP holds a reserved 500 a second of it, and campaigns are spread over a window. And marketing does not use SMS at all unless the user has no push token, because the same message costs 12 paise one way and nothing the other.”
“How do you know a notification arrived?”	“I do not, until the receipt webhook says so. The provider’s 200 means accepted, not delivered. I alarm when the delivered-to-sent ratio for a provider falls more than 20% below its 7-day baseline.”
“Fan out on write or on read?”	“Both, split at 100,000 followers. Five hundred accounts would otherwise be 65% of all inbox writes, and a single two-million-follower post would occupy the fan-out queue for ten seconds while everyone else waits. Below the threshold push; above it write once and let readers pull.”
“Two thirds of your fan-out is never read. Fix it.”	“Push only to followers seen in the last 7 days. Filtering to today’s actives would be cheaper still, but then a user returning tomorrow opens an empty app, and that is the one user you most want to keep.”
“Someone with two million followers deletes a post.”	“One write. <code>deleted_at</code> is set and the read path filters it during hydrate. Deleting two million inbox rows would be exactly the burst I designed the threshold to avoid, happening while the user watches and expects it to be instant.”
“Why does chat store one copy and the feed stores many?”	“Because of who reads. A chat message has a handful of recipients and every one of them will read it, so a shared row plus a per-device cursor is 7.9 times less storage. A post has thousands of followers and two thirds never come back, so pre-building the answer is worth it only for the ones who will.”
“Who decides message order?”	“One writer per chat, assigning a gapless sequence. Phone clocks are wrong by minutes, so ordering by client timestamp puts the answer before the question. When chat ownership moves, the new owner reads the max sequence first and the old owner must already have stopped.”
“Can you promise exactly-once delivery?”	“No — nobody can, over a network. I deliver at-least-once and make duplicates harmless: a unique (<code>chat_id</code> , <code>client_msg_id</code>) stops the sender’s retry, and the device ignores any sequence at or below its cursor.”

“A whole availability zone dies. Describe the next 60 seconds.”

“4.17 million sockets drop and every client wants to reconnect. Spread over 30 seconds that is 138,889 reconnects a second, which is a second outage, so the backoff window is 300 seconds instead. Each device replays from its own cursor, so nothing is lost — only delayed.”

TRAP

The most common Tier-3 failure in this chapter is **choosing one fan-out strategy and defending it**. Push and pull are both correct, for different users, in the same system, at the same moment — and a candidate who says “I would use fan-out on write” without naming the follower count at which they stop has not answered the question. The threshold, and the arithmetic that picks it, **is** the answer.

PRACTICE · TIER 3 target: 50 min

1. Design the rate limiter’s **cost** dimension: a search request costs 5 tokens and a cheap read costs 1. Say what changes in the lease, in **Retry-After**, and in what a client sees.
2. A notification campaign must respect quiet hours in 40 countries, a per-user frequency cap of 3 marketing messages a week, and a legal opt-out. Say where each check happens and why it cannot happen anywhere else.
3. Compute the cost of raising the news-feed inbox cap from 200 to 1,000 entries: memory, write amplification, and what the user gains. Then decide.
4. Add message search to the chat design without end-to-end encryption, then say exactly what breaks if the product later adds it.
5. A bug pushed 400 million inbox rows pointing at a post that was deleted an hour ago. Write the runbook for the first hour, the first day and the first week.

ANSWER KEY

1. The lease must be requested in **tokens**, not requests, so a pod leases 20 tokens and can serve twenty cheap reads or four searches with it. **Retry-After** uses the Piece 3 formula with the **cost** of the request that was refused, so a search waiting for 5 tokens at 10 tokens/second waits 500 ms while a cheap read waits 100 ms. What the client sees is the honest part: publish the cost of each endpoint in the docs, or callers cannot predict their own limits and will assume you are broken.
2. Quiet hours: at **send** time, converted from the user’s current time zone, because both the zone and the setting can change during a 2-hour spread. Frequency cap: at **send** time too, against a counter keyed by user and week — checking it at campaign-build time would let two overlapping campaigns each pass. Legal opt-out: at send time **and** enforced by the database, because it is the one check where being slightly stale is a legal problem rather than a product one.
3. Memory goes from $200,000,000 \times 200 \times 50 = 2 \text{ TB}$ to $200,000,000 \times 1,000 \times 50 = 10 \text{ TB}$ — 20 nodes become 100. Write amplification does **not** change: you still write the same number of rows, you just trim less often. The user gains 5x the scrollbar, which at 8 new posts a day is 125 days instead of 25 — far more than anyone scrolls. **Decision: no**. Raise it to 600 only for weekly-active users, where the gap between visits genuinely exceeds the cap.
4. Search needs an inverted index per user over the chats they are in, built by a consumer of the same message stream, partitioned by **user_id** rather than **chat_id** — the only place in the design that is not chat-partitioned, and you should say so. If end-to-end encryption is added later, the server sees only ciphertext and this index becomes impossible: search must move onto the device, over the history that device holds, which means it can no longer find anything from before that device was added. That is the honest trade, and it is why the encryption decision has to be made in Step 1 and not bolted on.
5. **First hour:** stop the fan-out workers for that post id, confirm the post is really tombstoned, and verify that readers are **not** seeing it — the tombstone filter should already be hiding it, which turns an incident into a cleanup. **First day:** leave the rows alone. 400 million rows at 50 bytes is 20 GB across 20 nodes, 1 GB each, and they age out within 14 days by themselves. Deleting them would be 400 million writes to fix a cosmetic problem. **First week:** add the assertion that would have caught it — a fan-out worker must refuse any post whose **deleted_at** is set — and a metric on rows written per post id, alarming above 100,000, which is exactly the threshold that should have sent it down the pull path anyway.

ONE-PAGE REVISION CARD

The seven steps, with minutes. Clarify 5 · estimate 5 · API 5 · data model 5 · diagram 8 · deep dive 10 · trade-offs 7. Say the step name out loud before you do it.

The estimation card.

1 day	86,400 s ($\approx 10^5$)	1 M/day	$\approx 11.6/s$
1 B/day	$\approx 11,574/s$	peak factor	2–3x global, 5–10x one country
1 KB × 1 M	1 GB	1 KB × 1 B	1 TB
1 Mbps	0.125 MB/s	MySQL box	~5,000 writes/s, ~20,000 reads/s
Redis box	~100,000 ops/s	WebSockets/box	~100,000 held

The shortcut: write the daily count in millions and multiply by 11.6. It is within 0.3% and it takes two seconds.

The five pieces this chapter built, and what each is really for.

base62 encoder	turns a number into a short, URL-safe string. $62^7 \approx 3.5$ trillion codes — seven characters is plenty, and you should say the power out loud.
key blocks	ten servers generate unique ids with no lock by claiming a block of 1,000 at a time. The only coordination is one increment per block, not one per id. The same trick reappears as token leases in Design 2.
token bucket	allows a burst up to the capacity, then settles to the refill rate. Tiny state: a count and a timestamp. Used again to shape SMS in Design 3 and to rate-limit typing indicators in Design 5.
sliding window counter	fixes the fixed-window flaw where a user sends 2x the limit across a boundary. More state, more accuracy — buy it only when you must.
notification dispatcher	one message, several channels, each of them unreliable. Dedupe key, retry budget, dead-letter queue. It is the core of Design 3.

The five designs, and the one number each turns on.

Design	The number	What it forces
URL shortener	100:1 read:write, $62^7 = 3.5$ trillion	cache first, database second; a counter, never a hash
Rate limiter	69,444 checks/s on shared state	one atomic script in the store, plus leases to touch it 20x less
Notifications	20 M blast = 33,333/s versus a 10-second OTP	one queue per class with reserved workers; shaping beats capacity
News feed	500 accounts = 65% of all fan-out writes	hybrid push/pull with the threshold at 100,000 followers
Chat	7.9 deliveries per message, 12.5 M open sockets	store once per chat with a per-device cursor; a sequence from one writer

The URL shortener in six lines.

- Hashing is not enough.** Hashes collide, and checking for a collision is a read on the write path. Generate ids from a coordinated **counter**, handed out in blocks.
- Redirects outnumber creates by roughly 100 to 1. Design the read path first; it is a single cache lookup and a 301/302.
- The data model is one KV row: code -> long_url, plus created-at and owner. There is no join anywhere in this system.

4. Cache with consistent hashing so one dead node costs $1/N$ of keys, not all of them. Coalesce concurrent misses so the store is not flooded.
5. A viral link is the **best** case, not the worst — it is one hot key served from memory.
6. At 10x the new problem is **geography**, not load. Replicate asynchronously across regions and fall back to the home region on a miss. Synchronous global replication would add 150 ms to every write to save a rare 150 ms on a read. Wrong trade.

The rate limiter in five lines.

1. **Counting per server is always wrong.** The full limit on each of 20 pods allows 20x; the limit divided by 20 gives a pinned client 5%. Both hide in a one-pod test.
2. **A check is not a read then a write.** Fifty concurrent requests against a limit of 10 all read 0 and all write 1 — 50 allowed, counter says 1. Run the whole check inside the store.
3. **Lease blocks of tokens**, exactly as the shortener leases blocks of ids. 20x fewer store calls under the limit, about 5x over it, and a 200 ms quiet period when a lease comes back empty.
4. **Fail open with a local cap** at limit-over-N — except login and OTP, which fail closed in code, because there the risk is abuse and not cost.
5. Retry-After is **computed and jittered**. A fixed value builds a synchronised herd out of your own error responses.

The notification system in five lines.

1. **One queue per class, with reserved workers.** One shared queue puts an OTP 58 minutes behind a 20 million blast: $20,000,000/5,787 = 3,456$ s. A priority field cannot reorder a log after the fact.
2. **The peak is self-inflicted, so shape it.** Spreading a blast over 120 minutes turns 33,333/s into 2,778/s and costs nothing.
3. **Money is a design input.** 25 million SMS a day at 12 paise is 109.5 crore a year. Route to the cheapest channel the user can actually receive.
4. **Retries back off, jitter, and stop.** 1, 4, 16, 64 seconds, then a dead-letter queue. Never retry a permanent failure — mark the dead device token and stop.
5. **“Sent” is a claim, “delivered” is evidence.** Receipts arrive by webhook; alarm when the delivered-to-sent ratio drops 20% below its 7-day baseline.

The news feed in five lines.

1. **Fan-out multiplies a tiny write rate into a huge one.** 92.6 posts/second becomes 18,518 inbox rows/second, which is the same size as the whole read path.
2. **No single strategy works.** Push below 100,000 followers, pull above it. 500 accounts would otherwise be 65% of all writes, and one post would block the queue for 10 seconds.
3. **Push only to followers seen in the last 7 days.** Two thirds of all rows are never read.
4. **Never delete an inbox row.** Tombstone the post and filter on read. One write instead of two million, effective on the reader’s very next request.
5. **Hydrate in parallel.** Sixty post-cache misses issued together are 8 ms; sixty issued in an await loop are 300 ms and a broken promise.

The chat system in five lines.

1. **Store once per chat, not once per recipient.** 438 TB/year instead of 3.46 PB, and there is no inbox to rebuild after a failure.
2. **A cursor per device, not per user.** A phone and a laptop are genuinely in different places in the conversation.
3. **Order comes from one writer per chat**, as a gapless sequence. Phone clocks are wrong by minutes and will put the answer before the question.
4. **Exactly-once does not exist. Make duplicates harmless.** Unique (chat_id, client_msg_id) for the sender’s retry; the device ignores any sequence at or below its cursor.
5. **Derive presence from the connection and publish only changes.** Heartbeat-and-broadcast is $12,500,000/30 = 416,667$ events a second for a green dot.

Push or pull? The question this chapter answers twice, differently.

	News feed	Chat
Recipients per item	200, sometimes 2,000,000	1, sometimes 24
Fraction who will read it	about a third	essentially all
Answer	push — but only below 100,000 followers	pull, from one shared copy
Why	pre-building the answer pays when many read it	copying pays nothing when few do

Failure modes and the honest answer to each.

What fails	User sees	What you say
one cache node	1 ms becomes 6 ms	consistent hashing; only $1/N$ of keys go cold
the whole cache tier	slow but working	the store is sized for 100% miss at average , and you shed traffic with 429s while it warms
the id service	creates fail, redirects fine	each pod holds unused ids — raise the block size to buy seconds of headroom, then 503 and let clients retry
the limiter store	nothing, briefly	fail open with a local cap and alarm at once; an approximate limit must never become the quiet normal state
an SMS provider	nothing	fail over to the second; capacity halves, so the OTP reservation makes marketing pause by itself
the fan-out queue	new posts appear late	freshness degrades, reads do not — the read path never touches the queue
one inbox node	1/20 of feeds empty	it is a cache; rebuild it from the follow graph and serve pull-based feeds meanwhile
a whole zone of gateways	4.17 M reconnect	randomise the backoff over 300 seconds, not 30 — size the window for the largest failure, not the likeliest

JavaScript traps in this chapter.

1. `sort()` with no comparator sorts numbers as text. Scores, counts and sequences all need $(a, b) \Rightarrow a - b$.
2. Read-modify-write across two `await`s is a race, even in single-threaded Node — the gap is the network, not the thread.
3. `await` inside a hydration loop turns 8 ms into 300 ms. Build the array of promises and `Promise.all` it.
4. `Map` and `Set` keep types and insertion order; a plain object stringifies its keys.
5. Numbers are exact only to $2^{53} - 1$. $62^9 = 1.35 \times 10^{16}$ already exceeds it, so a 9-character code needs `BigInt`.

Checklist before you say “done”.

1. Showed the division for every number you wrote on the board.
2. Named the **biggest** rate in the system and said where that data lives.
3. Named the atomic primitive that stops a double count or a double assignment.
4. Said which work is on the user’s critical path and which is behind a queue.
5. For anything shared across servers: said what happens when the shared thing is down.
6. For anything fanned out: gave the threshold at which you switch strategy, with arithmetic.

7. Gave both sides of every trade-off and then **decided**.

The sentence that separates a hire from a no-hire. Never “it depends” and stop. Say “**it depends on X; I will assume X because Y, so I choose Z; if X were false I would choose W.**” One sentence longer, and it is the whole difference.

HLD Case Studies II

Ride-hailing · food delivery · video streaming · search autocomplete · payments

The idea in one page

Chapter 11 designed five systems whose data was small and whose hard part was **coordination**. This chapter designs five whose data is awkward in a new way. One moves (a driver's position). One is enormous (video). One must answer before you finish typing. One must never, ever be wrong (money).

The seven steps do not change. What changes is **which step is hard**.

THE FIVE DESIGNS AND WHAT EACH ONE IS REALLY ABOUT

Design	The real lesson	The trap
Ride-hailing	Indexing things that move, and matching two sides of a market	Storing every GPS ping. It is 864 GB a day and nobody reads it.
Food delivery	A three-sided state machine with no distributed transaction	Treating “restaurant accepted” and “courier assigned” as one atomic step
Video streaming	Bandwidth, not requests. The CDN is the system.	Designing an origin that could serve the traffic. It cannot, and must not.
Search autocomplete	Precompute everything. The request must do no thinking.	Searching a trie at request time under a 50 ms budget
Payments	Idempotency, a double-entry ledger, and never trusting a third party	One row per payment that gets UPDATE-ed. Money is append-only.

The numbers card for this chapter

LEARN THESE BEFORE YOU WALK IN

Geography

- A geohash of 5 characters is about **4.9 km x 4.9 km**.
- 6 characters is about **1.2 km wide x 0.61 km tall**.
- 7 characters is about **153 m x 153 m**.
- 1 degree of latitude ≈ 111.3 km, everywhere. Longitude shrinks by $\cos(\text{lat})$.

Media

- Video bitrates: 240p 0.4 Mbps, 360p 0.8, 480p 1.2, 720p 2.5, 1080p 5.0 Mbps.
- 1 Mbps for 1 hour = 450 MB. 1 Gbps = 125 MB/s.
- A 4-second HLS chunk at 3 Mbps is $4 \times 3,000,000/8 = 1,500,000$ B = 1.5 MB.
- A good CDN offloads **95%** of bytes. The other 5% is your origin's whole job.

Latency budgets a product actually promises

- Autocomplete: **under 100 ms end to end**, so under 50 ms in your service.
- Driver dispatch: an offer on the driver's phone within **2 seconds** of the request.
- Payment authorisation: **under 3 seconds**, and the third party alone eats 800 ms of it.

Money

- Always integer minor units (cents, paise). **Never a float**.
- Double entry: every movement is two rows that sum to zero. Nothing is ever updated.

- 10 million payments/day = 115.7/s average; a sale day at 10x is 1,157/s.

TRAP

The most expensive mistake in this chapter is **persisting every event**. 400,000 drivers pinging every 4 seconds is 100,000 writes/second and 864 GB/day — of data that is **stale one second later**. A current position belongs in memory. Only the parts you will actually read again — a trip's trail, for a receipt or a dispute — go to disk, and even then downsampled.

Warm-up

WARM-UP

build the reflex

Example 1 [warm-up]

400,000 drivers are online at peak. Each phone sends its position every 4 seconds, in a 100-byte message. Find the write rate, the bandwidth, and the storage cost if you saved every ping for one day.

SOLUTION

Writes per second. Each driver sends $1/4 = 0.25$ messages a second.

$$400,000/4 = 100,000 \text{ pings/second}$$

Bandwidth.

$$100,000 \times 100 = 10,000,000 \text{ bytes/second} = 10 \text{ MB/s} = 80 \text{ Mbps}$$

That is small. The **count** is the problem, not the bytes.

Storage if you kept them all.

$$10,000,000 \times 86,400 = 864,000,000,000 \text{ bytes/day} = 864 \text{ GB/day}$$

$$864 \times 365/1000 = 315.4 \text{ TB/year}$$

What we actually store. Only the **latest** position per driver, in memory:

$$400,000 \times 60 \text{ B} = 24,000,000 \text{ B} = 24 \text{ MB}$$

Twenty-four megabytes replaces three hundred terabytes a year. That comparison is the whole answer to this design's first hard question.

Answer: 100,000 writes/s, 10 MB/s, 864 GB/day if kept — so keep only the latest, which is 24 MB

Example 2 [warm-up]

5 million rides a day. The average ride lasts 20 minutes. How many rides are in progress at any moment, on average and at a 3x peak?

SOLUTION

The trick: concurrency = rate $\times$ duration. Work in the same unit, here minutes.

$$\text{ride-minutes per day} = 5,000,000 \times 20 = 100,000,000$$

$$\text{minutes in a day} = 24 \times 60 = 1,440$$

$$\text{average concurrent rides} = 100,000,000/1,440 = 69,444$$

At a 3x peak:

$$69,444 \times 3 = 208,333 \text{ rides in progress}$$

Sanity check against the fleet: with 400,000 drivers online at peak, 208,333 on a trip means $208,333/400,000 = 52\%$ utilisation. That is believable for a busy evening, so the two numbers agree.

Answer: 69,444 average, 208,333 at peak — about 52% of the online fleet

SHORTCUT

Concurrency = arrival rate $\times$ holding time. This one line answers “how many simultaneous X” for rides, calls, video streams, open connections, and in-flight API calls. It is called Little’s Law. Use the same time unit on both sides and you cannot get it wrong.

Example 3 [warm-up]

100 million people watch 40 minutes of video a day at an average 3 Mbps. Find the average and peak concurrent viewers, and the peak egress in Tbps.

SOLUTION

Concurrent viewers — Little’s Law again.

$$\text{watch-minutes/day} = 100,000,000 \times 40 = 4,000,000,000$$

$$\text{average concurrent} = 4,000,000,000 / 1,440 = 2,777,778$$

$$\text{peak at } 3x = 2,777,778 \times 3 = 8,333,333 \text{ viewers}$$

Peak egress.

$$8,333,333 \times 3 \text{ Mbps} = 25,000,000 \text{ Mbps} = 25 \text{ Tbps}$$

Daily bytes.

$$4,000,000,000 \text{ min} \times 60 \text{ s} \times 3,000,000 \text{ bits} / 8 = 9.0 \times 10^{16} \text{ B} \\ = 90 \text{ PB/day}$$

Answer: 8.33 million concurrent at peak, 25 Tbps, 90 PB/day

25 Tbps is more than most countries’ entire international capacity. This is why a video design that draws “service $\rightarrow$ database $\rightarrow$ user” fails instantly. The right first sentence is: “**the CDN serves 95% of the bytes and my origin serves 1.25 Tbps.**”

Example 4 [warm-up]

A 10-minute video is transcoded into five renditions: 0.4, 0.8, 1.2, 2.5 and 5.0 Mbps. How many bytes does the finished video occupy? At 500,000 uploads a day, how much new storage per year?

SOLUTION

Total bitrate across the ladder.

$$0.4 + 0.8 + 1.2 + 2.5 + 5.0 = 9.9 \text{ Mbps}$$

Bytes for 10 minutes. 10 minutes = 600 seconds.

$$9.9 \times 10^6 \text{ bits/s} \times 600 \text{ s} = 5.94 \times 10^9 \text{ bits}$$

$$5.94 \times 10^9 / 8 = 742,500,000 \text{ B} = 742.5 \text{ MB per video}$$

Per day.

$$500,000 \times 742.5 \text{ MB} = 371,250,000 \text{ MB} = 371.25 \text{ TB/day}$$

Per year.

$$371.25 \times 365 = 135,506 \text{ TB} = 135.5 \text{ PB/year}$$

Answer: 742.5 MB per video, 371.25 TB/day, 135.5 PB/year

Example 5 [warm-up]

2 billion searches a day. The client debounces typing and sends 4 prefix requests per search. What QPS must autocomplete serve at peak, and how many in-memory nodes at 50,000 requests/s each?

SOLUTION

$$2,000,000,000 \times 4 = 8,000,000,000 \text{ prefix requests/day}$$

$$8,000,000,000 / 86,400 = 92,593 \text{ requests/second average}$$

$$92,593 \times 3 = 277,778 \text{ requests/second at peak}$$

$$277,778 / 50,000 = 5.6 \rightarrow 6 \text{ nodes}$$

Add headroom to survive losing one: run **8**. Note how small that is — the whole reason it is small is that a request does **no computation**, only a lookup. Change one word in the design (“we search the trie at request time”) and this number becomes 60 nodes.

Answer: 277,778 QPS at peak; 6 nodes of work, run 8

Example 6 [warm-up]

10 million payments a day, peak 10x on a sale day. The external payment provider takes 800 ms to answer. How many calls are in flight at peak?

SOLUTION

$$10,000,000 / 86,400 = 115.7 \text{ payments/second average}$$

$$115.7 \times 10 = 1,157 \text{ payments/second at peak}$$

In-flight calls, by Little’s Law:

$$1,157 \times 0.8 \text{ s} = 926 \text{ calls open at once}$$

That number decides your architecture. 926 open sockets is fine for one Node process doing I/O, but 926 **threads** would not be. And it means a 30-second provider stall parks $1,157 \times 30 = 34,710$ requests — which is why the call must be made by a worker reading a queue, not by the thread holding the user’s HTTP connection.

Answer: 1,157 payments/s at peak, 926 concurrent provider calls

Example 7 [warm-up]

A city is about 50 km by 27 km. You index drivers in 6-character geohash cells (1.2 km x 0.61 km). 60,000 drivers are online in that city. How many cells, how many drivers per cell, and how many candidates does a 9-cell search return?

SOLUTION

Cells.

$$50 / 1.223 = 40.9 \text{ cells across}$$

$$27 / 0.611 = 44.2 \text{ cells down}$$

$$40.9 \times 44.2 = 1,807 \text{ cells}$$

Drivers per cell.

$$60,000 / 1,807 = 33.2 \text{ drivers per cell}$$

A 9-cell search (the cell you are in, plus its 8 neighbours):

$$33.2 \times 9 = 299 \text{ candidates}$$

299 candidates is nothing: you can run an exact distance calculation on every one of them in well under a millisecond, then sort. **The cell index is a cheap filter, not the answer.**

Answer: 1,807 cells, 33 drivers each, 299 candidates from a 9-cell search

TRAP

Why nine cells and not one? Because the rider may be standing 10 metres from a cell border, and the nearest driver may be on the other side of it. Searching only your own cell finds a driver 900 m away and misses one 30 m away. **Always search the ring of neighbours.** This is the single most common bug in a geo-index answer.

Example 8 [warm-up]

400,000 drivers hold an open WebSocket. One machine comfortably holds 100,000 connections. How many machines, and how many if you must survive losing one of three availability zones?

SOLUTION

Base.

$$400,000/100,000 = 4 \text{ machines}$$

Surviving one zone of three. When one zone dies you must carry all the load on the remaining 2/3 of the fleet. So the fleet must be big enough that two thirds of it equals 4 machines' worth of capacity.

$$4/(2/3) = 4 \times 1.5 = 6 \text{ machines}$$

Spread them 2 / 2 / 2. Losing any one zone leaves 4, which is exactly enough.

What actually happens during that failure is the interesting part: 133,333 drivers reconnect at once. If they all retry immediately, the surviving 4 machines get a reconnect storm. Fix: the client waits a random 0 to 30 seconds before reconnecting, which spreads $133,333/30 = 4,444$ reconnects per second instead of 133,333 in one instant.

Answer: 4 machines of load, 6 provisioned as 2/2/2, plus randomised reconnect backoff

PRACTICE · WARM-UP target: 12 min

1. 3 million food orders a day, 35% of them in the 2-hour lunch window. Peak orders/second?
2. A courier fleet of 120,000 pings every 5 seconds. Write rate? If you keep 6 hours of trail at 40 bytes a point, how much memory?
3. A video service has 25 Tbps of peak egress and the CDN offloads 95%. What must the origin serve? If egress costs \$0.01 per GB, what is a day of traffic worth at 90 PB?
4. Autocomplete stores the top 10 suggestions at each of 50 million prefixes, 200 bytes per prefix. How much RAM, and over how many nodes at 32 GB each?
5. 10 million payments a day, two ledger rows each at 300 bytes. Storage per day, per year, and over a 7-year retention rule?

ANSWER KEY

1. $3,000,000 \times 0.35 = 1,050,000$ orders in the window; $1,050,000/7,200 \text{ s} = 145.8$ orders/second.
2. $120,000/5 = 24,000$ writes/second. Six hours of trail is $24,000 \times 3,600 \times 6 = 518,400,000$ points $\times 40 \text{ B} = 20.7 \text{ GB}$. Too much for one Redis node's comfort — downsample to one point per 15 seconds while on a trip and it becomes $20.7 \times 5/15 = 6.9 \text{ GB}$. Or just do not keep the trail of an **idle** courier at all, which removes most of it.
3. Origin = $25 \times 0.05 = 1.25 \text{ Tbps}$. $90 \text{ PB} = 90,000,000 \text{ GB}$; $90,000,000 \times 0.01 = \$900,000$ per day. That number is why large video companies build their own CDN and peer directly with ISPs.
4. $50,000,000 \times 200 = 10,000,000,000 \text{ B} = 10 \text{ GB}$. It fits on **one** 32 GB node — so shard for throughput and failure tolerance, not for size: 8 nodes each holding the full 10 GB, any of which can answer any query. That is a much better design than 8 shards, because it removes the routing layer entirely.
5. $10,000,000 \times 2 \times 300 = 6,000,000,000 \text{ B} = 6 \text{ GB/day}$; $\times 365 = 2,190 \text{ GB} = 2.19 \text{ TB/year}$; $\times 7 = 15.33 \text{ TB}$. Small. Money data is always small and always precious — the opposite shape from video.

Tier 1 — the low-level pieces

Five pieces. Every one of them has been asked on its own in a service-company round, and every one of them is the beating heart of a design later in this chapter.

Piece 1 — a geohash encoder

Example 9 [Infosys · pattern]

Write a function that turns a latitude and longitude into a short string, such that two nearby points share a long prefix.

SOLUTION

The idea. Repeatedly cut the world in half and write down which half you are in. Even-numbered bits cut **longitude**, odd-numbered bits cut **latitude**. Every 5 bits become one character of a 32-letter alphabet.

geohash.js

JS

```
const B32 = '0123456789bcdefghjkmnpqrstuvwxyz'; // no a, i, l or o

function geohash(lat, lng, precision = 6) {
  let latLo = -90, latHi = 90, lngLo = -180, lngHi = 180;
  let bit = 0, ch = 0, even = true, out = '';

  while (out.length < precision) {
    if (even) {
      // even bits split LONGITUDE
      const mid = (lngLo + lngHi) / 2;
      if (lng >= mid) { ch = (ch << 1) | 1; lngLo = mid; }
      else { ch = ch << 1; lngHi = mid; }
    } else {
      // odd bits split LATITUDE
      const mid = (latLo + latHi) / 2;
      if (lat >= mid) { ch = (ch << 1) | 1; latLo = mid; }
      else { ch = ch << 1; latHi = mid; }
    }
    even = !even;
    if (++bit === 5) { out += B32[ch]; bit = 0; ch = 0; }
  }
  return out;
}

// exact distance on a sphere, for the final filter
function haversineKm(a, b) {
  const R = 6371, toRad = d => d * Math.PI / 180;
  const dLat = toRad(b.lat - a.lat);
  const dLng = toRad(b.lng - a.lng);
  const h = Math.sin(dLat / 2) ** 2 +
    Math.cos(toRad(a.lat)) * Math.cos(toRad(b.lat)) *
    Math.sin(dLng / 2) ** 2;
  return 2 * R * Math.asin(Math.sqrt(h));
}
```

actual output

TEXT

```

Changi    w21zy2k
Marina    w21z73t
Marina+   w21z73t  shares prefix with Marina: 7 chars
Changi shares with Marina: 4 chars
Changi -> Marina = 17.10 km

```

Two points 60 metres apart produce the **same** 7-character hash. Two points 17 km apart share only 4 characters. That is the whole property: **prefix length is a proxy for closeness**.

CELL SIZE BY PRECISION — MEMORISE THE MIDDLE THREE

Chars	Cell size	Good for
4	19.6 km x 39.1 km	country-level bucketing
5	4.9 km x 4.9 km	a whole small city; too coarse for dispatch
6	1.22 km x 0.61 km	the dispatch default
7	153 m x 153 m	pickup-point clustering, “which building”

The cell is a **rectangle**, not a square, and the aspect flips at every character, because latitude and longitude do not get the same number of bits at every step.

TRAP

Geohash has two real flaws and you should name both before you are asked.

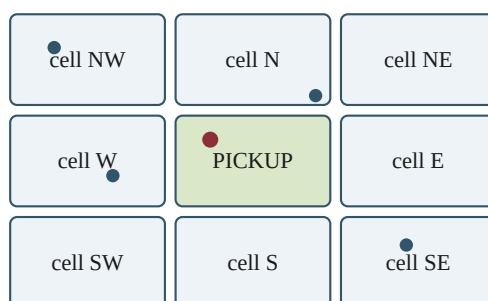
1. **Border blindness.** Neighbouring points can differ in the very first character. The fix is to always search the 8 neighbouring cells too.
2. **Uneven density.** A cell in a city centre may hold 5,000 drivers and one over water may hold zero. A fixed grid cannot adapt. If density varies by more than about 50x, use a **quadtree** (split a cell when it exceeds, say, 100 items) or an S2/H3 cell system instead. For a ride-hailing app inside one city, a fixed 6-character grid is honestly fine, and saying “fine, and here is when it stops being fine” is the answer that scores.

Piece 2 — the live index of things that move

Example 10 [Capgemini · pattern]

Build an in-memory index that answers “which available drivers are within 5 km of this point?” while 100,000 position updates a second are arriving.

SOLUTION



Step 1 cell lookup

9 cells x 33 drivers
= 299 candidates

Step 2 filter

drop **ON_TRIP**, drop stale
pings, older than 30 s

Step 3 exact

haversine, keep ≤ 5 km,
sort by distance

Searching only the centre cell would miss the driver just above its top border.

The 9-cell search. The cheap filter is the cell lookup; the exact filter is haversine. A driver just over a border is found because the ring is searched.

```

const CELL = 5; // 5 chars ~ 4.9 km; 6 for a dense city

class DriverIndex {
  constructor() {
    this.cells = new Map(); // cell -> Set(driverId)
    this.drivers = new Map(); // driverId -> {lat,lng,cell,status,ts}
  }

  // the 8 neighbours, found by nudging the coordinates instead of
  // doing bit arithmetic. Slower to run, far easier to get right.
  static neighbourCells(lat, lng, stepDeg = 0.045) {
    const out = new Set();
    for (const dLat of [-stepDeg, 0, stepDeg])
      for (const dLng of [-stepDeg, 0, stepDeg])
        out.add(geohash(lat + dLat, lng + dLng, CELL));
    return [...out];
  }

  upsert(id, lat, lng, status, ts) {
    const cell = geohash(lat, lng, CELL);
    const old = this.drivers.get(id);
    // moved to a new cell: unlink from the old one FIRST
    if (old && old.cell !== cell) this.cells.get(old.cell).delete(id);
    if (!this.cells.has(cell)) this.cells.set(cell, new Set());
    this.cells.get(cell).add(id);
    this.drivers.set(id, { lat, lng, cell, status, ts });
  }

  remove(id) {
    const d = this.drivers.get(id);
    if (!d) return;
    this.cells.get(d.cell)?.delete(id);
    this.drivers.delete(id);
  }

  nearby(lat, lng, radiusKm, now, staleMs = 30_000) {
    const found = [];
    for (const cell of DriverIndex.neighbourCells(lat, lng)) {
      for (const id of this.cells.get(cell) ?? []) {
        const d = this.drivers.get(id);
        if (d.status !== 'AVAILABLE') continue;
        if (now - d.ts > staleMs) continue; // a ghost: phone lost signal
        const km = haversineKm({ lat, lng }, d);
        if (km <= radiusKm) found.push({ id, km });
      }
    }
    return found.sort((a, b) => a.km - b.km); // numeric comparator
  }
}

```

actual output — d4 is busy, d5 is stale, d6 is 17 km away

TEXT

```

cells in use: [ 'w2lz7', 'w2lzy' ]
candidates within 5 km: [

```

```

{ id: 'd1', km: 0.07458808063127464 },
{ id: 'd2', km: 1.397664362032147 },
{ id: 'd3', km: 3.6540000323125237 }
]
candidates within 1 km: [ { id: 'd1', km: 0.07458808063127464 } ]

```

Why the stale check matters more than it looks. A driver whose phone lost signal is still in the index at their last known position. Offering them a ride wastes 12 seconds of the rider’s life and then re-dispatches. A 30-second staleness cut-off means you only ever offer to phones you have heard from recently. It costs one comparison.

TRAP

The unlink-before-link order is a real bug source. If you add the driver to the new cell before removing them from the old one, and a concurrent read runs in between, the driver appears **twice**. In JavaScript you get away with it inside one synchronous function, because nothing interleaves — but the moment `upsert` contains an `await`, the gap becomes real. Treat “remove from old, then add to new” as one indivisible step, and never put an `await` inside it.

Piece 3 — dispatch with a hold and a timeout

Example 11 [TCS NQT · pattern]

Two riders request at the same second and the same driver is nearest to both. Write the dispatcher so the driver is offered to exactly one of them, and so that ignoring an offer does not strand the rider.

SOLUTION

The rule: an offer is a **hold**. While a driver is holding an offer, no other dispatcher may offer them. The hold expires by itself.

dispatcher.js

JS

```

class Dispatcher {
  constructor(index, offerMs = 12_000) {
    this.index = index;
    this.offerMs = offerMs;
    this.offers = new Map(); // rideId -> {driverId, expiresAt, tried}
  }

  offer(rideId, lat, lng, now, tried = new Set()) {
    const candS = this.index.nearby(lat, lng, 5, now)
      .filter(c => !tried.has(c.id));
    if (candS.length === 0) return { rideId, status: 'NO_DRIVER' };

    const best = candS[0];
    const d = this.index.drivers.get(best.id);
    // the HOLD: status flips so no other dispatcher can pick him
    this.index.upsert(best.id, d.lat, d.lng, 'OFFERED', d.ts);
    this.offers.set(rideId, {
      driverId: best.id, expiresAt: now + this.offerMs, tried,
    });
    return {
      rideId, status: 'OFFERED', driverId: best.id,
      etaMin: +(best.km * 60 / 25).toFixed(1), // 25 km/h city average
    };
  }
}

```



```

accept(rideId, driverId, now) {
  const o = this.offers.get(rideId);
  if (!o || o.driverId !== driverId) return { ok: false, why: 'not yours' };
  if (now > o.expiresAt) return { ok: false, why: 'offer expired' };
  const d = this.index.drivers.get(driverId);
  this.index.upsert(driverId, d.lat, d.lng, 'ON_TRIP', d.ts);
  this.offers.delete(rideId);
  return { ok: true, status: 'MATCHED' };
}

timeout(rideId, lat, lng, now) { // the driver ignored the ping
  const o = this.offers.get(rideId);
  if (!o) return null;
  const d = this.index.drivers.get(o.driverId);
  // release the hold, remember we tried him, go again
  this.index.upsert(o.driverId, d.lat, d.lng, 'AVAILABLE', d.ts);
  o.tried.add(o.driverId);
  this.offers.delete(rideId);
  return this.offer(rideId, lat, lng, now, o.tried);
}
}

```

actual output

TEXT

```

offer 1 : { rideId: 'ride1', status: 'OFFERED', driverId: 'd1', etaMin: 0.2 }
ignored -> { rideId: 'ride1', status: 'OFFERED', driverId: 'd2', etaMin: 3.4 }
accept  : { ok: true, status: 'MATCHED' }
second rider now gets: { rideId: 'ride2', status: 'OFFERED', driverId: 'd1',
                        etaMin: 0.2 }

```

Read the trace. d1 was offered, ignored the ping, was released back to AVAILABLE, and d2 got the ride instead. When a **second** rider arrives, d1 is free again and is offered to them. Nobody was double-booked and nobody was lost.

THE THREE NUMBERS IN A DISPATCH POLICY, AND WHAT EACH ONE TRADES

- **Offer window (12 s).** Longer means fewer re-dispatches but a slower rider experience. 12 seconds is about as long as a person will stare at a “finding your driver” screen without losing faith.
- **Search radius (5 km).** Wider finds a driver in a quiet area but offers a 12-minute pickup. Start at 2 km, widen to 5 km after one failed round, then give up.
- **Staleness cut-off (30 s).** Tighter means fewer ghost offers but drops drivers in tunnels and underground car parks. 30 s is about 7 missed pings at a 4-second interval.

Piece 4 — a trie that has already done the thinking

Example 12 [Cognizant · pattern]

Return the 5 most popular completions of a prefix in under a millisecond. You may spend as much time as you like building the structure beforehand.

SOLUTION

The move that makes this easy: do not search at request time. Store the answer **at every node** while building. A request then walks the prefix and reads a list.

topk-trie.js

JS

```

class TrieNode {
  constructor() {
    this.children = new Map();
    this.top = []; // [{q, count}] kept sorted, length <= k
  }
}

class TopKTrie {
  constructor(k = 5) { this.root = new TrieNode(); this.k = k; }

  // insert one (query, count) pair, updating EVERY node on its path
  insert(query, count) {
    let node = this.root;
    this.#merge(node, query, count);
    for (const ch of query) {
      if (!node.children.has(ch)) node.children.set(ch, new TrieNode());
      node = node.children.get(ch);
      this.#merge(node, query, count);
    }
  }

  #merge(node, q, count) {
    const i = node.top.findIndex(e => e.q === q);
    if (i >= 0) node.top[i].count = count;
    else node.top.push({ q, count });
    // numeric sort desc, with an alphabetical tie-break for stability
    node.top.sort((a, b) => b.count - a.count || (a.q < b.q ? -1 : 1));
    if (node.top.length > this.k) node.top.length = this.k;
  }

  // the request path: walk, then READ. No search, no heap, no scan.
  suggest(prefix) {
    let node = this.root;
    for (const ch of prefix) {
      node = node.children.get(ch);
      if (!node) return [];
    }
    return node.top.map(e => e.q);
  }
}

```

actual output for a food-search log

TEXT

```

"c"      -> [ 'chicken rice', 'chilli crab', 'char kway teow',
              'chicken biryani', 'chicken curry' ]
"chi"    -> [ 'chicken rice', 'chilli crab', 'chicken biryani',
              'chicken curry', 'chicken wings' ]
"chic"   -> [ 'chicken rice', 'chicken biryani', 'chicken curry',
              'chicken wings' ]
"cha"    -> [ 'char kway teow', 'chapati' ]
"z"      -> []

```

nodes for 9 queries: 75 | each node stores up to 5 entries
 after a new query becomes #1: ['chicken rice stall', 'chicken rice',
 'chilli crab', 'chicken biryani',
 'chicken curry']

Time suggest: $O(p)$ where p is the prefix length — typically 3 to 8 character steps **Space** $O(N \times L \times k)$ — one top-k list per distinct prefix

Build is the expensive side: $O(Q \times L \times k \log k)$ for Q queries of length L . Build offline, hourly, never on the request path.

The trade you just made. You turned a request-time search of a whole subtree — which for the prefix "c" could be a million queries — into a pointer walk of 1 character. You paid for it in memory and in build time. That is the correct trade when reads outnumber writes by 80 million to one, which is exactly the case here.

TRAP

Do not call `sort()` without a comparator on the counts. `[100, 9, 1000].sort()` gives `[1, 100, 9]` — wait, as strings it gives `["100", "1000", "9"]`, so 9 sorts **last**. Your most popular suggestion silently disappears from the top-5. Always `(a, b) => b.count - a.count`.

Piece 5 — an idempotent, double-entry payment

Example 13 [Accenture · pattern]

A phone on a bad network sends the same payment request three times. Charge the user exactly once. Then prove, from the data, that no money was created or destroyed.

SOLUTION

THREE RULES THAT ARE NEVER NEGOTIABLE

1. **Money is an integer of the smallest unit.** 129.50 SGD is 12950. A float loses cents and no ledger will ever balance again.
2. **Every movement writes two rows that sum to zero.** One account is debited, another is credited. If the sum of every row in the whole system is not exactly zero, you have a bug, and you can detect it with one query.
3. **Nothing is ever updated or deleted.** A mistake is corrected by a **new** entry with the opposite sign. This is what makes an audit possible and what makes “who changed this?” answerable.

ledger.js — the part that cannot be wrong

JS

```
class Ledger {
  constructor() { this.entries = []; this.seq = 0; }

  post(txnId, legs) {
    const sum = legs.reduce((s, l) => s + l.amount, 0);
    if (sum !== 0) throw new Error(`unbalanced: legs sum to ${sum}, must be 0`);
    for (const l of legs) {
      this.entries.push({ seq: ++this.seq, txnId, ...l });
    }
  }

  balance(account) {
    return this.entries
      .filter(e => e.account === account)
      .reduce((s, e) => s + e.amount, 0);
  }
}
```

payment-service.js — the idempotency gate and the state machine

JS

```

const State = Object.freeze({
  CREATED: 'CREATED', AUTHORISED: 'AUTHORISED', CAPTURED: 'CAPTURED',
  FAILED: 'FAILED', REFUNDED: 'REFUNDED',
});

const ALLOWED = {
  CREATED: new Set(['AUTHORISED', 'FAILED']),
  AUTHORISED: new Set(['CAPTURED', 'FAILED']),
  CAPTURED: new Set(['REFUNDED']),
  FAILED: new Set([]),
  REFUNDED: new Set([]),
};

class PaymentService {
  constructor(psp) {
    this.psp = psp;
    this.byId = new Map();
    this.byIdemKey = new Map();
    this.ledger = new Ledger();
  }

  async pay({ idempotencyKey, payerId, merchantId, amountCents }) {
    // THE GATE. Check-and-claim must be one atomic step.
    // In a real system this is INSERT ... ON CONFLICT DO NOTHING.
    const seen = this.byIdemKey.get(idempotencyKey);
    if (seen) return { replayed: true, ...this.byId.get(seen) };

    const paymentId = `pay_${this.byId.size + 1}`;
    this.byIdemKey.set(idempotencyKey, paymentId);
    const p = {
      paymentId, payerId, merchantId, amountCents, state: State.CREATED,
    };
    this.byId.set(paymentId, p);

    // pass OUR id to the provider so the provider dedupes too
    let res;
    try {
      res = await this.psp.authorise({ ref: paymentId, amountCents });
    } catch (e) {
      return { replayed: false, ...this.#move(p, State.FAILED, e.message) };
    }
    if (!res.ok) {
      return { replayed: false, ...this.#move(p, State.FAILED, res.reason) };
    }

    this.#move(p, State.AUTHORISED);
    this.#move(p, State.CAPTURED);
    this.ledger.post(paymentId, [
      { account: `user:${payerId}`, amount: -amountCents },
      { account: `merchant:${merchantId}`, amount: +amountCents },
    ]);
    return { replayed: false, ...p };
  }

  refund(paymentId, amountCents) {
    const p = this.byId.get(paymentId);
    if (!p) return { ok: false, why: 'unknown payment' };
  }
}

```

```

    if (amountCents > p.amountCents) return { ok: false, why: 'too large' };
    this.#move(p, State.REFUNDED);
    // a NEW entry with the opposite sign. The old one is untouched.
    this.ledger.post(`${paymentId}_rf`, [
      { account: `merchant:${p.merchantId}`, amount: -amountCents },
      { account: `user:${p.payerId}`, amount: +amountCents },
    ]);
    return { ok: true, state: p.state };
  }

  #move(p, to, reason) {
    if (!ALLOWED[p.state].has(to)) {
      throw new Error(`illegal move ${p.state} -> ${to}`);
    }
    p.state = to;
    if (reason) p.reason = reason;
    return p;
  }
}

```

actual output

TEXT

```

first call: pay_1 CAPTURED replayed = false
retry call: pay_1 CAPTURED replayed = true
provider was called 1 time(s) -> the user was charged once
declined : pay_2 FAILED limit_exceeded

balances after one capture:
merchant:42    12950
user:7         -12950

refund 30.00: { ok: true, state: 'REFUNDED' }
merchant:42    9950
user:7         -9950
sum of every row in the ledger = 0 (must always be 0)
rejected: unbalanced: legs sum to -100, must be 0
rejected: illegal move REFUNDED -> REFUNDED

```

Four things are proved by that output, and you should say all four:

1. The retry returned the **same** payment and did not call the provider again.
2. The refund did not edit the original entry; it added an opposite one, and the balances moved from 12950 to 9950 as a result.
3. The sum of every row is zero — money was neither created nor destroyed.
4. Two classes of bug are impossible by construction: an unbalanced entry throws, and an illegal state move throws.

TRAP

The idempotency key must be claimed atomically, in the same store as the payment. If you write `if (!exists(key)) { insert(key); charge(); }` as two statements, two retries arriving in the same 5 ms both see “not exists” and both charge. The correct shape is a single `INSERT ... ON CONFLICT DO NOTHING (or PUT IF NOT EXISTS)`, and the **result of that insert** tells you whether you are the first caller. Checking a Redis cache first is a performance optimisation, never the guarantee.

Piece 6 — a saga, because there is no distributed transaction

Example 14 [Wipro · pattern]

A food order must hold the customer's payment, reserve the food, assign a courier, and burn a promo code. Four different services. If the third one fails, undo the first two.

SOLUTION

```
saga.js JS

class Saga {
  constructor(name) { this.name = name; this.steps = []; }

  step(name, run, compensate) {
    this.steps.push({ name, run, compensate });
    return this; // chainable
  }

  async execute(ctx) {
    const done = [];
    for (const s of this.steps) {
      try {
        const out = await s.run(ctx);
        Object.assign(ctx, out ?? {});
        done.push(s);
        ctx.trace.push(`OK ${s.name}`);
      } catch (err) {
        ctx.trace.push(`FAIL ${s.name}: ${err.message}`);
        // undo in REVERSE order: last done, first undone
        for (const d of done.reverse()) {
          try {
            await d.compensate(ctx);
            ctx.trace.push(`UNDO ${d.name}`);
          } catch (e2) {
            ctx.trace.push(`UNDO-FAILED ${d.name} -> manual queue`);
          }
        }
        return { ok: false, failedAt: s.name, ctx };
      }
    }
    return { ok: true, ctx };
  }
}
```

actual output — attempt 1 fails, attempt 2 succeeds

TEXT

```
--- attempt 1: no courier available ---
ORDER REJECTED at assign_courier
  OK   hold_payment
  OK   reserve_food
  FAIL assign_courier: no courier within 20 minutes
  UNDO reserve_food
  UNDO hold_payment
  world after rollback: {"held":0,"stock":{"biryani":2},
                        "couriersFree":0,"promoUses":0}

--- attempt 2: a courier comes online ---
ORDER PLACED
```

```
OK    hold_payment
OK    reserve_food
OK    assign_courier
OK    burn_promo
world after success: {"held":2400,"stock":{"biryani":1},
                     "couriersFree":0,"promoUses":1}
```

After the rollback, held is back to 0 and stock is back to 2 — exactly the state before the attempt. That is what a saga buys you: **eventual** atomicity, achieved by writing the undo yourself.

THE FOUR RULES OF A SAGA THAT ACTUALLY WORKS

1. **Every step must be idempotent.** A retry of hold_payment must not hold twice. Give each step a deterministic key derived from the order id.
2. **Every compensation must be idempotent too**, and must tolerate being called for a step that may not have fully finished.
3. **Some things cannot be compensated.** You cannot un-send an email or un-deliver food. Put those steps **last**, after everything that can fail has already succeeded.
4. **A compensation that fails goes to a human queue with an alarm.** It never silently disappears. The UNDO-FAILED line in the trace is not decoration.

PRACTICE · TIER 1 target: 35 min

1. Extend geohash with a decode that returns the centre of a cell and its error bounds. Verify that `decode(geohash(lat, lng, 7))` is within 100 m of the input.
2. `DriverIndex.neighbourCells` nudges by a fixed 0.045 degrees, which is right for 5-character cells. Make the step depend on CELL so 6 and 7 also work, and prove it by checking that exactly 9 distinct cells come back for a point in the middle of a cell.
3. Add surge to the dispatcher: if `nearby()` returns fewer than 3 drivers, multiply the fare by a factor. Choose the factor curve and justify the highest value you allow.
4. Make `TopKTrie` support **deletion** of a query whose popularity has collapsed. Explain why this is much harder than insertion and what a real system does instead.
5. Add a hold and capture split to `PaymentService`: authorise now, capture when the food is delivered, and expire the authorisation after 7 days. Which ledger entries move, and when?

ANSWER KEY

1. A 7-character cell is 153 m x 153 m, so the centre is at most $\sqrt{76.5^2 + 76.5^2} = 108$ m from any point inside it. Your check should therefore allow 110 m, not 100 m. Getting the **error bound** right is the point of the exercise.
2. Step should be roughly the cell's own height in degrees: $180/2^{\text{latbits}}$. For 6 characters latbits is 15, giving $180/32768 = 0.0055$ degrees. Nudging by a full cell width guarantees you land in the neighbour and not back in your own cell.
3. A sane curve: 1.0x at 3+ drivers, 1.3x at 2, 1.6x at 1, and **refuse to quote** at 0 rather than showing 3x. Cap at 2.0x. The reason to cap is not technical — an uncapped multiplier during an emergency is a public-relations disaster and in several places is illegal.
4. Deletion is hard because a query sits in the **top** list of **every node along its path**, and removing it leaves a hole that must be refilled from the node's subtree — which is exactly the expensive search you built this structure to avoid. Real systems do not delete: they **rebuild** the whole trie from the last 24 hours of logs, hourly, and swap the new one in atomically. Rebuilding 10 GB in the background is far cheaper than supporting deletion.
5. **authorise** moves nothing in the ledger — it is a **promise**, recorded in the payment row only. **capture** posts the two legs. Expiry posts nothing either; it just moves the payment to **EXPIRED**. **Only capture and refund touch the ledger**. Candidates who post an entry at authorisation end up with a ledger whose balance includes money that was never taken.

Design 1 — ride-hailing

Example 15 [Grab · pattern]

Design a service that matches riders with nearby drivers, tracks the trip, and prices it. 5 million rides a day, 400,000 drivers online at peak.

SOLUTION**Step 1 — Clarify**

Question I ask	Why the answer changes the design
How many drivers online at peak, and how often do they ping?	This is the largest write rate in the system by 1000x. It sets the whole architecture.
How fresh must a driver's position be?	2 seconds means memory. 30 seconds means you can batch and relax.
One city at a time, or cross-city trips?	City-scoped means every query and every shard can be city-scoped. Huge simplification.
Must I keep the route the car actually drove?	Yes for disputes and receipts — but downsampled, and only while on a trip.
Can a driver be offered two rides at once?	No. That makes the offer a hold , which makes dispatch a locking problem.
Is pricing fixed at request time or at drop-off?	Fixed up front means you quote before you know the route, and eat the variance.

Answers assumed: 400,000 drivers online at peak, pinging every 4 s; positions must be under 5 s old; trips are within one city; keep a downsampled trail for 90 days; one offer per driver at a time; **the fare is quoted up front and honoured.**

Step 2 — Scale estimate

The location firehose — the number that decides everything.

$$400,000/4 = 100,000 \text{ position updates/second}$$

$$100,000 \times 100 \text{ B} = 10,000,000 \text{ B/s} = 10 \text{ MB/s} = 80 \text{ Mbps}$$

If every ping were stored:

$$10,000,000 \times 86,400 = 864,000,000,000 \text{ B} = 864 \text{ GB/day}$$

Ride requests — three orders of magnitude smaller.

$$5,000,000/86,400 = 57.9 \text{ requests/second average}$$

$$57.9 \times 5 = 289 \text{ requests/second at a Friday-evening peak}$$

Trips in progress (Little's Law, 20-minute rides):

$$5,000,000 \times 20/1,440 = 69,444 \text{ average} \quad 69,444 \times 3 = 208,333 \text{ at peak}$$

The trail we actually keep: one point per 15 seconds, only while on a trip.

$$208,333/15 = 13,889 \text{ writes/second}$$

$$13,889 \times 40 \text{ B} = 555,560 \text{ B/s}$$

$$555,560 \times 86,400 = 48,000,000,000 \text{ B} = 48 \text{ GB/day}$$

That is **18 times less** than storing every ping, and it is the only version anybody will ever read.

The live index in memory:

$$400,000 \times 60 \text{ B} = 24,000,000 \text{ B} = 24 \text{ MB}$$

Connections. 400,000 driver WebSockets at 100,000 per machine = 4 machines of load; provision 6 as 2/2/2 so one zone can be lost.

THE BOARD SUMMARY

100,000 pings/s · 289 ride requests/s at peak · 208,333 trips in progress · **24 MB of live index** · 48 GB/day of trail.

The write load and the read load are in completely different systems. The pings never touch a database. The rides never touch the ping path. Say that sentence before you draw anything.

Step 3 — API surface

WebSocket for the things that stream, HTTP for the things that decide

TEXT

```
WS    /v1/driver/stream                (driver app, always open)
  up   { t:"ping", lat, lng, heading, speed, ts }      every 4 s
  down { t:"offer", rideId, pickup, dropoff, fareCents, expiresInMs }
  down { t:"cancelled", rideId, reason }

POST /v1/rides
  header Idempotency-Key: 3f9a-...
  body   { riderId, pickup:{lat,lng}, dropoff:{lat,lng}, productId }
  201    { rideId, state:"SEARCHING", quoteCents, quoteExpiresAt }

POST /v1/rides/{rideId}/offers/{offerId}/accept      (driver)
  200    { rideId, state:"MATCHED", riderName, pickup }
  409    { error:"offer_expired" } | 409 { error:"already_taken" }

GET  /v1/rides/{rideId}
  200    { rideId, state, driver:{id,name,plate,lat,lng}, etaSec }
  note   the rider app polls this every 3 s, OR holds its own WS

POST /v1/rides/{rideId}/complete                    (driver)
  body   { endLat, endLng, distanceM, durationS }
  200    { fareCents, paymentState }
```

SHORTCUT

Say why the driver uses a WebSocket and the rider mostly does not. The driver's phone must **receive** an unsolicited offer within 2 seconds — that is a push, and polling for it 400,000 times over would be absurd. The rider is mostly **asking** ("where is my driver?"), which is a poll, and polling every 3 seconds for the 5 minutes before pickup is 100 requests per ride — cheap and far simpler to operate. Choose the transport per **direction of initiative**.

Step 4 — Data model

three stores, because three very different access patterns

TEXT

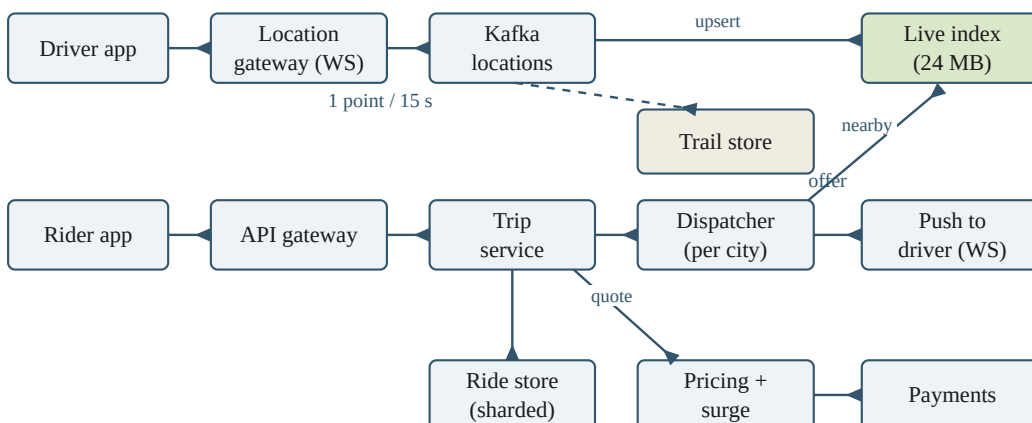
```
-- IN MEMORY (Redis or an in-process index), never on disk
driver:{id}      hash   lat,lng,cell,status,ts      TTL 60 s
cell:{geohash6}  set    driverIds                  updated on cell change
offer:{rideId}   string driverId                   TTL 12 s (the HOLD)
```

```
-- OLTP, sharded by city_id then hashed on ride_id
rides
ride_id      ulid      PK
city_id      int       SHARD KEY INPUT
rider_id     bigint
driver_id     bigint    null until MATCHED
state        enum      SEARCHING|MATCHED|ARRIVED|ON_TRIP|DONE|CANCELLED
quote_cents  bigint     integers only
fare_cents   bigint     null until DONE
requested_at, matched_at, started_at, ended_at
INDEX (rider_id, requested_at DESC)    -- "my trips"
INDEX (driver_id, requested_at DESC)   -- "my earnings"

-- APPEND-ONLY, time-partitioned, cheap storage
ride_trail
ride_id, seq, lat, lng, ts          one row per 15 s while ON_TRIP
PARTITION BY month, dropped after 90 days
```

Why city_id and not rider_id? Because every interesting query is city-scoped: the dispatcher for Singapore never needs a row from Jakarta. Sharding on city means a dispatcher instance owns a city, holds that city's index in its own memory, and never makes a network call to match. **The cost:** cities differ in size by 50x, so one shard is huge and another is tiny. We accept that and give the big cities their own shard, using the directory-router trick from Chapter 10.

Step 5 — Architecture



Solid = on a user's critical path. Dashed = asynchronous. The ping path writes to memory, never to a database. One dispatcher process owns one city and holds that city's index locally, so a match needs zero network calls.

The location path (top) and the ride path (bottom) share nothing but the in-memory index. That separation is the design.

Trace one ride, end to end:

1. Driver phones stream pings into the location gateway over their open WebSocket. The gateway does **no work** except validate and publish to Kafka.
2. A consumer applies each ping to the live index: update driver: {id}, and if the geohash cell changed, move the id between two sets. Cost per ping: two hash operations.
3. Separately, a second consumer writes one trail row every 15 seconds for drivers who are ON_TRIP. This is the only thing that touches a disk.
4. POST /v1/rides arrives. The trip service writes a SEARCHING row, asks pricing for a quote, and hands the request to the city's dispatcher.
5. The dispatcher runs nearby() in its own memory — 9 cells, 299 candidates, exact distances, sorted. It places a 12-second hold on the best driver and pushes an offer down that driver's WebSocket.

6. Driver accepts: the ride row moves to `MATCHED`, the hold becomes `ON_TRIP`, the rider is told. Driver ignores: the hold expires, the driver returns to `AVAILABLE`, and the dispatcher offers the next candidate — excluding the one who just ignored it.
7. At drop-off the fare is finalised and the payment is a separate, idempotent call.

Step 6a — Deep dive: never write a ping to a database

Suppose you did. 100,000 writes/second against a store doing 5,000 writes/second per node needs $100,000/5,000 = 20$ nodes, purely to record data that is **worthless one second later**, and 864 GB a day to store it.

THE THREE-LAYER ANSWER, WITH THE COST OF EACH

1. **Layer 1 — the live index, in memory.** 24 MB. Answers every dispatch query. Lost on restart, which costs nothing: within 4 seconds every online driver has pinged again and the index has rebuilt itself. **This is the rarest kind of state: important and disposable.**
2. **Layer 2 — Kafka, for 24 hours.** $10 \text{ MB/s} \times 86,400 = 864 \text{ GB}$ of retention. This is the replay buffer: if the indexing consumer has a bug, you fix it and replay. Kafka is cheap for this because it is a sequential append and never a random write.
3. **Layer 3 — the trail, on disk, downsampled.** 48 GB/day, 90 days = 4.32 TB, partitioned by month so deleting old data is a partition drop, not 4 billion DELETES.

What we gave up. You cannot ask “where was driver 55 at 10:04 last Tuesday if they were not on a trip?” We decided that question has no business value that justifies 864 GB a day. State the decision, not just the design.

Step 6b — Deep dive: two riders, one driver

At peak, 289 requests/second arrive across all cities. In a dense city centre at 6 p.m., two requests 400 m apart can legitimately see the same nearest driver within the same 50 ms.

Wrong fix: “check if the driver is available, then offer”. Two dispatchers both read `AVAILABLE`, both offer, and the driver’s phone shows two ride cards. Whichever they tap, the other rider is now stranded and must be re-dispatched, having already waited.

Right fix — the offer is a conditional write:

the only safe order

TEXT

```
WRONG:  if (status == AVAILABLE) { status = OFFERED; send(offer); }
RIGHT:  SET offer:{driverId} = rideId NX EX 12
        -> if the SET returns "already exists", this driver is taken:
            move to the next candidate, do not send anything
```

`NX` means “only if absent” and `EX 12` means “expire in 12 seconds”. The store decides, atomically, and the loser finds out in microseconds rather than after the driver taps.

What if the dispatcher crashes while holding offers? Nothing needs cleaning up. Every hold has a 12-second TTL, so the whole system self-heals within 12 seconds with no recovery code at all. **Expiry-based state is the cheapest form of fault tolerance there is.**

Step 6c — Deep dive: the quote you promised before you knew the route

The rider is quoted a fare before the trip starts. The trip then takes a different route, hits traffic, and costs you more than you charged.

WHERE THE MONEY ACTUALLY GOES, PER 1,000 RIDES

Suppose the quote is based on the predicted distance and duration, and the real trip differs. If 1,000 rides average a quote of 1,200 cents and the realised cost averages 1,260 cents, the loss is

$$1,000 \times (1,260 - 1,200) = 60,000 \text{ cents} = 600 \text{ currency units per 1,000 rides}$$

$$\text{as a fraction} = 60/1,200 = 5\% \text{ of revenue}$$

Five percent is not a rounding error; it is often the entire margin.

Three ways to close it, and what each costs:

1. **Quote a range, charge the actual.** Removes the risk entirely. **Cost:** riders hate it, and conversion drops measurably, because people will not tap a button that says “800 to 1,400”.
2. **Quote fixed, and improve the prediction.** Feed the estimator real trail data: this road at this hour takes this long. **Cost:** a whole model and pipeline, and it is never perfect.
3. **Quote fixed, and add a variance buffer.** Price at the **75th percentile** of predicted cost instead of the mean. **Cost:** you are slightly expensive on typical trips, which loses some bookings.

Decision: fixed quote priced at the 75th percentile, plus a hard rule that the fare only changes if the rider changes the destination. A fixed price is worth more in conversion than the 5% costs, and the one exception — the rider moved the pin — is the one case where a price change feels fair to a human.

Step 7 — Trade-offs, failure modes, 10x

Choice	Option A	Option B	Decision
Positions	persist every ping	memory + downsampled trail	B — 24 MB versus 864 GB/day for data read once
Geo index	database GEOSPATIAL index	B+T+H cells in memory	B — 100,000 writes/s is not a database workload
Driver transport	HTTP polling	WebSocket	B — offers are pushes and must land in 2 s
Rider transport	WebSocket	poll every 3 s	B — the rider asks, never receives unsolicited; polling is 100 calls/ride
Offer	read-then-write	SET NX EX	B — atomic, self-expiring, needs no crash recovery
Matching	global optimiser over all pairs	greedy nearest, per request	B — greedy is 2 ms and within a few percent of optimal at this density
Shard key	rider_id	city_id	B — dispatch is city-scoped, so a match needs zero cross-shard work
Fare	meter at drop-off	fixed quote at the 75th percentile	B — conversion is worth more than the 5% variance

What fails	What the user sees	What we do
The live index process dies	Dispatch stops in that city for 5 s	It rebuilds itself from the next round of pings — every driver pings within 4 s. Keep a hot standby consuming the same Kafka partitions so the gap is under a second.
Kafka is slow	Positions go stale, ETAs get worse	The gateway drops the oldest ping per driver rather than buffering. A newer position makes an older one worthless, so dropping is correct here — the rare

		case where losing data is the right answer.
A driver's phone loses signal mid-trip	Rider's map freezes	The trip does not cancel. The trail resumes when signal returns; the fare uses the completed-trip report, cross-checked against the trail.
Two dispatchers offer the same driver	Cannot happen	SET NX makes it impossible. The loser never sends anything.
Dispatcher crashes holding 50 offers	50 riders wait up to 12 extra seconds	Holds expire on their own. No recovery code, no cleanup job.
Surge multiplier misconfigured to 9x	Outrage	A hard cap in code, not in config, and an alarm on any city exceeding 2.0x for more than 10 minutes.
One city's shard is overloaded	Slow matching in that city only	The blast radius is exactly one city, which is why we sharded by city. Move that city to its own shard via the directory router.

At 10x (50 million rides/day, 4 million drivers online):

- Pings become $4,000,000/4 = 1,000,000/\text{second}$ and 100 MB/s. Kafka handles it with $100/10 = 10$ partitions of throughput, but we would use far more for consumer parallelism.
- The live index becomes $4,000,000 \times 60 = 240$ MB. Still trivially in memory — **but not in one process's memory if that process must also serve dispatch for every city**. The fix is the one we already built in: one dispatcher per city, each holding only its own city. At 10x this stops being an optimisation and becomes the only way it works.
- Trips in progress become 2,083,333 and the trail becomes 480 GB/day, 43.2 TB over 90 days. Now storage is real: compress the trail (consecutive points differ by a few metres, so delta-encoding cuts it by roughly 4x) and drop to one point per 30 seconds for the middle of a trip while keeping 5-second resolution near pickup and drop-off, where disputes actually happen.
- The genuinely new problem at 10x is **matching quality**, not throughput. Greedy nearest leaves value on the table when 40 riders and 40 drivers are in the same square kilometre. Batch requests into 2-second windows and solve an assignment problem over the batch. **Cost:** every rider waits up to 2 extra seconds. **Benefit:** measurably shorter pickups fleet-wide. **Decision:** batch only in the top 20 dense cells of a city, where the win is real, and stay greedy everywhere else.

Design 2 — food delivery

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 16 [Shopee · pattern]

Design a food-delivery service: 2 million orders a day, three parties per order (customer, restaurant, courier), and a lunch rush that concentrates a third of the day into two hours.

SOLUTION

Step 1 — Clarify

Questions and the assumptions I will state out loud: 2 million orders/day; 35% of them land in a 2-hour lunch window; 120,000 couriers online at peak, pinging every 5 s; a courier may carry up to 3 orders at once; restaurants confirm or reject within 90 seconds; payment is authorised at order time and captured at delivery; the customer must see live status.

Step 2 — Scale estimate

$$2,000,000/86,400 = 23.1 \text{ orders/second average}$$

$$2,000,000 \times 0.35 = 700,000 \text{ orders in the lunch window}$$

$$700,000/7,200 \text{ s} = 97.2 \text{ orders/second at lunch}$$

The peak factor here is $97.2/23.1 = 4.2 \times$, and it is **predictable to the minute**. That is a gift: you can scale up at 11:15 and down at 14:00 on a schedule, instead of reacting.

Courier pings.

$$120,000/5 = 24,000 \text{ updates/second}$$

A quarter of the ride-hailing firehose, handled by exactly the same three-layer design.

Trips, with batching. Three drops per trip:

$$97.2/3 = 32.4 \text{ courier trips/second at lunch}$$

Storage. One order plus its items and status history: about 1.5 KB.

$$2,000,000 \times 1,500 = 3,000,000,000 \text{ B} = 3 \text{ GB/day}$$

$$3 \times 365 = 1,095 \text{ GB/year} \approx 1.1 \text{ TB/year}$$

Status reads. A customer watches the map. Assume 60 status polls per order over 35 minutes:

$$2,000,000 \times 60 = 120,000,000 \text{ reads/day} = 1,389 \text{ reads/second average}$$

At lunch, $1,389 \times 4.2 = 5,834 \text{ reads/second}$. Every one of them is served from a cache keyed by order id, refreshed by the event stream.

Step 3 — API surface

the transitions are the API

TEXT

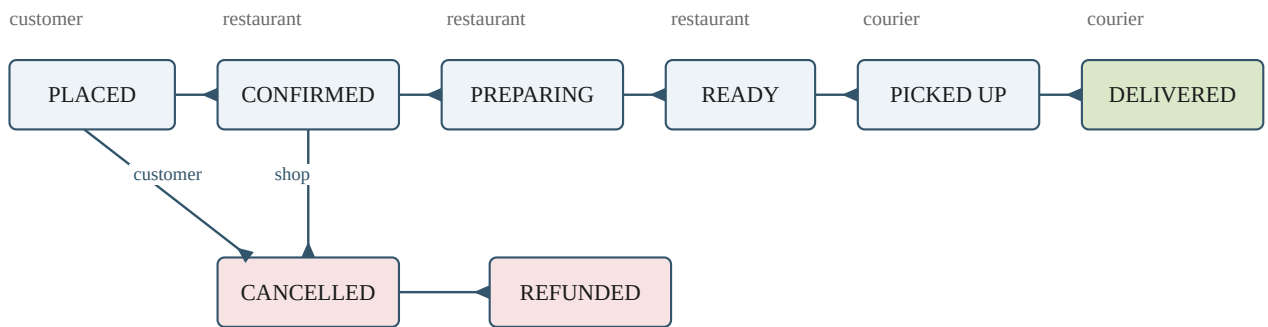
```
POST /v1/orders
  header Idempotency-Key: 7c21-...
  body   { customerId, restaurantId, items[], addressId, promoCode,
          paymentMethodId }
  201    { orderId, state:"PLACED", etaMinutes, totalCents }
  409    { error:"restaurant_closed" | "item_unavailable" }

POST /v1/orders/{orderId}/transitions      (restaurant / courier app)
  body   { to:"CONFIRMED"|"PREPARING"|"READY"|"PICKED_UP"|"DELIVERED",
          actor, at }
  200    { orderId, state }
  409    { error:"illegal_transition", from, to }

GET /v1/orders/{orderId}/live
  200    { state, courier:{lat,lng,etaSec}, preparedAt, asOf }
  note   served from cache; asOf tells the app how fresh it is

POST /v1/orders/{orderId}/cancel
  body   { by:"customer"|"restaurant", reason }
  200    { orderId, state:"CANCELLED", refundCents }
  note   refundCents depends on state: full before CONFIRMED,
          partial after PREPARING, none after PICKED_UP
```

Step 4 — The state machine is the data model



There is no arrow out of PICKED UP except DELIVERED: once food is in the bag, the order completes and a refund is a separate decision.

Every transition names who may trigger it. A transition not on this diagram returns 409, and that rule lives in one place in the code.

the tables

TEXT

```

orders
  order_id      ulid      PK
  city_id       int       SHARD KEY INPUT
  customer_id, restaurant_id, courier_id (null until assigned)
  state         enum
  total_cents   bigint    integers
  placed_at, confirmed_at, ready_at, picked_at, delivered_at
  version       int       optimistic lock on transitions
  INDEX (customer_id, placed_at DESC)
  INDEX (restaurant_id, state)      -- the restaurant's tablet

order_events      -- append-only, same shard
  order_id, seq, from_state, to_state, actor, at, note

courier_trips     -- one row per batched trip
  trip_id, courier_id, order_ids[], started_at, ended_at
  
```

Step 5 — Deep dive: there is no transaction across four services

Placing an order must hold the payment, reserve the items, find a courier, and burn the promo. Four services, four databases. A two-phase commit across them would block all four whenever the coordinator died — during the lunch rush.

We use the saga from Piece 6. The order of the steps is the design decision:

Step	Why it is here in the order	Compensation
1 hold payment	Fails most often (declined card). Fail early, before anyone does work.	release the hold — free, instant
2 reserve items	Cheap and reversible. Must happen before the restaurant is told.	return the stock
3 assign courier	The likeliest capacity failure at lunch. Still fully reversible.	release the courier
4 burn promo	Reversible but annoying to reverse; do it once everything else succeeded.	restore the promo use
5 notify restaurant	Not compensable. A printed ticket cannot be unprinted. Therefore last.	none — this is why it is last

TRAP

Order your saga steps by “likely to fail” first and “impossible to undo” last. Candidates usually list the steps in the order a product manager says them, which puts “tell the restaurant” early — and then the rollback has to phone a human. The ordering **is** the answer to this question.

Step 6 — Deep dive: batching three orders onto one courier

A courier carrying 3 orders costs a third as much per order, but every extra stop makes the **first** customer’s food later.

THE ARITHMETIC OF A BATCH, PER ORDER

Assume a single delivery is 18 minutes door to door and each extra stop adds 7 minutes.

Batch size	Total trip	Cost per order (at 100 per trip)	Worst customer waits
1	18 min	100	18 min
2	25 min	50	25 min
3	32 min	33.3	32 min
4	39 min	25	39 min

Going from 1 to 3 cuts cost per order by $(100 - 33.3)/100 = 66.7\%$ and adds 14 minutes to the unluckiest customer. Going from 3 to 4 saves a further $(33.3 - 25)/33.3 = 25\%$ and adds another 7 minutes.

Decision: cap the batch at 3, and only batch orders whose restaurants are within 800 m of each other and whose drop-offs are within 1.5 km. The 1-to-3 step is where almost all the saving is; the 3-to-4 step buys a quarter as much for the same 7-minute penalty. And the distance rules are what make the “+7 minutes per stop” assumption true in the first place — without them a batch is two trips pretending to be one.

Step 7 — Trade-offs and failures

Choice	Option A	Option B	Decision
Order placement	two-phase commit	saga with compensations	B — 2PC blocks everyone when the coordinator dies, at lunch
Restaurant notify	first step	last step	B — it is the one step that cannot be undone
Batch size	up to 4	up to 3, with distance rules	B — 1-to-3 captures 67% of the saving; 3-to-4 adds 7 min for 8%
Live status	query the order shard	cache keyed by order id, fed by events	B — 5,834 reads/s at lunch against 97 writes/s
Scaling for lunch	autoscale on CPU	scheduled scale-up at 11:15	B — the spike is 4.2x in 15 minutes; reactive autoscaling is always late
Cancellation	always refundable	refund depends on state	B — after PREPARING the food is real and someone paid for it

What fails

What the user sees

What we do

Restaurant never confirms	Order sits at PLACED	A 90-second timer auto-cancels with a full refund and suggests three nearby alternatives. Silence must always have a deadline.
No courier at lunch peak	Longer ETA, or “we cannot deliver right now”	Widen the search radius in stages, then raise courier pay for that zone, then stop accepting orders for it. Refusing an order beats accepting one you cannot deliver.
Payment authorisation expires before delivery	Charge fails at the end	Authorisations last 7 days; a 35-minute order never comes close. Alarm if any order exceeds 6 hours between authorise and capture.
A compensation fails	Nothing, yet	It goes to a human queue with an alarm. Never retried silently forever, never dropped.
The same transition arrives twice	Nothing	Transitions are idempotent by (order_id, to_state), and the state machine rejects anything not on the diagram with a 409.

PRACTICE · TIER 2 target: 40 min

1. Recompute the ride-hailing firehose for a 2-second ping interval instead of 4. What improves, what costs more, and would you do it?
2. A city has 2,000 drivers online and 900 ride requests in one minute. Compute the requests per available driver and say what the design should do when that ratio goes above 1.
3. Design the courier-batching decision as a function: given a new order and a list of in-progress trips, return the trip to join or null. State the inputs and the cut-offs.
4. The lunch peak is 4.2x and perfectly predictable. Write the scaling plan with times, target capacity and the rollback trigger.
5. A restaurant’s tablet loses Wi-Fi for 20 minutes during lunch. Walk through every order that arrives in that window and what the system does with each.

ANSWER KEY

1. Pings double to $400,000/2 = 200,000/s$ and 20 MB/s; Kafka retention doubles to 1.73 TB a day. What improves is position freshness — the worst-case staleness halves from 4 s to 2 s, which moves a map marker about 14 m at 25 km/h. **Decision: no.** Doubling the largest write rate in the system to move a dot 14 metres is not a trade worth making; spend that budget on interpolating the marker’s movement client-side instead, which costs nothing.
2. $900/60 = 15$ requests/second against 2,000 drivers. Over one minute that is 900 requests for 2,000 drivers, a ratio of 0.45 — comfortable. Above 1.0 you are **structurally** short of supply, and the honest responses in order are: widen the radius, raise the multiplier to pull more drivers online, then show longer ETAs, then stop quoting. Queuing riders invisibly is the one option that is always wrong.
3. Inputs: the new order’s restaurant location, drop-off, ready time; each trip’s current stops and courier position. Cut-offs: restaurants within 800 m, drop-offs within 1.5 km, resulting batch size ≤ 3 , and **no existing customer’s ETA increases by more than 8 minutes**. Return the trip with the smallest added distance that passes all four. That last cut-off is the one that keeps the batch honest.
4. Scale to 4.5x at 11:15, hold until 14:00, scale down in two steps by 14:30. Repeat for dinner. Rollback trigger: p99 order-placement latency above 800 ms, or saga step-3 failure rate above 2%, either of which means capacity is short despite the plan — then add 50% and page.
5. Orders arriving in that window get no confirmation, so each one hits the 90-second timer and auto-cancels with a full refund. That is correct but terrible for the restaurant. The fix is to detect the **tablet** being offline (it heartbeats every 30 s) and mark

the whole restaurant temporarily closed after two missed heartbeats, so new orders are never accepted at all. Detecting the dependency is better than compensating 40 separate orders.

Design 3 — video streaming

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 17 [Google · pattern]

Design a video platform: 100 million daily viewers watching 40 minutes each, 500,000 uploads a day. Make it work on a phone on a train.

SOLUTION

Step 1 — Clarify

Questions and the answers I assume: 100 million DAU, 40 minutes each; 500,000 uploads/day, 10 minutes average; playback must start in under 2 seconds; quality must adapt to a changing network; live streaming is **out of scope** (say this — it is a completely different system); content is global, so a viewer in Jakarta must not be served from Virginia.

Step 2 — Scale estimate

Concurrent viewers.

$$\begin{aligned}100,000,000 \times 40 &= 4,000,000,000 \text{ watch-minutes/day} \\4,000,000,000 / 1,440 &= 2,777,778 \text{ average concurrent} \\&\times 3 = 8,333,333 \text{ at peak}\end{aligned}$$

Egress — the headline number. At an average 3 Mbps:

$$\begin{aligned}8,333,333 \times 3 \text{ Mbps} &= 25,000,000 \text{ Mbps} = 25 \text{ Tbps at peak} \\4,000,000,000 \times 60 \times 3,000,000 / 8 &= 9.0 \times 10^{16} \text{ B} = 90 \text{ PB/day}\end{aligned}$$

What the origin must serve, if the CDN offloads 95%:

$$25 \times 0.05 = 1.25 \text{ Tbps}$$

That is still 20 machines' worth of 10 Gbps network cards, and it is the number the rest of the design has to hit.

Storage from uploads. Ladder total 9.9 Mbps, 10-minute average:

$$\begin{aligned}9.9 \times 10^6 \times 600 / 8 &= 742,500,000 \text{ B} = 742.5 \text{ MB per video} \\500,000 \times 742.5 \text{ MB} &= 371.25 \text{ TB/day} \\371.25 \times 365 &= 135,506 \text{ TB} = 135.5 \text{ PB/year}\end{aligned}$$

Transcoding compute. Five renditions of a 10-minute video, at half real time per rendition:

$$\begin{aligned}10 \times 5 \times 0.5 &= 25 \text{ CPU-minutes per video} \\500,000 \times 25 &= 12,500,000 \text{ CPU-minutes/day} \\12,500,000 / 1,440 &= 8,681 \text{ cores running continuously}\end{aligned}$$

THE BOARD SUMMARY

25 Tbps peak · 90 PB/day egress · 1.25 Tbps of origin · 135.5 PB/year of new storage · 8,681 cores of transcoding.

The first sentence out loud: “This is a bandwidth problem, not a request problem. The CDN is not an optimisation here — it is the product. At \$0.01 per GB, 90 PB a day is \$900,000 a day, which is why a platform this size builds its own edge and peers directly with ISPs.”

Step 3 — API surface

note that no API ever returns video bytes

TEXT

```
POST /v1/videos
  body { title, description, visibility }
  201 { videoId, uploadUrl, uploadToken }
  note uploadUrl is a pre-signed URL straight to object storage.
        The bytes NEVER pass through your API. This is the whole trick.

PUT <uploadUrl> (client -> object storage, resumable, chunked)
  note chunked so a phone on a train can resume, not restart

GET /v1/videos/{videoId}/manifest.m3u8
  200 the HLS master manifest: one line per rendition, with bitrate
        and a URL to that rendition's own chunk list
  note cached at the edge for 60 s; this is the only "API" playback hits

GET <cdn>/v/{videoId}/{rendition}/{chunkNo}.ts
  200 a 4-second chunk, ~1.5 MB at 3 Mbps
  note served by the CDN. Your origin sees this only on a cache miss.

POST /v1/videos/{videoId}/heartbeat
  body { positionSec, rendition, bufferHealthMs, droppedFrames }
  204
  note fire-and-forget analytics; never blocks playback
```

Step 4 — Data model

metadata is tiny; the bytes live somewhere else entirely

TEXT

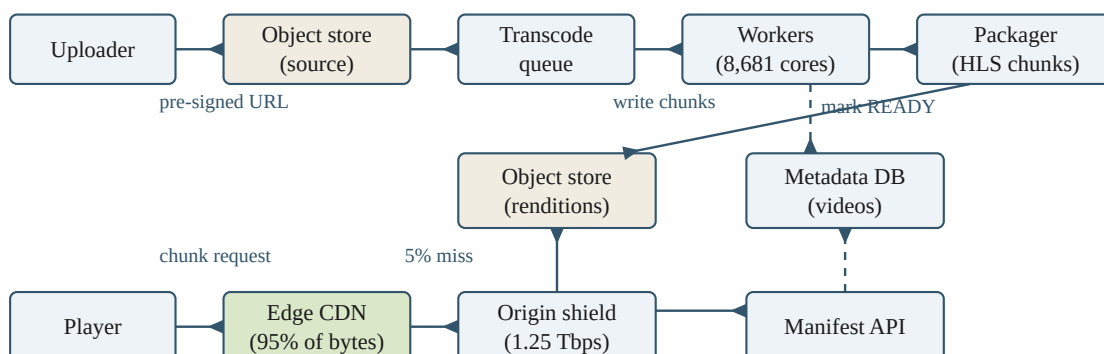
```
videos (sharded by video_id, hashed)
  video_id    ulid    PK
  owner_id    bigint
  title, description, tags
  duration_s  int
  state       enum    UPLOADING|TRANSCODING|READY|FAILED|TAKEN_DOWN
  renditions  json    [{h:1080,bitrate:5000000,path:"..."}, ...]
  created_at

-- object storage, NOT a database. Path is the key.
/v/{video_id}/source.mp4          -- deleted 30 days after READY
/v/{video_id}/1080p/{0000..NNNN}.ts -- 4-second chunks
/v/{video_id}/1080p/index.m3u8
/v/{video_id}/master.m3u8

watch_events -- append-only stream, not a table
  video_id, user_id, position_s, rendition, ts
```

Why the source file is deleted after 30 days: it is 30–40% of all your stored bytes and it is read exactly once, by the transcoder. Keeping it for 30 days covers “we need to re-encode with a better codec”. Keeping it forever costs roughly $135.5 \text{ PB/year} \times 0.35 = 47 \text{ PB}$ a year for nothing.

Step 5 — Architecture



Bytes never pass through an application server: upload goes straight to object storage, playback comes straight from the CDN. Your services only ever handle **metadata and manifests**, which are kilobytes.

Upload and transcode along the top; playback along the bottom. The two paths meet only at object storage, and the viewer never touches your API for bytes.

Step 6a — Deep dive: adaptive bitrate, and why the client decides

The phone on the train has 6 Mbps, then 0.5 Mbps in a tunnel, then 6 Mbps again. A fixed quality either stalls or looks bad.

HOW THE SWITCH ACTUALLY WORKS

1. The master manifest lists every rendition with its bitrate. The **player** picks.
2. The player measures how long the last chunk took. A 1.5 MB chunk that took 6 seconds means about $1.5 \times 8/6 = 2 \text{ Mbps}$ of real throughput.
3. It also watches its **buffer**: how many seconds of video are already downloaded.
4. Rule of thumb: step **down** immediately when the buffer falls below 10 seconds; step **up** only after several good chunks and a buffer above 25 seconds. Down fast, up slow — because a stall is far worse than a soft-looking picture.
5. Chunk length is the knob: 4 seconds is the usual compromise. **2-second chunks** adapt faster and start faster but double the request count and hurt compression. **10-second chunks** compress better but mean 10 seconds of the wrong quality after a network change.

Decision: 4-second chunks, with the first two chunks of every video also published at 2 seconds. The short opening chunks cut time-to-first-frame, which is the metric viewers actually feel, and the cost is two extra tiny files per video.

Why the client and not the server? Only the client knows its own buffer and its own screen. A server-side decision would need a control loop over a network it cannot measure. Handing the decision to the player also makes every chunk a plain, cacheable, immutable file — which is what lets a CDN serve 95% of your bytes without asking you anything.

Step 6b — Deep dive: making the CDN hit rate 95% and not 60%

WHERE THE HIT RATE ACTUALLY COMES FROM

Video viewing is extremely skewed: a small set of videos is most of the traffic. Suppose the top 10,000 videos are 80% of all watch time.

$$10,000 \times 742.5 \text{ MB} = 7,425,000 \text{ MB} = 7.4 \text{ TB}$$

Seven terabytes holds 80% of your traffic. That fits on the SSDs of a single edge rack, in every city you operate in. This one calculation is the argument for the whole CDN tier, and it is the calculation candidates never do.

Three rules that get you from 80% to 95%:

1. **Immutable URLs.** A chunk's content never changes, so cache it for a year. If you need to replace a video, publish new paths. Never invalidate.
2. **An origin shield.** Edges do not each fetch from origin; they fetch from one regional shield. With 200 edges, this turns 200 misses for a new video into 1.
3. **Pre-warm the predictable.** A popular creator's upload, a scheduled release: push it to edges **before** anyone asks. The first 100,000 viewers then find it already there.

What the remaining 5% costs. $25 \times 0.05 = 1.25 \text{ Tbps}$. At 10 Gbps per machine, $1,250/10 = 125$ machines of pure network, before any headroom. If the hit rate fell to 90%, that doubles to 250 machines. **A 5-point change in cache hit rate is 125 machines.** Say the number; it makes the whole tier concrete.

Step 6c — Deep dive: transcoding 500,000 videos a day without a queue backlog

8,681 cores running continuously is the **average**. Uploads are not flat — assume a 3x peak, so 26,043 cores at the busy hour.

THREE DECISIONS THAT MAKE THIS AFFORDABLE

1. **Split the video into chunks and transcode them in parallel.** A 10-minute video becomes 150 four-second chunks, each independently encodable. One video's latency drops from 25 minutes of CPU to about **30 seconds of wall clock** on 50 workers, and the work is now perfectly parallel and perfectly retryable.
2. **Prioritise the ladder.** Publish 360p first and mark the video READY the moment it exists. 1080p can finish minutes later. The creator sees a working video in under a minute; the expensive rendition arrives while nobody is looking.
3. **Use spot / preemptible capacity for the high renditions.** A chunk that is killed halfway is simply re-queued — it is idempotent by construction, keyed by (video_id, rendition, chunk_no). Typical saving is 60–80% of the compute bill for the renditions that are most of the cost.

What this costs: a video is briefly available only in 360p, so a viewer in the first two minutes may see a soft picture. **Decision: accept it.** The alternative is making every creator wait 25 minutes before their video exists at all.

Step 7 — Trade-offs, failure modes, 10x

Choice	Option A	Option B	Decision
Upload path	through your API	pre-signed URL to object storage	B — 371 TB/day must never touch an application server
Quality choice	server decides	client decides from buffer + throughput	B — only the client can measure its own link, and chunks stay cacheable

Chunk length	10 s, better compression	4 s, plus 2 s opening chunks	B — time-to-first-frame is the metric viewers feel
Source file	keep forever	delete 30 days after READY	B — saves 47 PB/year of write-once, read-once data
Transcode unit	whole video per worker	per chunk, parallel	B — 25 min of CPU becomes 30 s of wall clock, and retries are free
High renditions	on-demand, dedicated capacity	spot capacity, re-queue on kill	B — chunk work is idempotent, so preemption costs nothing but time
Cache invalidation	purge on change	immutable paths, never purge	B — a purge across 200 edges is slow and unreliable; new paths are instant

What fails	What the user sees	What we do
One edge node dies	A brief rebuffer, then normal	The player retries the same chunk URL; DNS or anycast routes it to a sibling edge. Immutable URLs mean any edge can serve it.
Origin shield is overwhelmed	Stalls on unpopular videos only	Popular content is already at the edge. Shed misses for rare videos with a 503 and let the player retry — degrade the tail, protect the head.
A transcode worker is preempted	Nothing	The chunk is re-queued. Keyed by (video_id, rendition, chunk_no), so a duplicate simply overwrites an identical file.
Transcoding backlog grows	Uploads take longer to appear	Alarm on queue age, not queue depth. Shed the 1080p work first: the video still becomes watchable, and the backlog clears 4x faster.
A video must be taken down	It disappears	Flip the metadata state and the manifest returns 410. Then purge — but the manifest block is instant and the purge is best-effort cleanup.
A region loses connectivity	Viewers there fail over to a neighbouring region	Latency rises, quality steps down, playback continues. Because bytes are immutable and stateless, failover is a routing change and nothing else.

At 10x (1 billion DAU-equivalent, 250 Tbps):

- 250 Tbps is beyond what any commercial CDN will sell you as a line item. You **are** the CDN: your own hardware inside ISP networks, which also removes most of the \$9,000,000-a-day transit bill.

- Storage becomes 1.36 EB a year of new video. Tiering becomes the main design: keep the hot 1% on SSD, the warm 9% on disk, and move the cold 90% to archival storage where the first byte takes minutes. A video nobody has watched for two years can take 3 minutes to start, and no one will ever notice.
- Transcoding becomes 86,810 cores. At this scale you stop buying general CPUs for it and move to dedicated encoding hardware, which is roughly an order of magnitude cheaper per stream.
- The genuinely new problem is **the long tail**. At 10x, the number of videos watched once a month grows faster than the head does, and those are exactly the ones that always miss the cache. **Decision: do not try to cache them.** Serve the tail from the origin at a lower starting rendition, accept a 400 ms time-to-first-frame instead of 150 ms, and spend the saved money keeping the head at 99% hit rate.

Design 4 — search autocomplete

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 18 [Amazon · pattern]

Design search-as-you-type: 2 billion searches a day, suggestions in under 100 ms, and new trending queries must appear within an hour.

SOLUTION

Step 1 — Clarify

Assumptions stated out loud: 2 billion searches/day; the client debounces and sends about 4 prefix requests per search; the budget is **100 ms end to end, so 50 ms in my service**; the top 10 suggestions are enough; personalisation is out of scope for v1 (say this — it changes the design completely); new trending queries must appear within **one hour**; offensive and banned terms must never be suggested.

Step 2 — Scale estimate

$$2,000,000,000 \times 4 = 8,000,000,000 \text{ prefix requests/day}$$

$$8,000,000,000/86,400 = 92,593 \text{ requests/second average}$$

$$92,593 \times 3 = 277,778 \text{ requests/second at peak}$$

Serving nodes. An in-memory lookup with no computation handles about 50,000 requests/s per node:

$$277,778/50,000 = 5.6 \rightarrow 6 \text{ nodes of work}$$

Run 8 so losing one changes nothing.

The data. Say 100 million distinct queries are worth suggesting, and precomputing the top 10 for prefixes up to 8 characters yields about 50 million distinct prefixes at 200 bytes each (10 suggestions, mostly shared string pointers):

$$50,000,000 \times 200 = 10,000,000,000 \text{ B} = 10 \text{ GB}$$

THE MOST IMPORTANT CONSEQUENCE OF THAT 10 GB

10 GB fits on one machine. So do not shard by prefix. Put a **complete copy** on every one of the 8 nodes, and let any node answer any request.

- No routing layer, no shard map, no rebalancing, no cross-shard query.
- A load balancer can send a request anywhere.
- Losing a node costs 1/8 of capacity and **zero** availability.

A candidate who shards a 10 GB dataset across 8 machines has added a distributed system to a problem that did not have one. **Say the size first, then decide.**

Write side. Rebuilding hourly from a 24-hour rolling log of 2 billion searches: the build job reads about 2×10^9 rows, aggregates by query, and emits the trie. That is a batch job of a few minutes on a modest cluster, run once an hour, entirely off the serving path.

Step 3 — API surface

one endpoint, and every field on it exists for a reason

TEXT

```
GET /v1/suggest?q=chick&lang=en&country=SG&limit=10
200 { suggestions: ["chicken rice", "chicken curry", ...],
      buildId: "2026-09-15T10", tookMs: 3 }
headers  Cache-Control: public, max-age=60
note     q is the RAW prefix; the server normalises (lowercase, trim,
collapse spaces, strip accents) before looking up
note     buildId lets the client and the CDN cache safely: when the
hourly build changes, the id changes and caches turn over
note     limit is capped at 10 server-side. A client asking for 1000
would turn a pointer read into a scan.
```

SHORTCUT

Put the buildId in the response and in the cache key. It turns cache invalidation — the hardest problem in the chapter — into a non-problem: nothing is ever invalidated, old entries simply stop being requested and expire. This is the same trick as immutable chunk URLs in the video design. Notice that you have now used it twice; interviewers notice when a candidate reuses a principle deliberately.

Step 4 — Data model

The serving structure is the TopKTrie from Piece 4, built offline and loaded as an immutable blob. The interesting model is the **pipeline** that produces it.

the build inputs

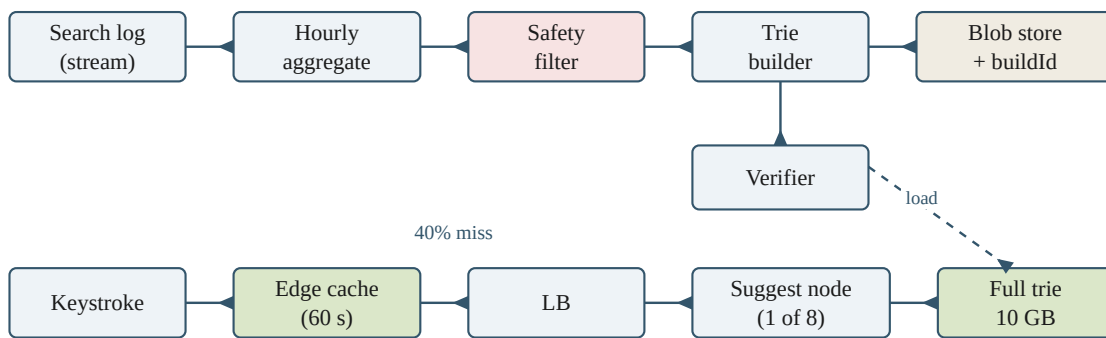
TEXT

```
search_log          -- the stream, 2 billion rows/day
  ts, raw_query, lang, country, result_clicked (bool)

query_stats          -- hourly aggregate, the build's input
  norm_query         text      PRIMARY KEY (norm_query, lang, country)
  count_24h          bigint
  ctr                float     clicks / impressions
  first_seen         timestamp
  blocked            bool      set by the safety pipeline, never by the build

trie_build           -- one row per hourly build
  build_id           text      "2026-09-15T10"
  blob_path          text      object storage path of the serialised trie
  size_bytes         bigint
  query_count        bigint
  state              enum      BUILDING|VERIFIED|LIVE|ROLLED_BACK
```

Step 5 — Architecture



A request does one prefix walk and one array read. No sorting, no scanning, no database, no network hop beyond the node itself. Every node holds the complete 10 GB trie, so there is no shard map and any node can answer anything.

The request path (bottom) touches nothing that can be slow. Everything expensive happens on the build path (top), once an hour.

Step 6a — Deep dive: the 50 ms budget, spent line by line

WHERE THE 100 MS ACTUALLY GOES

Segment	Budget	How we hold it
Phone to edge (mobile network)	30–50 ms	nothing you can do; it is physics
Edge cache hit	1 ms	60% of prefixes are cached; these never reach you
Edge to your region	10–20 ms	serve from the nearest region, always
Load balancer + your node	3 ms	prefix walk plus one array read
Response on the wire	2 ms	a suggestion list is under 1 KB

Total on a cache miss: about 70 ms. On a hit: about 40 ms.

Two rules follow, and they are the whole design:

1. **The client must debounce.** Firing on every keystroke sends 12 requests for a 12-letter query, of which 11 are thrown away. Wait 120 ms after the last keystroke, and cancel the previous in-flight request. This alone cuts request volume by roughly two thirds.
2. **The client must cache locally.** When the user types chick and then deletes back to chic, the answer for chic is already in the phone's memory. Never ask twice.

Step 6b — Deep dive: a new trending query in under an hour

The hourly rebuild means a query that starts trending at 10:05 appears at 11:00 — fifty-five minutes late. For a breaking news term that is far too slow.

THE TWO-LAYER ANSWER

1. **Layer 1 — the hourly trie.** Complete, safe, verified, 10 GB. It holds everything that was already popular.
2. **Layer 2 — a small hot overlay.** A separate structure, rebuilt every **60 seconds** from the last 15 minutes of the stream, holding only the few thousand queries whose rate has risen sharply. It is maybe 20 MB.
3. **At request time, merge:** look up the prefix in both, merge the two top-10 lists by score, take 10. This is a merge of two sorted lists of 10 — about 20 comparisons, well under 0.1 ms.

How you decide something is trending. Compare the last 15 minutes against the same query's previous hour, normalised per minute:

$$\text{score} = (\text{count in last 15 min} / 15) / (\text{count in previous hour} / 60 + 1)$$

The +1 stops a query that appeared twice yesterday from scoring infinity. Promote anything above, say, 5.0 with a minimum absolute count of 100 so that noise cannot trend.

What it costs: a second pipeline, a second structure in memory, and a merge on every request. **Decision: build it.** “Why doesn’t your search know about the thing that happened an hour ago?” is a product failure that users notice immediately, and 20 MB plus 20 comparisons is a very cheap fix.

Step 6c — Deep dive: never suggesting something terrible

Autocomplete says things **in your product’s voice**. A suggestion is an endorsement, and this is where these systems make the news.

FOUR FILTERS, APPLIED AT BUILD TIME, NOT AT REQUEST TIME

1. **Blocklist by exact normalised term.** Cheap, exact, and auditable. Applied after normalisation, so tricks with spacing and accents do not get through.
2. **Pattern rules.** No suggestion may complete a person’s name with a negative word. This is a rule about the **shape** of the suggestion, not about any single term.
3. **Minimum support.** Never suggest a query seen fewer than, say, 100 times in 24 hours across at least 20 distinct users. This alone removes almost all targeted manipulation, because making 20 real accounts is expensive and making 1 is not.
4. **A verification gate on every build.** The new trie is checked against a fixed set of test prefixes before it goes live. If any banned term appears, the build is marked ROLLED_BACK and the previous build stays live.

Why at build time? Because a request-time filter runs 277,778 times a second and a build-time filter runs once an hour. And because a build-time filter can be **reviewed** — you can look at the diff between two builds. You cannot review something that happens 24 billion times a day.

Step 7 — Trade-offs, failure modes, 10x

Choice	Decision
Data layout	Build fits; a full copy removes the routing layer entirely
When to rank	Pre-returns a 50 ms scan into a 0.05 ms read
Freshness	Only 15 minutes late on breaking terms is a visible product failure

Safety filtering	It — once an hour and reviewable, versus 24 billion times a day build quest with a ver- i- fi- ca- tion gate
Cache	Edge , 60% of requests never reach you, and invalidation is free the ser- vice keyed fast build dId
Personalisation	Alter . Personalising kills the shared edge cache, so it must be a separate, smaller, opt-in path — not the change to this one user's his- tory

What fails	What the user sees	What we do
A build is corrupt	Nothing	The verifier catches it and the previous build stays live. A stale trie is completely acceptable; a wrong one is not.
A suggest node dies	Nothing	7 of 8 remain, each holding the full data. Capacity drops 12.5%, availability does not move.
The hot-overlay pipeline stops	Trending terms stop appearing	Suggestions silently fall back to the hourly trie, which is still correct. Alarm on overlay age above 5 minutes.
Someone tries to manipulate a suggestion	Nothing	Minimum support of 100 searches from 20 distinct users, plus the build diff review.
Traffic doubles in an instant	Slightly slower	The edge cache absorbs most of it. The 8 nodes have 8x headroom over the 6 required.

At 10x (20 billion searches/day, 2.8 million requests/s at peak):

- $2,777,778/50,000 = 56$ nodes of work. Still small, because the request still does nothing. This is the payoff for the precompute decision, compounding.
- The trie grows with **distinct** queries, not with volume, so 10 GB becomes maybe 25 GB — not 100 GB. **Say that.** It is the difference between a design that scales and one that does not, and it comes from the shape of the data, not from any clever engineering.
- The new problem is **languages and scripts**. Prefix matching in a language without spaces, or one typed via transliteration, is not a prefix walk at all. **Decision: one trie per language, not one global trie**, plus a transliteration step that maps what the user typed into the target script before the lookup. Twelve languages at

25 GB each will no longer fit on one machine — and **that** is the moment sharding becomes correct. Shard by language first, because it needs no routing logic beyond a field already in the request.

Design 5 — payments

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 19 [Microsoft · pattern]

Design the payment service behind the previous four products: 10 million payments a day, 10x on sale days, several external providers, and a rule that a customer must never be charged twice.

SOLUTION

Step 1 — Clarify

Assumptions stated out loud: 10 million payments/day, 10x peak on a sale day; three external providers (card, wallet, bank transfer); the provider takes about 800 ms to answer; authorisation and capture are separate; refunds are 0.1% of payments; records must be kept **7 years**; and the absolute rule is **no double charge, ever, under any failure**.

Step 2 — Scale estimate

$$10,000,000/86,400 = 115.7 \text{ payments/second average}$$

$$115.7 \times 10 = 1,157 \text{ payments/second on a sale day}$$

In-flight provider calls (Little's Law, 800 ms each):

$$1,157 \times 0.8 = 926 \text{ open calls at peak}$$

Ledger rows. Two per payment, 300 bytes each:

$$10,000,000 \times 2 \times 300 = 6,000,000,000 \text{ B} = 6 \text{ GB/day}$$

$$6 \times 365 = 2,190 \text{ GB/year} = 2.19 \text{ TB/year}$$

$$2.19 \times 7 = 15.33 \text{ TB over the 7-year retention}$$

Idempotency keys. One per attempt, 100 bytes, kept 24 hours:

$$10,000,000 \times 100 = 1,000,000,000 \text{ B} = 1 \text{ GB/day}$$

Refunds. $10,000,000 \times 0.001 = 10,000$ a day = 0.12/second. Tiny, but they touch the same ledger and must be just as correct.

THE BOARD SUMMARY

1,157 payments/s at peak · 926 concurrent provider calls · 2.19 TB/year · 15.3 TB over 7 years.

The data is small and the correctness bar is absolute. That is the exact opposite of the video design, and the architecture should look opposite too: no cache on the write path, no eventual consistency about money, and a single strongly-consistent store per shard.

Step 3 — API surface

every write endpoint is idempotent; that is not optional here

TEXT

POST /v1/payments

header Idempotency-Key: 9f10-...

REQUIRED. 400 if missing.

```

body    { customerId, merchantId, amountCents, currency, method,
          orderRef, captureMode:"auto"|"manual" }
201     { paymentId, state:"AUTHORISED"|"CAPTURED", providerRef }
200     { ... }    <- a REPLAY: same key, same body, first answer returned
409     { error:"idempotency_key_reused_with_different_body" }
402     { paymentId, state:"FAILED", reason:"insufficient_funds" }

POST /v1/payments/{paymentId}/capture
header Idempotency-Key: ...
body    { amountCents }      // may be less than authorised, never more
200     { paymentId, state:"CAPTURED", capturedCents }

POST /v1/payments/{paymentId}/refunds
header Idempotency-Key: ...
body    { amountCents, reason }
201     { refundId, state:"REFUND_PENDING" }

POST /v1/webhooks/{provider}          (provider -> us)
note    signature-verified, replay-safe, and ALWAYS returns 200 fast.
        The handler writes to a queue and does nothing else.

GET /v1/payments/{paymentId}
200     { paymentId, state, amountCents, capturedCents, refundedCents,
          events:[...] }

```

TRAP

The 409 on a reused key with a different body is not pedantry. If a client reuses a key by accident and you quietly return the first payment, you have just told them their 5,000-rupee charge succeeded when what actually succeeded was a 50-rupee one. Store a hash of the request body alongside the key and compare. Same key and same body means replay; same key and different body means a client bug, and you must say so loudly.

Step 4 — Data model

append-only everywhere it matters

TEXT

```

payments                                (sharded by customer_id, hashed)
payment_id          ulid      PK
customer_id         bigint    SHARD KEY INPUT
merchant_id         bigint
amount_cents        bigint    INTEGER. Never a float. Never.
currency            char(3)
state               enum      CREATED|AUTHORISED|CAPTURED|FAILED|REFUNDED
provider            text
provider_ref        text
version             int
created_at, updated_at
UNIQUE (customer_id, idempotency_key)    -- the gate, in the database

payment_events      -- append-only, never updated
payment_id, seq, from_state, to_state, actor, at, raw_provider_payload

ledger_entries      -- append-only: never updated or deleted
entry_id            bigserial

```

```

txn_id      text      groups the legs of one movement
account     text      "user:7" | "merchant:42" | "fees" | "provider:card"
amount      bigint    signed minor units; legs of a txn SUM TO ZERO
currency    char(3)
posted_at   timestamp
INDEX (account, posted_at)

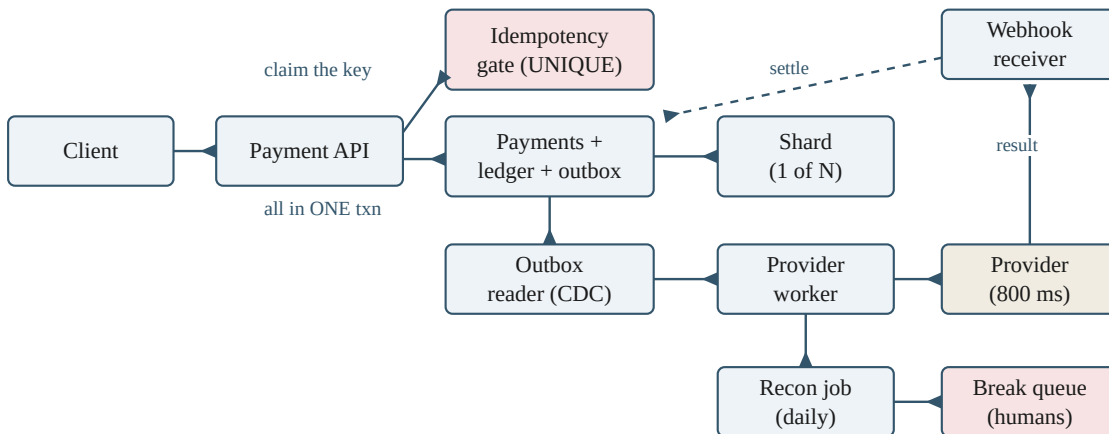
```

```

outbox      -- same shard, same transaction
id, payment_id, payload, published_at NULL

```

Step 5 — Architecture



The API returns as soon as the local transaction commits. Talking to the provider is a worker's job, so a provider stall never holds a user's connection, and a provider outage never loses a payment.

The user's request never waits on a provider retry. The gate, the ledger and the outbox all commit in one local transaction on one shard.

Trace one payment:

1. `POST /v1/payments` arrives with an Idempotency-Key.
2. The service opens one transaction on the customer's shard and does **all** of this in it: insert the idempotency key (unique constraint; a duplicate fails here and we return the first result), insert the payments row, insert the payment_events row, insert an outbox row. Commit.
3. Return 201 with state `CREATED`. Total: about 5 ms. **No provider has been contacted yet.**
4. A CDC reader tails the outbox and hands the job to a provider worker. The worker makes the 800 ms call, passing our payment_id as the provider's own idempotency reference.
5. On success it posts the two ledger legs and moves the state to `AUTHORISED` — again in one local transaction on the same shard.
6. The provider's webhook arrives later and confirms. It is signature-verified, written to a queue, and processed idempotently by (provider, provider_event_id).
7. Every night, the reconciliation job compares our ledger against the provider's settlement file, row by row. Anything that does not match goes to the break queue for a human.

Step 6a — Deep dive: the three ways a double charge actually happens

NAME ALL THREE; CANDIDATES USUALLY NAME ONE

1. **The client retried.** Handled by the idempotency key, claimed with a unique constraint in the same transaction as the payment. A duplicate insert fails and we return the stored first response.
2. **We retried, and the provider had already succeeded.** Our first call timed out at 5 s but the provider processed it at 6 s. Our retry creates a second charge. **Handled by passing our own payment_id to the provider as its idempotency reference** — every serious provider supports this, and a retry with the

same reference returns the original result instead of charging again. If a provider does **not** support it, you must query the provider for the status of that reference before retrying, never retry blindly.

3. **Two of our own processes worked the same outbox row.** Handled by claiming the row with a conditional update (`UPDATE ... SET claimed_by = me WHERE claimed_by IS NULL`) and by the fact that the provider call itself is idempotent on `payment_id`. Belt and braces, because this is the one where the money is real.

The fourth case nobody mentions: the transaction committed but the response never reached the client, so the user taps again with a **new** idempotency key. Now it is genuinely a second request, correctly processed, and the customer is legitimately charged twice for what they think is one thing. **The fix is not in the payment service.** The **order** service must derive the idempotency key from the order, not generate a fresh one per tap — and the app must show the pending charge. Being able to say where a fix does not belong is a senior answer.

Step 6b — Deep dive: the provider is down for 20 minutes

At 1,157 payments/second, 20 minutes of outage is

$$1,157 \times 20 \times 60 = 1,388,400 \text{ payments that cannot be authorised}$$

FOUR RESPONSES, IN INCREASING ORDER OF DESPERATION

1. **Retry with backoff and a cap.** Base 1 s, doubling, 6 attempts, full jitter: total window $1 \times (2^6 - 1) = 63$ s. This covers a blip, not an outage.
2. **Fail over to a second provider.** Requires that both are integrated and that routing is a config change, not a deploy. **Cost:** the second provider charges more, and its authorisation rates differ, so your success rate moves.
3. **Queue and authorise later.** Accept the order, tell the user “payment pending”, authorise when the provider returns. **Cost:** some of those authorisations will decline afterwards, and you must then unwind an order the customer thinks is placed. Only acceptable for low-value, cancellable goods.
4. **Shed load honestly.** Return 503 with a retry-after. Ugly, but it does not create 1.4 million ambiguous payments.

Decision: (1) then (2) then (4). Never (3) for anything of real value. An accepted order with an unauthorised payment is a promise you may not be able to keep, and unwinding 1.4 million of them is a far worse day than 20 minutes of honest 503s. Route to the second provider automatically when the primary’s error rate exceeds 5% over 60 seconds, and route back only after 10 clean minutes.

Step 6c — Deep dive: reconciliation, the job that catches everything else

Every night the provider sends a settlement file: everything they think happened.

THE THREE-WAY MATCH, AND WHAT EACH MISMATCH MEANS

Match on (`provider_ref`, `amount`, `currency`) across three sources: **our ledger**, **the provider file**, and **the merchant’s expected payout**.

Mismatch	What it means	Action
In provider file, not in our ledger	We charged and then crashed before posting the entry	Post the missing entry with the provider’s timestamp. This is money we took and did not record — the most dangerous kind.
In our ledger, not in the provider file	We recorded a charge that never happened	Reverse it and alarm. A customer may be looking at a charge that does not exist.

Amounts differ	Fees, currency conversion, or a partial capture	Expected for fees; post the fee leg. Unexpected for the rest; break queue.
Duplicate provider_ref	A genuine double charge slipped through	Refund immediately, automatically, and page a human. Do not wait to be asked.

Scale of the job: 10 million rows on each side. A sorted merge on provider_ref is $O(n \log n)$ to sort and $O(n)$ to walk — a few minutes. **Alarm threshold: any unexplained break at all.** Money is the one place where “0.01% is fine” is not fine: 0.01% of 10 million payments at an average of 800 rupees is $10,000,000 \times 0.0001 \times 800 = 800,000$ rupees a day.

Step 7 — Trade-offs, failure modes, 10x

Choice	Option A	Option B	Decision
Payment record	one row, UP-DATE-ed through states	append-only events plus a ledger	B — you can answer “what happened and when”, and an audit is possible
Idempotency	check a Redis key first	unique constraint in the same transaction	B — Redis is a speed-up, never the guarantee
Provider call	inside the user’s request	outbox plus a worker	B — 926 in-flight calls must not be 926 held user connections
Money type	decimal or float	integer minor units	B — there is no defensible version of A
Consistency	eventual, like the profile store	strongly consistent per shard	B — the data is 2 TB/year; there is no scale argument for weakening it
Provider outage	queue and authorise later	retry, fail over, then shed	B — an unauthorised accepted order is a promise you may not keep
Reconciliation	sample 1%	match every row, every night	B — 0.01% of missed breaks is 800,000 rupees a day

What fails	What the user sees	What we do
We crash after charging, before recording	A charge with no order	Reconciliation finds it that night and posts the entry. The provider reference is the link, which is why we send our own id as that reference.

A webhook arrives twice	Nothing	Processed idempotently on (provider, provider_event_id).
A webhook never arrives	Payment stuck at CREATED	A sweeper polls the provider for anything older than 5 minutes in a non-terminal state. Never rely on a webhook as the only path.
The shard leader fails mid-transaction	One request errors; the client retries	The transaction either committed or did not. The idempotency key makes the retry safe either way — this is exactly why the key is claimed inside the transaction.
A refund is requested twice	One refund	REFUNDED has no outgoing transitions, and the refund itself carries an idempotency key.
Sale day, 10x traffic	Nothing	1,157/s across N shards is small. The real constraint is the provider's rate limit — negotiate it in advance and implement a token bucket per provider so you shape your own traffic rather than being throttled mid-sale.

At 10x (100 million payments/day):

- 1,157/s average, 11,574/s at peak. $11,574/5,000 = 2.3 \rightarrow 3$ shards on write capacity; storage becomes 21.9 TB/year and 153 TB over 7 years, giving $153,000/500 = 306$ shards at a 500 GB shard size. **Storage decides, exactly as in Chapter 10** — so archive ledger entries older than 18 months to cold storage with the same append-only guarantees, and the online set stays small.
- In-flight provider calls become $11,574 \times 0.8 = 9,259$. No single provider will accept that from one client. You **must** run several providers in parallel and route by cost, success rate and their individual rate limits.
- The genuinely new problem at 10x is **reconciliation time**. 100 million rows per side per night is still a sorted merge, but it no longer fits in one machine's memory and it no longer finishes before the morning. **Decision: reconcile continuously, in hourly windows, instead of nightly in one batch.** Same logic, 24 times smaller each run, and a break is found within an hour instead of within a day — which for money is the difference that matters.

Interview drill — what the interviewer pushes on

They ask	You answer
“Where do you store driver locations?”	“In memory, and nowhere else on the hot path. 400,000 drivers is 24 MB. Persisting every ping would be 100,000 writes a second and 864 GB a day of data that is worthless one second later. I keep a downsampled trail on disk only while a driver is on a trip, which is 48 GB a day.”
“Why nine geohash cells and not one?”	“Because a rider standing at a cell border has their nearest driver on the other side of it. Searching one cell finds someone 900 metres away and misses someone 30 metres away. The cell lookup is a cheap filter — about 299 candidates — and haversine is the exact filter on top.”
“Two riders, one nearest driver. What happens?”	“The offer is a hold, taken with SET offer:driverId NX EX 12. The store decides atomically and the loser moves to the next candidate in microseconds. The 12-second TTL also means a dispatcher crash needs no recovery code at all.”

“Why not a distributed transaction for a food order?”	“Because a two-phase commit blocks every participant when the coordinator dies, and it will die during the lunch rush. I use a saga: steps ordered with the likeliest failure first and the impossible-to-undo step last, every step idempotent, and a failed compensation goes to a human queue with an alarm.”
“How do you serve 25 Tbps?”	“I do not. The CDN serves 95% of it and my origin serves 1.25 Tbps. The top 10,000 videos are 7.4 TB, which fits on one edge rack, and that is where 80% of watch time is. Chunks are immutable so nothing is ever invalidated.”
“Who chooses the video quality?”	“The player, from its own buffer depth and the measured download time of the last chunk. Step down fast below 10 seconds of buffer, step up slowly above 25. Only the client can measure its own link — and letting it decide is what keeps every chunk a plain, cacheable file.”
“Autocomplete in 50 ms. How?”	“By doing no work at request time. The top 10 are precomputed at every trie node by an hourly build job, so a request is a prefix walk and one array read. The whole structure is 10 GB, which fits on one machine, so every node holds a full copy and there is no shard map at all.”
“A term starts trending right now.”	“A second, tiny overlay rebuilt every 60 seconds from the last 15 minutes, merged with the hourly trie at request time. Twenty megabytes and about twenty comparisons. The hourly build alone would be up to 55 minutes late, which users notice.”
“Guarantee no double charge.”	“Three mechanisms, because there are three causes. The client’s retry is stopped by a unique constraint on the idempotency key, claimed inside the same transaction as the payment. Our own retry is stopped by passing our payment id to the provider as its idempotency reference. Two workers on one outbox row are stopped by a conditional claim. And reconciliation catches whatever all three missed.”
“Why is your ledger append-only?”	“Because an UPDATE destroys the evidence. Every movement is two rows summing to zero, a refund is a new opposite entry, and I can verify the whole system with one query: the sum of every row must be exactly zero.”
“Float or integer for money?”	“Integer minor units, always. 0.1 plus 0.2 is not 0.3 in a double, and once your ledger is out by a fraction of a cent it never balances again.”
“Which of these five is hardest?”	“They are hard in different places, and naming the place is the answer. Ride-hailing is hard at the write rate, video at the bandwidth, autocomplete at the latency budget, payments at correctness, and food delivery at coordination across parties who do not share a database.”

TRAP

The most common Tier-3 failure in this chapter is answering a **bandwidth** problem with a **request** architecture. If a candidate draws “player → load balancer → video service → database” for a streaming question, the interview is effectively over, because no arrangement of that picture carries 25 Tbps. Compute the egress in the first two minutes and let the number choose the architecture.

PRACTICE · TIER 3 target: 50 min

1. Design surge pricing end to end: what you measure, over what window, how you compute the multiplier, how you stop it oscillating, and what hard limits you enforce in code.
2. A food-delivery courier goes offline mid-trip with the food. Enumerate every state that must change, every party that must be told, and what the customer's refund is at each order state.
3. Compute the full cost of adding 4K (15 Mbps) to the video ladder: extra storage per year, extra transcode cores, and extra egress if 5% of watch time moves to it. Then decide.
4. Add personalised autocomplete for signed-in users without destroying the shared edge cache. Describe the two-path design and the extra latency it costs.
5. A provider's settlement file shows 412 charges we have no ledger entry for, totalling 680,000 rupees. Write the runbook: what you do in the first hour, the first day, and the first week.

ANSWER KEY

1. Measure **unfilled requests per available driver** in a cell over a rolling 5 minutes — not raw demand, because raw demand is high everywhere at 6 p.m. Multiplier steps at fixed thresholds (1.0 / 1.3 / 1.6 / 2.0), never a continuous function, so it cannot oscillate on noise. Add hysteresis: go up after 2 minutes above a threshold, come down only after 5 minutes below it. Hard caps in **code**, not config: never above 2.0x, never during a declared emergency, and an alarm on any cell above 1.6x for more than 10 minutes.
2. States: the order goes to **DELIVERY_FAILED**, the courier trip is closed, the courier's other batched orders are **reassigned** (this is the part candidates forget), the payment authorisation is voided or refunded, and the restaurant is told only if they need to remake. Refunds: full before **PICKED_UP**; full plus an apology credit after, because the customer did nothing wrong and the food is gone either way. Trying to charge for undelivered food to protect margin costs far more in churn than the food did.
3. Ladder goes from 9.9 to 24.9 Mbps, a $24.9/9.9 = 2.5 \times$ increase in stored bytes: 135.5 PB/year becomes 340.8 PB/year, so **+205 PB/year**. Transcode: 4K is roughly 4x the cost of 1080p per second, pushing per-video CPU from 25 to about 45 minutes, so cores go from 8,681 to $500,000 \times 45/1,440 = 15,625$ — **+6,944 cores**. Egress: 5% of watch time at 15 Mbps instead of 3 Mbps adds $90 \text{ PB} \times 0.05 \times (15/3 - 1) = 18 \text{ PB/day}$, which at \$0.01/GB is \$180,000 a day. **Decision: transcode 4K only for videos that pass a popularity threshold in their first 48 hours.** Most uploads are never watched enough to justify 2.5x storage, and the ones that are will earn it.
4. Path 1 is the shared, cacheable, anonymous path exactly as designed. Path 2 runs **only for signed-in users**, fetches the shared top-10 from path 1 (still a cache hit) and re-ranks it against a small per-user history held in a fast store keyed by user id. Re-ranking 10 items costs under 1 ms; the extra fetch costs about 3 ms. **Never** put the user id in the shared cache key — that turns one cache entry into 200 million.
5. **First hour:** freeze nothing, page the on-call, confirm the file is genuine and not a duplicate delivery, and check whether those 412 references exist in **payment_events** but not in **ledger_entries** — that distinguishes “we crashed before posting” from “we never saw these at all”. **First day:** post the missing entries with the provider's timestamps, notify the affected customers if they were charged for something they did not receive, and open an incident. **First week:** find the code path that lost them, add a metric that would have caught it in an hour instead of a night, and move reconciliation to hourly windows so the detection gap shrinks from 24 hours to 1.

ONE-PAGE REVISION CARD

The four numbers that start each design.

- Ride-hailing: $400,000/4 = 100,000$ pings/s. Everything follows from this.
- Food delivery: 35% of the day in 2 hours → a **predictable** 4.2x peak.
- Video: $8,333,333 \times 3$ Mbps = 25 Tbps. It is a bandwidth problem.
- Autocomplete: 277,778 req/s against a **10 GB** dataset. It fits on one machine.
- Payments: 1,157/s at peak, 2.19 TB/year. Small data, absolute correctness.

Little's Law is the workhorse. concurrency = rate $\times$ duration. Rides: $5,000,000 \times 20/1,440 = 69,444$. Viewers: $100,000,000 \times 40/1,440 = 2,777,778$. Provider calls: $1,157 \times 0.8 = 926$. Same formula, three designs.

Geohash card. 5 chars ≈ 4.9 km; 6 chars $\approx 1.22 \times 0.61$ km; 7 chars ≈ 153 m. **Always search the 8 neighbours.** Cell lookup is the cheap filter; haversine is the exact one. Drop pings older than 30 seconds.

The three-layer rule for data that moves. Memory for **now** (24 MB, disposable, rebuilds in 4 seconds). A log for **replay** (Kafka, 24 h). Disk for what will be **read again**, downsampled (48 GB/day instead of 864 GB/day).

Holds beat locks. SET key NX EX 12. Atomic, self-expiring, no crash recovery, no cleanup job. Used for dispatch offers; the same shape works for seat holds and inventory reservations.

Saga ordering is the answer. Likeliest-to-fail first; impossible-to-undo last. Every step and every compensation idempotent. A failed compensation goes to a human queue with an alarm, never a silent retry loop.

Video: the CDN is the system. 95% offload, immutable chunk paths so nothing is ever invalidated, an origin shield so 200 edge misses become 1, and pre-warming for predictable releases. Top 10,000 videos = 7.4 TB = 80% of watch time. The player chooses quality; down fast, up slow; 4-second chunks with shorter opening chunks.

Autocomplete: precompute or lose. Store the top-k **at every node** at build time. A request is a prefix walk and one array read. Filter for safety at build time (once an hour, and reviewable), never at request time (24 billion times a day). Hourly trie plus a 60-second hot overlay, merged per request.

Payments: four non-negotiables.

1. Integer minor units. Never a float.
2. Double entry, two legs summing to zero, append-only. A refund is a new opposite entry.
3. Idempotency key claimed by a **unique constraint inside the same transaction**. Redis is a speed-up, never the guarantee.
4. Reconcile every row, every night — or better, every hour. 0.01% missed is 800,000 rupees a day.

Three causes of a double charge, and the three different fixes. Client retry → idempotency key. Our retry → send our own id as the provider's reference. Two workers on one row → conditional claim. Reconciliation catches the rest.

JavaScript traps in this chapter.

1. `sort()` without a comparator sorts numbers as text: distances, counts and timestamps all need `(a, b) => a - b`.
2. Unlink from the old cell **before** linking to the new one, and never put an `await` between the two.
3. Map and Set keep types and insertion order; a plain object stringifies keys.
4. Money as a float. $0.1 + 0.2 !== 0.3$.
5. 926 concurrent I/O calls in one Node process is fine; 926 CPU-bound ones is not — move those to `worker_threads` or a separate service.

Checklist before you say “done”.

1. Computed the **biggest** write rate and said where it is stored — memory, log, or disk.
2. Said which transport each direction uses, and why (push versus poll).
3. Named the atomic primitive that stops a double assignment (SET NX, unique constraint).
4. Ordered the saga by failure likelihood and undoability.

5. For anything with media: computed egress in Tbps **before** drawing boxes.
6. For anything with a latency budget: spent the budget line by line.
7. For anything with money: integers, double entry, append-only, idempotency, reconciliation.
8. Gave both sides of every trade-off and then **decided**.

Four sentences that score marks.

1. “The current position of a driver is important and disposable — 24 megabytes in memory that rebuilds itself in four seconds.”
2. “This is a bandwidth problem, not a request problem; the CDN serves 95% of the bytes and my origin serves 1.25 Tbps.”
3. “The request does no thinking. The hourly build already did all of it.”
4. “The idempotency key is claimed by a unique constraint inside the same transaction as the payment, so a retry cannot win a race it should lose.”

Last-Week Quick Reference

Everything in this book compressed to what you can hold in your head on the day.

This appendix repeats nothing new. Every number here was derived in a chapter. Read it twice the night before and once on the morning of the round. If a line surprises you, go back to the chapter it came from — the chapter number is in the margin of each block.

A.1 • Latency numbers to memorise

These are the ones that actually change a design decision. Round numbers on purpose; an interviewer wants the **order of magnitude**, not three decimal places.

Operation	Time	vs. memory	What it means for the design
L1 cache read	0.5 ns	0.005x	free; never a bottleneck
main memory read	100 ns	1x	the unit everything else is measured in
read 1 MB from memory	30 µs	300x	serialising a big object is not free
in-process Map lookup	0.1 µs	1x	a local cache is ~5,000x a network hop
SSD random read	100 µs	1,000x	one uncached row read
same-DC network round trip	500 µs	5,000x	every extra service hop costs this
Redis GET, same DC	0.5–1 ms	~10,000x	dominated by the round trip, not Redis
indexed SQL row, cached	0.1–0.5 ms	–	the good case
indexed SQL row, from SSD	1–5 ms	–	the normal case
disk seek (spinning)	10 ms	100,000x	why nobody puts a hot index on one
scan 1 M rows	0.2–1 s	–	a missing index, visible to the user
CDN edge hit	10–30 ms	–	physics of the last mile, not of your server
Mumbai ↔ Singapore RTT	~60 ms	–	one sync cross-region write per request is too many
Mumbai ↔ Virginia RTT	~250 ms	–	only a nearby copy fixes this

SHORTCUT

Two sentences carry most of the marks here. **“Memory is about 5,000 times faster than a network hop, so I will keep the hot set in-process.”** And **“250 ms across an ocean is physics; I cannot optimise it away, I can only put a copy closer.”**

What one machine does — the capacity table

app server, JSON	1,000–3,000 RPS	L7 proxy node	20,000–50,000 RPS
SQL node, indexed reads	10,000–20,000/s	SQL node, durable writes	2,000–5,000/s
SQL, single locked row	~500 updates/s	Redis	~100,000 ops/s
one Kafka-style partition	~10 MB/s	WebSockets held per box	~100,000
1 Gbps NIC	125 MB/s	restore from backup	~200 MB/s

Two consequences you should be able to state instantly. **Shard size:** at 200 MB/s a 1 TB shard restores in about 83 minutes and a 6.5 TB one in about 9 hours — so keep shards between 200 GB and 1 TB. **Consumer count:** a consumer group can never have more working consumers than the topic has partitions.

A.2 • Estimation cheat-sheet

THE FIVE LINES — WRITE THESE ON THE BOARD IN THIS ORDER

traffic	DAU × actions ÷ 86,400 × peak factor	→ QPS
storage	rows/day × bytes/row × 365 × years × RF	→ bytes (× 1.3 for indexes)
bandwidth	peak QPS × response bytes	→ B/s (× 8 for bits)
memory	hot items × bytes/item × 1.3	→ cache size
machines	peak QPS ÷ per-server QPS ÷ 0.65	→ node count

Conversions you must not stop to derive

1 M/day	= 11.6/s	1 B/day	= 11,574/s
2^{10}	$\approx 10^3$ (KB)	2^{20}	$\approx 10^6$ (MB)
2^{30}	$\approx 10^9$ (GB)	2^{32}	$= 4.29 \times 10^9$
1 KB × 1 M	= 1 GB	1 KB × 1 B	= 1 TB
1 MB × 1 M	= 1 TB	1 Gbps	= 125 MB/s
1 day	86,400 s	1 month	$\approx 2.6 \times 10^6$ s
1 year	$\approx 3.15 \times 10^7$ s	bits vs bytes	capital B is bytes

Peak factors — pick one and say why

Traffic shape	Peak / average	Why
global consumer app	2x	users spread across time zones flatten the curve
one-country consumer app	3x	one evening peak, roughly 12–15% of the day in one hour
one-country work app	4–5x	everyone logs in inside the same 30 minutes
sale, match, or ticket drop	10–50x	the peak is the product; design for it explicitly

TRAP

Flat traffic would be $100\%/24 = 4.2\%$ per hour. A real consumer evening peak is 12–15% in the busiest hour. That is where the 3x comes from — say the derivation, do not just say “3x”.

The four formulas that decide designs

Little’s Law	$L = \lambda W$. Things in flight = arrival rate × time each one stays. Sizes every thread pool, connection pool and queue. 10,000 RPS at 200 ms = 2,000 in flight.
Cache relief	Database load falls by $1/(1 - h)$. A 90% hit rate is 10x relief; 99% is 100x. Going 92% → 97% is a 2.67x cut, which is why the last few points are worth fighting for.
Needed hit rate	$h = 1 - \text{db capacity} / \text{peak QPS}$. Work backwards from what the database can take, and you get the cache target instead of guessing it.
Fan-out tail	$P(\text{slow}) = 1 - p^n$. At p99 per shard, 16 shards give 14.9% slow requests and 64 shards give 47.4%. Scatter-gather turns a good p99 into a bad one.
Utilisation tax	latency multiplier = $1/(1 - \rho)$. 50% busy is 2x, 80% is 5x, 90% is 10x. This is why you size for 60–70%, not 95%.
Quorum	$W + R > N$. Overlap = $W + R - N$ nodes. Survives $N - W$ failures for writes, $N - R$ for reads. $N = 5, W = 3, R = 3$ is the balanced default.

Amplifiers — the things that multiply your estimate

Amplifier	Factor	Where it hides
replication	x3	every byte you store is three bytes of disk
indexes	+30%	and one extra write per index on every insert
fan-out on write	x followers	one post becomes one row per follower’s feed

retries at n layers	3^n	3 hops x 3 attempts = 27 requests for one user action
compaction headroom	x1.5	LSM stores need free space to merge
logs and telemetry	often > the data	a 1 KB event with 2 KB of logs around it

A.3 · The seven-step checklist

Say the step name out loud before you do it. The interviewer is ticking boxes, and an unspoken step is an unticked box.

#	Min	What you leave on the board	The sentence that scores the mark
1	5	3 features in scope, 3 out. One non-functional target (latency or availability).	“Let me fix the boundary first: X, Y and Z are in; auth and analytics are out.”
2	5	Peak QPS, storage per year, peak bandwidth — with the division shown.	“50 M DAU x 20 actions = 1 B/day = 11,600/s average, 3x peak $\approx$ 35,000/s.”
3	5	3–5 endpoints with request and response fields.	“Creates take an idempotency key; lists take a cursor, never an offset.”
4	5	2–4 tables or classes, with the key circled and one index named per query.	“The partition key is <code>user_id</code> because the hot query is ‘my last 50’, which then touches one shard.”
5	10	One diagram. Read path and write path marked separately.	“Write path is solid, read path is dashed. The cache sits here, and here is what invalidates it.”
6	10	The one or two genuinely hard parts, solved, with the failure case handled.	“The hard part is X. Here is the mechanism, and here is what happens when it fails mid-way.”
7	5	Two options, one decision, one 10x remark.	“I chose A over B, and I paid for it with C. At 10x, the first thing to break is D.”

THE CLOSING 60 SECONDS — SAY ALL FOUR

1. **The bottleneck.** “The binding constraint here is write throughput on the orders table.”
2. **The failure.** “If the cache region dies, reads fall back to the database at 5x load; I shed the feed endpoint first to protect checkout.”
3. **The 10x.** “At 10x, the single-leader write path is what breaks. I would shard by merchant next.”
4. **The regret.** “If I had another hour, I would revisit the fan-out choice — I picked write-time fan-out, and celebrity accounts make that expensive.”

The LLD version of the same seven steps

1 Clarify	Which rules change often? Those become strategy objects. Which never change? Those become constants.
2 Estimate	Not QPS — cardinality . “10,000 items, 50 users” tells you a <code>Map</code> is enough and a scan is not.
3 API	The public methods of the main class, with arguments and return types.
4 Data model	Classes, fields, and the maps that index them. State the invariant that must always hold.
5 Draw	A class sketch: boxes, arrows for “has-a” and “is-a”. Or write the code — code is the picture in LLD.
6 Deep dive	The concurrency case, or the rule that changes most often. Usually one of those two.
7 Trade-offs	Why this pattern and not that one. What a new requirement would cost you in edits.

TRAP

The most common LLD failure is designing for requirements nobody gave you. Six interfaces for a vending machine is not senior — it is slow. Add an abstraction only when you can name the **second** thing that will use it.

A.4 · The trade-off table

Every row is a real decision. Learn the **Decide when** column — that is the half people forget, and it is the half that scores.

Decision	Option A	Option B	Decide when
----------	----------	----------	-------------

SQL vs NoSQL	SQL: joins, constraints, real transactions	NoSQL: one access pattern, huge volume, easy sharding	SQL unless you can name the one query shape and the volume that breaks a single node. Default to SQL.
Normalise vs de-normalise	Normalised: one source of truth, no drift	De-normalised: fewer joins, faster reads	Normalise first. De-normalise only for a price snapshot, a constraint, or a measured hot join.
Strong vs eventual consistency	Strong: correct, slower, fails on partition	Eventual: available, stale, needs merge rules	Money, inventory and usernames are strong. Profiles, counts and feeds are eventual. Different data, different answer, same product.
Cache-aside vs write-through	Aside: simple, a miss costs one DB read	Through: always warm, every write costs more	Cache-aside as the default. Write-through only for a small, always-hot set that must never miss.
Update cache vs delete cache	Update: no miss after the write	Delete: cannot go stale, one miss	Always delete. Two concurrent updates can land in the wrong order; two deletes cannot.
Fan-out on write vs on read	On write: feed reads are one lookup	On read: cheap writes, expensive reads	Fan-out on write for ordinary users; on read for accounts above ~100 k followers. Hybrid, and say so.
Sync vs async work	Sync: user sees the result and the error	Async: fast response, needs status and retries	Sync only for what the user must see to trust the action. Everything else goes on a queue with a DLQ.
At-least-once vs exactly-once	At-least-once: duplicates, simple, the default	Exactly-once effect : unique key on the effect	Always at-least-once delivery. Buy exactly-once effect with a unique constraint, not with a cache.
Single leader vs multi-leader	Single: no write conflicts, one region is far	Multi: local writes, real conflicts to merge	Single leader unless cross-region write latency is the product problem. Conflicts cost more than you expect.
Optimistic vs pessimistic locking	Optimistic: no waiting, retries under contention	Pessimistic: one waits, throughput ~1/hold time	Optimistic below ~10% conflict rate. Above it, retries storm — 200 buyers on one row produced 19,900 retries.
hash % N vs consistent hashing	Modulo: trivial, moves ~ $N/(N+1)$ of keys	Consistent: moves ~ $1/(N+1)$ of keys	Consistent hashing with 100–200 vnodes, or fixed buckets with a bucket -> shard map. Never plain modulo on a growing cluster.
Vertical vs horizontal scaling	Vertical: no code change, a hard ceiling	Horizontal: no ceiling, distributed problems	Vertical first — it is cheaper than you think. Go horizontal when the blast

			radius, not the QPS, is the reason.
Retry vs fail fast	Retry: hides a blip, can amplify 3^n	Fail fast: honest, user sees the error	Retry only idempotent calls, with jitter, a budget of ~10% of traffic, and a circuit breaker above it.
Rate limit: token vs sliding window	Token bucket: allows bursts, tiny state	Sliding window: accurate, more state	Token bucket by default; sliding window log only when the limit is a contractual promise you must be exact about.

A.5 • The failure list — and the one-line fix

Failure	What the user sees	The fix you name
stale cache	an old price, an old name	delete on write, never update; short TTL
thundering herd	a latency spike every TTL	singleflight + stale-while-revalidate
hot key	one shard at 100%, rest idle	in-process cache for 1–2 s; split the key
cache penetration	DB load from keys that do not exist	cache the 404 for 30 s
cache avalanche	everything expires together	TTL jitter $\pm 20\%$, warm on boot
retry storm	the outage lasts longer than the cause	budget, jitter, circuit breaker
hot shard	p99 fine, p99.9 terrible	pick a key with no time prefix; measure skew
split brain	two leaders, lost writes	majority election, leases, fencing tokens
queue lag	work is done, but hours late	alarm on lag, autoscale consumers, shed
poison message	one consumer stuck forever	retry count, then DLQ, then alarm
scatter-gather tail	everything is slow at once	fewer shards per query, or hedge
unbounded growth	it works, then it stops	TTL, archive, or partition-and-drop from day one

A.6 • The night-before self-test

Twelve questions. If you can answer all twelve out loud in ten minutes, you are ready.

PRACTICE • TIER 3 target: 10 min, spoken, no notes

1. Say the seven steps in order, with the minute budget.
2. 30 M DAU, 12 actions each. Average QPS? Peak at 3x?
3. How many bytes is 1 KB per row, 5 M rows per day, kept 3 years, RF 3?
4. Memory read vs same-DC round trip — what is the ratio, and what does it tell you to do?
5. Give the quorum that survives two node failures for reads and writes at $N = 5$.
6. You cache at 92%. What hit rate halves the database load again?
7. Name a shard key that fails the “must never change” test, and what you would use instead.
8. On a write: cache first or database first? Delete or update? Why?
9. Exactly-once: what exactly does not exist, and what do you build instead?
10. Name two pieces of data in one product that sit on opposite sides of CAP.
11. 16 shards, p99 per shard. What fraction of scatter-gather reads are slow?
12. Your design at 10x — what breaks first, and what do you change?

ANSWER KEY

1. Clarify 5, estimate 5, API 5, data model 5, draw 10, deep dive 10, trade-offs 5. 2. $30 \text{ M} \times 12 = 360 \text{ M/day}$; $360/86,400 = 4,167/\text{s}$; peak $\approx 12,500/\text{s}$. 3. $5 \text{ M} \times 1 \text{ KB} = 5 \text{ GB/day}$; $\times 365 \times 3 = 5.48 \text{ TB}$; $\times 3 = 16.4 \text{ TB}$; with indexes $\approx 21.4 \text{ TB}$. 4. 100 ns vs 500 $\mu\text{s} = 5,000\times$. Keep the hot set in-process; batch what must cross

the network. **5.** $N = 5, W = 3, R = 3$: $W + R = 6 > 5$, survives $N - W = 2$ write failures and $N - R = 2$ read failures. **6.** Relief is $1/(1 - h)$. At 92% it is 12.5x; halving DB load needs 25x, so $h = 1 - 1/25 = 96\%$. **7.** Anything mutable — `city`, `plan_tier`, `merchant_region`. Use an immutable id (`user_id`, `order_id`), or a bucket derived from one. **8.** Database first, then **delete** the cache key. Never the reverse, never an update. **9.** Exactly-once **delivery** does not exist. Build exactly-once **effect**: a unique key on the effect, written in the same transaction as the offset. **10.** Payment ledger is CP (refuse rather than double-charge); profile photo is AP (serve a stale one). Usernames CP, follower counts AP. **11.** $1 - 0.99^{16} = 14.9\%$. **12.** Any honest answer with a named component, a named symptom, and a named change. “Single-leader writes on **orders**; I see it as lock waits above 500/s; I shard by merchant and move the ledger to its own cluster.”

One last thing, and it matters more than any table on these pages. An interviewer is not checking whether you already know the answer. They are checking whether they would want you in the room when the system is on fire at 3 a.m. Think out loud, write the numbers down, name the failure before they ask, and **decide**. That is the whole book.